SBTK handbook

Documentation of SBTK (Small Body Toolkit) version 1.0.0: density distributions, gravity coefficients and moments of inertia of small bodies from polyhedral shape models, and estimation of densities from gravity and inertia measurements.

Generated from the SBTK source on 2026-11-02. The website version is at https://ashcav1.github.io/sbtk-docs/.

Contents

1. SBTK: Small Body Toolkit
2. Assumptions and limitations
3. Installation
4. First script
5. Units
6. Coordinate systems and origins
7. Gravity coefficients
8. Frames of a gravity field
9. Moments of inertia
10. Shape models
11. Density distributions
12. Gravity and moment calculations
13. Measurement sets
14. Uncertainties and the expansion point
15. Reading the gravity coefficients
16. Density estimation
17. Fitting part of the coefficients
18. Goodness of fit and model comparison
19. Models with components
20. Searching the geometry
21. Sector grids
22. Point masses
23. Results folders and figures
24. Exchange packages
25. Density models by component type
26. Calculations
27. Tutorial scripts
28. Project template
29. Bennu
30. Hera blind test
31. Didymos and Dimorphos values
32. Testing with ROB data
33. Code of the ROB solutions
34. ROB-DATA1: Didymos
35. ROB-DATA2: Didymos
36. ROB-DATA3: Didymos
37. ROB-DATA4: Dimorphos
38. ROB-DATA5: Dimorphos
39. ROB-DATA6: Dimorphos
40. API reference
41. sbtk
42. sbtk.units
43. sbtk.shape
44. sbtk.body
45. sbtk.interior
46. sbtk.gravity
47. sbtk.data
48. sbtk.interpret
49. sbtk.invert
50. sbtk.search
51. sbtk.compare
52. sbtk.exchange
53. sbtk.io
54. sbtk.plot
55. Data structures
56. Error identifiers
57. Internal functions
58. Validation and tests

SBTK: Small Body Toolkit

SBTK is a MATLAB toolkit for the interior density distribution of asteroids and other small bodies. It computes the gravity coefficients, center of mass and moments of inertia of a density model inside a polyhedral shape model, and it estimates the densities of such a model from measured gravity coefficients and moments of inertia. This site documents version 1.0.0.

Capabilities

The following are implemented and covered by the test suite (Validation).

Density models
: A background density, uniform components (spheres, triaxial ellipsoids, tori with any axis, layers below the surface, latitude–longitude–radius sectors) and an optional Gaussian random density field, clipped to a closed polyhedral shape model that is star-shaped about the expansion origin (Density distributions).

Gravity and moment calculations
: Fully normalized spherical-harmonic coefficients to any degree, the center of mass and the inertia tensor of such a model, from exact polyhedral integrals over the cones between the expansion origin and each surface facet (Gravity and moment calculations).

Potential and acceleration at points
: The gravitational potential and acceleration outside, on or inside the body: exact for a homogeneous polyhedron and for spheres inside the body, and for the other departures from the background density on a grid of uniform cubes (Potential and acceleration at points).

Frames
: Coefficients about the center of mass in the shape-model axes, the spin axes or the principal axes of inertia, and bodies, measurements and models carried into another frame (Frames of a gravity field).

Measurement sets
: Gravity coefficients with a full covariance, moment-of-inertia elements, linear combinations of them, the libration ratio $(B - A) / C$ and the center of mass, simulated (exact values, or with uncertainties and Gaussian noise) or read from files (Measurement sets).

Reading the coefficients
: The measured coefficients minus those of the homogeneous body, about the center of figure, read as the center-of-mass offset, the principal axes of degree 2 and each degree's directions of excess and deficit; the anomaly each candidate component makes, and a forward stepwise selection of candidates (Reading the gravity coefficients).

Density estimation
: Weighted least squares for the background density and the relative density of each component, with the total mass as an equality constraint, bounds on the densities, optional Tikhonov regularization (its parameter by the discrepancy principle or the lowest Akaike information criterion), optional estimation of the position of spheres, ellipsoids and tori and of the axis of ellipsoids and tori, linearized 1σ uncertainties, and a χ^2 test of the fit (Density estimation).

Kinds of solution
: Models with components; a search of their geometry (every combination of given sizes, positions and axes, then Gauss–Newton, then compact masses kept by the likelihood-ratio test); sector grids that fill the body; and regularized point masses (Models with components, Searching the geometry, Sector grids, Point masses).

Comparison with a true model
: For synthetic tests: the fraction of the variance of the true density explained by the estimate (R^2), the largest value the chosen components allow, and a cell-by-cell comparison of density grids (Goodness of fit and model comparison).

Exchange packages
: Plain-text packages of measurements, interior models and solutions with their units, frame and normalization, for comparisons between teams (Exchange packages).

The assumptions behind these calculations and what SBTK does not do are listed in Assumptions and limitations.

Status and requirements

SBTK 1.0.0 has not been publicly released; its source repository is private. It requires MATLAB R2024b or newer with the Optimization Toolbox and no other. The polyhedral gravity kernel is a compiled C function, provided for macOS and buildable on other platforms (Installation).

Organization of this documentation

Section	Contents
Getting started	Installation and a complete first script.
Conventions	Units, coordinate systems, coefficient normalization, moments of inertia.
Models and calculations	Shape models, density distributions, the forward model, the potential at points.
Measurements and inversion	Measurement sets, uncertainties and the expansion point, reading the coefficients, the estimation problem, fitting part of the coefficients, χ^2 testing and comparison statistics.
Kinds of solution	Models with components, the search of their geometry, sector grids and point masses.
Files	Results folders, figures and the exchange-package format.
Examples	One example per component type and per calculation; the eleven tutorial scripts and the maker models of the Hera blind test.
Projects	The project template, the Bennu and Hera blind-test projects, and the Didymos and Dimorphos values.
Testing with ROB data	The six ROB packages of the Hera blind test, solved with each kind of solution.

Section	Contents
Reference	Every supported function, every data structure and field, every error identifier; internal functions separately.
Validation	What each test checks and its tolerance.

A single-page version and a PDF handbook contain the same pages.

Credits

SBTK was written by A. Cavale (University of Colorado Boulder). The polyhedral gravity kernel `ComputePolyhedralCSBar_mex` was written by Y. Takahashi (University of Colorado Boulder). The Bennu project reproduces the inversion of Cavale et al., "Constraints on the Internal Density Distribution of Bennu Using OSIRIS-REx Mission Data" (Icarus). Shape models: DART stereophotoclinometry models of Didymos (v003) and Dimorphos (v004); OSIRIS-REx shape model of Bennu.

Assumptions and limitations

This page lists the physical and numerical assumptions of SBTK 1.0.0 and the calculations it does not perform. Each item links to the page where the behavior is described.

Physical model

- The body is rigid and its density does not change with time. The gravity field is described by spherical-harmonic coefficients; SBTK does not evaluate their series. The potential and the acceleration at points are computed from the shape model: exactly for the background density and for spheres inside the body, and for the other departures from the background density on a grid of uniform cubes, which follows their boundaries to within a cell (Potential and acceleration at points).
- The density is a background density plus uniform components plus an optional Gaussian random field (Density distributions). Gradients within a component, other than through the random field, are not modeled.
- The shape model is fixed and exact: its uncertainty is not propagated into the coefficients or the estimates.
- The total mass is treated as exact. A measured GM enters only as the mass of the model; its uncertainty is not used (`sbtk.exchange.read_shadr` reads it and states that it is not used).

Geometry

- The shape model must be closed, consistently ordered and star-shaped about the expansion origin: every facet must face away from it, except small overhangs holding less than 10^{-6} of the volume (Shape models). Bodies with larger overhangs cannot be integrated. A mesh ordered inward is reversed.
- Layers and sectors are defined along rays from the expansion origin (the center of mass), not along the local surface normal (Coordinate systems).
- The z-axis of the shape-model frame is taken as the spin axis and as a principal axis in the definition of the libration ratio (Moments of inertia). For a homogeneous body the axis of the largest moment is 3.8° from the shape-model z-axis for Didymos and 0.7° for Dimorphos.
- Measurements, models and estimates of one body are in one frame, that of its shape model. `sbtk.data.to_frame` turns the axes of a measurement set only when its inertia rows are exact (the values of `sbtk.data.simulate`): rows with uncertainties would mix with correlations that the rows cannot hold. An inversion result cannot be transformed; invert in the frame wanted (Frames of a gravity field).
- The spin axes are those of a body in principal-axis rotation (z along the axis of the largest moment); the rotation state itself is not modeled, and a spin pole known with an error is represented only by the noise of $\bar{C}_{21}$, $\bar{S}_{21}$.
- When a field with uncertainties is moved to its center of mass or rotated into its principal axes, the covariance is propagated through the translation and rotation as linear maps; the uncertainty of the center of mass and of the principal axes themselves is not added.

Numerical integration

- Facet-cone integrals of the whole body are exact up to rounding (Gravity and moment calculations).
- A sphere entirely inside the body is integrated as a point mass, which is exact.
- Other components are integrated by sampling their radial extent along 9 sub-rays per facet and averaging the exact cone integrals over them. The result is exact when the radial interval is the same along every sub-ray of a cone and otherwise converges with the mesh resolution. Sector boundaries in latitude and longitude are resolved at the sub-ray spacing. The validation tests bound the errors for particular cases (Validation).
- The crossings of a ray with a sphere, ellipsoid or torus are exact (roots of a quadratic or quartic). The integrals of these components are continuous in their position and orientation, but smooth only on scales larger than the ray spacing: where a ray grazes a component, its piece inside changes like a square root. The derivatives used to estimate geometry are therefore taken over steps of 0.1 m and 0.01° (Estimated geometry).
- The random density field is integrated with one value per radial shell (24 shells per facet cone).

Estimation

- The measurements are linear in the densities. Positions and axes of components, when estimated, are found by Gauss–Newton iteration with a trust region and optional a priori constraints and limits; the geometry of layers and sectors, and of every component not marked for estimation, is fixed in a fit. Sizes, depths and sector bounds have no Gauss–Newton derivatives; they can only be searched over given values (Searching the geometry). An estimated geometry can absorb signal from structure the model lacks (Estimated geometry).
- The uncertainties are the linearized covariance with the mass constraint and fixed parameters eliminated. The inequality bounds on the densities are not reflected in the covariance; `fit.at_bound` flags densities at a bound.
- The χ^2 test assumes Gaussian measurement errors with the stated covariance.
- Simulated noise is Gaussian, drawn from the stated covariance of the coefficients and from the σ of each inertia row independently (`sbtk.data.add_noise`).
- The libration ratio obtained from a libration amplitude (`sbtk.data.libration_ratio_from_amplitude`) uses planar forced libration to first order in the eccentricity, about a point-mass primary; free libration, the primary's oblateness and the mutual dynamics of a binary are not included.

Data formats

- Coefficients must be fully normalized without the Condon–Shortley phase (Gravity coefficients). `sbtk.exchange.read_shadr` rejects files whose header does not state full normalization; other conversions are the user's responsibility.
- The exchange format gives the center of mass separately from the gravity coefficients, so a covariance between the center of mass and the other coefficients cannot be written (`sbtk:exchange:correlatedCenterOfMass`).

- The SBTK measurement text format stores only the diagonal of the coefficient covariance; the `.mat` file keeps the full matrix (Results folders and figures).

Software

- MATLAB R2024b or newer with the Optimization Toolbox (`lsqlin`). The compiled kernel is provided for macOS (Apple silicon and Intel) and must be compiled once on other platforms (Installation).
- Version 1.0.0 is not publicly released.

Installation

Requirements

- MATLAB R2024b or newer. SBTK 1.0.0 is tested with R2025a.
- Optimization Toolbox, for `lsqlin` in the inversion. No other toolbox is needed.
- A compiled copy of the polyhedral gravity kernel `ComputePolyhedralCSBar_mex` for the platform. Binaries for macOS on Apple silicon (`.mexmaca64`) and Intel (`.mexmaci64`) are included; on other platforms it must be compiled once (below).

No other toolboxes or external libraries are used.

Repository layout

Folder	Contents
`toolbox/+sbtk/`	The toolkit: one function per file, one folder per package (`+units`, `+shape`, ...).
`sbtk_setup.m`	Adds `toolbox/` to the MATLAB path for the current session.
`data/shapes/`	Shape models in OBJ format, lengths in km.
`data/gravity/`	The OSIRIS-REx gravity field of Bennu (JPL, 2019).
`data/hera_rse/`	A README; the radio-science files of the Hera end-to-end test are copied here (below).
`examples/`	Eleven tutorial scripts, `ex01_quick_start.m` ... `ex11_geometry_search.m` (Tutorial scripts), and the maker models of the Hera blind test (`make_didymos_*.m`, `make_dimorphos_*.m`).
`projects/`	Runner scripts for the project template, Bennu and the Hera blind test, with the ROB solutions in `projects/hera_hlp6/rob/` (Projects).
`results/`	Output of every run, created on demand (Results folders).
`tests/`	Unit and integration tests, longer studies and test reports (Validation).
`tools/`	Maintenance scripts (reduced shape models, benchmark, README figures).
`docs/`	Notes in Markdown and earlier documents.
`documentation/`	Sources and tools of this documentation.

Setting the path

Scripts in `examples/` and `projects/` add `toolbox/` to the path themselves, so they can be run directly, also section by section. For the Command Window or other scripts, run `sbtk_setup.m` once per MATLAB session:

MATLAB

```
run("/path/to/SBTK/sbtk_setup.m")     % replace /path/to/SBTK with the location of the repository
```

It prints the version and the repository folder. `sbtk.version` and `sbtk.root` return them.

Runner scripts locate themselves with `fileparts(which('<script name>'))` rather than `mfilename`, because under **Run Section** MATLAB executes a temporary copy of the script. A copied runner must have the name inside `which` changed to its own file name.

Compiling the gravity kernel

On platforms without a compiled kernel, with a C compiler configured for MATLAB (`mex -setup C`):

MATLAB

```
cd(fullfile(sbtk.root, "toolbox", "+sbtk", "+internal"))
mex -O ComputePolyhedralCSBar_mex.c
```

Then run the tests; the regression test compares the kernel output with stored values.

Files that are not in the repository

File	Needed by	Source
`data/shapes/g_01165mm_spc_obj_didy_0000n00000_v003.obj` (Didymos, 1.165 m sampling, about 160 MB)	`sbtk.shape.bundled("didymos_highres")`, `tools/make_reduced_shapes.m`, `examples/ex01_quick_start.m`, the Didymos maker models and runners of `projects/hera_hlp6`	DART shape models on ESA's SPICE server, `spiftp.esac.esa.int/data/SPICE/HERA/kernels/dsk/`
`data/shapes/g_00243mm_spc_obj_dimo_0000n00000_v004.obj` (Dimorphos, 0.243 m, about 160 MB)	`sbtk.shape.bundled("dimorphos_highres")`, `tools/make_reduced_shapes.m`, the Dimorphos maker models and runners of `projects/hera_hlp6`	same

File	Needed by	Source
Radio-science files of the Hera end-to-end test, in `data/hera_rse/`	`projects/hera_hlp6/hlp6_uncertainties.m`: the simulation, inversion and ROB runners of `projects/hera_hlp6` and the studies in `tests/studies`	Hera working group; not distributed (`data/hera_rse/README.md`, Hera blind test)

All other shape models used in the examples and tests are in `data/shapes/`.

Checking the installation

```
MATLAB
run(fullfile(sbtk.root, "tests", "run_tests.m"))
```

This runs 196 unit and integration tests in about 35 minutes, most of it the HLP6 maker models on the full-resolution shape models, and writes `tests/reports/test_report.txt`. Every test should pass (Validation).

First script

The script below runs the complete SBTK sequence on the shape model of the asteroid Didymos: it defines a body and a known interior (the true model), computes the exact gravity coefficients and moments of inertia of that interior, turns them into measurements by giving them uncertainties and adding noise, estimates the densities of the interior from those measurements, and compares the estimate with the true model. It takes about 15 s.

Script

Replace `/path/to/SBTK` with the location of the repository and run the script.

MATLAB script: first_script.m

```
run("/path/to/SBTK/sbtk_setup.m")

% 1. Body: shape model, total mass [kg], reference radius of the gravity field [km]
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
sbtk.body.summary(body)

% 2. A known interior: a dense core and a low-density surface layer
components = [sbtk.interior.sphere("core", body.centroid_km, 0.170, +450), ...   % center [km]: the center of figure
              sbtk.interior.layer("regolith", 0, 0.012, -900)];        % from 0 to 12 m below the surface
interior = sbtk.interior.create(components);

% 3. The true model and its exact gravity coefficients (degrees 1-4) and moments of inertia
[exact, truth] = sbtk.data.simulate(body, interior, 4, "first_script");

% 4. Measurements: 1-sigma uncertainties, then one realization of noise drawn from them
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), ...   % degree 1: 2 m in the center of mass
                               1e-4, 2e-4, 4e-4], ...                           % degrees 2, 3, 4
                              2e-3, ...                                         % each inertia element / (M R_ref^2)
                              0.02);                                            % libration ratio
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
sbtk.data.summary(obs)

% 5. Estimate the relative densities of the same components (the values given above are not used)
fit = sbtk.invert.solve(body, components, obs);
sbtk.invert.summary(fit)

% 6. Compare the estimate with the true model
cmp = sbtk.compare.with_truth(body, truth, fit);
sbtk.compare.summary(cmp)
sbtk.plot.truth_vs_estimate(body, truth, fit);
```

Output

Output

```
SBTK 1.0.0 ready (/path/to/SBTK)

Didymos
  shape didymos_g_9309mm_spc_obj_0000n00000_v003: 49152 facets, 24578 vertices
  volume 0.20336 km^3, bulk density 2593 kg/m^3, mass 5.273e+11 kg
  reference radius 430 m; center of figure [15.9 42.3 -34.0] m from the shape origin

Didymos measurements (first_script)
  mass 5.273e+11 kg, reference radius 430 m, gravity to degree 4
  24 gravity coefficients: 3 of degree 1, 5 of degree 2, 7 of degree 3, 9 of degree 4
  7 moment-of-inertia measurements: Ixx Iyy Izz Ixy Ixz Iyz libration
  coefficients are about origin_km = [15.9 42.3 -34.0] m (shape-model coordinates)
  coefficient covariance: diagonal (independent uncertainties)
  example: C20 = -3.4877e-02 +- 1.0e-04
  notes: Simulated by SBTK from interior first_script. Values include simulated measurement noise (seed 1).

Didymos inversion of first_script: 27 measurements
  chi2 = 27.77, dof = 25.0, reduced chi2 = 1.11, p = 0.32 (consistent with the measurements at the 5% level)

  component    density [kg/m^3]      relative density [kg/m^3]
  background     2741 +-    51
  core           2871 +-   115          +131 +-   164   not resolved (|relative density| < 1 sigma)
  regolith       1040 +-   323         -1701 +-   373

  largest normalized residual: S42, 2.0 sigma

Didymos: estimate compared with the true model
  R2 = 0.37: fraction of the variance of the true density about its mean explained by the estimate
  R2_max = 1.00: the largest R2 these components can reach
  RMS density error 267 kg/m^3; standard deviation of the true density 337 kg/m^3
  reduced chi2 of the fit 1.11 (p = 0.32)

  component         estimate [kg/m^3]       true    deviation
  core                 2871 +-   115       3083       1.8 sigma
  regolith             1040 +-   323       1733       2.1 sigma
```

The figure shows the true density, the estimated density and their difference on the planes $z = 0$ and $y = 0$ through the center of mass (Figures).

What each step does

1. **Body.** `sbtk.shape.bundled` reads the DART shape model of Didymos (version 3, 9.3 m sampling, 49,152 facets) in km. `sbtk.body.from_mass` attaches the total mass $M = 5.273{\times}10^{11}$ kg and the reference radius $R_{\mathrm{ref}} = 0.43$ km of the coefficients; the bulk density M / V follows from the shape volume. The center of figure is 57 m from the origin of the shape-model file, which is not the body's center (Coordinate systems and origins).
2. **Interior.** The core is a sphere of radius 170 m centered at the center of figure, `body.centroid_km`, 450 kg/m³ denser than the background; positions of spheres, ellipsoids and tori are shape-model coordinates, and the origin of this file is 57 m from the center of figure (Where to place components). The regolith is everything within 12 m of the surface, measured along rays from the center of mass, 900 kg/m³ lighter than the background. The background density is not given: it is set so that the total mass equals M (Density distributions).
3. **True model.** `sbtk.data.simulate` computes the fully normalized gravity coefficients of degree 1 to 4, the six inertia elements and the libration ratio of this interior, about its center of mass (Gravity and moment calculations). It returns the true model (`truth`) and its exact values (`exact`), with no uncertainties: none are involved in the forward model. Degree-1 coefficients give the center of mass (Gravity coefficients).
4. **Measurements.** `sbtk.data.set_uncertainties` gives every coefficient the 1σ of its degree and every inertia quantity its 1σ; `sbtk.data.add_noise` adds one Gaussian realization drawn from them with seed 1 (Measurement sets).
5. **Estimation.** `sbtk.invert.solve` fits the background density and the relative density of each component to the 24 coefficients and, with the standard settings of `sbtk.invert.options`, to `Ixx`, `Iyy` and `Izz`, with the total mass as a constraint (Density estimation). The reduced χ^2 of 1.11 with $p = 0.32$ means that the model fits the data within their uncertainties (Goodness of fit).
6. **Comparison.** `sbtk.compare.with_truth` evaluates both densities throughout the body. $R^2 = 0.37$ is the fraction of the variance of the true density that the estimate explains; $R^2_{\mathrm{max}} = 1$ because the estimate uses the true geometry. Each component's estimate is within 2.1σ of its true density. The core is not resolved (its relative density is smaller than its 1σ): a core concentric with the body changes the field much as the background and the regolith layer do, so at these uncertainties the three are hard to tell apart. An off-center core, or more precise inertia measurements, separate them better (Tutorial ex08).

Next steps

- Density models by component type and Calculations: short examples of every model type and calculation.
- Results folders and figures: saving runs with logs, settings and standard figures.
- Exchange packages: measurement packages for blind tests between teams.

Units

SBTK uses one set of units in every function, structure and file. Every stored quantity that has a unit carries it at the end of its name; a name without a unit suffix is dimensionless.

Quantity	Unit	Name suffix	Example
Lengths, positions, radii, depths	km	`_km`	`center_km = [0.02 0 0]` (20 m along x)
Densities and relative densities	kg/m³	`_kgm3`	`drho_kgm3 = -900`
Mass	kg	`_kg`	`mass_kg = 5.273e11`
Volume	km³	`_km3`	`volume_km3 = 0.20336`
Gravitational parameter GM	km³/s²	`_km3_s2`	`GM_km3_s2 = 3.519e-8`
Angles: latitude, longitude, rotations	degrees	`_deg`	`lat_deg = [40 90]`
Gravity coefficients $\bar{C}_{lm}$, $\bar{S}_{lm}$	dimensionless		fully normalized (Gravity coefficients)
Moments of inertia in measurement sets	$I\,/\,(MR_{\mathrm{ref}}^2)$, dimensionless	`_norm`	$I_{zz} \approx 0.35$ for a homogeneous Didymos with $R_{\mathrm{ref}} = 0.43$ km
Moments of inertia in reports and text files	kg m²	`_kgm2`, `_kg_m2`	`inertia_tensor_kg_m2`
Gravitational constant G	km³ kg⁻¹ s⁻²		6.67430×10^{-20} (CODATA 2018), `sbtk.units.G`

`sbtk.units.table` prints this table in MATLAB.

Where units are given at input

Two inputs carry a unit chosen by the caller:

- **Shape files.** `sbtk.shape.load` and `sbtk.shape.from_arrays` require the length unit of the vertices, `"km"`, `"m"` or `"cm"`; there is no default. `sbtk.units.length_scale` gives the conversion factor to km.
- **Moments of inertia in kg m².** `sbtk.units.moi_to_norm` converts them to $I\,/\,(MR_{\mathrm{ref}}^2)$ before `sbtk.data.add_moi`; `sbtk.units.norm_to_moi` converts back.

All other arguments take the units of the table; their names say which (`sbtk.interior.sphere(name, center_km, radius_km, drho_kgm3)`).

Conversions inside the toolkit

Volumes are in km³ and densities in kg/m³, so the mass of a region of density ρ and volume V is $\rho\, V \times 10^9$ kg. Moments of inertia are normalized with R_{ref} in km, so $I\,/\,(MR_{\mathrm{ref}}^2)$ needs no further factor; converting to kg m² multiplies by $M\,(10^3 R_{\mathrm{ref}})^2$.

A center-of-mass offset d (km) along one axis corresponds to the fully normalized degree-1 coefficient $d\,/\,(\sqrt{3}\;R_{\mathrm{ref}})$ (`sbtk.units.offset_to_degree1`); this is how a center-of-mass uncertainty enters `sbtk.data.uncertainties`.

Coordinate systems and origins

Shape-model frame

Every position in SBTK is given in the **shape-model frame**: the origin and axes of the shape model in use, lengths in km. The z-axis is taken as the spin axis. Measurements, models and estimates of one body are in the same frame. `sbtk.body.to_frame`, `sbtk.data.to_frame` and `sbtk.interior.to_frame` carry a body, its measurements and its interior into another frame, for example the spin or principal axes of a true model, which then becomes the shape-model frame of everything that follows; `sbtk.gravity.center_of_mass_frame` expresses a gravity field in the shape-model, spin or principal axes (Frames of a gravity field).

Three points of the shape-model frame are distinct in general:

Point	Definition	Where SBTK stores it
Shape-model origin	The point (0, 0, 0) of the shape file, chosen by its makers. For the DART Didymos model v003 it is 57 m from the center of figure; for the OSIRIS-REx Bennu model it is the center of mass found from the tracking data, 1.4 m from the center of figure; the HLP6 makers keep the origin of the DART files, and a package moved to the center of mass has its origin there (The shape model of a package).	the origin of every coordinate
Center of figure	Centroid of the volume enclosed by the shape model: the center of mass of a homogeneous body.	`body.centroid_km`, `props.centroid_km`
Center of mass	Center of mass of the density model. It differs from the center of figure when the density is not uniform.	`truth.com_km`, `obs.com_minus_cof_km`, `center_of_mass.txt` of a measurement package

Expansion origin

Gravity coefficients, first moments and the inertia tensor are computed **about** a point, the expansion origin. It is stored as `origin_km` in measurement sets (`obs.origin_km`), true models (`truth.origin_km`), inversion results (`fit.origin_km`) and geometries (`geo.origin_km`).

- `sbtk.data.simulate` expands about the true center of mass of the simulated interior. Because layers and sectors are measured from the expansion origin, the center of mass depends on the origin; the simulation repeats the calculation about the new center of mass until it moves by less than 1 mm.
- `sbtk.data.expand_about` expands a simulated field and its inertia about another point, for example `[0 0 0]`, the origin of the shape model file, about which the HLP6 makers give their data.
- Measurement packages read with `sbtk.exchange.read_data` are about the origin of their shape model file, as given, or about the *measured* center of mass; in the second case the inversion moves its predictions to the model's own center of mass (Density estimation).
- `sbtk.gravity.geometry` takes any expansion origin about which the shape is star-shaped.

Positions of components

Components are positioned in two different ways:

- **Spheres, ellipsoids and tori** are placed in shape-model coordinates. `center_km = [0 0 0]` is the shape-model origin, not the center of figure. A torus lies about its axis, the third of `component.axes`, through `component.center_km`, in the plane at `height_km` along that axis; as constructed this is the z-axis through the origin. To place components from the center of figure, as recommended, use `body.centroid_km`: a sphere at `body.centroid_km + offset`, a torus with `center_km = body.centroid_km` (Where to place components).
- **Layers and sectors** are measured along rays from the expansion origin: a layer's depth is the distance below the surface along the ray through the point; a sector's radial fraction is the distance from the origin divided by the distance from the origin to the surface along the same ray, and its latitude and longitude are measured about the origin.

Latitude and longitude

For a vector $\mathbf{x} = (x, y, z)$ from the point of reference, latitude ϕ is measured from the xy-plane toward $+z$ and longitude λ from $+x$ toward $+y$:

$$\mathbf{x} = d\,(\cos\phi\cos\lambda,\ \cos\phi\sin\lambda,\ \sin\phi)\,.$$

`sbtk.interior.latlon` returns $\mathbf{x}$ from ϕ, λ (degrees) and d (km). Sector longitudes are compared in $[-180°, 180°)$ with ranges that may extend beyond ±180°; for example $[150°, 240°]$ includes $-170°$.

Ellipsoid orientation

An ellipsoid's semi-axes (a, b, c) lie along x, y, z before rotation. The rotation is $R = R_z(\psi)\,R_y(\theta)\,R_x(\varphi)$ with ψ the yaw (about z), θ the pitch (about y) and φ the roll (about x), given in degrees as `[yaw pitch roll]`. The columns of R (`component.axes`) are the ellipsoid's axes in the shape-model frame, and a point $\mathbf{x}$ is inside when $\sum_i \left([R^{\mathsf{T}}(\mathbf{x} - \mathbf{c})]_i \,/\, a_i \right)^2 < 1$.

Axes of the inertia tensor and the coefficients

Coefficients and inertia elements are given about the center of mass but in the axes of the shape-model frame, not in principal axes of inertia, so $\bar{C}_{21}$, $\bar{S}_{21}$ and $\bar{S}_{22}$ are in general not zero. For a homogeneous body, the axis of the largest moment of inertia is 3.8° from the shape-model z-axis for Didymos (DART v003, 1.165 m model) and 0.7° for Dimorphos (v004, 0.243 m model), and the axis of the smallest moment is turned 32.2° and 7.8° from x toward $-y$; in the

shape-model axes the homogeneous Didymos has, about its center of mass with R_{ref} = 410 m, $\bar{C}_{21} = 4.4{\times}10^{-3}$, $\bar{S}_{21} = 9.4{\times}10^{-4}$ and $\bar{S}_{22} = -7.6{\times}10^{-3}$. In the spin axes of a field (z along its axis of the largest moment) $\bar{C}_{21} = \bar{S}_{21} = 0$, and in its principal axes also $\bar{S}_{22} = 0$; the interior-model package states the principal axes of the true model (Frames of a gravity field). A field and its shape model must be in the same frame: when they are not, carry the shape model into the frame of the field with `sbtk.body.to_frame`. The libration ratio uses the principal moments in the xy-plane computed from the tensor (Moments of inertia), and takes z as a principal axis, which holds exactly in the spin and principal axes.

Gravity coefficients

Expansion of the potential

Outside the smallest sphere about the expansion origin that encloses the body (the Brillouin sphere), the gravitational potential is

$$U(r,\phi,\lambda) = \frac{GM}{r}\sum_{l=0}^{L}\left(\frac{R_{\mathrm{ref}}}{r}\right)^{l}\sum_{m=0}^{l}\bar{P}_{lm}(\sin\phi)\left[\bar{C}_{lm}\cos(m\lambda)+\bar{S}_{lm}\sin(m\lambda)\right]$$

with r, latitude ϕ and longitude λ measured from the expansion origin in the axes of the shape-model frame (Coordinate systems). $U > 0$ and the acceleration is $+\nabla U$. $\bar{C}_{00} = 1$. R_{ref} is a chosen reference radius; SBTK computes the coefficients and does not evaluate the series. The potential and acceleration at points are computed from the shape model directly (Potential and acceleration at points).

For a density $\rho(\mathbf{x})$ of total mass M the fully normalized coefficients are

$$\bar{C}_{lm} = \frac{1}{(2l+1)M}\int \rho\left(\frac{r}{R_{\mathrm{ref}}}\right)^{l}\bar{P}_{lm}(\sin\phi)\cos(m\lambda)\,dV,$$

$$\bar{S}_{lm} = \frac{1}{(2l+1)M}\int \rho\left(\frac{r}{R_{\mathrm{ref}}}\right)^{l}\bar{P}_{lm}(\sin\phi)\sin(m\lambda)\,dV.$$

Normalization

The coefficients are **fully normalized** (the 4π normalization of geodesy) and the associated Legendre functions have **no Condon–Shortley phase** $(-1)^m$:

$$\bar{P}_{lm} = N_{lm}\,P_{lm}, \qquad N_{lm} = \sqrt{\frac{(2-\delta_{m0})(2l+1)(l-m)!}{(l+m)!}}, \qquad P_{lm}(x) = (1-x^2)^{m/2}\frac{d^m}{dx^m}P_l(x),$$

so that the mean of $[\bar{P}_{lm}(\sin\phi)\cos(m\lambda)]^2$ over the sphere is 1. Unnormalized coefficients are $C_{lm} = N_{lm}\bar{C}_{lm}$. Examples: $N_{20} = \sqrt{5}$, $N_{21} = \sqrt{5/3}$, $N_{22} = \sqrt{5/12}$, $N_{30} = \sqrt{7}$. The zonal coefficients are $J_l = -C_{l0} = -\sqrt{2l+1}\ \bar{C}_{l0}$.

Coefficients in another convention must be converted before use: multiply by $(-1)^m$ to remove a Condon–Shortley phase, and by $1/N_{lm}$ to normalize. `sbtk.exchange.read_shadr` checks the normalization state in the file header and rejects unnormalized files; `sbtk.io.read_jpl_gravity` converts J_l to $\bar{C}_{l0}$.

Reference radius

Changing the reference radius from R to R' multiplies a coefficient of degree l by $(R/R')^l$; its variance by $(R/R')^{2l}$. `sbtk.data.use_uncertainties` applies this when it takes uncertainties from a data set with another reference radius. The examples on this site use $R_{\mathrm{ref}} = 0.43$ km for Didymos and 0.10 km for Dimorphos.

Order of coefficients

Vectors of coefficients are ordered by degree, and within a degree as C_{l0}, then C_{lm}, S_{lm} for $m = 1 \ldots l$:

`C00, C10, C11, S11, C20, C21, S21, C22, S22, C30, C31, S31, …`

`sbtk.gravity.labels` returns the names to a given degree and `pk` describes the order. Measurement sets (`obs`) leave out C_{00} and store the names in `obs.coef_labels`. Where coefficients are given as matrices (`sbtk.data.from_coefficients`), `C(l+1, m+1)` holds $\bar{C}_{lm}$ and `S(l+1, m+1)` holds $\bar{S}_{lm}$.

Degree 1 and the center of mass

About an expansion origin O, with (x_c, y_c, z_c) the center of mass minus O,

$$\bar{C}_{10} = \frac{z_c}{\sqrt{3}\ R_{\mathrm{ref}}}, \qquad \bar{C}_{11} = \frac{x_c}{\sqrt{3}\ R_{\mathrm{ref}}}, \qquad \bar{S}_{11} = \frac{y_c}{\sqrt{3}\ R_{\mathrm{ref}}}.$$

About the center of mass these vanish. A measured center of mass enters a measurement set as these three coefficients with their uncertainties (`sbtk.data.add_center_of_mass`, `sbtk.units.offset_to_degree1`).

Frame of the coefficients: `coef_about`

Each measurement set states what its coefficients are about (`obs.coef_about`):

- `'origin'`: every coefficient, degree 1 included, and the inertia are about `obs.origin_km`. Simulated measurements are of this kind, with the origin at the true center of mass; `sbtk.data.expand_about` expands them about another point, for example `[0 0 0]`, the origin of the shape model file. Packages about the origin of their shape model file, as the HLP6 makers send, are read as this kind.
- `'center_of_mass'`: the coefficients of degree 2 and higher are about the body's center of mass, as a radio-science solution is; the degree-1 coefficients give the measured position of the center of mass relative to `obs.origin_km`. Packages about the center of mass and fields given to `sbtk.data.add_center_of_mass` are of this kind.

The inversion treats both as described in Density estimation. `sbtk.data.move_origin` changes the origin of `'center_of_mass'` data; only the degree-1 values change.

Frames of a gravity field

A set of gravity coefficients means nothing without the point it is expanded about and the axes its latitude and longitude are measured in. Changing the point or the axes changes every coefficient but not the potential outside the body. This page covers:

- the two changes used for small bodies: moving the expansion point to the center of mass, and turning the axes into the *spin axes* or the *principal axes* of inertia;
- what each change makes zero;
- the frames used for past measured and modeled fields;
- how the frame affects a comparison with a homogeneous body;
- which frame a team that makes data sets (*maker*) or estimates interiors from them (*finder*) should use, including for a body that may be tumbling, such as Dimorphos.

Frame	Expansion point	Axes	Coefficients that vanish
Shape-model origin	origin of the shape-model file	shape-model axes	none
Center of mass, shape-model axes	center of mass	shape-model axes	$\bar{C}_{10}$, $\bar{C}_{11}$, $\bar{S}_{11}$
Center of mass, spin axes	center of mass	z along the axis of the largest moment, x toward the shape-model x-axis	degree 1, and $\bar{C}_{21}$, $\bar{S}_{21}$
Center of mass, principal axes	center of mass	principal axes of inertia	degree 1, and $\bar{C}_{21}$, $\bar{S}_{21}$, $\bar{S}_{22}$

In SBTK, `sbtk.data.simulate` expands coefficients about the center of mass, in the axes of the shape model in use; `sbtk.data.expand_about` expands them about another point. The functions that work with frames are:

- `sbtk.gravity.center_of_mass_frame` gives any field in the shape-model, spin or principal axes, with its principal axes and moments.
- `sbtk.plot.difference_from_homogeneous` compares fields with the homogeneous body in any of the three.
- `sbtk.body.to_frame`, `sbtk.data.to_frame` and `sbtk.interior.to_frame` carry a body, its measurements and its interior or true model into another frame. Every later step (simulation, packages, density grids, plots and comparisons) then works in that frame.

Summary

- **The information is the same in every frame.** The terms a frame makes zero become its definition: three coordinates of the center of mass and up to three angles. Both must be known relative to the shape model.
- **Spin axes** reproduce fields measured by radio science from a body that rotates about its axis of the largest moment (OSIRIS-REx at Bennu): $\bar{C}_{21} = \bar{S}_{21} = 0$, but $\bar{S}_{22} \neq 0$ because the x-axis is set by a surface feature, not by the mass distribution.
- **Principal axes** suit rotational dynamics and bodies without a fixed spin axis (tumbling), and also a synchronous satellite.
- **Makers** should send the measurements, the shape-model file, the true model and the density grid in one frame: spin axes for a body like Didymos, principal axes for Dimorphos (Which frame to use).
- **Finders** should work in the frame of the package they receive and report in it.
- **Against a homogeneous body**, the frame-independent results are the angles between the principal axes of the field and of the homogeneous body (`axes_deg`) and the power of each degree.

Moving the expansion point to the center of mass

Degree 1

About an expansion point O, with $d = (x_c, y_c, z_c)$ the center of mass minus O, the degree-1 coefficients are (Gravity coefficients)

$$\bar{C}_{10} = \frac{z_c}{\sqrt{3}\ R_{\mathrm{ref}}}, \qquad \bar{C}_{11} = \frac{x_c}{\sqrt{3}\ R_{\mathrm{ref}}}, \qquad \bar{S}_{11} = \frac{y_c}{\sqrt{3}\ R_{\mathrm{ref}}},$$

so they vanish about the center of mass and only there. A radio-science field is expanded about the center of mass because the spacecraft orbit is integrated about it. The position of the center of mass relative to the shape model is then a separate result, not part of the coefficients.

Higher degrees

Moving the expansion point changes the coefficients of every degree. The coefficients of degree l about the new point are a linear combination of those of degree $\leq l$ about the old point, weighted by powers of the displacement: lower degrees enter higher ones, never the reverse. For degree 2 this is the parallel-axis theorem. With $I_O = I_{\mathrm{cm}} + M\left(|d|^2 1 - d\, d^{\mathsf{T}} \right)$ and the relations between the inertia tensor and the degree-2 coefficients (Moments of inertia), the unnormalized coefficients about O are

$$C_{20}^O = C_{20} + \frac{2z_c^2 - x_c^2 - y_c^2}{2R^2}, \quad C_{21}^O = C_{21} + \frac{x_c z_c}{R^2}, \quad S_{21}^O = S_{21} + \frac{y_c z_c}{R^2},$$

$$C_{22}^O = C_{22} + \frac{x_c^2 - y_c^2}{4R^2}, \quad S_{22}^O = S_{22} + \frac{x_c y_c}{2R^2},$$

where C_{2m}, S_{2m} are about the center of mass. The change is of second order in $|d\,|\ /\ R$. For the Didymos model below, the shape-model origin is 57 m from the center of mass ($|d\,|\ /\ R_{\mathrm{ref}}$ = 0.13). As a result, $\bar{C}_{21}$ is $3.67{\times}10^{-3}$ about the center of mass but $1.49{\times}10^{-3}$ about the origin.

SBTK applies the translation to all degrees at once with a matrix built from the solid harmonics of point masses (`sbtk.internal.sh_translation`). The map is linear and finite, so the result is exact to rounding. The covariance of a measurement set is propagated through the same matrix.

Rotating the axes

Principal axes from the degree-2 coefficients

About the center of mass, the degree-2 coefficients fix the inertia tensor up to a multiple of the identity (MacCullagh's formula). With unnormalized coefficients,

$$\frac{I - I_{zz}1}{MR_{\mathrm{ref}}^2} = \begin{pmatrix} C_{20} - 2C_{22} & -2S_{22} & -C_{21} \\ -2S_{22} & C_{20} + 2C_{22} & -S_{21} \\ -C_{21} & -S_{21} & 0 \end{pmatrix}.$$

Its eigenvectors are the principal axes of inertia. Its eigenvalues are the principal moments $A \le B \le C$ minus I_{zz}, divided by MR_{ref}^2. SBTK orders the principal axes so that:

- x' is the axis of the smallest moment, y' of the middle one and z' of the largest;
- each axis points to the positive side of the shape-model axis nearest to it;
- the frame is right-handed (`sbtk.internal.principal_axes`).

An eigenvector has no sign, so a field in principal axes must state these choices. Reversing x', for example, changes the sign of every coefficient that is odd in x, such as $\bar{C}_{31}$. When two moments are equal (a body symmetric about an axis), the axes in their plane are not defined.

What a rotation changes

A rotation mixes the $2l + 1$ coefficients of each degree among themselves and with no others. For fully normalized coefficients, the mixing matrix of each degree is orthogonal, so the power of each degree,

$$\sigma_l^2 = \sum_{m=0}^{l} \left(\bar{C}_{lm}^2 + \bar{S}_{lm}^2 \right),$$

is unchanged. SBTK builds the rotation matrix of each degree from the solid harmonics of points and their rotated images (`sbtk.internal.sh_rotation`). It is exact to rounding and also rotates a covariance.

Rotating the coefficients gives the same result as computing the field of the rotated body. The homogeneous field of the shape model with its vertices rotated into the principal axes equals the rotated homogeneous coefficients to 10^{-12} (`testDifferenceInPrincipalAxes`). Simulating a model whose body, components and random field are all carried into another frame also gives the rotated field, density and inertia (`testChangeOfFrameEqualsSimulationInIt`, Validation).

Principal axes

In the principal axes the products of inertia vanish, so

$$\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0, \qquad C_{20} = \frac{A + B - 2C}{2MR^2}, \qquad C_{22} = \frac{B - A}{4MR^2} \ \text{(unnormalized)},$$

and the whole degree-2 power is in $\bar{C}_{20}$ and $\bar{C}_{22}$. Degree 1 stays zero. No rotation makes the coefficients of degree 3 and higher zero: their values change but their power does not.

Spin axes

The spin axes remove only the tilt of the axis of the largest moment:

- z' is the axis of the largest moment (the third principal axis);
- x' is the shape-model x-axis projected onto the plane normal to z';
- $y' = z' \times x'$.

Of all frames with z' along that axis, this one turns the shape-model axes the least.

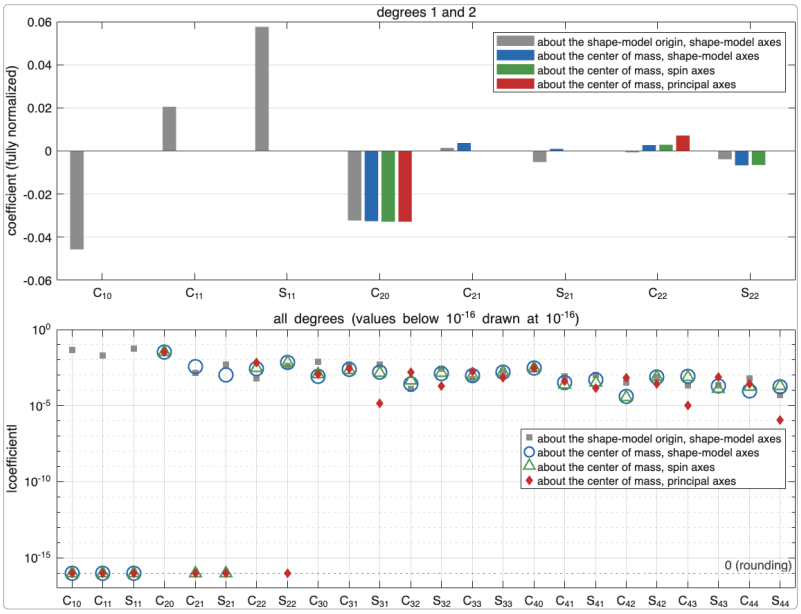
The products of inertia that involve z vanish, so $\bar{C}_{21} = \bar{S}_{21} = 0$. $\bar{S}_{22}$ is in general not zero, and the axis of the smallest moment lies at longitude

$$\lambda_{22} = \frac{1}{2} \operatorname{atan2}\left(\bar{S}_{22}, \bar{C}_{22} \right).$$

These are the conditions of a body-fixed frame of a body in principal-axis rotation, whose spin axis is the axis of its largest moment. The prime meridian here is the direction of the shape-model x-axis.

One field in four frames

Below is the true model of the documentation example (core, torus, regolith, boulder and a random density field in the 9.3 m Didymos model, Results folders and figures) in the four frames, made by `documentation/tools/make_samples.m`:



The degree-1 and degree-2 coefficients (top) and the magnitude of every coefficient (bottom) of one field: about the shape-model origin (gray), and about its center of mass in the shape-model axes (blue), the spin axes (green) and the principal axes (red). Values at the dotted line are zero to rounding.

Output

```
Example true model of Didymos (shape model didymos_g_9309mm_spc_obj_0000n00000_v003), R_ref = 0.43 km

Coefficients of the true model
coef  shape-model origin    center of mass    center of mass    center of mass
       shape-model axes  shape-model axes         spin axes    principal axes
C10       -4.572228e-02      0.000000e+00     -1.289007e-18     -3.710172e-20
C11        2.051808e-02      0.000000e+00      4.758569e-17      1.083458e-17
S11        5.758028e-02      0.000000e+00     -7.278009e-19      1.071706e-18
C20       -3.226924e-02     -3.256747e-02     -3.279488e-02     -3.279488e-02
C21        1.485208e-03      3.665234e-03      4.219222e-18     -1.689108e-17
S21       -5.134086e-03      9.837596e-04      1.553676e-17     -2.088580e-17
C22       -6.163254e-04      2.746776e-03      2.863367e-03      7.152014e-03
S22       -3.893035e-03     -6.638447e-03     -6.553811e-03     -5.354679e-17
```

- **About the shape-model origin**, degree 1 is the largest part of the field: the origin is 57 m from the center of mass.
- **About the center of mass**, degree 1 is zero. The parallel-axis terms above change the degree-2 coefficients; $\bar{S}_{21}$ even changes sign. Degrees 3 and 4 change too.
- **In the spin axes**, $\bar{C}_{21}$ and $\bar{S}_{21}$ are zero to rounding. $\bar{C}_{22}$ and $\bar{S}_{22}$ barely change, because the tilt is only 4.0°.
- **In the principal axes**, $\bar{S}_{22}$ is zero as well. $\bar{C}_{22}$ grows from 2.9×10^{-3} to 7.2×10^{-3}: the axes turn by 33° about z, and the degree-2 power is the same in all three axes.

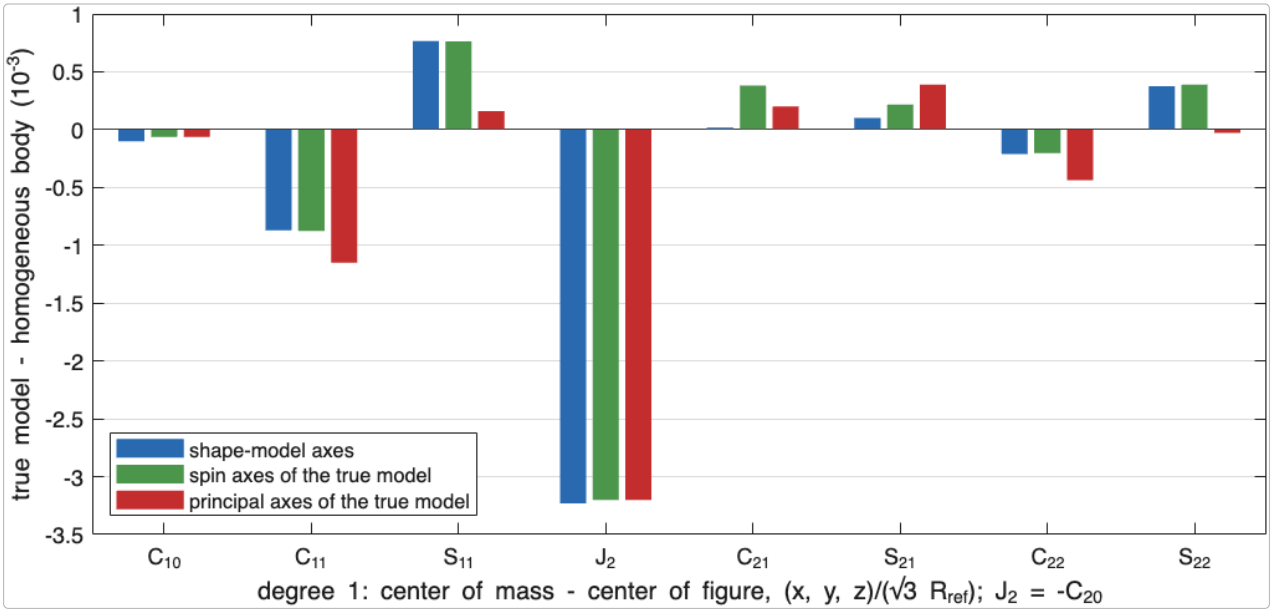
MATLAB script: A field in four frames

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
cof = body.centroid_km;                                          % positions from the center of figure
interior = sbtk.interior.create([sbtk.interior.sphere("core", cof, 0.170, +450), ...
                                 sbtk.interior.sphere("boulder", cof + sbtk.interior.latlon(25, 60, 0.250), 0.040, +900)]);
[exact, truth] = sbtk.data.simulate(body, interior, 4, "frames");

pk = sbtk.internal.sh_packing(4);
about_origin = truth.coef*sbtk.internal.sh_translation(-truth.origin_km, pk, body.R_ref_km)';
cs = sbtk.gravity.center_of_mass_frame(truth, body, "shape");        % degree 1 = 0
cz = sbtk.gravity.center_of_mass_frame(truth, body, "spin");         % and C21 = S21 = 0
cp = sbtk.gravity.center_of_mass_frame(truth, body, "principal");    % and C21 = S21 = S22 = 0
disp(table(cs.labels(1:8)', about_origin(2:9)', cs.coef(1:8)', cz.coef(1:8)', cp.coef(1:8)', 'VariableNames', ...
           ["coefficient" "about the origin" "shape axes" "spin axes" "principal axes"]))
power = @(c, l) sum(c(pk.deg(2:end) == l).^2);
fprintf("degree-2 power: %.6e (shape), %.6e (spin), %.6e (principal)\n", power(cs.coef, 2), power(cz.coef, 2), ...
        power(cp.coef, 2))
fprintf("axis of the largest moment %.2f deg from the shape-model z-axis; smallest at longitude %.1f deg\n", ...
        acosd(cp.principal(3, 3)), 0.5*atan2d(cz.coef(cz.labels == "S22"), cz.coef(cz.labels == "C22")))
fprintf("principal moments minus their mean, I/(M R^2): %.5f %.5f %.5f\n", cp.moments)
```

Comparison with the homogeneous body

`sbtk.plot.difference_from_homogeneous` subtracts the field of the homogeneous body (density M / V) from a field. Both are taken about their own center of mass and expressed in the same axes, which belong to the field (`frame` argument). Degree 1 shows the center of mass minus the center of figure in those

axes.



The example true model minus the homogeneous body, degrees 1 and 2, in units of 10^{-3}, in the shape-model axes (blue), in the spin axes of the true model (green) and in its principal axes (red). Degree 1 is the center of mass minus the center of figure; the zonal term is shown as $J_2 = -\bar{C}_{20}$.

Output

```
True model minus the homogeneous body (both about their own center of mass)
coef   shape-model axes         spin axes    principal axes
J2        -3.231612e-03     -3.202163e-03     -3.202163e-03
C21        1.721657e-05      3.803159e-04      1.995238e-04
S21        1.005986e-04      2.168021e-04      3.896582e-04
C22       -2.103466e-04     -2.023064e-04     -4.383788e-04
S22        3.748967e-04      3.900043e-04     -2.924388e-05

axis of the largest moment from the shape-model z-axis:   true model 3.99 deg, homogeneous body 3.56 deg
axis of the smallest moment from the x-axis, in longitude: true model -33.3 deg, homogeneous body -33.2 deg
principal axes of the homogeneous body from those of the true model: tilt 0.44 deg, turn 0.11 deg
in the spin axes, axis of the smallest moment at (1/2) atan2(S22, C22) = -33.2 deg
principal moments minus their mean, I/(M R_ref^2): true model -0.03368 -0.01521 0.04889
                                                   homogeneous body -0.03663 -0.01703 0.05367
center of mass - center of figure: -0.6 0.6 -0.1 m
```

What each frame shows:

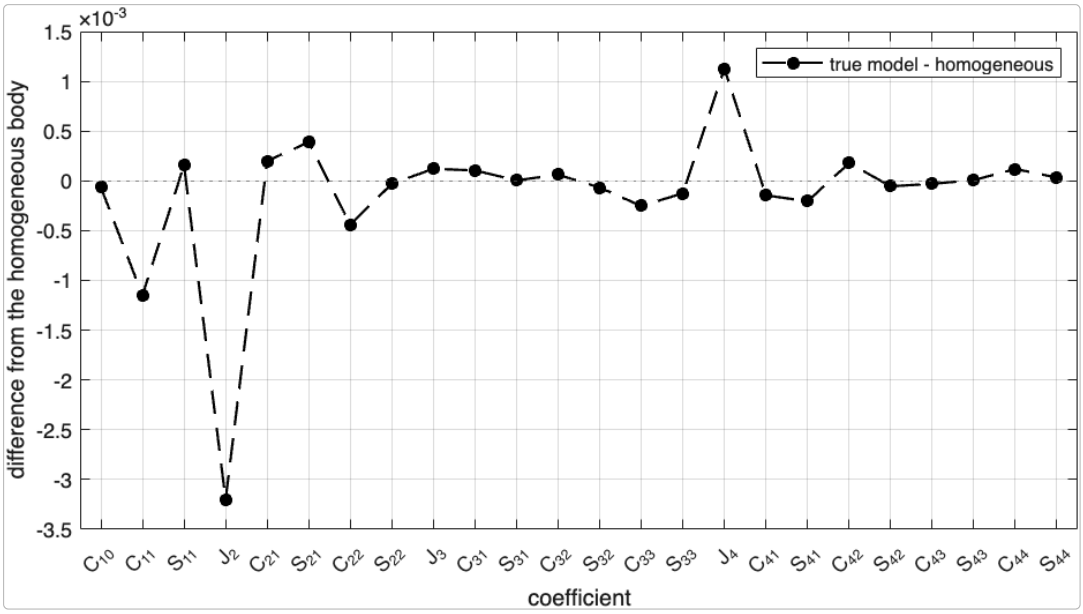
- **Shape-model axes.** Both fields keep their own $\bar{C}_{21}$, $\bar{S}_{21}$, $\bar{S}_{22}$. The differences mix two things: how far the interior turns the principal axes, and how much it changes the moments.
- **Spin axes.** The field has $\bar{C}_{21} = \bar{S}_{21} = 0$. The differences in these two terms are therefore minus the homogeneous body's own terms, and measure the tilt between the two axes of the largest moment (0.44° here). The differences in $\bar{C}_{22}$ and $\bar{S}_{22}$ contain the turn of the equatorial axes and the change in $B - A$.
- **Principal axes.** The field has $\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0$, so all of the misalignment is in the homogeneous body's terms: $\bar{C}_{21}$, $\bar{S}_{21}$ for the tilt and $\bar{S}_{22}$ for the turn (0.11° here). The difference in $\bar{C}_{22}$ is then the change in $B - A$.

For small angles, a rotation ε about one principal axis gives the homogeneous body a product of inertia proportional to ε times the difference of the other two moments:

$$|\bar{C}_{21}| \approx \frac{C-A}{\sqrt{5/3}\ MR^2}\,|\varepsilon_y|, \qquad |\bar{S}_{21}| \approx \frac{C-B}{\sqrt{5/3}\ MR^2}\,|\varepsilon_x|, \qquad |\bar{S}_{22}| \approx \frac{B-A}{2\sqrt{5/12}\ MR^2}\,|\varepsilon_z|\ .$$

A mass excess m at (x, y, z) adds the products of inertia $-mxz$, $-myz$ and $-mxy$. An excess at mid-latitude, away from the z-axis, tilts the axis of the largest moment. An excess in a direction between the x'- and y'-axes turns the axes in the equatorial plane.

In the example, $(C-A)\,/\,(MR^2) = 0.083$ and the tilt is 0.44°, which gives about $5{\times}10^{-4}$, shared between $\bar{C}_{21}$ and $\bar{S}_{21}$ according to the direction of the tilt ($2.0{\times}10^{-4}$ and $3.9{\times}10^{-4}$ in the principal axes): several times the 1σ of 10^{-4} assumed for degree 2. The plot returns the tilt and the turn as `axes_deg`. They, and the power of each degree, do not depend on the frame. Which frame to plot in depends on what the plot is for: the spin axes show the tilt directly in $\bar{C}_{21}$, $\bar{S}_{21}$, as a mission field would; the principal axes isolate all orientation differences in the homogeneous body's terms.



The example true model minus the homogeneous body, degrees 1 to 4, in the principal axes of the true model. Degree 1 is the center of mass minus the center of figure; zonal terms are shown as $J_l = -\bar{C}_{l0}$.

Frames used for measured and modeled fields

Body-fixed frames of measured fields

Gravity fields from radio science, and the shape models made with them, are given in a body-fixed (cartographic) frame:

- z is the spin pole;
- x points to the prime meridian, which is defined by a surface feature;
- the origin is the center of mass found from the tracking data.

NEAR at Eros, Dawn at Vesta and Ceres, and OSIRIS-REx at Bennu all used frames of this kind. For a body in principal-axis rotation, the spin pole is the axis of the largest moment, so $\bar{C}_{21} = \bar{S}_{21} = 0$. The prime meridian, however, has nothing to do with the mass distribution, so $\bar{S}_{22}$ is not zero.

Example: Bennu

The OSIRIS-REx field of Bennu (JPL, 2019) and the Bennu shape model of the Bennu project, with the homogeneous Bennu in the same frame (`documentation/tools/make_samples.m`):

Output

```
Bennu: OSIRIS-REx gravity field (JPL 2019), shape model bennu_higher_qual, R_ref = 0.290 km
Body-fixed frame of the shape model and the field: z the spin pole, x the prime meridian, origin the center of mass
center of figure (centroid of the shape model): 1.34 -0.44 0.24 m from the origin (1.43 m)

coef           measured      1-sigma       homogeneous
C20       -1.926101e-02     5.20e-05     -1.875290e-02
C21       -2.178217e-14     7.75e-10      4.653539e-04
S21        3.000970e-15     7.75e-10     -9.420563e-05
C22        3.064995e-03     5.27e-05      3.538331e-03
S22       -1.094504e-03     1.07e-04     -8.917091e-04

axis of the largest moment from z:      measured 0.000 deg, homogeneous 0.754 deg
axis of the smallest moment, longitude: measured -9.83 deg, homogeneous -7.08 deg
between the axes of the largest moment of the field and of the homogeneous body: 0.754 deg
```

- **The origin is the center of mass.** The field has no degree-1 terms. The centroid of the shape model (the center of figure) is 1.4 m from the origin, which itself is a sign of a non-uniform interior.
- $\bar{C}_{21}$ **and** $\bar{S}_{21}$ **are zero.** Their values are ~10^{-14} with a 1σ of 8×10^{-10}, so they were constrained to zero by taking the spin pole as the axis of the largest moment.
- **The frame is not the principal frame of the shape.** The homogeneous Bennu has $\bar{C}_{21} = 4.7\times10^{-4}$: its axis of the largest moment is 0.75° from the spin pole.
- $\bar{S}_{22}$ **is not zero.** The measured field's axis of the smallest moment is at longitude −9.8°; the homogeneous body's is at −7.1°.

An interior model of Bennu must therefore tilt the axis of the largest moment by 0.75° and turn the equatorial axes by 2.7°. In Cavale et al. (Icarus) this is what the boulders that target $\bar{C}_{21}/\bar{S}_{21}$ and $\bar{C}_{22}/\bar{S}_{22}$ do. Rotating the field 9.8° about z would make $\bar{S}_{22}$ zero. It would also separate the field from the frame of the shape model, the maps and the navigation, and gain nothing, since the angle is already given by $\bar{S}_{22}$ / $\bar{C}_{22}$.

Principal-axis frames

Principal axes are used for:

- rotational dynamics: the equations of rotation (Euler's equations) are diagonal in them;
- model fields made for dynamics studies before a body is visited;
- tumbling asteroids, whose rotation state is solved in the principal axes of the body (for example 4179 Toutatis and 99942 Apophis).

Which frame to use

Makers

The HLP6 makers agreed (20 October 2026) on the first of these two choices.

1. **The frame and origin of the shape model file, as distributed** (the HLP6 choice). The package names the shape model file, which every team has. The gravity field is expanded about its origin, so degree 1 gives the center of mass and is not zero, and $\bar{C}_{21}$, $\bar{S}_{21}$ are those of the whole body in the shape-model axes (the homogeneous 1.165 m Didymos v003 model alone has $\bar{C}_{21} = 4.4{\times}10^{-3}$ about its center of mass with R_{ref} = 410 m). The moments of inertia are about the same origin. Make the exact values with `sbtk.data.simulate` and expand them about `[0 0 0]` with `sbtk.data.expand_about`. Nothing is moved or rotated, so every team can compare with its own computation on the file as it is; a package of the homogeneous body lets the makers check that they compute the same field (Hera blind test).
2. **Moved to the center of mass and rotated** (kept in SBTK; tutorial ex09). The origin is the center of mass of the true model, and the axes are - the **spin axes** for a body observed to rotate about one axis (Didymos): $\bar{C}_{21} = \bar{S}_{21} = 0$ and $\bar{S}_{22} \neq 0$, as a mission field would look. The homogeneous 9.3 m Didymos v003 model is already 3.6° off the shape-model z-axis (3.8° at 1.165 m), and the example true model 4.0°; - the **principal axes** for a body that may be tumbling, or a synchronous satellite (Dimorphos; Tumbling bodies).

The body, the exact values and the true model are carried into that frame (code below), and the package contains the moved and rotated shape model.

In either case, use one frame for everything you send: the measurements, the shape model file, the interior-model package and the density grid, so that the receiving teams' density grids coincide with yours. Define the interior in the frame of the shape model file.

Points to check:

The next three points concern the moved and rotated frame.

- **The frame of the true model is exact.** A real spin pole has an error ε, which leaves $|\bar{C}_{21}|$, $|\bar{S}_{21}| \approx (C-A)\,\varepsilon\,/\,(\sqrt{5/3}\;MR^2)$, about $1.1{\times}10^{-4}$ for Didymos at $\varepsilon = 0.1°$. A package of exact values has $\bar{C}_{21} = \bar{S}_{21} = 0$ exactly; the 1σ a receiving team gives them stands for the pole error. If that 1σ is much smaller, the data define the pole more precisely than a mission would.
- **The rotation carries information about the interior.** It is the same information the field gives in the shape-model axes ($\bar{C}_{21}$, $\bar{S}_{21}$), and the same a mission's pole and field give together, no more. `conventions.txt` records the change of frame from the original shape-model file.
- **The origin is the center of mass.** Mission body-fixed frames put the origin at the estimated center of mass (Bennu: the centroid is 1.4 m from the origin). The shape model, the gravity field and the moments of inertia then share one origin, and a receiving team has nothing to move. The origin of the original file has no physical meaning (Didymos v003: 57 m from the center of figure). Moving the origin adds no information: a package of exact values gives the exact center of mass in `center_of_mass.txt` anyway, together with the center of figure, and their offset, which the interior sets, is the same in every frame. For a package with noise, a mission's origin is the *measured* center of mass; translate to it after `sbtk.data.add_noise` (a translation applies to any set).
- **Uncertainties belong to a frame.** A table of one σ per coefficient describes the solution it comes from, in that solution's axes. In other axes the same covariance has correlations, and two coefficients of one degree can have very different σ (in a principal-axis solution, for example, $\sigma(\bar{C}_{22}) \ll \sigma(\bar{S}_{22})$). Applying such a table coefficient by coefficient in other axes is an approximation. `sbtk.data.to_frame` rotates a full covariance exactly.
- **Components tied to the rotation.** A component defined in the shape-model frame, such as a torus about its z-axis, is tilted relative to the spin axes by the angle between them. If it should lie about the spin axis, as an equatorial torus does, set its axis with `sbtk.interior.align_with_spin_axis` before simulating (Axes of components). `interior_model.txt` gives the geometry in the frame of the package.

Finders

1. **Work in the frame of the package.** Its shape model and its measurements are in the same frame. Define the components of the inversion model in that frame.
2. **A package in the frame and about the origin of the shape model file** (the HLP6 choice): fit the coefficients, degree 1 included, and the moments of inertia as they are, about the origin of the file, with no change of frame. `sbtk.exchange.read_data` returns them about `[0 0 0]` (`coef_about = 'origin'`), and the inversion predicts the model's coefficients and inertia about the same point. The homogeneous body already has a degree 1 (its center of figure, 57 m from the origin of the Didymos v003 file) and a $\bar{C}_{21}$, $\bar{S}_{21}$; the interior has to account for the difference, in particular the offset of the center of mass from the center of figure.
3. **If a field and its shape model are in different frames** (for example a field in principal axes and a shape model in its own frame, with the rotation given), rotate the shape model with `sbtk.body.to_frame`, not the measurements.
4. **Report the solution and its density grid in the frame of the package**, so that the cells coincide with those of the interior-model package.
5. **To interpret the orientation of the field**, compare it with the homogeneous body in the spin or principal axes, and quote the angles `axes_deg`.

Packages moved to the center of mass and rotated

MATLAB script: Measurement and interior-model packages in the spin axes

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
cof = body.centroid_km;                                                  % place from the center of figure
torus = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
torus.center_km = cof;
interior = sbtk.interior.create([sbtk.interior.sphere("core", cof, 0.170, +450), torus]);
interior = sbtk.interior.align_with_spin_axis(body, interior, "torus", 0.01, 20);   % torus about the spin axis
[exact, truth] = sbtk.data.simulate(body, interior, 4, "spin example");      % in the frame of the shape-model file

spin  = sbtk.gravity.center_of_mass_frame(truth, body, "spin");             % or "principal"
body  = sbtk.body.to_frame(body, spin.com_km, spin.axes, "didymos_v003_com_spin_axes");   % origin: center of mass
truth = sbtk.interior.to_frame(truth, spin.com_km, spin.axes);
exact = sbtk.data.to_frame(exact, spin.com_km, spin.axes);                   % C21 = S21 = 0

folder = tempname;                                                       % exact values: no uncertainties, no noise
data  = sbtk.exchange.write_data(folder, "DOC-DATA9-Didymos", body, exact, "Example team");
model = sbtk.exchange.write_model(folder, "DOC-MOD9-Didymos", "DOC-DATA9-Didymos", body, truth, exact, 0.040, ...
                                  "Example team");
[received, rbody] = sbtk.exchange.read_data(data);                           % what a receiving team reads
fprintf("%s: C21 %+.1e, S21 %+.1e\n", rbody.shape.name, received.coef(5:6))
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
received = sbtk.data.set_uncertainties(received, unc);                       % the receiving team's uncertainties
```

The package contains `DOC-DATA9-Didymos_didymos_v003_com_spin_axes.obj`, named after the package because no other package has this shape model: the DART shape model with every vertex moved to the center of mass of the true model and rotated into its spin axes, $x' = (x - c)A$ with c the center of mass and the columns of A the spin axes, both in the frame of the original file. The center of mass is at the origin of that file, and the gravity field, the moments of inertia and every position of both packages are in its frame, so a receiving team uses it as it is. Its `conventions.txt` states what the origin and the axes are (read from the values: the center of mass at the origin, $\bar{C}_{21} = \bar{S}_{21} = 0$) and the change of frame from the original file. `interior_model.txt` describes the torus with its axis in the new frame: along z, because it was aligned with the spin axis. The package has exact values without uncertainties (Packages of exact values). The HLP6 runners have a FRAME setting that does this (Hera blind test).

Tumbling bodies and Dimorphos

The gravity field does not depend on the rotation. The coefficients and the principal axes describe the mass distribution in a body-fixed frame and do not change with the rotation state.

What changes for a tumbling body is the meaning of "spin axis". In principal-axis rotation the body spins about its axis of the largest moment. A tumbling body (non-principal-axis rotation) has angular momentum fixed in space, but its angular velocity moves through the body, so no body-fixed axis is the spin axis. The rotation is described in the principal axes, in which Euler's equations are diagonal.

Dimorphos. Before the DART impact, Dimorphos was taken to rotate synchronously with its orbit about Didymos. In that state the body-fixed frame (z the orbit normal, x toward Didymos on average) is close to its principal axes. The impact changed its orbit, and simulations (Agrusa et al., 2021, Icarus) showed that it could have excited non-principal-axis rotation. Its present rotation state is not known; Hera will measure it. The DART shape model v004 is from before the impact, and the impact may have changed the shape.

For making and finding data sets this means:

- **Frame.** Use the principal axes. They suit a tumbling body and, within a few degrees, a synchronous one. The spin axes remain a valid body-fixed frame for a tumbler, but their z-axis is no longer a spin axis.
- **What the rotation measures.**
- A librating synchronous body gives the libration ratio $(B - A) / C$ (Moments of inertia).
- A tumbling body gives two ratios of principal moments, for example A / C and B / C, because Euler's equations depend only on ratios. Together with the degree-2 field, which fixes $B - A$ and $C - (A + B) / 2$ in units of MR^2, two ratios fix all three moments, including the trace that the gravity field cannot give.

A tumbling Dimorphos would therefore constrain the radial density structure better than a librating one. - **Moment-ratio measurements in SBTK.** In principal axes the ratios are the linear rows $I_{xx} - r_A I_{zz} = 0$ and $I_{yy} - r_B I_{zz} = 0$, with σ = $\sigma_r I_{zz}$, added with `sbtk.data.add_moi_row`. In other axes the weights follow from the principal axes of the degree-2 field. The exchange format gives inertia elements and the libration ratio but not moment ratios. A group can give I_{xx}, I_{yy}, I_{zz} in principal axes, which are A, B, C, or agree on an extension of the format. - **The interior model is made as before.** The true interior is defined in the shape-model frame and carried into the principal axes; the rotation state affects only the rotation measurements. Finders invert as usual in the frame of the package.

MATLAB script: Ratios of principal moments of a tumbling Dimorphos

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Dimorphos", sbtk.shape.bundled("dimorphos"), 2.700e9, 0.10);
interior = sbtk.interior.create(sbtk.interior.ellipsoid("core", body.centroid_km + [0.004 0 0], [0.045 0.040 0.028], ...
                                                        [15 0 0], +400));
[exact, truth] = sbtk.data.simulate(body, interior, 3, "tumbling example");
cp = sbtk.gravity.center_of_mass_frame(truth, body, "principal");
exact = sbtk.data.to_frame(exact, [0 0 0], cp.axes);           % Ixx = A, Iyy = B, Izz = C
I = exact.moi_tensor_norm;
rA = I(1, 1)/I(3, 3);  rB = I(2, 2)/I(3, 3);  sigma_r = 0.005;
obs = sbtk.data.select_moi(exact, strings(1, 0));
obs = sbtk.data.add_moi_row(obs, "A_over_C", [1 0 -rA 0 0 0], 0, sigma_r*I(3, 3));
obs = sbtk.data.add_moi_row(obs, "B_over_C", [0 1 -rB 0 0 0], 0, sigma_r*I(3, 3));
fprintf("A/C = %.4f, B/C = %.4f; off-diagonal elements in principal axes: %.1e\n", rA, rB, max(abs(I(~eye(3)))))
```

Moments of inertia

Inertia tensor

The inertia tensor about the expansion origin, in the axes of the shape-model frame, is

$$I_{ij} = \int \rho \left(r^2 \delta_{ij} - x_i x_j \right) dV,$$

so the products of inertia carry the minus sign: $I_{xy} = -\int \rho \, xy \, dV$. SBTK stores the six independent elements in the order $[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}]$, normalized by MR_{ref}^2 in measurement sets and models (`truth.inertia_norm`, `fm.inertia_norm`) and in kg m² in reports (Units).

Relation to the degree-2 coefficients

About the center of mass, in the same axes and with the same R_{ref}, the unnormalized degree-2 coefficients are

$$C_{20} = \frac{I_{xx} + I_{yy} - 2I_{zz}}{2MR^2}, \quad C_{21} = -\frac{I_{xz}}{MR^2}, \quad S_{21} = -\frac{I_{yz}}{MR^2}, \quad C_{22} = \frac{I_{yy} - I_{xx}}{4MR^2}, \quad S_{22} = -\frac{I_{xy}}{2MR^2},$$

and $\bar{C}_{lm} = C_{lm} \,/\, N_{lm}$ (Gravity coefficients). These relations are verified by `testDegreeTwoAndInertia` (Validation).

The degree-2 field fixes five combinations of the six elements. The sixth, the trace (for example through the polar moment I_{zz}), is the information that a moment-of-inertia measurement adds to the gravity field: it measures how strongly the mass is concentrated toward the center, which the gravity coefficients alone cannot distinguish from other radial distributions of the same field. Tutorial ex08 compares inversions with different inertia measurements.

Measurement rows

A moment-of-inertia measurement is one linear combination of the elements:

$$\mathbf{w} \cdot \frac{[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}]}{MR_{\mathrm{ref}}^2} = v \pm \sigma,$$

stored as a `moi` row with `weights` $\mathbf{w}$, `value` v and `sigma` σ. A single element has one weight equal to 1 (`sbtk.data.add_moi`); any other combination is added with `sbtk.data.add_moi_row`. `sbtk.data.select_moi` chooses the rows a data set contains and `opts.moi_labels` (`opts.moi_labels`) the rows an inversion fits.

Libration ratio

The libration ratio is

$$k = \frac{B - A}{C},$$

with $A \le B$ the principal moments of inertia about the axes in the xy-plane and $C = I_{zz}$; the spin axis z is taken as a principal axis, which holds exactly in the spin or principal axes of the field (Frames of a gravity field). For a tumbling body the rotation gives ratios of principal moments instead (Tumbling bodies). With a the angle of the axis of A from the x-axis toward $+y$,

$$B - A = \cos(2a)\,(I_{yy} - I_{xx}) - \sin(2a) \cdot 2I_{xy},$$

which is linear in the tensor once a is fixed. `sbtk.data.add_libration` takes a from the measured $\bar{C}_{22}$ and $\bar{S}_{22}$, since $(\cos(2a), \sin(2a))$ is proportional to (C_{22}, S_{22}) about the center of mass, and adds the linear row

$$\cos(2a)\,(I_{yy} - I_{xx}) - \sin(2a) \cdot 2I_{xy} - k\, I_{zz} = 0 \pm \sigma_k \, I_{zz},$$

labeled `libration`, with weights $[-\cos(2a), \cos(2a), -k, -2\sin(2a), 0, 0]$ (`sbtk.internal.libration_weights`). Together with C_{22} and S_{22}, which give $B - A$, the ratio measures C with a relative precision of about $\sigma_k \,/\, k$. If $C_{22} = S_{22} = 0$ the principal axes in the xy-plane are undefined and the function stops with `sbtk:data:axisymmetric`.

Libration ratio from a libration amplitude

For a synchronously rotating satellite in a Keplerian orbit of eccentricity e about a point-mass primary, the forced libration angle γ of the long axis from the direction to the primary obeys, to first order in e,

$$\ddot{\gamma} + 3n^2 k\, \gamma = 2en^2 \sin M,$$

with mean motion n and mean anomaly M, so $\gamma = G \sin M$ with

$$G = \frac{2e}{3k - 1}, \qquad k = \frac{1}{3}\left(1 + \frac{2e}{G}\right).$$

`sbtk.data.libration_ratio_from_amplitude` returns k and its 1σ from G, e and their uncertainties by first-order error propagation. $G < 0$ for $k < 1 \,/\, 3$ (elongations up to about 1.4 : 1). The sensitivity of G to k grows near $k = 1 \,/\, 3$, where the free libration period equals the orbital period. Free libration, the

primary's oblateness and the mutual dynamics of a binary are not included.

Shape models

A shape model is a closed triangular mesh, stored in SBTK as a `shape` structure with vertices in km.

Reading and creating shape models

Function	Input
`sbtk.shape.load`	Wavefront OBJ file: `v x y z` and `f i j k` records; normals, texture coordinates and faces written as `f i/t/n` are accepted, other records ignored. The length unit of the file (`"km"`, `"m"` or `"cm"`) is a required argument.
`sbtk.shape.bundled`	A model shipped with SBTK, by name (table below).
`sbtk.shape.from_arrays`	Face and vertex arrays with their length unit.
`sbtk.shape.ellipsoid`	A triaxial ellipsoid: an icosahedron subdivided `level` times (20·4^level facets) with vertices projected onto the ellipsoid.
`sbtk.shape.write_obj`	Writes a shape model as OBJ with vertices in km.

Requirements

The forward model integrates over the cones from the expansion origin to every facet (Gravity and moment calculations). The mesh must therefore be

- **closed**: every edge is shared by exactly two facets that traverse it in opposite directions;
- **ordered outward**: the enclosed volume computed from the facets is positive. A mesh ordered inward is reversed by `sbtk.gravity.polyhedron`, and by `sbtk.gravity.geometry` when every facet faces the expansion origin;
- **star-shaped about the expansion origin**: every facet faces away from it, so that each ray from the origin crosses the surface once.

`sbtk.shape.properties` reports all three, the last about the centroid (`props`). `sbtk.gravity.geometry` stops with `sbtk:gravity:notStarShaped` when facets face the expansion origin, unless their cones hold less than 10^{-6} of the volume. Such small overhangs occur in high-resolution models: seen from a point 4.6 m from the center of figure of the 0.243 m Dimorphos model, two facets of 0.025 m² face it, with 2.5×10^{-9} of the volume. Their cones count with their sign, which is exact for the body (the signed cones of any closed mesh add up to its volume integrals) and leaves an error of the order of their volume in layers and sectors.

MATLAB script: Checking a shape model

```
run("/path/to/SBTK/sbtk_setup.m")
shape = sbtk.shape.bundled("didymos");
p = sbtk.shape.properties(shape)
fprintf("volume %.5f km^3, centroid [%.1f %.1f %.1f] m\n", p.volume_km3, 1e3*p.centroid_km)
```

Shape models in the repository

Files in `data/shapes/`, lengths in km, in the body-fixed frame of each model (z along the spin axis).

`bundled` name	File	Sampling	Facets	Volume (km³)	Description
`"didymos"`	`didymos_g_9309mm_spc_obj_0000n00000_v003.obj`	9.3 m	49,152	0.20336	DART stereophotoclinometry (SPC) model of Didymos, version 3
`"didymos_highres"`	`g_01165mm_spc_obj_didy_0000n00000_v003.obj`	1.165 m	3,145,728	0.20381	The same model at full resolution (not in the repository, Installation)
`"didymos_every8"`	`g_01165mm_spc_obj_didy_0000n00000_v003_every8.obj`	9.3 m	49,152	0.20375	Every 8th grid point of the 1.165 m model
`"dimorphos"`	`g_00243mm_spc_obj_dimo_0000n00000_v004_every8.obj`	1.94 m	49,152	0.0017597	Every 8th grid point of the DART SPC model of Dimorphos, version 4 (before the DART impact)
`"dimorphos_highres"`	`g_00243mm_spc_obj_dimo_0000n00000_v004.obj`	0.243 m	3,145,728	0.0017602	The Dimorphos model at full resolution (not in the repository)
`"bennu"`	`bennu_higher_qual.obj`		196,608		OSIRIS-REx shape model of Bennu
`"example_asteroid"`	`example_asteroid.obj`				A synthetic body of about 1.2 km, used by the project template
`"dimorphos_placeholder"`	`dimorphos_placeholder.obj`		20,480		A synthetic stand-in used by tests; not a mission model

The 9.3 m Didymos file is distributed separately by the DART team and is not a subsample of the 1.165 m model. For a homogeneous body with $R_{\mathrm{ref}} = 0.43$ km, `tools/make_reduced_shapes.m` gives these differences from the full-resolution models:

Model compared with the full resolution	Volume	Center of figure	Largest coefficient difference, degree 2 / 3 / 4
`didymos_every8`	−0.030%	0.01 m	1.2e-5 / 5.0e-6 / 2.8e-6
`didymos` (9.3 m file)	−0.224%	0.35 m	3.1e-4 / 1.7e-4 / 4.2e-5
`dimorphos` (every 8th point, $R_{\mathrm{ref}} = 0.10$ km)	−0.031%	0.00 m	1.5e-5 / 1.3e-6 / 5.1e-6

Differences of this size between the shape used to simulate data and the shape used to invert them act as a systematic error in the coefficients. Measurement packages therefore name the shape model they were made with, and include it unless it is one of SBTK's files as distributed (Exchange packages). The HLP6 makers agreed on `didymos_highres`, the 1.165 m model (Didymos and Dimorphos values).

Reducing high-resolution models

High-resolution shape models have more facets than coefficients to degree 4 or 5 require. Two functions reduce them:

- `sbtk.shape.reduce_spc` keeps every `step`-th grid point of an SPC model stored in its grid order (6 cube faces of $(q+1)^2$ vertices, $12q^2$ facets), triangulated as the original. The grid is radial, so the result stays star-shaped about the body's center; a cell whose triangles would face the center is split along its other diagonal.
- `sbtk.shape.reduce` applies MATLAB's `reducepatch` to any mesh and checks that the result is closed, outward and star-shaped about its centroid; a general reduction can fold facets, in which case it stops with `sbtk:shape:reduce`.

MATLAB

```
big   = sbtk.shape.load("g_00243mm_spc_obj_dimo_0000n00000_v004.obj", "km");   % q = 512, 3,145,728 facets
small = sbtk.shape.reduce_spc(big, 8);                                        % q = 64, 49,152 facets
sbtk.shape.write_obj(small, "g_00243mm_spc_obj_dimo_0000n00000_v004_every8.obj");
```

Bodies

A body (`body`) is a shape model with its total mass and the reference radius of its gravity coefficients. `sbtk.body.from_mass` takes the mass; `sbtk.body.create` takes the bulk density and computes the mass from the volume. The bulk density is M / V; the background density of a model with components differs from it (Density distributions).

MATLAB fragment. Requires the variables defined in Checking a shape model.

```
body = sbtk.body.from_mass("Didymos", shape, 5.273e11, 0.43);    % mass [kg], reference radius [km]
sbtk.body.summary(body)
sbtk.plot.shape(body);
```

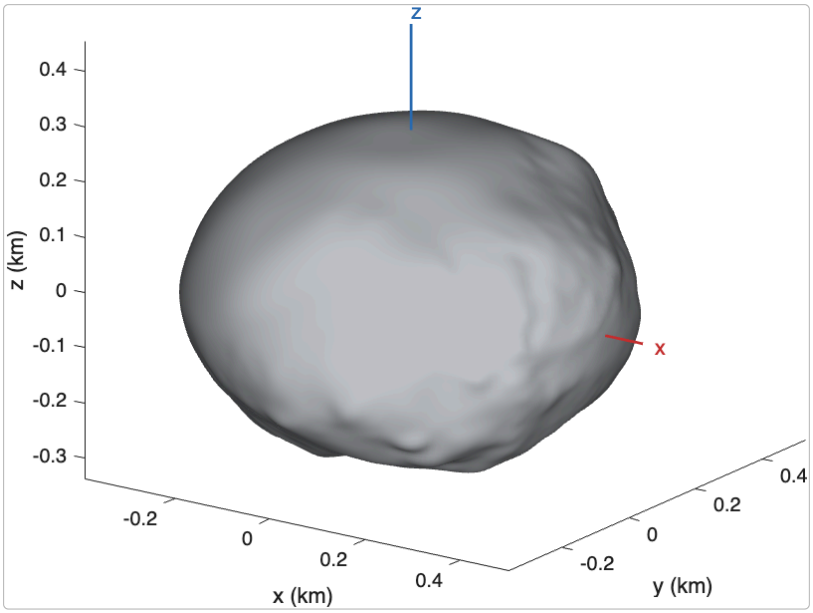


Figure produced by `sbtk.plot.shape` for the 9.3 m Didymos model: the surface, with the shape-model axes drawn from the center of figure (x red, z blue; in this view the y-axis is behind the body).

Density distributions

Density model

The density at a point $\mathbf{x}$ inside the body is

$$\rho(\mathbf{x}) = \rho_{\mathrm{bg}} + \sum_{c\,:\,\mathbf{x}\,\in\,c} \Delta\rho_c + \delta\rho(\mathbf{x}),$$

where

- ρ_{bg} is the **background density**, the density outside every component;
- $\Delta\rho_c$ is the **relative density** of component c: its density minus the background density (`component.drho_kgm3`). Where components overlap their relative densities add. Everything outside the shape model is excluded;
- $\delta\rho$ is an optional Gaussian **random density field**.

The background density is not an input of a simulation: `sbtk.gravity.forward` sets it so that the total mass equals the mass of the body,

$$\rho_{\mathrm{bg}} = \frac{M - \sum_c \Delta\rho_c V_c - \int \delta\rho \, dV}{V},$$

with V_c the volume of component c inside the body. In an inversion it is estimated or held at the bulk density (Density estimation). `sbtk.data.simulate` stops with `sbtk:data:negativeDensity` if the density is negative anywhere.

An interior model (`interior`) holds the components and the random field; `sbtk.interior.create` builds it. `sbtk.interior.density_at` evaluates the density of any model at given points.

Component types

Each constructor returns one `component`. A point $\mathbf{x}$ (shape-model frame) belongs to the component when the condition in the table holds. $\mathbf{o}$ is the expansion origin, t_s the distance from $\mathbf{o}$ to the surface along the ray through $\mathbf{x}$, $r = |\mathbf{x} - \mathbf{o}|$.

Type	Constructor	Membership condition	Measured from
Sphere	`sbtk.interior.sphere(name, center_km, radius_km, drho_kgm3)`	$\lvert\mathbf{x} - \mathbf{c}\rvert < r_s$	shape-model origin
Ellipsoid	`sbtk.interior.ellipsoid(name, center_km, semi_axes_km, angles_deg, drho_kgm3)`	$\sum_i \left([R^{\mathsf{T}}(\mathbf{x} - \mathbf{c})]_i / a_i \right)^2 < 1$	shape-model origin
Torus	`sbtk.interior.torus(name, ring_radius_km, tube_radius_km, height_km, drho_kgm3)`	$\left(\sqrt{q_1^2 + q_2^2} - R_r \right)^2 + (q_3 - h)^2 < r_t^2$ with $q = A^{\mathsf{T}}(x - c)$	its axis: the third axis of `component.axes` A through `component.center_km` c (the shape-model z-axis unless set)
Layer	`sbtk.interior.layer(name, top_depth_km, bottom_depth_km, drho_kgm3)`	$d_{\mathrm{top}} \le t_s - r < d_{\mathrm{bottom}}$	rays from $\mathbf{o}$
Sector	`sbtk.interior.sector(name, frac, lat_deg, lon_deg, drho_kgm3)`	$f_{\mathrm{in}} \le r / t_s < f_{\mathrm{out}}$, $\phi_1 \le \phi < \phi_2$ ($\le 90°$ if $\phi_2 = 90°$), λ in $[\lambda_1, \lambda_2)$ modulo 360°	rays from $\mathbf{o}$

Notes on the types:

- **Sphere.** A sphere entirely inside the body is integrated exactly as a point mass; one that crosses the surface is clipped. Its center can be estimated in an inversion (`component.estimate_position`).
- **Ellipsoid.** $R = R_z(\mathrm{yaw}) R_y(\mathrm{pitch}) R_x(\mathrm{roll})$ (Coordinate systems).
- **Torus.** Lies about its axis, the shape-model z-axis through the origin as constructed; set `component.center_km` to move it (for example to the center of figure) and turn it with `sbtk.interior.set_axis`. It may be buried, cross the surface or extend outside the body. Only the part inside the body counts.
- **In another frame.** `sbtk.interior.to_frame` moves and turns every component with the frame: centers, the axes of ellipsoids, the axis of a torus and the axes in which a sector's latitude and longitude are measured (`component.axes`), and the random field. The density at every point is unchanged (Frames of a gravity field).
- **Layer.** Depth is measured along the ray from the expansion origin, not along the surface normal; `layer("regolith", 0, 0.012, …)` is the outermost 12 m along each ray.
- **Sector.** Radial fractions follow the shape: `sector("shell", [0.5 0.75], [-90 90], [-180 180], …)` is a shell between half and three quarters of the way to the surface. `sbtk.interior.sector` stops with `sbtk:interior:sectorRange` unless each range increases and the longitude range spans at most 360°.

`sbtk.interior.latlon` converts latitude, longitude and distance to a position, for example to place a sphere. `sbtk.interior.summary` prints a list of components.

Where to place components

Spheres, ellipsoids and tori are placed in the coordinates of the shape-model file, so a center `[0 0 0]` is the **origin of the file**. Layers and sectors are measured along rays from the expansion origin, which in a simulation is the **center of mass**. The **center of figure** is the centroid of the shape model (`body.centroid_km`), the center of mass of a homogeneous body (Coordinate systems).

Place components from the center of figure:

- **It depends only on the shape.** The center of mass of a model depends on its components, so positions measured from it would depend on what they define.
- **"Central" means concentric with the figure.** An offset of a component from the center of figure is what moves the center of mass away from it, which the data measure (degree 1). That offset should be chosen, not inherited from where the file has its origin.
- **The origin of a shape-model file can be far from the body's center.** It is 57 m from the center of figure in the DART Didymos model v003 (13% of the reference radius of 430 m), 0.75 m in the DART Dimorphos model v004, and 1.4 m in the OSIRIS-REx Bennu model, whose origin is the center of mass found from the tracking data.

MATLAB fragment. Requires the variables defined in An interior model of Didymos.

```
cof = body.centroid_km;                                % center of figure
core = sbtk.interior.sphere("core", cof + [0.010 0 0], 0.170, +450);   % 10 m from the center of figure along x
t = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
t.center_km = cof;                                     % about the z-axis through the center of figure
```

In the interior of Cavale et al. (Icarus) for Bennu (Bennu) every position is from the origin of the Bennu shape model, which is its measured center of mass: there the two choices differ by 1.4 m. With a pre-mission model such as the DART Didymos v003 they differ by 57 m. A central core placed at the origin is then 57 m off center, and in a Didymos model a core of radius 170 m and relative density +450 kg/m³ moves the center of mass about 1 m from the center of figure.

Axes of components

The axis of a torus, the axes of an ellipsoid and the axes in which a sector's latitude and longitude are measured are `component.axes`; their third axis is "the axis" of the component.

- `sbtk.interior.set_axis(c, direction)` turns a component so that its axis points along a direction (the smallest rotation, about its center).
- `sbtk.interior.align_with_spin_axis(body, interior, names, tol_deg, max_iter)` sets the axis of the named components along the axis of the largest moment of the whole model: the spin axis of a body in principal-axis rotation, about which an equatorial torus forms. That axis depends on the components themselves, so the function iterates (simulate to degree 2, take the axis, turn the components, repeat) until the axis moves less than `tol_deg`, at most `max_iter` times (`sbtk.internal.fixed_point`, `sbtk:iteration:notConverged`). Other components are not changed.

MATLAB fragment. Requires the variables defined in An interior model of Didymos.

```
cof = body.centroid_km;
t = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
t.center_km = cof;
model = sbtk.interior.create([sbtk.interior.sphere("boulder", cof + sbtk.interior.latlon(25, 60, 0.250), 0.040, +900), t], ...
                             sbtk.interior.no_random_field());
[model, spin_axis, n] = sbtk.interior.align_with_spin_axis(body, model, "torus", 0.01, 20);
fprintf("torus axis %.2f deg from the shape-model z-axis after %d simulations\n", acosd(spin_axis(3)), n)
```

For a true model these are modeling choices. In an inversion the geometry of a torus or an ellipsoid (center, height, axis) can be estimated instead, with bounds and priors (Density estimation).

Sector grids

`sbtk.interior.sector_grid(frac_edges, lat_edges_deg, lon_edges_deg)` divides the whole body into sectors, one per radial shell, latitude band and longitude band, named `s<shell><lat><lon>` (`s111` is the innermost shell, the southernmost band, the first longitude band). This is a density model without an assumed structure. Because the sectors fill the body, the background density and a common change of all sector densities are indistinguishable: an inversion must hold the background at the bulk density (`background = "bulk"`) or use regularization (Sector grids).

Random density field

`sbtk.interior.random_field(sigma_kgm3, corr_length_km, seed)` is a Gaussian random field with standard deviation σ and correlation $\exp(-|\Delta\mathbf{x}|^2/(2L^2))$, represented by 300 Fourier modes (`random_field`). It is defined at every point independently of the expansion origin and is the same for the same seed. `sbtk.interior.no_random_field` gives no field. A random field is used in simulations; inversions do not estimate one.

Prescribed components

A component with `component.estimate_density` set to false keeps its relative density in an inversion, for example a surface layer of known density. The constructors set it to true.

Example

MATLAB script: An interior model of Didymos

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
cof = body.centroid_km;                                          % positions from the center of figure
torus = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
torus.center_km = cof;                                           % about the z-axis through it
components = [ ...
    sbtk.interior.sphere("core", cof, 0.170, +450), ...
    torus, ...
    sbtk.interior.layer("regolith", 0, 0.012, -900), ...
    sbtk.interior.sphere("boulder", cof + sbtk.interior.latlon(25, 60, 0.250), 0.040, +900)];
interior = sbtk.interior.create(components, sbtk.interior.random_field(40, 0.06, 7));
sbtk.interior.summary(interior.components)

geo = sbtk.gravity.geometry(body, body.centroid_km, 0);          % degree 0: no coefficients needed
fm  = sbtk.gravity.forward(geo, interior, body.mass_kg);
fprintf("background density %.0f kg/m^3\n", fm.rho_bg_kgm3)
model = interior;
model.rho_bg_kgm3 = fm.rho_bg_kgm3;                               % background + components + field
rho = sbtk.interior.density_at(model, body.centroid_km + [0 0 0; 0.30 0 0], geo)
```

Here the geometry is built about the center of figure, so the layer is measured from it; `sbtk.data.simulate` uses the center of mass instead and returns the model with its background density as `truth`. The last line evaluates the density at the center of figure and 300 m from it along x. Further examples: Density models by component type.

Gravity and moment calculations

The forward model computes, for a density model inside a shape model, the gravity coefficients (Gravity coefficients), the center of mass and the inertia tensor (Moments of inertia) about an expansion origin $\mathbf{o}$.

Facet cones

`sbtk.gravity.geometry(body, origin_km, degree)` divides the body into tetrahedral cones, one per surface facet, with apex at $\mathbf{o}$ and the facet as base. This partition is exact when the shape is star-shaped about $\mathbf{o}$ (small overhangs count with their sign: Shape models). For every cone f it stores, per unit density (`geo`):

Quantity	Definition	Field
Spherical-harmonic integrals	$T_{f,lm} = \int_f (r / R_{\text{ref}})^l \bar{Y}_{lm}\, dV$, with $\bar{Y}_{lm} = \bar{P}_{lm}(\sin\phi)\{\cos, \sin\}(m\lambda) / (2l+1)$	`geo.T`
Volume	V_f	`geo.V`
First moments	$\int_f \mathbf{x}\, dV$	`geo.m1`
Second moments	$\int_f [x^2, y^2, z^2, xy, xz, yz]\, dV$	`geo.Q`

$T_{f,lm}$ comes from the polyhedral kernel `sbtk.internal.ComputePolyhedralCSBar_mex` applied to the tetrahedron of each cone. With this definition a body of uniform density ρ has $\bar{C}_{lm} = \rho \sum_f T_{f,lm} / M$ (with the unit factor of Units).

Scaling within a cone

Shrinking a cone about its apex by a factor s multiplies $r^l\, dV$ by s^{l+3}. The part of cone f between the scaled facets s_a and s_b $(0 \le s_a < s_b \le 1)$ therefore contributes exactly

$$T_{f,lm}\,(s_b^{l+3} - s_a^{l+3}), \qquad V_f\,(s_b^3 - s_a^3), \qquad \mathbf{m}_{1,f}\,(s_b^4 - s_a^4), \qquad Q_f\,(s_b^5 - s_a^5)\,.$$

The validation test `testConeScaling` checks this law against the kernel to a relative error below 10^{-13}.

Integrals of components

`sbtk.gravity.component_integrals(geo, c)` returns the integrals of the part of a component inside the body per unit density (`integrals`):

- **Sphere entirely inside the body:** exact point-mass integrals.
- **Layer:** along each of 9 sub-rays per facet, the radial interval between the two depths gives s_a and s_b directly.
- **Sector:** the radial fractions give s_a and s_b; whether a sub-ray lies within the latitude and longitude range is tested at the middle of the interval.
- **Sphere crossing the surface, ellipsoid, torus:** a sub-ray crosses the component's surface where a polynomial in the distance along it vanishes: a quadratic for a sphere or an ellipsoid, a quartic for a torus (solved as the eigenvalues of its companion matrix). Its real roots between the expansion origin and the body's surface split the sub-ray, and each piece whose midpoint is inside the component contributes $s_b^p - s_a^p$, so a ray may enter and leave the component several times, and a ray that only grazes it contributes a correspondingly short piece.

For each facet the contributions are averaged over its 9 sub-rays and multiplied by the cone's integrals. The result is exact when the radial interval is the same along every sub-ray of a cone, and converges with mesh resolution otherwise (Assumptions and limitations).

`sbtk.gravity.random_field_integrals` evaluates the random field at the centroid of each of 24 radial shells of every cone (`samples`) and integrates each shell exactly with that value.

Coefficients, center of mass and inertia

`sbtk.gravity.forward(geo, interior, mass_kg)` combines the integrals of the body (U, V, $\mathbf{m}_1$, Q), of the components (U_c, V_c, ...) and of the random field (U_δ, ...):

$$U = \rho_{\text{bg}} U_{\text{body}} + \sum_c \Delta\rho_c U_c + U_\delta, \qquad \bar{C}_{lm}, \bar{S}_{lm} = \frac{U_{lm}}{M},$$

$$\mathbf{x}_{\text{cm}} = \mathbf{o} + \frac{\rho_{\text{bg}} \mathbf{m}_{1,\text{body}} + \sum_c \Delta\rho_c \mathbf{m}_{1,c} + \mathbf{m}_{1,\delta}}{M}, \qquad I_{xx} = Q_{yy} + Q_{zz},\ I_{xy} = -Q_{xy},\ \ldots$$

with ρ_{bg} chosen so that the mass is M (Density distributions). It returns the coefficients to degree L, the center of mass and $[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}] / (MR_{\text{ref}}^2)$ (`fm`).

Simulated measurements

`sbtk.data.simulate(body, interior, degree, name)` runs the forward model about the center of mass:

1. geometry and forward model about the shape-model origin;
2. geometry and forward model about the resulting center of mass, repeated until the center of mass and the expansion origin differ by less than 1 mm (layers and sectors depend on the origin);

3. a check that the density is not negative at the sample points of `sbtk.internal.body_samples` (24 radial shells per cone), otherwise `sbtk:data:negativeDensity`;
4. the exact coefficients of degree 1 to L, the six inertia elements and the libration row, with the ratio $k = \sqrt{(I_{yy} - I_{xx})^2 + 4I_{xy}^2} \;/\; I_{zz}$ of the principal moments, as a measurement set without uncertainties (`obs`), and the true model (`truth`).

No measurement uncertainties enter the forward model. To make measurements of the true model, `sbtk.data.set_uncertainties` (or `sbtk.data.use_uncertainties`, `sbtk.data.set_covariance`, `sbtk.data.set_moi_sigma`) gives the exact values uncertainties and `sbtk.data.add_noise` draws one noisy realization (Measurement sets). Selecting which inertia quantities a data set contains: `sbtk.data.select_moi`.

Potential and acceleration at points

The spherical-harmonic expansion converges only outside the sphere about its expansion point that encloses the body, and slowly near it. For points closer in, or on the surface, SBTK computes the potential and the acceleration from the shape model directly:

- `sbtk.gravity.polyhedron`: a homogeneous polyhedron, exactly, with the closed form of Werner and Scheeres (1997, *Celestial Mechanics and Dynamical Astronomy* 65, 313–344), a sum over the edges and faces of the shape model. It holds outside, on and inside the body; on an edge the edge terms are zero.
- `sbtk.gravity.field_at`: any model. The background density fills the polyhedron, exactly. A sphere entirely inside the body is exact too: a point mass outside it, a uniform ball inside it. The other departures from the background density (spheres that reach the surface, ellipsoids, tori, layers, sectors, the random field) are taken on the density grid of `sbtk.exchange.density_grid`, each cell a uniform cube with the closed form of Nagy, Papp and Benedek (2000, *Journal of Geodesy* 74, 552–560). A homogeneous model, or one whose components are all spheres inside the body, is exact; otherwise the cubes follow the boundaries of the components to within a cell.

The potential is positive, $U = G \int \rho \,/\, |\, r - r' \,|\, dV$, and the acceleration is its gradient; the rotation of the body is not included. `tests/unit/test_gravity.m` checks that the two closed forms give the same cube to 10^{-10}, that the divergence of the acceleration is $-4\pi G\rho$ inside the body, and that the acceleration is the gradient of the potential.

For the homogeneous Didymos (1.165 m model, 2600 kg/m³, R_{ref} = 410 m), the expansion about the origin of the file compared with the exact potential (the sphere that encloses the body has a radius of 449.7 m):

Distance from the origin	Expansion to degree 10	to degree 30
492 m (1.2 R_{ref})	1.5×10^{-4}	7.8×10^{-7}
600 m	6.7×10^{-6}	2.2×10^{-10}
707 m	1.9×10^{-7}	1.9×10^{-12}
2449 m	5.8×10^{-13}	8×10^{-15}

(relative difference of the potential at one point each). The polyhedron takes about 1 s per point for 3 million facets.

Computation time

Times measured by `tools/benchmark.m` on Didymos (49,152 facets, degree 4), MATLAB R2025a on an Apple-silicon laptop:

Step	Time
Geometry, degree 4	0.22 s
Simulation: 4 components and a random field, center-of-mass frame	3.6 s
Inversion setup: 3 components	1.3 s
Inversion run	0.25 s
Comparison with the true model	0.53 s

The geometry scales with the number of facets and with the number of coefficients: the agreed 3,145,728-facet Didymos model takes 4 s to read and 56 s to integrate to degree 10 (4.5 GB), and a forward model with four components then 14 s. On that model the comparison with a true model (`sbtk.compare.with_truth`) takes 16 s and a 20 m density grid 14 s: the surface along a direction is found among the facets sorted by direction (`sbtk.internal.surface_distance_indexed`), not among all 3 million.

Example

MATLAB script: Coefficients, center of mass and inertia of a model

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
interior = sbtk.interior.create(sbtk.interior.sphere("core", [0 0 0], 0.170, +450));

geo = sbtk.gravity.geometry(body, body.centroid_km, 4);       % about the center of figure, to degree 4
fm  = sbtk.gravity.forward(geo, interior, body.mass_kg);
lab = sbtk.gravity.labels(4);
fprintf("C20 = %.5e, C22 = %.5e\n", fm.coef(strcmp(lab, "C20")), fm.coef(strcmp(lab, "C22")))
fprintf("center of mass - center of figure = [%.2f %.2f %.2f] m\n", 1e3*(fm.com_km - body.centroid_km))
fprintf("Izz / (M R_ref^2) = %.4f\n", fm.inertia_norm(3))
```

The core is centered at the shape-model origin, 57 m from the center of figure, so the center of mass moves toward it and the degree-1 coefficients about the center of figure are not zero.

Measurement sets

A measurement set (`obs`) holds fully normalized gravity coefficients with their covariance, moment-of-inertia measurement rows, the total mass, the reference radius and the expansion origin.

Sources

Source	Function	Notes
A simulated interior	`sbtk.data.simulate`	Exact values about the true center of mass, without uncertainties (they are NaN until set, below); `coef_about = 'origin'`.
An SBTK measurement package	`sbtk.exchange.read_data`	Coefficients with their full covariance, about the origin of the shape model file for packages about that origin, as the HLP6 packages are (`coef_about = 'origin'`), otherwise about the measured center of mass (`'center_of_mass'`); returns the body as well (Exchange packages).
A gravity file in the PDS SHADR layout	`sbtk.exchange.read_shadr`	Header row (R, GM, σGM, degree, order, normalization state, ...) then rows `l, m, C, S, sigma_C, sigma_S`; diagonal covariance. The caller gives the shape model and the point the coefficients are about.
A JPL SSD gravity file	`sbtk.io.read_jpl_gravity`	MATLAB-assignment form with `NAMES`, `VALS`, `COV`; parsed as text; full covariance; J_l converted to $\bar{C}_{l0}$.
Coefficient matrices	`sbtk.data.from_coefficients`	`C(l+1, m+1)`, `S(l+1, m+1)` and their 1σ; diagonal covariance.
The SBTK text format	`sbtk.io.read_measurements`, `sbtk.io.load_measurements`	One keyword per line; diagonal covariance only (Results folders).

Uncertainties and covariance

- **Exact values have none.** `sbtk.data.simulate` returns the true model's exact values without uncertainties: `coef_cov` and every inertia σ are NaN. Measurements need them: `sbtk.data.add_noise`, `sbtk.invert.setup` and `sbtk.exchange.write_data` stop with `sbtk:data:noUncertainties` otherwise.
- **By degree.** `sbtk.data.set_uncertainties(obs, unc)` gives every coefficient of degree l the 1σ of `sbtk.data.uncertainties` for that degree (a diagonal covariance), every inertia element its 1σ (as a fraction of MR_{ref}^2), and the libration row the 1σ of the ratio times $I_{zz} / (MR_{\mathrm{ref}}^2)$.
- **From another data set.** `sbtk.data.use_uncertainties` gives every coefficient that a reference data set also contains the reference's variances and covariances, converted to the reference radius of the target by $(R' / R)^l$, with R' the reference's radius and R the target's. If the reference is about another point, its covariance is first moved to the point of the target, $K' = TKT^{\mathsf{T}}$ (Uncertainties and the expansion point). Coefficients the reference lacks keep their variances and become uncorrelated with the others.
- **Full covariance.** `sbtk.data.set_covariance` replaces the coefficient covariance with any symmetric positive definite matrix in the order of `obs.coef_labels`.
- **Inertia rows.** `sbtk.data.set_moi_sigma` sets the σ of named rows; the field `libration` gives the 1σ of the ratio.

Noise

`sbtk.data.add_noise(obs, seed)` returns one realization: the coefficients plus $L\mathbf{z}$, with L the lower Cholesky factor of the coefficient covariance and $\mathbf{z}$ standard normal; each inertia row plus its σ times a standard normal variable. The same seed gives the same noise; `obs.noise_seed` records it.

Inertia measurements and the center of mass

Function	Adds
`sbtk.data.add_moi`	One element `Ixx` ... `Iyz` as $I / (MR_{\mathrm{ref}}^2)$ (`sbtk.units.moi_to_norm` converts from kg m²).
`sbtk.data.add_moi_row`	Any linear combination of the six elements.
`sbtk.data.add_libration`	The libration ratio $(B - A) / C$ as a linear row (Moments of inertia). Requires $\bar{C}_{22}$ and $\bar{S}_{22}$.
`sbtk.data.libration_ratio_from_amplitude`	Computes the ratio and its 1σ from a forced libration amplitude.
`sbtk.data.add_center_of_mass`	The measured center of mass as degree-1 coefficients; sets `coef_about` to `'center_of_mass'`.

Selecting measurements

`sbtk.data.select_degree` keeps the coefficients up to a degree, `sbtk.data.select_coefficients` a list of coefficients, and `sbtk.data.select_moi` a list of inertia rows; the covariance keeps the matching rows and columns. `sbtk.data.move_origin` changes the expansion origin of data about the center of mass. `sbtk.data.summary` prints the contents of a measurement set.

Example

MATLAB script: Measurements from coefficient matrices

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Dimorphos", sbtk.shape.bundled("dimorphos"), 2.700e9, 0.10);

% coefficient matrices, C(l+1, m+1) = C_lm, fully normalized, about the shape-model origin
L = 2;
C = zeros(L + 1);  S = zeros(L + 1);
C(1,1) = 1;  C(3,1) = -0.08;  C(3,3) = 0.03;  S(3,3) = 0.002;     % C00, C20, C22, S22
Cs = 1e-3*ones(L + 1);  Ss = 1e-3*ones(L + 1);                     % 1-sigma of each
obs = sbtk.data.from_coefficients(body, "example", [0 0 0], C, S, Cs, Ss);

% a polar moment given in kg m^2, and a libration ratio
Izz = sbtk.units.moi_to_norm(1.6e14, body.mass_kg, body.R_ref_km);
obs = sbtk.data.add_moi(obs, "Izz", Izz, 0.01*Izz);
obs = sbtk.data.add_libration(obs, 0.20, 0.03, Izz);

% 30% correlation between all coefficients
sig = sqrt(diag(obs.coef_cov));
obs = sbtk.data.set_covariance(obs, (0.3 + 0.7*eye(numel(sig))).*(sig*sig'));
sbtk.data.summary(obs)
```

Calculations has further examples: simulated data with uncertainties from another data set, selection of inertia rows, and reading files.

Uncertainties and the expansion point

A gravity field can be written about any point, and its coefficients depend on that point. So do their uncertainties. The Hera radio-science (RSE) team gives the uncertainties of a field about the center of mass; the HLP6 packages give coefficients about the origin of the shape model file, 56 m from the center of mass of Didymos in UCB-DATA1. This lesson starts from what a coefficient measures, derives how the coefficients and their uncertainties change when the expansion point moves, works the Didymos and Dimorphos cases by hand, and ends with the intuition and what to do.

1. What a coefficient measures

A coefficient of degree l is a moment of the mass distribution of order l, taken about the expansion point. With x, y, z measured from that point, M the mass and R the reference radius, the coefficients of degree 0 to 2 are (unnormalized):

Coefficient	Moment	Meaning
$C_{00} = 1$	$\frac{1}{M}\int \rho \, dV$	the mass
C_{10}, C_{11}, S_{11}	$\frac{1}{MR}\int \rho\, z\, dV$, $\frac{1}{MR}\int \rho\, x\, dV$, $\frac{1}{MR}\int \rho\, y\, dV$	the center of mass from the point, $\bar{z}\,/\,R$, $\bar{x}\,/\,R$, $\bar{y}\,/\,R$
C_{20}	$\frac{1}{MR^2}\int \rho \left(z^2 - \frac{1}{2}(x^2+y^2)\right) dV$	flattening along z
C_{22}	$\frac{1}{4MR^2}\int \rho\,(x^2 - y^2)\, dV$	elongation along x

The fully normalized coefficients of the packages are $\bar{C}_{lm} = C_{lm} \,/\, N_{lm}$, with $N_{10} = N_{11} = \sqrt{3}$, $N_{20} = \sqrt{5}$ and $N_{22} = \sqrt{5\,/\,12}$ (Gravity coefficients). About the center of mass, degree 1 is zero by definition; about any other point it gives where the center of mass is.

2. Moving the point: a warm-up in one dimension

Take mass along a line and its second moment about the origin, $\int \rho\, x^2\, dx$. About a point at $x = h$ instead:

$$\int \rho\,(x-h)^2\, dx = \int \rho\, x^2\, dx \;-\; 2h \int \rho\, x\, dx \;+\; h^2 \int \rho\, dx\,.$$

The second moment about the new point is the old second moment, plus the old **first** moment times h, plus the old **zeroth** moment (the mass) times h^2. This is the parallel-axis theorem. Two things follow, and they hold for every degree:

- a moment about the new point mixes in the moments of **lower** order about the old one, weighted by powers of the offset; never higher ones;
- if a lower moment is uncertain, the higher moments about the new point inherit that uncertainty, multiplied by the offset (the lever arm).

3. The rules for degrees 1 and 2

Let $h = (h_x, h_y, h_z)$ be the new expansion point minus the old one, so that the coordinates about the new point are $x - h_x$, $y - h_y$, $z - h_z$. Putting these into the moments of section 1:

Degree 1 picks up only the mass:

$$C_{10}' = C_{10} - \frac{h_z}{R}, \qquad C_{11}' = C_{11} - \frac{h_x}{R}, \qquad S_{11}' = S_{11} - \frac{h_y}{R}\,.$$

Degree 2 picks up degree 1 and the mass. Expanding $(z-h_z)^2 - \frac{1}{2}\left((x-h_x)^2 + (y-h_y)^2\right)$ and $(x-h_x)^2 - (y-h_y)^2$:

$$C_{20}' = C_{20} - 2\frac{h_z}{R}C_{10} + \frac{h_x}{R}C_{11} + \frac{h_y}{R}S_{11} + \frac{h_z^2 - \frac{1}{2}(h_x^2 + h_y^2)}{R^2},$$

$$C_{22}' = C_{22} - \frac{h_x}{2R}C_{11} + \frac{h_y}{2R}S_{11} + \frac{h_x^2 - h_y^2}{4R^2}\,.$$

In fully normalized coefficients (divide each side by its N_{lm}):

$$\bar{C}_{20}' = \bar{C}_{20} + \sqrt{\tfrac{3}{5}}\left(- 2\frac{h_z}{R}\bar{C}_{10} + \frac{h_x}{R}\bar{C}_{11} + \frac{h_y}{R}\bar{S}_{11}\right) + \text{const},$$

$$\bar{C}_{22}' = \bar{C}_{22} + \frac{3}{\sqrt{5}}\left(- \frac{h_x}{R}\bar{C}_{11} + \frac{h_y}{R}\bar{S}_{11}\right) + \text{const}\,.$$

The constants hold only h and $C_{00} = 1$. For every degree at once the rule is linear, $c' = T\,c$, with T the translation matrix of solid harmonics (`sbtk.internal.sh_translation`): degree l about the new point is a combination of degrees 0 to l about the old one, with powers of $h\,/\,R$.

4. From values to uncertainties

A linear map carries a covariance K to $K' = T\,K\,T^{\mathsf{T}}$. For one coefficient this means: the variance of a sum of independent terms is the sum of their variances, each with the square of its weight, and a constant adds nothing. The offset h is known exactly (the shape model file is agreed) and $C_{00} = 1$, so only the uncertain coefficients count. With uncorrelated inputs:

$$\sigma^2(\bar{C}'_{1m}) = \sigma^2(\bar{C}_{1m}) \qquad \text{(degree 1 does not change)}$$

$$\sigma^2(\bar{C}'_{20}) = \sigma^2(\bar{C}_{20}) + \frac{12}{5}\left(\frac{h_z}{R}\right)^2\sigma^2(\bar{C}_{10}) + \frac{3}{5}\left(\frac{h_x}{R}\right)^2\sigma^2(\bar{C}_{11}) + \frac{3}{5}\left(\frac{h_y}{R}\right)^2\sigma^2(\bar{S}_{11})$$

$$\sigma^2(\bar{C}'_{22}) = \sigma^2(\bar{C}_{22}) + \frac{9}{5}\left(\frac{h_x}{R}\right)^2\sigma^2(\bar{C}_{11}) + \frac{9}{5}\left(\frac{h_y}{R}\right)^2\sigma^2(\bar{S}_{11})$$

The 1σ of degree 1 is the uncertainty of the center of mass: $\sigma_x = \sqrt{3}\ R\,\sigma(\bar{C}_{11})$, $\sigma_y = \sqrt{3}\ R\,\sigma(\bar{S}_{11})$, $\sigma_z = \sqrt{3}\ R\,\sigma(\bar{C}_{10})$. With it the added terms read as a lever arm times a position error, over R^2; for example the z term of $\bar{C}_{20}$ is $\frac{2}{\sqrt{5}}\,h_z\,\sigma_z / R^2$.

If the inputs are correlated, each pair adds twice its covariance times the two weights; TKT^{T} includes that. The RSE `.tab` files give only one 1σ per coefficient, so the examples below treat them as uncorrelated, as `sbtk.data.use_uncertainties` does.

---

5. Example: the zonal coefficient of degree 2 of Didymos (UCB-DATA1)

The package gives degree 1 about the origin of the file; it puts the center of mass at $c = (15.03,\ 42.27,\ -33.98)$ m. The RSE field is about the center of mass, so the expansion point moves from the center of mass to the origin: $h = -c = (-15.03,\ -42.27,\ 33.98)$ m, and with $R = 410$ m, $h / R = (-0.0367,\ -0.1031,\ 0.0829)$. The RSE 1σ, converted to $R = 410$ m (factor $(430 / 410)^l$):

	$\sigma(\bar{C}_{20})$	$\sigma(\bar{C}_{10})$	$\sigma(\bar{C}_{11})$	$\sigma(\bar{S}_{11})$
RSE, R = 410 m	8.70 × 10⁻⁶	1.378 × 10⁻⁴ (σ_z = 9.8 cm)	1.240 × 10⁻⁵ (σ_x = 0.9 cm)	1.304 × 10⁻⁵ (σ_y = 0.9 cm)

The terms of $\sigma(\bar{C}'_{20})$:

Term	Value
own, $\sigma(\bar{C}_{20})$	8.70 × 10⁻⁶
z: $\sqrt{12/5}\ (0.0829)(1.378\times10^{-4})$	1.77 × 10⁻⁵
x: $\sqrt{3/5}\ (0.0367)(1.240\times10^{-5})$	3.5 × 10⁻⁷
y: $\sqrt{3/5}\ (0.1031)(1.304\times10^{-5})$	1.04 × 10⁻⁶
total, $\sqrt{\text{sum of squares}}$	**1.97 × 10⁻⁵, 2.27 times the RSE value**

The z term dominates: the 34 m height of the origin above the center of mass is the lever arm, and the center of mass is least certain along z (9.8 cm against 0.9 cm in x and y).

---

6. Example: the sectoral coefficient of degree 2 of Dimorphos (UCB-DATA4)

The center of mass is at $c = (-1.11,\ -2.26,\ -0.25)$ m from the origin of the Dimorphos file, so $h = (1.11,\ 2.26,\ 0.25)$ m and, with $R = 100$ m, $h / R = (0.0111,\ 0.0226,\ 0.0025)$.

	$\sigma(\bar{C}_{22})$	$\sigma(\bar{C}_{11})$	$\sigma(\bar{S}_{11})$
RSE	4.04 × 10⁻⁶	2.462 × 10⁻³ (σ_x = 0.43 m)	3.380 × 10⁻³ (σ_y = 0.59 m)

Term	Value
own, $\sigma(\bar{C}_{22})$	4.04 × 10⁻⁶
x: $\frac{3}{\sqrt{5}}(0.0111)(2.462\times10^{-3})$	3.67 × 10⁻⁵
y: $\frac{3}{\sqrt{5}}(0.0226)(3.380\times10^{-3})$	1.03 × 10⁻⁴
total	**1.09 × 10⁻⁴, 27 times the RSE value**

The offset is only 2.5 m, but the center of mass of Dimorphos is known to about half a meter, and $\bar{C}_{22}$ itself is known very well (4×10^{-6}). A small lever arm on a large position error swamps a small own uncertainty.

The other coefficients, worked the same way (answers in section 10): Dimorphos $\bar{C}_{20}$ grows 1.60 times, Didymos $\bar{C}_{22}$ only 1.003 times, because $\bar{C}_{22}$ couples only to x and y, where the center of mass of Didymos is known to a centimeter.

---

7. Example: degree 1 does not change

$\bar{C}'_{10} = \bar{C}_{10} - h_z / (\sqrt{3}\ R)$, and h_z is known exactly. For UCB-DATA1 the value goes from about zero (about the center of mass) to $-33.98\ \text{m} / (\sqrt{3}\cdot 410\ \text{m}) = -0.0478$, the $\bar{C}_{10}$ in the package, while the 1σ stays 1.378×10^{-4}. Degree 1 is the position of the center of mass; measuring it from a different fixed point changes the number, not how well it is known.

---

8. Example: the moments of inertia

The inertia elements move by the parallel-axis theorem, $I_{ij}(\text{origin}) = I_{ij}(\text{c.m.}) + M\left(\ |c|^2\delta_{ij} - c_i c_j\ \right)$, so $I_{xx}(\text{origin}) = I_{xx}(\text{c.m.}) + M(c_y^2 + c_z^2)$. An error in c changes it by $2M(c_y\,\delta c_y + c_z\,\delta c_z)$, and in units of MR^2 the added 1σ is

$\frac{2}{R^2}\sqrt{(c_y\sigma_y)^2 + (c_z\sigma_z)^2}$.

For Dimorphos (UCB-DATA4): $\frac{2}{(100\ \mathrm{m})^2}\sqrt{(2.26{\cdot}0.585)^2 + (0.25{\cdot}0.418)^2}\ \mathrm{m}^2 = 2.65{\times}10^{-4}$, against the RSE 1σ of I_{xx}, $1.1081{\times}10^{4}$ kg km² $/\,(MR^2) = 4.10{\times}10^{-4}$ (the file labels it kg m²; Didymos and Dimorphos values): the total is $4.89{\times}10^{-4}$, 19% more than the RSE 1σ. For Didymos the added term is $4{\times}10^{-5}$ against an RSE 1σ of $2.8{\times}10^{-2}$, which is negligible. (The inertia errors are also correlated with degree 1, which SBTK, keeping inertia and coefficients uncorrelated, cannot carry; it adds the variance only.)

---

9. The intuition

- **Coefficients describe the mass as seen from a point.** About the center of mass, the RSE measures the shape of the field very well and the position of the center of mass separately. About another point, every coefficient also encodes where the center of mass sits relative to that point, so the position error enters them.
- **The offset is a lever arm.** As in the parallel-axis theorem, the error a position uncertainty causes grows with the distance: no offset, no added uncertainty. That is why the field is best described about the center of mass.
- **It reaches every higher degree.** Degree l mixes in all lower degrees with powers of $h\,/\,R$, so the uncertainty of the center of mass reaches degree 2, 3 and above, with weights that fall as $h\,/\,R$ is small.
- **It matters most where a coefficient is known best.** A coefficient whose own 1σ is tiny (Dimorphos $\bar{C}_{22}$) is overwhelmed; one with a large own 1σ hardly changes.
- **Nothing is lost.** $K' = TKT^{\mathsf{T}}$ has correlations between degree 1 and the rest. A fit that uses the full covariance gives the same answer about the origin as about the center of mass. The mistake is to keep the RSE 1σ, which belong to the center of mass, for coefficients about the origin: that claims those coefficients are known up to 27 times better than they are, and a fit then weights them too much.

---

10. What to do, and answers

To weight data about a fixed point with uncertainties given about the center of mass, do one of these two; they give the same fit:

1. move the covariance to the point of the data, $K' = TKT^{\mathsf{T}}$: `sbtk.data.use_uncertainties` does it when the reference set is about another point, and `projects/hera_hlp6/hlp6_uncertainties.m` gives the RSE field the center of mass the package's degree 1 gives;
2. move the coefficients to that center of mass (`sbtk.data.expand_about`) and use the RSE 1σ there.

The numbers of sections 5 to 8 can be reproduced with the formulas of section 4:

MATLAB

```
R = 0.41;  h = [-15.03 -42.27 33.98]/1e3;                    % Didymos, UCB-DATA1 [km]
s20 = 8.70e-6;  s10 = 1.378e-4;  s11 = 1.240e-5;  t11 = 1.304e-5;  % RSE 1-sigma at R = 0.41 km
c20 = sqrt(s20^2 + 12/5*(h(3)/R)^2*s10^2 + 3/5*(h(1)/R)^2*s11^2 + 3/5*(h(2)/R)^2*t11^2);
fprintf('Didymos C20: %.3g, %.2f times the RSE value\n', c20, c20/s20)          % 1.97e-05, 2.27
R = 0.10;  h = [1.11 2.26 0.25]/1e3;                         % Dimorphos, UCB-DATA4 [km]
s22 = 4.04e-6;  s11 = 2.462e-3;  t11 = 3.380e-3;
c22 = sqrt(s22^2 + 9/5*(h(1)/R)^2*s11^2 + 9/5*(h(2)/R)^2*t11^2);
fprintf('Dimorphos C22: %.3g, %.1f times the RSE value\n', c22, c22/s22)        % 1.09e-04, 27.0
```

and for any package, with the full covariance, by SBTK:

MATLAB

```
[obs, body] = sbtk.exchange.read_data("UCB-DATA1-Didymos");
addpath(fullfile(sbtk.root, "projects", "hera_hlp6"));
obs = hlp6_uncertainties(obs, body);      % RSE covariance moved from the center of mass to the file origin
```

Exercises. Answers below each.

1. *Dimorphos $\bar{C}_{20}$ (UCB-DATA4).* With $\sigma(\bar{C}_{20}) = 5.09{\times}10^{-5}$, $\sigma(\bar{C}_{10}) = 2.41{\times}10^{-3}$ and the rest from section 6: terms z $9.3{\times}10^{-6}$, x $2.12{\times}10^{-5}$, y $5.92{\times}10^{-5}$; total $8.14{\times}10^{-5}$, **1.60 times** the RSE value.
2. *Didymos $\bar{C}_{22}$ (UCB-DATA1).* With $\sigma(\bar{C}_{22}) = 2.645{\times}10^{-5}$: terms x $6.1{\times}10^{-7}$, y $1.8{\times}10^{-6}$; total $2.652{\times}10^{-5}$, **1.003 times**: the center of mass of Didymos is known to 0.9 cm in x and y.
3. *How high above the center of mass must the expansion point of Didymos be for $\sigma(\bar{C}_{20})$ to double?* Doubling means the added variance is three times the own: $\tfrac{12}{5}(h_z/R)^2\sigma^2(\bar C_{10}) = 3\sigma^2(\bar C_{20})$, so $h_z = R\,\sqrt3\,\sigma(\bar C_{20})/(\sqrt{12/5}\,\sigma(\bar C_{10})) = 410\ \text{m} \times 0.0706 \approx$ **29 m**. The origin of the file is 34 m above it, hence a little more than double.
4. *Why does $\sigma(\bar{C}_{10})$ not change?* Its only new term is $h_z\,/\,(\sqrt{3}\ R)$ times $C_{00} = 1$, and both are exact (section 7).

Reading the gravity coefficients

An inversion estimates the densities of the components it is given; it does not say which components to give it. This page is the step before: how to read the gravity coefficients of an unknown body to decide which components to model, and where to start them. It derives what each kind of structure does to the coefficients, gives a procedure with the SBTK functions for each step, works an example, and reports how well the procedure finds known interiors (What the study found). The procedure applies to any body with a shape model; the examples use Didymos.

The anomaly: what the interior adds

The coefficients of a body of constant density follow from its shape alone. The density structure is in the difference between the measured coefficients and those of the **homogeneous body**, the same shape with the bulk density $\bar{\rho} = M / V$ everywhere. Write the density as $\rho(r) = \bar{\rho} + \delta\rho(r)$; since the mass is known, $\int \delta\rho \, dV = 0$. About a fixed point, the fully normalized coefficients are linear in the density, so the difference is

$$\Delta\bar{C}_{lm} = \bar{C}_{lm} - \bar{C}^{\mathrm{hom}}_{lm} = \frac{1}{MR^l} \int \delta\rho(r) \; | \, r - r_f |^l \, \bar{Y}_{lm}(r - r_f) \, dV,$$

the **anomaly** of each coefficient. SBTK expresses it about the center of figure r_f, the homogeneous body's center of mass and a fixed point of the shape, so that the anomaly of a model is the sum of the anomalies of its components. Measurements about another point (a package about the origin of its shape-model file, or a field about the measured center of mass) are moved there exactly, with their covariance (Uncertainties and the expansion point). `sbtk.interpret.anomaly` computes it, for measurements or for a model:

- `an.value` the anomaly, `an.sigma` its 1σ (the homogeneous field is exact), `an.significance` the ratio;
- `an.com_offset_km` the degree-1 part as the center of mass minus the center of figure;
- per degree, the RMS of the anomaly and of its 1σ (`an.signal_rms`, `an.sigma_rms`): a degree whose anomaly is not well above its 1σ carries no information about the interior;
- the inertia rows of the measurements with the homogeneous body's values about the same point.

`sbtk.plot.significance` draws every measurement with its ±1σ interval against the constant-density value, colored by whether it differs from it by less than 1σ, 1 to 3σ or more than 3σ. `sbtk.interpret.summary` prints the rest of this page's quantities.

Signatures of components

A relative density $\Delta\rho_c$ in a component of volume V_c lowers the density of the rest of the body by $\Delta\rho_c V_c / V$, because the mass is fixed. The anomaly it makes is $\Delta\rho_c$ times the component's **signature**

$$s_c = \frac{1}{M}\left(U_c - \frac{V_c}{V} U_{\mathrm{body}} \right),$$

with U the integrals $\int (r / R)^l \bar{Y}_{lm} \, dV$ of the component and of the body about the center of figure (`sbtk.gravity.component_integrals`). The anomaly of any model is $\sum_c \Delta\rho_c \, s_c$ (checked to 10^{-12} in `testSignaturesAddUp`). `sbtk.interpret.signatures` computes the signatures of candidate components, and fits each one alone to the measured anomaly: its density, the reduction of χ^2 from the homogeneous body, and the correlation of its signature with the anomaly.

What each kind of structure does

The homogeneous body's own coefficients set the scale. Each degree $l \geq 2$ of a field can be split into a scaled homogeneous field and the rest:

$$\bar{C}_{l\cdot} = k_l \, \bar{C}^{\mathrm{hom}}_{l\cdot} + \mathrm{rest}_l, \qquad k_l = \frac{h_l^{\mathsf{T}} K_l^{-1} c_l}{h_l^{\mathsf{T}} K_l^{-1} h_l},$$

with h_l and c_l the $2l + 1$ coefficients of degree l of the homogeneous body and of the field, and K_l the covariance of c_l: k_l is the scale of the homogeneous field fitted to degree l by weighted least squares (`an.ratio`, `an.residual`). For a model, which has no uncertainties, K_l is the identity. The χ^2 of the rest with K_l has $2l$ degrees of freedom (`an.residual_chi2`). Structure that is symmetric about the center of figure changes only k_l; everything else shows in the rest.

Structure	Degree 1	Scaling k_l	The rest
Dense core at the center of figure, mass excess qM	0	$1 - q$ at every degree	none
The same core moved by d	$q\,d\,/\,(\sqrt{3}\,R)$	$1 - q$, to order $(d / R)^2$	$q\,p_l(d)$, of order $q(d / R)^l$
Light core ($q < 0$)	$q\,d\,/\,(\sqrt{3}\,R)$	$1 + \lvert q \rvert$	as above
Surface layer, thickness t, relative density $\Delta\rho$	0	$1 + \frac{\Delta\rho}{\bar{\rho}}(s^3 - s^{l+3}) \approx 1 + \frac{\Delta\rho}{\bar{\rho}}\frac{lt}{r}$	small
Ring or torus about the z-axis, radius a	0	changes the even degrees	$J_2, J_4, \ldots$ only
The same, raised by h	$C_{10} = q_t h\,/\,(\sqrt{3}\,R)$	as above	adds $J_3 \approx 0.98\,(a / R)^2\,\bar{C}_{10}$
Elongated core (ellipsoid)	as a core	as a core	$\bar{C}_{22}, \bar{S}_{22}$: its elongation and direction
Boulder of mass $q_b M$ at d_b	$q_b\,d_b\,/\,(\sqrt{3}\,R)$	small	every order, falling as $(d_b / R)^l$

A compact mass. A sphere inside the body is a point mass: its integrals are $V_c\,p(d)$, with $p_l(d)$ the coefficients of a unit point mass at d from the center of figure (`sbtk.internal.point_mass_sh`). With $q = \Delta\rho_c V_c / M$, the field is exactly

$$c_l = (1 - q)\,h_l + q\,p_l(d) \quad \text{for every } l \geq 1\,.$$

At degree 1, $h_1 = 0$ and $p_1(d) = d\,/\,(\sqrt{3}\,R)$ (as C_{11}, S_{11}, C_{10}): the center of mass moves by $q\,d$. At degree l the core's own term is of order $q(d\,/\,R)^l$; for $d \to 0$ it vanishes and every degree is the homogeneous one scaled by $1 - q$. A dense core makes every coefficient smaller in the same proportion; a light core makes them larger.

A surface layer. SBTK's layers are defined along rays from the center, so a layer from depth 0 to t is the body minus the body scaled by $s = 1 - t\,/\,r$ along each ray (r the distance to the surface). In the cone of each facet the integrals of degree l scale with s^{l+3}; for a layer thin compared with the radius of a nearly round body, s is about the same on every ray, the layer's signature is $\frac{1}{M}(s^3 - s^{l+3})\,U_{\text{body}}$, and

$$k_l = 1 + \frac{\Delta\rho}{\bar{\rho}}\left(s^3 - s^{l+3}\right) \approx 1 + \frac{\Delta\rho}{\bar{\rho}}\,\frac{l\,t}{\bar{r}}.$$

A light layer ($\Delta\rho < 0$) scales the coefficients down like a dense core, but more at higher degree: a ratio k_l that falls with degree is a surface layer, a constant one a central mass.

A ring or torus. A thin ring of mass $q_t M$ and radius a about the z-axis through the center of figure, at height h, has only zonal terms, $\bar{C}_{l0} = q_t\,(r\,/\,R)^l P_l(\sin\varphi)\,/\,\sqrt{2l+1}$ with $r^2 = a^2 + h^2$, $\sin\varphi = h\,/\,r$. In the equatorial plane only the even degrees remain: $P_2(0) = -\frac{1}{2}$, $P_4(0) = \frac{3}{8}$, so a dense ring raises J_2 and lowers J_4, a light one does the opposite, and the sectoral and tesseral coefficients are untouched. Raised by h, the ring adds $\bar{C}_{10} = q_t h\,/\,(\sqrt{3}\,R)$ and $\bar{C}_{30} \approx -\frac{3}{2} q_t\,h\,a^2\,/\,(\sqrt{7}\,R^3)$, so $J_3 \approx \frac{3\sqrt{3}}{2\sqrt{7}}(a\,/\,R)^2\,\bar{C}_{10}$. A core moved by the same amount along z makes the same $\bar{C}_{10}$ with a J_3 of order $(d\,/\,R)^3$: J_3 **beside** C_{10} **tells a raised torus from a raised core.**

An elongated core has a degree-2 field of its own, $\bar{C}_{22}$ and $\bar{S}_{22}$ in proportion to the difference of its two equatorial axes; the angle $\frac{1}{2}\,atan2(S_{22}, C_{22})$ of the rest of degree 2 is the direction of its long axis (or short axis, if it is light).

A boulder far from the center makes degree 1 and a rest in every degree and order, falling as $(d_b\,/\,R)^l$; its direction is the direction of the degree-1 anomaly when nothing else moves the center of mass.

Degree 1 and the core

Degree 1 is the first moment of the density anomaly:

$$M\,(r_{\text{cm}} - r_f) = \sum_c \Delta\rho_c\,V_c\,(\bar{r}_c - r_f),$$

with $\bar{r}_c$ the centroid of component c. It fixes the product of a core's mass excess and its offset, not either alone: **a dense core at** d **and a light core at** $-d$ **give the same degree 1**, and so does a lighter core farther out. The mass excess must come from elsewhere, and degree 2 gives it if the core is the only central mass. `sbtk.interpret.central_mass` solves the two equations of a single compact mass,

$$q\,d = r_{\text{cm}} - r_f, \qquad c_2 - h_2 = q\,(p_2(d) - h_2),$$

by iteration from $q = 1 - k_2$ and $d = (r_{\text{cm}} - r_f)\,/\,q$, with their 1σ from the full covariance. For a sphere alone the answer is exact (`testCentralMassOfOneSphere`). With other structure, q is the net central condensation of all of it: a light surface layer adds to it, a light torus too, and d then comes out too short. That is why a model fit does not end the question of the core's position:

- `sbtk.invert.scan_position` fits the densities with the core placed at each of a list of points, here along the measured center-of-mass offset, and returns the χ^2 profile. It shows both sides of the ambiguity (a dense core on one side, a light one on the other) and how flat the valley between core mass and offset is;
- Gauss–Newton on the core position (`component.estimate_position`), started from the lowest point of the profile, refines it with every coefficient at once;
- `sbtk.invert.recenter` instead moves the core so that the model's center of mass equals the measured one at the fitted densities, $s = M(r_{\text{cm}} - r_{\text{cm}}^{\text{model}})\,/\,(\Delta\rho_c V_c)$, and repeated with new densities it is a fixed-point iteration. It converges in a few steps when the core is the only component that can move the center of mass; when other components can match degree 1 themselves, it stops where it starts (What the study found).

Procedure

1. **Read.** `an = sbtk.interpret.anomaly(obs, body, L)` with every coefficient to degree L and the uncertainties you will use; `sbtk.interpret.summary`, `sbtk.plot.significance`. Note which degrees carry signal, whether the center of mass is off the center of figure, and the inertia rows.
2. **Scaling and rest.** `sbtk.plot.degree_ratio`: k_l below 1 at every degree is central mass, falling with degree a light surface, above 1 a light center or a dense surface. The rest, coefficient by coefficient (`an.residual_significance`): only even zonal terms point to a ring or torus about z; J_3 with C_{10} to a raised torus; C_{22}, S_{22} to an elongated center; every order in degrees 2–4 to something off center. **The geometry** (`sbtk.interpret.geometry`, `sbtk.plot.anomaly_map`): the center-of-mass offset as a direction; the principal axes of the second moment of degree 2 (C_{20}, C_{21}, S_{21}, C_{22}, S_{22}), the axis mass was moved toward and the one it was moved away from, with their 1σ; and each degree as a map on the sphere with its directions of strongest excess and deficit. A compact mass gives one direction at every degree, a ring or belt the even degrees with extremes along the equator and at the poles. This is the reading that places components: a core along the offset, a torus about the axis degree 2 moved mass away from, a boulder where degrees 2–4 agree.
3. **One compact mass.** `sbtk.interpret.central_mass`: the size and offset of the central mass if it were alone.
4. **Candidates.** `sbtk.interpret.signatures` for the component types step 2 suggests, at several sizes and positions (`sbtk.interior.sphere_grid` for positions anywhere); `sbtk.interpret.select` adds them one at a time by the largest reduction of χ^2, with an a priori 1σ of each density. It is a guide: similar signatures (a core and a surface layer) can share the anomaly between them.
5. **Model and core position.** Set up the model (`sbtk.invert.setup`); `sbtk.invert.scan_position` along the center-of-mass offset; Gauss–Newton from the lowest point, with the geometry the rest points to free (torus height, ellipsoid axes).

6. **Test.** The χ^2 test (Goodness of fit); the anomaly of the fit against the measured one (`sbtk.plot.anomaly`); and a fit to the low degrees only, predicting the others (`sbtk.invert.predict`, `sbtk.plot.coefficient_residuals`; Fitting part of the coefficients).

Example

`examples/ex10_reading_coefficients.m` runs the procedure on a hidden Didymos interior: a dense core (+500 kg/m³, radius 140 m) 30 m from the center of figure, a light torus (−400 kg/m³) 12 m above the equator and a light 15 m surface layer (−600 kg/m³), with coefficients to degree 5 about the origin of the shape-model file and 1σ close to the Hera radio-science values.

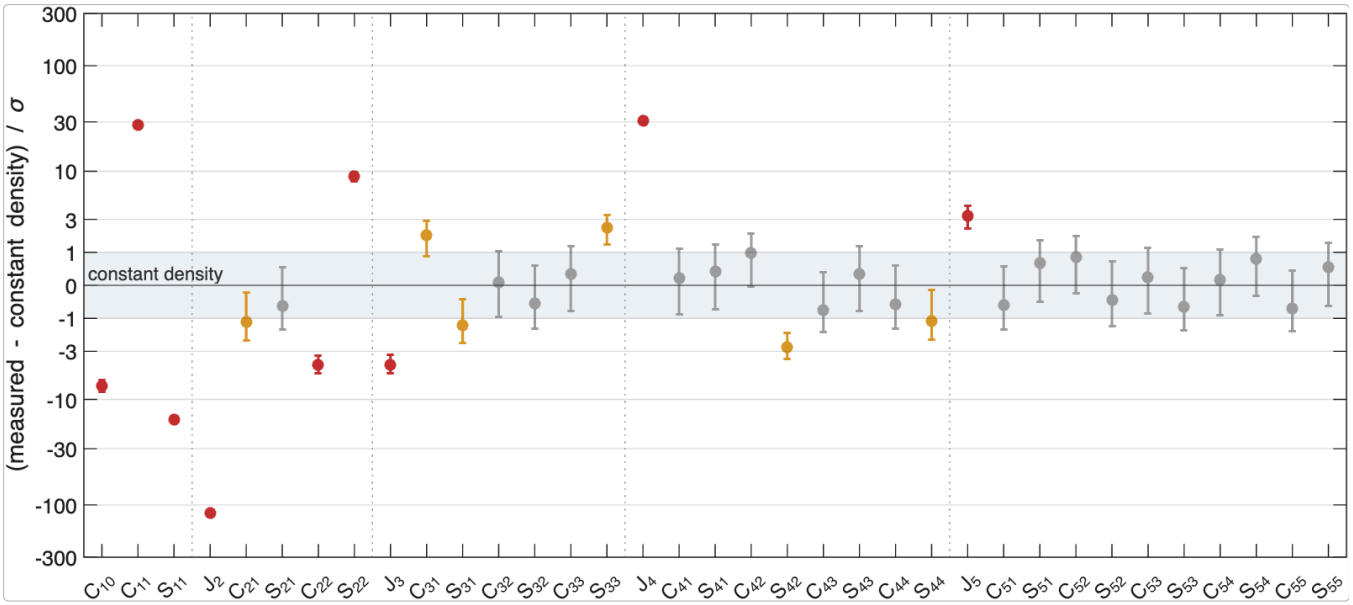
MATLAB script: Reading the coefficients of a hidden interior

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.create("Didymos", sbtk.shape.bundled("didymos"), 2600, 0.41);
cof  = body.centroid_km;
torus = sbtk.interior.torus("torus", 0.36, 0.05, 0, -400);
torus.center_km = cof + [0 0 0.012];
hidden = [sbtk.interior.sphere("core", cof + [0.025 -0.015 0.008], 0.14, +500), torus, ...
          sbtk.interior.layer("regolith", 0, 0.015, -600)];
exact = sbtk.data.simulate(body, sbtk.interior.create(hidden), 5, "reading example");
exact = sbtk.data.expand_about(sbtk.data.select_moi(exact, {}), [0 0 0]);
obs = sbtk.data.set_uncertainties(exact, sbtk.data.uncertainties([1.3e-5 2e-5 3e-5 3e-5 3e-5], 1, 0.02));

an = sbtk.interpret.anomaly(obs, body, 5);              % 1. read
sbtk.interpret.summary(an);
cm = sbtk.interpret.central_mass(an);                   % 3. one compact mass

model = [sbtk.interior.sphere("core", cof, 0.15, 0), sbtk.interior.torus("torus", 0.37, 0.055, 0, 0), ...
         sbtk.interior.layer("regolith", 0, 0.012, 0)];
model(2).center_km = cof;
opts = sbtk.invert.options(moi_labels={});
prob = sbtk.invert.setup(body, model, obs, opts);
u  = an.com_offset_km/norm(an.com_offset_km);           % 5. the core along the center-of-mass offset
sc = sbtk.invert.scan_position(prob, "core", cof + (-0.10:0.005:0.10)'*u);
model(1).center_km = sc.centers_km(sc.best, :);
model(1).estimate_position = true;
model(2).estimate_position = [false false true];
fit = sbtk.invert.solve(body, model, obs, opts);
sbtk.invert.summary(fit)
```

Output: sbtk.interpret.summary(an)

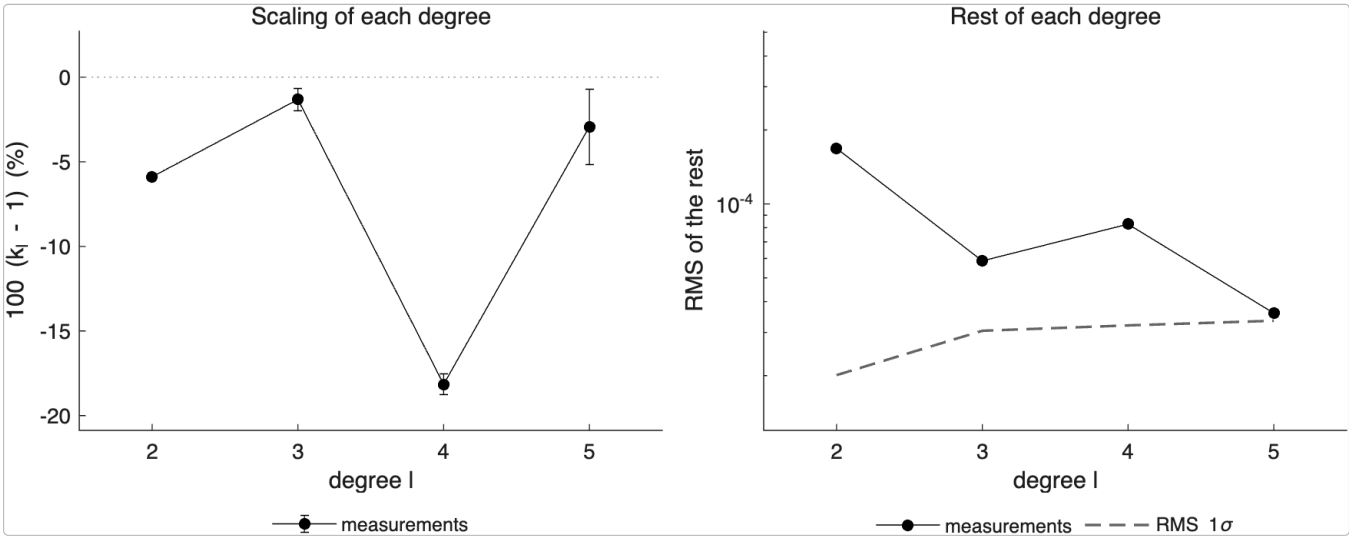
```
Didymos (reading example): anomaly about the center of figure, degrees 1 to 5
  center of mass − center of figure: [0.26 −0.15 −0.07] m +− [0.01 0.01 0.01] m, |offset| 0.31 m
  chi-square of the offset: 1096 for 3 degrees of freedom
  degree  RMS anomaly  RMS 1-sigma     ratio k_l +− 1-sigma      residual chi2 / dof
  1         2.485e-04    1.300e-05
  2         1.079e-03    2.010e-05       0.94115 +− 0.00050               348.8 / 4
  3         6.282e-05    3.045e-05       0.98683 +− 0.00670                25.95 / 6
  4         3.283e-04    3.202e-05       0.81849 +− 0.00607                58.64 / 8
  5         3.766e-05    3.345e-05       0.97082 +− 0.02231               12.95 / 10
  one compact mass with the same degrees 1 and 2: 5.893% +− 0.050% of the mass (3.12e+10 kg) at [4.4 −2.5 −1.1] m +− [0.2 0.2 0.2] m from the center of figure
```

1. Read. 15 of the 35 coefficients differ from constant density by more than 1σ and 9 by more than 3σ. The center of mass is 0.31 m from the center of figure, measured to 1 cm: something is off center. J_2 is 120σ below the homogeneous value and J_4 30σ above it.



`sbtk.plot.significance`: each coefficient's anomaly with its ±1σ interval against the constant-density value (the line at 0), on a symmetric logarithmic axis. Gray: within 1σ; amber: 1 to 3σ; red: beyond 3σ.

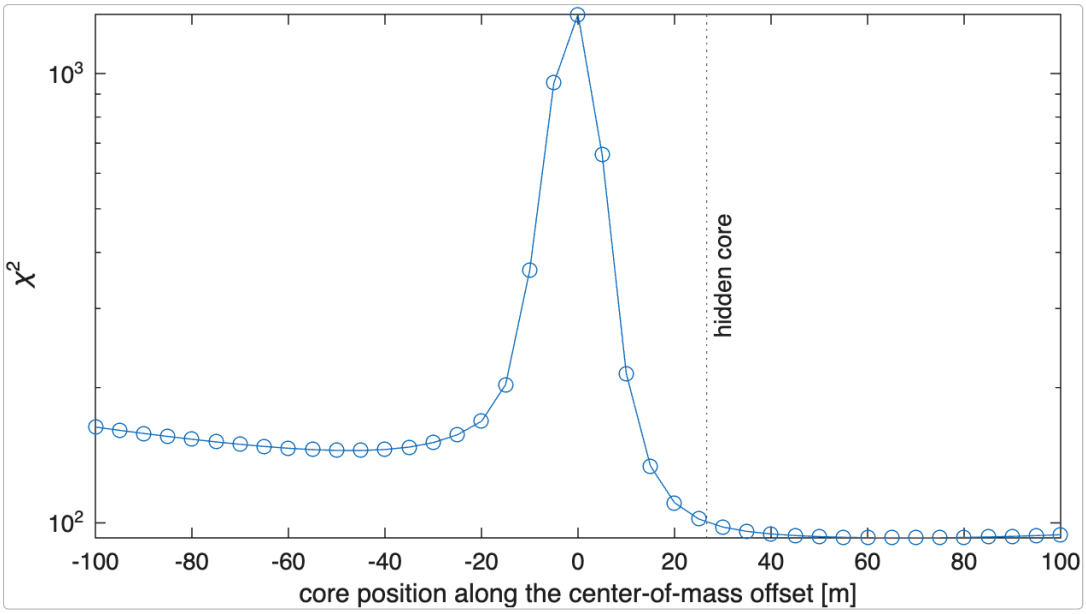
2. Scaling and rest. The homogeneous field is scaled down by 5.9% in degree 2, 18% in degree 4 and about 1–3% in degrees 3 and 5: the even degrees much more than the odd ones. A central mass alone would scale every degree alike and a surface layer would scale degree 4 more than degree 2 but degree 3 in between; the even–odd alternation is the signature of a ring about the z-axis. The rest confirms it: χ^2 of 349 for the 4 degrees of freedom of degree 2 and 59 for the 8 of degree 4, against 26 and 13 for degrees 3 and 5, and in degree 2 most of it is J_2. J_3 (−4σ) and J_5 (+3σ) point to a ring that is not in the equatorial plane.



`sbtk.plot.degree_ratio`: left, $100\,(k_l - 1)$, the change of each degree as a scaled homogeneous field, in % of the homogeneous field; right, the RMS of the rest of each degree and the RMS 1σ of the measurements (dashed).

3. One compact mass. Degrees 1 and 2 as one mass give 5.9% of the mass, 5.2 m from the center of figure. The hidden core is 1.1% of the mass 30 m from it: the 5.9% is the net central condensation of the core, the light torus and the light surface layer together, and the offset divided by it comes out about 6 times too short (5 m instead of 30 m). On its own, a compact mass is the wrong reading here, as step 2 already said.

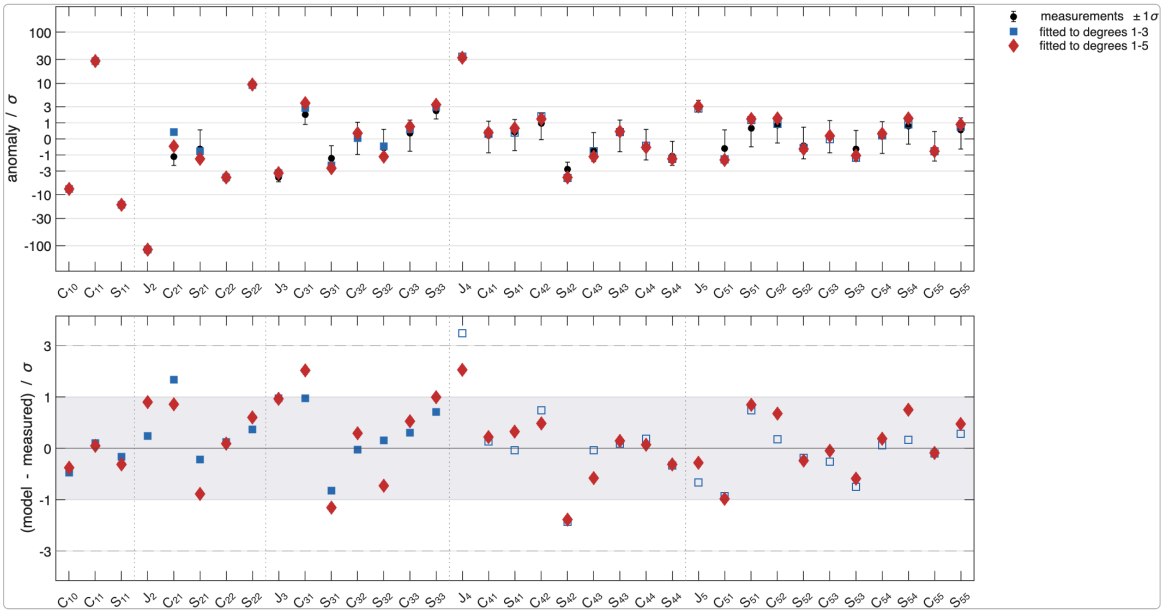
5. Model and core position. The model has the three components step 2 points to, with the sizes a receiving team would guess (core radius 150 m, torus radii 370 and 55 m about the z-axis, a 12 m layer). The χ^2 profile of the core along the center-of-mass offset has two valleys: a light core 45 m on the negative side ($\chi^2 \approx 145$) and a dense core on the positive side, flat from 40 to 100 m ($\chi^2 \approx 93$). The hidden core is at 27 m along this line.



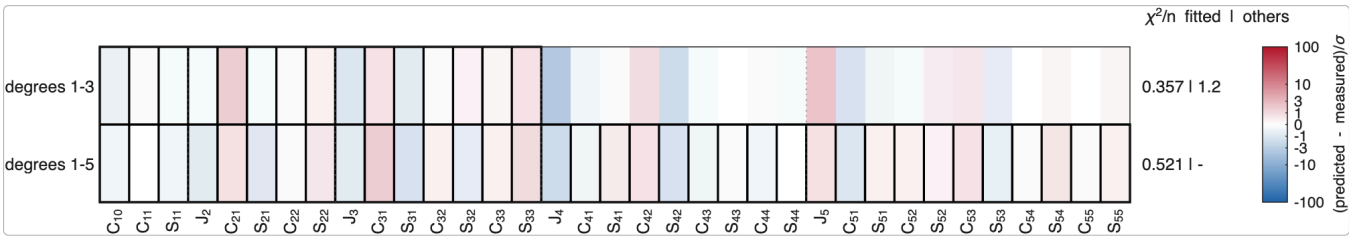
`sbtk.invert.scan_position`: χ^2 of the densities fitted with the core at each point along the measured center-of-mass offset. The dotted line is the hidden core.

Gauss–Newton from the lowest point, with the torus height free, ends at a reduced χ^2 of 0.65 ($p = 0.92$): the model is consistent with the measurements. It puts the torus 12.2 m above the equator (hidden: 12 m, ±1.9 m) with a relative density of −359 ± 15 kg/m³ (hidden: −400). The core, however, has +122 ± 22 kg/m³ at 127 m from the center of figure, and the layer −1418 ± 184 kg/m³: a weak core far out and a very light surface explain the measurements as well as the hidden dense core and lighter surface, within the 1σ of the measurements and with this model's slightly wrong sizes. With the hidden sizes the fit recovers every density exactly ($\chi^2 \approx 10^{-9}$). Degree 1 fixes the core's first moment; how it splits into mass and offset depends on the degree-2 scaling, which the layer shares. The position 1σ of Gauss–Newton (18 m) describes the valley near the solution, not the length of the valley: read the χ^2 profile.

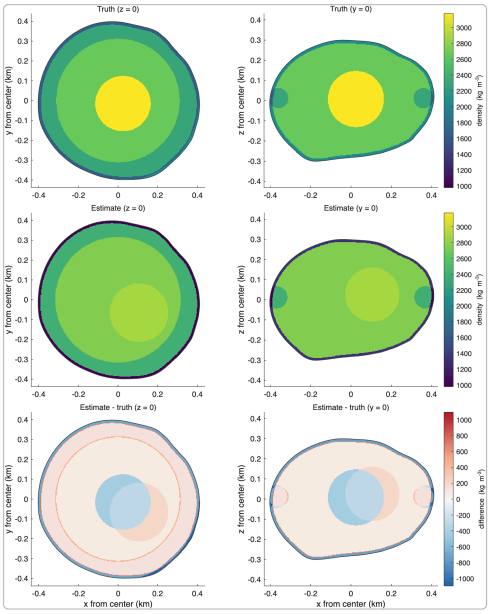
6. Test. Fitted to degrees 1–3 only, the same model predicts the 20 coefficients of degrees 4 and 5 with a χ^2 of 1.2 per coefficient: the model passes a test on coefficients it was not fitted to. The test does not separate the core from the layer either: at this precision degrees 4 and 5 do not tell the two explanations apart.



`sbtk.plot.anomaly`: the measured anomaly (black, ±1σ) and those of the model fitted to degrees 1–3 (squares) and to degrees 1–5 (diamonds); filled: fitted, open: predicted. Below, (model − measured)/σ.



`sbtk.plot.coefficient_residuals`: one row per fit, one column per coefficient; outlined: the coefficients the fit was fitted to. Right: χ^2 per coefficient of the fitted coefficients and of the others.



The hidden interior (top), the fit to degrees 1–5 (middle) and their difference (bottom), on two cuts through the center of mass.

What the study found

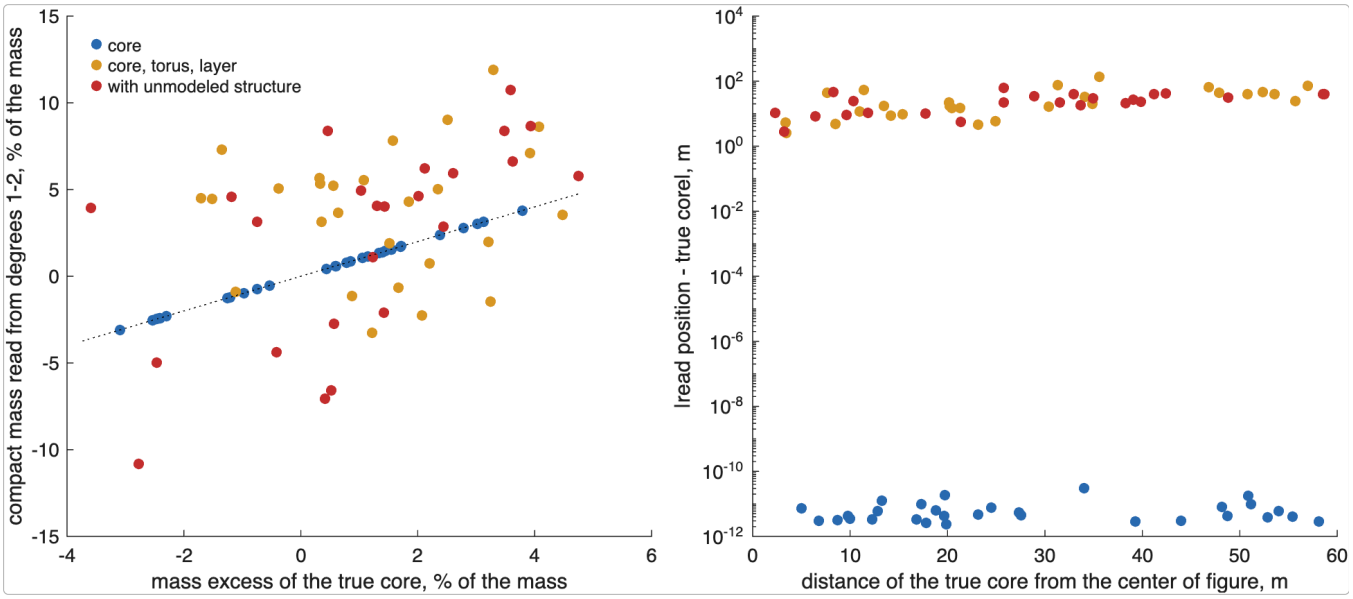
`tests/studies/coefficient_reading_study.m` applies the procedure to 110 random Didymos interiors whose answers are known, and `tests/studies/coefficient_reading_figures.m` summarizes it. Each true interior gives the exact coefficients to degree 5 about the origin of the shape-model file, weighted by the Hera radio-science 1σ (`hlp6_uncertainties.m`), and the same with one realization of noise. The interiors come in four families:

Family	True models	Contents (random)
no core	20	a torus about the z-axis (ring radius 300–400 m, tube 35–65 m, ±150 to ±700 kg/m³, up to 25 m above or below the equator) and a light surface layer (8–30 m, −200 to −1100 kg/m³)
core	30	one sphere (radius 100–200 m, +200 to +900 kg/m³, or −200 to −700 in one case of four) up to 60 m from the center of figure
core, torus, layer	30	the three together
with unmodeled structure	30	the three, the torus tilted up to 4°, 1 to 3 dense boulders (25–50 m) and a Gaussian random field

Each data set is fitted with 8 sets of coefficients, from degree 1 and J_2 to every coefficient to degree 5, with two models of fixed sizes, the sizes a team would have to guess: a core of radius 150 m, and that core with a torus (radii 370 and 55 m) and a 12 m layer. The core is placed by up to five procedures: at the center of figure; Gauss–Newton from there; the χ^2 profile along the center-of-mass offset, then Gauss–Newton (also with the torus height free); the fixed-point iteration of `sbtk.invert.recenter`. That is 15,840 fits; on the 49,152-facet model a fit takes 2 s (core fixed) to 8 s (profile and Gauss–Newton), 50 s with the torus height free, and the study 3 hours on 10 workers.

Reading without a fit

- **A core alone** scales every degree by the same factor ($k_l = 0.992$, the median $1 - q$), leaves no rest, and `sbtk.interpret.central_mass` returns its mass excess and position exactly in all 30 models.
- **A torus and a layer** scale degree 4 down most (median $k_4 = 0.89$, against $k_2 = 0.96$ and $k_3 = 0.96$) and leave a rest in degrees 2 and 4 (median χ^2 108 and 154) much larger than in degrees 3 and 5 (9 and 6): the signature of the even degrees. The compact mass from degrees 1 and 2 is then the net central condensation: it converges in 90% of the models, its mass is off by a factor (median 1.25 of the core's, with 10% of the models below −2.9 and 10% above 9.3) and its position 20 m (median) from the core.
- **Boulders and a random field** add a rest in the odd degrees: median χ^2 84 in degree 3 and 24 in degree 5, against 9 and 6 without them. A rest in the odd degrees is the sign that a core, a torus and a layer will not be enough.



The compact mass read from degrees 1 and 2 (`sbtk.interpret.central_mass`) against the true core: left its mass excess, right the distance of its position from the true core. A core alone is read exactly (on the diagonal; positions within 10^{-10} m); with a torus and a layer, or more, the reading is off by tens of meters and by a factor in mass.

Degree 1 and the core

- **Degree 1 and J_2 determine a core that is alone.** The four numbers fix its four unknowns: without noise the core is recovered exactly; at the Hera precision it is 4.6 m from the true core (median), within 10 m in 83% of the models, with its mass excess within 2%. More coefficients do not move it closer (4.1–4.6 m with every set to degree 5): the position is limited by how well degree 1 and J_2 are measured relative to the core's mass. Gauss–Newton, the profile and the fixed-point iteration give the same result.
- **Matching degree 1 by moving the core is an iteration that converges when the core is the only component that moves the center of mass.** With the core model, `sbtk.invert.recenter` converges in all 240 fits with noise, in 3 steps (median). With the three-component model, fitted to degree 1 and J_2, it stops at its first step with the core at the center of figure: the torus and the layer, cut by the irregular surface, have their own degree 1, and their fitted densities already match the measured one. With more coefficients it converges in 53–77% of the fits, after 6–32 steps, and ends 15–30 m from the core.
- **Gauss–Newton from the center of figure** with the three-component model finds a lone core within 10 m in 17% of the models with degree 1 and J_2 (its mass excess wrong by about 300%); from the lowest point of the χ^2 profile, in 83%.
- **The χ^2 profile chooses the side.** Its lowest point is on the side of the true core in every model of a lone core without noise and in 93% with noise; with a torus and a layer in 79%, with unmodeled structure in 80%. Most profiles (64–81%) have two valleys, a dense core on one side and a light one on the other.

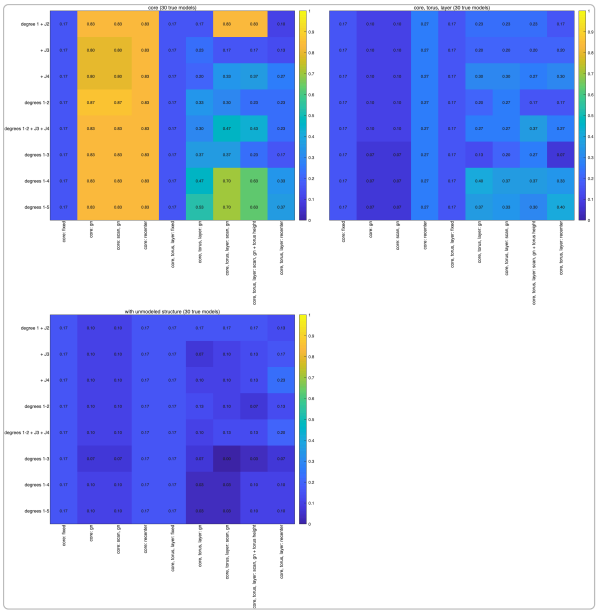
Which coefficients

With the right model (a lone core fitted with the core model) every set predicts the coefficients it was not fitted to with χ^2 of 1.0 per coefficient, as the noise: degree 1 and J_2 are enough. With a torus and a layer, the three-component model of guessed sizes, placed by the profile and Gauss–Newton, with noise:

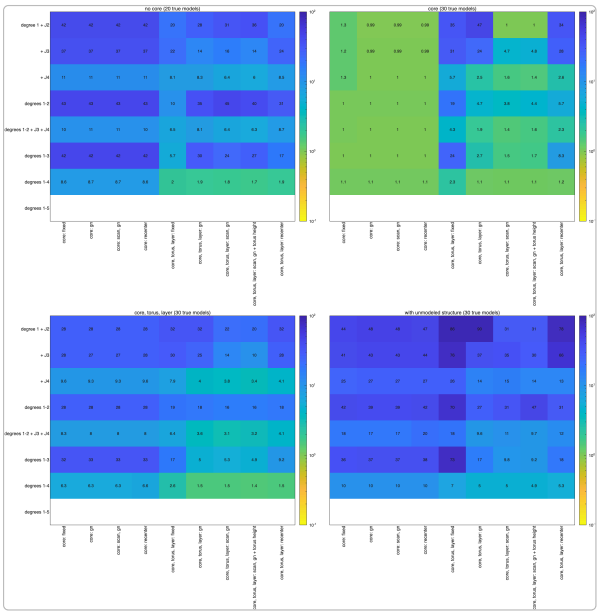
Coefficients fitted	Number	Core within 10 m	Core error (median)	Mass excess error (median)	Consistent	χ^2 per coefficient of the others
degree 1 + J_2	4	23%	22 m	151%	(underdetermined)	21.5
+ J_3	5	20%	47 m	134%	(underdetermined)	13.5
+ J_4	6	30%	22 m	53%	(no degree of freedom)	3.8
degrees 1–2	8	27%	22 m	77%	63%	16.3
degrees 1–2 + J_3 + J_4	10	27%	14 m	47%	40%	3.1
degrees 1–3	15	20%	20 m	88%	47%	5.3
degrees 1–4	24	37%	14 m	56%	37%	1.5
degrees 1–5	35	33%	15 m	46%	33%	–

Without noise the same model places the core within 10 m in 53–60% of the models from degrees 1–4 or 1–5 (median 8–10 m).

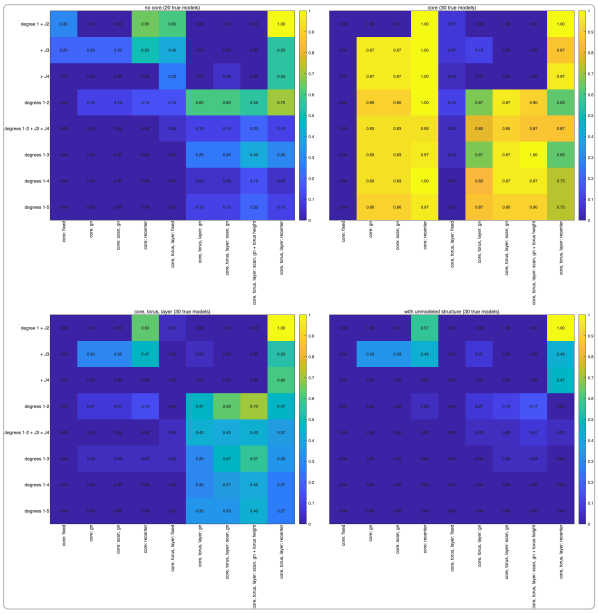
- **J_4 is the coefficient that separates a torus and a layer from a core.** Added to degree 1 and J_2, it lowers the χ^2 of the other coefficients from 21.5 to 3.8 per coefficient and the error of the core's mass excess from 151% to 53%. J_3 adds little: at this precision it is mostly noise, and fitting it moves the core to fit the noise (47 m).
- **A fit to degrees 1–2 can be consistent and wrong.** Degree 2 has five numbers; 63% of these fits pass the χ^2 test and still predict the other coefficients at 16 per coefficient. For interiors without any core the same model passes on degrees 1–2 in 60% of the models and predicts the rest at 45. Test a model on coefficients it was not fitted to (Fitting part of the coefficients).
- **The low degrees predict the high ones once J_4 is in**: fitted to degrees 1–4, the model predicts degree 5 at 1.5 per coefficient.



Fraction of fits whose core is within 10 m of the true core, with noise, for each set of coefficients (rows) and each model and procedure (columns), for the families with a core.



Median χ^2 per coefficient of the coefficients not fitted, with noise. Below about 1.5 the fitted model predicts the rest as well as the noise allows.



Fraction of fits consistent with the coefficients they were fitted to ($p \geq 0.05$). Fits with no more coefficients than unknowns have no degrees of freedom and are not counted as consistent; the fixed-point iteration moves the core outside the fit, so its fits do not count the three coordinates as unknowns and pass more often.

What limits the finding

- **The sizes of the components.** The models' sizes are fixed and the true ones random. With the true sizes the fit recovers every density exactly (the Example); with guessed sizes the densities and the core position absorb the difference, and the core is found within 10 m in about a third of the models with noise and the mass excess to 50%. Search the sizes as well as the densities (Searching the geometry), as in ROB-DATA3.
- **Structure the model does not have.** With boulders and a random field the three-component model places the core within 10 m in at most a fifth of the fits, and is consistent with the coefficients in at most 7% of the fits to degrees 1–2 + J_3 + J_4 or 1–3 and in none of the fits to degrees 1–4 or 1–5. The rest in the odd degrees says so before any fit.
- **Noise** costs about a factor two in the core position with three components (8–10 m without noise, 14–15 m with).

In practice

1. If every degree is scaled alike and there is no rest, the interior is a core: fit a core to degree 1 and J_2, and test it on the rest.
2. If degree 4 is scaled down more than degree 2 and the rest is in the even degrees, add a torus and a layer, and fit at least degree 1, J_2 and J_4; degrees 1–4 when the low degrees are not enough.
3. If there is a rest in the odd degrees, three components are not enough; do not trust a core position from them.
4. Place the core by its χ^2 profile, not by Gauss–Newton from the center of figure; the fixed-point iteration only for a core that is alone.
5. Vary the sizes of the components, and add compact masses only where the measurements ask for them (Searching the geometry).
6. Accept a model only if it predicts coefficients it was not fitted to.

Density estimation

The inversion estimates the background density and the relative densities of a set of components from a measurement set, and optionally parts of their geometry: the position of spheres, ellipsoids and tori and the axis of ellipsoids and tori (Estimated geometry).

Procedure

`sbtk.invert.setup(body, components, obs, opts)` builds the geometry about the measured center of mass (`fit.origin_km`; below) and the design matrix once (`prob`); `sbtk.invert.run(prob)` solves. `sbtk.invert.solve` does both. Reusing `prob` is faster when the same problem is solved more than once:

- with other settings that do not select measurements (`prob.opts.regularization`, bounds, positions);
- for a homogeneous body (`sbtk.invert.homogeneous`);
- for other realizations of the same measurements (`sbtk.invert.replace_data`).

Estimation problem

Let $\mathbf{y}$ be the n measurements (the selected coefficients followed by the inertia rows named in `opts.moi_labels`) with covariance $\Sigma = LL^{\mathsf{T}}$. The predictions are linear in the densities:

$$\mathbf{y}_{\mathrm{pred}} = \rho_{\mathrm{bg}}\,\mathbf{g}_{\mathrm{body}} + \sum_{c=1}^{n_c} \Delta\rho_c\,\mathbf{g}_c,$$

where $\mathbf{g}_{\mathrm{body}}$ and $\mathbf{g}_c$ are the measurements of a unit density in the whole body and in component c (`prob.gbody`, `prob.G`). The unknowns $\mathbf{x} = (\rho_{\mathrm{bg}}, \Delta\rho_1, \ldots, \Delta\rho_{n_c})$ minimize

$$\lVert L^{-1}(\mathbf{y}_{\mathrm{pred}} - \mathbf{y}) \rVert^2 + \lambda \sum_c \left(\frac{\Delta\rho_c}{\bar{\rho}}\right)^2$$

subject to

- the **mass constraint** $\rho_{\mathrm{bg}} V + \sum_c \Delta\rho_c V_c = M$, with V_c the volume of component c inside the body and M = `obs.mass_kg`;
- the **bounds** $0 \le \rho_{\mathrm{bg}} \le \rho_{\max}$ and $0 \le \rho_{\mathrm{bg}} + \Delta\rho_c \le \rho_{\max}$ for every component, with $\rho_{\max}$ = `opts.rho_max_kgm3`;
- **fixed parameters**: with `opts.background = "bulk"` the background is held at the bulk density $\bar{\rho} = M / V$; components with `component.estimate_density` false keep their relative density.

λ is the Tikhonov parameter `opts.regularization`. The problem is solved with `lsqlin`, first with the active-set algorithm and, if that does not converge, with the interior-point algorithm; if neither converges the run stops with `sbtk:invert:solver`.

Estimated geometry

Besides the densities, an inversion can estimate the geometry of components. The geometry parameters are listed by `sbtk.internal.geometry_parameters`:

Field of the component	Components	Parameters
`component.estimate_position = true`	sphere, ellipsoid, torus	the three center coordinates, km (`<name>_x`, `_y`, `_z`)
`component.estimate_position = [false false true]`	sphere, ellipsoid, torus	only the coordinates of the mask, for example the height of a torus
`component.estimate_axis = true`	torus	the direction of its axis: two tilts about its own first and second axes, deg (`<name>_tilt1`, `_tilt2`)
`component.estimate_axis = true`	ellipsoid	its orientation: the two tilts of its third axis and the turn about it, deg (`<name>_tilt1`, `_tilt2`, `_turn`), without the rotation about an axis of symmetry

A rotation about an axis of symmetry leaves the component as it is, so no measurement can determine it, and it is not a parameter. An ellipsoid with two equal semi-axes has two rotations, whichever of its axes is the axis of symmetry, and one with three equal semi-axes has none.

Setting either field on a component without a center or an axis stops the inversion with `sbtk:invert:geometryParameter`. The geometry is found by Gauss–Newton iteration:

- **Each step** linearizes the measurements in the geometry parameters and solves the bounded problem. The derivatives are central differences of the integrals: 10^{-6} km for a sphere inside the body (a point mass), 10^{-4} km and 10^{-2} deg for components integrated along rays, whose integrals are smooth only on scales larger than the ray spacing (Assumptions and limitations).
- **Trust region.** The steps are limited to `opts.step_km` for a center coordinate and `opts.step_deg` for a tilt or turn. A step that does not lower the objective is undone and the trust region halved; a successful step doubles it, up to those limits.
- **A priori constraints.** With `opts.position_sigma_km` or `opts.axis_sigma_deg` finite, each center coordinate, tilt or turn has an a priori constraint of that 1σ about its starting value. These are needed when the measurements do not determine all of the geometry, and they keep a component from absorbing structure the model lacks (below).
- **Hard limits.** `opts.max_shift_km` bounds the distance of an estimated center from its start, `opts.max_tilt_deg` the angle of an estimated axis from its starting direction, and `opts.max_turn_deg` the turn of an ellipsoid about its axis. Spheres are always kept inside the body.
- **Stopping.** Iteration stops after `opts.max_iter` steps, or earlier when any of these holds:
- the objective changes by less than 10^{-10} relative;

- an accepted step changes every parameter by less than `opts.geometry_tol` of its step limit;
- the trust region falls below 10^{-6} km.

`fit.geometry_converged` is false when the steps ran out first, and `sbtk.invert.summary` says so.

Without the step test, a model that lacks structure the data contain keeps lowering χ^2 by ever smaller steps. That runs every repetition about the center of mass to its limit.

For data about the center of mass the fit is repeated about the model's own center of mass (below), and each repetition continues from the geometry of the previous one. The priors and limits always refer to the geometry the components were given.

Absorbed structure. An estimated geometry can absorb structure the model lacks, so freeing it can lower χ^2 while moving the component away from the truth. Example: a Didymos true model has a torus about its spin axis at the center of figure, plus a boulder and a random density field. The data are its exact values weighted by the Hera radio-science precision, in the frame and about the origin of the DART file. An inversion model with a core, the torus (about the z-axis through the center of figure, 3.9° from the true torus axis) and the regolith has a reduced χ^2 of 157 ($R^2 = 0.49$) with fixed geometry. With the torus axis and height estimated (a priori 1σ of 1° and 10 m, limits of 10° and 50 m), χ^2 drops to 49 while R^2 falls to -1.92: the torus moves 50 m up, to its limit, and tilts 6.8° from z to stand in for the boulder, and the regolith density falls to 19 ± 176 kg/m³. At this precision such priors are weak. Priors, or limits tight enough to bind, must express how well the geometry is actually known.

Local minima. A fit whose geometry absorbs structure can have more than one local minimum. The same Didymos fit, in the frame with the center of mass as origin and the spin axes, settled with the torus 41 m above the center of figure (reduced χ^2 39) or, with the derivatives taken over smaller steps, 47 m below it (reduced χ^2 37); which minimum Gauss–Newton reaches depends on its path. Compare fits started from different geometries before reading a position or an axis as a result.

`sbtk.interior.align_with_spin_axis` is the counterpart for making a true model: it sets the axis of tori, ellipsoids or sectors along the model's own axis of the largest moment by fixed-point iteration (Density distributions).

Uncertainties

The covariance of the parameters is the linearized one at the solution, with the mass constraint and the fixed parameters eliminated:

$$P = Z\,(Z^{\mathsf{T}} J^{\mathsf{T}} J Z)^{-1} Z^{\mathsf{T}},$$

where J is the whitened Jacobian of the measurements with the regularization and a priori rows appended, and the columns of Z span the null space of the mass constraint and the fixed parameters. If $Z^{\mathsf{T}} J^{\mathsf{T}} J Z$ is singular (the measurements do not determine every free parameter), the pseudo-inverse is used and `fit.determined` is false. The bounds are not included in P; `fit.at_bound` marks densities at a bound.

The density of a component and its 1σ are $\rho_{\mathrm{bg}} + \Delta\rho_c$ and $\sqrt{P_{\mathrm{bg,bg}} + P_{cc} + 2P_{\mathrm{bg},c}}$ (`fit.rho_kgm3`, `fit.rho_sigma_kgm3`). `fit.resolved` marks components whose relative density exceeds its 1σ. The validation tests check over 40 noise realizations that the standard deviation of (estimate − truth)/σ is within 0.25 of 1, with diagonal and with strongly correlated covariances (Validation).

Regularization

Models with many components, such as a sector grid, or weak data need Tikhonov regularization to keep noise from producing large alternating densities. `sbtk.invert.choose_regularization` with the rule `"discrepancy"` returns the largest λ for which

$$\chi^2 \le \max\left(n,\ \chi^2_{\min} + \sqrt{2n}\ \right),$$

with n the number of measurements and $\chi^2_{\min}$ the value at $\lambda = 10^{-2}$ (the discrepancy principle): χ^2 may rise by about one standard deviation of its distribution. It searches $\log_{10}\lambda \in [-2, 8]$ by bisection with 9 solves; the result is returned, not stored, so set `prob.opts.regularization` before the final run. With the rule `"aic"` it returns the λ of the lowest AIC, $\chi^2 + 2p_{\mathrm{eff}}$, for comparing a regularized model with other models on equal terms. These rules and the smoothest consistent solution are on the pages of the sector grids and point masses.

Data about a fixed point or about the center of mass

When `obs.coef_about` is `'origin'`, the coefficients, degree 1 included, and the inertia are about the fixed point `obs.origin_km`: for a package in the frame and about the origin of its shape model file, as the HLP6 makers send, `[0 0 0]`. Layers and sectors are measured along rays from the center of mass, as in `sbtk.data.simulate`, so the model's geometry is built about the center of mass the degree-1 coefficients give (`fit.origin_km`; if that is within 3σ of `obs.origin_km`, the data do not tell the two apart and `obs.origin_km` is used). Its coefficients, degree 1 included, and its inertia are moved to `obs.origin_km` exactly (`sbtk.internal.sh_translation`, the parallel-axis theorem; `fit.com_shift_km` is that point minus `fit.origin_km`). Degree 1 then compares the model's center of mass with the measured one, and no iteration is needed. Measuring layers and sectors from the origin of the file instead, 52 to 57 m from the center of mass of Didymos, changes a sector by about 25% of its volume, and a fit with exactly the components of the true model no longer recovers it (`testLayersFromMeasuredCenterOfMass`).

When `obs.coef_about` is `'center_of_mass'`, the coefficients of degree 2 and higher are about the body's center of mass, which is known only through its measurement (the degree-1 rows). Expanding the model about the *measured* center of mass would carry the error of that measurement into the predictions: a translation of the expansion point changes the coefficients of degree l by terms proportional to those of lower degrees (Uncertainties and the expansion point). `sbtk.invert.run` therefore

1. keeps the model where it is: its geometry is built about the measured center of mass (`fit.origin_km`), from which layers and sectors are measured;
2. moves its predicted coefficients of degree ≥ 2 and inertia to the model's own center of mass, by the exact translation of solid harmonics (`sbtk.internal.sh_translation`) and the parallel-axis theorem;
3. iterates (Gauss–Newton, including the dependence of the center of mass on the densities, `prob.D`) until that center moves by less than 1 mm (`fit.frame_iterations`, `fit.frame_offset_km`);

4. compares the model's center of mass with the measured one through the degree-1 rows.

Here `fit.com_shift_km` is the model's center of mass minus `fit.origin_km`. The tests `testCenterOfMassFrameMatchesTheTrueFrame` and `testPredictionsAboutTheModelCenterOfMass` check that the densities agree with an inversion about the true center of mass to within 0.2σ and that the predictions equal a forward model computed from scratch about the fitted center of mass (Validation).

Settings

Setting (`opts`)	Standard value	Meaning
`opts.moi_labels`	`{'Ixx','Iyy','Izz'}`	Inertia rows fitted; `{}` for gravity coefficients only.
`opts.regularization`	0	Tikhonov parameter λ.
`opts.background`	`"estimate"`	`"bulk"` holds the background at M / V.
`opts.rho_max_kgm3`	4500	Upper bound on every density, kg/m³.
`opts.max_iter`	200	Largest number of Gauss–Newton steps for estimated geometry; `fit.geometry_converged` is false when they end first.
`opts.step_km`	0.010	Trust-region radius of a center coordinate, km.
`opts.step_deg`	1	Trust-region radius of an axis tilt or turn, deg.
`opts.position_sigma_km`	`Inf`	A priori 1σ of center coordinates, km.
`opts.axis_sigma_deg`	`Inf`	A priori 1σ of axis tilts and turns, deg.
`opts.max_shift_km`	`Inf`	Largest distance of an estimated center from its start, km.
`opts.max_tilt_deg`	`Inf`	Largest angle of an estimated axis from its start, deg.
`opts.max_turn_deg`	`Inf`	Largest turn of an estimated ellipsoid about its axis from its start, deg.
`opts.geometry_tol`	10^{-3}	Converged when every geometry step is below this fraction of its step limit.

`sbtk.invert.options(Name=Value)` changes any of them; the fields can also be set afterwards.

Example

MATLAB script: Estimation with several settings

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
unc  = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
cof = body.centroid_km;                                   % positions from the center of figure
true_components = [sbtk.interior.sphere("core", cof + [0.03 -0.02 0.01], 0.17, +500), ...
                   sbtk.interior.torus("torus", 0.37, 0.055, 0.015, -500), ...
                   sbtk.interior.layer("regolith", 0, 0.012, -900)];
true_components(2).center_km = cof;                       % the torus about the z-axis through cof
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(true_components), 4, "estimation");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);

components = [sbtk.interior.sphere("core", cof, 0.17, 0), ...        % geometry fixed, densities estimated
              sbtk.interior.torus("torus", 0.37, 0.055, 0, 0), ...
              sbtk.interior.layer("regolith", 0, 0.012, 0)];
components(2).center_km = cof;
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
hom  = sbtk.invert.homogeneous(prob);                     % is structure required?
fit  = sbtk.invert.run(prob);
sbtk.invert.summary(fit)

moving = components;  moving(1).estimate_position = true;             % estimate the core center as well
fit_p  = sbtk.invert.solve(body, moving, obs);
fprintf("core center [%.0f %.0f %.0f] m from the center of figure +- [%.0f %.0f %.0f] m\n", ...
        1e3*(fit_p.components(1).center_km - cof), 1e3*fit_p.center_sigma_km(1, :))

raised = components;  raised(2).estimate_position = [false false true];  % estimate the height of the torus
fit_z  = sbtk.invert.solve(body, raised, obs, sbtk.invert.options(max_shift_km=0.030));
fprintf("torus height %.1f m +- %.1f m (true 15 m)\n", 1e3*(fit_z.components(2).center_km(3) - cof(3)), 1e3*fit_z.center_sigma_km(2, 3))
```

The true core is at (30, −20, 10) m from the center of figure and the true torus 15 m above it; the fixed-geometry fit places both at the center of figure. More examples: Calculations.

Fitting part of the coefficients

A model can be fitted to some of the coefficients and tested on the others. This answers two questions: which coefficients determine which parts of a model (can degree 1 and J_2 alone place a core?), and whether a model fitted to the well-measured low degrees predicts the higher ones, which is a test on data it was not fitted to.

Functions

Step	Function
Keep some coefficients	`sbtk.data.select_coefficients(obs, labels)`, `sbtk.data.select_degree(obs, L)`
Fit them	`sbtk.invert.setup`, `sbtk.invert.run` (or `sbtk.invert.solve`)
Predict every coefficient from the fit	`sbtk.invert.predict(fit, body, obs)` with the full set
Map the residuals of several fits	`sbtk.plot.coefficient_residuals`
Compare anomalies coefficient by coefficient	`sbtk.plot.anomaly` with `sbtk.interpret.anomaly` of each fit

`sbtk.invert.predict` evaluates the fitted model (its components with their fitted geometry and densities) for every measurement of the full set: for the fitted rows it gives the fit's own predictions, for the others new ones. It reports the χ^2 of each group with its own covariance (`pred.chi2_fitted`, `pred.chi2_other`). The fit and the full set must give the same measured center of mass, so keep degree 1 in every subset of data about a fixed point; otherwise the model's layers and sectors would be measured from another point and `sbtk:invert:predictOrigin` stops the prediction.

A subset with fewer coefficients than free parameters leaves the fit underdetermined: `fit.determined` is false, the solution is one of many, and its χ^2 is 0 by construction. A core alone has four unknowns (its relative density and three coordinates), which degree 1 and J_2 determine exactly; a core, a torus and a layer have six.

MATLAB fragment. Requires the variables defined in Reading the coefficients of a hidden interior.

```
part = sbtk.data.select_coefficients(obs, ["C10" "C11" "S11" "C20"]);   % degree 1 and J2
core = sbtk.interior.sphere("core", cm.center_km, 0.15, 0);
core.estimate_position = true;
fit1 = sbtk.invert.solve(body, core, part, opts);
pred1 = sbtk.invert.predict(fit1, body, obs);
fit3 = sbtk.invert.solve(body, model, sbtk.data.select_degree(obs, 3), opts);
pred3 = sbtk.invert.predict(fit3, body, obs);
fprintf('degree 1 + J2: chi2 per coefficient of the others %.3g\n', pred1.chi2_other/pred1.n_other);
fprintf('degrees 1-3:   chi2 per coefficient of degrees 4-5 %.3g\n', pred3.chi2_other/pred3.n_other);
fig = sbtk.plot.coefficient_residuals({pred1, pred3}, ["core, degree 1 + J2", "three components, degrees 1-3"]);
```

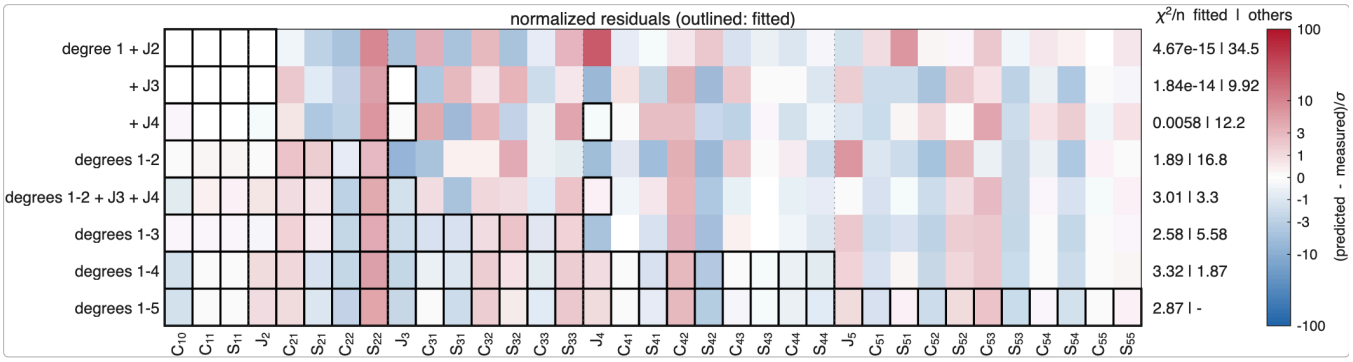
How well the fitted coefficients are matched

A fit to part of the coefficients has to be judged on two groups: the coefficients it was fitted to, which it matches by construction as well as its parameters allow, and the others, which it predicts. `sbtk.plot.coefficient_residuals` shows both for several fits at once, one row per fit and one column per measurement, with the fitted cells outlined; `sbtk.plot.anomaly` shows the anomaly of a few fits against the measured one, filled where fitted and open where predicted.

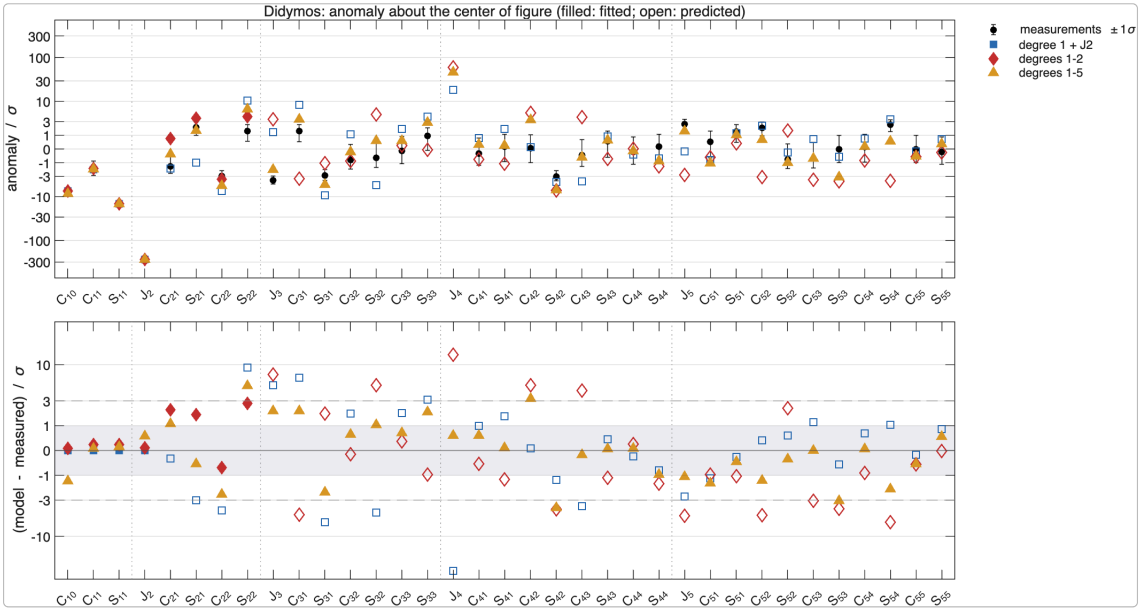
`tests/studies/coefficient_reading_figures.m` does this for one of the study's true interiors (the first of the family "core, torus, layer": a core of +776 kg/m³ and radius 121 m, 54 m from the center of figure, a torus of −652 kg/m³ and a layer of −421 kg/m³), with noise at the Hera precision, fitted with the three-component model of guessed sizes to each of the eight sets, the core placed by its χ^2 profile and Gauss–Newton:

Output

```
truth_051: core +776 kg/m^3, radius 121 m, 53.6 m from the center of figure; torus -652; layer -421
coefficients              fitted  chi2/n fitted  chi2/n others    core drho core error m
degree 1 + J2                  4       4.67e-15          34.5         1799         38.6
+ J3                           5       1.84e-14          9.92          883         28.5
+ J4                           6         0.0058          12.2         -121        187.0
degrees 1-2                    8           1.89          16.8          388         45.7
degrees 1-2 + J3 + J4         10           3.01           3.3          189         78.1
degrees 1-3                   15           2.58          5.58          300         34.9
degrees 1-4                   24           3.32          1.87          560          9.4
degrees 1-5                   35           2.87             -          433         16.8
```



(model – measured)/σ of every coefficient, for the model fitted to each of the eight sets (rows). Outlined: the coefficients of the fit. Right: χ^2 per coefficient of the fitted coefficients and of the others.



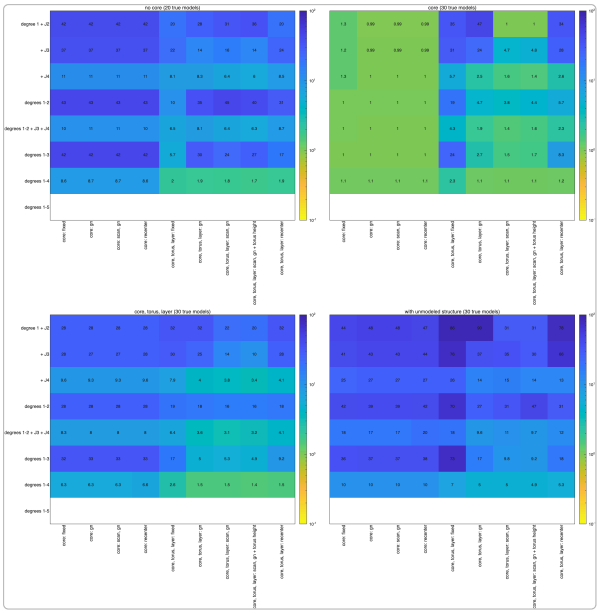
The measured anomaly and the anomalies of the fits to degree 1 + J_2, to degrees 1–2 and to degrees 1–5; filled where fitted, open where predicted.

- With 4, 5 or 6 coefficients for 6 unknowns, the fit matches its coefficients exactly (χ^2 per coefficient 10^{-15} to 10^{-2}): the row is white where it is outlined. That says nothing about the model; the other coefficients do (34.5, 9.9 and 12.2 per coefficient), and the core ends 29 to 187 m from the true one.
- From degrees 1–2 the fit has more coefficients than unknowns and its own χ^2 rises (1.9 to 3.3 per coefficient): the model with these sizes cannot match the coefficients to their 1σ. J_4 brings the prediction of the others from 16.8 to 3.3; degrees 1–4 predict degree 5 at 1.9, and the core is then 9 m from the true one.
- $\bar{S}_{22}$ is several σ off in every fit. The true interior has the same three kinds of component; the model's, of other sizes, do not make its $\bar{S}_{22}$ at any densities and core position.

Results of the study

What the study found reports the 15,840 fits of `tests/studies/coefficient_reading_study.m`. For the choice of coefficients:

- **A core that is alone** is determined by degree 1 and J_2, and the fit predicts every other coefficient as well as the noise allows (1.0 per coefficient). More coefficients do not improve its position (4–5 m at the Hera precision).
- **A core with a torus and a layer** needs J_4: degree 1 + J_2 + J_4 predicts the others at 3.8 per coefficient against 21.5 without J_4; degrees 1–2 + J_3 + J_4 at 3.1; degrees 1–4 predict degree 5 at 1.5. J_3 adds little and, with noise, moves the core to fit the noise.
- **Degrees 1–2 alone are not a test**: 63% of those fits are consistent with their coefficients and still predict the others at 16 per coefficient; for interiors with no core at all, 60% and 45.
- The **fixed-point iteration** that moves the core to match degree 1 (`sbtk.invert.recenter`) converges in about 3 steps for a core alone, and not at all when other components can match degree 1 themselves.



Median χ^2 per coefficient of the coefficients not fitted, with noise, for each set (rows), model and procedure (columns) and family. Blank: every coefficient fitted.

Goodness of fit and model comparison

Chi-square test

For a fit with residuals $\mathbf{r} = \mathbf{y}_{\text{pred}} - \mathbf{y}$ and measurement covariance Σ:

Quantity	Definition	Field
χ^2	$\mathbf{r}^{\mathsf{T}}\Sigma^{-1}\mathbf{r}$, with the full covariance	`fit.chi2`
Effective number of parameters	$p_{\text{eff}} = \operatorname{tr}(J_d P J_d^{\mathsf{T}})$, the trace of the data resolution matrix (J_d whitened measurement Jacobian, P parameter covariance); it accounts for fixed parameters and regularization	
Degrees of freedom	$\nu = n - p_{\text{eff}}$	`fit.dof`
Reduced χ^2	χ^2 / ν; close to 1 when the model and the stated uncertainties are correct	`fit.chi2_reduced`
p-value	$P(\chi^2_\nu \geq \chi^2)$, the probability that a correct model gives a χ^2 at least as large, computed as the upper regularized incomplete gamma function $Q(\nu/2, \chi^2/2)$	`fit.p_value`
Consistent	$p \geq 0.05$: the model is consistent with the measurements at the 5% significance level	`fit.consistent`

`sbtk.invert.goodness_of_fit` computes the last three from χ^2 and ν. When $\nu \leq 10^{-6}$ (at least as many parameters as measurements) the test is undefined: the reduced χ^2 and the p-value are NaN and `consistent` is false.

Homogeneous body

`sbtk.invert.homogeneous` fits a body of uniform density M / V to the same measurements; it has no free parameters ($\nu = n$). If this fit is already consistent with the measurements, the data do not require internal structure at their precision.

Normalized residuals

$(y_{\text{pred}} - y) / \sigma$ for every measurement (`fit.normalized_residual`, `sbtk.plot.residuals`). For a correct model about 68% lie within ±1. Large residuals concentrated in one group indicate structure that the model lacks: in the degree-1 rows, a model that cannot place the center of mass; at one degree, structure at the corresponding scale.

When a fit is rejected ($p < 0.05$), its densities and uncertainties describe the best model of the chosen form, not the body. At high measurement precision, small structures that a component model omits, such as a random density field of a few percent of the bulk density or a boulder tens of meters across, can be enough to reject an otherwise correct model (Hera blind test).

Comparing models: the Akaike information criterion

Models of different kinds fitted to the same measurements (a few components, a sector grid, thousands of point masses) are compared by the Akaike information criterion,

$$\text{AIC} = \chi^2 + 2\,p_{\text{eff}},$$

the form it takes for Gaussian measurement errors of known covariance, up to a constant that is the same for every model. χ^2 rewards the fit and $2p_{\text{eff}}$ penalizes the flexibility that bought it: a parameter is worth adding only if it lowers χ^2 by more than 2. Lower is better, and only differences between models fitted to the same measurements mean anything. As a guide (Burnham and Anderson, *Model Selection and Multimodel Inference*, 2002), a difference below 2 gives no preference, 4 to 7 a clear one, and above 10 a strong one.

- **Regularized models.** For a sector grid or point masses, p_{eff} is the trace of the data resolution matrix and depends on the Tikhonov parameter λ: more smoothing, fewer effective parameters and a larger χ^2. To compare them with other kinds on equal terms, each is taken at the λ of its lowest AIC (Sector grids, Point masses). The smoothest consistent solution (the largest λ with $p \geq 0.05$) shows the least structure the measurements require, but its AIC is higher by design.
- **Sizes chosen by a search.** A size that `sbtk.search.run` chose on its grid was fitted to the measurements too: it counts as one parameter in ν, p and AIC, besides the densities and the centers and axes Gauss–Newton refined (Searching the geometry).
- **What AIC does not say.** AIC ranks how well each model accounts for the measurements it was fitted to, for its number of parameters. It does not say which is closest to the true interior: several models can fit equally well and differ inside. Test the candidates on data they were not fitted to, such as higher degrees, other coefficients (Fitting part of the coefficients) or the gravity at a surface point (Testing with ROB data).

Comparison with a true model

In synthetic tests the true model is known. `sbtk.compare.with_truth(body, truth, fit)` evaluates both densities at the centroids of 24 radial shells of every facet cone (1,179,648 points for the 49,152-facet Didymos model), each weighted by the shell volume w_i, with each model's layers and sectors measured from its own origin. A model with more than 50,000 facets is sampled in the cones of 50,000 of them, chosen at random with a fixed seed: 1.2 million points for the 3,145,728-facet model instead of 75 million, with R^2 within 0.002 of the value from 9.4 million points. It returns (`cmp`):

$$R^2 = 1 - \frac{\sum_i w_i(\rho_{\text{est},i} - \rho_{\text{true},i})^2}{\sum_i w_i(\rho_{\text{true},i} - \bar{\rho}_{\text{true}})^2},$$

the fraction of the volume-weighted variance of the true density about its mean that the estimate explains: 1 for an exact estimate, 0 for one no better than a uniform density, negative for a worse one.

- $R^2_{\max}$: the largest R^2 attainable with the same components, from a weighted least-squares fit of the true density by the background and the components. It measures the limit set by the choice of components; R^2 close to $R^2_{\max}$ means the measurements determined the densities as well as that geometry allows.
- The volume-weighted RMS error and the standard deviation of the true density.
- For each component present in both models: estimate, 1σ, true density and $|\text{estimate} - \text{true}| \ / \ \sigma$.

For many fits of the same components against one true model, `sbtk.compare.reference` samples the true model once and `sbtk.compare.evaluate` compares each fit in milliseconds. `sbtk.compare.summary` prints a comparison and `sbtk.plot.comparison` plots it.

Comparison of density grids

Any two models can be compared on a common cubic grid, independently of how they are parametrized. `sbtk.exchange.density_grid(body, model, spacing_km)` evaluates a model at cell centers that lie at integer multiples of the spacing along the axes of the shape-model frame; cells whose center is inside the shape model (tested along rays from the center of figure) are kept, so all models of a body have the same cells (`grid`). `sbtk.compare.grids(truth, estimate)` compares the cells present in both grids (`gridcmp`): R^2 with equal weights, RMS and mean error, and the fraction of cells whose error is at most twice the 1σ of the estimate. Both grids must have the same spacing (`sbtk:compare:gridSpacing`). This is how solutions in exchange packages are compared with interior-model packages.

Calibration of the uncertainties

The uncertainties of a fit can be checked with repeated noise realizations: if they are correct, the standard deviation of $(\hat{\rho} - \rho_{\text{true}}) \ / \ \sigma$ over many realizations is close to 1.

MATLAB script: Calibration over noise realizations

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
unc  = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
components = [sbtk.interior.sphere("core", body.centroid_km, 0.17, +450), sbtk.interior.layer("regolith", 0, 0.012, -900)];
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(components), 4, "calibration");
exact = sbtk.data.set_uncertainties(exact, unc);

prob = sbtk.invert.setup(body, components, sbtk.data.add_noise(exact, 1), sbtk.invert.options());
z = zeros(30, 3);
for k = 1:30
    f = sbtk.invert.run(sbtk.invert.replace_data(prob, sbtk.data.add_noise(exact, 100 + k)));
    z(k, :) = ([f.rho_bg_kgm3, [f.components.drho_kgm3]] - [truth.rho_bg_kgm3, [components.drho_kgm3]]) ...
              ./[f.rho_bg_sigma_kgm3, f.drho_sigma_kgm3];
end
fprintf("standard deviation of (estimate - truth)/sigma: %.2f\n", std(z(:)))
```

Models with components

A model with components describes the interior as a background density and a few bodies of uniform relative density: a core, a surface layer, a torus, spheres, ellipsoids, sectors (Density distributions). Each is chosen to account for part of the measured anomaly. Such a model has few parameters and can say precisely where a core is and how dense; its risk is the choice of shapes, which the measurements test only through the fit.

What is estimated

Quantity	How	Page
The background density and each component's relative density	linear least squares with the mass constraint and the bounds $0 \le \rho \le \rho_{\max}$	Density estimation
The center of a sphere, ellipsoid or torus, the axis of a torus or ellipsoid	Gauss–Newton with a trust region, a priori constraints and limits	Estimated geometry
Sizes: a radius, the ring and tube radii of a torus, a layer's thickness	a grid search, densities solved at every combination	Searching the geometry
A component's place along a line, such as the center-of-mass offset	the χ^2 profile, then Gauss–Newton from its lowest point	Degree 1 and the core
Compact masses where the residuals ask for them	added one at a time, each kept if it passes the likelihood-ratio test	Searching the geometry

Choosing the components

The anomaly C – C_CD, the measured coefficients minus those of the same shape at constant density, says what to try (Reading the gravity coefficients):

The measurements show	Try
the center of mass off the center of figure (degree 1)	a core along the offset: dense on that side, or light on the opposite side
every coefficient scaled by the same factor	a central mass (dense below 1, light above)
a scaling that falls or rises with degree	a surface layer
only even zonal terms (J_2, J_4) changed	a torus or ring about the spin axis
$\bar{C}_{22}$, $\bar{S}_{22}$ changed	an elongated core, or bodies at the ends of an equatorial axis
the moments of inertia larger (or smaller) than at constant density	mass farther from (closer to) the center: a dense surface or a light center
every order of degrees 2 to 4 changed	something off center and not symmetric: compare with a sector grid or point masses

Fitting a model

MATLAB

```
cof = body.centroid_km;
rm  = (3*body.volume_km3/(4*pi))^(1/3);                    % mean radius
core  = sbtk.interior.sphere("core", cof, 0.40*rm, 0);
torus = sbtk.interior.torus("torus", 0.95*rm, 0.15*rm, 0, 0);
torus.center_km = cof;                                     % about the z-axis through the center of figure
layer = sbtk.interior.layer("regolith", 0, 0.03*rm, 0);
opts = sbtk.invert.options(moi_labels={'Ixx', 'Iyy', 'Izz'});

% place the core by its chi-square profile along the center-of-mass offset
an = sbtk.interpret.anomaly(obs, body, obs.degree);
u  = an.com_offset_km/norm(an.com_offset_km);
sc = sbtk.invert.scan_position(sbtk.invert.setup(body, [core, torus, layer], obs, opts), "core", ...
                               cof + (-0.40:0.025:0.40)'*rm*u);
core.center_km = sc.centers_km(sc.best, :);
core.estimate_position = true;                             % then Gauss-Newton from there
fit = sbtk.invert.solve(body, [core, torus, layer], obs, opts);
fprintf("reduced chi-square %.2f, AIC %.1f\n", fit.chi2_reduced, fit.chi2 + 2*(fit.n_obs - fit.dof))
```

`projects/hera_hlp6/analyze_rob.m` fits the models of `rob_candidates.m` to every ROB package in this way, from a core at the center of figure to models with four or five components, runs two searches of the geometry (Searching the geometry), and ranks them all by AIC. The runner `rob<n>_components.m` of each package fits the model of lowest AIC (Code of the ROB solutions).

Rules

- **Start simple, add one component at a time**, and keep it only if it lowers the AIC: it must lower χ^2 by more than 2 for each parameter it adds, a size chosen on a grid included (Goodness of fit).
- **Place before refining.** Degree 1 fixes only the product of a core's mass excess and its offset: a dense core on one side and a light core on the other give the same degree 1. Gauss–Newton started at the center of figure can end on the wrong side. On synthetic interiors with a lone core, a core, torus and layer model started from the lowest point of the χ^2 profile found the core within 10 m in 83% of them, started from the center in 17% (What the study found).
- **Check that the geometry converged.** Where a density and a position trade against each other, as a dense core's density and its offset do, Gauss–Newton can need more than a hundred steps, as for the core of ROB-DATA4. `fit.geometry_converged` false means the `opts.max_iter` steps (200 by default) ran out first, and `sbtk.invert.summary` says so: raise it and fit again.
- **A density at a bound** (`fit.at_bound`) means the shape or place is wrong, or the component too small for the mass the measurements want there.

- **A density within 1σ of zero** (`fit.resolved` false) is not resolved: the component can be dropped.
- **Free geometry can absorb structure the model lacks**, and can have several minima. Compare fits started from different places, and give the geometry the a priori uncertainty it really has (Estimated geometry).
- **Test on what was not fitted.** A model that fits the measured degrees can still be wrong inside: compare it with a solution that assumes no shapes (point masses), and with data it was not fitted to (Fitting part of the coefficients).

Searching the geometry

The densities of a model are linear in the measurements; its geometry is not. `sbtk.invert.run` estimates the densities, and by Gauss–Newton the centers and axes it is asked to, from where the model starts. It does not estimate sizes (the radii of a torus, the thickness of a layer), it finds the minimum of the valley it starts in, and it fits only the components it is given. What the study found showed what that costs: with guessed sizes a core is placed within 10 m in about a third of the interiors, and with boulders the model does not fit at all. `sbtk.search` searches the geometry systematically, for any model and any set of measurements.

The method

For fixed geometry, the densities cost one small least-squares solve. With every center, size and axis fixed, the measurements are linear in the densities, and with the mass constraint one density follows from the others. The χ^2 of a geometry is then the minimum over the densities, from the component integrals alone. The integrals of a component depend only on its own geometry: a grid that varies the ring and tube radii and the axis of a torus, the thickness of a layer and the position of a core needs integrals for each combination of the values of each component's own parameters, and then a least-squares solve for every combination of the components. 140 torus shapes, 4 layer thicknesses and 17 core positions are 9,520 combinations from 161 integrals; on the 49,152-facet Didymos model a torus integral takes 0.9 s and a solve a fraction of a millisecond.

The search goes from the coarse to the fine and adds parameters only where the measurements need them:

1. **Grid** (`sbtk.search.grid`): every combination of the values of the searched parameters, densities without bounds; the best few fitted again with the bounds of `sbtk.invert.run`. The χ^2 profile of each parameter, the lowest χ^2 over all others at each of its values, shows which values the measurements allow (`sbtk.plot.search_profiles`).
2. **Gauss–Newton from the best points** (`sbtk.search.refine`): the centers and axes with derivatives refined from each of the best combinations, which start in different valleys; the fits ranked by χ^2.
3. **Compact masses** (`sbtk.search.add_masses`): the residuals of the model tell where one more mass would lower χ^2 most. For every position of a grid of candidates inside the body, a point mass (a sphere inside the body is exactly one) is added and the densities fitted again: one solve per position, a map of where and with which sign the measurements ask for mass. The best few positions are refined by Gauss–Newton with the position derivatives, then with all the estimated geometry free, and the mass is kept only if it lowers χ^2 by at least the $1-\alpha$ quantile of χ^2 with 4 degrees of freedom, the likelihood-ratio test for a model with a center and a density more (9.49 for $\alpha = 0.05$). The step repeats until the model is consistent with the measurements ($p \geq 0.05$), a mass is not significant, or a largest number is reached (`sbtk.plot.search_masses`).

`sbtk.search.run` does the three steps in order, each from the best result of the one before, and logs each. A value chosen on the grid is fitted to the measurements too: each searched parameter that Gauss–Newton does not refine afterwards, such as a size, counts as one parameter in the degrees of freedom, p and AIC of every fit the search reports, beside the densities and the centers and axes Gauss–Newton refines. Called alone, `sbtk.search.add_masses` takes their number as `n_fixed`.

What can be searched

Any component of any model, by name, with `sbtk.search.parameter`:

Component	Fields	Gauss–Newton
sphere	`center_km`, `radius_km`	center
ellipsoid	`center_km`, `semi_axes_km`, `axis`	center, orientation
torus	`center_km`, `ring_radius_km`, `tube_radius_km`, `height_km`, `axis`	center (height), axis
layer	`depth_km` (top and bottom)	–
sector	`frac`, `lat_deg`, `lon_deg`, `axis`	–

Sizes have no derivatives in SBTK and are searched on the grid; centers and axes are searched coarsely on the grid (the side of a core, the tilt of a torus) and refined by Gauss–Newton. `sbtk.search.directions` spreads axis directions evenly in a cone. Several parameters of one component vary together. The measurements can be any set: all coefficients, a subset (`sbtk.data.select_coefficients`), inertia rows; the search targets whatever `obs` contains.

Two limits hold for any search:

- **A sphere inside the body is a point mass.** The coefficients see its mass and position, not its radius: a dense small sphere and a less dense large one give the same coefficients. Its radius matters only through the density bounds (`opts.rho_max_kgm3` caps its mass).
- **A thin layer is seen through its mass per area**, thickness times density, unless the measurements are precise enough to separate them. Its profile is then flat along the thickness.

Functions

Step	Function
Parameters and axis directions	`sbtk.search.parameter`, `sbtk.search.directions`
Grid	`sbtk.search.grid`, `sbtk.plot.search_profiles`
Gauss–Newton from several starts	`sbtk.search.refine`
Compact masses	`sbtk.search.add_masses`, `sbtk.plot.search_masses`
All steps	`sbtk.search.run`, `sbtk.search.settings`

What the search finds

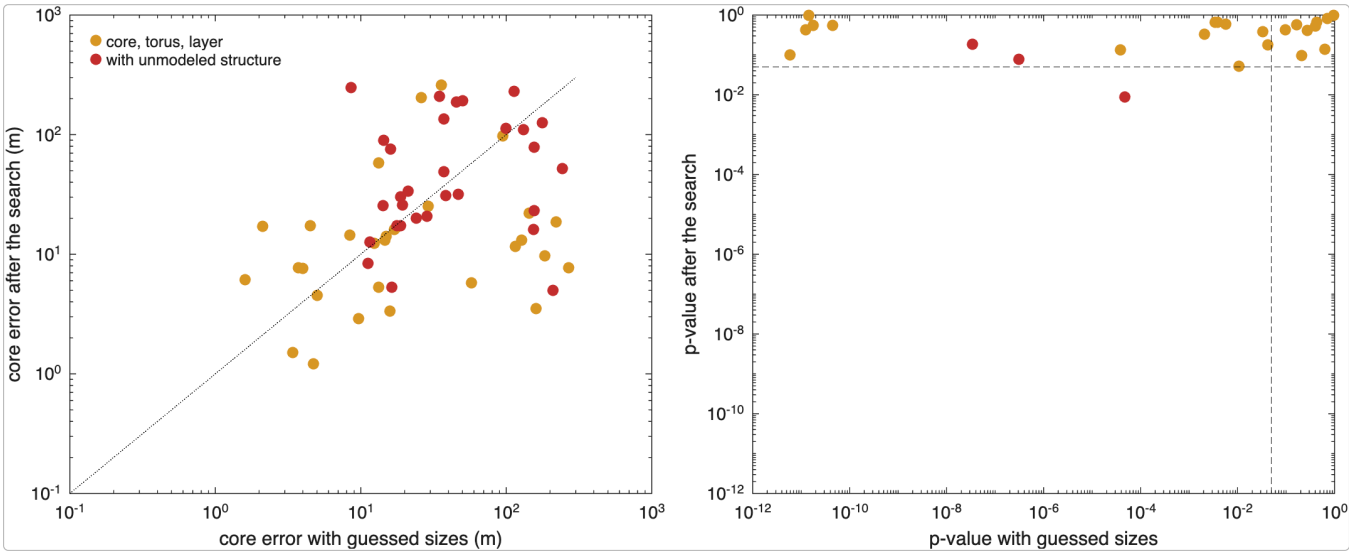
`tests/studies/geometry_search_study.m` searches the 60 interiors of the coefficient reading study that have a torus and a layer, exact and with the Hera radio-science noise, with a core, a torus and a layer: the core along the center-of-mass offset (−80 to 80 m), the ring radius (300 to 400 m) and tube radius (35 to 65 m), 7 axis directions up to 4° from z and 4 layer thicknesses on the grid (9,520 combinations from 161 integrals), Gauss–Newton on the core position and the torus height and axis from the best three starts, then up to four compact masses. It compares the result with the true interior and with the same model of guessed sizes. 120 searches took 2.7 hours on 10 workers, 7 to 17 minutes each.

Output

```
Geometry search study: medians over the true models of each family (noisy data, degrees 1-5).
search: sbtk.search.run (grid of sizes, axis and core position; Gauss-Newton; compact masses).
fixed sizes: the same model with guessed sizes, core placed by its chi-square profile and Gauss-Newton
(coefficient_reading_study.m). Boulder found: a mass within 40 m of a true boulder.

== core, torus, layer (30 true models)
                                          search  fixed sizes
core within 10 m (fraction)                 0.43         0.33
core error (median, m)                      12.7         15.4
mass excess error (median, of the true)     0.18         0.46
consistent (p >= 0.05, fraction)            1.00         0.33
density R2 (median)                         0.26         0.23
ring radius error (median, m)                7.1
tube radius error (median, m)               11.6
torus height error (median, m)               2.2
torus axis error (median, deg)              0.62
layer thickness error (median, m)            7.9
masses added (mean)                         0.07
true boulders (mean)                        0.00
boulders found (fraction of true)           0.00
time per true model (median, s)              454

== with unmodeled structure (30 true models)
                                          search  fixed sizes
core within 10 m (fraction)                 0.10         0.03
core error (median, m)                      32.7         36.0
mass excess error (median, of the true)     0.62         0.55
consistent (p >= 0.05, fraction)            0.27         0.00
density R2 (median)                        -0.21        -0.31
ring radius error (median, m)               13.8
tube radius error (median, m)                8.1
torus height error (median, m)               5.3
torus axis error (median, deg)              1.91
layer thickness error (median, m)           11.2
masses added (mean)                         3.20
true boulders (mean)                        1.77
boulders found (fraction of true)           0.02
time per true model (median, s)             1030
```

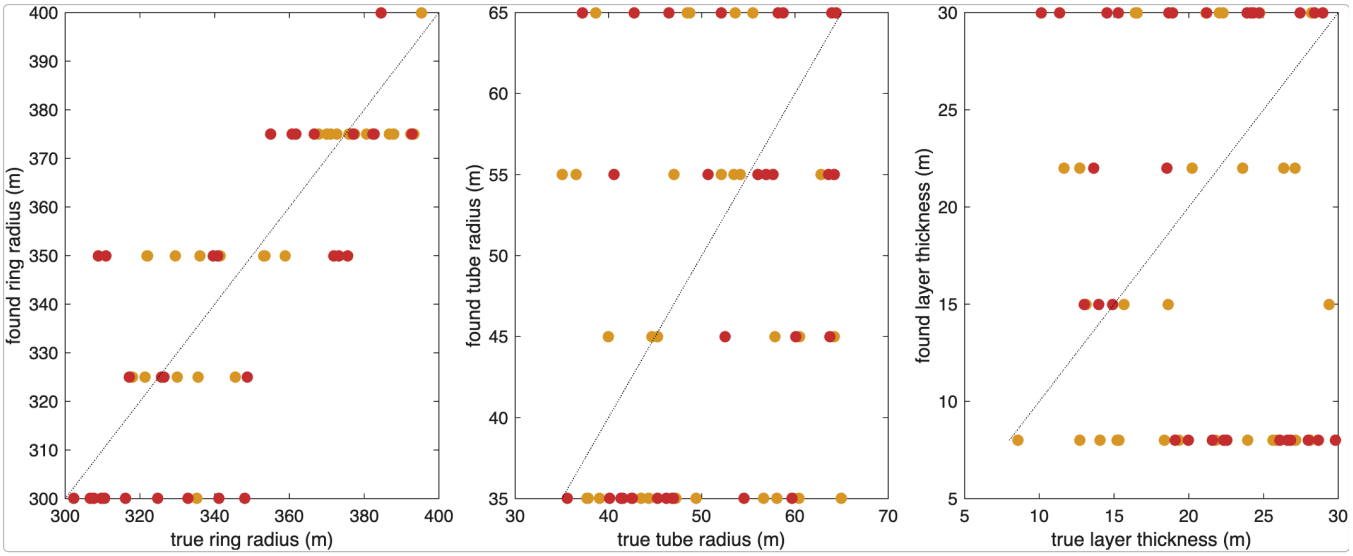
Output

```
Geometry search study: medians over the true models of each family (exact data, degrees 1–5).
search: sbtk.search.run (grid of sizes, axis and core position; Gauss–Newton; compact masses).
fixed sizes: the same model with guessed sizes, core placed by its chi-square profile and Gauss–Newton
(coefficient_reading_study.m). Boulder found: a mass within 40 m of a true boulder.

== core, torus, layer (30 true models)
                                          search  fixed sizes
core within 10 m (fraction)                0.83         0.53
core error (median, m)                      3.5          8.4
mass excess error (median, of the true)    0.11         0.38
consistent (p >= 0.05, fraction)           1.00         0.57
density R2 (median)                        0.45         0.05
ring radius error (median, m)               6.9
tube radius error (median, m)               8.9
torus height error (median, m)              0.6
torus axis error (median, deg)             0.06
layer thickness error (median, m)           8.7
masses added (mean)                        0.00
true boulders (mean)                       0.00
boulders found (fraction of true)          0.00
time per true model (median, s)             437

== with unmodeled structure (30 true models)
                                          search  fixed sizes
core within 10 m (fraction)                0.13         0.10
core error (median, m)                     26.7         26.9
mass excess error (median, of the true)    0.29         0.40
consistent (p >= 0.05, fraction)           0.57         0.03
density R2 (median)                       -0.28        -0.12
ring radius error (median, m)              15.7
tube radius error (median, m)              10.5
torus height error (median, m)              3.1
torus axis error (median, deg)             1.65
layer thickness error (median, m)           8.6
masses added (mean)                        2.67
true boulders (mean)                       1.77
boulders found (fraction of true)          0.02
time per true model (median, s)             948
```



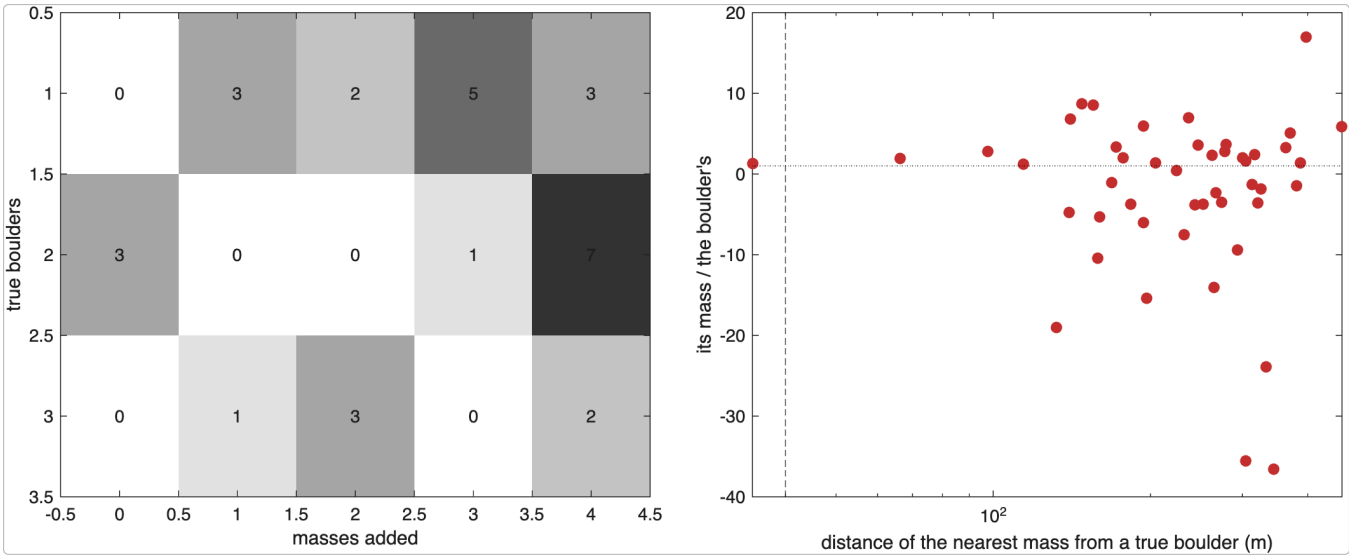
With noise: the core's distance from the true core (left) and the p-value of the fit (right), after the search against with guessed sizes, one point per interior.

Interiors of a core, a torus and a layer. The search fits the coefficients in every interior ($p \geq 0.05$ in all 30, against 10 of 30 with guessed sizes), places the core within 10 m in 43% (83% without noise, against 33% and 53%), and determines the core's mass excess to 18% (11%), against 46% (38%). The torus comes out with its ring radius within 7 m, its height within 2 m and its axis within 0.6° (median, with noise). Its tube radius and the thickness of the layer are not determined: at degree 5 a thin torus and a thin layer are seen through their mass (cross-section or thickness times density), and the grid ends at its first or last value as often as near the true one.



With noise: the ring radius, tube radius and layer thickness found against the true ones (grid steps of 25 m, 10 m and 7 m), colors as in the figure above.

Interiors with boulders and a random field. The search fits the coefficients in 27% of them with noise (none with guessed sizes) and 57% without, but the core is no better placed (within 10 m in 10% and 13%), and **the compact masses do not find the boulders**: about three are added per interior, each passing the likelihood-ratio test, at a median 230 m from the nearest true boulder (2% within 40 m), with masses up to tens of times the boulders' and of either sign. They fit the coefficients, and the density moves further from the true one (median R^2 of the density −0.10 before them, −0.21 after). The boulders are 25 to 50 m across, at most 10^{-3} of the mass, 120 to 260 m from the center: to degree 5 their fields are smooth, and other components (the torus's tilt, the random field, a mass elsewhere) make the same coefficients. A compact mass that passes the test says that the model lacks something the coefficients see, not where a boulder is.



Without noise: masses added against the number of true boulders (left); the nearest added mass to each true boulder, its distance and its mass over the boulder's (right). The dashed line is 40 m.

In short: search the sizes and the axis on a grid and refine by Gauss–Newton from each valley; it finds the parameters the coefficients determine (the core, the ring radius, the axis) and shows by its profiles which they do not (a thin tube or layer). Add compact masses to see whether the model lacks structure, not to locate boulders, unless they are large and shallow enough to dominate a degree.

Point masses on the same interiors

The same 60 interiors fitted with point masses, which assume no shapes, are on the Point masses page.

Sector grids

A sector grid divides the whole body into blocks bounded by latitude, longitude and depth, each with its own density. Like point masses it assumes no shapes, but its blocks are large: tens of them instead of thousands of points, each reaching from one radial fraction of the way to the surface to another. It shows in which directions and at which depths the body is denser or lighter than its bulk density.

Building a grid

`sbtk.interior.sector_grid(frac_edges, lat_edges_deg, lon_edges_deg)` returns one sector per radial shell, latitude band and longitude band (Density distributions). Radial edges are fractions of the distance from the expansion origin to the surface along each ray, so every shell follows the shape. The grids of the ROB pages:

Grid	Radial edges	Latitude edges	Longitude edges	Sectors
1 × 4 × 8	0, 1	−90°, −45°, 0°, 45°, 90°	every 45° from −180°	32, each from the center to the surface
2 × 4 × 8	0, 0.5, 1	the same	the same	64: an inner and an outer shell

Sectors near the poles are smaller than those at the equator, and inner sectors smaller than outer ones; their volumes enter the fit through their integrals.

The fit

A sector grid is fitted like any model with components (Density estimation), with two differences:

- **The background is held at the bulk density** (`opts.background = "bulk"`). The sectors fill the body, so the background and a common change of every sector density are the same thing.
- **Tikhonov regularization** penalizes the relative densities, $\lambda \sum_c (\Delta\rho_c / \bar{\rho})^2$, so that the measurements, which determine fewer combinations than there are sectors, do not produce large alternating densities. The bounds $0 \le \rho \le \rho_{\max}$ still apply. The effective number of parameters is the trace of the data resolution matrix (`fit.dof`).

Choosing the regularization parameter

Rule	λ	Use
lowest AIC	`sbtk.invert.choose_regularization` with `"aic"`: the lowest $\chi^2 + 2p_{\text{eff}}$, by a scan of $\log_{10}\lambda$ from −3 to 8 in steps of 0.25, then golden-section search	comparing sector grids with other kinds of model on equal terms (Goodness of fit)
smoothest consistent	the largest with $p \ge 0.05$; not a rule of `sbtk.invert.choose_regularization`: a bisection on $\log_{10}\lambda$ with `sbtk.invert.run` at each step	the least structure the measurements require
discrepancy principle	`sbtk.invert.choose_regularization` with `"discrepancy"`: the largest with $\chi^2 \le \max(n, \chi^2_{\min} + \sqrt{2n})$	noisy measurements of known precision (Regularization)

MATLAB

```
sectors = sbtk.interior.sector_grid([0 1], -90:45:90, -180:45:180);   % 1 x 4 x 8
prob = sbtk.invert.setup(body, sectors, obs, sbtk.invert.options(background="bulk", moi_labels={'Ixx', 'Iyy', 'Izz'}));
prob.opts.regularization = sbtk.invert.choose_regularization(prob, "aic");
fit = sbtk.invert.run(prob);
```

Reading the result

- **Each sector's relative density and its 1σ** (`fit.components(c).drho_kgm3`, `fit.drho_sigma_kgm3`). The regularization correlates neighboring sectors, and a sector whose relative density is within 1σ of zero is not resolved.
- **What a grid can represent.** A sector reaches from its inner to its outer radial fraction. One shell cannot put a dense core under a normal surface or a light ring 300 m out under a normal crust: on ROB-DATA3, a light ring about the spin axis, both grids stay far from the measurements (ROB-DATA3). Where the anomaly is spread over a direction and its depth, a grid fits with few assumptions: on ROB-DATA2, a tilted, off-center anomaly, a grid of 32 sectors has the lowest AIC, tied with a model with components that needs four compact masses (ROB-DATA2).
- **The edges are a choice.** A structure that straddles an edge is split between two sectors. Compare grids with different edges, or with point masses (Point masses), before reading a single sector.

Point masses

A point-mass solution fills the body with a grid of small masses and fits all of them together to the measurements. It assumes no shapes: where the solution puts mass comes from the measurements and the regularization alone. It is the mass-concentration (mascon) model of gravity-field work, here solved for the relative density of each point.

With thousands of points and tens of measurements, the measurements determine only a few combinations of the masses, and the regularization chooses the rest. Read the result as where the anomaly is concentrated, not as a density map.

Building the grid

`sbtk.interior.sphere_grid(body, spacing_km, radius_km)` places one sphere at every point of a cubic grid centered on the center of figure whose sphere lies inside the body. A sphere inside the body is integrated exactly as a point mass of its volume times its relative density (Density distributions), so with a radius well below the spacing the spheres are point masses and the grid cell around each stands for its mass.

Body	Spacing	Points
Didymos (mean radius 365 m)	41 m (mean radius / 9)	2,667
Dimorphos (mean radius 75 m)	8.3 m	2,674

Finer grids change little: the measurements, not the grid, limit the resolution.

The fit

`sbtk.interpret.point_masses(prob, weighting, rule)` solves the linear problem on a problem prepared by `sbtk.invert.setup` with the grid as components. With the mass of the body known, a relative density x_k in point k lowers the density of the rest of the body by $x_k V_k / V$; the columns a_k of A are the measurements of a unit relative density in point k with that share of the homogeneous body taken away, whitened by the Cholesky factor of the covariance. The relative densities minimize

$$\|Ax - r\|^2 + \lambda \|Wx\|^2,$$

with r the whitened difference between the measurements and the homogeneous body. The solution is $x = W^{-1}A_w^{\mathsf{T}}(K + \lambda I)^{-1}r$ with $A_w = AW^{-1}$ and $K = A_w A_w^{\mathsf{T}}$, an $n \times n$ system for n measurements however many points there are. There are no bounds: a point can be lighter than empty space or denser than rock, because each stands for the mass of its cell. The effective number of parameters is $p_{\text{eff}} = \operatorname{tr} K(K + \lambda I)^{-1}$.

Weighting

W sets which solutions the regularization prefers among the many that fit.

Weighting	W	Effect
`"equal"`	I	the minimum-norm solution: the smallest relative densities that fit. Points near the surface change the coefficients most per unit density, so the anomaly is put there, more so at higher degree
`"sensitivity"`	$\operatorname{diag}(\|a_k\|)$	each point's penalty in proportion to its sensitivity, so deep points are not suppressed for being less visible; the anomaly is spread over depth (a form of the depth weighting of potential-field inversion)

On the six ROB packages neither weighting is better everywhere. Their AIC differs by less than 4. Above the fitted degrees sensitivity weighting reproduces ROB's exact values far better on ROB-DATA1 and better on ROB-DATA5, equal weighting far better on ROB-DATA4 and better on ROB-DATA6, and the two are close on ROB-DATA2 and 3 (Testing with ROB data). The ROB pages show sensitivity weighting.

Choosing the regularization parameter

The rule sets λ, and with it how much structure the solution has:

Rule	λ	Use
`"aic"`	the lowest AIC, $\chi^2 + 2p_{\text{eff}}$ (a scan of $\log_{10}\lambda$ from −12 to 12, then golden-section search)	comparing point masses with other kinds of model on equal terms (Goodness of fit)
`"consistent"`	the largest with $p \geq 0.05$ and at least one degree of freedom left (scan, then bisection)	the smoothest solution the measurements allow: the least structure they require
`"discrepancy"`	the largest with $\chi^2 \leq n$ (bisection)	the discrepancy principle; with few measurements it leaves χ^2 near n, not consistent

If no λ is consistent, `"consistent"` takes the one with the largest p-value and sets `pm.consistent` false. With measurements that are nearly exact (or as precise as Hera's for Dimorphos), the AIC rule fits far more closely than the consistent one: on the ROB Dimorphos packages it lowers the AIC of the point masses by 14 to 21.

Reading the result

`pm` (`pm`) holds the relative density and the mass anomaly of every point (the masses sum to zero), the totals and mass-weighted centroids of the positive and negative parts, χ^2, p_{eff} and the p-value, and the normalized residual of every measurement. `pm.model` is the solution as a model: `sbtk.interpret.anomaly`, `sbtk.gravity.forward` and `sbtk.gravity.field_at` evaluate it like any fit, so its coefficients of any degree and its field at a point

can be compared with other solutions. `sbtk.plot.point_masses` draws it averaged over cells of the grid spacing; `sbtk.plot.solution_cuts` draws it beside other solutions on the same color scale.

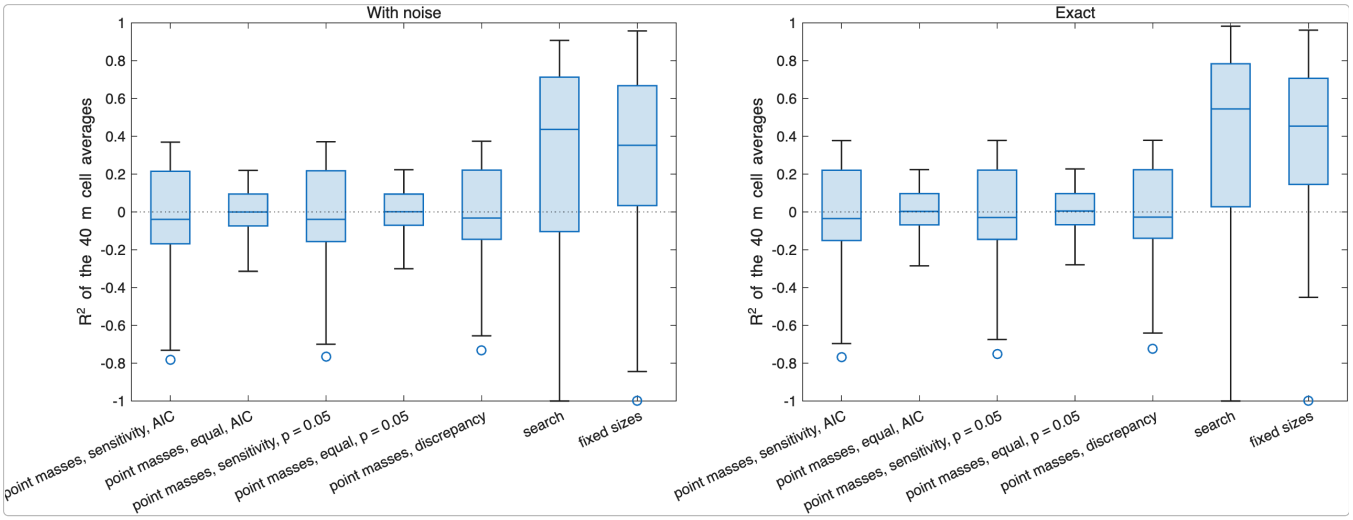
- **Where, not what.** The positive and negative parts show the side and the depth range of an anomaly. They do not show its shape: a dense sphere comes out as a smooth lobe, a ring as a band.
- **Point masses against components.** Where the point masses put mass where a model of a few components does, the measurements, not the choice of components, place it. Where they differ, the measurements allow both.
- **Not a density map.** On 60 synthetic interiors whose truth is known, the cell averages of the point masses explain little of the true density at their own resolution, much less than a model of the right components (Point masses on known interiors).

Point masses on known interiors

`tests/studies/point_mass_study.m` fits the 60 interiors of the geometry search study (core, torus and layer, with and without boulders and a random density field), exact and with noise, with 1,389 point masses every 40 m, each weighting and rule. Every solution and the true interior are averaged over the same 40 m cells, the resolution of the point masses, and compared by the R^2 of the cell averages (the fraction of the variance of the true cell averages explained):

Output

```
Point masses and models with components: medians over the true models (degrees 1–5).
cell R2: R2 of the density averaged over 40 m cells, against the true interior averaged over the same cells.

== core, torus, layer, noisy data (30 true models)
                                      cell R2      chi2/nu   consistent   parameters
point masses, sensitivity, AIC          −0.08        0.911         1.00         32.2
point masses, equal, AIC                −0.04        0.916         1.00         32.5
point masses, sensitivity, p = 0.05     −0.06         2.17         0.93         29.7
point masses, equal, p = 0.05           −0.04         2.21         0.87         29.9
point masses, discrepancy               −0.05         4.16         0.00         26.6
search                                   0.70         1.03         1.00          9.0
fixed sizes                              0.60         1.85         0.33          6.0

== with unmodeled structure, noisy data (30 true models)
                                      cell R2      chi2/nu   consistent   parameters
point masses, sensitivity, AIC           0.07        0.942         1.00         32.6
point masses, equal, AIC                 0.01        0.951         1.00         32.4
point masses, sensitivity, p = 0.05      0.08         2.24         0.97         30.2
point masses, equal, p = 0.05            0.02         2.17         0.93         29.6
point masses, discrepancy                0.08         4.52         0.00         27.2
search                                   0.02         1.98         0.27         25.0
fixed sizes                              0.30         9.46         0.00          6.0

== core, torus, layer, exact data (30 true models)
                                      cell R2      chi2/nu   consistent   parameters
point masses, sensitivity, AIC          −0.06        0.854         1.00         31.6
point masses, equal, AIC                −0.03        0.851         1.00         32.0
point masses, sensitivity, p = 0.05     −0.06         2.07         0.93         28.7
point masses, equal, p = 0.05           −0.03         2.12         0.97         29.2
point masses, discrepancy               −0.05         3.82         0.00         25.8
search                                   0.76       0.0408         1.00          9.0
fixed sizes                              0.68        0.915         0.57          6.0

== with unmodeled structure, exact data (30 true models)
                                      cell R2      chi2/nu   consistent   parameters
point masses, sensitivity, AIC           0.09        0.866         1.00         32.2
point masses, equal, AIC                 0.02        0.888         1.00         31.6
point masses, sensitivity, p = 0.05      0.09         2.16         0.90         29.6
point masses, equal, p = 0.05            0.02         2.07         0.97         28.7
point masses, discrepancy                0.09         4.26         0.00         26.8
search                                   0.11          1.6         0.57         21.0
fixed sizes                              0.38         9.09         0.03          6.0
```



R^2 of the 40 m cell averages of each solution against the true interior, with noise (left) and without (right), 60 interiors each (box: the middle half; line: the median).

The rule changes how closely the point masses fit, not what they find. With the AIC rule they fit every interior (median reduced χ^2 0.85 to 0.95) with about 32 effective parameters of the 35 coefficients; with the consistent rule they fit almost every interior with about 29. Either way their median R^2 stays between −0.08 and 0.09. On the interiors of a core, a torus and a layer the search and the model of fixed sizes reach 0.60 to 0.76. With boulders and a random density field every kind falls to 0.38 or below.

Example

MATLAB

```
grid = sbtk.interior.sphere_grid(body, 0.041, 0.008);                % Didymos: 2,667 points
prob = sbtk.invert.setup(body, grid, obs, sbtk.invert.options(moi_labels={'Ixx', 'Iyy', 'Izz'}));
pm   = sbtk.interpret.point_masses(prob, "sensitivity", "aic");
fprintf("%.1f effective parameters, reduced chi-square %.2f, AIC %.1f\n", pm.p_eff, pm.chi2_reduced, ...
        pm.chi2 + 2*pm.p_eff)
fig  = sbtk.plot.point_masses(body, pm, 0.041);
```

Results folders and figures

Run folders

`sbtk.io.new_run(project, name)` creates the folder `results/<project>/<yyyy-mm-dd_HHMMSS>_<name>/` in the repository and starts recording everything printed in the Command Window to `log.txt` in it, until `sbtk.io.finish_run` closes the log. It returns an `out` structure, which every save and report function takes. A new run never overwrites an earlier one: a run of the same name started in the same second waits for the next second.

- `sbtk.io.latest_run(project, name)` returns the newest folder whose name after the time stamp equals `name` exactly: `"didymos_T1"` does not match `didymos_T1_inversion`. Inversion scripts use it to find the newest simulation, and the figure scripts of `projects/hera_hlp6/rob/` the `solution.mat` of the newest run of each solution.
- `sbtk.io.subfolder(out, name)` makes the save and report functions write into a subfolder while the log stays in the run folder. The Hera simulation runners use it to keep the true model apart from the data package (Hera blind test).

MATLAB

```
out = sbtk.io.new_run("my_project", "T1_inversion");     % results/my_project/<date_time>_T1_inversion/
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
sbtk.io.report_inversion(out, body, sbtk.invert.run(prob), sbtk.invert.homogeneous(prob));
sbtk.io.finish_run(out);
```

This fragment requires `body`, `components` and `obs` as defined in the first script.

Files

File	Written by	Contents	Read with
`log.txt`	`sbtk.io.new_run`	Everything printed during the run.	a text editor
`settings.json`	`sbtk.io.save_settings`, called by the report functions	Body, degree, uncertainties, components, random field, noise seed and inversion settings of the run.	`jsondecode(fileread(file))`
`measurements.txt`, `.mat`	`sbtk.io.save_measurements`	The measurement set (`obs`); the text has only the diagonal of the coefficient covariance.	`sbtk.io.load_measurements`
`truth.txt`, `.mat`	`sbtk.io.save_truth`	The true model of a simulation (`truth`).	`sbtk.io.load_truth`
`fit.txt`, `.mat`	`sbtk.io.save_fit`	Densities with 1σ, χ^2 test, every residual (`fit`).	`sbtk.io.load_fit`
`homogeneous.txt`	`sbtk.io.report_inversion`	The same data fitted by a homogeneous body.	
`comparison.txt`, `.mat`	`sbtk.io.save_comparison`	R^2, $R^2_{\max}$, RMS error, per-component comparison (`cmp`).	`load`
`figures/*.png`	`sbtk.io.save_figure` (200 dpi)	The figures below.	

The report functions write a standard set in one call:

Function	Writes
`sbtk.io.report_simulation(out, body, obs, truth)`	`measurements.*`, `truth.*`, `settings.json`, `figures/truth_3d.png`, `truth_slices.png`, `measurements.png`
`sbtk.io.report_inversion(out, body, fit, homogeneous)`	`fit.*`, `homogeneous.txt`, `settings.json`, `figures/residuals.png`, `densities.png`, `estimate_3d.png`, `estimate_slices.png`
`sbtk.io.report_comparison(out, body, truth, fit, cmp)`	`comparison.*`, `settings.json`, `figures/comparison.png`, `truth_vs_estimate.png`

Text formats

Measurements (`sbtk.io.write_measurements`, `sbtk.io.read_measurements`): one keyword per line, values separated by spaces, `#` lines are comments. `coef` lines hold l, m, $\bar{C}_{lm}$, $\bar{S}_{lm}$ and their 1σ (NaN for a coefficient that is not in the set); `inertia` lines hold the label, value, 1σ and the six weights. The first lines of a file:

Output

```
# Didymos measurements: didymos_example (SBTK 1.0.0)
# Simulated by SBTK from interior didymos_example. Values include simulated measurement noise (seed 1).
# Lines: keyword then values. coef = gravity coefficient: degree l, order m, C, S and their
# 1-sigma uncertainties (fully normalized, Kaula, no Condon-Shortley phase; dimensionless).
# inertia = one moment-of-inertia measurement on I/(M R_ref^2): label, value, 1-sigma, then
# its weights on Ixx Iyy Izz Ixy Ixz Iyz. Lengths in km, mass in kg.
body Didymos
name didymos_example
mass_kg 5.272999999999999e+11
GM_km3_s2 3.519358390000000e-08
R_ref_km 4.300000000000000e-01
degree 4
origin_km 1.528148834993066e-02 4.288469572137803e-02 -3.405288560049095e-02
coef_about origin   (origin: coefficients about origin_km; center_of_mass: degree >= 2 about the center of mass)
com_minus_cof_km -6.480457727157593e-04 5.708425733001801e-04 -7.435287904191540e-05
noise_seed 1   (NaN = exact values or real data)
libration_sigma 2.000000000000000e-02   (1-sigma of the libration ratio; NaN = none)
# gravity field about origin_km: l m C S sigma_C sigma_S (NaN: not in the set)
coef 1 0 -1.743147996446580e-03  0.000000000000000e+00 2.685350089254074e-03 0.000000000000000e+00
coef 1 1  3.171855274043025e-03 -2.036642465133304e-03 2.685350089254074e-03 2.685350089254074e-03
coef 2 0 -3.267843300100998e-02  0.000000000000000e+00 1.000000000000000e-04 0.000000000000000e+00
coef 2 1  3.580678708085953e-03  9.264930696445466e-04 1.000000000000000e-04 1.000000000000000e-04
coef 2 2  2.690908094312223e-03 -6.620608536237520e-03 1.000000000000000e-04 1.000000000000000e-04
coef 3 0  8.207508643558385e-04  0.000000000000000e+00 2.000000000000000e-04 0.000000000000000e+00
coef 3 1 -2.161190799036665e-03  1.345792464405454e-03 2.000000000000000e-04 2.000000000000000e-04
coef 3 2 -1.185030577784691e-04  1.023223669886617e-03 2.000000000000000e-04 2.000000000000000e-04
coef 3 3 -7.109585763745573e-04 -1.687530510730563e-03 2.000000000000000e-04 2.000000000000000e-04
coef 4 0  3.025957726300028e-03  0.000000000000000e+00 4.000000000000000e-04 0.000000000000000e+00
coef 4 1 -1.108795363729265e-03 -9.890871671108035e-04 4.000000000000000e-04 4.000000000000000e-04
coef 4 2  5.087506921525737e-04  1.560247138787098e-03 4.000000000000000e-04 4.000000000000000e-04
```

Inversion results (`sbtk.io.write_fit`): lines beginning with `#` describe the lines below them.

Output

```
# Didymos inversion of didymos_example (SBTK 1.0.0)
# chi-square, degrees of freedom, reduced chi-square, p-value (consistent at 5%: true)
chi2 25.9429   dof 24.000   chi2_reduced 1.0810   p_value 0.3561
n_measurements 27
regularization 0
moi_used Ixx Iyy Izz
origin_km 1.528148834993066e-02 4.288469572137803e-02 -3.405288560049095e-02
coef_about origin   com_shift_km 0.000000000 0.000000000 0.000000000   (predictions about origin_km + com_shift_km)
# density <component> <relative density> <its 1-sigma> <density> <its 1-sigma>   [kg/m^3]
#   (+ center x y z and their 1-sigma in km, if the center was estimated; + axis x y z and the
#    1-sigma of the rotations about the first, second and third axes in deg, if estimated)
density background 0 0 2666.936 129.322
density core 642.329 533.014 3309.265 405.126
density torus -414.298 108.921 2252.638 52.039
density regolith -1082.035 716.363 1584.901 588.859
# component <name> <type> <geometry>
component core sphere center [15.9 42.3 -34.0] m, radius 170.0 m
component torus torus ring 370.0 m, tube 55.0 m, ring center [15.9 42.3 -34.0] m, axis [0.0000 0.0000 1.0000]
component regolith layer 0.0 to 12.0 m below the surface
# residual <label> <measured> <predicted> <sigma> <(predicted - measured)/sigma>
residual C10 -1.743147996447e-03  8.859490892470e-05 2.685350e-03  0.682
residual C11  3.171855274043e-03  7.515844247840e-04 2.685350e-03 -0.901
```

True models (`sbtk.io.write_truth`) and **comparisons** (`sbtk.io.write_comparison`) use the same layout.

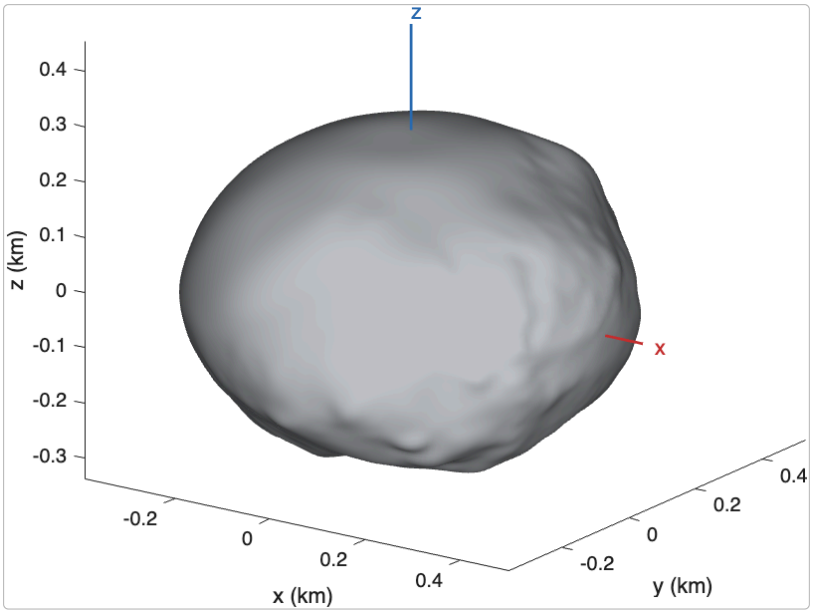
Figures

Every plot function returns a figure handle. `sbtk.io.save_figure(out, fig, name)` saves it as `figures/<name>.png` at 200 dpi in a run folder; `exportgraphics` saves it anywhere. Densities are drawn with a sequential color map (dark blue to yellow); signed differences with blue for lower and red for higher density.

The figures below come from one synthetic Didymos run, made by `documentation/tools/make_samples.m`: a core (+450 kg/m³) at the center of figure, an equatorial torus (−500) about the z-axis through it, a 12 m regolith layer (−900), a boulder of radius 40 m (+900) and a random field of 40 kg/m³ with a 60 m correlation length; measurements to degree 4 with the uncertainties of the first script; an inversion for core, torus and regolith (the boulder and the field are not in the inversion model). The fit has $\chi^2 / \nu = 1.08$ ($p = 0.36$), the homogeneous body $\chi^2 / \nu = 45.0$; $R^2 = 0.93$ with $R^2_{\max} = 0.98$.

Shape model

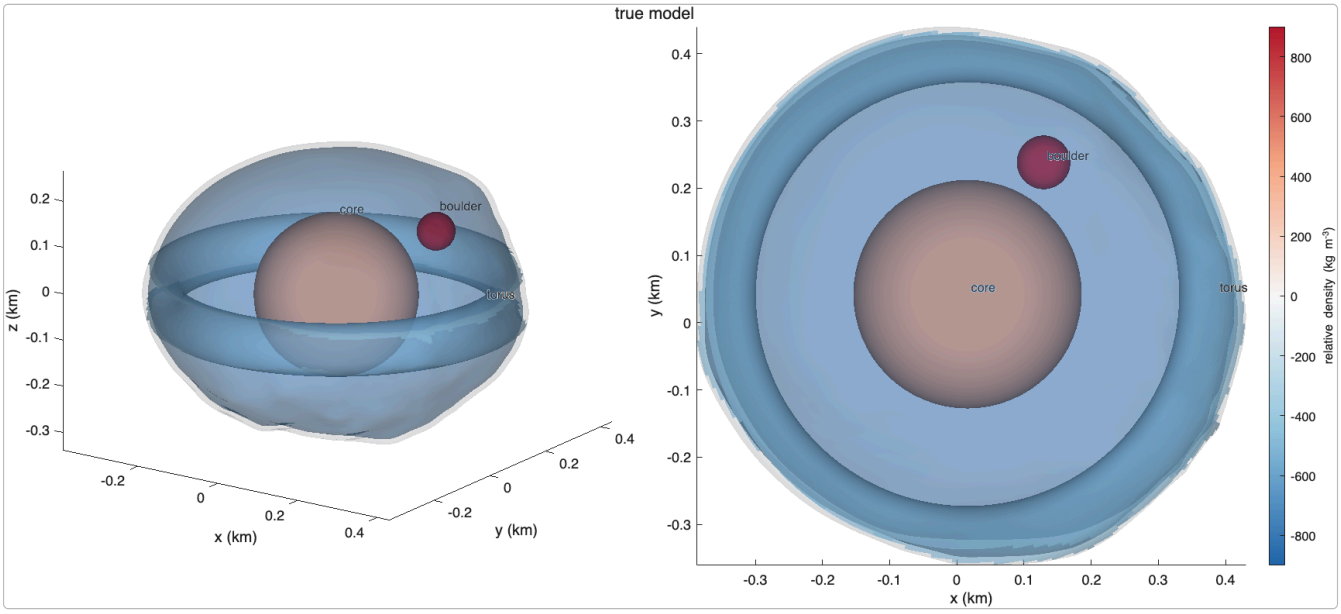
`sbtk.plot.shape(body)`: the mesh with its center of figure and axes.



Didymos shape model (DART v003, 9.3 m sampling).

Interior in three dimensions

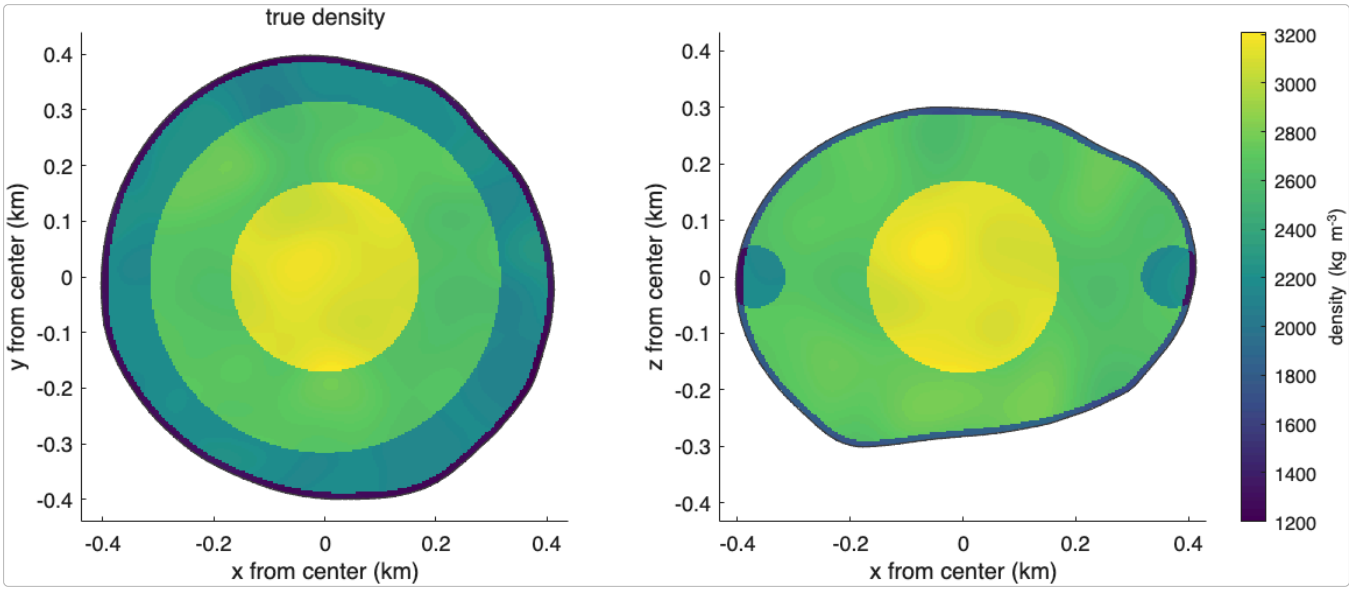
`sbtk.plot.interior3d(body, model, title)`: two views, oblique and down the z-axis. The surface is drawn translucent; spheres, ellipsoids and tori where they lie inside the body, with their names; layers by their lower boundary; sectors as solids. Color is the relative density; sectors and layers are the more opaque the larger their relative density. Saved as `truth_3d.png` and `estimate_3d.png` by the report functions.



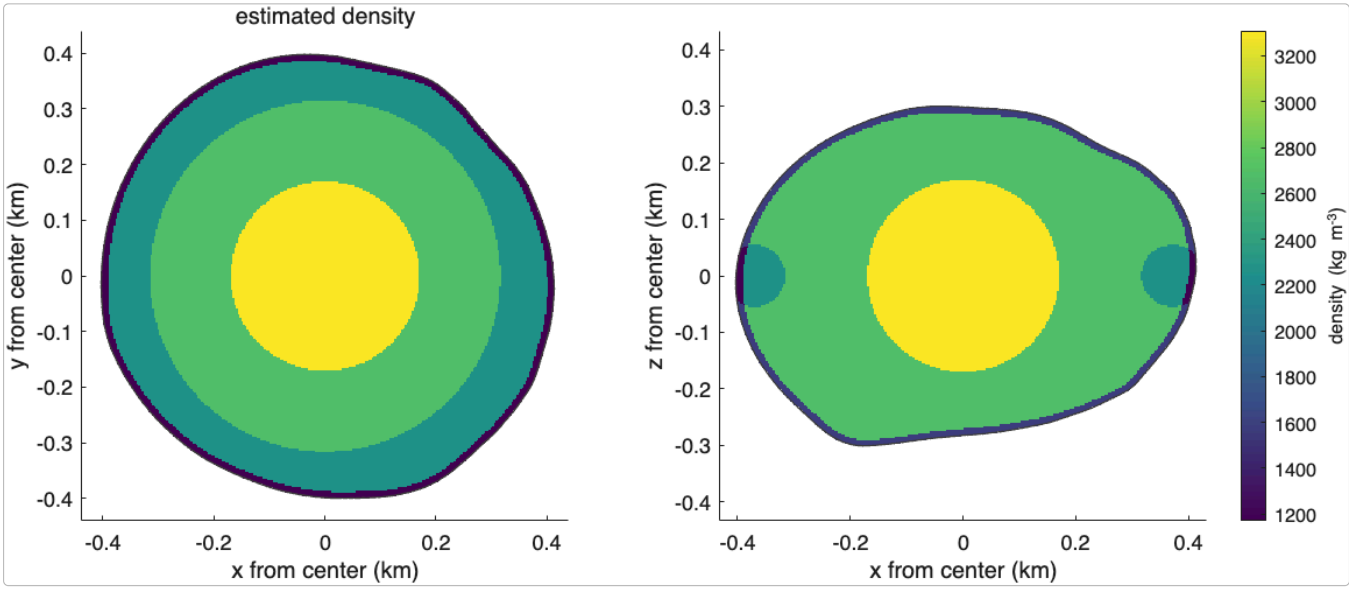
True model: core, torus, lower boundary of the regolith layer, boulder.

Density slices

`sbtk.plot.slices(body, {models}, {titles})`: the density on the plane $z = 0$ (left) and the plane $y = 0$ (right), both through the first model's origin, one row per model with a common color scale. Saved as `truth_slices.png` and `estimate_slices.png`.



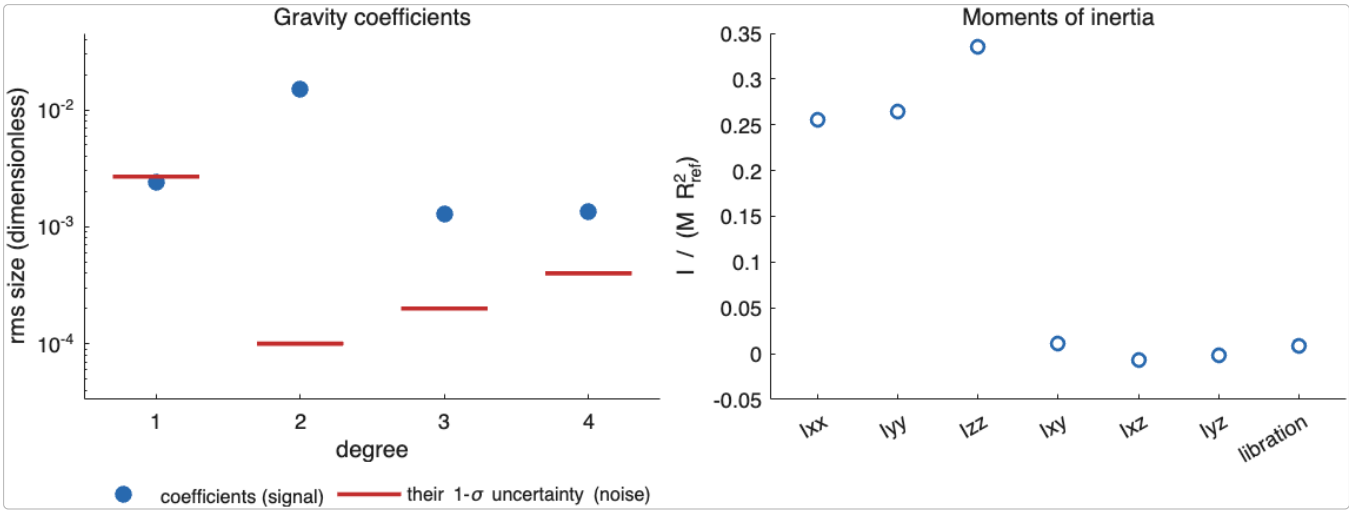
True density; the random field appears as small-scale variation.



Estimated density of the three-component model on the same planes.

Measurements

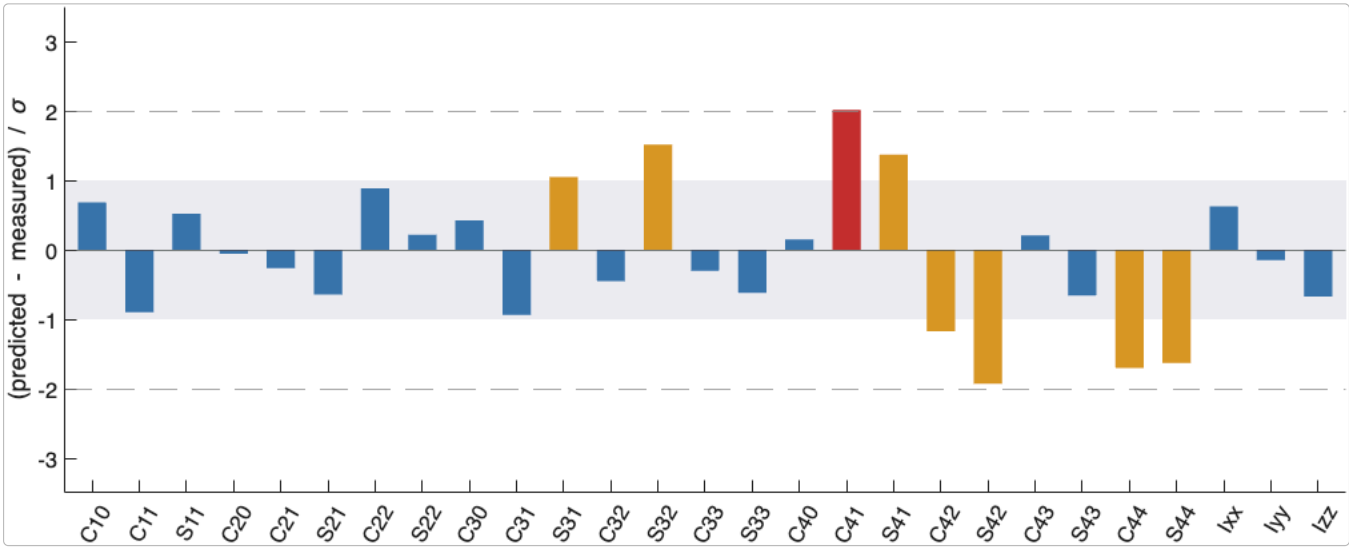
`sbtk.plot.measurements(obs)`: left, the root-mean-square size of the coefficients of each degree (points) and of their 1σ uncertainties (line); degrees where the points lie well above the line carry information about the interior. Degree 1 is close to zero about the center of mass. Right, each inertia measurement with its 1σ. Saved as `measurements.png`.



Coefficients of degrees 1 to 4 and the seven inertia rows of the example.

Normalized residuals

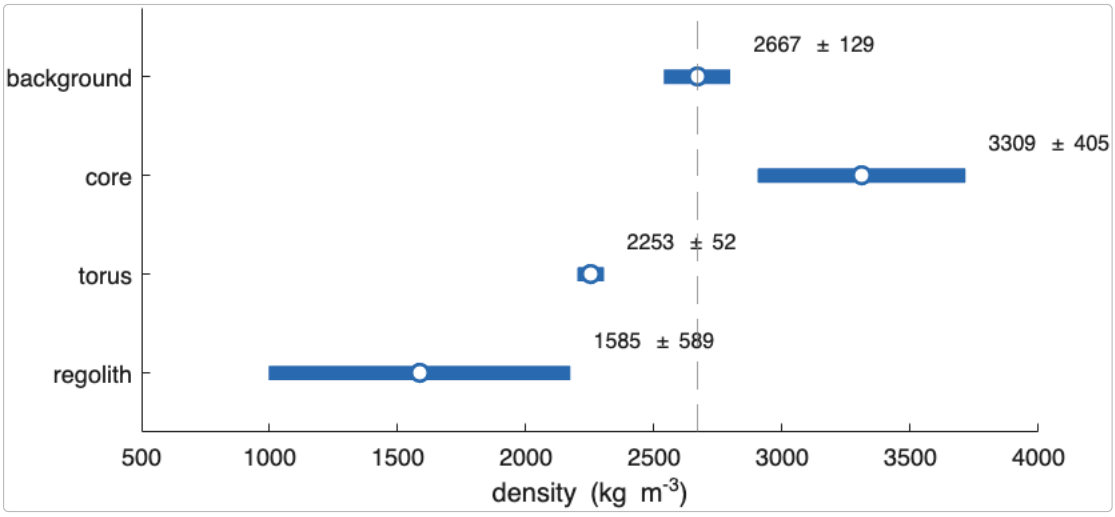
`sbtk.plot.residuals(fit)`: (predicted – measured)/σ for every measurement; gray band ±1σ, dashed lines ±2σ. Saved as `residuals.png`.



Residuals of the three-component fit.

Estimated densities

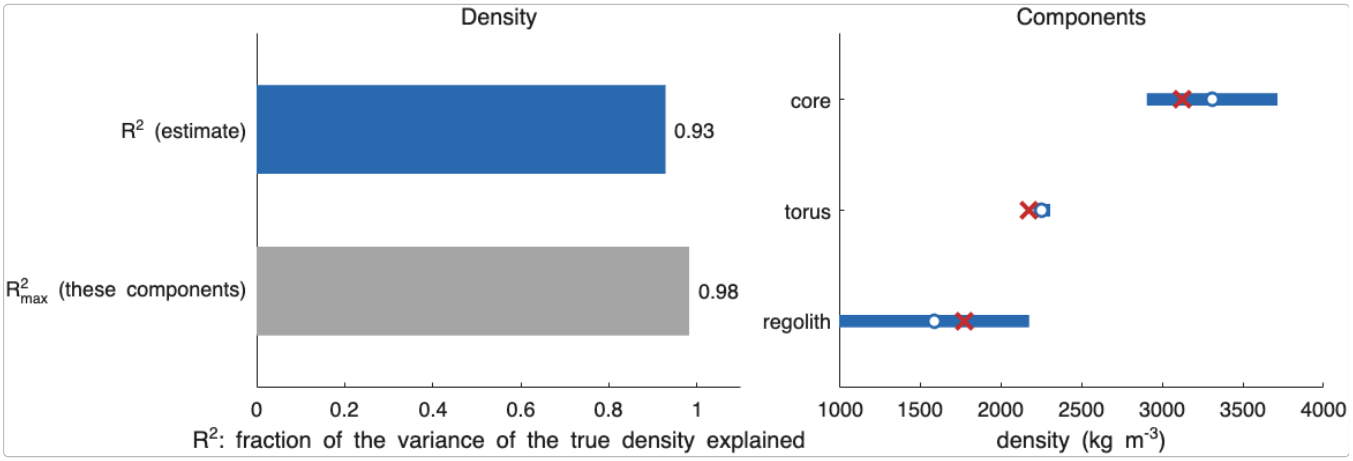
`sbtk.plot.densities(fit)`: the density of the background and each component with its ±1σ interval; the dashed line is the background. Components whose relative density is smaller than its 1σ are drawn in gray. Saved as `densities.png`.



Estimated densities of the example.

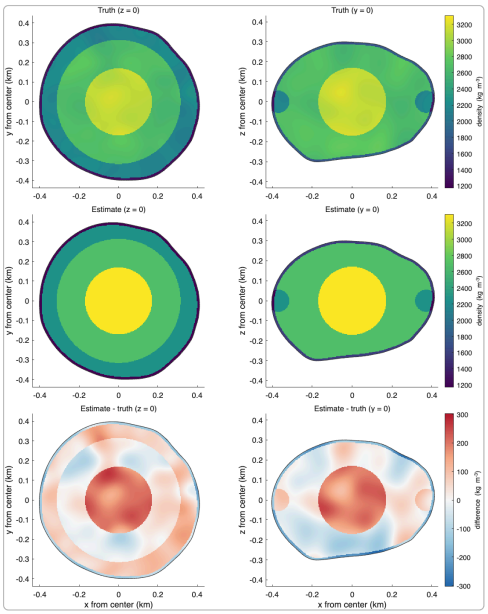
Comparison with the true model

`sbtk.plot.comparison(cmp)`: left, R^2 of the estimate and $R^2_{\max}$; right, each component's estimate with ±1σ and its true density (cross). Saved as `comparison.png`.



Comparison of the example fit with the true model.

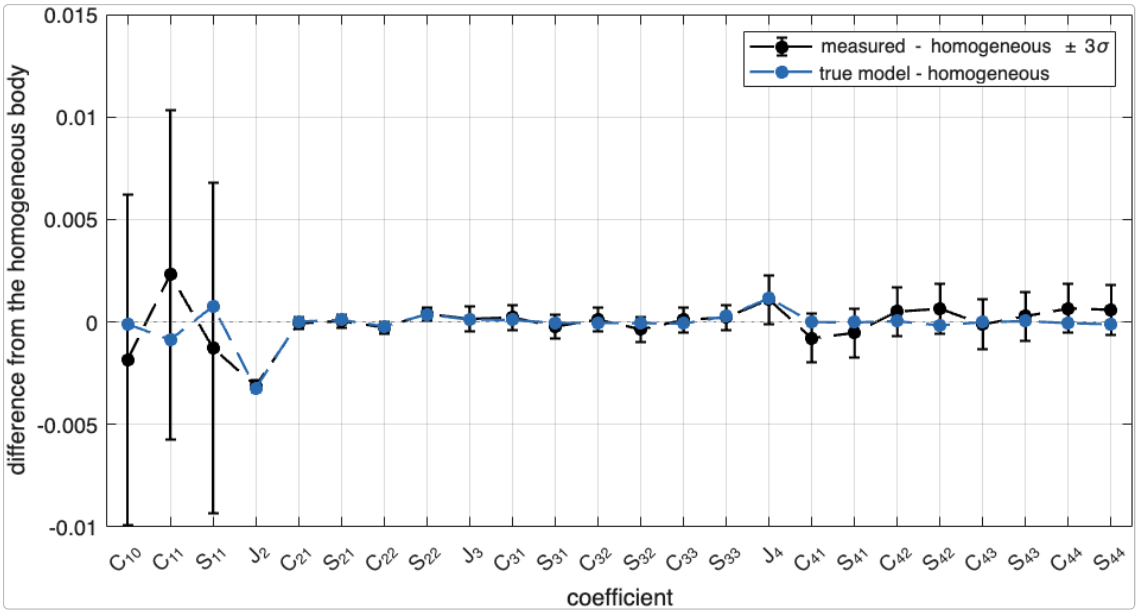
`sbtk.plot.truth_vs_estimate(body, truth, fit)`: rows for the true density, the estimate (same color scale) and their difference, on the two planes through the true center of mass. Saved as `truth_vs_estimate.png`.



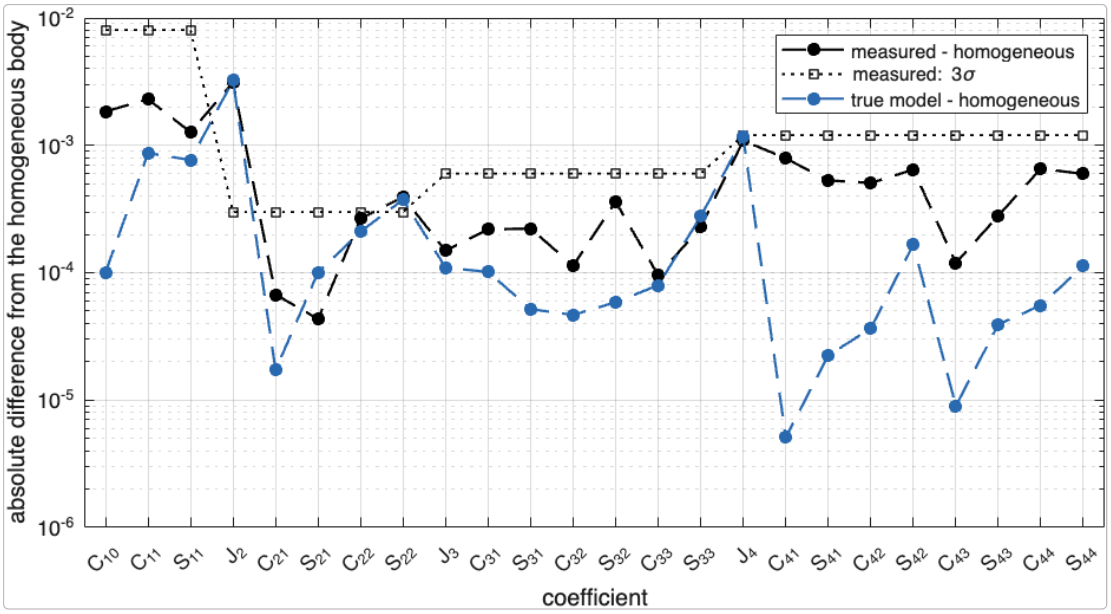
True density, estimate and difference.

Difference from the homogeneous body

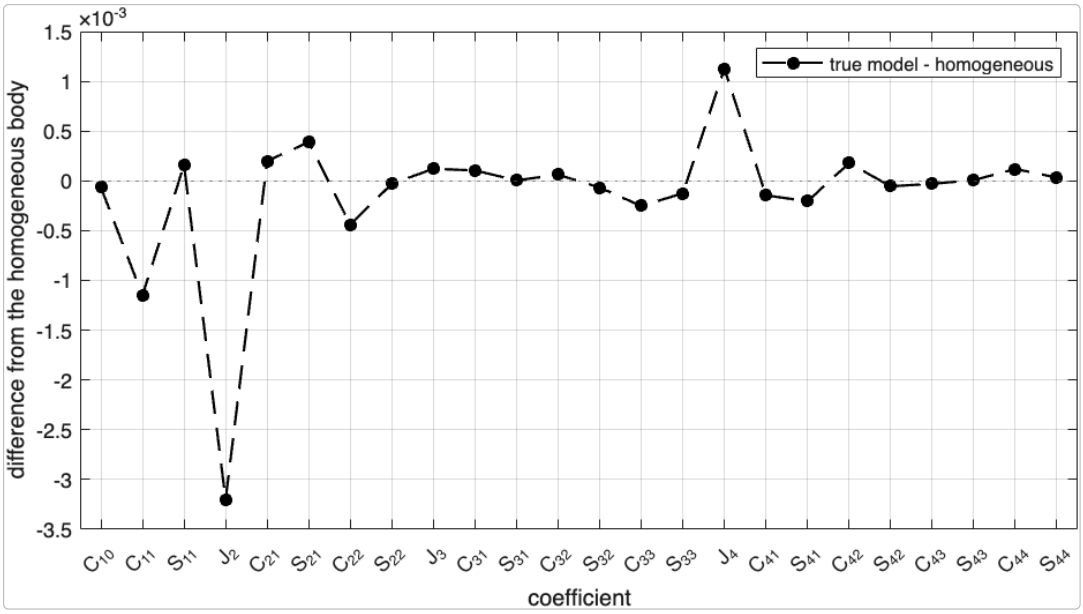
`sbtk.plot.difference_from_homogeneous(body, sets, names, scale, frame)`: for each set of gravity coefficients (a measurement set, a true model or an inversion result), the coefficients minus those of the homogeneous body (density M / V), both about their own center of mass, as radio-science fields are. Degree 1 shows the set's center of mass minus the center of figure. A measurement set about another point (for example a field about the shape-model origin) is first moved to its center of mass by the exact translation of solid harmonics. Zonal coefficients are shown as $J_l = -\bar{C}_{l0}$. With `scale = "linear"` the signed difference has ±3σ bars for sets with uncertainties; with `"log"` the absolute difference is drawn with 3σ as a separate line, which shows which departures from a uniform density the measurements resolve. With `frame = "shape"` both fields are in the shape-model axes; with `"spin"` in the spin axes of the set (z along its axis of the largest moment), where the set's $\bar{C}_{21}$, $\bar{S}_{21}$ vanish; with `"principal"` in its principal axes, where also $\bar{S}_{22}$ vanishes. The differences in the vanishing terms are the homogeneous body's own terms in those axes: they measure how far the principal axes of the interior are tilted and turned from those of the homogeneous body (Frames of a gravity field). Not saved by the report functions; save it with `sbtk.io.save_figure`.



Measured coefficients (±3σ) and those of the true model, minus the homogeneous body, in the shape-model axes.



The same on a logarithmic scale, with the 3σ level of the measurements.



The true model minus the homogeneous body in the principal axes of the true model.

Exchange packages

Exchange packages carry measurements, interior models and solutions between teams, for example in a blind test where one team simulates measurements of an interior it keeps hidden and other teams invert them. They are plain text with the units, frame and normalization written in them, and can be read without SBTK.

Package	Name	Written by	Contents	Released
Measurements	`<team>-DATA<n>-<comment>`	`sbtk.exchange.write_data`	Gravity field, covariance, center of mass, moments of inertia, shape model	with the data set
Interior model	`<team>-MOD<n>-<comment>`	`sbtk.exchange.write_model`	True model, density grid, noise-free measurements, figures	after the solutions
Solution	`<team>-SOL_<data team><n>`	`sbtk.exchange.write_solution`	Estimated densities, covariance, fitted measurements, density grid, figures	with the solution

`sbtk.exchange.solution_name` forms solution names; `sbtk.exchange.locate` finds the package of a given kind in a run folder and its `redacted/` and `unredacted/` subfolders.

File layout

Each file starts with metadata lines `# key: value`, followed by one line of column names and comma-separated rows. Every column and key with a unit ends with it; NaN marks a value that is not provided.

Reader	Statement
MATLAB	`T = readtable('gravity_field.txt', 'FileType', 'text', 'CommentStyle', '#');`
Python with pandas	`T = pandas.read_csv('gravity_field.txt', comment='#')`
Python, standard library	`rows = list(csv.DictReader(l for l in open('gravity_field.txt') if not l.startswith('#')))`
SBTK	`sbtk.exchange.read_data(folder)`, `sbtk.exchange.read_grid(file)`

Measurement package

File	Contents
`README.txt`	Summary (shape model, GM and mass, bulk density, degree and reference radius of the field; the uncertainties and the noise when the values have them) and the list of files.
`gravity_field.txt`	`l,m,C,S`, fully normalized, with `sigma_C,sigma_S` when the values have uncertainties: from degree 1 about the origin of the shape model file, or from degree 2 about the center of mass (Expansion point).
`gravity_field_unnormalized.txt`	The same coefficients unnormalized, $C_{lm} = N_{lm}\bar{C}_{lm}$, $S_{lm} = N_{lm}\bar{S}_{lm}$ (Gravity coefficients), and their 1σ when they have one.
`gravity_covariance.txt`	Full covariance of the normalized coefficients; rows and columns labeled. Only when the values have uncertainties.
`center_of_mass.txt`	Center of mass in the shape-model frame (with its 3 × 3 covariance when it has one) and the center of figure.
`moments_of_inertia.txt`	`quantity,value,unit` (and `sigma`): inertia elements as $I / (MR_{\text{ref}}^2)$ about the same point as the gravity field and/or the libration ratio. The products of inertia are tensor elements, $I_{xy} = -\int \rho\, xy\, dV$ (Moments of inertia); the definition of the libration ratio is given when the file has it.
`conventions.txt`	Units; the frame (what its origin and axes are, and any change of frame from the original shape-model file); the normalization, with MATLAB and Python code; degree 1 and the center of mass.
`<shape>.obj`	The shape model the data were made with, in km, in the frame of every position of the package (The shape model of a package). Not included when it is one of SBTK's shape model files as distributed (`data/shapes/`), such as the agreed HLP6 models: the files then name it.
`potential_acceleration.txt`	Optional (`sbtk.exchange.write_potential`): the gravitational potential and acceleration at given points, in SI units (Potential and acceleration at points).

The package contains nothing about the interior: not the noise seed, the model name or any component; `testNothingAboutTheInterior` checks this (Validation).

Expansion point

The gravity field and the moments of inertia are given about one of two points, chosen by the measurement set that `sbtk.exchange.write_data` writes:

Point	Measurement set	`gravity_field.txt`	`sbtk.exchange.read_data` returns
The origin of the shape model file (the HLP6 choice)	about `[0 0 0]`, e.g. from `sbtk.data.expand_about`	degree 1 and above; degree 1 gives the center of mass	the coefficients as given, `coef_about = 'origin'`, `origin_km = [0 0 0]`
The center of mass, as a radio-science solution	any other set	degree 2 and above; degree 1 is zero and not listed	degree ≥ 2 about the measured center of mass, degree 1 as 0 ± σ (the uncertainty of the center of mass), `coef_about = 'center_of_mass'`

For a set about the center of mass the coefficients are written as they are: `obs.origin_km` is taken to be the center of mass and the degree-1 terms its measured offset, as the simulations make them. A set about another point is first expanded about `[0 0 0]` with `sbtk.data.expand_about`.

`center_of_mass.txt` gives the center of mass and the center of figure in both, and `conventions.txt` states which point the package uses. The body is built from the package's shape model and GM; `sbtk.exchange.read_data` reads the shape model from the package, or from `data/shapes/` when the

package names a file it does not include. A libration ratio, which is defined about the center of mass, becomes the row $(B - A) - k\,I_{zz} = 0$ with σ = $\sigma_k I_{zz}$, using the measured I_{zz} if the package has one and that of a homogeneous body otherwise.

The files state only what applies to the package. Those of a measurement package with uncertainties and noise, made by `documentation/tools/make_samples.m`:

Output

```
DOC-DATA1-Didymos: measurements of Didymos
Prepared by Example team

SUMMARY
  shape model       didymos_g_9309mm_spc_obj_0000n00000_v003.obj
  GM                3.51935839e-08 km^3/s^2, mass 5.273000e+11 kg
  bulk density      2592.98 kg/m^3 (mass / shape model volume 0.203356 km^3)
  gravity field     degrees 2 to 4, reference radius 0.43 km, about the center of mass
  center of mass    see center_of_mass.txt file
  inertia           see moments_of_inertia.txt file
  frame             see conventions.txt file
  values            with 1-sigma uncertainties (covariance of the coefficients in gravity_covariance.txt) and Gaussian noise drawn from them

FILES
  gravity_field.txt
  gravity_field_unnormalized.txt
  gravity_covariance.txt
  center_of_mass.txt
  moments_of_inertia.txt
  conventions.txt
```

Output

```
# product: gravity field, spherical-harmonic coefficients
# data_set: DOC-DATA1-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
# expansion_point: the center of mass
# axes: shape model axes
# normalization: fully normalized (conventions.txt)
# reference_radius_km: 0.43
# GM_km3_s2: 3.51935839e-08
# mass_kg: 5.273000e+11
# max_degree: 4
# C00: 1
# uncertainties: 1-sigma; full covariance in gravity_covariance.txt
# noise: Gaussian, drawn from the 1-sigma uncertainties
l,m,C,S,sigma_C,sigma_S
2,0,-3.267843300100998e-02,0.000000000000000e+00,1.000000000000000e-04,0.000000000000000e+00
2,1,3.580678708085953e-03,9.264930696445466e-04,1.000000000000000e-04,1.000000000000000e-04
2,2,2.690908094312223e-03,-6.620608536237520e-03,1.000000000000000e-04,1.000000000000000e-04
3,0,8.207508643558385e-04,0.000000000000000e+00,2.000000000000000e-04,0.000000000000000e+00
3,1,-2.161190799036665e-03,1.345792464405454e-03,2.000000000000000e-04,2.000000000000000e-04
3,2,-1.185030577784691e-04,1.023223669886617e-03,2.000000000000000e-04,2.000000000000000e-04
3,3,-7.109585763745573e-04,-1.687530510730563e-03,2.000000000000000e-04,2.000000000000000e-04
4,0,3.025957726300028e-03,0.000000000000000e+00,4.000000000000000e-04,0.000000000000000e+00
4,1,-1.108795363729265e-03,-9.890871671108035e-04,4.000000000000000e-04,4.000000000000000e-04
```

Output

```
# product: center of mass and center of figure
# data_set: DOC-DATA1-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
# uncertainties: 1-sigma; 3 x 3 covariance in the cov columns
# noise: Gaussian, drawn from the 1-sigma uncertainties
axis,center_of_mass_km,sigma_km,center_of_figure_km,cov_x_km2,cov_y_km2,cov_z_km2
x,0.017643828580,0.002000000000,0.015929534123,4.000000000000001e-06,0.000000000000000e+00,0.000000000000000e+00
y,0.041367841384,0.002000000000,0.042313853148,0.000000000000000e+00,4.000000000000001e-06,0.000000000000000e+00
z,-0.035351150585,0.002000000000,-0.033978532721,0.000000000000000e+00,0.000000000000000e+00,4.000000000000001e-06
```

Output

```
# product: moments of inertia about the center of mass
# data_set: DOC-DATA1-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
# axes: shape model axes
# normalization: values are I / (M R_ref^2)
# reference_radius_km: 0.43
# mass_kg: 5.273000e+11
# libration_ratio: (B - A) / C, A <= B principal moments about axes in the xy-plane, C = Izz
# uncertainties: 1-sigma, uncorrelated
# noise: Gaussian, drawn from the 1-sigma uncertainties
quantity,value,sigma,unit
Ixx,2.553972158114155e-01,2.000000000000000e-03,M R_ref^2
Iyy,2.641173924187360e-01,2.000000000000000e-03,M R_ref^2
Izz,3.346618502230035e-01,2.000000000000000e-03,M R_ref^2
Ixy,1.137452222834483e-02,2.000000000000000e-03,M R_ref^2
Ixz,-7.467290512494866e-03,2.000000000000000e-03,M R_ref^2
Iyz,-1.855098124365382e-03,2.000000000000000e-03,M R_ref^2
libration_ratio,8.070726840715539e-02,2.000000000000000e-02,1
```

The shape model of a package

A package's shape model is the one the data were made with, in the frame of every position of the package. The HLP6 makers use the agreed shape model file as distributed, unchanged, in its own frame (Frames of a gravity field). Every team has that file, and it never changes, so a package made with it does not include it: `README.txt` and the header of every file name it. ex09 sends the original shape model moved and rotated into the new frame with `sbtk.body.to_frame`: every vertex becomes $x' = (x - c)A$, with c the center of mass of the true model and the columns of A the new axes, both in the frame of the original file. The shape itself does not change, only its coordinates.

Example	Shape model of the package	Origin	Axes
Didymos, spin axes	`TEAM-DATA1-Didymos_didymos_v003_com_spin_axes.obj`	center of mass	z the axis of the largest moment, x the DART x-axis projected onto the plane normal to z
Dimorphos, principal axes	`TEAM-DATA2-Dimorphos_dimorphos_v004_com_principal_axes.obj`	center of mass	principal axes: x the axis of the smallest and z of the largest moment

Each true model has its own center of mass and axes, so each package has its own shape model, and its file name starts with the name of the package: `<package>_<shape name>.obj` (`sbtk.internal.shape_file_name`). The interior-model package of the data set and a solution of it use the same name. A shape model taken unchanged from a file keeps the file's name.

The center of mass is then at the origin (to rounding), and the center of figure at $-(r_{\mathrm{CoM}} - r_{\mathrm{CoF}})A$. The gravity field, the moments of inertia and `center_of_mass.txt`, and in the interior-model package every component and the density grid, are in this frame, so a receiving team uses the shape model as it is, with nothing to move or rotate. The FRAME section of `conventions.txt` states the origin and the axes, and the change of frame from the original file ($o = c$ and A), so the coordinates of the original file are $x = x'A^T + c$. The shape model records this change as `shape.frame_origin_km` and `shape.frame_axes`, combined over successive changes of frame. The FRAME section of such a package, made by `documentation/tools/make_samples.m`:

Output

```
FRAME
  All positions, the gravity field and the moments of inertia of this package are given
  in the frame of its shape model file, DOC-DATA3-Didymos_didymos_v003_com_spin_axes.obj:
    origin  the center of mass of the body
    z       the axis of the largest moment of inertia
    x       the x-axis of the original file projected onto the plane normal to z
    y       completes the right-handed frame (y = z cross x)
  The shape model file was made by moving and rotating the original file,
    didymos_g_9309mm_spc_obj_0000n00000_v003.obj
  into this frame:
    x' = (x - o) A
    o = [0.015281496 0.042884748 -0.034053123] km  (the center of mass, in the original file)
    A = [  0.997920540676  0.000000000000 -0.064456143974
          -0.001688015795  0.999657019950 -0.026134136041
           0.064434036803  0.026188594157  0.997578273840 ]
  where x and x' are the coordinates of a point (row vectors, km) in the original file and
  in this one, and o and the columns of A are the origin and the axes of this frame in the
  original file.
```

Potential and acceleration at points

`sbtk.exchange.write_potential` adds `potential_acceleration.txt` to a package: one row per point with its number `point`, its position `x_m,y_m,z_m` in the frame of the shape model file, the potential `U_m2_s2` and the acceleration `ax_m_s2,ay_m_s2,az_m_s2`, in SI units as the HLP6 makers' cross-validation asks. The potential is positive, $U = G \int \rho / \mid r - r' \mid dV$, the acceleration is its gradient, and the rotation of the body is not included. The header states how the values were computed (`sbtk.gravity.field_at`): exactly for a homogeneous body and for spheres entirely inside the body, and with the other departures from the background density on a grid of cubes (Potential and acceleration at points). The function works for any model; the HLP6 makers give it for the homogeneous Didymos (`examples/make_didymos_homogeneous.m`).

Packages of exact values

A set without any uncertainties, such as the exact values of a true model (`sbtk.data.simulate`), is written as a package of exact values:

- `gravity_field.txt` has the columns `l,m,C,S`, `center_of_mass.txt` the columns `axis,center_of_mass_km,center_of_figure_km`, and `moments_of_inertia.txt` the columns `quantity,value,unit`;
- there is no `gravity_covariance.txt`;
- no file has an `uncertainties` or `noise` line.

A maker's package of exact values in the frame and about the origin of the shape model file, as the HLP6 makers send them, made by `documentation/tools/make_samples.m`:

Output

```
DOC-DATA2-Didymos: measurements of Didymos
Prepared by Example team

SUMMARY
  shape model       didymos_g_9309mm_spc_obj_0000n00000_v003.obj
  GM                3.51935839e-08 km^3/s^2, mass 5.273000e+11 kg
  bulk density      2592.98 kg/m^3 (mass / shape model volume 0.203356 km^3)
  gravity field     degrees 1 to 4, reference radius 0.43 km, about the origin of the shape model file
  center of mass    see center_of_mass.txt file
  inertia           see moments_of_inertia.txt file
  frame             see conventions.txt file

FILES
  gravity_field.txt
  gravity_field_unnormalized.txt
  center_of_mass.txt
  moments_of_inertia.txt
  conventions.txt
```

Output

```
# product: center of mass and center of figure
# data_set: DOC-DATA2-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
axis,center_of_mass_km,center_of_figure_km
x,0.015281496496,0.015929534123
y,0.042884747979,0.042313853148
z,-0.034053123055,-0.033978532721
```

In a blind test the makers send such packages, and each receiving team applies uncertainties of its own choosing. `sbtk.exchange.read_data` reads them with NaN uncertainties. The inversion then stops with `sbtk:data:noUncertainties` until uncertainties are given, for example with `sbtk.data.use_uncertainties` from a precision file and `sbtk.data.set_moi_sigma`, after truncating with `sbtk.data.select_degree` to the degrees the precision covers. A set with some uncertainties but not all is not written (`sbtk:data:noUncertainties`).

Interior-model package

File	Contents
`README.txt`	Summary (mass, bulk and background density, number of components, random field, density grid) and the list of files.
`interior_model.txt`	`name,type,relative_density_kgm3,density_kgm3,geometry` for the background and every component; metadata: mass, bulk and background density, the point layers and sectors are measured from, the random field, the principal axes of inertia (unit vectors in the shape-model axes) and the principal moments $A \le B \le C$ (Frames of a gravity field).
`density_grid.txt`	`x_km,y_km,z_km,density_kgm3,density_sigma_kgm3`: the true density at the cell centers (σ = 0), random field included.
`noise_free_data/`	The measurement files without noise.
`figures/`	`true_density_3d.png`, `true_density_slices.png`.

Output

```
# product: true interior model
# model: DOC-MOD1-Didymos
# data_set: DOC-DATA1-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
# frame: shape model frame
# mass_kg: 5.273000e+11
# bulk_density_kgm3: 2592.984
# background_density_kgm3: 2672.002
# center_of_mass_km: 0.015281 0.042885 -0.034053
# layers_and_sectors_from_km: 0.015281 0.042885 -0.034053
# overlaps: relative densities add; components are clipped to the shape model
# principal_axes: x' = [0.835027009 -0.548783743 0.039576476], y' = [0.546420442 0.835555749 0.057195193], z' = [-0.064456144 -0.026134136 0.997578274] (x'
smallest, z' largest moment)
# principal_moments: A B C = 0.253270820 0.271737242 0.335835620 (I / (M R_ref^2))
# random_field: Gaussian, standard deviation 40.0 kg/m^3, correlation exp(-d^2/(2 L^2)) with L = 0.0600 km, seed 7
# relative_density: density minus background density
name,type,relative_density_kgm3,density_kgm3,geometry
background,background,0.000,2672.002,"everything not in a component"
core,sphere,450.000,3122.002,"center [15.9 42.3 -34.0] m; radius 170.0 m"
torus,torus,-500.000,2172.002,"ring 370.0 m; tube 55.0 m; ring center [15.9 42.3 -34.0] m; axis [0.0000 0.0000 1.0000]"
regolith,layer,-900.000,1772.002,"0.0 to 12.0 m below the surface"
boulder,sphere,900.000,3572.002,"center [129.2 238.5 71.7] m; radius 40.0 m"
```

Solution package

File	Contents
`README.txt`	Method, the χ^2 test of the fit and of a homogeneous body, the measurements used.
`solution.txt`	Background and each component: density and relative density with 1σ, estimated or prescribed, geometry.
`solution_covariance.txt`	Covariance of the estimated parameters.
`fitted_measurements.txt`	`label,measured,predicted,sigma,normalized_residual`; the degree-1 rows give the center of mass, the other coefficients are about the point the measurements are about (the measured center of mass, or the origin of the shape model file).
`density_grid.txt`	Estimated density and its 1σ at the cell centers, propagated from the parameter covariance.
`figures/`	Residuals, densities, 3-D view, slices.

Output

```
# product: estimated interior
# solution: DOC-SOL_DOC1
# data_set: DOC-DATA1-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
# fit: chi2 24.69, degrees of freedom 24.0, reduced chi2 1.03, p-value 0.423 (consistent with the measurements at the 5% level)
# homogeneous_body: reduced chi2 44.86, p-value 3.61e-238
# measurements_used: C10 C11 S11 C20 C21 S21 C22 S22 C30 C31 S31 C32 S32 C33 S33 C40 C41 S41 C42 S42 C43 S43 C44 S44 Ixx Iyy Izz
# regularization: 0 (Tikhonov parameter)
# layers_and_sectors_from_km: 0.017644 0.041368 -0.035351
# relative_density: density minus background density
# uncertainties: 1-sigma; covariance in solution_covariance.txt
name,type,density_kgm3,density_sigma_kgm3,relative_density_kgm3,relative_density_sigma_kgm3,density,geometry
background,background,2665.863,129.767,0.000,0.000,estimated (total mass as a constraint),"everything not in a component"
core,sphere,3310.739,406.372,644.877,534.712,estimated,"center [15.9 42.3 -34.0] m; radius 170.0 m"
torus,torus,2252.418,52.225,-413.445,109.017,estimated,"ring 370.0 m; tube 55.0 m; ring center [15.9 42.3 -34.0] m; axis [0.0000 0.0000 1.0000]"
regolith,layer,1591.601,591.154,-1074.262,719.114,estimated,"0.0 to 12.0 m below the surface"
```

Density grid

Cell centers lie at integer multiples of `spacing_km` along each axis of the shape-model frame, so grids of the same spacing coincide regardless of who made them. A cell is listed when its center is inside the shape model, tested along rays from the center of figure, so all models of a body have the same cells. Each cell stands for the volume `spacing_km`^{3} and carries the density at its center. `sbtk.compare.grids` compares two grids (Comparison of density grids); the comparison of the example solution with its model:

Output

```
density grids: 3179 common cells (3179 true, 3179 estimated)
  R2 = 0.922 (fraction of the variance of the true density explained)
  RMS error 98.3 kg/m^3, mean error +1.2 kg/m^3, standard deviation of the true density 351.3 kg/m^3
  97.9% of the common cells have abs(estimate - true) <= 2 sigma of the estimate
```

conventions.txt

Every measurement package includes this file, written by `sbtk.internal.exchange_conventions`; the tests check the normalization code it prints. That of the maker's package above, in the frame and about the origin of the shape model file:

Output

```
UNITS
  Lengths and positions in km, mass in kg, time in s, GM in km^3/s^2, densities in kg/m^3,
  angles in degrees.

FRAME
  All positions, the gravity field and the moments of inertia of this package are given
  in the frame of its shape model file, didymos_g_9309mm_spc_obj_0000n00000_v003.obj:
    origin  the origin of the shape model file (different from the CoF and CoM)
    axes    those of the shape model file as distributed

GRAVITY FIELD COEFFICIENT NORMALIZATION
  The files give fully normalized coefficients:
    Cbar_lm = C_lm / N_lm,   Sbar_lm = S_lm / N_lm,   Pbar_lm = N_lm P_lm,
    N_lm = sqrt( (2 - delta_m0) (2l + 1) (l - m)! / (l + m)! )
  where P_lm are the associated Legendre functions without the factor (-1)^m. For example,
    N_20 = sqrt(5), N_21 = sqrt(5/3), N_22 = sqrt(5/12), N_30 = sqrt(7).
  Zonal coefficients: J_l = -C_l0 = -sqrt(2l + 1) Cbar_l0.
  gravity_field_unnormalized.txt gives the same coefficients unnormalized:
    C_lm = N_lm Cbar_lm,   S_lm = N_lm Sbar_lm.

  MATLAB
    N = sqrt((2 - (m == 0)) .* (2*l + 1) .* factorial(l - m) ./ factorial(l + m));
  Python
    N = math.sqrt((2 - (m == 0)) * (2*l + 1) * math.factorial(l - m) / math.factorial(l + m))

DEGREE 1 AND THE CENTER OF MASS
  The gravity field is expanded about the origin of the shape model file. Its degree-1
  coefficients give the center of mass (xc, yc, zc) relative to that origin. The moments
  of inertia are about the same origin.
```

Other formats

Format	Function	Notes
PDS SHADR layout	`sbtk.exchange.read_shadr`	Header row then `l, m, C, S, sigma_C, sigma_S`; commas or spaces; `#` comments. The caller gives the shape model and the point the coefficients are about.
JPL SSD gravity file	`sbtk.io.read_jpl_gravity`	Full covariance; J_l converted.
Coefficient matrices	`sbtk.data.from_coefficients`	
Uncertainties of another data set	`sbtk.data.use_uncertainties`	Converted to the target reference radius by $(R / R')^l$.

Density models by component type

One example per component type and model feature. Each example after the first is a fragment: it uses the body and uncertainties defined in the setup script below, which must be run first. Building a true model and its exact values (`sbtk.data.simulate`) needs no uncertainties; only the examples that make measurements and estimate densities use `unc`. Geometry definitions: Density distributions.

Setup

MATLAB script: Body and uncertainties used by the examples on this page

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);    % mass [kg], R_ref [km]
unc  = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);   % for measurements
```

Sphere

A sphere is given by its center (shape-model frame, km), radius and relative density. A sphere entirely inside the body is integrated as a point mass. `sbtk.interior.latlon` gives a position by latitude, longitude and distance; added to the center of figure, as here, it is measured from there (Where to place components).

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
core    = sbtk.interior.sphere("core", body.centroid_km, 0.170, +450);
boulder = sbtk.interior.sphere("boulder", body.centroid_km + sbtk.interior.latlon(25, 60, 0.250), 0.040, +900);
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create([core, boulder]), 4, "spheres");
fprintf("center of mass - center of figure: [%.1f %.1f %.1f] m\n", 1e3*exact.com_minus_cof_km)
```

Ellipsoid

Semi-axes along x, y, z before the rotation by yaw (about z), pitch (about y) and roll (about x), in degrees.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
lens = sbtk.interior.ellipsoid("lens", body.centroid_km + [0.05 0.10 0.12], [0.20 0.08 0.05], [40 -30 15], +300);
disp(lens.axes)                                  % columns: the ellipsoid axes in the shape-model frame
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(lens), 4, "ellipsoid");
sbtk.plot.slices(body, {truth}, {'ellipsoid'});
```

Torus

A torus about the z-axis: ring radius, tube radius, height of its plane. Only the part inside the body counts, so a torus that crosses the surface at the equator represents a belt of different density along the equator. Set its `center_km` to place it about the center of figure, and align it with the spin axis with `sbtk.interior.align_with_spin_axis` (Axes of components).

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
torus = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
torus.center_km = body.centroid_km;                          % about the z-axis through the center of figure
geo = sbtk.gravity.geometry(body, body.centroid_km, 0);
I = sbtk.gravity.component_integrals(geo, torus);
fprintf("torus volume inside the body: %.5f km^3 (%.1f%% of the body)\n", I.V, 100*I.V/body.volume_km3)
```

Layer

Top and bottom depth below the surface, along rays from the expansion origin.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
regolith = sbtk.interior.layer("regolith", 0, 0.012, -900);          % the outermost 12 m
mantle   = sbtk.interior.layer("mantle", 0.012, 0.080, -200);        % from 12 m to 80 m deep
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create([regolith, mantle]), 4, "layers");
fprintf("background density %.0f kg/m^3\n", truth.rho_bg_kgm3)
```

Sector

Radial fractions of the distance from the expansion origin to the surface, a latitude range and a longitude range (degrees). Longitude ranges may extend beyond ±180°.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
block = sbtk.interior.sector("block", [0.3 0.9], [40 90], [150 240], +200);    % northern block across 180 deg
shell = sbtk.interior.sector("shell", [0.5 0.75], [-90 90], [-180 180], -300); % a shell that follows the shape
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create([block, shell]), 4, "sectors");
sbtk.plot.slices(body, {truth}, {'sectors'});
```

Sector grid

A grid of sectors that fills the body is a model without assumed structure. As an estimation model it needs the background held at the bulk density and, with many sectors, regularization. Here the true interior is a dense core and the estimate uses 36 sectors.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(sbtk.interior.sphere("core", body.centroid_km, 0.17, +450)), ...
                                    4, "core");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);       % measurements of it
sectors = sbtk.interior.sector_grid([0 0.5 0.85 1], [-90 -30 30 90], [-180 -90 0 90 180]);   % 3 x 3 x 4
prob = sbtk.invert.setup(body, sectors, obs, sbtk.invert.options(background="bulk"));
prob.opts.regularization = sbtk.invert.choose_regularization(prob, "discrepancy");
fit = sbtk.invert.run(prob);
fprintf("lambda %.3g, reduced chi2 %.2f, dof %.1f\n", fit.regularization, fit.chi2_reduced, fit.dof)
sbtk.plot.slices(body, {truth, fit}, {'true density', 'sector estimate'});
```

Random density field

Standard deviation (kg/m³), correlation length (km) and seed. The field is added to the components; it is used in simulations and is not estimated by the inversion.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
field = sbtk.interior.random_field(40, 0.06, 7);                    % sigma 40 kg/m^3, correlation length 60 m
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(sbtk.interior.no_components(), field), 4, "field");
grid = sbtk.exchange.density_grid(body, truth, 0.020);              % density at 20 m cell centers inside the body
fprintf("%d cells: mean %.0f, standard deviation %.1f kg/m^3\n", numel(grid.density_kgm3), ...
        mean(grid.density_kgm3), std(grid.density_kgm3))
```

Homogeneous body

`sbtk.interior.no_components` without a random field gives a uniform density M / V.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(sbtk.interior.no_components()), 4, "uniform");
fprintf("background %.1f kg/m^3, bulk density %.1f kg/m^3\n", truth.rho_bg_kgm3, body.bulk_density_kgm3)
```

Prescribed component

A component with `estimate_density = false` keeps its relative density in the inversion; here a regolith layer of known density while the core is estimated.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
truth_components = [sbtk.interior.sphere("core", body.centroid_km, 0.17, +450), sbtk.interior.layer("regolith", 0, 0.012, -900)];
exact = sbtk.data.simulate(body, sbtk.interior.create(truth_components), 4, "prescribed");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
model = truth_components;
model(1).drho_kgm3 = 0;                  % estimated: the value is not used
model(2).estimate_density = false;       % prescribed: -900 kg/m^3 is kept
fit = sbtk.invert.solve(body, model, obs);
sbtk.invert.summary(fit)
```

Estimated sphere position

The center of a sphere with `estimate_position = true` is estimated with its density; `position_sigma_km` adds an a priori constraint about the starting position. The center, height and orientation of tori and ellipsoids can be estimated the same way (Estimated geometry).

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
truth_components = sbtk.interior.sphere("core", body.centroid_km + [0.03 -0.02 0.01], 0.17, +500);
exact = sbtk.data.simulate(body, sbtk.interior.create(truth_components), 4, "position");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
model = sbtk.interior.sphere("core", body.centroid_km, 0.17, 0);       % started at the center of figure
model.estimate_position = true;
fit = sbtk.invert.solve(body, model, obs, sbtk.invert.options(position_sigma_km=0.050));
fprintf("center [%.1f %.1f %.1f] m from the center of figure +- [%.1f %.1f %.1f] m after %d iterations\n", ...
        1e3*(fit.components(1).center_km - body.centroid_km), 1e3*fit.center_sigma_km(1, :), fit.iterations)
```

Calculations

Short examples of individual calculations. Fragments on this page use the variables of the setup script below.

Setup

MATLAB script: Body, interior and measurements used by the examples on this page

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
components = [sbtk.interior.sphere("core", body.centroid_km, 0.170, +450), sbtk.interior.layer("regolith", 0, 0.012, -900)];
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(components), 4, "calculations");   % the true model
unc  = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
exact = sbtk.data.set_uncertainties(exact, unc);                    % exact values with uncertainties
obs = sbtk.data.add_noise(exact, 1);                                % one noisy realization
```

Unit conversions

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
Izz = sbtk.units.moi_to_norm(1.6e16, body.mass_kg, body.R_ref_km)    % kg m^2 -> I / (M R_ref^2)
I_kgm2 = sbtk.units.norm_to_moi(Izz, body.mass_kg, body.R_ref_km)    % and back
c = sbtk.units.offset_to_degree1(0.002, body.R_ref_km)               % 2 m center-of-mass offset -> degree 1
f = sbtk.units.length_scale("m")                                     % m -> km: 1e-3
```

Volume and moments of one component

`sbtk.gravity.component_integrals` returns the integrals of the part of a component inside the body, per unit density, about the origin of the geometry.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
geo = sbtk.gravity.geometry(body, truth.origin_km, 4);
I = sbtk.gravity.component_integrals(geo, components(2));           % the regolith layer
fprintf("regolith volume %.5f km^3 = %.1f%% of the body\n", I.V, 100*I.V/geo.V_body)
mass_deficit_kg = components(2).drho_kgm3*I.V*1e9                  % its mass relative to the background
```

Coefficients and inertia about another origin

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
geo0 = sbtk.gravity.geometry(body, [0 0 0], 4);                     % about the shape-model origin
fm = sbtk.gravity.forward(geo0, sbtk.interior.create(components), body.mass_kg);
lab = sbtk.gravity.labels(4);
fprintf("C11 about the shape origin %.4e; about the center of mass %.1e\n", ...
        fm.coef(strcmp(lab, "C11")), truth.coef(strcmp(lab, "C11")))
```

Difference from the homogeneous body

The gravity coefficients of the true model and of the measurements, minus those of a body of uniform density, with the 3σ of the measurements (Figures). The second output holds the plotted values; `axes_deg` gives the angles between the principal axes of each set and of the homogeneous body.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
[fig, d] = sbtk.plot.difference_from_homogeneous(body, {obs, truth}, ["measured" "true model"], "linear", "shape");
fprintf("%s: %+.2e\n", [d(2).labels(1:4); string(d(2).difference(1:4))])   % C10, C11, S11, J2 of the true model
sbtk.plot.difference_from_homogeneous(body, {obs, truth}, ["measured" "true model"], "log", "shape");
[~, dp] = sbtk.plot.difference_from_homogeneous(body, {truth}, "true model", "linear", "principal");
fprintf("principal axes of the true model vs the homogeneous body: tilt %.2f deg, turn %.2f deg\n", dp.axes_deg)
```

Field about the center of mass, in spin and principal axes

`sbtk.gravity.center_of_mass_frame` moves a field to its center of mass (degree 1 becomes 0) and, with `"spin"`, turns its axis of the largest moment to z ($\bar{C}_{21} = \bar{S}_{21} = 0$) or, with `"principal"`, rotates it into its principal axes of inertia ($\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0$) (Frames of a gravity field).

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
cs = sbtk.gravity.center_of_mass_frame(truth, body, "shape");
cz = sbtk.gravity.center_of_mass_frame(truth, body, "spin");
cp = sbtk.gravity.center_of_mass_frame(truth, body, "principal");
disp(table(cs.labels(1:8)', cs.coef(1:8)', cz.coef(1:8)', cp.coef(1:8)', 'VariableNames', ...
           ["coefficient" "shape axes" "spin axes" "principal axes"]))
fprintf("axis of the largest moment %.2f deg from the shape-model z-axis\n", acosd(cp.principal(3, 3)))
mo = sbtk.gravity.center_of_mass_frame(obs, body, "principal");      % a measurement set: with its covariance
fprintf("C22 in principal axes %.4e +- %.1e\n", mo.coef(mo.labels == "C22"), mo.sigma(mo.labels == "C22"))
```

A body and its true model in another frame

`sbtk.body.to_frame` and `sbtk.interior.to_frame` carry the shape model and the true model into the frame with the center of mass as origin and the spin axes; the density grid is then on cells of that frame (Frames of a gravity field).

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
spin = sbtk.gravity.center_of_mass_frame(truth, body, "spin");
body_s  = sbtk.body.to_frame(body, spin.com_km, spin.axes, "didymos_v003_com_spin_axes");
truth_s = sbtk.interior.to_frame(truth, spin.com_km, spin.axes);
fprintf("center of mass %s km in the shape-model file, %s km in the new frame\n", mat2str(truth.com_km, 4), ...
        mat2str(truth_s.com_km, 4))
fprintf("center of figure %s km in the new frame\n", mat2str(body_s.centroid_km, 4))
grid_s = sbtk.exchange.density_grid(body_s, truth_s, 0.040);
fprintf("%d cells of 40 m in the spin axes\n", numel(grid_s.density_kgm3))
```

Density at points and on a grid

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
geo = sbtk.gravity.geometry(body, truth.origin_km, 0);              % geometry in the model's frame
x = linspace(-0.3, 0.3, 61)';
rho = sbtk.interior.density_at(truth, truth.origin_km + [x, 0*x, 0*x], geo);
plot(1e3*x, rho); xlabel("x from the center of mass [m]"); ylabel("density [kg/m^3]");

grid = sbtk.exchange.density_grid(body, truth, 0.040);              % 40 m cells
fprintf("%d cells, mean density %.0f kg/m^3\n", numel(grid.density_kgm3), mean(grid.density_kgm3))
```

Uncertainties from another data set

`sbtk.data.use_uncertainties` gives simulated coefficients the covariance of a reference data set, converted to the target reference radius. Here the reference has $R_{\mathrm{ref}} = 0.40$ km.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
L = 4;
[C, S] = deal(zeros(L + 1));  C(1, 1) = 1;
sig = 5e-5*ones(L + 1);
reference = sbtk.data.from_coefficients(sbtk.body.from_mass("Didymos", body.shape, body.mass_kg, 0.40), ...
                                        "precision", [0 0 0], C, S, sig, sig);
noisy = sbtk.data.add_noise(sbtk.data.use_uncertainties(exact, reference), 2);
fprintf("1-sigma of C20: %.3e (reference %.1e at 0.40 km)\n", sqrt(noisy.coef_cov(4, 4)), 5e-5)
```

Inertia measurements

`sbtk.data.select_moi` chooses the inertia rows of a data set; `opts.moi_labels` the rows a fit uses. `sbtk.data.libration_ratio_from_amplitude` converts a libration amplitude into a libration ratio.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
gravity_only = sbtk.data.select_moi(obs, {});
with_izz = sbtk.data.add_moi(gravity_only, "Izz", exact.moi(3).value, 2e-3);
with_lib = sbtk.data.add_libration(gravity_only, exact.libration_ratio, 0.001, exact.moi(3).value);
[k, sk] = sbtk.data.libration_ratio_from_amplitude(-12, 0.2, 0.027, 0.001)   % G [deg], its sigma, e, its sigma

f0 = sbtk.invert.solve(body, components, gravity_only, sbtk.invert.options(moi_labels={}));
f1 = sbtk.invert.solve(body, components, with_izz, sbtk.invert.options(moi_labels={'Izz'}));
f2 = sbtk.invert.solve(body, components, with_lib, sbtk.invert.options(moi_labels={'libration'}));
fprintf("regolith density 1-sigma: gravity only %.0f, with Izz %.0f, with the libration ratio %.0f kg/m^3\n", ...
        f0.rho_sigma_kgm3(2), f1.rho_sigma_kgm3(2), f2.rho_sigma_kgm3(2))
```

Measured center of mass

A gravity field about the center of mass has no degree-1 terms; `sbtk.data.add_center_of_mass` adds the measured position, after which the inversion expands about the model's own center of mass (Density estimation).

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
field = sbtk.data.select_coefficients(obs, setdiff(obs.coef_labels, {'C10', 'C11', 'S11'}, "stable"));
field = sbtk.data.add_center_of_mass(field, obs.origin_km + [0.0005 0 0], 0.002);   % measured 0.5 m off
fit = sbtk.invert.solve(body, components, field);
fprintf("frame iterations %d, model center of mass − origin: [%.2f %.2f %.2f] m\n", fit.frame_iterations, 1e3*fit.com_shift_km)
```

Reading a SHADR-layout gravity file

The example writes a small file in the PDS SHADR layout and reads it back. The coefficients are about the shape-model origin, so the file includes degree-1 terms and `origin_km = [0 0 0]`.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
geo0 = sbtk.gravity.geometry(body, [0 0 0], 2);
fm = sbtk.gravity.forward(geo0, sbtk.interior.create(components), body.mass_kg);
lab = sbtk.gravity.labels(2);
coef = @(name) fm.coef(strcmp(lab, name));
file = [tempname '.tab'];
rows = sprintf("%.4f, %.6e, 0, 2, 2, 1, 0, 0", body.R_ref_km, body.GM_km3_s2);   % R, GM, sigma GM, degree, order, normalized
rows(2) = sprintf("1, 0, %.10e, 0, 1e-5, 1e-5", coef("C10"));
rows(3) = sprintf("1, 1, %.10e, %.10e, 1e-5, 1e-5", coef("C11"), coef("S11"));
rows(4) = sprintf("2, 0, %.10e, 0, 1e-5, 1e-5", coef("C20"));
rows(5) = sprintf("2, 1, %.10e, %.10e, 1e-5, 1e-5", coef("C21"), coef("S21"));
rows(6) = sprintf("2, 2, %.10e, %.10e, 1e-5, 1e-5", coef("C22"), coef("S22"));
writelines(rows, file);
[shadr, shadr_body] = sbtk.exchange.read_shadr(file, "Didymos", body.shape, [0 0 0], 2);
sbtk.data.summary(shadr)
```

Packages for another team

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
folder = tempname;
data  = sbtk.exchange.write_data(folder, "TEAM-DATA1-Didymos", body, obs, "Team name");
model = sbtk.exchange.write_model(folder, "TEAM-MOD1-Didymos", "TEAM-DATA1-Didymos", body, truth, exact, 0.040, "Team name");
[obs2, body2] = sbtk.exchange.read_data(data);                      % what the other team reads
prob = sbtk.invert.setup(body2, components, obs2, sbtk.invert.options());
solution = sbtk.exchange.write_solution(folder, sbtk.exchange.solution_name("OTHER", obs2.name), body2, ...
                                        sbtk.invert.run(prob), sbtk.invert.homogeneous(prob), 0.040, "Other team");
cmp = sbtk.compare.grids(sbtk.exchange.read_grid(fullfile(model, "density_grid.txt")), ...
                         sbtk.exchange.read_grid(fullfile(solution, "density_grid.txt")));
sbtk.compare.grid_summary(cmp);
```

Many fits against one true model

`sbtk.compare.reference` samples the true model once; `sbtk.compare.evaluate` then compares each fit in milliseconds. `sbtk.invert.replace_data` reuses the setup for each noise realization.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
ref = sbtk.compare.reference(body, truth, components, prob.geo.origin_km);
R2 = zeros(1, 20);
for k = 1:20
    fit = sbtk.invert.run(sbtk.invert.replace_data(prob, sbtk.data.add_noise(exact, 10 + k)));
    R2(k) = sbtk.compare.evaluate(ref, fit).R2;
end
fprintf("R2 over 20 noise realizations: %.2f +− %.2f\n", mean(R2), std(R2))
```

Tutorial scripts

The repository folder `examples/` contains eleven tutorial scripts with sections. Each adds `toolbox/` to the path itself and can be run whole or section by section. All eleven run in the test suite (`tests/integration/test_examples.m`, also section by section for ex01). The code below is included from the repository files.

Script	Topic	Run time
`ex01_quick_start.m`	Body, interior, simulation, inversion, comparison on the 1.165 m Didymos model	about 90 s
`ex02_shapes_and_bodies.m`	Shape models from files, arrays and ellipsoids; checks; bodies	about 5 s
`ex03_forward_model.m`	Every component type, a random field, the simulated measurements, plots	about 20 s
`ex04_inversion.m`	Homogeneous test, inertia data, sphere positions, regularization, calibration	about 30 s
`ex05_import_data.m`	Measurements from matrices, kg m², a full covariance, the text format	about 5 s
`ex06_results_folders.m`	Run folders, reports, logs, settings	about 30 s
`ex07_exchange_packages.m`	Measurement, interior-model and solution packages	about 40 s
`ex08_moments_of_inertia.m`	What inertia data and the libration ratio add to the gravity field	about 20 s
`ex09_moved_and_rotated_package.m`	A package with its shape model moved to the center of mass and rotated into the spin axes	about 40 s
`ex10_reading_coefficients.m`	Reading the coefficients of a hidden interior, placing the core along the center-of-mass offset, testing on coefficients not fitted	about 30 s
`ex11_geometry_search.m`	Searching the geometry of a model: a grid over sizes, axis and core position, Gauss–Newton from each valley, compact masses	about 10 min

The same folder has the maker models of the Hera blind test, `make_didymos_1.m` to `make_dimorphos_3.m`, and the homogeneous reference `make_didymos_homogeneous.m` (Maker models).

ex01: quick start

The sequence of the first script on the full-resolution Didymos model (3,145,728 facets), which is not in the repository (Installation). With `"didymos"` instead of `"didymos_highres"` it runs in about 15 s.

MATLAB script: examples/ex01_quick_start.m

```
%% EX01_QUICK_START  Simulate measurements of an interior, invert them, compare with the true model. (~90 s with the 1.165 m shape model)
%
% The SBTK workflow on the Didymos shape model:
%   1. a body: shape model, bulk density, reference radius
%   2. forward model: components -> gravity coefficients and moments of inertia
%   3. inversion: measurements + component geometry -> densities with 1-sigma uncertainties
%   4. comparison with the true model

addpath(fullfile(fileparts(fileparts(which('ex01_quick_start'))), 'toolbox'));

%% 1. Body
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos_highres"), 5.273e11, 0.43);   % mass [kg], R_ref [km] (Hera end-to-end test)
sbtk.body.summary(body)

%% 2. Forward model: a dense core and a low-density surface layer
components = [sbtk.interior.sphere("core", body.centroid_km, 0.170, +450), ...  % center [km] (the center of figure), radius [km], relative density [kg/m^3]
              sbtk.interior.layer("regolith", 0, 0.012, -900)];           % from 0 to 12 m below the surface
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(components), 4, "quick_start");   % exact values

% measurements: 1-sigma uncertainties, then one noise realization
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), ...  % degree 1: 2 m center of mass
                               1e-4, 2e-4, 4e-4], ...                          % degrees 2, 3, 4
                              2e-3, ...                                        % each MOI / (M R_ref^2)
                              0.02);                                           % libration ratio
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
sbtk.data.summary(obs)

%% 3. Inversion: the same components, relative densities unknown
% (the relative densities given with the components are not used: they are estimated)
fit = sbtk.invert.solve(body, components, obs);
sbtk.invert.summary(fit)

%% 4. Comparison with the true model
cmp = sbtk.compare.with_truth(body, truth, fit);
sbtk.compare.summary(cmp)
sbtk.plot.densities(fit);
sbtk.plot.truth_vs_estimate(body, truth, fit);
```

ex02: shapes and bodies

Shape models from an OBJ file with a stated unit, from arrays (a tetrahedron in meters) and as an ellipsoid; the checks of `sbtk.shape.properties`; writing OBJ; a body; the unit table.

MATLAB script: examples/ex02_shapes_and_bodies.m

```
%% EX02_SHAPES_AND_BODIES  Shape models from files or arrays, units, checks. (~5 s)
%
% Any closed polyhedral shape works if it is star-shaped about its center of
% mass (every facet visible from it). Lengths are converted to km on input:
% the unit of the file must be given.

addpath(fullfile(fileparts(fileparts(which('ex02_shapes_and_bodies'))), 'toolbox'));

%% From an OBJ file (unit stated explicitly)
didymos = sbtk.shape.load(fullfile(sbtk.root, "data", "shapes", "didymos_g_9309mm_spc_obj_0000n00000_v003.obj"), "km");
% the shape models that ship with SBTK also load by name:
%   sbtk.shape.bundled("didymos"), ("bennu"), ("dimorphos_placeholder"), ("example_asteroid")
props = sbtk.shape.properties(didymos);
fprintf('%s: %d facets, volume %.5f km^3, centroid [%.1f %.1f %.1f] m, star-shaped: %d\n', didymos.name, ...
        props.n_facets, props.volume_km3, 1e3*props.centroid_km, props.star_shaped_about_centroid);

%% From arrays (here a tetrahedron given in meters)
V = [0 0 0; 100 0 0; 0 100 0; 0 0 100] - 25;          % vertices [m]
F = [1 3 2; 1 2 4; 1 4 3; 2 3 4];                      % facets, outward
tetra = sbtk.shape.from_arrays("tetra", F, V, "m");
disp(sbtk.shape.properties(tetra))

%% A triaxial ellipsoid (exact test shape), saved to OBJ
ell = sbtk.shape.ellipsoid("test_ellipsoid", [0.40 0.35 0.30], 4);     % 5,120 facets
file = fullfile(tempdir, 'test_ellipsoid.obj');
sbtk.shape.write_obj(ell, file);
ell2 = sbtk.shape.load(file, "km");

%% Bodies: shape + bulk density + reference radius of the gravity field
body = sbtk.body.create("Ellipsoid", ell2, 2200, 0.40);
sbtk.body.summary(body)
sbtk.plot.shape(body);
sbtk.units.table()
```

---

ex03: forward model

Every component type and a random field, the simulated measurements as a table, the density at given points, and the three-dimensional, slice and measurement figures.

MATLAB script: examples/ex03_forward_model.m

```
%% EX03_FORWARD_MODEL  Every component type, a random density field, and the simulated measurements. (~20 s)
%
% Components are regions of uniform relative density (density minus the
% background density). Overlapping relative densities add and components are
% clipped to the body. The background density follows from the total mass.

addpath(fullfile(fileparts(fileparts(which('ex03_forward_model'))), 'toolbox'));
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);

%% Components
% Spheres, ellipsoids and tori are placed in shape-model coordinates, here from the center of
% figure (the origin of this shape-model file is 57 m from it); layers and sectors are
% measured from the center of mass.
cof = body.centroid_km;
components = [ ...
  sbtk.interior.sphere("core", cof, 0.170, +250), ...                                    % center, radius
  sbtk.interior.ellipsoid("lens", cof + [0.05 0.10 0.12], [0.20 0.08 0.05], [40 -30 15], +300), ... % center, semi-axes, yaw/pitch/roll
  sbtk.interior.torus("torus", 0.370, 0.055, 0.000, -500), ...                           % ring radius, tube radius, height
  sbtk.interior.layer("regolith", 0.000, 0.012, -900), ...                               % top, bottom depth
  sbtk.interior.sector("block", [0.3 0.9], [40 90], [150 240], +200), ...                % radial fraction, lat, lon
  sbtk.interior.sphere("boulder", cof + sbtk.interior.latlon(25, 60, 0.250), 0.040, +900)];   % position from lat/lon/distance
components(3).center_km = cof;                                                           % the torus about the z-axis through cof
sbtk.interior.summary(components)

%% Gaussian random density field (standard deviation 40 kg/m^3, correlation length 60 m, seed 7)
interior = sbtk.interior.create(components, sbtk.interior.random_field(40, 0.06, 7));

%% Forward model: the true model and its exact gravity coefficients and moments of inertia
[exact, truth, geo] = sbtk.data.simulate(body, interior, 4, "example_interior");

%% Measurements of it: uncertainties, then noise drawn from them
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
fprintf('background density %.0f kg/m^3, center of mass %.1f m from the center of figure\n', ...
        truth.rho_bg_kgm3, 1e3*norm(obs.com_minus_cof_km));

% What the measurements look like
disp(table(obs.coef_labels(1:8), obs.coef(1:8), sqrt(diag(obs.coef_cov(1:8, 1:8))), ...
           'VariableNames', {'label', 'value', 'sigma'}))
disp(struct2table(obs.moi))

%% Density anywhere inside (truth frame = center of mass)
rho = sbtk.interior.density_at(truth, [0 0 0; 0.3 0 0] + truth.origin_km, geo);
fprintf('density at the center %.0f, 300 m out along x %.0f kg/m^3\n', rho);

%% Plots
sbtk.plot.interior3d(body, truth, "example interior");
sbtk.plot.slices(body, {truth}, {'truth'});
sbtk.plot.measurements(obs);
```

ex04: inversion options

The homogeneous test; fits with and without inertia data; estimation of a sphere center; a 36-sector grid with Tikhonov regularization chosen by the discrepancy principle; and the standard deviation of (estimate − truth)/σ over 50 noise realizations using `sbtk.invert.replace_data`.

MATLAB script: examples/ex04_inversion.m

```
%% EX04_INVERSION  Inversion options: homogeneous model, MOI, sphere positions, regularization. (~30 s)
%
% setup does the slow part once (geometry and each component's effect on every
% measurement); run solves in milliseconds, so trying options is cheap.

addpath(fullfile(fileparts(fileparts(which('ex04_inversion'))), 'toolbox'));
body  = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
unc   = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
cof   = body.centroid_km;                  % components are placed from the center of figure
truth_components = [sbtk.interior.sphere("core", cof + [0.03 -0.02 0.01], 0.17, +500), ...
              sbtk.interior.torus("torus", 0.37, 0.055, 0, -500), sbtk.interior.layer("regolith", 0, 0.012, -900)];
truth_components(2).center_km = cof;       % the torus about the z-axis through the center of figure
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(truth_components), 4, "ex04");
exact = sbtk.data.set_uncertainties(exact, unc);
obs = sbtk.data.add_noise(exact, 1);

%% Set up once
% Component geometry is fixed (the core is placed at the center of figure, 37 m from the true
% center); relative densities are estimated, so their values here are not used.
components = [sbtk.interior.sphere("core", cof, 0.17, 0), sbtk.interior.torus("torus", 0.37, 0.055, 0, 0), ...
             sbtk.interior.layer("regolith", 0, 0.012, 0)];
components(2).center_km = cof;
opts = sbtk.invert.options();             % every setting and its standard value
disp(opts)
prob = sbtk.invert.setup(body, components, obs, opts);

%% Is a homogeneous body consistent with the measurements?
u = sbtk.invert.homogeneous(prob);
fprintf('homogeneous body: reduced chi2 %.2f, p = %.2g\n', u.chi2_reduced, u.p_value);

%% Fit, and the same with gravity only (no MOI): MOI shrink the uncertainties
fit = sbtk.invert.run(prob);
sbtk.invert.summary(fit)
fit_g = sbtk.invert.solve(body, components, obs, sbtk.invert.options(moi_labels={}));
fprintf('core 1-sigma: %.0f kg/m^3 with MOI, %.0f without\n', fit.rho_sigma_kgm3(1), fit_g.rho_sigma_kgm3(1));

%% Estimate the core position as well
moving = components;  moving(1).estimate_position = true;
fit_p = sbtk.invert.solve(body, moving, obs);
fprintf('core center [%.0f %.0f %.0f] m +- [%.0f %.0f %.0f] m (truth [30 -20 10] m from the shape origin)\n', ...
        1e3*fit_p.components(1).center_km, 1e3*fit_p.center_sigma_km(1, :));

%% Tikhonov regularization for many components (36-sector grid), parameter by the discrepancy principle
% The sectors fill the body, so the background is held at the bulk density.
sectors = sbtk.interior.sector_grid([0 0.5 0.85 1], [-90 -30 30 90], [-180 -90 0 90 180]);
probc = sbtk.invert.setup(body, sectors, obs, sbtk.invert.options(background="bulk"));
probc.opts.regularization = sbtk.invert.choose_regularization(probc, "discrepancy");
fitc = sbtk.invert.run(probc);
fprintf('sector grid: lambda %.3g, reduced chi2 %.2f (dof %.1f)\n', fitc.regularization, fitc.chi2_reduced, fitc.dof);

%% Calibration of the uncertainties: 50 noise realizations
z = zeros(50, 4);
for k = 1:50
    f = sbtk.invert.run(sbtk.invert.replace_data(prob, sbtk.data.add_noise(exact, 100 + k)));
    z(k, :) = ([f.rho_bg_kgm3, [f.components.drho_kgm3]] - [truth.rho_bg_kgm3, [truth_components.drho_kgm3]]) ...
              ./[f.rho_bg_sigma_kgm3, f.drho_sigma_kgm3];
end
fprintf('standard deviation of (estimate - truth)/sigma: %.2f (1 for calibrated uncertainties)\n', std(z(:)));
sbtk.plot.residuals(fit);
```

ex05: importing data

A measurement set from coefficient matrices; inertia given in kg m²; a libration ratio; a full covariance; the text format written and read back; selection of degrees.

MATLAB script: examples/ex05_import_data.m

```
%% EX05_IMPORT_DATA  Measurements from arrays, kg m^2, a full covariance, and text files. (~5 s)
%
% Real data arrive as coefficient matrices with uncertainties (and maybe a full
% covariance) and as moments of inertia. This builds an SBTK measurement set
% from them, in the units SBTK uses.

addpath(fullfile(fileparts(fileparts(which('ex05_import_data'))), 'toolbox'));
body = sbtk.body.from_mass("Dimorphos", sbtk.shape.bundled("dimorphos"), 2.700e9, 0.10);

%% Coefficient matrices: C(l+1, m+1) = C_lm, fully normalized about origin_km
L = 2;
C = zeros(L + 1);  S = zeros(L + 1);
C(1,1) = 1;  C(3,1) = -0.08;  C(3,3) = 0.03;  S(3,3) = 0.002;     % C00, C20, C22, S22
Cs = 1e-3*ones(L + 1);  Ss = 1e-3*ones(L + 1);                     % 1-sigma of each
obs = sbtk.data.from_coefficients(body, "example_import", [0 0 0], C, S, Cs, Ss);

%% Moments of inertia, given in kg m^2 -> I/(M R_ref^2)
Izz_kgm2 = 1.6e14;
Izz = sbtk.units.moi_to_norm(Izz_kgm2, body.mass_kg, body.R_ref_km);
obs = sbtk.data.add_moi(obs, "Izz", Izz, 0.01*Izz);
obs = sbtk.data.add_libration(obs, 0.20, 0.03, Izz);               % (B-A)/C and its 1-sigma

%% A full covariance (here: 30% correlation between all coefficients)
sig = sqrt(diag(obs.coef_cov));
corr = 0.3*ones(numel(sig)) + 0.7*eye(numel(sig));
obs = sbtk.data.set_covariance(obs, corr.*(sig*sig'));
sbtk.data.summary(obs)

%% The SBTK text format: write it, read it back
% (for data to send to other teams see ex07_exchange_packages.m)
file = fullfile(tempdir, 'example_measurements.txt');
sbtk.io.write_measurements(obs, file);
type(file)
back = sbtk.io.load_measurements(file);     % (the text keeps only the diagonal of the covariance)

%% Use fewer degrees
obs1 = sbtk.data.select_degree(obs, 1);
fprintf('%d coefficients up to degree 1\n', numel(obs1.coef));
```

ex06: results folders

A simulation, an inversion of the newest simulation found with `sbtk.io.latest_run`, and a comparison, each in its own run folder with log, settings and figures (Results folders).

MATLAB script: examples/ex06_results_folders.m

```
%% EX06_RESULTS_FOLDERS  Standard outputs: run folders, reports, logs, settings. (~30 s)
%
% Every run gets results/<project>/<date_time>_<name>/ with standard file names,
% settings.json (every number used) and log.txt (everything printed). The
% report_* functions save, print and plot the standard set in one call.

addpath(fullfile(fileparts(fileparts(which('ex06_results_folders'))), 'toolbox'));
shape = sbtk.shape.ellipsoid("ellipsoid", [0.40 0.35 0.30], 4);
body  = sbtk.body.create("Ellipsoid", shape, 2200, 0.40);
unc   = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.40), 1e-4, 2e-4], 2e-3, 0.02);

%% Simulation
components = [sbtk.interior.sphere("core", [0.03 0 0], 0.15, +600), sbtk.interior.layer("regolith", 0, 0.02, -700)];
out = sbtk.io.new_run("examples", "ellipsoid_T1");
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(components), 3, "ellipsoid_T1");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
sbtk.io.report_simulation(out, body, obs, truth);
sbtk.io.finish_run(out);

%% Inversion of the newest simulation
obs = sbtk.io.load_measurements(sbtk.io.latest_run("examples", "ellipsoid_T1"));
out = sbtk.io.new_run("examples", "ellipsoid_T1_inversion");
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
fit = sbtk.invert.run(prob);
sbtk.io.report_inversion(out, body, fit, sbtk.invert.homogeneous(prob));
sbtk.io.finish_run(out);

%% Comparison with the true model
truth = sbtk.io.load_truth(sbtk.io.latest_run("examples", "ellipsoid_T1"));
out = sbtk.io.new_run("examples", "ellipsoid_T1_comparison");
sbtk.io.report_comparison(out, body, truth, fit, sbtk.compare.with_truth(body, truth, fit));
sbtk.io.finish_run(out);
dir(out.folder)
```

ex07: exchange packages

A blind test in miniature: one team writes a measurement package and an interior-model package; another reads the measurement package, inverts it and writes a solution package; the solution is compared with the hidden model on the density grid (Exchange packages).

MATLAB script: examples/ex07_exchange_packages.m

```
%% EX07_EXCHANGE_PACKAGES  Data for other teams: measurement, interior-model and solution packages. (~40 s)
%
% A blind test: one team simulates measurements of an interior it keeps hidden and
% sends only the measurement package, of exact values; another team gives them its own
% uncertainties, inverts them and sends a
% solution package; at the end both are compared with the hidden model on a common
% density grid. Every package is plain text with its units, frame and normalization
% written in it (docs/exchange_format.md). projects/hera_hlp6 does the same with
% results folders.

addpath(fullfile(fileparts(fileparts(which('ex07_exchange_packages'))), 'toolbox'));
body = sbtk.body.create("Example asteroid", sbtk.shape.bundled("example_asteroid"), 2000, 0.60);
unc  = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.005, 0.60), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
folder = tempname;                        % where the packages go (normally a results folder)

%% The team that makes the data
hidden = [sbtk.interior.sphere("core", body.centroid_km + [0.03 0 0], 0.25, +500), ...   % 30 m from the center of figure
          sbtk.interior.layer("regolith", 0, 0.02, -800)];
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(hidden), 4, "hidden_model");
data = sbtk.exchange.write_data(folder, "TEAM-DATA1-example", body, exact, "Team, contact");       % send now: exact values
model = sbtk.exchange.write_model(folder, "TEAM-MOD1-example", "TEAM-DATA1-example", body, truth, ...
                                  exact, 0.04, "Team, contact");                                  % send at the end
type(fullfile(data, 'README.txt'))

%% The team that inverts it: everything comes from the package
[obs2, body2] = sbtk.exchange.read_data(data);          % shape model, GM and reference radius included
obs2 = sbtk.data.add_noise(sbtk.data.set_uncertainties(obs2, unc), 1);   % its own uncertainties (and noise, optionally)
components = [sbtk.interior.sphere("core", body2.centroid_km, 0.25, 0), sbtk.interior.layer("regolith", 0, 0.02, 0)];
prob = sbtk.invert.setup(body2, components, obs2, sbtk.invert.options());
fit  = sbtk.invert.run(prob);
sbtk.invert.summary(fit)
solution = sbtk.exchange.write_solution(folder, sbtk.exchange.solution_name("OTHER", obs2.name), body2, fit, ...
                                        sbtk.invert.homogeneous(prob), 0.04, "Other team, contact");

%% At the end: the solution against the hidden model, cell by cell
cmp = sbtk.compare.grids(sbtk.exchange.read_grid(fullfile(model, 'density_grid.txt')), ...
                         sbtk.exchange.read_grid(fullfile(solution, 'density_grid.txt')));
sbtk.compare.grid_summary(cmp);
```

ex08: moments of inertia and the libration ratio

The same inversion with no inertia data, I_{zz}, the libration ratio at two precisions, and I_{xx}, I_{yy}, I_{zz}; the libration ratio from an observed libration amplitude; adding measured inertia values (Moments of inertia).

MATLAB script: examples/ex08_moments_of_inertia.m

```
%% EX08_MOMENTS_OF_INERTIA  What moments of inertia and the libration ratio add to the gravity field. (~20 s)
%
% The degree-2 gravity coefficients fix five combinations of the inertia tensor
% (C20 ~ (Ixx + Iyy - 2 Izz), C22 ~ (Iyy - Ixx), C21, S21, S22 ~ products of inertia),
% but not its trace. A dense core and a light surface layer can therefore give the
% same gravity field, and the inversion cannot separate them. A moment of inertia
% such as the polar moment Izz (from the spin state) adds the missing combination:
% it measures how concentrated the mass is toward the center.
%
% The libration ratio (B - A)/C (principal moments: A <= B about the axes in the
% xy-plane, C = Izz) comes from the libration amplitude of a synchronously rotating body
% (e.g. Dimorphos). The degree-2 field gives B - A, so the ratio gives C: it carries the
% same information as a polar moment, with a relative precision of sigma/ratio.

addpath(fullfile(fileparts(fileparts(which('ex08_moments_of_inertia'))), 'toolbox'));
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);

%% Simulated measurements: every inertia quantity is available
truth_components = [sbtk.interior.sphere("core", body.centroid_km, 0.170, +450), ...   % at the center of figure
                    sbtk.interior.layer("regolith", 0, 0.012, -900)];
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(truth_components), 4, "ex08");
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], ...
                              2e-3, ...        % 1-sigma of each inertia element, as a fraction of M R_ref^2
                              0.02);           % 1-sigma of the libration ratio
exact = sbtk.data.set_uncertainties(exact, unc);
obs = sbtk.data.add_noise(exact, 1);
disp(struct2table(rmfield(obs.moi, 'weights')))    % Ixx Iyy Izz Ixy Ixz Iyz libration, as I/(M R_ref^2)

%% The same inversion with different inertia measurements
% opts.moi_labels chooses which of the data's inertia rows are fitted. The libration
% ratio of this body is about 0.056, so the simulated 1-sigma of 0.02 (36%) says little;
% a ratio measured to 0.001 (1.8%) is added as a separate data set (sbtk.data.add_libration).
gravity  = sbtk.data.select_moi(obs, {});                              % gravity coefficients only
precise  = sbtk.data.add_libration(gravity, exact.libration_ratio, 0.001, obs.moi(3).value);
cases = {'none (gravity only)',    obs,     {}
         'Izz',                    obs,     {'Izz'}
         'libration +- 0.02',      obs,     {'libration'}
         'libration +- 0.001',     precise, {'libration'}
         'Ixx Iyy Izz',            obs,     {'Ixx', 'Iyy', 'Izz'}};
fprintf('\n%-22s %18s %18s %12s %8s\n', 'inertia used', 'core [kg/m^3]', 'regolith [kg/m^3]', 'correlation', 'chi2/nu');
for k = 1:size(cases, 1)
    fit = sbtk.invert.solve(body, truth_components, cases{k, 2}, sbtk.invert.options(moi_labels=cases{k, 3}));
    fprintf('%-22s %10.0f +- %4.0f %10.0f +- %4.0f %12.2f %8.2f\n', cases{k, 1}, fit.rho_kgm3(1), ...
            fit.rho_sigma_kgm3(1), fit.rho_kgm3(2), fit.rho_sigma_kgm3(2), fit.corr(2, 3), fit.chi2_reduced);
end
fprintf('%-22s %10.0f %20.0f\n', 'true', truth.rho_bg_kgm3 + truth_components(1).drho_kgm3, ...
        truth.rho_bg_kgm3 + truth_components(2).drho_kgm3);
% correlation: between the core and regolith relative densities (near 1 = the data
% cannot tell them apart).
% Izz halves the regolith uncertainty. The libration ratio gives C = (B - A)/ratio, so it
% constrains C as well as the ratio is known relative to its value: +-0.001 is 1.8% of
% C and helps, Izz +-0.002 is 0.5% and helps more.

%% A libration ratio from the observed libration amplitude
% A synchronous satellite in an orbit of eccentricity e librates with amplitude
% 2e/(3 ratio - 1) (forced libration, first order in e). Near ratio = 1/3 (elongation
% about 1.4 : 1) the amplitude is large and fixes the ratio precisely; a nearly
% axisymmetric body librates little and the ratio stays uncertain.
for amp = [-4 -12]
    [k, sk] = sbtk.data.libration_ratio_from_amplitude(amp, 0.2, 0.027, 0.001);   % deg, deg, e, sigma e
    fprintf('amplitude %5.1f +- 0.2 deg, e = 0.027 +- 0.001: (B - A)/C = %.3f +- %.4f (%.0f%%)\n', ...
            amp, k, sk, 100*sk/k);
end

%% Your own inertia measurements
% Start from the gravity coefficients (a package, a SHADR file, coefficient matrices,
% ...) and add each measured quantity. Values given in kg m^2 convert with
% sbtk.units.moi_to_norm; inertia is about the center of mass, shape-model axes.
Izz_kgm2 = sbtk.units.norm_to_moi(obs.moi(3).value, body.mass_kg, body.R_ref_km);   % e.g. a measured polar moment
mine = sbtk.data.add_moi(gravity, "Izz", sbtk.units.moi_to_norm(Izz_kgm2, body.mass_kg, body.R_ref_km), 2e-3);
mine = sbtk.data.add_libration(mine, exact.libration_ratio, 0.001, mine.moi(1).value);  % (B - A)/C, 1-sigma, Izz
fit = sbtk.invert.solve(body, truth_components, mine, sbtk.invert.options(moi_labels={'Izz', 'libration'}));
sbtk.invert.summary(fit)
```

ex09: moved and rotated package

The HLP6 makers send their data in the frame and about the origin of the shape model file as distributed. This tutorial keeps the other way: the shape model, the true model and the exact values are moved to the center of mass of the true model and rotated into its spin axes (or principal axes), so degree 1

and $\bar{C}_{21}$, $\bar{S}_{21}$ are zero. The package contains the moved and rotated shape model, named after the package, and `conventions.txt` gives the change of frame from the original file; the original coordinates are recovered from it (Frames of a gravity field, The shape model of a package).

MATLAB script: examples/ex09_moved_and_rotated_package.m

```
%% EX09_MOVED_AND_ROTATED_PACKAGE  A package with the shape model moved to the center of mass and rotated. (~40 s)
%
% The HLP6 makers agreed to send their data in the frame and about the origin of the shape
% model file as distributed (examples/make_didymos_1.m): degree 1 and C21, S21 are then not
% zero. This tutorial keeps the alternative. The shape model, the true model and the exact
% values are moved to the center of mass of the true model and rotated into its spin axes
% (z the axis of the largest moment, x the shape-model x-axis projected onto the plane
% normal to z) or its principal axes, so degree 1 and C21, S21 (and S22) are zero. The
% package then contains the moved and rotated shape model, named after the package, and
% conventions.txt gives the change of frame from the original file
% (website: Frames of a gravity field; Exchange packages, "The shape model of a package").

addpath(fullfile(fileparts(fileparts(which('ex09_moved_and_rotated_package'))), 'toolbox'));
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
cof  = body.centroid_km;                                % the origin of the file is 57 m from it
folder = tempname;                                      % where the packages go (normally a results folder)

%% A true model, made in the frame of the shape model file
torus = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
torus.center_km = cof;
interior = sbtk.interior.create([sbtk.interior.sphere("core", cof + [0.02 -0.01 0], 0.150, 400), torus]);
interior = sbtk.interior.align_with_spin_axis(body, interior, "torus", 0.01, 20);   % torus about the spin axis
[exact, truth] = sbtk.data.simulate(body, interior, 4, "moved_and_rotated");

%% Move and rotate: origin at the center of mass, spin axes ("principal" for the principal axes)
spin  = sbtk.gravity.center_of_mass_frame(truth, body, "spin");
body  = sbtk.body.to_frame(body, spin.com_km, spin.axes, "didymos_v003_com_spin_axes");
truth = sbtk.interior.to_frame(truth, spin.com_km, spin.axes);
exact = sbtk.data.to_frame(exact, spin.com_km, spin.axes);    % before select_moi: needs the full tensor
exact = sbtk.data.select_moi(exact, ["Ixx" "Iyy" "Izz"]);
fprintf('largest-moment axis %.2f deg from the DART z-axis; center of mass %.1f m from the file origin\n', ...
        acosd(spin.principal(3, 3)), 1e3*norm(spin.com_km));

%% The packages
data  = sbtk.exchange.write_data(folder, "TEAM-DATA1-Didymos", body, exact, "Team, contact");
model = sbtk.exchange.write_model(folder, "TEAM-MOD1-Didymos", "TEAM-DATA1-Didymos", body, truth, exact, ...
                                  0.040, "Team, contact");
disp(string({dir(data).name})')                         % ... TEAM-DATA1-Didymos_didymos_v003_com_spin_axes.obj
type(fullfile(data, 'conventions.txt'))

%% What a receiving team reads
[obs, rbody] = sbtk.exchange.read_data(data);
[~, i] = ismember(["C21" "S21"], string(obs.coef_labels));
fprintf('center of mass %s km; C21 %.1e, S21 %.1e\n', mat2str(obs.origin_km, 3), obs.coef(i));
% the original coordinates: x = x' A' + o
original = rbody.shape.vertices_km*body.shape.frame_axes' + body.shape.frame_origin_km;
fprintf('max difference from the DART vertices: %.1e km\n', ...
        max(abs(original - sbtk.shape.bundled("didymos").vertices_km), [], 'all'));
```

ex10: reading the coefficients

The procedure of Reading the gravity coefficients on a hidden Didymos interior (a dense core 30 m from the center of figure, a light torus raised 12 m and a light surface layer): the anomaly and which coefficients differ from constant density; each degree's scaling and the rest; one compact mass from degrees 1 and 2; the core's χ^2 profile along the center-of-mass offset; Gauss–Newton with the torus height free; and the model fitted to degrees 1–3 predicting degrees 4–5.

MATLAB script: examples/ex10_reading_coefficients.m

```
%% EX10_READING_COEFFICIENTS  Read the gravity coefficients of an unknown interior, then fit the components they point to.
%
% A worked example of the procedure in the documentation (Reading the gravity
% coefficients). The interior is hidden in the first section, as in a blind test; the
% rest uses only the measurements:
%   1. the anomaly: measured minus homogeneous coefficients, about the center of figure,
%      and which of them differ from constant density (sbtk.interpret.anomaly,
%      sbtk.interpret.summary, sbtk.plot.significance);
%   2. each degree as a scaled homogeneous field plus the rest (sbtk.plot.degree_ratio):
%      the scaling says how much mass is concentrated toward the center, the rest which
%      coefficients carry structure that is not centrally symmetric;
%   3. one compact mass from degrees 1 and 2 (sbtk.interpret.central_mass);
%   4. a model with the components the reading suggests; chi-square along the measured
%      center-of-mass offset for the core position (sbtk.invert.scan_position);
%   5. Gauss-Newton from the lowest point, with the torus height free;
%   6. the same model fitted to degrees 1-3 only predicts degrees 4-5
%      (sbtk.invert.predict): a test on coefficients it was not fitted to.
% Figures go to results/examples/<date_time>_ex10_reading_coefficients/figures/. About 30 s.

clear; close all;
root = fileparts(fileparts(mfilename('fullpath')));
addpath(fullfile(root, 'toolbox'));
out = sbtk.io.new_run("examples", "ex10_reading_coefficients");

%% The hidden interior and its measurements
body = sbtk.body.create("Didymos", sbtk.shape.bundled("didymos"), 2600, 0.41);
cof  = body.centroid_km;                                % center of figure
torus = sbtk.interior.torus("torus", 0.36, 0.05, 0, -400);
torus.center_km = cof + [0 0 0.012];
hidden = [sbtk.interior.sphere("core", cof + [0.025 -0.015 0.008], 0.14, +500), torus, ...
          sbtk.interior.layer("regolith", 0, 0.015, -600)];
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(hidden), 5, "reading example");
exact = sbtk.data.expand_about(sbtk.data.select_moi(exact, {}), [0 0 0]);   % about the origin of the shape model file
unc = sbtk.data.uncertainties([1.3e-5 2e-5 3e-5 3e-5 3e-5], 1, 0.02);       % 1-sigma by degree, close to the Hera RSE values
obs = sbtk.data.set_uncertainties(exact, unc);

%% 1. The anomaly
an = sbtk.interpret.anomaly(obs, body, 5);
lines = sbtk.interpret.summary(an);
sbtk.io.save_figure(out, sbtk.plot.significance(an), "significance");

%% 2. Scaling and the rest, degree by degree
sbtk.io.save_figure(out, sbtk.plot.degree_ratio(an, {}, strings(1, 0)), "degree_ratio");

%% 3. One compact mass from degrees 1 and 2
cm = sbtk.interpret.central_mass(an);

%% 4. A model and the core position along the center-of-mass offset
model = [sbtk.interior.sphere("core", cof, 0.15, 0), sbtk.interior.torus("torus", 0.37, 0.055, 0, 0), ...
         sbtk.interior.layer("regolith", 0, 0.012, 0)];
model(2).center_km = cof;
opts = sbtk.invert.options(moi_labels={});
prob = sbtk.invert.setup(body, model, obs, opts);
u  = an.com_offset_km/norm(an.com_offset_km);
t  = (-0.10:0.005:0.10)';
sc = sbtk.invert.scan_position(prob, "core", cof + t*u);
fig = figure('Color', 'w', 'Position', [100 100 700 360]);
semilogy(1e3*t, sc.chi2, 'o-');
xline(1e3*(hidden(1).center_km - cof)*u', ':', 'hidden core', 'LabelVerticalAlignment', 'middle');
xlabel('core position along the center-of-mass offset [m]');  ylabel('\chi^2');
sbtk.io.save_figure(out, fig, "scan");

%% 5. Gauss-Newton from the lowest point, the torus height free
model(1).center_km = sc.centers_km(sc.best, :);
model(1).estimate_position = true;
model(2).estimate_position = [false false true];
fit = sbtk.invert.solve(body, model, obs, opts);
sbtk.invert.summary(fit)

%% 6. Fitted to degrees 1-3, tested on degrees 4-5
fit3 = sbtk.invert.solve(body, model, sbtk.data.select_degree(obs, 3), opts);
preds = {sbtk.invert.predict(fit3, body, obs), sbtk.invert.predict(fit, body, obs)};
sbtk.io.save_figure(out, sbtk.plot.coefficient_residuals(preds, ["degrees 1-3", "degrees 1-5"]), "coefficient_residuals");
sbtk.io.save_figure(out, sbtk.plot.anomaly(an, {sbtk.interpret.anomaly(fit3, body, 5), sbtk.interpret.anomaly(fit, body, 5)}, ...
                                           ["fitted to degrees 1-3", "fitted to degrees 1-5"]), "anomaly");
fprintf('fitted to degrees 1-3: chi2 per coefficient %.2f (fitted), %.2f (degrees 4-5)\n', ...
        preds{1}.chi2_fitted/preds{1}.n_fitted, preds{1}.chi2_other/preds{1}.n_other);
sbtk.io.save_figure(out, sbtk.plot.truth_vs_estimate(body, truth, fit), "truth_vs_estimate");
sbtk.io.finish_run(out);
```

ex11: searching the geometry

The search of Searching the geometry on a hidden Didymos interior (a dense core, a torus of other radii than the model's and tilted 3°, a light layer, a dense boulder and a light one): the model with guessed sizes; a grid over the core position along the center-of-mass offset, the torus radii and axis and the layer thickness, with the χ^2 profile of each; Gauss–Newton from the lowest point of each valley; compact masses added while each passes the likelihood-ratio test; and the result against the hidden interior.

MATLAB script: examples/ex11_geometry_search.m

```
%% EX11_GEOMETRY_SEARCH  Search the geometry of a model systematically: sizes, axis, core position and compact masses.
%
% A worked example of the documentation page Searching the geometry. The interior is
% hidden in the first section, as in a blind test: a dense core, a torus of other radii
% than the model's and tilted 3 deg, a light layer, a dense boulder and a light one. The
% rest uses only the measurements (coefficients to degree 5 with 1-sigma close to the Hera
% radio-science values) and a model of a core, a torus and a layer:
%   1. the model with guessed sizes, the core placed by Gauss-Newton: the starting point;
%   2. a grid over the core position along the center-of-mass offset, the ring and tube
%      radii and the axis of the torus, and the layer thickness (sbtk.search.grid), and
%      the chi-square profile of each (sbtk.plot.search_profiles);
%   3. Gauss-Newton on the core position and the torus height and axis from the lowest
%      point of each valley of the profiles (sbtk.search.refine);
%   4. compact masses added where the residuals ask for them, each kept only if it passes
%      the likelihood-ratio test (sbtk.search.add_masses, sbtk.plot.search_masses);
%   5. the result against the hidden interior.
% sbtk.search.run does steps 2-4 in one call. The 49,152-facet Didymos model keeps the
% torus integrals at about 1 s each. Figures go to
% results/examples/<date_time>_ex11_geometry_search/figures/. About 10 minutes.

clear; close all;
root = fileparts(fileparts(mfilename('fullpath')));
addpath(fullfile(root, 'toolbox'));
out = sbtk.io.new_run("examples", "ex11_geometry_search");

%% The hidden interior and its measurements
body = sbtk.body.create("Didymos", sbtk.shape.bundled("didymos_every8"), 2600, 0.41);
cof  = body.centroid_km;                                % center of figure
torus = sbtk.interior.torus("torus", 0.330, 0.045, 0, -450);
torus.center_km = cof + [0 0 0.010];
torus = sbtk.interior.set_axis(torus, [sind(3)*cosd(40), sind(3)*sind(40), cosd(3)]);
hidden = [sbtk.interior.sphere("core", cof + [0.035 -0.020 0.010], 0.140, +600), torus, ...
          sbtk.interior.layer("regolith", 0, 0.018, -700), ...
          sbtk.interior.sphere("boulder1", cof + sbtk.interior.latlon(20, 60, 0.200), 0.045, +1500), ...
          sbtk.interior.sphere("boulder2", cof + sbtk.interior.latlon(-30, -120, 0.180), 0.045, -1200)];
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(hidden), 5, "search example");
exact = sbtk.data.expand_about(sbtk.data.select_moi(exact, {}), [0 0 0]);   % about the origin of the shape model file
unc = sbtk.data.uncertainties([1.3e-5 2e-5 3e-5 3e-5 3e-5], 1, 0.02);       % 1-sigma by degree, close to the Hera RSE values
obs = sbtk.data.set_uncertainties(exact, unc);

%% 1. The model with guessed sizes
an = sbtk.interpret.anomaly(obs, body, 5);
u  = an.com_offset_km/norm(an.com_offset_km);            % direction of the center-of-mass offset
core = sbtk.interior.sphere("core", cof, 0.150, 0);
core.estimate_position = true;
ring = sbtk.interior.torus("torus", 0.370, 0.055, 0, 0);
ring.center_km = cof;
ring.estimate_position = [false false true];            % its height
ring.estimate_axis = true;
model = [core, ring, sbtk.interior.layer("regolith", 0, 0.012, 0)];
opts = sbtk.invert.options(moi_labels={});
prob = sbtk.invert.setup(body, model, obs, opts);
guessed = sbtk.invert.run(prob);
fprintf('guessed sizes: chi2 %.1f for %d coefficients, p = %.2g\n', guessed.chi2, guessed.n_obs, guessed.p_value);

%% 2. Grid
t  = (-0.080:0.010:0.080)';
R  = (0.300:0.025:0.400)';
a  = (0.035:0.010:0.065)';
[d, tilt] = sbtk.search.directions([0 0 1], 4, 4);      % the z-axis and 6 directions 4 deg from it
th = [0.008; 0.015; 0.022; 0.030];
P = [sbtk.search.parameter("core", "center_km", cof + t*u, 1e3*t, "core along the offset (m)"), ...
     sbtk.search.parameter("torus", "ring_radius_km", R, R, "ring radius (km)"), ...
     sbtk.search.parameter("torus", "tube_radius_km", a, a, "tube radius (km)"), ...
     sbtk.search.parameter("torus", "axis", d, tilt, "tilt of the axis (deg)"), ...
     sbtk.search.parameter("regolith", "depth_km", [zeros(4, 1), th], 1e3*th, "thickness (m)")];
gr = sbtk.search.grid(prob, P, 3);
fprintf('grid: %d combinations from %d integrals in %.0f s; best chi2 %.1f\n', gr.n_combinations, ...
        gr.n_integrals, gr.time_s, gr.best_fit.chi2);
sbtk.io.save_figure(out, sbtk.plot.search_profiles(gr), "profiles");

%% 3. Gauss-Newton from the lowest point of each valley
rf = sbtk.search.refine(prob, gr.components);
fprintf('refined: chi2 %s\n', mat2str(rf.chi2, 4));

%% 4. Compact masses
cand = sbtk.interior.sphere_grid(body, 0.040, 0.045);     % candidate positions every 40 m
ms = sbtk.search.add_masses(prob, rf.best_fit, cand, 4, 0.05, 3, 3);   % 3: the ring and tube radii and the
                                                          % layer thickness, chosen on the grid
fprintf('masses: %d kept (%s); chi2 %s\n', ms.n_masses, ms.stop_reason, mat2str(ms.chi2, 4));
sbtk.io.save_figure(out, sbtk.plot.search_masses(ms, body), "masses");
```

```
%% 5. The result against the hidden interior
fit = ms.best_fit;
sbtk.invert.summary(fit)
for c = fit.components
    fprintf('%-9s %+6.0f kg/m^3, center - cof [%5.1f %5.1f %5.1f] m\n', c.name, c.drho_kgm3, 1e3*(c.center_km - cof));
end
fprintf('torus: ring %.0f m, tube %.0f m, axis %.1f deg from z; layer %.0f m\n', 1e3*fit.components(2).ring_radius_km, ...
        1e3*fit.components(2).tube_radius_km, acosd(fit.components(2).axes(3, 3)), 1e3*fit.components(3).depth_km(2));
cmp = sbtk.compare.with_truth(body, truth, fit);
cmp0 = sbtk.compare.with_truth(body, truth, guessed);
fprintf('density R2: guessed sizes %.2f, after the search %.2f\n', cmp0.R2, cmp.R2);
sbtk.io.save_figure(out, sbtk.plot.truth_vs_estimate(body, truth, fit), "truth_vs_estimate");
sbtk.io.finish_run(out);
```

Project template

A project is a folder of runner scripts for one body that write to `results/<project>/`. The folder `projects/template/` is the starting point for a new body. Body-specific numbers are kept in one settings function, `my_body.m`; nothing body-specific is in the toolbox.

Files

File	Purpose
`my_body.m`	Returns the body (shape model and its length unit, bulk density, reference radius), the gravity degree and the density-grid spacing.
`simulate_measurements.m`	Simulates exact measurements of a prescribed interior and writes a measurement package and an interior-model package.
`invert_measurements.m`	Gives the measurements of a package uncertainties, estimates component densities and writes a solution package.
`compare_with_truth.m`	Compares an inversion with the true model of a simulation, by component and on the density grid.

As delivered, the template runs on `data/shapes/example_asteroid.obj`, a synthetic body of about 1.2 km. `tests/integration/test_template_project.m` runs the three scripts.

Use

1. Copy `projects/template/` to `projects/<body>/`.
2. In `my_body.m`, replace the shape file, its length unit, the bulk density (or use `sbtk.body.from_mass`), the reference radius, the degree and the grid spacing. The shape must be closed and star-shaped about its center of mass (Shape models).
3. Set `project` at the top of each script.
4. In `simulate_measurements.m`, edit the sections PROJECT, NAME AND DELIVERY and INTERIOR. The simulation writes exact values, without uncertainties or noise.
5. In `invert_measurements.m`, edit DATA (the package to invert), UNCERTAINTIES (the 1σ the receiving team gives the exact values), MODEL and OPTIONS.
6. Run the scripts in order. Each run writes its own folder (Results folders); the inversion finds the newest simulation of the given name with `sbtk.io.latest_run`.

A simulation run writes

Simulation run folder

```
results/<project>/<date_time>_<name>/
  log.txt                                everything printed (includes the true model)
  redacted/<team>-DATA<n>-<name>/        measurement package: nothing about the interior
  unredacted/<team>-MOD<n>-<name>/       interior-model package: true model, density grid, the same exact values
  unredacted/                            measurements.*, truth.*, settings.json, figures/
```

Code

MATLAB: projects/template/my_body.m

```
function [body, degree, grid_km] = my_body()
%MY_BODY  Everything specific to your asteroid, used by every runner in this project.
%
%   [body, degree, grid_km] = my_body()
%
%   Replace the shape file (and its length unit), the bulk density, the reference
%   radius, the degree and the grid spacing with your own. The measurement
%   uncertainties are set by the receiving team in each inversion run (invert_measurements.m).

shape = sbtk.shape.load(fullfile(sbtk.root, 'data', 'shapes', 'example_asteroid.obj'), ...   % your shape model
                        "km");                                                               % its length unit: "km", "m" or "cm"
body  = sbtk.body.create("Example asteroid", shape, ...
            2000, ...       % bulk density [kg/m^3]
            0.60);          % reference radius of the gravity coefficients [km]
degree = 4;                 % highest degree of the gravity field
grid_km = 0.040;            % cell edge of the density grids in model and solution packages [km]
end
```

MATLAB script: projects/template/simulate_measurements.m

```
%% SIMULATE_MEASUREMENTS  Synthetic measurements of a prescribed interior, for any body.
%
% Edit NAME and INTERIOR; the body is defined in my_body.m. The measurements are the exact
% values of the interior, without uncertainties or noise: the receiving team applies its own
% (invert_measurements.m). Writes results/<project>/<date_time>_<name>/:
%   redacted/<team>-DATA<n>-<name>/    measurement package to send (nothing about the interior)
%   unredacted/<team>-MOD<n>-<name>/   interior-model package (true model, density grid,
%                                      the same exact values, figures)
%   unredacted/                        measurements.*, truth.*, settings.json, figures/
%   log.txt
% Package format: docs/exchange_format.md.

clear; close all;
here = fileparts(which('simulate_measurements'));
addpath(here, fullfile(fileparts(fileparts(here)), 'toolbox'));
[body, degree, grid_km] = my_body();

%% PROJECT, NAME AND DELIVERY
project     = "my_asteroid";
name        = "T1";
team        = "TEAM";                % team identifier in package names
number      = 1;                     % package numbers: <team>-DATA<number>, <team>-MOD<number>
prepared_by = "Team name, contact";

%% INTERIOR
% Components with their relative density (density minus background, kg/m^3);
% lengths in km. See sbtk.interior: sphere, ellipsoid, torus, layer, sector. Spheres,
% ellipsoids and tori are placed from the center of figure (cof), layers and sectors are
% measured from the center of mass.
cof = body.centroid_km;
components = [ ...
    sbtk.interior.sphere("core",     cof,                                    0.250, +500), ...
    sbtk.interior.layer("regolith",  0.000, 0.020,                                 -800), ...
    sbtk.interior.sphere("boulder",  cof + sbtk.interior.latlon(10, -40, 0.35), 0.060, +900)];
field = sbtk.interior.no_random_field();     % or sbtk.interior.random_field(sigma kg/m^3, correlation length km, seed)

%% RUN
out = sbtk.io.new_run(project, name);
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(components, field), degree, name);   % the true model
sbtk.io.report_simulation(sbtk.io.subfolder(out, "unredacted"), body, exact, truth);
data_set = sprintf("%s-DATA%d-%s", team, number, name);
sbtk.exchange.write_data(fullfile(out.folder, "redacted"), data_set, body, exact, prepared_by);
sbtk.exchange.write_model(fullfile(out.folder, "unredacted"), sprintf("%s-MOD%d-%s", team, number, name), ...
                          data_set, body, truth, exact, grid_km, prepared_by);
sbtk.io.finish_run(out);
```

MATLAB script: projects/template/invert_measurements.m

```
%% INVERT_MEASUREMENTS  Component densities from gravity coefficients and moments of inertia, for any body.
%
% Edit DATA, UNCERTAINTIES, MODEL and OPTIONS. Writes results/<project>/<date_time>_<name>_inversion/:
% fit.txt / .mat, homogeneous.txt, settings.json, log.txt, figures/.

clear; close all;
here = fileparts(which('invert_measurements'));
addpath(here, fullfile(fileparts(fileparts(here)), 'toolbox'));
[~, ~, grid_km] = my_body();
team = "TEAM";  prepared_by = "Team name, contact";

%% DATA
project = "my_asteroid";
name    = "T1";
data_folder = sbtk.exchange.locate(sbtk.io.latest_run(project, name), "DATA");   % or any package folder
[obs, body] = sbtk.exchange.read_data(data_folder);

%% UNCERTAINTIES (1-sigma): ours, since the package has exact values
unc = sbtk.data.uncertainties( ...
    [sbtk.units.offset_to_degree1(0.005, body.R_ref_km), ... % degree 1: 5 m center of mass
     1e-4, 2e-4, 4e-4], ...                                  % degrees 2..degree (fully normalized)
    2e-3, ...                                                % each moment of inertia, as a fraction of M R_ref^2
    0.02);                                                   % libration ratio (B-A)/C
obs = sbtk.data.set_uncertainties(obs, unc);
% obs = sbtk.data.add_noise(obs, 1);                         % noise drawn from them, if wanted

%% MODEL (geometry fixed; relative densities estimated)
cof = body.centroid_km;                                      % center of figure of the package's shape model
components = [ ...
    sbtk.interior.sphere("core",     cof,          0.250, 0), ...
    sbtk.interior.layer("regolith",  0.000, 0.020,        0)];
% Without assumed structure:
% components = sbtk.interior.sector_grid([0 0.45 0.8 1], [-90 0 90], [-180 -90 0 90 180]);

%% OPTIONS
opts = sbtk.invert.options();
opts.moi_labels = {'Ixx', 'Iyy', 'Izz'};   % moment-of-inertia measurements used ({} = gravity only)

%% RUN
out  = sbtk.io.new_run(project, name + "_inversion");
prob = sbtk.invert.setup(body, components, obs, opts);
if numel(components) > 6
    prob.opts.regularization = sbtk.invert.choose_regularization(prob, "discrepancy");    % many components
end
fit = sbtk.invert.run(prob);
hom = sbtk.invert.homogeneous(prob);
sbtk.io.report_inversion(out, body, fit, hom);
sbtk.exchange.write_solution(out.folder, sbtk.exchange.solution_name(team, obs.name), body, fit, hom, grid_km, prepared_by);
sbtk.io.finish_run(out);
```

MATLAB script: projects/template/compare_with_truth.m

```
%% COMPARE_WITH_TRUTH  Compare an inversion with the true model of a simulation, for any body.
%
% Reports R2 (fraction of the variance of the true density explained by the
% estimate), R2_max for the same components, the RMS density error and each
% component's estimated and true density; then compares the density grids of the
% solution package and the interior-model package cell by cell. Writes
% results/<project>/<date_time>_<inversion>_comparison/: comparison.txt / .mat,
% grid_comparison.txt, figures/comparison.png, figures/truth_vs_estimate.png.

clear; close all;
here = fileparts(which('compare_with_truth'));
addpath(here, fullfile(fileparts(fileparts(here)), 'toolbox'));
body = my_body();

project       = "my_asteroid";
truth_run     = "T1";              % simulation
inversion_run = "T1_inversion";    % inversion

simulation = sbtk.io.latest_run(project, truth_run);
inversion  = sbtk.io.latest_run(project, inversion_run);
truth = sbtk.io.load_truth(simulation);
fit   = sbtk.io.load_fit(inversion);
out   = sbtk.io.new_run(project, inversion_run + "_comparison");
sbtk.io.report_comparison(out, body, truth, fit, sbtk.compare.with_truth(body, truth, fit));

% the solution package against the interior-model package, cell by cell
cmp = sbtk.compare.grids(sbtk.exchange.read_grid(fullfile(sbtk.exchange.locate(simulation, "MOD"), "density_grid.txt")), ...
                         sbtk.exchange.read_grid(fullfile(sbtk.exchange.locate(inversion, "SOL"), "density_grid.txt")));
writelines(sbtk.compare.grid_summary(cmp), fullfile(out.folder, "grid_comparison.txt"));
sbtk.io.finish_run(out);
```

Bennu

`projects/bennu/` estimates the interior of the asteroid Bennu from the OSIRIS-REx gravity field (JPL, 2019; degree 10, R_{ref} = 0.290 km) following Cavale et al., "Constraints on the Internal Density Distribution of Bennu Using OSIRIS-REx Mission Data" (Icarus), and reproduces that paper's inversion.

Files

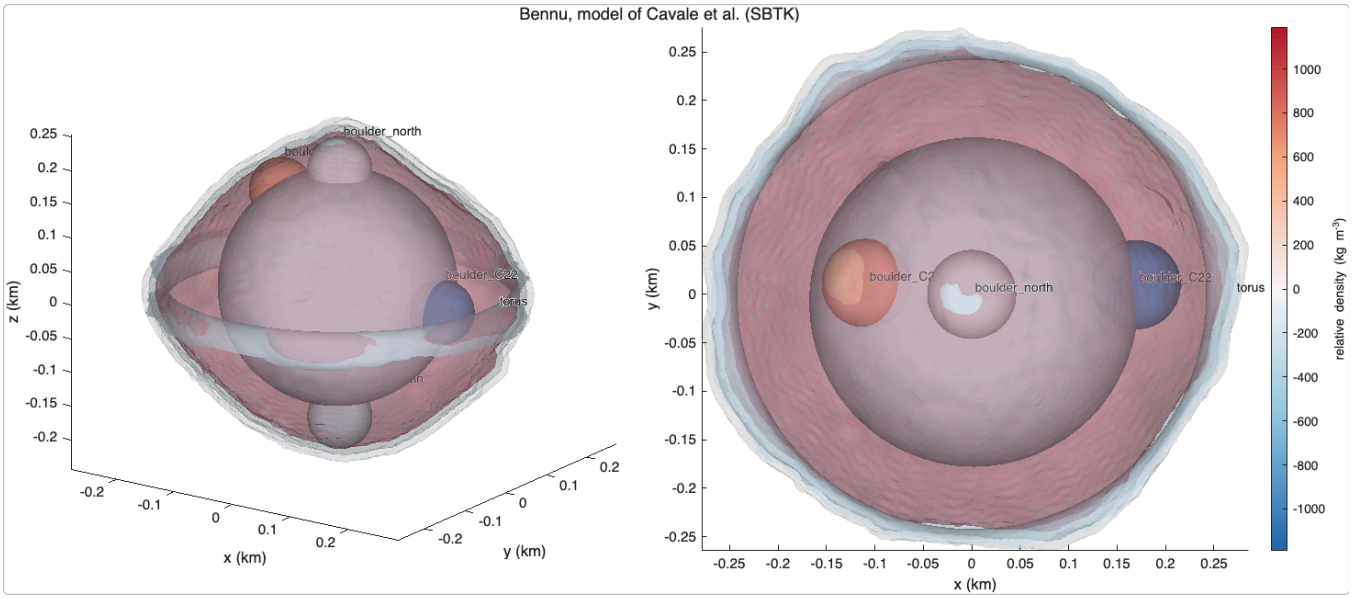
File	Purpose
`bennu_body.m`	Shape model `bennu_higher_qual.obj` (196,608 facets, body-fixed frame with origin at the center of mass), bulk density 1190 kg/m³, R_{ref} = 0.290 km.
`bennu_measurements.m`	The field as a measurement set, with the paper's weighting (C10 = C11 = S11 = 0, J_2, C21, S21, C22, S22, J_3, J_4 with equal weights) or with all coefficients of degrees 2–4 and the full JPL covariance. Reads the file with `sbtk.io.read_jpl_gravity`.
`bennu_paper_components.m`	The paper's nominal model: prescribed surface (0–10 m, −297.5 kg/m³) and subsurface (10–20 m, +1190 kg/m³) layers, a core of radius 170 m with estimated center, an equatorial torus and four boulders. Positions are from the origin of the shape model, as in the paper; that origin is Bennu's center of mass from the OSIRIS-REx tracking data, 1.4 m from the center of figure (Where to place components).
`invert_bennu_paper.m`	Reproduces the paper's inversion with the background held at the bulk density and compares the result with the paper's values (`tests/reference/bennu_paper_nominal.json`).
`invert_bennu_boulder_positions.m`	The same with the centers of the two boulders that target C21/S21 and C22/S22 also estimated, with an a priori 1σ of 10 m per coordinate.
`check_bennu_full_field.m`	Compares the reproduced interior's coefficients of degrees 2–4 with the full field and covariance.
`bennu_compare_with_paper.m`	Prints a fit next to the paper's values.

Reproduction of the paper

Relative densities from `invert_bennu_paper.m` (`comparison_with_paper.txt` in its run folder):

Relative density (kg/m³)	Paper	SBTK
core	−241.9	−242.1
torus	−239.4	−234.6
boulder C21	+664.5	+660.8
boulder C22	−1049.3	−1037.3
boulder north	−148.0	−146.4
boulder south	−251.5	−250.9
core center (m)	(1.86, −7.89, 12.59)	(2.06, −7.91, 12.56)

The paper's code counted a surface facet as part of the torus only when all three of its vertices were inside the tube; SBTK integrates the exact intersection, whose volume is 2.4% larger, which accounts for most of the torus difference. `tests/integration/test_bennu_paper.m` checks the reproduction against the paper's values: the core relative density within 1 kg/m³, the boulders within 2% of the paper's relative densities, the torus mass contrast within 1% of the paper's, the core center within 0.5 m, and the predicted coefficients the paper did not fit (C31 … S44) within 0.15 of the JPL 1σ of those the paper's code predicts (Validation).



Estimated interior of Bennu in the paper configuration, from `invert_bennu_paper.m` (`sbtk.plot.interior3d`): oblique view and view down the z-axis, components labeled and colored by relative density.

Hera blind test

`projects/hera_hlp6/` contains the runner scripts SBTK uses in a blind test of interior-modeling methods for the binary asteroid Didymos–Dimorphos, the target of ESA's Hera mission. In such a test one team simulates measurements of an interior it keeps hidden and shares only the measurements; other teams estimate the interior; the hidden models are released afterwards and compared with the estimates on a common density grid. The makers deliver exact values, without uncertainties or noise; each receiving team weights them with uncertainties of its own choosing. SBTK's inversion runners use those of the Hera end-to-end simulation, read from files on the Hera working-group server (Didymos and Dimorphos values); the runners need copies in `data/hera_rse/`.

Files

File	Purpose
`hlp6_didymos.m`, `hlp6_dimorphos.m`	Settings shared by every run of one body: shape model, mass, reference radius, degree, density-grid spacing.
`hlp6_team.m`	Team identifier and contact written into package names and files.
`hlp6_uncertainties.m`	The receiving team's uncertainties: the Hera radio-science precision of the gravity coefficients (to degree 5 for Didymos, 3 for Dimorphos, to which the coefficients are truncated), the inertia elements and the libration ratio.
`simulate_didymos.m`, `simulate_dimorphos.m`	Simulate measurements of a hidden interior; write the measurement package and the interior-model package.
`invert_didymos.m`, `invert_dimorphos.m`	Invert a measurement package (by default the redacted package of the newest simulation); write a solution package.
`invert_didymos_sectors.m`	The same with a 36-sector grid, the background held at the bulk density and regularization by the discrepancy principle.
`invert_received_data.m`	Invert another team's package (SBTK format) or a gravity file in the PDS SHADR layout.
`compare_didymos.m`, `compare_dimorphos.m`	Compare an inversion with the hidden model, by component and on the density grid.
`rob_data.m`	Read a ROB package (`shgrav.txt`, `moi.txt`) with the receiving team's uncertainties.
`analyze_rob.m`, `rob_candidates.m`	Read a ROB package and rank every candidate model by AIC: how the models of the ROB runners were chosen (Testing with ROB data).
`rob/data<n>/`	One runner per ROB solution (components, sector grid, point masses) and the script of each package's figures and tables (Code of the ROB solutions).
`rob_surface_point.m`	What the Dimorphos surface point adds: a buried sphere under it.

Settings

Setting	Didymos (agreed by the makers, 21 October 2026)	Dimorphos
Shape model	DART v003, 1.165 m sampling, 3,145,728 facets	DART v004 (before the impact), 0.243 m sampling, 3,145,728 facets
Bulk density	2600 kg/m³	1534 kg/m³
Mass	5.299121×10^{11} kg (GM $3.5367926256900638 \times 10^{-6}$ km³/s²)	2.700×10^{9} kg
Reference radius	0.41 km	0.10 km
Degrees the makers send	1 to 10	1 to 10
Degrees the receiving team fits	1 to 5	1 to 3
Density-grid spacing	0.020 km	0.005 km
Frame of the packages	the frame and origin of the DART shape model file	the frame and origin of the DART shape model file

Where each of these values comes from, and the alternatives: Didymos and Dimorphos values. A simulation on the full-resolution models takes 10 to 25 minutes: the geometry of 3 million facets to degree 10 takes about a minute and 4.5 GB, and a simulation repeats it until the center of mass settles.

Simulation runs

A simulation runner has four sections:

1. **NAME AND DELIVERY**: the run name, the package number n, and the inertia quantities the package contains (any of `Ixx` … `Iyz`, `libration`, or none; `sbtk.data.select_moi`).
2. **INTERIOR**: components and random field (Density distributions), in the axes of the DART shape-model file with positions measured from the center of figure (`cof = body.centroid_km`), not from the origin of the file, which is 57 m from it for Didymos (Where to place components). The torus lies about the z-axis through `cof`. `align_with_spin_axis` names the components whose axis is set along the axis of the largest moment of the whole true model (Didymos: `"torus"`, which forms about the spin axis; Dimorphos: none), with `sbtk.interior.align_with_spin_axis`.
3. **FRAME OF THE PACKAGES**: `"shape"`, as the HLP6 makers agreed: the frame and origin of the DART shape model file as distributed, with the gravity field (degree 1 included) and the moments of inertia about that origin. The alternatives `"spin"` and `"principal"` move everything to the center of mass of the true model and rotate it into its spin or principal axes (Frames of a gravity field).
4. **RUN**: `sbtk.interior.create` and `sbtk.interior.align_with_spin_axis`; `sbtk.data.simulate` (the true model and its exact values, without uncertainties); `sbtk.data.select_moi` and, for `"shape"`, `sbtk.data.expand_about` to the origin of the file (for `"spin"` or `"principal"`, `sbtk.gravity.center_of_mass_frame` and `sbtk.body.to_frame`, `sbtk.interior.to_frame`, `sbtk.data.to_frame`, with the shape model moved and

rotated: The shape model of a package); then `sbtk.io.report_simulation`, `sbtk.exchange.write_data` (a package of exact values: no 1σ columns, no covariance file, Exchange packages) and `sbtk.exchange.write_model`.

The run folder separates what may be shared from what may not:

Simulation run folder of projects/hera_hlp6

```
results/hera_hlp6/<date_time>_<name>/
  log.txt                          everything printed, including the true model
  redacted/<team>-DATA<n>-<body>/  the measurement package to share
  unredacted/<team>-MOD<n>-<body>/ the interior-model package, shared after the estimates
  unredacted/                      measurements.*, truth.*, settings.json, figures/
```

Only the package folder in `redacted/` is shared: `log.txt` and everything in `unredacted/` contain the true model.

Maker models

`examples/make_didymos_1.m` to `make_dimorphos_3.m` are six true models made the same way, one script each, into `results/didymos_models/` and `results/dimorphos_models/`, as packages 1 to 6. Didymos and Dimorphos share one numbering because a solution name carries only the number; the T1 demo sets of the simulate scripts are 7 and 8. They send exact values of every degree to 10, normalized and unnormalized, and I_{xx}, I_{yy}, I_{zz}, in the frame and about the origin of the DART shape model file (degree 1 and $\bar{C}_{21}$, $\bar{S}_{21}$ are not zero). The packages name that file but do not include it.

The interiors of these models are not described on this site, which is public, until the true models are released; `projects/hera_hlp6/README.md` in the repository lists them. `examples/make_didymos_homogeneous.m` makes the reference the makers compare, as their cross-validation asks (Didymos and Dimorphos values): Didymos at 2600 kg/m³, every degree to 10 and all six inertia elements, in the same frame and about the same origin, and the potential and acceleration at the 20 agreed points (`sbtk.exchange.write_potential`; package `UCB-DATA0-Didymos-homogeneous`). Its degree 1 gives the center of figure. If two makers compute the same homogeneous field from the same shape model file, they work in the same frame with the same conventions. `tests/integration/test_maker_scripts.m` runs `make_didymos_3`, `make_dimorphos_3` and `make_didymos_homogeneous` and checks their packages.

Inversion runs

`invert_didymos.m` reads the redacted package of the newest simulation of the given name with `sbtk.exchange.locate` and `sbtk.exchange.read_data`, exactly as another team receives it. Its coefficients, degree 1 included, and its moments of inertia are about the origin of the shape model file, and the inversion predicts the model's about the same point, with no change of frame (Density estimation). `hlp6_uncertainties.m` then gives the exact values the receiving team's uncertainties (the Hera radio-science precision, truncating the coefficients to the degree it covers). Its model places the core and the torus at the center of figure of the package's shape model, as a receiving team would, and estimates the torus axis and height (Estimated geometry): `estimate_axis = true`, `estimate_position = [false false true]`, with a priori 1σ of 1° and 10 m and limits of 10° and 50 m. It writes `fit.*`, `homogeneous.txt`, figures and the solution package `<team>-SOL_<data team><n>/`. `invert_received_data.m` does the same for a package from another team, or, with the commented lines, for a SHADR-layout gravity file, for which the shape model and the point the coefficients are about must be given.

Comparison runs

`compare_didymos.m` loads the newest simulation and inversion of the given names and the body from the simulation's measurement package (so that all three are in the frame of the packages), compares the fit with the true model (`sbtk.compare.with_truth`) and compares the solution package's density grid with the interior-model package's (`sbtk.compare.grids`), which is the comparison every team's solution receives.

Example runs

The delivered interiors (named T1) include a boulder and a random density field that the delivered inversion models do not contain. Weighted by the uncertainties of the end-to-end simulation, the χ^2 test rejects the inversion models of both bodies, and the estimated densities are biased accordingly. At this precision an inversion model has to include structure down to the scale of such features to pass the test (Goodness of fit). For Didymos T1, whose torus is aligned with the spin axis of the true model (3.9° from the z-axis of the DART file), the model with the torus about the z-axis and fixed geometry has a reduced χ^2 of 157 and $R^2 = 0.49$. Estimating the torus axis and height lowers χ^2 to 49 but R^2 to -1.92: the torus moves 50 m, to its limit, and tilts 6.8° to take up the signal of the boulder, which the model lacks (Estimated geometry). On the full-resolution shape model the run takes about 25 minutes. For Dimorphos, an ellipsoidal core and the regolith give a reduced χ^2 of 47 and $R^2 = 0.04$ (on the 49,152-facet version of the shape model).

Didymos and Dimorphos values

Every value SBTK uses to model Didymos and Dimorphos, where it comes from and where to find it. Each value is one of:

- **agreed**: the value the Hera blind-test group uses;
- **server**: from a file on the Hera working-group server (folder paths below are relative to its top folder);
- **published**: from a paper or a public data product (references at the end of the page);
- **derived**: computed from other values on this page;
- **assumed**: an SBTK choice, not a measurement.

The HLP6 makers agreed on the Didymos values on 21 October 2026 [HLP6 cross-validation 2026]: the 1.165 m shape model in its own frame, bulk density 2600 kg/m³, reference radius 410 m, degrees to 10, and the potential and acceleration at 20 given points. There are no agreed Dimorphos values yet.

The values are set in the repository in `projects/hera_hlp6/hlp6_didymos.m` and `projects/hera_hlp6/hlp6_dimorphos.m` (shape model, mass or bulk density, reference radius, degree, grid spacing), `projects/hera_hlp6/hlp6_uncertainties.m` (the receiving team's uncertainties) and the maker scripts `examples/make_didymos_1.m` to `make_dimorphos_3.m`.

Shape models

The shape models are the DART stereophotoclinometry (SPC) models, Didymos version 3 and Dimorphos version 4, from before the DART impact [Daly et al. 2023; Daly et al. 2024]. They are published in two places:

- **PDS**: NASA Planetary Data System, Small Bodies Node, bundle `pds4-dart_shapemodel-v1.0` [DART shape models 2023]. Each collection holds the global model at four samplings (OBJ files) with the digital terrain maps (FITS):
- Didymos: https://pds-smallbodies.astro.umd.edu/holdings/pds4-dart_shapemodel-v1.0/data_derived_didymos_model_v003/
- Dimorphos: https://pds-smallbodies.astro.umd.edu/holdings/pds4-dart_shapemodel-v1.0/data_derived_dimorphos_model_v004/
- **ESA SPICE**: the full-resolution models only, as OBJ and digital shape kernel (`.bds`) files, https://spiftp.esac.esa.int/data/SPICE/HERA/kernels/dsk/. The working-group server has the same two files in `Hera_WG/End-to-End Test/Shape/`, and the agreed Didymos file in `Hera_WG/End-to-End Test/HLP6G BlindTest/inputs/Reference models/Didymos/`.

The volume and the volume-equivalent diameter $D = (6V/\pi)^{1/3}$ are **derived** from each file.

`sbtk.shape.bundled` name	File (in `data/shapes/`)	Sampling	Facets	Volume (km³)	D (m)	Where it comes from
`didymos`	`didymos_g_9309mm_spc_obj_0000n00000_v003.obj`	9.3 m	49,152	0.20336	729.6	**published**: PDS, Didymos collection (3.4 MB), the file unchanged. The version the Hera radio-science (RSE) team uses; SBTK's blind-test packages until 20 October 2026.
`didymos_highres`	`g_01165mm_spc_obj_didy_0000n00000_v003.obj`	1.165 m	3,145,728	0.20381	730.1	**agreed**, **published**: the HLP6 reference model of Didymos. ESA SPICE (162 MB) and the server. PDS has the 1.165 m model as `didymos_g_1165mm_spc_obj_0000n00000_v003.obj`. MD5 `0a585bc46d19f29778d7721970958723`. Not in git.
`didymos_every8`	`g_01165mm_spc_obj_didy_0000n00000_v003_every8.obj`	9.3 m	49,152	0.20375	730.1	**derived**: every 8th grid point of `didymos_highres` (`tools/make_reduced_shapes.m`)
`dimorphos`	`g_00243mm_spc_obj_dimo_0000n00000_v004_every8.obj`	1.94 m	49,152	0.0017597	149.8	**derived**: every 8th grid point of `dimorphos_highres`. SBTK's blind-test Dimorphos packages until 20 October 2026.
`dimorphos_highres`	`g_00243mm_spc_obj_dimo_0000n00000_v004.obj`	0.243 m	3,145,728	0.0017602	149.8	**published**, **assumed**: ESA SPICE (162 MB) and the server. SBTK's blind-test Dimorphos packages (like the agreed Didymos model, the full resolution). PDS has it as `dimorphos_g_0243mm_spc_obj_0000n00000_v004.obj`. MD5 `d8263a016169434e09fd10f8b065f0a1`. Not in git.
(not in SBTK yet)	`dimorphos_g_1940mm_spc_obj_0000n00000_v004.obj`	1.94 m				**published**: PDS, Dimorphos collection (3.4 MB). The 1.94 m Dimorphos model the RSE team uses.

Points to know:

- **Use the same file as the other teams.** The agreed Didymos file is the 1.165 m model. The 9.3 m file is a separate product, not a subsample of it: its volume is 0.22% smaller, its center of figure is 0.35 m away, and its degree-2 coefficients differ by up to 3×10^{-4}, about the size of the measurement uncertainties. Data made with one file and inverted with another carry that difference as an error. `data/shapes/README.md` compares all the versions.
- **Packages do not include the file.** A package made with one of these files names it (Exchange packages).
- **Dimorphos.** The models are from before the DART impact, which may have changed the shape.
- **Frame.** Each file has its own body-fixed frame, z along the spin axis. The HLP6 makers keep it ("native reference frame of the shape model, not barycentric"; Frames of a gravity field). The center of figure is 56.765 m from the origin of the agreed Didymos file, at (+15.723, +42.487, −34.204) m, and 0.751 m from that of the Dimorphos file, at (−0.741, −0.117, −0.007) m.

Mass and GM

Quantity	Didymos	Dimorphos	Kind	Source
Bulk density	2600 kg/m³		**agreed** (Didymos)	[HLP6 cross-validation 2026]

Quantity	Didymos	Dimorphos	Kind	Source
Mass M	5.299121 × 10^{11} kg	2.700 × 10^{9} kg	**derived** (Didymos: bulk density × volume of the agreed file); **server** (Dimorphos)	Dimorphos: the mass the RSE end-to-end test uses, from the Didymos Reference Model, issue 6.2 (`Hera_WG/End-to-End Test/Didymos Reference model/Didymos Reference Model issue 6_2_clean.pdf`) and [Makadia et al. 2026]. The group's end-to-end input values are in `Hera_WG/End-to-End Test/Endtoendtest_inputs.xlsx`.
Gravitational constant G	6.67430 × 10^{-20} km³ kg^{-1} s^{-2}	same	**published**	CODATA 2018 [Tiesinga et al. 2021] (`sbtk.units.G`)
$GM = GM$	3.5367926256900638 × 10^{-8} km³/s²	1.802061 × 10^{-10} km³/s²	**agreed** (Didymos), **derived**	Didymos: [HLP6 cross-validation 2026]. G × 2600 kg/m³ × the volume of the agreed file gives the same value to 13 digits, so the group uses CODATA 2018 too.
Mass used until 20 October 2026	5.273 × 10^{11} kg		**server**	the RSE end-to-end test (Reference Model 6.2)
GM estimated in the RSE end-to-end test	3.5192927 × 10^{-8} ± 4.2724 × 10^{-13} km³/s²	1.8049683 × 10^{-10} ± 1.8642 × 10^{-13} km³/s²	**server**	header lines of `Didymos_gravity.tab` and `Dimorphos_gravity.tab` (`Hera_WG/End-to-End Test/Radio Science/Gravity/`; copies in `data/hera_rse/`). `RSE_solution_MOIs_Deg2Gravity_Didymos.txt` gives 3.519242 × 10^{-8}. These are test estimates, scattered about the value the test was made with, not values to use.
Mass from the RSE GM	5.27290 × 10^{11} kg	2.70436 × 10^{9} kg	**derived**	GM / G

The maker packages give GM and the mass (GM / G). `sbtk.exchange.read_data` takes the mass from GM, so a package from a team that used another G gives that team's mass.

Bulk density

The bulk density is the background density of a homogeneous model (Density distributions). For Didymos it is **agreed**, 2600 kg/m³, and sets the mass. For Dimorphos it is **derived**: 2.700 × 10^{9} kg over the volume of the 0.243 m model, 1534 kg/m³.

Before the agreement SBTK took the mass of the RSE end-to-end test, 5.273 × 10^{11} kg, which gives 2593 kg/m³ with the 9.3 m file and 2587 kg/m³ with the 1.165 m file.

Gravity field settings

Setting	Didymos	Dimorphos	Kind	Source
Reference radius R_{ref}	0.41 km	0.10 km	**agreed** (Didymos), **server** (Dimorphos)	Didymos: [HLP6 cross-validation 2026]. Dimorphos: the RSE `.tab` file. The RSE Didymos file uses 0.43 km; its coefficients and their 1σ convert to 0.41 km (below). The RSE inertia files state "0.4 m" and "0.1 m", a labeling error.
Degree of the receiving team's fit	5	3	**server**	the degrees of the RSE `.tab` files; finders truncate to them
Degree the makers send	10	10	**agreed** (Didymos), **assumed** (Dimorphos)	[HLP6 cross-validation 2026]: `max_l` 10
Expansion point and axes	origin and axes of the shape model file	same	**agreed**	the HLP6 makers, 20 and 21 October 2026 (Exchange packages)
Inertia data the makers send	I_{xx}, I_{yy}, I_{zz} about the same origin	same	**assumed**	Hera blind test

Coefficients given with another reference radius R' convert as $\bar{C}'_{lm} = \bar{C}_{lm}\,(R / R')^{l}$ (Gravity coefficients).

Potential and acceleration at the cross-validation points

The makers also give the gravitational potential and acceleration of the homogeneous Didymos at 20 points [HLP6 cross-validation 2026], in m in the frame of the agreed file (`projects/hera_hlp6/cross_validation/points.csv`):

Points	Distance from the origin of the file
1: on the surface at latitude 45°, longitude 45°	332 m
2: near an equilibrium point of the homogeneous body	360 m
3–8, 9–14, 15–20: six points each on spheres of radius 1.2, 1.5 and 2.0 R_{ref} (Fibonacci lattice)	492, 615, 820 m

SBTK computes them exactly from the polyhedron of the shape model (Potential and acceleration at points). The sphere about the origin that encloses Didymos has a radius of 449.7 m, so points 1 and 2 are inside it, where the spherical-harmonic expansion does not converge. At 1.2 R_{ref} the expansion to degree 10 differs from the exact potential by up to 1.5 × 10^{-4} of it; a team that evaluates the coefficients there will differ by that much. At point 2 the gravitational acceleration, 2.40 × 10^{-4} m/s², points to the center of mass and equals the centrifugal acceleration of the 2.26 h rotation about an axis through it: the point is an equilibrium point, as stated.

Uncertainties of the receiving team

The makers send exact values. SBTK's inversion runners give them the precision of the RSE end-to-end test (`projects/hera_hlp6/hlp6_uncertainties.m`), read from the RSE files (**server**, `Hera_WG/End-to-End Test/Radio Science/Gravity/`, copies in `data/hera_rse/`, whose `README.md` describes them; the

files are marked "test product only, not for scientific use"). The blind-test group's precision files are in `Hera_WG/End-to-End Test/HLP6G BlindTest/inputs/uncertainties/` (`UniBo-PREC1-Gravity_MOIs`).

Quantity	Didymos	Dimorphos	Source file
Gravity coefficients, 1σ	degree 2: 8 × 10^{-6} ($\bar{C}_{20}$) to 1.3 × 10^{-4} ($\bar{C}_{21}$); degrees 3–5: 1 × 10^{-5} to 5 × 10^{-5}; center of mass about 0.1 m	degree 2: 4 × 10^{-6} ($\bar{C}_{22}$) to 4 × 10^{-4} ($\bar{C}_{21}$); degree 3: about 1 × 10^{-2}	`Didymos_gravity.tab`, `Dimorphos_gravity.tab` (one 1σ per coefficient, degree 1 included)
I_{xx}, I_{yy}, I_{zz}, 1σ	2.4853 × 10^{8}, 2.4852 × 10^{8}, 2.4853 × 10^{9} kg km²	1.1081 × 10^{4}, 1.1359 × 10^{4}, 1.4283 × 10^{4} kg km²	`RSE_solution_MOIs_Deg2Gravity_Didymos.txt`, `..._Dimorphos.txt` (labeled kg m², values in kg km²)
I_{xy}, I_{xz}, I_{yz}, 1σ	4.5012 × 10^{6}, 1.6210 × 10^{7}, 4.5518 × 10^{6} kg km²	1.3759 × 10^{3}, 1.3099 × 10^{4}, 8.6039 × 10^{3} kg km²	same
Libration ratio, 1σ	0.02	0.02	**assumed** (placeholder)

The Didymos coefficients and their 1σ are given for R_{ref} = 0.43 km; `sbtk.data.use_uncertainties` converts the 1σ of degree l to the agreed 0.41 km by the factor $(0.43 / 0.41)^{l}$.

The RSE fields are given in principal-axis frames (for the homogeneous body, the axis of the smallest moment is turned 32.2° (Didymos) and 7.8° (Dimorphos) from the shape-model x-axis toward $-y$, with the axis of the largest moment tilted 3.8° and 0.7° from the shape-model z-axis). Their 1σ table is applied in the axes of the packages without that rotation, which is an approximation (Frames of a gravity field).

The RSE fields are also about the center of mass, while the packages are about the origin of the shape model file. The covariance is therefore moved to that point exactly, $K' = TKT^{\mathsf{T}}$ (`sbtk.data.use_uncertainties`): moving a field to another point mixes the degrees, so the uncertainty of the center of mass (degree 1) enters the other coefficients. About the origin of the file, $\bar{C}_{20}$ of Didymos (UCB-DATA1) has 2.3 times the RSE 1σ and some degree-3 coefficients up to 1.8 times; $\bar{C}_{22}$ of Dimorphos (UCB-DATA4, center of mass 2.5 m from the origin) has 27 times the RSE 1σ, which is only $4{\times}10^{-6}$. The inertia elements, about the same point, get the variance the center of mass adds through the parallel-axis theorem: negligible for Didymos, a 1σ 12 to 19% larger than the RSE value for Dimorphos. A team that applies the RSE 1σ unchanged to coefficients about the file origin weights those coefficients too much. Uncertainties and the expansion point derives these numbers.

Modeling choices

Setting	Didymos	Dimorphos	Kind
Density-grid spacing of model and solution packages	20 m (about 25,000 cells)	5 m (about 15,000 cells)	**assumed**
Upper bound on a component's density in the inversion	4500 kg/m³	4500 kg/m³	**assumed** (`opts.rho_max_kgm3`)

References

- **Daly et al. 2023.** R. T. Daly et al., "Successful kinetic impact into an asteroid for planetary defence," *Nature* 616, 443–447 (2023). doi:10.1038/s41586-023-05810-5.
- **Daly et al. 2024.** R. T. Daly et al., "An Updated Shape Model of Dimorphos from DART Data," *The Planetary Science Journal* 5(1), 24 (2024). doi:10.3847/PSJ/ad0b07.
- **Makadia et al. 2026.** R. Makadia, S. R. Chesley, D. Herald, D. Farnocchia, N. L. Chabot, S. P. Naidu, A. S. Rivkin, A. Siakas, D. Souami, P. Tanga, S. Tsavdaridis, K. Tsiganis, S. Bouquillon, S. Eggl, "Direct detection of an asteroid's heliocentric deflection: The Didymos system after DART," *Science Advances* 12(10), eaea4259 (2026). doi:10.1126/sciadv.aea4259.
- **Tiesinga et al. 2021.** E. Tiesinga, P. J. Mohr, D. B. Newell, B. N. Taylor, "CODATA recommended values of the fundamental physical constants: 2018," *Reviews of Modern Physics* 93(2), 025010 (2021). doi:10.1103/RevModPhys.93.025010.
- **DART shape models 2023.** T. Daly, O. Barnouin, C. Ernst, H. Nair, R. Espiritu, D. Waller, "DART (Double Asteroid Redirection Test) Digital Terrain Maps for didymos_model_v003, Version 1.0," NASA Planetary Data System (2023). doi:10.26007/bm57-x327. The same authors, "... for dimorphos_model_v004, Version 1.0," NASA Planetary Data System (2023). doi:10.26007/0nss-vd15. Bundle `urn:nasa:pds:dart_shapemodel`, https://pds-smallbodies.astro.umd.edu/holdings/pds4-dart_shapemodel-v1.0/.
- **HLP6 cross-validation 2026.** Hera working group, "Hera Blind-Test interior HLP6: Homogeneous Didymos data for makers cross-validation test" (README of the makers, 21 October 2026). On the server with the agreed shape model (`Hera_WG/End-to-End Test/HLP6G BlindTest/inputs/Reference models/Didymos/`); copy in `projects/hera_hlp6/cross_validation/`.
- **ESA SPICE kernels for Hera**, shape models (digital shape kernels and OBJ files): https://spiftp.esac.esa.int/data/SPICE/HERA/kernels/dsk/.
- **Hera working-group server**, folder `Hera_WG/End-to-End Test/`: `Shape/` (full-resolution shape models), `Radio Science/Gravity/` (RSE end-to-end gravity fields and inertia), `Didymos Reference model/` (Didymos Reference Model, issue 6.2), `HLP6G BlindTest/` (the blind-test packages), `Endtoendtest_inputs.xlsx` (input values of the end-to-end test).

Testing with ROB data

The Royal Observatory of Belgium (ROB), one of the teams that make interiors for the HLP6 blind test, distributed six measurement packages of hidden interiors: three of Didymos (ROB-DATA1 to 3) and three of Dimorphos (ROB-DATA4 to 6). This section reads each package with the procedure of Reading the gravity coefficients and fits three kinds of solution to the same measurements: a model with components chosen from that reading, a sector grid and point masses. Each is made by its own script (Code of the ROB solutions). It then tests every solution on what it was not fitted to: ROB's exact coefficients to degree 10 and, for Dimorphos, the gravity at one point on the surface. The hidden interiors are not known to us.

The packages

Each package has

- `shgrav.txt`: fully normalized gravity coefficients of degrees 0 to 10 about the origin of the DART shape-model file, in its axes, with ROB's reference radius and GM in the header;
- `moi.txt`: the six inertia elements $I / (MR_{\mathrm{ref}}^2)$ about the same origin;
- for Dimorphos also the gravitational potential and acceleration at one point on the surface, as the GRASS gravimeter on a lander would measure them.

The values are exact (the 1σ columns of ROB's coefficients are 10^{-8} to 10^{-5}, far below what a mission measures; those of the inertia elements are 10% of each value). The pages weight them with the Hera uncertainties instead (Uncertainties). The headers give each package's mass and reference radius:

Package	Body	Shape model	Mass (kg)	Bulk density (kg/m³)	R_{ref} (km)
ROB-DATA1	Didymos	DART v003, 1.165 m	5.30417×10^{11}	2602.5	0.41
ROB-DATA2	Didymos	DART v003, 1.165 m	5.30966×10^{11}	2605.2	0.41
ROB-DATA3	Didymos	DART v003, 1.165 m	5.29912×10^{11}	2600.0	0.41
ROB-DATA4	Dimorphos	DART v004, 0.243 m	2.71071×10^{9}	1540.0	0.09
ROB-DATA5	Dimorphos	DART v004, 0.243 m	2.70889×10^{9}	1539.0	0.09
ROB-DATA6	Dimorphos	DART v004, 0.243 m	2.80558×10^{9}	1593.9	0.09

Uncertainties

The packages are weighted with the precision of the Hera radio-science end-to-end simulation, as in the blind test (`projects/hera_hlp6/rob_data.m`, which calls `hlp6_uncertainties.m`; Didymos and Dimorphos values):

- every coefficient's 1σ from the RSE gravity field, with its full covariance moved from the RSE center of mass to the origin of the shape-model file (Uncertainties and the expansion point) and scaled to the package's reference radius;
- the coefficients truncated to the degrees the RSE field has, and every one of them fitted: **degrees 1 to 5 for Didymos** (35 coefficients) and **1 to 3 for Dimorphos** (15). The fits do not use $\bar{C}_{00}$: the mass is the package's. For Dimorphos the 1σ of degree 3, about 3×10^{-2}, is 20 to 600 times its anomaly, so degree 3 is fitted but constrains nothing. For Didymos degree 5 is resolved on ROB-DATA2 and 3 (up to 6σ) and not on ROB-DATA1;
- each inertia element's 1σ from the RSE inertia solution, with the variance the center of mass adds about the origin. For Didymos they are about 2.8×10^{-2} of MR^2 and carry no information; for Dimorphos about 5×10^{-4}, and they do.

What the six packages show

- **A model with a few components, chosen by reading the coefficients, fits with the fewest parameters.** It ranks first by AIC on ROB-DATA1, 3, 4 and 5: by 37 and 18 on Didymos, by 4.6 on ROB-DATA4, and by 1.9 on ROB-DATA5, a tie with the point masses. On ROB-DATA6 it ties with the point masses (0.1 apart); the sector grid is 2.1 behind. On ROB-DATA2 no model of a core, a torus and a layer fits; a dense and a light sphere with four compact masses does, and ties with the sector grid and the point masses.
- **Fitting best is not the same as being right above the fitted degrees.** There ROB's exact values side with the point masses on ROB-DATA2, 3 and 6, and with the model with components on ROB-DATA1, 4 and 5. The degrees the measurements resolve, to 4 or 5 for Didymos and to 2 for Dimorphos, fix where the anomaly is; they do not fix how compact it is.
- **Given no shapes, the point masses often find the geometry of the model with components.** On ROB-DATA1 they find its light column along the spin axis and put the mass at the two ends of one equatorial axis. On ROB-DATA3 they put a dense center inside a light equatorial ring. On ROB-DATA4 they put the excess toward –y, where the core is. On ROB-DATA5 they find the core's size and elongation but not its side. Where they agree with the model with components, the measurements, not the choice of shapes, placed the anomaly.
- **Where the two differ, the measurements allow both, and the truth goes either way.** On ROB-DATA5 the core's offset to +x, which the point masses leave out because degree 1 is measured at only 0.7σ, is right. On ROB-DATA6 the point masses' light region at the center is closer to the truth than the light core to one side.
- **Sector grids fit an anomaly spread over a direction and depth (ROB-DATA2), but cannot make a ring (ROB-DATA3).** Above the fitted degrees they do worse than constant density on ROB-DATA1, 3 and 5.
- **For Dimorphos the moments of inertia and one surface point matter.** At the Hera precision the moments carry no information for Didymos. For Dimorphos they are as strong as degree 2 (ROB-DATA4 and 6). The GRASS point decides where it lies over the anomaly: on ROB-DATA4 the compact core predicts it 3.7% too high and the point masses 9.3% too low, where constant density is 13.3% low.

Results

Package	What the measurements show	Best by AIC	Tied (within 2 in AIC)	Closest to ROB's exact values above the fitted degrees
ROB-DATA1	mass from the spin axis to two ends of an equatorial axis, more at –x	a light column along the spin axis, two dense equatorial ellipsoids	none	components (0.92; point masses –0.40)
ROB-DATA2	center of mass 2.3 m south, degree 2 tilted 58°, a large degree 3	sector grid 1 × 4 × 8	components, point masses	point masses (0.17; sector grid 0.05, components –0.20)
ROB-DATA3	J_2 570σ lower and J_4 95σ higher: a light ring about the spin axis	dense core, light torus, dense layer	none	point masses (0.92; components 0.79)
ROB-DATA4	mass toward ±y, more at –y; the moments	a dense core 50 m toward –y, a torus, a layer	none	components (0.81; sector grid 0.25)
ROB-DATA5	mass toward ±x, close to the x-axis	one dense sphere 28 m toward +x	point masses	components, the same sphere (0.91; point masses 0.79)
ROB-DATA6	the moments 32–62σ larger: light inside, dense outside	point masses	components	sector grid and point masses (0.50 and 0.46; components –0.72)

The fitted degrees are 1 to 5 for Didymos and 1 to 3 for Dimorphos. Above them: the fraction of ROB's exact coefficients a solution reproduces over the degrees not fitted, 6 to 10 and 4 to 10 (1: all; 0: no better than constant density; negative: worse).

Point masses: sensitivity or equal weighting

Point masses can be weighted in two ways (Point masses). Both were fitted to every package, each with the regularization parameter of lowest AIC:

Package	AIC, sensitivity	AIC, equal	Exact values above the fitted degrees, sensitivity	Exact values above the fitted degrees, equal	GRASS acceleration, sensitivity	GRASS acceleration, equal
ROB-DATA1	62.1	66.0	–0.40	–3.03		
ROB-DATA2	65.1	67.2	0.17	0.24		
ROB-DATA3	68.8	67.9	0.92	0.95		
ROB-DATA4	17.7	17.3	–0.33	0.30	–9.3%	–6.6%
ROB-DATA5	10.3	12.4	0.79	0.64	–0.6%	–0.4%
ROB-DATA6	16.6	17.4	0.46	0.65	–0.7%	+1.6%

Neither weighting is better everywhere. The AIC of the two differs by less than 4 on every package. Above the fitted degrees sensitivity weighting is far better on ROB-DATA1 and better on ROB-DATA5, equal weighting far better on ROB-DATA4 and better on ROB-DATA6; on ROB-DATA2 and 3 they are close. The pages show sensitivity weighting, which spreads the anomaly over depth instead of putting it under the surface; each page's `weighting.md` table in the results folder has both.

How to read these pages

Every package page has the same six sections: what the measurements show, where the mass is, how well each solution fits, the model with components, the solutions side by side, and the test against ROB's exact values (for Dimorphos also against the GRASS point).

C – C_CD. Each measured coefficient minus the coefficient of the same shape at constant density (CD), both about the center of figure: what the interior adds to the gravity field. 0 is constant density. The bars are ±1σ of the Hera radio-science solution (Uncertainties). Zonal terms are shown as $J_l = -\bar{C}_{l0}$; degree 1 is the center-of-mass offset ($\bar{C}_{11}, \bar{S}_{11}, \bar{C}_{10} = x, y, z$ over $\sqrt{3}\ R_{\mathrm{ref}}$). The sign of a coefficient says where mass was added or taken away: J_2 higher, toward the equator; $\bar{C}_{22} > 0$, toward the ends of the x-axis; $\bar{C}_{21} > 0$, toward +x in the north and –x in the south (Reading the gravity coefficients).

The maps. Each resolved degree of C – C_CD, up to the highest with a measurement more than 3σ from constant density, drawn as a pattern on the sphere: red where a mass added near the surface in that direction would produce it, blue where a mass taken away would (`sbtk.plot.anomaly_map`).

The kinds of solution. All are fitted to the same measurements, every coefficient the Hera uncertainties cover and I_{xx}, I_{yy}, I_{zz}:

Kind	What it assumes	Page
Components	a few shapes chosen by reading the coefficients: a core, a torus, layers, spheres, ellipsoids, polar caps; their densities and some positions fitted, sizes fixed or searched on a grid, compact masses added where they pass the likelihood-ratio test	Models with components, Searching the geometry
Sector grid	32 or 64 blocks of latitude, longitude and depth, each with its own density	Sector grids
Point masses	about 2,700 masses on a grid	Point masses

Relative density. In the side-by-side figures, the density minus the bulk density, so that every kind is drawn on the same scale. In the component tables and the three-dimensional views, a component's density minus the background density, the quantity the fit estimates.

Reduced χ² and p. χ^2 is the sum over the measurements of ((solution – measured)/σ)², with the correlations of the coefficients. The reduced χ^2 divides it by the degrees of freedom; about 1 means the solution fits to the noise. p is the probability that noise alone gives a χ^2 this large; $p \geq 0.05$ is consistent with the

measurements. The measurements are ROB's exact values weighted with the Hera uncertainties, without noise added, so a reduced χ^2 well below 1, as for every solution on Dimorphos, means a solution matches them more closely than the uncertainties require.

Parameters and AIC. The number of parameters is how many independent quantities a solution takes from the measurements: one per density or coordinate of a model with components, and one per size chosen on a grid; for the regularized sector grids and point masses, the effective number, the trace of the data resolution matrix. The Akaike information criterion, AIC = χ^2 + 2 × effective parameters, ranks solutions of different kinds on the same terms: lower is better, a difference below 2 is a tie, above 10 a strong preference (Goodness of fit). Sector grids and point masses are regularized with the parameter of lowest AIC.

Exact values. ROB's packages give exact coefficients to degree 10; the solutions are fitted to degrees 1 to 5 for Didymos and 1 to 3 for Dimorphos. Each solution is tested on all ten: the fraction of each degree it reproduces, 1 – |solution – exact|²/|exact|². 1 is all of it, 0 no better than constant density, negative worse. The degrees above the fitted ones, 6 to 10 for Didymos and 4 to 10 for Dimorphos, are a test on coefficients no solution was fitted to.

GRASS. For Dimorphos, ROB gives the acceleration at one point on the surface, as the GRASS gravimeter on a lander would measure it. Each solution's prediction is compared in magnitude and direction.

Method

For every package the runners in `projects/hera_hlp6/rob/` fit the three solutions and make the figures and tables of its page (Code of the ROB solutions); `projects/hera_hlp6/analyze_rob.m` chose their models:

1. **Reading** (`sbtk.interpret.anomaly`, `sbtk.interpret.geometry`): C – C_CD coefficient by coefficient, the center-of-mass offset, the axes of degree 2, each degree as a map.
2. **Models with components** chosen from the reading (`rob_candidates.m`): a core placed by its χ^2 profile along the center-of-mass offset and then by Gauss–Newton, layers, tori, ellipsoids, a light column along the spin axis, and polar caps and spheres where the reading put mass; and two searches (`sbtk.search.run`): a core, a torus and a layer with their sizes on a grid, and a dense and a light sphere started where the point masses put the mass; both with compact masses added while they pass the likelihood-ratio test. The model of lowest AIC is the package's model with components. The fits run on the 49,152-facet versions of the shape models with the measurements moved to them (`sbtk.data.to_reduced_shape`).
3. **Sector grids** (1 × 4 × 8 and 2 × 4 × 8; the one of lower AIC) and **point masses** (both weightings), each with the regularization parameter of lowest AIC.
4. **Comparison** of the three solutions, coefficient by coefficient, on two planes through the body, against ROB's exact values to degree 10, and for Dimorphos at the GRASS surface point.

Code of the ROB solutions

Every solution on these pages is made by its own script in `projects/hera_hlp6/rob/`: for each package, one runner per kind of solution and one script for the figures and tables of its page. Change a model or an option in a runner and run it again; the figure script reads the newest run of each runner.

Script	What it does	Run folder in `results/hera_hlp6/`
`rob<n>_components.m`	fits the model with components chosen for the package; the header says why	`<date_time>_ROB-DATA<n>_components`
`rob<n>_sectors.m`	fits a sector grid with the Tikhonov parameter of the lowest AIC	`<date_time>_ROB-DATA<n>_sectors`
`rob<n>_point_masses.m`	fits about 2,700 point masses with the Tikhonov parameter of the lowest AIC; `weighting` is `"sensitivity"` or `"equal"`	`<date_time>_ROB-DATA<n>_point_masses_<weighting>`
`rob<n>_figures.m`	the figures and tables of the page, one section per figure	`<date_time>_ROB-DATA<n>_figures`

Run them in this order: the point masses (once with each weighting), the sector grid, the model with components (`rob2_components.m` starts from the point masses), the figures. The Didymos runners take seconds to a few minutes. Each Dimorphos runner adds the GRASS surface point: about a minute for the model with components and the sector grid, up to half an hour for the point masses.

The runners share a few functions in the same folder:

Function	What it does
`rob_package(number)`	reads ROB-DATA<number> with the Hera radio-science uncertainties: degrees 1 to 5 for Didymos, 1 to 3 for Dimorphos, and I_{xx}, I_{yy}, I_{zz}, on the 49,152-facet shape model; and ROB's exact values to degree 10
`rob_solution(name, kind, result)`	a fit or point masses in the form the figures read: the model, χ^2, the number of parameters, AIC, p
`rob_load(number, run)`	the solution of the newest run of a runner
`rob_grass(number, body_full, model)`	the acceleration a model predicts at the GRASS surface point, against ROB's
`rob_reproduced(a, exact, degrees)`	the fraction of ROB's exact anomaly a solution reproduces
`rob_tables(...)`, `rob_grass_table(...)`	the Markdown tables of a page

Every runner has the same sections. DATA reads the package. MODEL is the interior that is fitted. OPTIONS are those of `sbtk.invert.options`. RUN fits, reports in a new run folder and saves `solution.mat`. For Dimorphos, SURFACE POINT predicts the GRASS acceleration.

The models with components

Each package has its own model, chosen by reading the coefficients and ranked against the other candidates by AIC (see How the models were chosen).

MATLAB script: projects/hera_hlp6/rob/data1/rob1_components.m

```
%% ROB1_COMPONENTS  ROB-DATA1 (Didymos): a light column along the spin axis and two dense equatorial ellipsoids.
%
% The reading of the coefficients (documentation: Testing with ROB data > ROB-DATA1): J2 is
% higher and C22, S22 say mass has moved from the spin axis toward the two ends of one
% equatorial axis, at longitudes about -20 and 160 deg; degree 1 says the 160 deg end is
% heavier. A light column (a prolate ellipsoid about the spin axis through the center of figure)
% takes mass from along the axis; a dense ellipsoid at each end of the equatorial axis puts it
% there. Of the models tried (rob_candidates.m, ranked by analyze_rob.m) this one has the
% lowest AIC. Fits the background density, the three relative densities and the centers of
% the two ellipsoids to the coefficients of degrees 1-5 and Ixx, Iyy, Izz.
% Writes results/hera_hlp6/<date_time>_ROB-DATA1_components/: solution.mat (read by
% rob1_figures.m), fit.txt/.mat, homogeneous.txt, settings.json, log.txt, figures/.

clear; close all;
here = fileparts(mfilename('fullpath'));                  % this package's folder
rob  = fileparts(here);                                   % rob/: the functions the runners share
hlp6 = fileparts(rob);                                    % hera_hlp6/: rob_data.m, hlp6_uncertainties.m
addpath(rob, hlp6, fullfile(fileparts(fileparts(hlp6)), 'toolbox'));

%% DATA
number = 1;                                               % ROB-DATA1
[obs, body] = rob_package(number);                        % degrees 1-5 and Ixx, Iyy, Izz, Hera uncertainties

%% MODEL
cof = body.centroid_km;                                   % positions are from the center of figure
rm  = (3*body.volume_km3/(4*pi))^(1/3);                   % mean radius [km]
u   = [cosd(-20) sind(-20) 0];                            % the equatorial axis of degree 2
column = sbtk.interior.ellipsoid("column", cof, [0.25 0.25 0.70]*rm, [0 0 0], 0);
east   = sbtk.interior.ellipsoid("east", cof + 0.6*rm*u, [0.35 0.20 0.20]*rm, [-20 0 0], 0);
west   = sbtk.interior.ellipsoid("west", cof - 0.6*rm*u, [0.35 0.20 0.20]*rm, [160 0 0], 0);

east.estimate_position = true;
west.estimate_position = true;
components = [column east west];

%% OPTIONS
opts = sbtk.invert.options(moi_labels={'Ixx', 'Iyy', 'Izz'});
opts.background = "estimate";                             % the background density is fitted too

%% RUN
out  = sbtk.io.new_run("hera_hlp6", sprintf("ROB-DATA%d_components", number));
prob = sbtk.invert.setup(body, components, obs, opts);
fit  = sbtk.invert.run(prob);
sbtk.io.report_inversion(out, body, fit, sbtk.invert.homogeneous(prob));
solution = rob_solution("light column along the spin axis, two equatorial ellipsoids", "components", fit);
fprintf('AIC %.1f with %.0f parameters, reduced chi-square %.2f, p = %.2g\n', solution.aic, solution.parameters, ...
        solution.chi2_reduced, solution.p_value);

save(fullfile(out.folder, "solution.mat"), 'solution');
sbtk.io.finish_run(out);
```

MATLAB script: projects/hera_hlp6/rob/data2/rob2_components.m

```
%% ROB2_COMPONENTS  ROB-DATA2 (Didymos): a dense and a light sphere, and compact masses where the measurements ask for them.
%
% The reading of the coefficients (documentation: Testing with ROB data > ROB-DATA2): the center
% of mass is 2.3 m off center toward the south, degree 2 is tilted 58 deg from the equator and
% degree 3 is large. No model of a core, a torus and a layer fits (rob_candidates.m,
% analyze_rob.m). The point masses (rob2_point_masses.m: run it first) put the excess below the
% equator toward -x and -y and the deficit toward +y and the north. This model starts a dense
% and a light sphere at the centroids of the point masses' positive and negative mass and fits
% their densities and centers; then compact masses (spheres of a quarter of the mean radius,
% from a grid of candidate positions) are added one at a time while each passes the
% likelihood-ratio test at the 5% level, every position refitted each time (sbtk.search.run).
% Of the models tried this one has the lowest AIC. Fitted to the coefficients of degrees 1-5
% and Ixx, Iyy, Izz. A few minutes.
% Writes results/hera_hlp6/<date_time>_ROB-DATA2_components/: solution.mat (read by
% rob2_figures.m), search.txt, fit.txt/.mat, homogeneous.txt, settings.json, log.txt, figures/.

clear; close all;
here = fileparts(mfilename('fullpath'));                  % this package's folder
rob  = fileparts(here);                                   % rob/: the functions the runners share
hlp6 = fileparts(rob);                                    % hera_hlp6/: rob_data.m, hlp6_uncertainties.m
addpath(rob, hlp6, fullfile(fileparts(fileparts(hlp6)), 'toolbox'));

%% DATA
number = 2;                                               % ROB-DATA2
[obs, body] = rob_package(number);                        % degrees 1-5 and Ixx, Iyy, Izz, Hera uncertainties

%% MODEL
rm = (3*body.volume_km3/(4*pi))^(1/3);                    % mean radius [km]
r  = load(fullfile(sbtk.io.latest_run("hera_hlp6", sprintf("ROB-DATA%d_point_masses_sensitivity", number)), ...
                   "point_masses.mat"));                  % where the point masses put the mass
dense = sbtk.interior.sphere("dense", r.pm.positive_centroid_km, 0.30*rm, 0);
light = sbtk.interior.sphere("light", r.pm.negative_centroid_km, 0.30*rm, 0);
dense.estimate_position = true;
light.estimate_position = true;
components = [dense light];
candidates = sbtk.interior.sphere_grid(body, rm/4, rm/4);  % compact masses: every quarter of the radius

%% OPTIONS
opts = sbtk.invert.options(moi_labels={'Ixx', 'Iyy', 'Izz'});
opts.background = "estimate";                             % the background density is fitted too
settings = sbtk.search.settings(keep=1, max_masses=6, mass_starts=3);
params = struct([]);                                      % no geometry searched on a grid

%% RUN
out = sbtk.io.new_run("hera_hlp6", sprintf("ROB-DATA%d_components", number));
sr  = sbtk.search.run(body, components, obs, opts, params, candidates, settings);
fit = sr.best_fit;
writelines(formattedDisplayText(sr.log, 'SuppressMarkup', true), fullfile(out.folder, "search.txt"));
sbtk.io.report_inversion(out, body, fit, sbtk.invert.homogeneous(sbtk.invert.setup(body, components, obs, opts)));
solution = rob_solution(sprintf("dense and light spheres, %d compact masses", sr.masses.n_masses), "components", fit);
fprintf('AIC %.1f with %.0f parameters, reduced chi-square %.2f, p = %.2g\n', solution.aic, solution.parameters, ...
        solution.chi2_reduced, solution.p_value);
save(fullfile(out.folder, "solution.mat"), 'solution');
sbtk.io.finish_run(out);
```

MATLAB script: projects/hera_hlp6/rob/data3/rob3_components.m

```
%% ROB3_COMPONENTS  ROB-DATA3 (Didymos): a dense core, a light thick torus about the spin axis and a dense surface layer.
%
% The reading of the coefficients (documentation: Testing with ROB data > ROB-DATA3): J2 is 570
% sigma lower and J4 95 sigma higher than at constant density, and little else changes: a light
% ring about the spin axis, mass moved out of the equatorial plane. The torus is large and
% thick: of the four ring and tube radii tried (1.00 or 1.05, 0.25 or 0.30 of the mean radius;
% rob_candidates.m, ranked by analyze_rob.m) this one has the lowest AIC. The core is placed
% by its chi-square profile along the center-of-mass offset, then fitted with the densities.
% Fits the background density, the three relative densities and the core's center to the
% coefficients of degrees 1-5 and Ixx, Iyy, Izz.
% Writes results/hera_hlp6/<date_time>_ROB-DATA3_components/: solution.mat (read by
% rob3_figures.m), fit.txt/.mat, homogeneous.txt, settings.json, log.txt, figures/.

clear; close all;
here = fileparts(mfilename('fullpath'));                  % this package's folder
rob  = fileparts(here);                                   % rob/: the functions the runners share
hlp6 = fileparts(rob);                                    % hera_hlp6/: rob_data.m, hlp6_uncertainties.m
addpath(rob, hlp6, fullfile(fileparts(fileparts(hlp6)), 'toolbox'));

%% DATA
number = 3;                                               % ROB-DATA3
[obs, body] = rob_package(number);                        % degrees 1-5 and Ixx, Iyy, Izz, Hera uncertainties

%% MODEL
cof = body.centroid_km;                                   % positions are from the center of figure
rm  = (3*body.volume_km3/(4*pi))^(1/3);                   % mean radius [km]
core  = sbtk.interior.sphere("core", cof, 0.40*rm, 0);
core.estimate_position = true;
torus = sbtk.interior.torus("torus", 1.05*rm, 0.30*rm, 0, 0);   % about the z-axis
torus.center_km = cof;
layer = sbtk.interior.layer("regolith", 0, 0.03*rm, 0);        % from the surface to 3% of the radius deep
components = [core torus layer];

%% OPTIONS
opts = sbtk.invert.options(moi_labels={'Ixx', 'Iyy', 'Izz'});
opts.background = "estimate";                             % the background density is fitted too

%% RUN
out  = sbtk.io.new_run("hera_hlp6", sprintf("ROB-DATA%d_components", number));
% 1. Place the core: chi-square along the measured center-of-mass offset, the rest of the
%    geometry fixed (sbtk.invert.scan_position), from -0.4 to 0.4 mean radii of the center.
an = sbtk.interpret.anomaly(obs, body, obs.degree);
u  = an.com_offset_km/norm(an.com_offset_km);
fixed = components;
[fixed.estimate_position] = deal(false);
scan = sbtk.invert.scan_position(sbtk.invert.setup(body, fixed, obs, opts), "core", cof + (-0.40:0.025:0.40)'*rm*u);
components(1).center_km = scan.centers_km(scan.best, :);
% 2. Fit the densities and the core's position from there (Gauss-Newton).
prob = sbtk.invert.setup(body, components, obs, opts);
fit  = sbtk.invert.run(prob);
sbtk.io.report_inversion(out, body, fit, sbtk.invert.homogeneous(prob));
solution = rob_solution("core, torus, layer", "components", fit);
fprintf('AIC %.1f with %.0f parameters, reduced chi-square %.2f, p = %.2g\n', solution.aic, solution.parameters, ...
        solution.chi2_reduced, solution.p_value);

save(fullfile(out.folder, "solution.mat"), 'solution');
sbtk.io.finish_run(out);
```

MATLAB script: projects/hera_hlp6/rob/data4/rob4_components.m

```
%% ROB4_COMPONENTS  ROB-DATA4 (Dimorphos): a core, a torus about the spin axis and a surface layer.
%
% The reading of the coefficients (documentation: Testing with ROB data > ROB-DATA4): C22 is 50
% sigma lower (mass toward the ends of the y-axis) and the center of mass is 1.8 m toward -y,
% so the -y end carries more; J2 is 21 sigma higher. A dense core on the -y side makes C22 and
% the offset; a torus about the spin axis and a surface layer the part of J2 the core does not.
% Of the models tried (rob_candidates.m and two searches of the geometry, ranked by
% analyze_rob.m) this one has the lowest AIC. The core is placed by its chi-square profile
% along the center-of-mass offset, then fitted with the densities; Gauss-Newton moves it
% outward and lowers its density (degree 2 trades one against the other). Fits the background
% density, the three relative densities and the core's center to the coefficients of degrees
% 1-3 and Ixx, Iyy, Izz; then predicts the acceleration at the GRASS surface point (several
% minutes).
% Writes results/hera_hlp6/<date_time>_ROB-DATA4_components/: solution.mat (read by
% rob4_figures.m), fit.txt/.mat, homogeneous.txt, settings.json, log.txt, figures/.

clear; close all;
here = fileparts(mfilename('fullpath'));                  % this package's folder
rob  = fileparts(here);                                   % rob/: the functions the runners share
hlp6 = fileparts(rob);                                    % hera_hlp6/: rob_data.m, hlp6_uncertainties.m
addpath(rob, hlp6, fullfile(fileparts(fileparts(hlp6)), 'toolbox'));

%% DATA
number = 4;                                               % ROB-DATA4
[obs, body, ~, ~, body_full] = rob_package(number);       % degrees 1-3 and Ixx, Iyy, Izz, Hera uncertainties

%% MODEL
cof = body.centroid_km;                                   % positions are from the center of figure
rm  = (3*body.volume_km3/(4*pi))^(1/3);                   % mean radius [km]
core  = sbtk.interior.sphere("core", cof, 0.40*rm, 0);
core.estimate_position = true;
torus = sbtk.interior.torus("torus", 0.95*rm, 0.15*rm, 0, 0);   % about the z-axis
torus.center_km = cof;
layer = sbtk.interior.layer("regolith", 0, 0.03*rm, 0);       % from the surface to 3% of the radius deep
components = [core torus layer];

%% OPTIONS
opts = sbtk.invert.options(moi_labels={'Ixx', 'Iyy', 'Izz'});
opts.background = "estimate";                             % the background density is fitted too

%% RUN
out  = sbtk.io.new_run("hera_hlp6", sprintf("ROB-DATA%d_components", number));
% 1. Place the core: chi-square along the measured center-of-mass offset, the rest of the
%    geometry fixed (sbtk.invert.scan_position), from -0.4 to 0.4 mean radii of the center.
an = sbtk.interpret.anomaly(obs, body, obs.degree);
u  = an.com_offset_km/norm(an.com_offset_km);
fixed = components;
[fixed.estimate_position] = deal(false);
scan = sbtk.invert.scan_position(sbtk.invert.setup(body, fixed, obs, opts), "core", cof + (-0.40:0.025:0.40)'*rm*u);
components(1).center_km = scan.centers_km(scan.best, :);
% 2. Fit the densities and the core's position from there (Gauss-Newton).
prob = sbtk.invert.setup(body, components, obs, opts);
fit  = sbtk.invert.run(prob);
sbtk.io.report_inversion(out, body, fit, sbtk.invert.homogeneous(prob));
solution = rob_solution("core, torus, layer", "components", fit);
fprintf('AIC %.1f with %.0f parameters, reduced chi-square %.2f, p = %.2g\n', solution.aic, solution.parameters, ...
        solution.chi2_reduced, solution.p_value);

%% SURFACE POINT
% The acceleration the model predicts at the GRASS point, on the full-resolution shape model.
solution.grass = rob_grass(number, body_full, fit);
fprintf('GRASS: acceleration %+.1f%% of ROB''s, %.1f deg from it\n', solution.grass.error_percent, solution.grass.angle_deg);

save(fullfile(out.folder, "solution.mat"), 'solution');
sbtk.io.finish_run(out);
```

MATLAB script: projects/hera_hlp6/rob/data5/rob5_components.m

```
%% ROB5_COMPONENTS  ROB-DATA5 (Dimorphos): one dense core.
%
% The reading of the coefficients (documentation: Testing with ROB data > ROB-DATA5): C22 is 27
% sigma higher (mass toward the ends of the x-axis), Ixx and Izz smaller (mass close to the
% x-axis and to the spin axis), and the center of mass 0.3 m toward +x (1 sigma): mass gathered
% on the x-axis inside the body, on the +x side. A dense core placed by its chi-square profile
% along the center-of-mass offset, then fitted with its density. Adding a torus or a layer
% lowers chi-square by less than AIC asks; of the models tried (rob_candidates.m and two
% searches of the geometry, ranked by analyze_rob.m) this one has the lowest AIC. Fits the background density, the
% core's relative density and its center to the coefficients of degrees 1-3 and Ixx, Iyy,
% Izz; then predicts the acceleration at the GRASS surface point (several minutes).
% Writes results/hera_hlp6/<date_time>_ROB-DATA5_components/: solution.mat (read by
% rob5_figures.m), fit.txt/.mat, homogeneous.txt, settings.json, log.txt, figures/.

clear; close all;
here = fileparts(mfilename('fullpath'));                % this package's folder
rob  = fileparts(here);                                 % rob/: the functions the runners share
hlp6 = fileparts(rob);                                  % hera_hlp6/: rob_data.m, hlp6_uncertainties.m
addpath(rob, hlp6, fullfile(fileparts(fileparts(hlp6)), 'toolbox'));

%% DATA
number = 5;                                             % ROB-DATA5
[obs, body, ~, ~, body_full] = rob_package(number);     % degrees 1-3 and Ixx, Iyy, Izz, Hera uncertainties

%% MODEL
cof = body.centroid_km;                                 % positions are from the center of figure
rm  = (3*body.volume_km3/(4*pi))^(1/3);                 % mean radius [km]
core = sbtk.interior.sphere("core", cof, 0.40*rm, 0);
core.estimate_position = true;
components = core;

%% OPTIONS
opts = sbtk.invert.options(moi_labels={'Ixx', 'Iyy', 'Izz'});
opts.background = "estimate";                           % the background density is fitted too

%% RUN
out  = sbtk.io.new_run("hera_hlp6", sprintf("ROB-DATA%d_components", number));
% 1. Place the core: chi-square along the measured center-of-mass offset, the rest of the
%    geometry fixed (sbtk.invert.scan_position), from -0.4 to 0.4 mean radii of the center.
an = sbtk.interpret.anomaly(obs, body, obs.degree);
u  = an.com_offset_km/norm(an.com_offset_km);
fixed = components;
[fixed.estimate_position] = deal(false);
scan = sbtk.invert.scan_position(sbtk.invert.setup(body, fixed, obs, opts), "core", cof + (-0.40:0.025:0.40)'*rm*u);
components(1).center_km = scan.centers_km(scan.best, :);
% 2. Fit the densities and the core's position from there (Gauss-Newton).
prob = sbtk.invert.setup(body, components, obs, opts);
fit  = sbtk.invert.run(prob);
sbtk.io.report_inversion(out, body, fit, sbtk.invert.homogeneous(prob));
solution = rob_solution("core", "components", fit);
fprintf('AIC %.1f with %.0f parameters, reduced chi-square %.2f, p = %.2g\n', solution.aic, solution.parameters, ...
        solution.chi2_reduced, solution.p_value);

%% SURFACE POINT
% The acceleration the model predicts at the GRASS point, on the full-resolution shape model.
solution.grass = rob_grass(number, body_full, fit);
fprintf('GRASS: acceleration %+.1f%% of ROB''s, %.1f deg from it\n', solution.grass.error_percent, solution.grass.angle_deg);

save(fullfile(out.folder, "solution.mat"), 'solution');
sbtk.io.finish_run(out);
```

MATLAB script: projects/hera_hlp6/rob/data6/rob6_components.m

```
%% ROB6_COMPONENTS  ROB-DATA6 (Dimorphos): a core, a torus about the spin axis and a surface layer.
%
% The reading of the coefficients (documentation: Testing with ROB data > ROB-DATA6): all three
% moments of inertia are 30 to 60 sigma larger than at constant density (light inside, dense
% outside), C22 is 97 sigma lower (mass toward the ends of the y-axis), J2 79 sigma higher, and
% the center of mass is 1.3 m toward +x. A light core on the x-axis takes mass from along x and
% moves the center of mass; a dense torus in the equatorial plane raises J2; a dense surface
% layer raises the moments. Of the models tried (rob_candidates.m and two searches of the
% geometry, ranked by analyze_rob.m) this one has the lowest AIC. The core is placed by its chi-square profile
% along the center-of-mass offset, then fitted with the densities. Fits the background density,
% the three relative densities and the core's center to the coefficients of degrees 1-3 and Ixx,
% Iyy, Izz; then predicts the acceleration at the GRASS surface point (several minutes).
% Writes results/hera_hlp6/<date_time>_ROB-DATA6_components/: solution.mat (read by
% rob6_figures.m), fit.txt/.mat, homogeneous.txt, settings.json, log.txt, figures/.

clear; close all;
here = fileparts(mfilename('fullpath'));                  % this package's folder
rob  = fileparts(here);                                   % rob/: the functions the runners share
hlp6 = fileparts(rob);                                    % hera_hlp6/: rob_data.m, hlp6_uncertainties.m
addpath(rob, hlp6, fullfile(fileparts(fileparts(hlp6)), 'toolbox'));

%% DATA
number = 6;                                               % ROB-DATA6
[obs, body, ~, ~, body_full] = rob_package(number);       % degrees 1-3 and Ixx, Iyy, Izz, Hera uncertainties

%% MODEL
cof = body.centroid_km;                                   % positions are from the center of figure
rm  = (3*body.volume_km3/(4*pi))^(1/3);                   % mean radius [km]
core  = sbtk.interior.sphere("core", cof, 0.40*rm, 0);
core.estimate_position = true;
torus = sbtk.interior.torus("torus", 0.95*rm, 0.15*rm, 0, 0);   % about the z-axis
torus.center_km = cof;
layer = sbtk.interior.layer("regolith", 0, 0.03*rm, 0);       % from the surface to 3% of the radius deep
components = [core torus layer];

%% OPTIONS
opts = sbtk.invert.options(moi_labels={'Ixx', 'Iyy', 'Izz'});
opts.background = "estimate";                             % the background density is fitted too

%% RUN
out  = sbtk.io.new_run("hera_hlp6", sprintf("ROB-DATA%d_components", number));
% 1. Place the core: chi-square along the measured center-of-mass offset, the rest of the
%    geometry fixed (sbtk.invert.scan_position), from -0.4 to 0.4 mean radii of the center.
an = sbtk.interpret.anomaly(obs, body, obs.degree);
u  = an.com_offset_km/norm(an.com_offset_km);
fixed = components;
[fixed.estimate_position] = deal(false);
scan = sbtk.invert.scan_position(sbtk.invert.setup(body, fixed, obs, opts), "core", cof + (-0.40:0.025:0.40)'*rm*u);
components(1).center_km = scan.centers_km(scan.best, :);
% 2. Fit the densities and the core's position from there (Gauss-Newton).
prob = sbtk.invert.setup(body, components, obs, opts);
fit  = sbtk.invert.run(prob);
sbtk.io.report_inversion(out, body, fit, sbtk.invert.homogeneous(prob));
solution = rob_solution("core, torus, layer", "components", fit);
fprintf('AIC %.1f with %.0f parameters, reduced chi-square %.2f, p = %.2g\n', solution.aic, solution.parameters, ...
        solution.chi2_reduced, solution.p_value);

%% SURFACE POINT
% The acceleration the model predicts at the GRASS point, on the full-resolution shape model.
solution.grass = rob_grass(number, body_full, fit);
fprintf('GRASS: acceleration %+.1f%% of ROB''s, %.1f deg from it\n', solution.grass.error_percent, solution.grass.angle_deg);

save(fullfile(out.folder, "solution.mat"), 'solution');
sbtk.io.finish_run(out);
```

The sector grids and the point masses

These differ between packages only in the package number, the grid of ROB-DATA2 (one shell, the others two), and the GRASS section of the Dimorphos packages.

MATLAB script: projects/hera_hlp6/rob/data4/rob4_sectors.m

```
%% ROB4_SECTORS  ROB-DATA4 (Dimorphos): a sector grid, 2 × 4 × 8.
%
% Fits a density to each of 64 sectors (the inner and the outer half of the radius, 4 latitude bands,
% 8 longitude bands) with the background at the bulk density and the Tikhonov parameter of the lowest AIC
% (sbtk.invert.choose_regularization), to the coefficients of degrees 1–3 and Ixx, Iyy, Izz
% with the Hera radio-science uncertainties. Of the two grids tried (analyze_rob.m: 1 x 4 x 8
% and 2 x 4 x 8), this one has the lower AIC.
% Then the acceleration the solution predicts at the GRASS surface point (rob_grass.m, several minutes).
% Writes results/hera_hlp6/<date_time>_ROB-DATA4_sectors/: solution.mat (read by
% rob4_figures.m), fit.txt/.mat, homogeneous.txt, settings.json, log.txt, figures/.

clear; close all;
here = fileparts(mfilename('fullpath'));                  % this package's folder
rob  = fileparts(here);                                   % rob/: the functions the runners share
hlp6 = fileparts(rob);                                    % hera_hlp6/: rob_data.m, hlp6_uncertainties.m
addpath(rob, hlp6, fullfile(fileparts(fileparts(hlp6)), 'toolbox'));

%% DATA
number = 4;                                               % ROB-DATA4
[obs, body, ~, ~, body_full] = rob_package(number);       % degrees 1–3 and Ixx, Iyy, Izz, Hera uncertainties

%% MODEL
lat = [-90 -45 0 45 90];                                  % 4 latitude bands [deg]
lon = -180:45:180;                                        % 8 longitude bands [deg]
sectors = sbtk.interior.sector_grid([0 0.5 1], lat, lon);   % shells: fractions of the radius

%% OPTIONS
opts = sbtk.invert.options(moi_labels={'Ixx', 'Iyy', 'Izz'});
opts.background = "bulk";                                 % the sectors fill the body

%% RUN
out  = sbtk.io.new_run("hera_hlp6", sprintf("ROB-DATA%d_sectors", number));
prob = sbtk.invert.setup(body, sectors, obs, opts);
prob.opts.regularization = sbtk.invert.choose_regularization(prob, "aic");
fit  = sbtk.invert.run(prob);
sbtk.io.report_inversion(out, body, fit, sbtk.invert.homogeneous(prob));
solution = rob_solution("2 × 4 × 8", "sectors", fit);
fprintf('lambda %.3g; AIC %.1f with %.1f parameters, reduced chi-square %.2f, p = %.2g\n', fit.regularization, ...
        solution.aic, solution.parameters, solution.chi2_reduced, solution.p_value);

%% SURFACE POINT
% The acceleration the solution predicts at the GRASS point, on the full-resolution shape model.
solution.grass = rob_grass(number, body_full, fit);
fprintf('GRASS: acceleration %+.1f%% of ROB''s, %.1f deg from it\n', solution.grass.error_percent, solution.grass.angle_deg);

save(fullfile(out.folder, "solution.mat"), 'solution');
sbtk.io.finish_run(out);
```

MATLAB script: projects/hera_hlp6/rob/data4/rob4_point_masses.m

```
%% ROB4_POINT_MASSES  ROB-DATA4 (Dimorphos): point masses on a grid filling the body.
%
% About 2,700 small spheres, one every 8.3 m (a ninth of the mean radius), fitted together to the
% coefficients of degrees 1-3 and Ixx, Iyy, Izz with the Hera radio-science uncertainties,
% with the Tikhonov parameter of the lowest AIC (sbtk.interpret.point_masses). No shapes are
% assumed: where the measurements put the mass. Run it once with each weighting; the pages show
% "sensitivity" and compare it with "equal".
% Then the acceleration the solution predicts at the GRASS surface point (rob_grass.m, several minutes).
% Writes results/hera_hlp6/<date_time>_ROB-DATA4_point_masses_<weighting>/: solution.mat
% (read by rob4_figures.m), point_masses.mat, log.txt, figures/point_masses.png.

clear; close all;
here = fileparts(mfilename('fullpath'));                  % this package's folder
rob  = fileparts(here);                                   % rob/: the functions the runners share
hlp6 = fileparts(rob);                                    % hera_hlp6/: rob_data.m, hlp6_uncertainties.m
addpath(rob, hlp6, fullfile(fileparts(fileparts(hlp6)), 'toolbox'));

%% DATA
number = 4;                                               % ROB-DATA4
[obs, body, ~, ~, body_full] = rob_package(number);       % degrees 1-3 and Ixx, Iyy, Izz, Hera uncertainties
weighting = "sensitivity";                                % "sensitivity" or "equal"

%% MODEL
rm = (3*body.volume_km3/(4*pi))^(1/3);                    % mean radius [km]
spacing = rm/9;                                           % [km]
grid = sbtk.interior.sphere_grid(body, spacing, spacing/5);

%% OPTIONS
opts = sbtk.invert.options(moi_labels={'Ixx', 'Iyy', 'Izz'});

%% RUN
out  = sbtk.io.new_run("hera_hlp6", sprintf("ROB-DATA%d_point_masses_%s", number, weighting));
prob = sbtk.invert.setup(body, grid, obs, opts);
pm   = sbtk.interpret.point_masses(prob, weighting, "aic");
sbtk.io.save_figure(out, sbtk.plot.point_masses(body, pm, spacing), "point_masses");
solution = rob_solution(sprintf("%d points", numel(grid)), "point masses", pm);
fprintf('lambda %.3g; AIC %.1f with %.1f effective parameters, reduced chi-square %.2f, p = %.2g\n', pm.lambda, ...
        solution.aic, solution.parameters, solution.chi2_reduced, solution.p_value);

%% SURFACE POINT
% The acceleration the solution predicts at the GRASS point, on the full-resolution shape model.
solution.grass = rob_grass(number, body_full, pm.model);
fprintf('GRASS: acceleration %+.1f%% of ROB''s, %.1f deg from it\n', solution.grass.error_percent, solution.grass.angle_deg);

save(fullfile(out.folder, "solution.mat"), 'solution');
save(fullfile(out.folder, "point_masses.mat"), 'pm');
sbtk.io.finish_run(out);
```

The figures and tables

After DATA AND SOLUTIONS, each section makes the figures of one part of the page (only the ROB-DATA2 page shows that of THE SECTOR GRID); THE GRASS POINT and TABLES write its tables. The Didymos scripts have no GRASS section.

MATLAB script: projects/hera_hlp6/rob/data4/rob4_figures.m

```
%% ROB4_FIGURES  The figures and tables of the ROB-DATA4 page, from the newest run of each solution.
%
% Run first: rob4_components.m, rob4_sectors.m and rob4_point_masses.m (once with weighting
% "sensitivity", once with "equal"). Run DATA AND SOLUTIONS, then any section: each makes one
% figure (SOLUTIONS SIDE BY SIDE before TABLES, which uses its cuts). Writes
% results/hera_hlp6/<date_time>_ROB-DATA4_figures/: figures/<name>.png and tables/<name>.md, as
% the documentation page (Testing with ROB data > ROB-DATA4) shows them. A few minutes, most of
% them for the GRASS point of constant density.

clear; close all;
here = fileparts(mfilename('fullpath'));                % this package's folder
rob  = fileparts(here);                                 % rob/: the functions the runners share
hlp6 = fileparts(rob);                                  % hera_hlp6/: rob_data.m, hlp6_uncertainties.m
addpath(rob, hlp6, fullfile(fileparts(fileparts(hlp6)), 'toolbox'));

%% DATA AND SOLUTIONS
number = 4;                                             % ROB-DATA4 (Dimorphos)
[obs, body, given, full, body_full] = rob_package(number);
S = [rob_load(number, "components"), rob_load(number, "sectors"), rob_load(number, "point_masses_sensitivity")];
W = [S(3), rob_load(number, "point_masses_equal")];     % the two weightings of the point masses
names = ["Components", "Sector grid", "Point masses"];
style = [1 3 4];                                        % orange circle, green diamond, purple triangle
shown = [1 2 3];                                        % in the figures: those consistent with the measurements
L = obs.degree;                                         % the fitted degree
an = sbtk.interpret.anomaly(obs, body, L);              % the measurements minus constant density
an_full = sbtk.interpret.anomaly(full, body_full, L);   % the same on the full-resolution shape
exact = sbtk.interpret.anomaly(given, body, 10);        % ROB's exact values to degree 10
M = max(an_full.deg(abs(an_full.value./an_full.sigma) > 3)); % the highest degree resolved at 3 sigma
rm = (3*body.volume_km3/(4*pi))^(1/3);                  % mean radius [km]
out = sbtk.io.new_run("hera_hlp6", sprintf("ROB-DATA%d_figures", number));

%% MEASURED COEFFICIENTS
% C - C_CD of every fitted coefficient with its 1-sigma, and the moments of inertia.
fig = sbtk.plot.coefficient_comparison(an, {}, zeros(0, 3), strings(1, 0), L, zeros(1, 0));
sbtk.io.save_figure(out, fig, "measured");

%% ANOMALY MAPS
% Each resolved degree from 2 as a map of where a mass near the surface would make it.
fig = sbtk.plot.anomaly_map(sbtk.interpret.geometry(sbtk.interpret.anomaly(full, body_full, M)));
sbtk.io.save_figure(out, fig, "anomaly_map");

%% SOLUTIONS AGAINST THE MEASUREMENTS
% Each solution's C - C_CD with the measured values, and (solution - measured)/sigma.
models = arrayfun(@(s) sbtk.interpret.anomaly(s.model, body, L), S(shown), 'UniformOutput', false);
moi = zeros(numel(shown), 3);
for i = 1:numel(shown)
    [~, j] = ismember(an.moi_labels, S(shown(i)).labels);
    moi(i, :) = S(shown(i)).normalized_residual(j);
end
fig = sbtk.plot.coefficient_comparison(an, models, moi, names(shown), L, style(shown));
sbtk.io.save_figure(out, fig, "compare");

%% THE MODEL WITH COMPONENTS
% Seen obliquely and down the spin axis; the part of each coefficient each component makes.
fig = sbtk.plot.interior3d(body, S(1).model, "");
sbtk.io.save_figure(out, fig, "view_components");
fig = sbtk.plot.contributions(an, S(1).model, body, M);
sbtk.io.save_figure(out, fig, "contributions");

%% THE SECTOR GRID
fig = sbtk.plot.interior3d(body, S(2).model, "");
sbtk.io.save_figure(out, fig, "view_sectors");

%% SOLUTIONS SIDE BY SIDE
% Relative density on the planes z = 0 and y = 0 through the center of figure; the point
% masses as the mean of cells a ninth of the mean radius across.
[fig, cs] = sbtk.plot.solution_cuts(body, {S(shown).model}, names(shown), rm/9);
sbtk.io.save_figure(out, fig, "solutions");

%% ROB'S EXACT VALUES
% The fraction of each degree of ROB's exact C - C_CD each solution reproduces (rob_reproduced.m).
F = zeros(numel(shown), 10);
for i = 1:numel(shown)
    a = sbtk.interpret.anomaly(S(shown(i)).model, body, 10);
    F(i, :) = arrayfun(@(l) rob_reproduced(a, exact, l), 1:10);
end
fig = rob_plot_reproduced(F, names(shown), L, style(shown));
sbtk.io.save_figure(out, fig, "exact");

%% THE GRASS POINT
% The acceleration each solution predicts at the GRASS surface point (computed by its runner)
% and that of constant density, against ROB's.
```

```
homogeneous = struct('rho_bg_kgm3', body_full.bulk_density_kgm3, 'components', sbtk.interior.no_components(), ...
                     'random_field', sbtk.interior.no_random_field(), 'origin_km', body_full.centroid_km);
rob_grass_table(fullfile(out.folder, "tables"), S(1:2), W, rob_grass(number, body_full, homogeneous));

%% TABLES
rob_tables(fullfile(out.folder, "tables"), S, W, an, an_full, exact, body, cs, names(shown));
close all
sbtk.io.finish_run(out);
```

How the models were chosen

`projects/hera_hlp6/analyze_rob.m` fits every candidate of `rob_candidates.m` to a package, two searches of the geometry (`sbtk.search.run`) and the point masses, and ranks them all by the Akaike information criterion, AIC = χ^2 + 2 × the number of parameters. A regularized fit counts its effective number of parameters; a search counts each size it chose on its grid as well as the densities and positions it estimated. The model of each kind with the lowest AIC is the one its runner fits. To try another model, copy a component runner, change MODEL, and compare the AIC it prints with the others (Testing with ROB data).

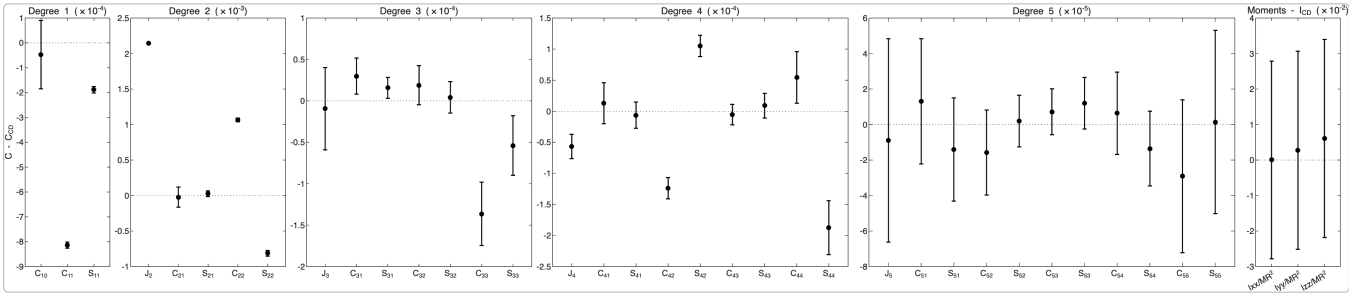
ROB-DATA1: Didymos

Degree 2 of ROB-DATA1 moves mass from the spin axis toward the two ends of one equatorial axis, and degree 1 says that one end carries more than the other. A model of three components built on that reading, a light column along the spin axis and a dense ellipsoid at each end of the axis, fits every measured coefficient with 9 parameters and ranks first by 37 in AIC. The point masses, given no shapes, find the same light column. Above the fitted degrees, 1 to 5, the model with components reproduces 92% of ROB's exact values over degrees 6 to 10, where the point masses and the sector grid do worse than constant density.

Kind	Model	Parameters	Reduced χ^2	p	AIC	Exact values, degrees 1–5	Exact values, degrees 6–10
Components	**light column along the spin axis, two equatorial ellipsoids**	9.0	0.24	1.00	25.0	1.00	0.92
Sector grid	2 × 4 × 8	28.3	0.52	0.87	61.7	1.00	−3.82
Point masses	2667 points	28.7	0.51	0.88	62.1	1.00	−0.40

The best solution of each kind, by AIC (bold: the lowest). Parameters: the effective number of parameters. Exact values: the fraction of ROB's exact coefficients a solution reproduces, over the fitted degrees 1–5 and over degrees 6–10, which were not (1: all; 0: no better than constant density; negative: worse). See How to read these pages.

1. What the measurements show



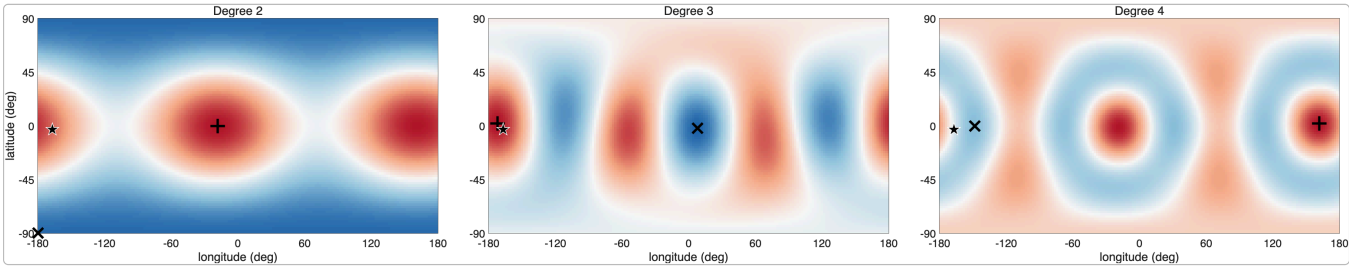
The measured coefficients minus those of the same shape at constant density, C − C_CD, with ±1σ of the Hera radio-science solution, degrees 1 to 5, and the three moments of inertia minus those at constant density. Each panel has its own scale; 0 is constant density.

Measurement	C − C_CD	1σ	In σ	What it says
$\bar{C}_{11}$	-8.14×10^{-4}	1.24×10^{-5}	−66	center of mass toward −x
$\bar{S}_{11}$	-1.88×10^{-4}	1.30×10^{-5}	−14	center of mass toward −y
$\bar{C}_{20}$	-2.14×10^{-3}	8.70×10^{-6}	−246	J_2 higher: mass toward the equator
$\bar{C}_{22}$	1.07×10^{-3}	2.64×10^{-5}	+40	mass toward ±x more than ±y
$\bar{S}_{22}$	-8.12×10^{-4}	3.93×10^{-5}	−21	mass toward longitudes −45° and 135°
$\bar{C}_{33}$	-1.36×10^{-4}	3.83×10^{-5}	−4	order 3, strongest at longitudes −173°, −53°, 67°
$\bar{C}_{42}$	-1.24×10^{-4}	1.69×10^{-5}	−7	order 2, strongest at longitudes −110°, 70°
$\bar{S}_{42}$	1.05×10^{-4}	1.70×10^{-5}	+6	order 2, strongest at longitudes −110°, 70°
$\bar{S}_{44}$	-1.88×10^{-4}	4.35×10^{-5}	−4	order 4, strongest at longitudes −108°, −18°, 72°, 162°

The measurements more than 3σ from constant density.

Three things stand out. J_2 is 246σ higher than at constant density: mass has moved from the spin axis toward the equator. $\bar{C}_{22}$ and $\bar{S}_{22}$ say the move is not even around the equator but toward one axis. And the center of mass is 0.59 m from the center of figure toward −x: one end of that axis is heavier. Degrees 3 and 4 are small but measured. Degree 5 is fitted, but no coefficient of it is more than 1σ from constant density. The moments of inertia are not: for Didymos their 1σ is larger than any interior changes them.

2. Where the mass is

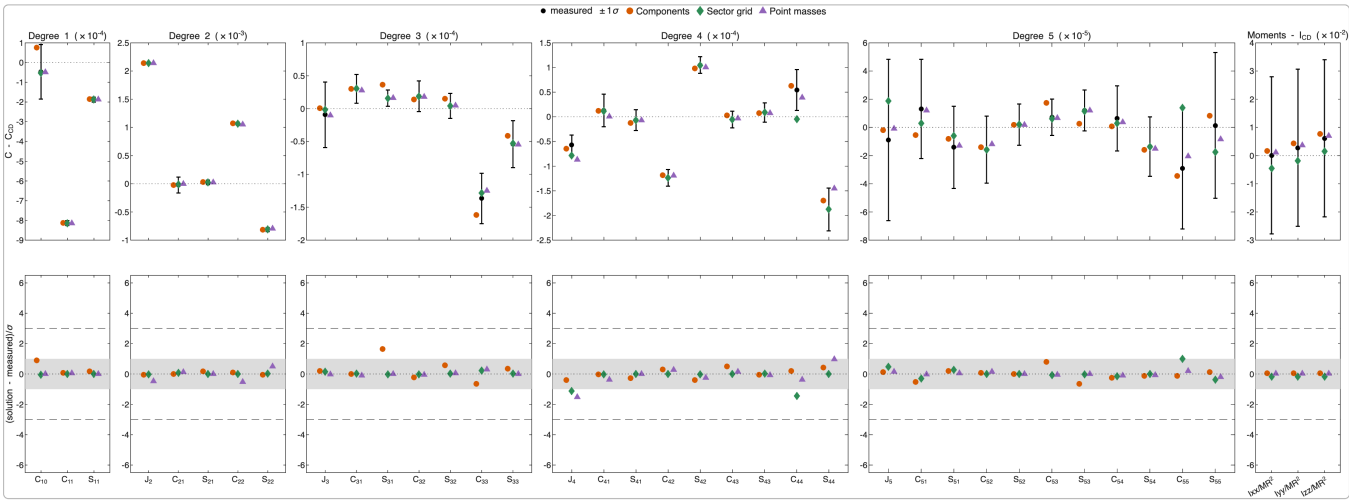


Degrees 2, 3 and 4 of C − C_CD as maps of latitude and longitude from the center of figure. Red where a mass added near the surface in that direction would produce the pattern, blue where a mass taken away would; each map on its own symmetric color scale. +: strongest excess; x: strongest deficit; star: direction of the center-of-mass offset.

Degree 2 puts the excess on the equator at longitudes −19° and 161° (±2°) and the deficit at both poles. The center of mass points to longitude −167°, 32° from the 161° end. Degree 3 has its strongest excess at −172° and degree 4 at −18° and 162°: the same axis, the same heavier end.

So the model built for this package takes mass from along the spin axis with a light column, a prolate ellipsoid about the axis through the center of figure (semi-axes 0.25, 0.25 and 0.70 of the mean radius), and puts it at the two ends of the axis with a dense ellipsoid at each, started 0.6 of the mean radius out at longitudes −20° and 160° and elongated along the axis. The densities of all three and the centers of the two ellipsoids are fitted.

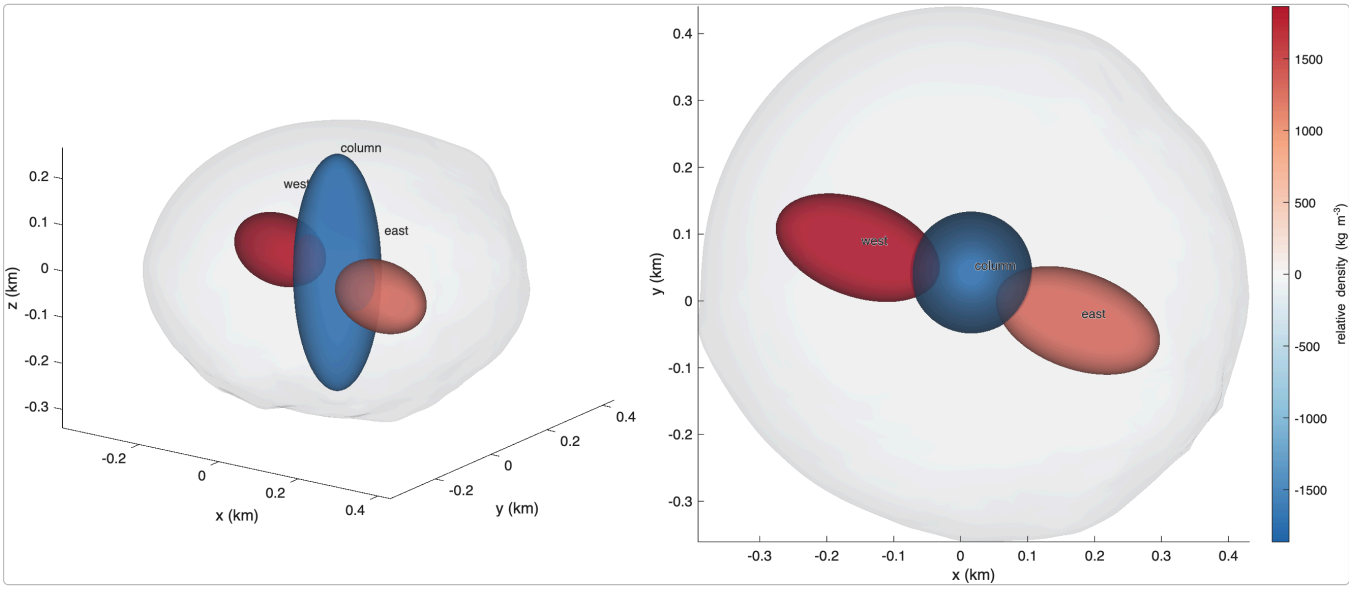
3. How well each solution fits



Top: C − C_CD measured (black, ±1σ) and of the best solution of each kind. Bottom: (solution − measured)/σ; the gray band is ±1σ, the dashed lines ±3σ.

The model with components, the sector grid and the point masses all match every fitted coefficient within 2σ. They differ in what it costs: 9 parameters for the components, about 28 effective parameters for the others. The components rank first by 37 in AIC, a strong preference. The model of five components tried before it, polar caps, a dense sphere at each end of the axis and a light core, fits with 14 parameters and ranks 23 behind.

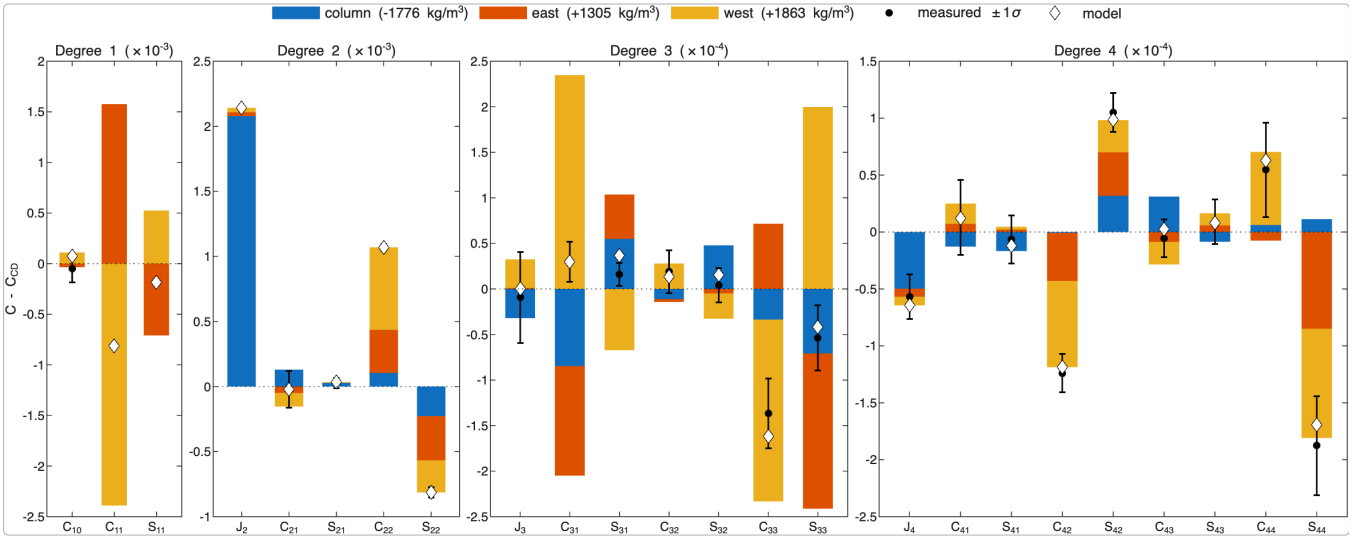
4. The model with components



The model with components, seen obliquely (left) and down the spin axis (right), colored by relative density.

Component	Shape and place	Relative density (kg/m³)	Density (kg/m³)	Mass anomaly (kg); makes most of
column	ellipsoid, semi-axes 91 × 91 × 256 m, at the center	−1776 ± 49	861	-1.58×10^{10}; $\bar{C}_{20}$ −239σ, $\bar{S}_{22}$ −6σ
east	ellipsoid, semi-axes 128 × 73 × 73 m, 175 m from the center toward lat −1°, lon −24°	+1305 ± 145	3942	3.72×10^{9}; $\bar{C}_{11}$ +127σ, $\bar{S}_{11}$ −54σ
west	ellipsoid, semi-axes 128 × 73 × 73 m, 174 m from the center toward lat 3°, lon 168°	+1863 ± 163	4500	5.32×10^{9}; $\bar{C}_{11}$ −193σ, $\bar{S}_{11}$ +40σ

Background density 2637 kg/m³ (bulk density 2603 kg/m³). Makes most of: the component's part of each coefficient in units of its 1σ.

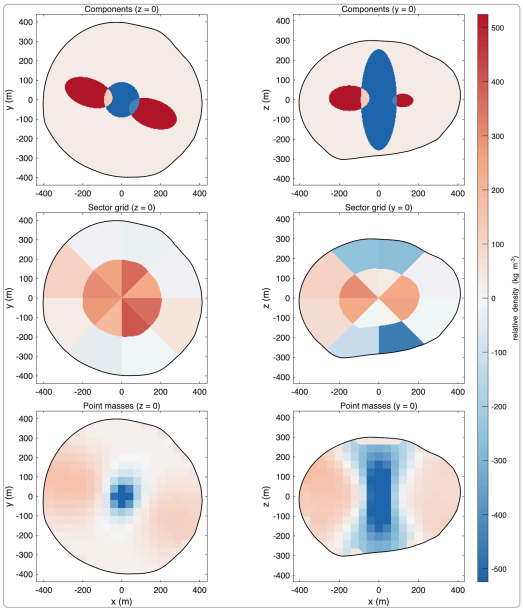


The part of each measured coefficient each component makes (stacked bars), the measured value with ±1σ (black) and the model's sum (white diamond).

Each component does what the reading asked of it:

- **The light column** (−1776 ± 49 kg/m³, semi-axes 91, 91 and 256 m, at the center) makes nearly all of J_2: 239σ of $\bar{C}_{20}$. Its density, 861 kg/m³, is a third of the bulk density.
- **The west and east ellipsoids** (+1863 and +1305 kg/m³, 174 and 175 m from the center at longitudes 168° and −24°) make most of $\bar{C}_{22}$ and $\bar{S}_{22}$. Being unequal, they move the center of mass toward −x: their parts of $\bar{C}_{11}$, −193σ and +127σ, leave the measured −66σ. The west ellipsoid is at the density bound of 4500 kg/m³: the excess at the 161° end is more than an ellipsoid of this size can hold.
- The background, 2637 kg/m³, is 34 kg/m³ above the bulk density, to make up the mass the column takes away.

5. Point masses and components side by side



The best solution of each kind on the planes z = 0 (left) and y = 0 (right) through the center of figure, colored by relative density on one scale. Components and sectors are drawn with their own boundaries; the point masses are drawn as the mean relative density of 41 m cells.

Solution	Mass moved (% of M)	Excess centered at (m)	Deficit centered at (m)
Components	2.6	(−21, −4, 2)	(1, 0, 0)
Sector grid	1.7	(−21, −5, 1)	(10, 4, 1)
Point masses	1.5	(−34, −5, 0)	(4, 4, 2)

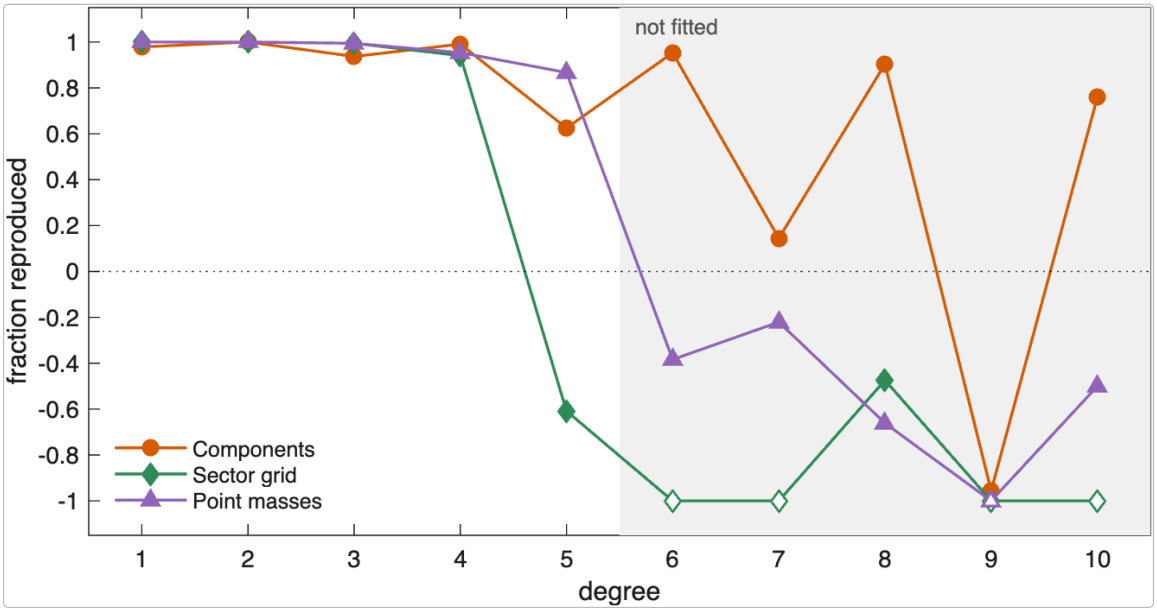
Correlation	Components	Sector grid	Point masses
Components	1.00	0.11	0.68
Sector grid	0.11	1.00	0.26
Point masses	0.68	0.26	1.00

Mass moved: the mass above the bulk density, as a percentage of the mass of the body. Excess and deficit centers are from the center of figure. Correlation: of the relative densities of the solutions, 41 m cell by 41 m cell.

The point masses were given no shapes, yet they find the light column. On the y = 0 plane they draw a light band about 100 m wide along the spin axis, from 250 m below the center to 250 m above it, where the column is; on the z = 0 plane it is a light region about 100 m across at the center. Both solutions add the mass at the equator, more on the −x side: the excess of the components is centered 21 m toward −x, that of the point masses 34 m. The difference is how compact the dense material is. The components gather it into two ellipsoids; the point masses spread it over the outer equator. Their relative densities correlate 0.68 cell by cell.

The sector grid cannot make a column: its sectors are blocks of latitude and longitude. It puts light caps at the poles of its outer shell and a denser inner shell instead, and correlates 0.11 with the components and 0.26 with the point masses.

6. Against ROB's exact values



The fraction of each degree of ROB's exact C − C_CD that each solution reproduces, 1 − |solution − exact|²/|exact|². 1: all of it; 0: no better than constant density; values below −1 are drawn at −1 as open markers. The gray region was not fitted.

All three reproduce 94 to 100% of each of degrees 1 to 4. Degree 5 is fitted, but its 1σ is larger than its anomaly, so the fit does not hold it: the point masses reproduce 87% of it, the components 62%, and the sector grid does worse than constant density. Above degree 5, which no solution was fitted to, the model with components reproduces 92% over degrees 6 to 10: 95% of degree 6, 90% of degree 8 and 76% of degree 10, though only 14% of degree 7, and it fails at degree 9. The point masses (−0.40) and the sector grid (−3.82) do worse than constant density at every degree above 5. A light column along the spin axis with dense material at the two ends of one equatorial axis is close to what the hidden interior has.

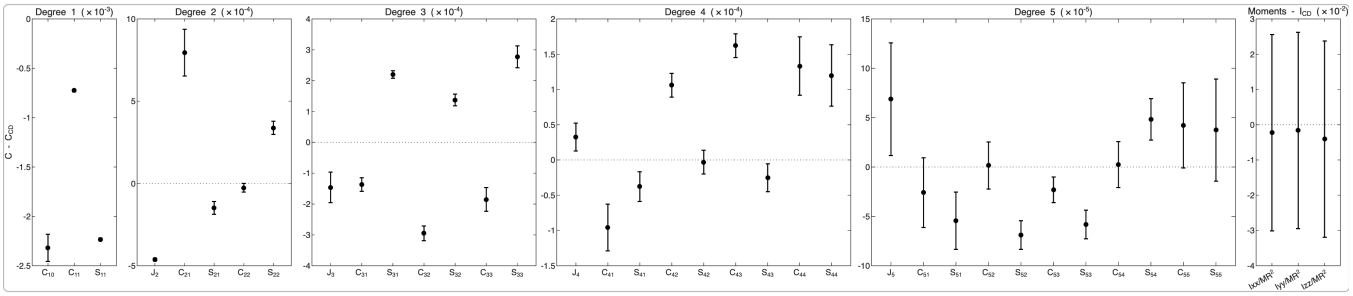
ROB-DATA2: Didymos

ROB-DATA2 is the package where simple shapes fail. The center of mass is 2.3 m off center toward the south, degree 2 is tilted 58° from the equator, and degree 3, small in the other Didymos packages, is large. No model built from a core, a torus and a layer comes near the measurements. A dense and a light sphere started where the point masses put the mass, with four compact masses added where the measurements ask for them, does fit, and ties in AIC with a grid of 32 sectors and with the point masses. The sector grid and the point masses agree closely, on a dense region below the equator toward –x and –y and a light region above it toward +y: their excess centers are 10 m apart and their relative densities correlate 0.78. Above the fitted degrees, 1 to 5, the point masses reproduce 17% of ROB's exact values and the sector grid 5%; the model with components does worse than constant density.

Kind	Model	Parameters	Reduced χ^2	p	AIC	Exact values, degrees 1–5	Exact values, degrees 6–10
Sector grid	**1 × 4 × 8**	25.1	1.05	0.40	63.8	1.00	0.05
Components	dense and light spheres, 4 compact masses	24.0	1.16	0.30	64.3	0.99	−0.20
Point masses	2667 points	30.4	0.57	0.80	65.1	1.00	0.17

The best solution of each kind, by AIC (bold: the lowest). Parameters: the effective number of parameters. Exact values: the fraction of ROB's exact coefficients a solution reproduces, over the fitted degrees 1–5 and over degrees 6–10, which were not (1: all; 0: no better than constant density; negative: worse). See How to read these pages.

1. What the measurements show



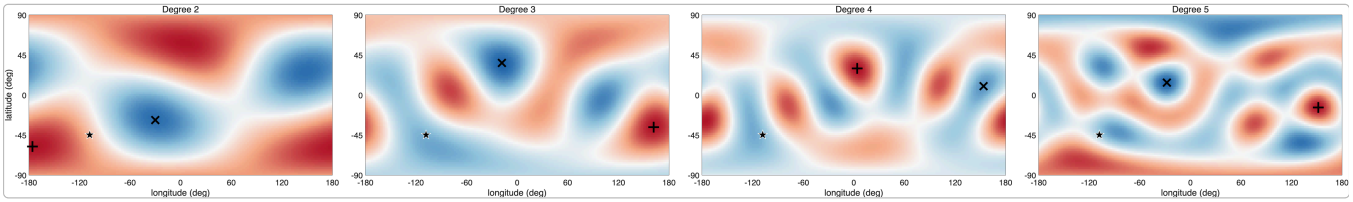
The measured coefficients minus those of the same shape at constant density, C – C_CD, with ±1σ of the Hera radio-science solution, degrees 1 to 5, and the three moments of inertia minus those at constant density. Each panel has its own scale; 0 is constant density.

Measurement	C – C_CD	1σ	In σ	What it says
$\bar{C}_{10}$	-2.32×10^{-3}	1.38×10^{-4}	−17	center of mass toward –z (south)
$\bar{C}_{11}$	-7.25×10^{-4}	1.24×10^{-5}	−58	center of mass toward –x
$\bar{S}_{11}$	-2.23×10^{-3}	1.30×10^{-5}	−171	center of mass toward –y
$\bar{C}_{20}$	4.62×10^{-4}	8.74×10^{-6}	+53	J_2 lower: mass toward the poles or the center
$\bar{C}_{21}$	7.94×10^{-4}	1.42×10^{-4}	+6	mass toward +x in the north, –x in the south
$\bar{S}_{21}$	-1.48×10^{-4}	3.98×10^{-5}	−4	mass toward –y in the north, +y in the south
$\bar{S}_{22}$	3.38×10^{-4}	3.93×10^{-5}	+9	mass toward longitudes 45° and –135°
$\bar{C}_{31}$	-1.37×10^{-4}	2.18×10^{-5}	−6	order 1, strongest at longitudes 122°
$\bar{S}_{31}$	2.20×10^{-4}	1.25×10^{-5}	+18	order 1, strongest at longitudes 122°
$\bar{C}_{32}$	-2.95×10^{-4}	2.35×10^{-5}	−13	order 2, strongest at longitudes –102°, 78°
$\bar{S}_{32}$	1.37×10^{-4}	1.90×10^{-5}	+7	order 2, strongest at longitudes –102°, 78°
$\bar{C}_{33}$	-1.85×10^{-4}	3.83×10^{-5}	−5	order 3, strongest at longitudes –79°, 41°, 161°
$\bar{S}_{33}$	2.77×10^{-4}	3.58×10^{-5}	+8	order 3, strongest at longitudes –79°, 41°, 161°
$\bar{C}_{42}$	1.06×10^{-4}	1.69×10^{-5}	+6	order 2, strongest at longitudes –1°, 179°
$\bar{C}_{43}$	1.62×10^{-4}	1.67×10^{-5}	+10	order 3, strongest at longitudes –123°, –3°, 117°
$\bar{C}_{44}$	1.33×10^{-4}	4.15×10^{-5}	+3	order 4, strongest at longitudes –170°, –80°, 10°, 100°
$\bar{S}_{52}$	-6.88×10^{-5}	1.46×10^{-5}	−5	order 2, strongest at longitudes –44°, 136°
$\bar{S}_{53}$	-5.81×10^{-5}	1.45×10^{-5}	−4	order 3, strongest at longitudes –157°, –37°, 83°

The measurements more than 3σ from constant density.

The center of mass is 2.3 m from the center of figure, four times as far as in ROB-DATA1 and ROB-DATA3: toward –y (171σ), –x (58σ) and the south (17σ). J_2 is 53σ lower than at constant density: mass has moved toward the poles or the center. $\bar{C}_{21}$ and $\bar{S}_{21}$ are not zero, so the anomaly is not symmetric about the equator, and every order of degree 3 is measured. Degree 5 is resolved too: $\bar{S}_{52}$ and $\bar{S}_{53}$ are 5σ and 4σ from constant density.

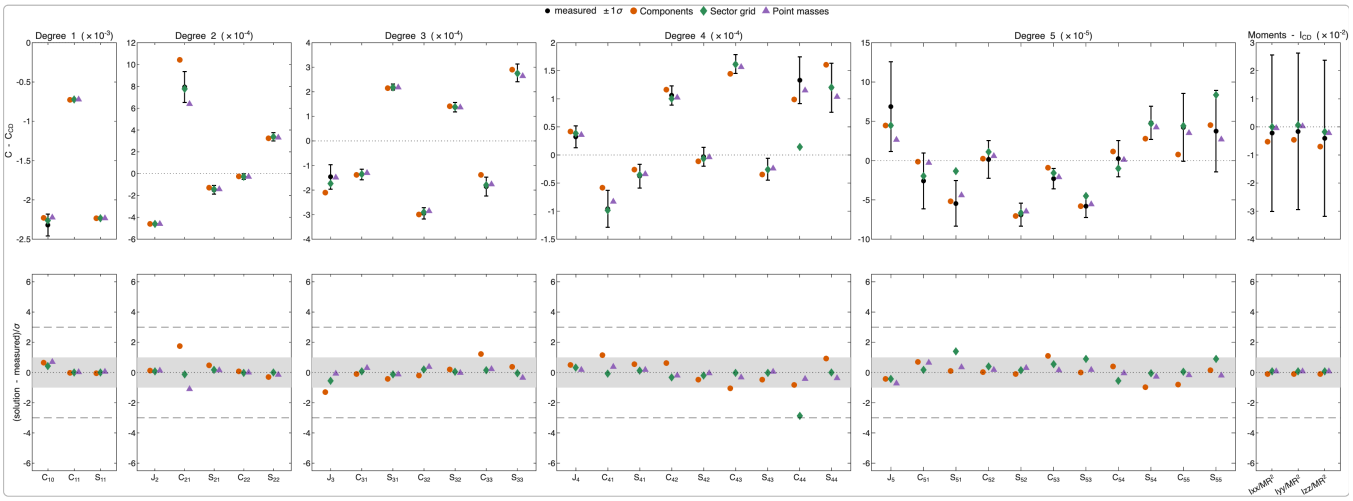
2. Where the mass is



Degrees 2 to 5 of C – C_CD as maps of latitude and longitude from the center of figure. Red where a mass added near the surface in that direction would produce the pattern, blue where a mass taken away would; each map on its own symmetric color scale. +: strongest excess; x: strongest deficit; star: direction of the center-of-mass offset.

Degree 2 puts the excess at latitude −58°, longitude −176°, and at the opposite point: an axis tilted 58° from the equator. The center of mass points to −45°, −108°, the southern side. Degree 3 has its excess at −36°, 162° and its deficit at the opposite point, and degree 5 its excess at −14°, 152°. Degrees 1, 2, 3 and 5 all point to the southern hemisphere, between longitudes 150° and −110° across 180°.

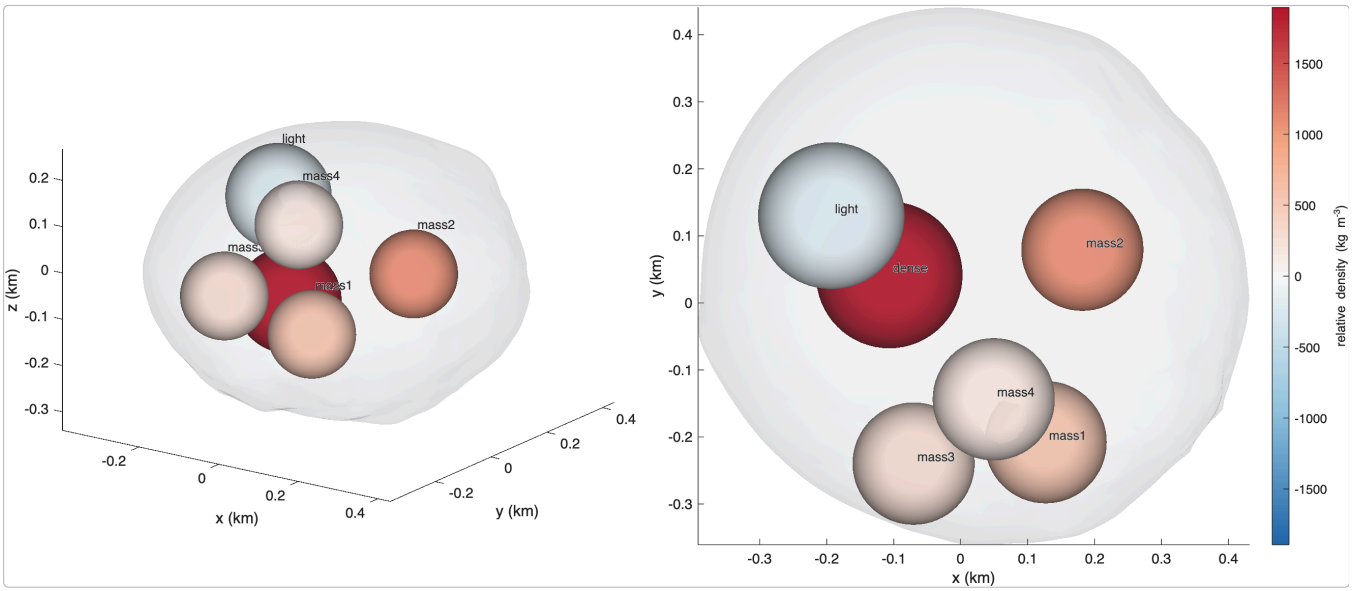
3. How well each solution fits



Top: C – C_CD measured (black, ±1σ) and of the best solution of each kind. Bottom: (solution – measured)/σ; the gray band is ±1σ, the dashed lines ±3σ.

The three match every fitted coefficient within 3σ: the point masses within about 1σ, the model with components within 2σ, the sector grid within 3σ. The sector grid needs 25 effective parameters, the model with components 24 and the point masses 30. Their AIC spans 1.3, a tie. The models of a core, a torus and a layer do not fit: a core placed along the center-of-mass offset with a tilted torus and a layer has a reduced χ^2 of 20, and with three compact masses added, 16. A core can move the center of mass and a torus can change J_2, but neither makes a tilted elongation together with a large degree 3.

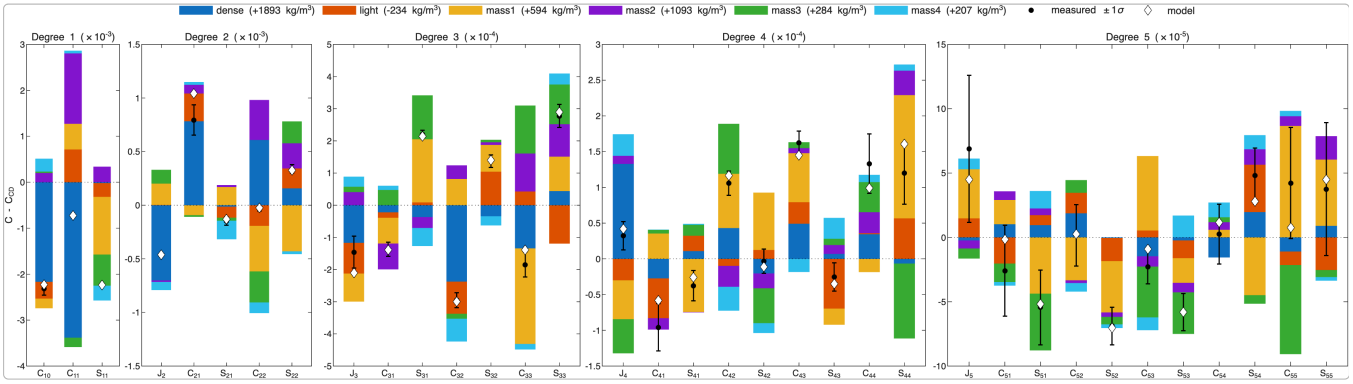
4. The model with components



The model with components, seen obliquely (left) and down the spin axis (right), colored by each component's density relative to the background density.

Component	Shape and place	Relative density (kg/m³)	Density (kg/m³)	Mass anomaly (kg); makes most of
dense	sphere, radius 110 m, 145 m from the center toward lat −33°, lon −180°	+1893 ± 213	4420	1.04×10^{10}; $\bar{C}_{11}$ −273σ, $\bar{C}_{20}$ +80σ
light	sphere, radius 110 m, 250 m from the center toward lat 25°, lon 157°	−234 ± 112	2293	-1.29×10^{9}; $\bar{C}_{11}$ +58σ, $\bar{S}_{11}$ −23σ
mass1	sphere, radius 91 m, 277 m from the center toward lat −9°, lon −66°	+594 ± 226	3121	1.89×10^{9}; $\bar{S}_{11}$ −96σ, $\bar{C}_{11}$ +45σ
mass2	sphere, radius 91 m, 172 m from the center toward lat 7°, lon 13°	+1093 ± 159	3620	3.48×10^{9}; $\bar{C}_{11}$ +124σ, $\bar{S}_{11}$ +26σ
mass3	sphere, radius 91 m, 295 m from the center toward lat 2°, lon −107°	+284 ± 253	2811	9.04×10^{8}; $\bar{S}_{11}$ −52σ, $\bar{C}_{11}$ −17σ
mass4	sphere, radius 91 m, 250 m from the center toward lat 41°, lon −80°	+207 ± 267	2734	6.56×10^{8}; $\bar{S}_{11}$ −25σ, $\bar{C}_{20}$ +9σ

Background density 2527 kg/m³ (bulk density 2606 kg/m³). Makes most of: the component's part of each coefficient in units of its 1σ.



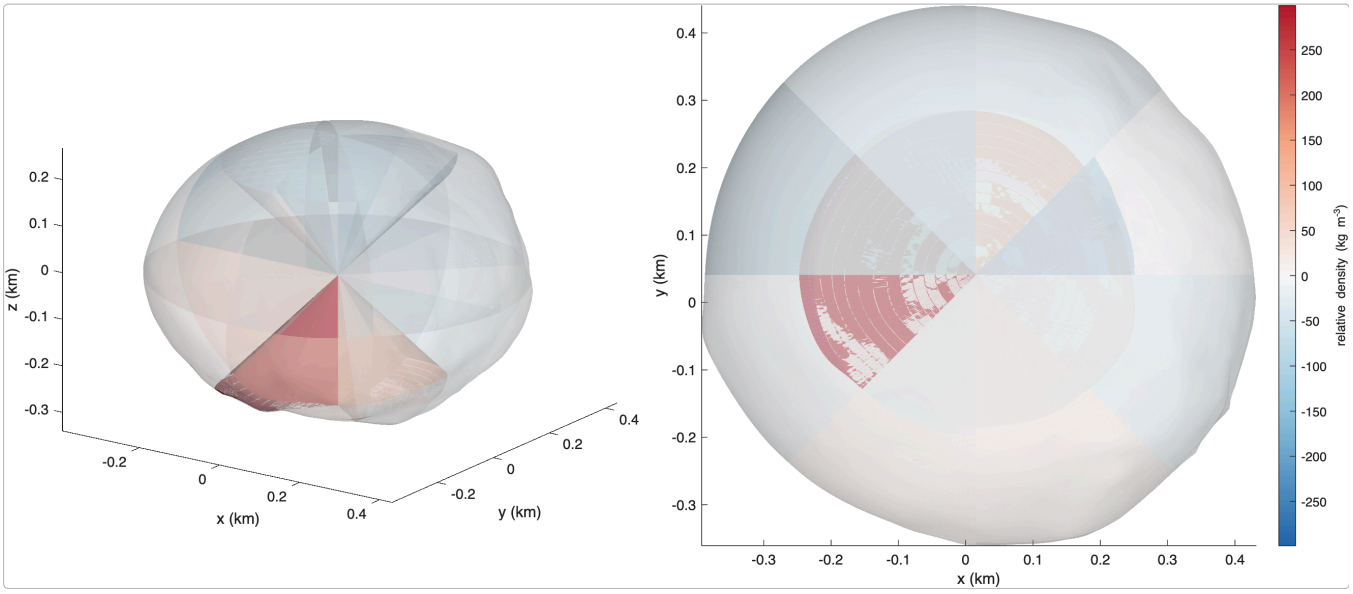
The part of each measured coefficient each component makes (stacked bars), the measured value with ±1σ (black) and the model's sum (white diamond).

The model starts from the point masses. A dense and a light sphere, each of 0.3 mean radii, are placed at the centroids of the point masses' excess and deficit, and their densities and centers are fitted. Compact masses, spheres of 91 m radius, are then added one at a time where the residuals ask for them, each kept while it passes the likelihood-ratio test at the 5% level, and every position is refitted each time. Four pass.

- **The dense sphere** (+1893 ± 213 kg/m³, radius 110 m, 145 m from the center toward latitude −33°, longitude 180°) makes most of the center-of-mass offset toward −x and the south, and of the change of J_2.
- **The light sphere** (−234 ± 112 kg/m³, 250 m from the center toward latitude 25°, longitude 157°) is the deficit above the equator.
- **The four compact masses** (+207 to +1093 kg/m³, 172 to 295 m from the center, three of them near the equator toward longitudes −66° to −107° and 13°) move the center of mass toward −y and shape degrees 3 to 5. Two are not resolved: the fourth (+207 ± 267 kg/m³) and the one toward −107° (+284 ± 253 kg/m³).

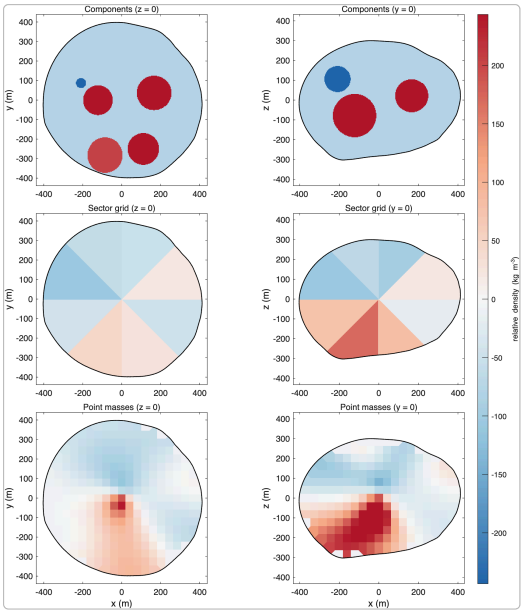
Together the six spheres move 2.8% of the mass of the body, three times as much as the sector grid or the point masses.

5. Point masses, sector grid and components side by side



The best sector grid (1 × 4 × 8), seen obliquely (left) and down the spin axis (right), colored by relative density.

Each of the 32 sectors reaches from the center to the surface. The densest are south of −45° latitude: +299 ± 54 kg/m³ toward longitudes −180° to −135°, +170 ± 53 toward 135° to 180° and +133 ± 67 toward 45° to 90°. The lightest is between 0° and 45° latitude toward longitudes 135° to 180° (−106 ± 10 kg/m³). The sectors north of 45° are not resolved.



The best solution of each kind on the planes z = 0 (left) and y = 0 (right) through the center of figure, colored by relative density on one scale. Components and sectors are drawn with their own boundaries; the point masses are drawn as the mean relative density of 41 m cells.

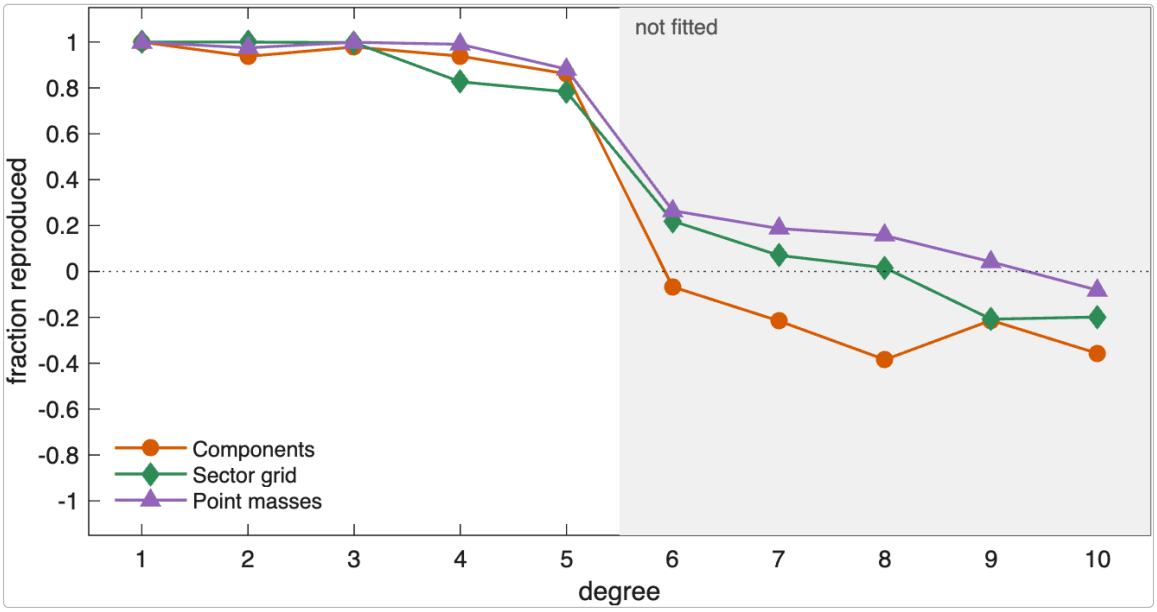
Solution	Mass moved (% of M)	Excess centered at (m)	Deficit centered at (m)
Components	2.8	(−37, −31, −47)	(−21, 26, 10)
Sector grid	0.9	(−72, −64, −108)	(−24, 98, 53)
Point masses	1.0	(−63, −64, −103)	(−11, 100, 59)

Correlation	Components	Sector grid	Point masses
Components	1.00	0.29	0.36
Sector grid	0.29	1.00	0.78
Point masses	0.36	0.78	1.00

Mass moved: the mass above the bulk density, as a percentage of the mass of the body. Excess and deficit centers are from the center of figure. Correlation: of the relative densities of the solutions, 41 m cell by 41 m cell.

The sector grid and the point masses are built differently: 32 large blocks against 2,667 point masses, with different regularization. They still place the anomaly in the same way. Each moves about 1% of the mass of the body. Each centers its excess about 140 m from the center toward latitude −49°, longitude −135°, within 10 m of the other, and its deficit about 115 m toward +y and the north. On the y = 0 plane the point masses gather the excess into a lobe 100 to 250 m below the center, where the sector grid has its densest southern sectors. The model with components puts its excess in the same direction but nearer the center, 67 m from it, and its deficit 35 m from it; it correlates 0.29 and 0.36 with the other two.

6. Against ROB's exact values



The fraction of each degree of ROB's exact C − C_CD that each solution reproduces, 1 − |solution − exact|²/|exact|². 1: all of it; 0: no better than constant density; values below −1 are drawn at −1 as open markers. The gray region was not fitted.

All three reproduce degrees 1 to 3. Of degrees 4 and 5 the point masses reproduce 99% and 88%, the model with components 94% and 86%, the sector grid 83% and 78%. Above degree 5, which none was fitted to, the point masses and the sector grid fall toward zero: 22 to 26% of degree 6, less of each degree after it. The model with components does worse than constant density at every degree above 5. Over degrees 6 to 10 the point masses reproduce 17%, the sector grid 5%, and the model with components −0.20: its compact masses fit degrees 1 to 5 with structure the hidden interior does not have. The large-scale structure the sector grid and the point masses agree on is real; its finer form is beyond what degrees 1 to 5 can show.

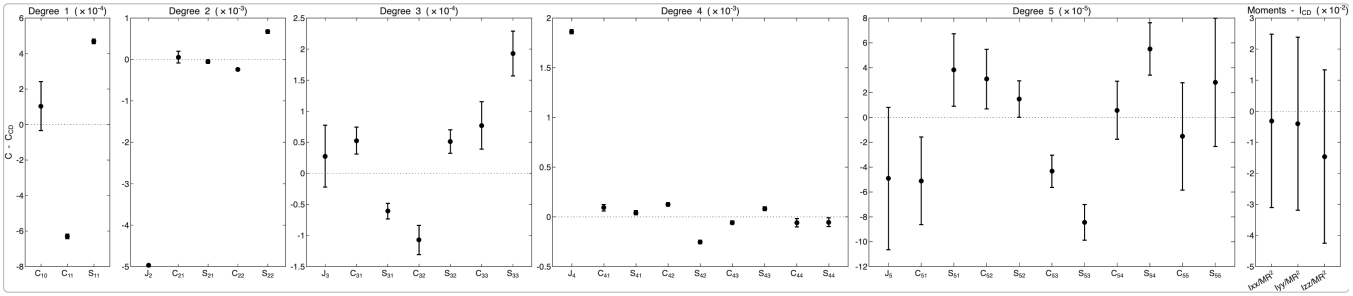
ROB-DATA3: Didymos

In ROB-DATA3 the even zonal coefficients change far more than anything else: J_2 is 570σ lower and J_4 95σ higher than at constant density. Only something symmetric about the spin axis does that, and J_2 lower means less mass in the equatorial plane. A model of three components built on that reading fits with 6 parameters: a dense core, a light thick torus about the spin axis and a dense surface layer. The point masses, given no shapes, find a dense center inside a light equatorial ring and correlate 0.68 with it cell by cell. The sector grids, whose sectors run from the center to the surface, cannot make a ring and fail. Against ROB's exact values the torus model holds to degree 7, and the point masses to degree 9.

Kind	Model	Parameters	Reduced χ²	p	AIC	Exact values, degrees 1–5	Exact values, degrees 6–10
Components	**core, torus, layer**	6.0	1.22	0.18	51.1	1.00	0.79
Point masses	2667 points	33.9	0.25	0.91	68.8	1.00	0.92
Sector grid	2 × 4 × 8	28.8	411.15	< 0.01	3851.8	0.94	−1.10

The best solution of each kind, by AIC (bold: the lowest). Parameters: the effective number of parameters. Exact values: the fraction of ROB's exact coefficients a solution reproduces, over the fitted degrees 1–5 and over degrees 6–10, which were not (1: all; 0: no better than constant density; negative: worse). See How to read these pages.

1. What the measurements show



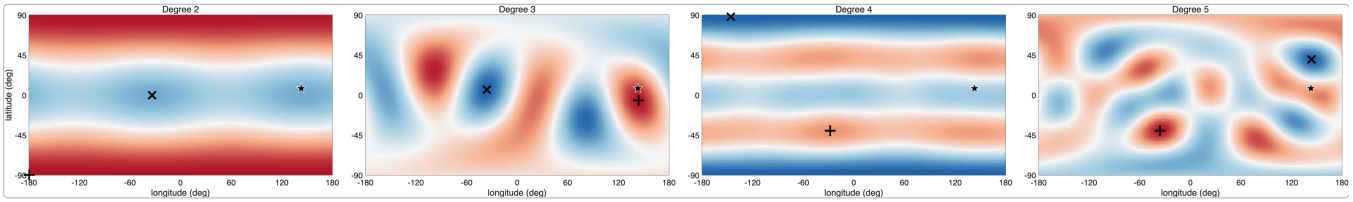
The measured coefficients minus those of the same shape at constant density, C − C_CD, with ±1σ of the Hera radio-science solution, degrees 1 to 5, and the three moments of inertia minus those at constant density. Each panel has its own scale; 0 is constant density.

Measurement	C − C_CD	1σ	In σ	What it says
$\bar{C}_{11}$	-6.30×10^{-4}	1.24×10^{-5}	−51	center of mass toward −x
$\bar{S}_{11}$	4.69×10^{-4}	1.30×10^{-5}	+36	center of mass toward +y
$\bar{C}_{20}$	4.96×10^{-3}	8.70×10^{-6}	+570	J_2 lower: mass toward the poles or the center
$\bar{C}_{22}$	-2.45×10^{-4}	2.64×10^{-5}	−9	mass toward ±y more than ±x
$\bar{S}_{22}$	6.73×10^{-4}	3.93×10^{-5}	+17	mass toward longitudes 45° and −135°
$\bar{S}_{31}$	-6.06×10^{-5}	1.25×10^{-5}	−5	order 1, strongest at longitudes −49°
$\bar{C}_{32}$	-1.07×10^{-4}	2.35×10^{-5}	−5	order 2, strongest at longitudes −103°, 77°
$\bar{S}_{33}$	1.93×10^{-4}	3.58×10^{-5}	+5	order 3, strongest at longitudes −97°, 23°, 143°
$\bar{C}_{40}$	-1.86×10^{-3}	1.97×10^{-5}	−95	J_4 higher: mass toward mid-latitudes (about 30° to 70°)
$\bar{C}_{42}$	1.26×10^{-4}	1.69×10^{-5}	+7	order 2, strongest at longitudes −32°, 148°
$\bar{S}_{42}$	-2.53×10^{-4}	1.70×10^{-5}	−15	order 2, strongest at longitudes −32°, 148°
$\bar{C}_{43}$	-5.87×10^{-5}	1.67×10^{-5}	−4	order 3, strongest at longitudes −78°, 42°, 162°
$\bar{S}_{43}$	8.15×10^{-5}	1.97×10^{-5}	+4	order 3, strongest at longitudes −78°, 42°, 162°
$\bar{C}_{53}$	-4.33×10^{-5}	1.29×10^{-5}	−3	order 3, strongest at longitudes −159°, −39°, 81°
$\bar{S}_{53}$	-8.44×10^{-5}	1.45×10^{-5}	−6	order 3, strongest at longitudes −159°, −39°, 81°

The measurements more than 3σ from constant density.

J_2 and J_4 change by 13% and 35% of their values at constant density; the odd degrees change by a few percent. A change confined to the even zonal terms is the signature of a ring or torus about the spin axis. The sign says the ring is light: J_2 lower means mass has moved out of the equatorial plane, toward the poles or the center. The center of mass is 0.56 m off center, toward −x and +y, and $\bar{C}_{22}$, $\bar{S}_{22}$ and $\bar{S}_{42}$ change by 9 to 17σ: there is something off center as well, but it is small. Degree 5 is resolved, but only just: $\bar{S}_{53}$ is 6σ and $\bar{C}_{53}$ 3σ from constant density.

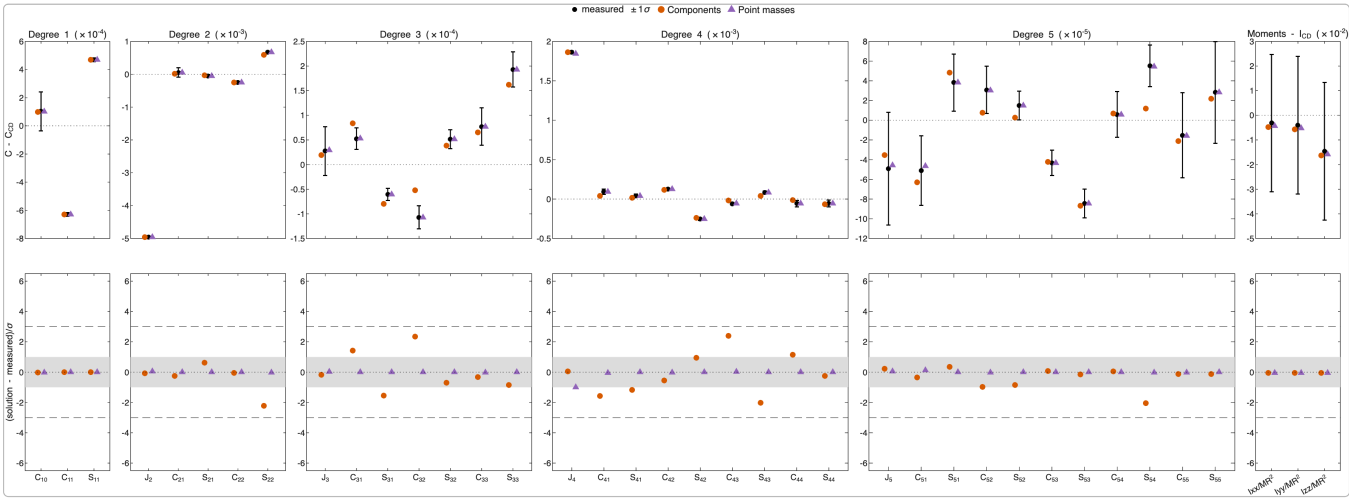
2. Where the mass is



Degrees 2 to 5 of C − C_CD as maps of latitude and longitude from the center of figure. Red where a mass added near the surface in that direction would produce the pattern, blue where a mass taken away would; each map on its own symmetric color scale. +: strongest excess; x: strongest deficit; star: direction of the center-of-mass offset.

Degree 2 is nearly a band: excess at both poles, deficit all around the equator. Degree 4 puts its excess at mid-latitudes and its deficit at the poles and the equator. Together they describe mass taken from an equatorial ring and put either nearer the center or at mid-latitudes. The model built for this package therefore has a light torus about the spin axis, a dense core at the center, and a surface layer whose density is free. The torus's ring and tube radii were chosen by the χ^2 of each pair (ring radius 1.00 or 1.05, tube radius 0.25 or 0.30 of the mean radius); the best is a ring of 383 m and a tube of 110 m.

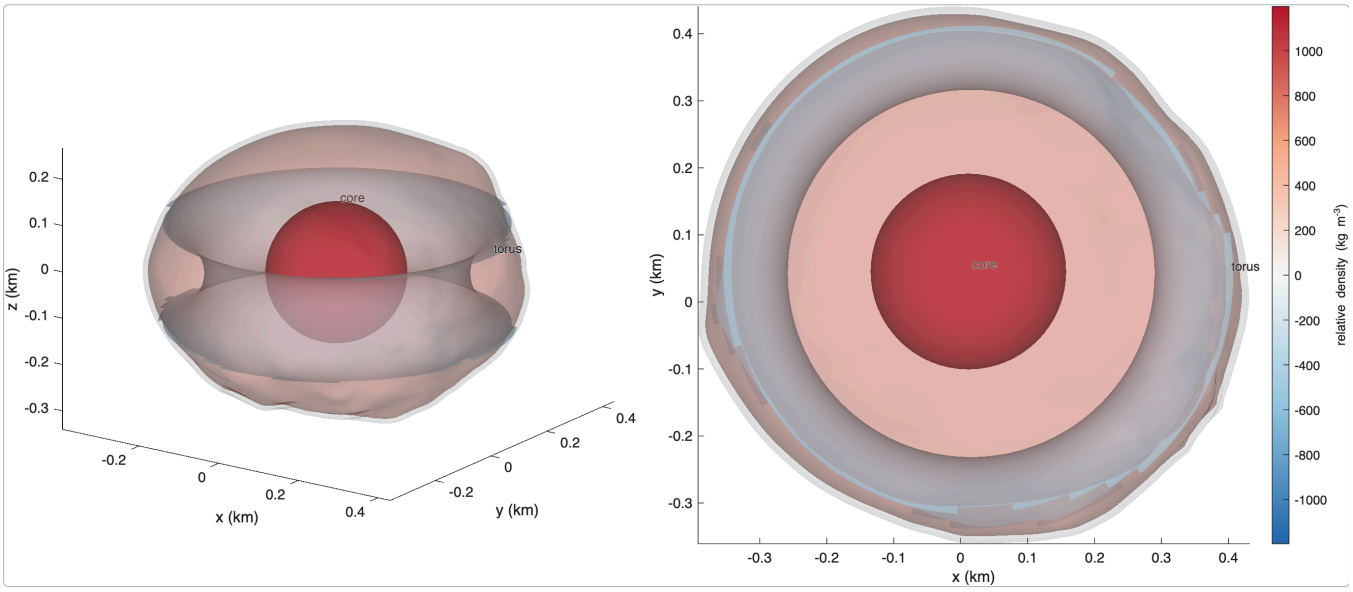
3. How well each solution fits



Top: C − C_CD measured (black, ±1σ) and of the best solution of each kind that fits. Bottom: (solution − measured)/σ; the gray band is ±1σ, the dashed lines ±3σ.

Two kinds fit: the model with components and the point masses. The model with components needs 6 parameters, the point masses 34 effective parameters; by AIC the components rank first, 18 ahead. The sector grids cannot fit (reduced χ^2 above 400). Each sector runs from the center to the surface, so a sector grid can make the equator lighter only along the whole radius, which also takes mass from the center.

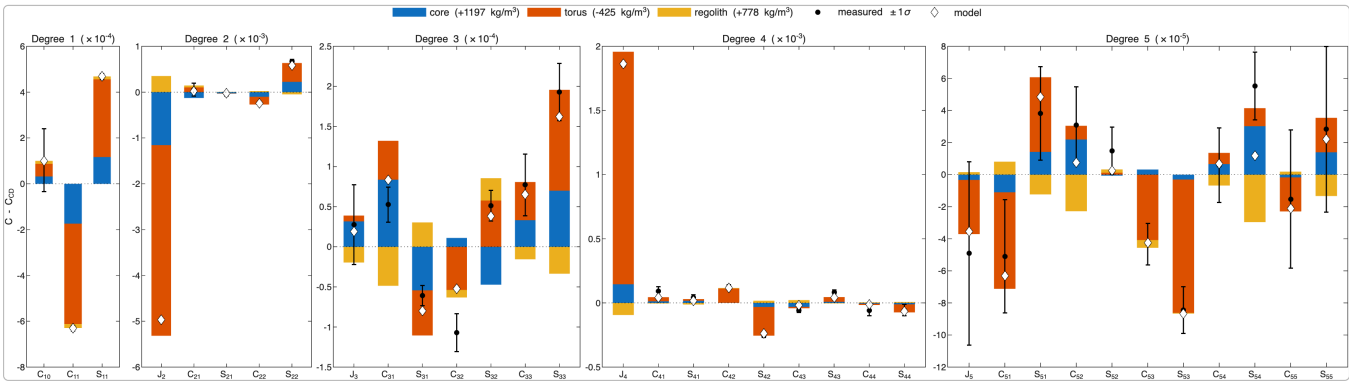
4. The model with components



The model with components, seen obliquely (left) and down the spin axis (right), colored by relative density.

Component	Shape and place	Relative density (kg/m³)	Density (kg/m³)	Mass anomaly (kg); makes most of
core	sphere, radius 146 m, 5 m from the center toward lat 9°, lon 146°	+1197 ± 104	3749	1.56×10^{10}; $\bar{C}_{20}$ +133σ, $\bar{C}_{11}$ −14σ
torus	torus, ring radius 383 m, tube radius 110 m, axis 0° from z, center 0 m from the center	−425 ± 7	2126	-1.93×10^{10}; $\bar{C}_{20}$ +478σ, $\bar{C}_{40}$ −92σ
regolith	surface layer, 0 to 11 m deep	+778 ± 252	3330	1.37×10^{10}; $\bar{C}_{20}$ −40σ, $\bar{C}_{40}$ +5σ

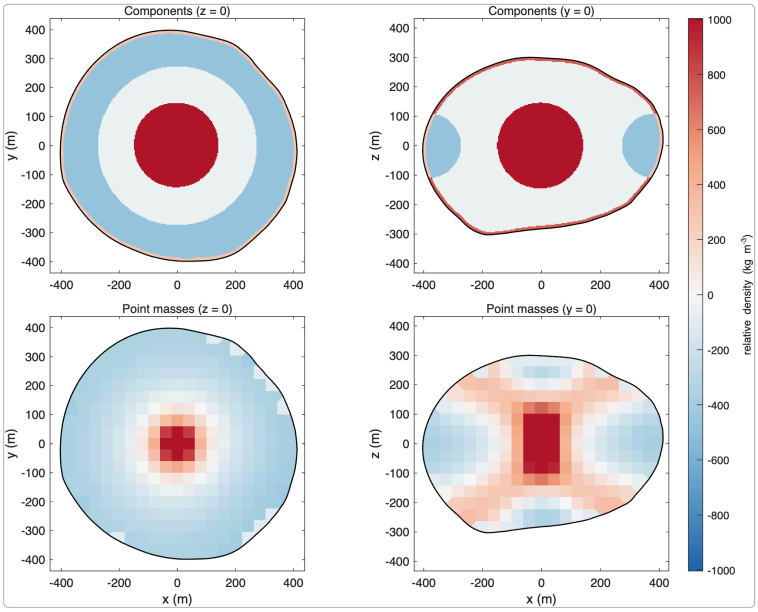
Background density 2552 kg/m³ (bulk density 2601 kg/m³). Makes most of: the component's part of each coefficient in units of its 1σ.



The part of each measured coefficient each component makes (stacked bars), the measured value with ±1σ (black) and the model's sum (white diamond).

- **The torus** (−425 ± 7 kg/m³) makes nearly all of J_2 and J_4: 478σ of $\bar{C}_{20}$ and 92σ of $\bar{C}_{40}$. Its ring of 383 m lies just inside the equatorial surface; the part of its tube inside the body is a light belt about 120 m deep under the equator.
- **The core** (+1197 ± 104 kg/m³, radius 146 m, 5 m from the center) puts back at the center part of the mass the torus takes away, and moves the center of mass toward −x and +y.
- **The surface layer** (+778 ± 252 kg/m³, 11 m thick) corrects J_2 by −40σ.

5. Point masses and components side by side



The best solution of each kind that fits, on the planes z = 0 (left) and y = 0 (right) through the center of figure, colored by relative density on one scale. Components are drawn with their own boundaries; the point masses are drawn as the mean relative density of 41 m cells.

Solution	Mass moved (% of M)	Excess centered at (m)	Deficit centered at (m)
Components	3.3	(−3, 2, −1)	(6, −5, −1)
Point masses	3.9	(−3, 3, 1)	(9, −6, −2)

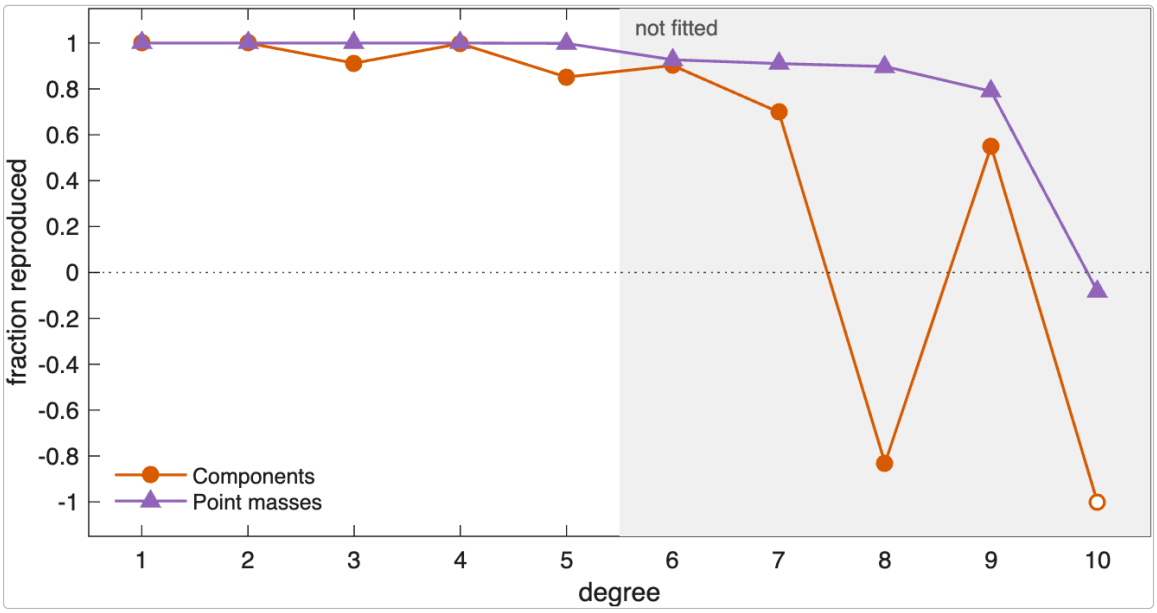
Correlation	Components	Point masses
Components	1.00	0.68
Point masses	0.68	1.00

Mass moved: the mass above the bulk density, as a percentage of the mass of the body. Excess and deficit centers are from the center of figure. Correlation: of the relative densities of the solutions, 41 m cell by 41 m cell.

The point masses come close to the model with components. On both planes they put a dense block at the center, about the size of the core, inside a light equatorial region that reaches the surface where the torus does. On the y = 0 plane they also put dense material at mid-latitudes near the surface, which the model with components gives to its dense surface layer. The two correlate 0.68 cell by cell, as on ROB-DATA1.

---

6. Against ROB's exact values



The fraction of each degree of ROB's exact C − C_CD that each solution reproduces, 1 − |solution − exact|²/|exact|². 1: all of it; 0: no better than constant density; values below −1 are drawn at −1 as open markers. The gray region was not fitted.

Both reproduce 85 to 100% of each fitted degree, 1 to 5, and predict 90 to 93% of degree 6. The model with components holds to degree 7 (70%), fails at degree 8 and reproduces 55% of degree 9. The point masses hold to degree 9 (79 to 93%). Over degrees 6 to 10 the point masses reproduce 92% and the model with components 79%. The light equatorial ring and the dense center are what the hidden interior has. The point masses, by not committing to a torus of one size and one density, describe its finer form better than the torus does.

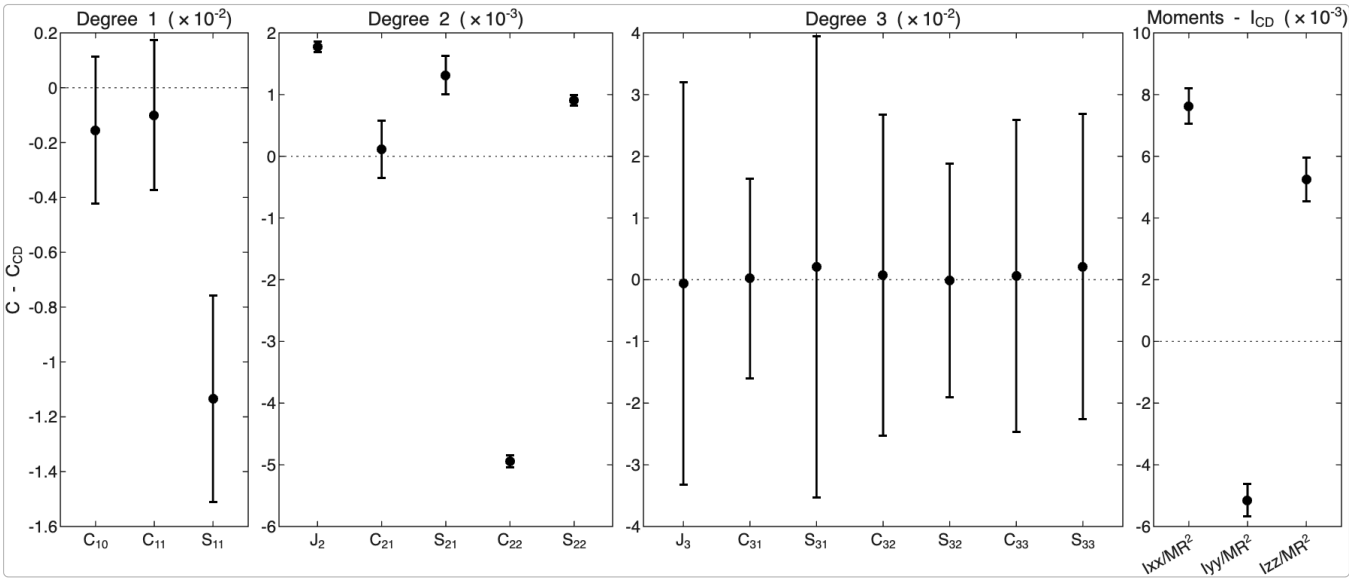
ROB-DATA4: Dimorphos

ROB-DATA4 is the package where the measurements barely decide and the truth does. Degree 2 and the moments of inertia say mass has moved toward the two ends of the y-axis, and degree 1 that the −y end is heavier. A dense core placed along the center-of-mass offset, with a torus and a surface layer, fits with 6 parameters and ranks first by AIC, 4.6 ahead of the sector grid and 5.4 ahead of the point masses. All three solutions put the excess on the −y side; they differ in how compact it is. ROB's exact values favor the compact core: it reproduces 81% of degrees 4 to 10, which no solution was fitted to, the sector grid 25%, and the point masses do worse than constant density. At the GRASS surface point above it the core predicts an acceleration 3.7% too large, the sector grid and the point masses 7 to 9% too small.

Kind	Model	Parameters	Reduced χ^2	p	AIC	Exact values, degrees 1–3	Exact values, degrees 4–10
Components	**core, torus, layer**	6.0	0.02	1.00	12.3	0.98	0.81
Sector grid	2 × 4 × 8	8.0	0.10	1.00	16.9	0.89	0.25
Point masses	2674 points	8.6	0.05	1.00	17.7	0.88	−0.33

The best solution of each kind, by AIC (bold: the lowest). Parameters: the effective number of parameters. Exact values: the fraction of ROB's exact coefficients a solution reproduces, over the fitted degrees 1–3 and over degrees 4–10, which were not (1: all; 0: no better than constant density; negative: worse). See How to read these pages.

1. What the measurements show



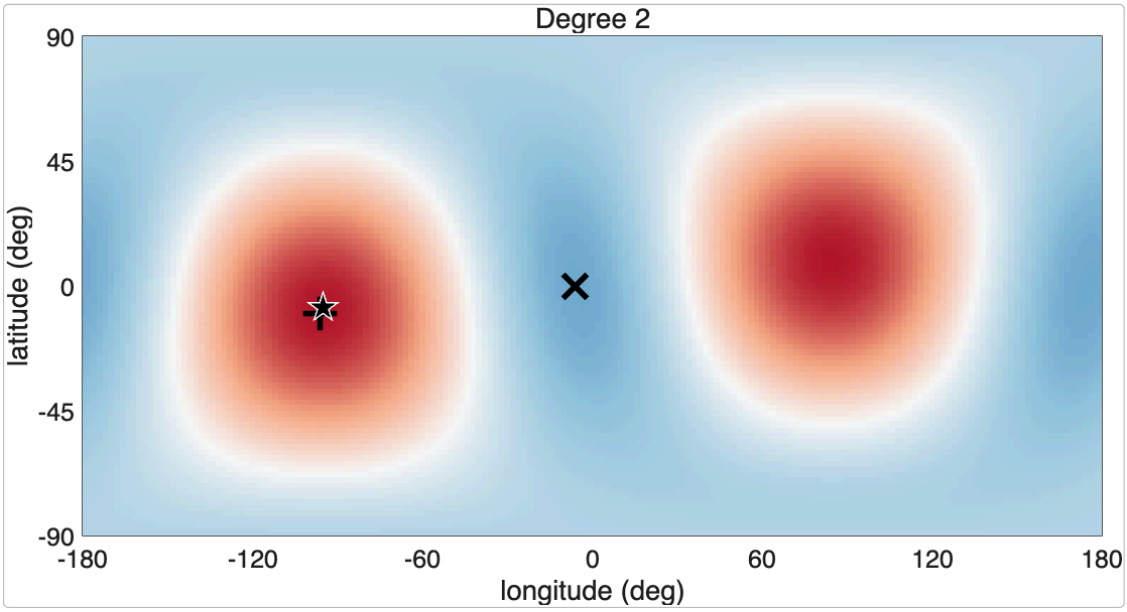
The measured coefficients minus those of the same shape at constant density, C − C_CD, with ±1σ of the Hera radio-science solution, degrees 1 to 3, and the three moments of inertia minus those at constant density. Each panel has its own scale; 0 is constant density. Degree 3 is fitted, but its 1σ is about 20 times its anomaly.

Measurement	C − C_CD	1σ	In σ	What it says
$\bar{S}_{11}$	-1.13×10^{-2}	3.76×10^{-3}	−3	center of mass toward −y
$\bar{C}_{20}$	-1.77×10^{-3}	8.57×10^{-5}	−21	J_2 higher: mass toward the equator
$\bar{S}_{21}$	1.32×10^{-3}	3.13×10^{-4}	+4	mass toward +y in the north, −y in the south
$\bar{C}_{22}$	-4.94×10^{-3}	9.93×10^{-5}	−50	mass toward ±y more than ±x
$\bar{S}_{22}$	9.09×10^{-4}	8.75×10^{-5}	+10	mass toward longitudes 45° and −135°
$I_{xx}\,/\,MR^2$	7.62×10^{-3}	5.74×10^{-4}	+13	mass farther from the x-axis
$I_{yy}\,/\,MR^2$	-5.15×10^{-3}	5.26×10^{-4}	−10	mass closer to the y-axis
$I_{zz}\,/\,MR^2$	5.24×10^{-3}	7.12×10^{-4}	+7	mass farther from the z-axis

The measurements more than 3σ from constant density.

$\bar{C}_{22}$ is 50σ below its value at constant density: mass has moved toward the ends of the y-axis. The moments of inertia agree: I_{yy} is smaller (mass closer to the y-axis), I_{xx} and I_{zz} larger. J_2 is 21σ higher, mass toward the equator. The center of mass is 1.8 m toward −y, 3σ: the −y end carries more.

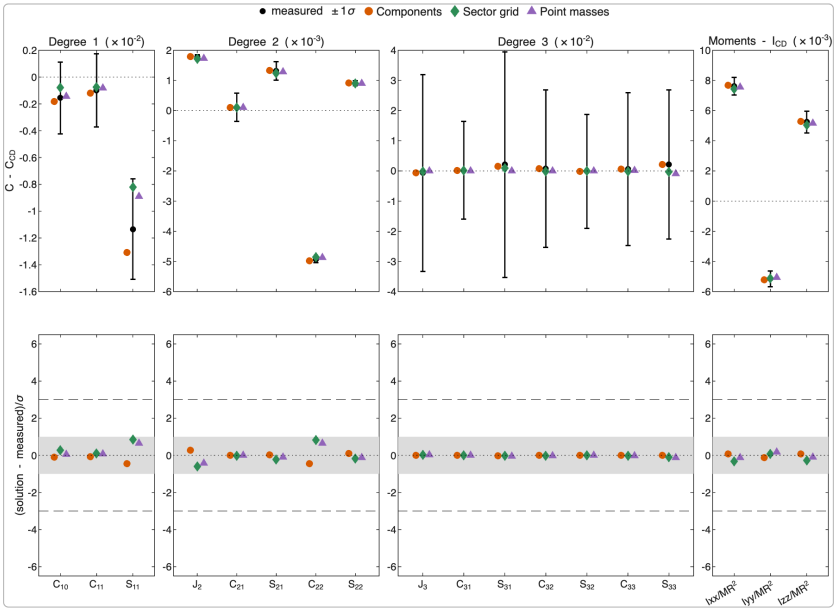
2. Where the mass is



Degree 2 of C − C_CD as a map of latitude and longitude from the center of figure. Red where a mass added near the surface in that direction would produce the pattern, blue where a mass taken away would. +: strongest excess; x: strongest deficit; star: direction of the center-of-mass offset.

Degree 2 puts the excess at both ends of an axis through latitude −9°, longitude −95° and the opposite point: the y-axis, tilted 9°. The center of mass points to the same place, −8°, −95°. The model built for this package therefore places a dense core along the center-of-mass offset by its χ^2 profile and then fits its position by Gauss–Newton, with a torus and a surface layer for the part of J_2 the core does not make.

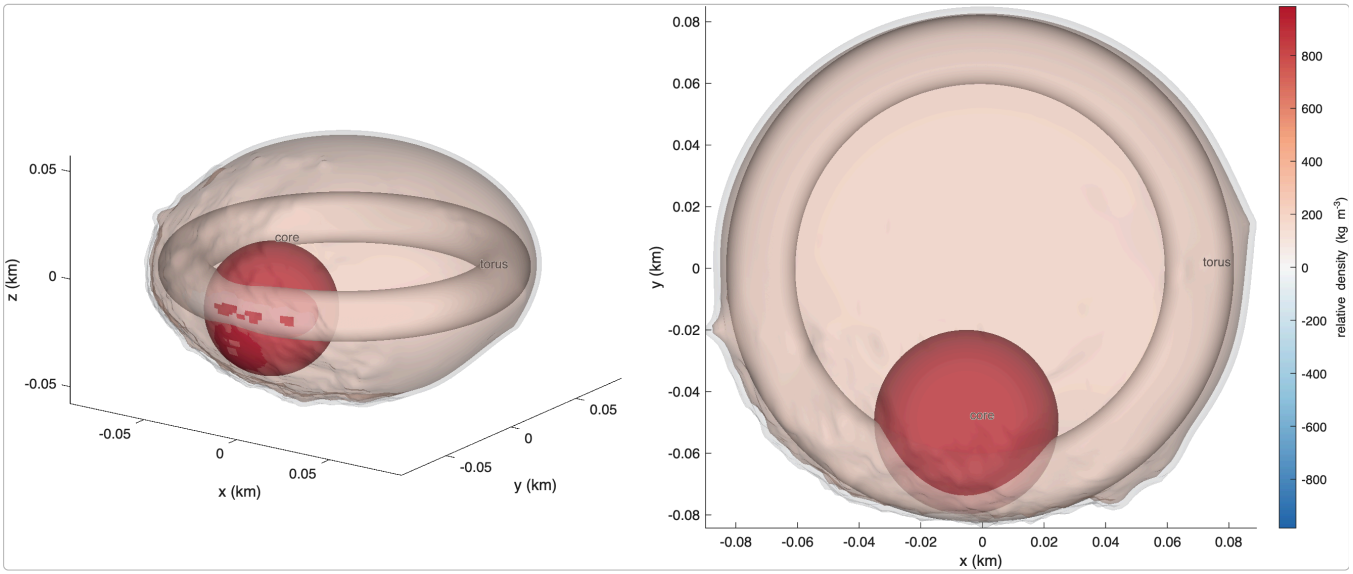
3. How well each solution fits



Top: C − C_CD measured (black, ±1σ) and of the best solution of each kind. Bottom: (solution − measured)/σ; the gray band is ±1σ, the dashed lines ±3σ.

All three match every measurement within 1σ. The model with components needs 6 parameters, the sector grid and the point masses 8.0 and 8.6 effective parameters. The components rank first by 4.6 in AIC, a preference but not a strong one. With 18 measurements, degree 2 and the moments determine where the excess is but not its form.

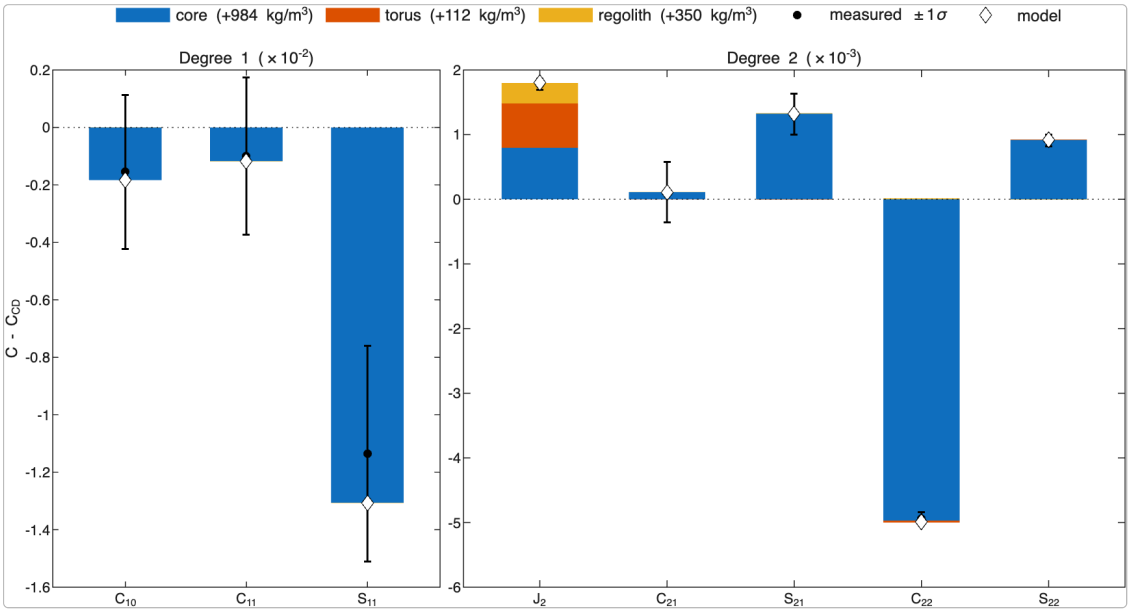
4. The model with components



The model with components, seen obliquely (left) and down the spin axis (right), colored by each component's density relative to the background density.

Component	Shape and place	Relative density (kg/m³)	Density (kg/m³)	Mass anomaly (kg); makes most of
core	sphere, radius 30 m, 50 m from the center toward lat −8°, lon −95°	+984 ± 514	2420	1.11×10^{8}; $\bar{C}_{22}$ −50σ, $\bar{S}_{22}$ +10σ
torus	torus, ring radius 71 m, tube radius 11 m, axis 0° from z, center 0 m from the center	+112 ± 104	1548	1.99×10^{7}; $\bar{C}_{20}$ −8σ, $\bar{C}_{22}$ −0σ
regolith	surface layer, 0 to 2 m deep	+350 ± 475	1786	5.29×10^{7}; $\bar{C}_{20}$ −4σ, $\bar{C}_{22}$ +0σ

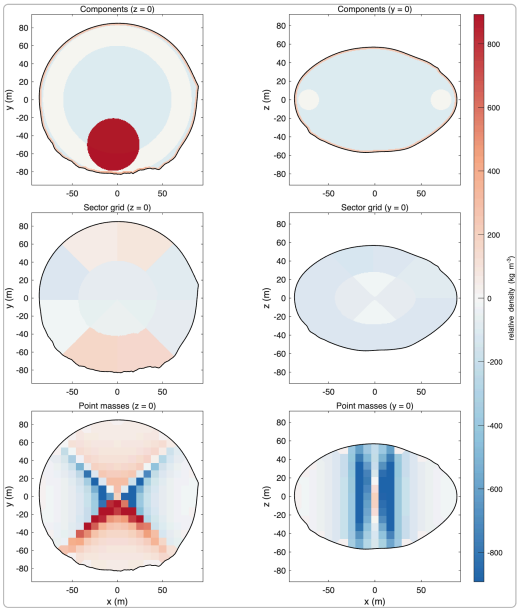
Background density 1436 kg/m³ (bulk density 1540 kg/m³). Makes most of: the component's part of each coefficient in units of its 1σ.



The part of each measured coefficient each component makes (stacked bars), the measured value with ±1σ (black) and the model's sum (white diamond).

The core alone makes $\bar{C}_{22}$ and $\bar{S}_{22}$ and the center-of-mass offset. It is a sphere of 30 m radius, 50 m from the center toward –y, 984 ± 514 kg/m³ denser than the background. The torus and the layer make the rest of J_2, and neither is resolved. Degree 2 and the moments allow a denser core closer in or a lighter one farther out: from where its χ^2 profile placed it, Gauss–Newton moves the core outward and lowers its density, and the wide 1σ of its density says how loosely the two are held apart.

5. Point masses and components side by side



The best solution of each kind on the planes z = 0 (left) and y = 0 (right) through the center of figure, colored by relative density on one scale. Components and sectors are drawn with their own boundaries; the point masses are drawn as the mean relative density of 8.3 m cells.

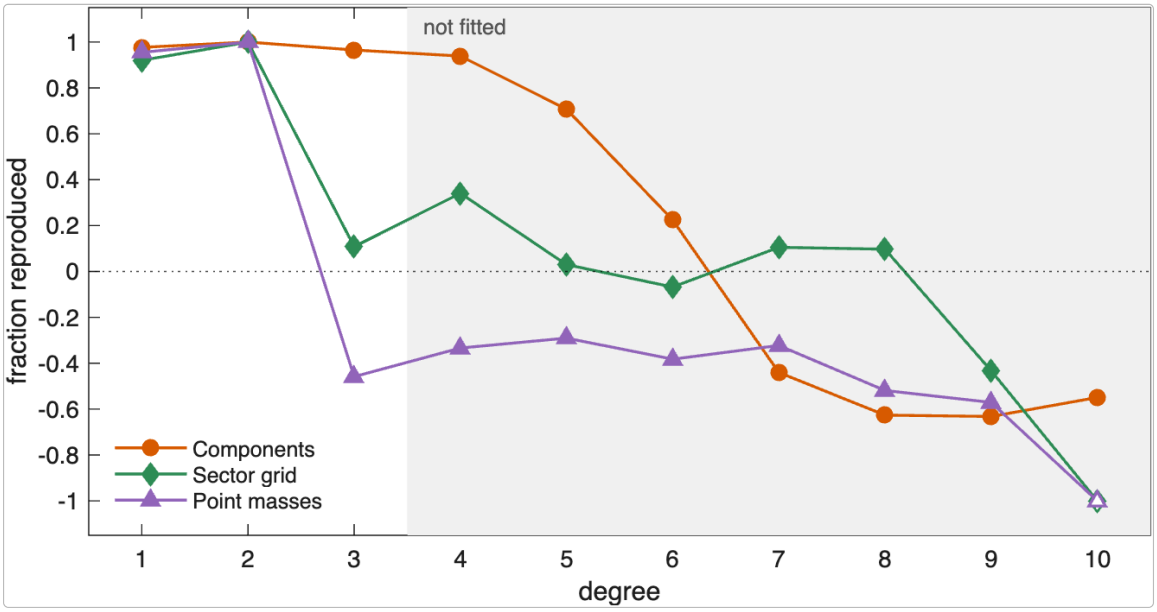
Solution	Mass moved (% of M)	Excess centered at (m)	Deficit centered at (m)
Components	3.7	(−4, −49, −7)	(0, 5, 1)
Sector grid	2.6	(−3, −34, −3)	(1, 8, 2)
Point masses	6.7	(−2, −15, −4)	(0, 5, 0)

Correlation	Components	Sector grid	Point masses
Components	1.00	0.49	0.15
Sector grid	0.49	1.00	0.34
Point masses	0.15	0.34	1.00

Mass moved: the mass above the bulk density, as a percentage of the mass of the body. Excess and deficit centers are from the center of figure. Correlation: of the relative densities of the solutions, 8.3 m cell by 8.3 m cell.

All three put the excess on the −y side: its center is 49 m toward −y for the components, 34 m for the sector grid and 15 m for the point masses. The point masses mimic the core. On the z = 0 plane they gather a dense lobe 10 to 50 m toward −y, where the core is, with light bands on either side, but they keep it closer to the center and spread it out. The sector grid spreads it over the whole −y sector of its outer shell. The coefficients cannot choose between these forms: the cell-by-cell correlation is 0.49 between the components and the sector grid, 0.34 between the sector grid and the point masses, and 0.15 between the components and the point masses.

6. Against ROB's exact values and the GRASS point



The fraction of each degree of ROB's exact C − C_CD that each solution reproduces, 1 − |solution − exact|²/|exact|². 1: all of it; 0: no better than constant density; values below −1 are drawn at −1 as open markers. The gray region was not fitted.

Beyond degree 2 the three separate. Degree 3 is fitted, but its 1σ is so large that the fit does not constrain it: the core reproduces 97% of it, the sector grid 11%, and the point masses do worse than constant density. Over degrees 4 to 10, which no solution was fitted to, the core reproduces 81%: 94% of degree 4, 71% of degree 5, 22% of degree 6, and none of degrees 7 to 10. The sector grid reproduces 25% and the point masses do worse than constant density (−0.33). A compact core makes the low degrees the hidden interior has; a spread excess does not.

Solution	Acceleration minus ROB's (% of ROB's)	Angle from ROB's
Components	+3.7%	1.5°
Sector grid	−7.3%	0.7°
Point masses, sensitivity weighting	−9.3%	0.8°
Point masses, equal weighting	−6.6%	0.3°
Constant density	−13.3%	0.9°

The acceleration each solution predicts at the GRASS surface point minus ROB's, in magnitude (% of ROB's magnitude) and direction, on the full-resolution shape model.

The GRASS point lies on the surface at the −y end, above the core. There the excess matters most: constant density predicts an acceleration 13.3% too small. The core predicts it 3.7% too large, 1.5° from ROB's direction; the sector grid and the point masses, whose excess is farther from the surface, 7.3% and 9.3% too small. One surface measurement over the anomaly separates solutions that the gravity coefficients cannot. Here the truth lies between them: the core puts too much mass close to the point, the spread solutions too little.

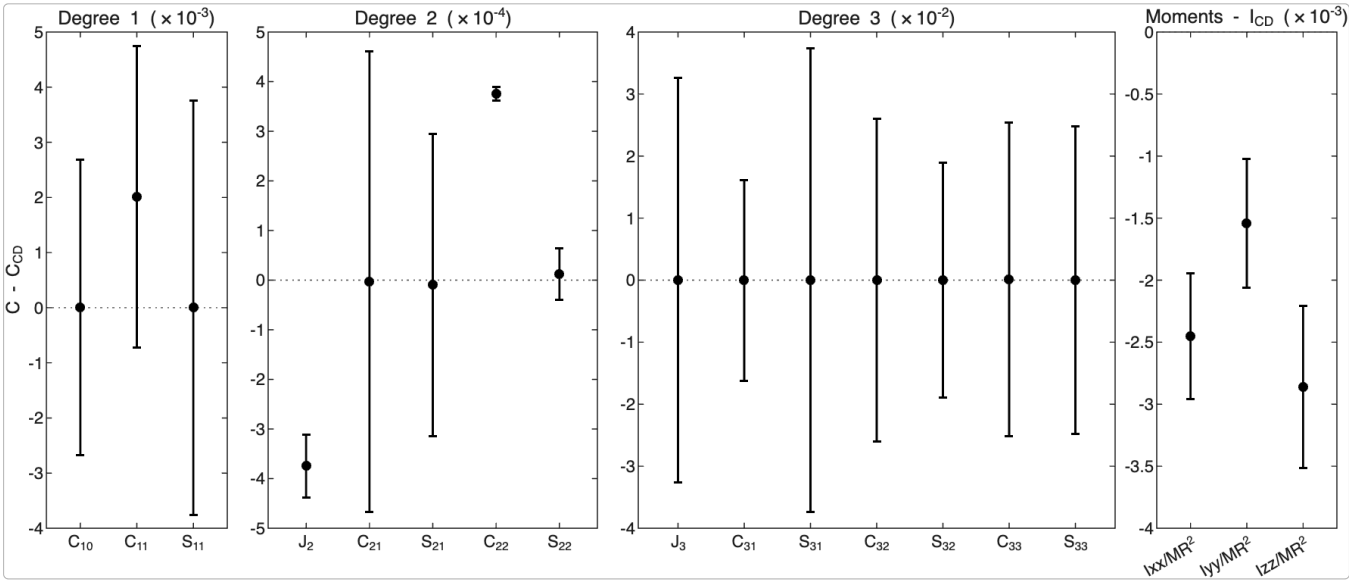
ROB-DATA5: Dimorphos

ROB-DATA5 has the smallest anomaly of the six packages. Only J_2, $\bar{C}_{22}$ and two moments of inertia differ from constant density by more than 3σ. A single dense sphere 28 m from the center toward +x explains them with 4 parameters and ranks first. The point masses tie with it in AIC. They put the same mass at the center, elongated along x like the sphere, but not to one side: the center-of-mass offset the sphere makes is only 0.7σ in the measurements, and the point masses do not pay for it. ROB's exact values show the offset is real. The sphere reproduces 91% of degrees 1 to 3 and 87 to 99% of each degree from 6 to 10. It reproduces 99% of degree 10, more than any solution of the other five packages (at most 76%, on ROB-DATA1).

Kind	Model	Parameters	Reduced χ²	p	AIC	Exact values, degrees 1–3	Exact values, degrees 4–10
Components	**core**	4.0	0.03	1.00	8.4	0.91	0.91
Point masses	2674 points	4.5	0.10	1.00	10.3	0.15	0.79
Sector grid	2 × 4 × 8	7.8	0.10	1.00	16.6	0.98	−0.70

The best solution of each kind, by AIC (bold: the lowest). Parameters: the effective number of parameters. Exact values: the fraction of ROB's exact coefficients a solution reproduces, over the fitted degrees 1–3 and over degrees 4–10, which were not (1: all; 0: no better than constant density; negative: worse). See How to read these pages.

1. What the measurements show



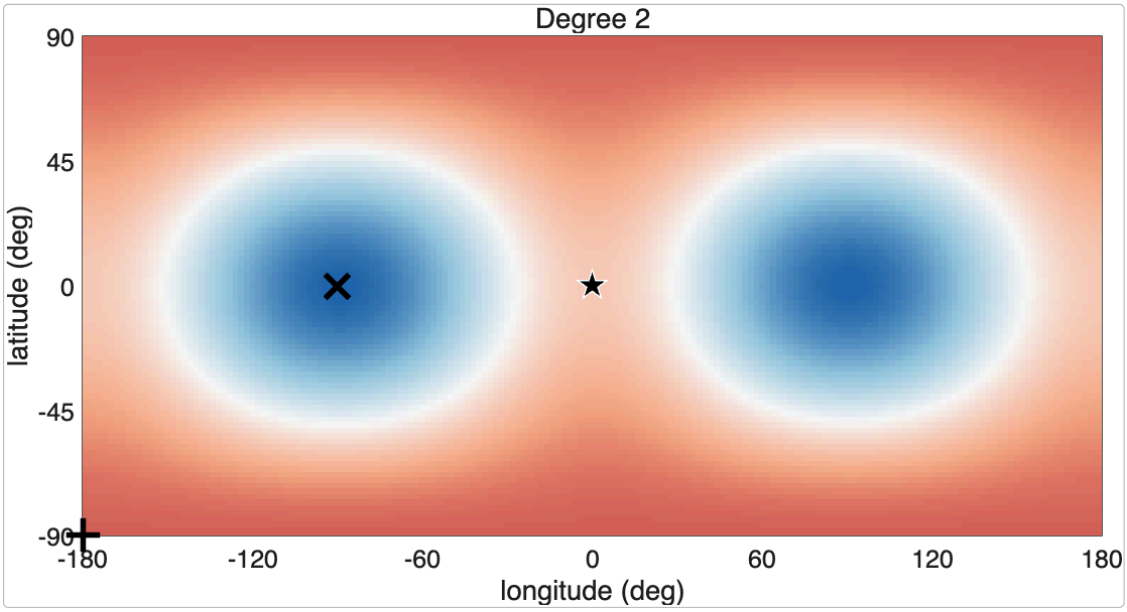
The measured coefficients minus those of the same shape at constant density, C − C_CD, with ±1σ of the Hera radio-science solution, degrees 1 to 3, and the three moments of inertia minus those at constant density. Each panel has its own scale; 0 is constant density. Degree 3 is fitted, but its 1σ is about 600 times its anomaly.

Measurement	C − C_CD	1σ	In σ	What it says
$\bar{C}_{20}$	3.74×10^{-4}	6.33×10^{-5}	+6	J_2 lower: mass toward the poles or the center
$\bar{C}_{22}$	3.75×10^{-4}	1.38×10^{-5}	+27	mass toward ±x more than ±y
$I_{xx}\,/\,MR^2$	-2.45×10^{-3}	5.05×10^{-4}	−5	mass closer to the x-axis
$I_{zz}\,/\,MR^2$	-2.86×10^{-3}	6.53×10^{-4}	−4	mass closer to the z-axis

The measurements more than 3σ from constant density.

$\bar{C}_{22}$ is 27σ higher than at constant density: mass has moved toward the ends of the x-axis. I_{xx} is 5σ smaller (mass closer to the x-axis) and I_{zz} 4σ smaller (mass closer to the spin axis). J_2 is 6σ lower. The center of mass is 0.31 m toward +x, 0.7σ: by itself, not a detection.

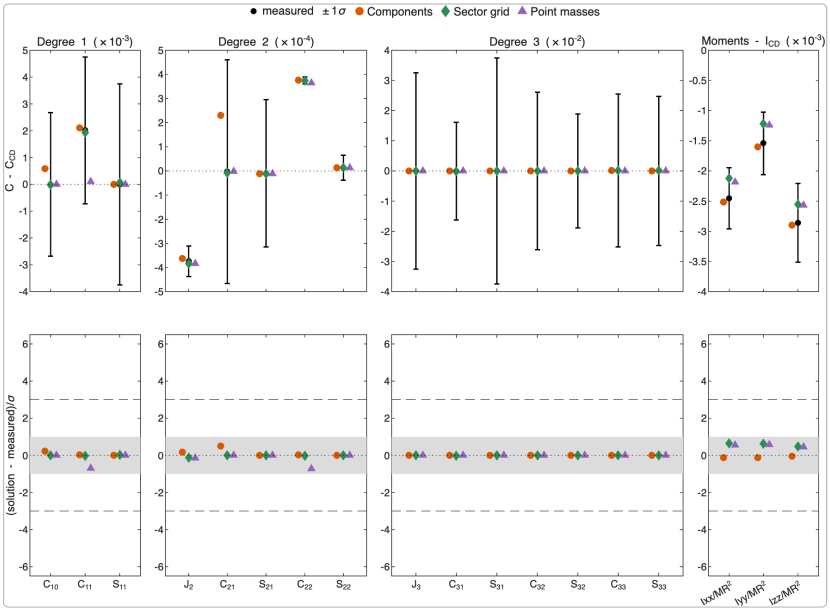
2. Where the mass is



Degree 2 of C − C_CD as a map of latitude and longitude from the center of figure. Red where a mass added near the surface in that direction would produce the pattern, blue where a mass taken away would. +: strongest excess; x: strongest deficit; star: direction of the center-of-mass offset.

Degree 2 takes mass from the ends of the y-axis and puts it at the poles and the ends of the x-axis. The smaller I_{xx} and I_{zz} say the added mass is close to the x-axis and to the center, not near the surface. Mass gathered on the x-axis inside the body makes this, and the 0.7σ offset toward +x suggests it is on the +x side. The model built for this package places a dense core along that offset by its χ² profile.

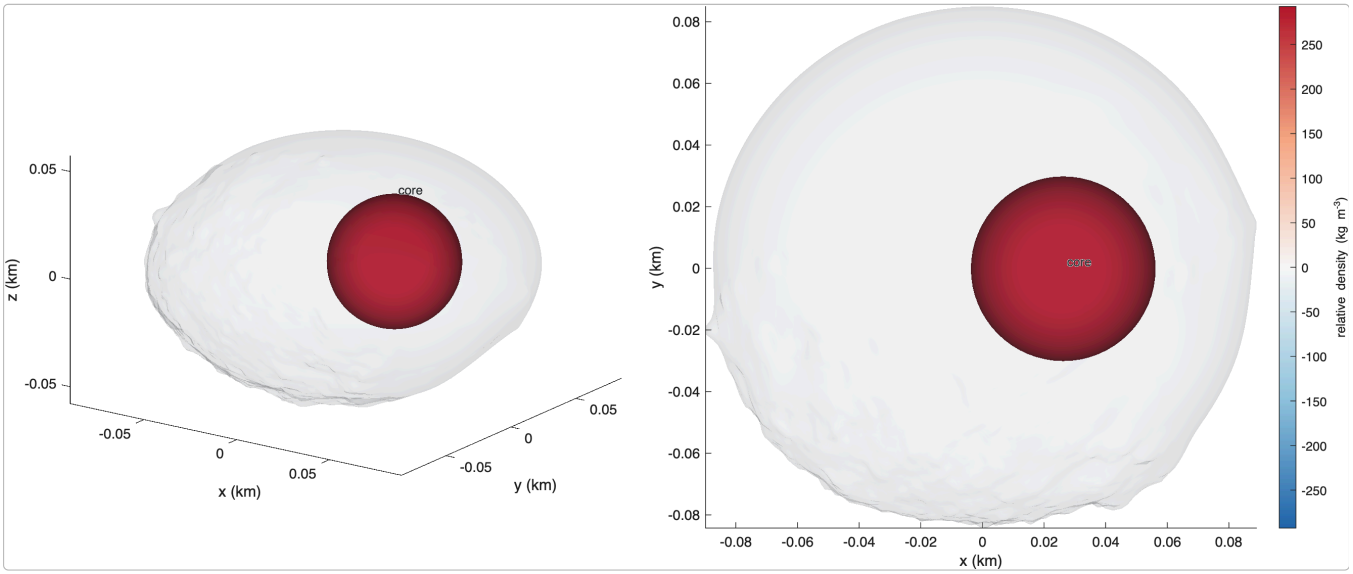
3. How well each solution fits



Top: C − C_CD measured (black, ±1σ) and of the best solution of each kind. Bottom: (solution − measured)/σ; the gray band is ±1σ, the dashed lines ±3σ.

Every kind fits within 1σ. The single sphere needs 4 parameters and the point masses 4.5; they are 1.9 apart in AIC, a tie. The sector grid fits as closely with 7.8 effective parameters and ranks 8 behind. Adding a torus or a layer to the sphere lowers χ² by less than the 2 per parameter AIC asks.

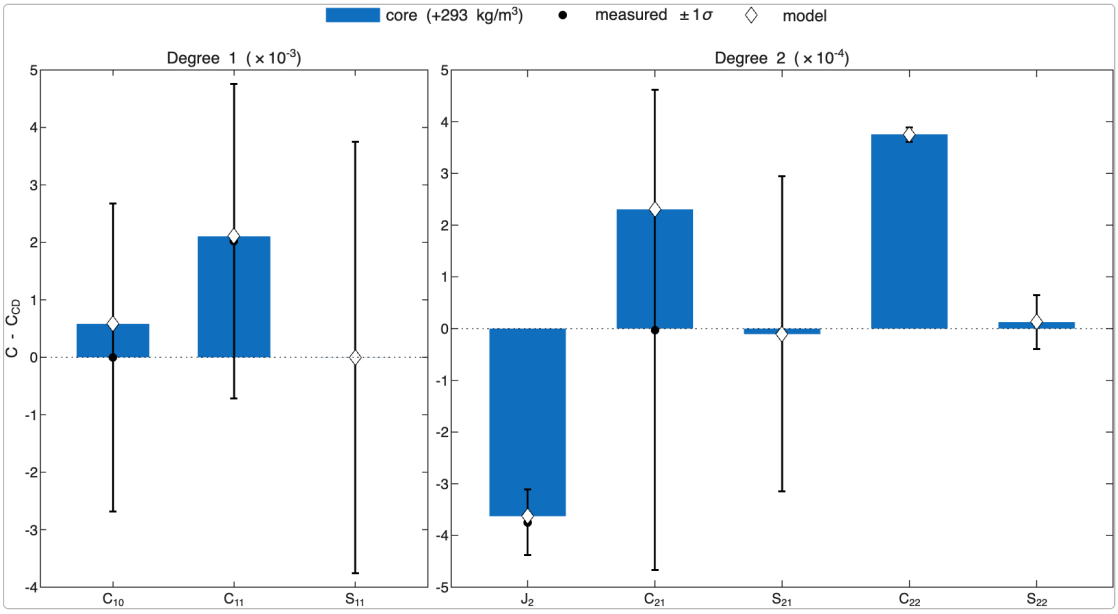
4. The model with components



The model with components, seen obliquely (left) and down the spin axis (right), colored by the core's density relative to the background density.

Component	Shape and place	Relative density (kg/m³)	Density (kg/m³)	Mass anomaly (kg); makes most of
core	sphere, radius 30 m, 28 m from the center toward lat 15°, lon 0°	+293 ± 25	1813	3.30×10^{7}; $\bar{C}_{22}$ +27σ, $\bar{C}_{20}$ +6σ

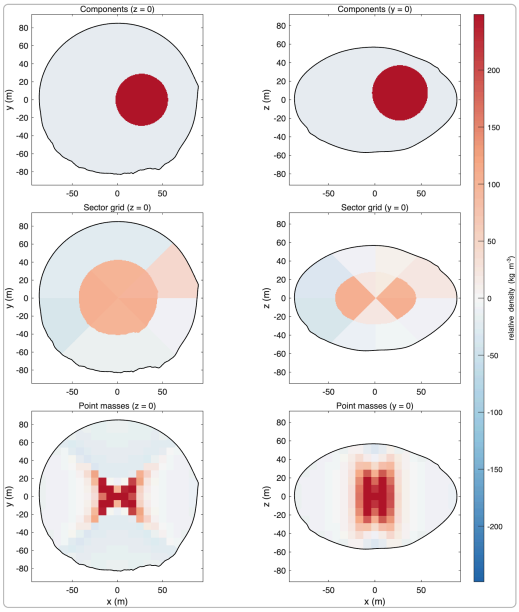
Background density 1521 kg/m³ (bulk density 1539 kg/m³). Makes most of: the component's part of each coefficient in units of its 1σ.



The part of each measured coefficient the core makes, the measured value with ±1σ (black) and the model's sum (white diamond).

The core is a sphere of 30 m radius, 28 m from the center toward +x and 15° above the equator, 293 ± 25 kg/m³ denser than the background. Its mass anomaly of 3.3 × 10⁷ kg is 1.2% of the mass of the body. It makes $\bar{C}_{22}$, J_2 and the center-of-mass offset.

5. Point masses and components side by side



The best solution of each kind on the planes z = 0 (left) and y = 0 (right) through the center of figure, colored by relative density on one scale. Components and sectors are drawn with their own boundaries; the point masses are drawn as the mean relative density of 8.3 m cells.

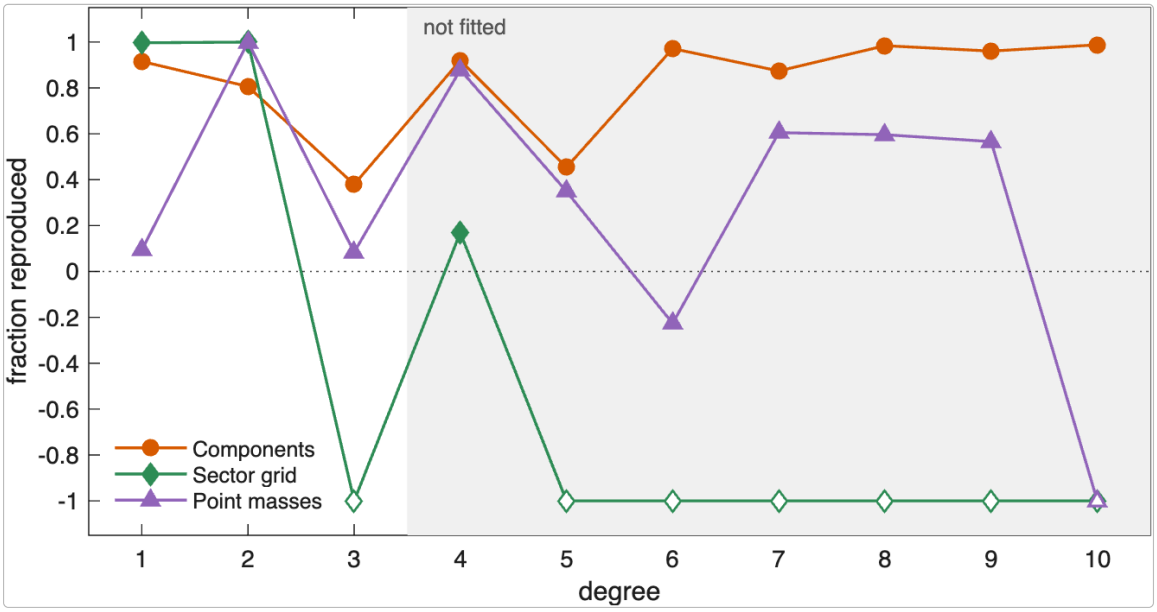
Solution	Mass moved (% of M)	Excess centered at (m)	Deficit centered at (m)
Components	1.1	(27, 0, 7)	(−3, 0, −1)
Sector grid	0.9	(11, 2, 0)	(−21, 0, 0)
Point masses	0.8	(1, 0, 0)	(−1, 0, 0)

Correlation	Components	Sector grid	Point masses
Components	1.00	0.46	0.41
Sector grid	0.46	1.00	0.47
Point masses	0.41	0.47	1.00

Mass moved: the mass above the bulk density, as a percentage of the mass of the body. Excess and deficit centers are from the center of figure. Correlation: of the relative densities of the solutions, 8.3 m cell by 8.3 m cell.

The point masses find the core's size and elongation but not its place. They gather a dense column about 50 m across along x at the center, nearly the extent of the sphere along x, symmetric about the center instead of on the +x side. The excess of the sphere is centered 27 m toward +x, that of the point masses 1 m. Degree 2 is the same for mass at +x and at −x. Only the odd degrees tell them apart: degree 3 is fitted but its 1σ is far larger than its anomaly, and degree 1, at 0.7σ, is too weak for the point-mass regularization to follow. The sector grid puts the excess in its inner shell, 11 m toward +x.

6. Against ROB's exact values and the GRASS point



The fraction of each degree of ROB's exact C − C_CD that each solution reproduces, 1 − |solution − exact|²/|exact|². 1: all of it; 0: no better than constant density; values below −1 are drawn at −1 as open markers. The gray region was not fitted.

The exact values settle it. The sphere reproduces 91% of degree 1, the center-of-mass offset the measurements barely saw; the point masses 10%. Over degrees 4 to 10, which no solution was fitted to, the sphere reproduces 91% and the point masses 79%, and the sector grid does worse than constant density (−0.70). From degree 6 on the sphere reproduces 87 to 99% of each degree, while the point masses fail at degrees 6 and 10. A compact sphere on the +x side is what the hidden interior has.

Solution	Acceleration minus ROB's (% of ROB's)	Angle from ROB's
Components	+0.2%	0.1°
Sector grid	+0.2%	0.1°
Point masses, sensitivity weighting	−0.6%	0.1°
Point masses, equal weighting	−0.4%	0.0°
Constant density	−0.6%	0.1°

The acceleration each solution predicts at the GRASS surface point minus ROB's, in magnitude (% of ROB's magnitude) and direction, on the full-resolution shape model.

The GRASS point lies at the +x end of the body, above the sphere, but the anomaly is too small there to decide. Constant density is off by only 0.6%. The sphere and the sector grid predict the acceleration to 0.2%, the point masses to 0.6%, all within 0.2° in direction.

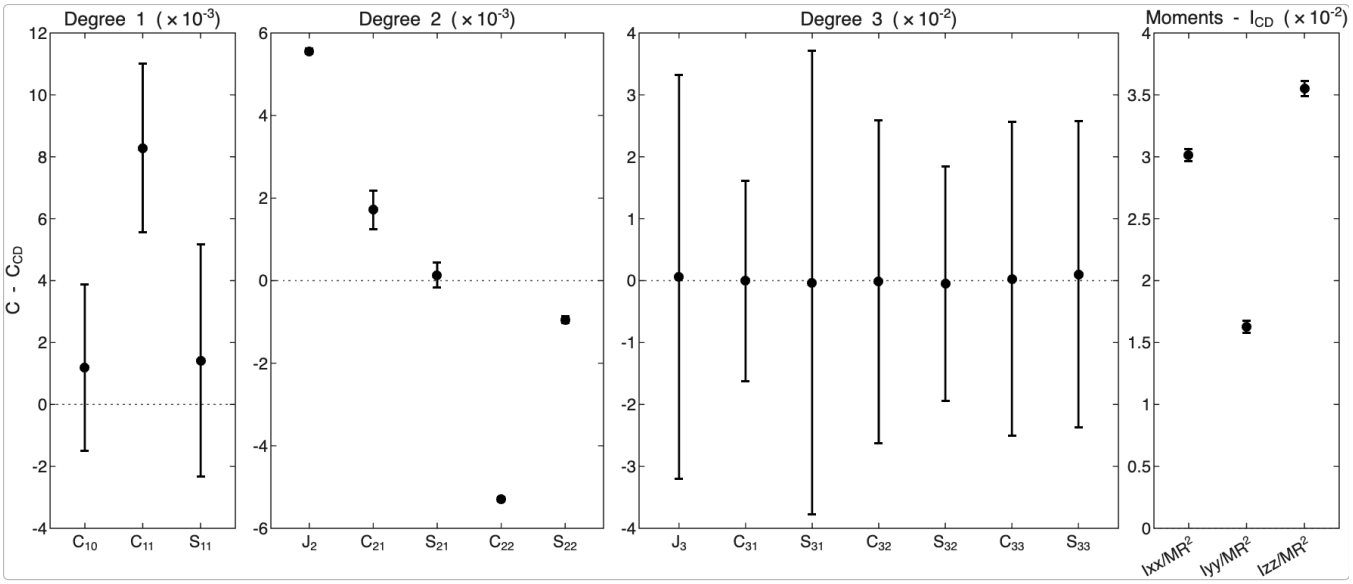
ROB-DATA6: Dimorphos

In ROB-DATA6 all three moments of inertia are 32 to 62σ larger than at constant density: mass lies farther from the center than in a uniform body, so the inside is light and the outside dense. Degree 2 adds that mass has moved toward the ends of the y-axis. A model with a light core toward –x, a dense equatorial torus and a dense crust fits with 6 parameters. The point masses tie with it in AIC. They also make a light inside and a dense outside, but put the light region at the center, elongated along x, instead of to one side. Here ROB's exact values and the GRASS point side with the point masses. Over degrees 4 to 10, which no solution was fitted to, they reproduce 46% where the components do worse than constant density (−0.72), and they predict the GRASS acceleration to 0.7% against 4.1%.

Kind	Model	Parameters	Reduced χ²	p	AIC	Exact values, degrees 1–3	Exact values, degrees 4–10
Point masses	**2674 points**	7.8	0.11	1.00	16.6	0.92	0.46
Components	core, torus, layer	6.0	0.39	0.97	16.7	0.69	−0.72
Sector grid	2 × 4 × 8	9.2	0.04	1.00	18.7	0.96	0.50

The best solution of each kind, by AIC (bold: the lowest). Parameters: the effective number of parameters. Exact values: the fraction of ROB's exact coefficients a solution reproduces, over the fitted degrees 1–3 and over degrees 4–10, which were not (1: all; 0: no better than constant density; negative: worse). See How to read these pages.

1. What the measurements show



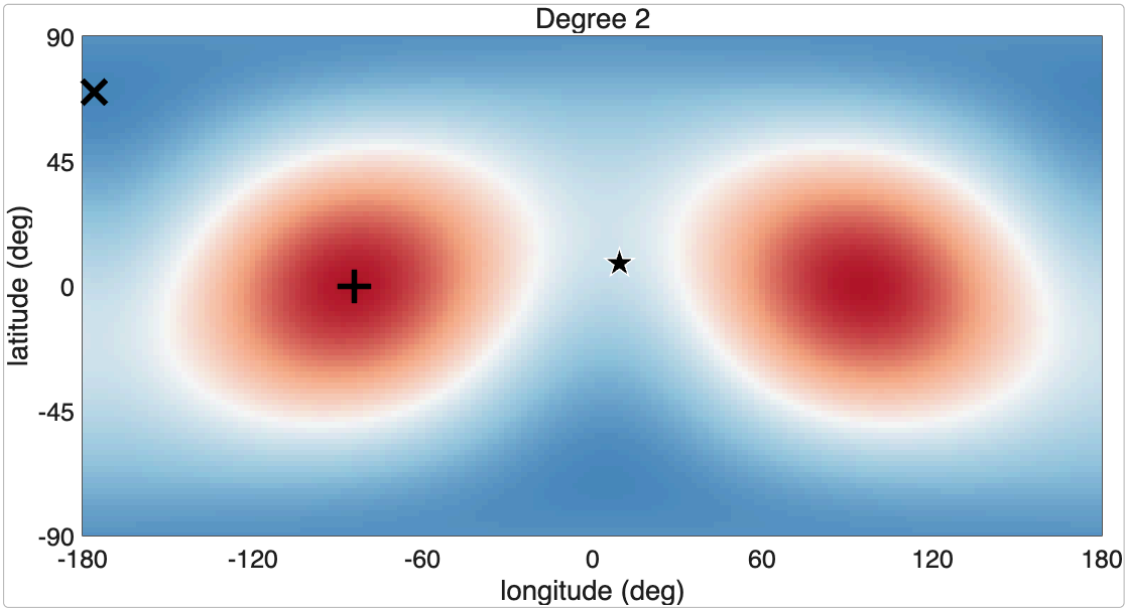
The measured coefficients minus those of the same shape at constant density, C – C_CD, with ±1σ of the Hera radio-science solution, degrees 1 to 3, and the three moments of inertia minus those at constant density. Each panel has its own scale; 0 is constant density. Degree 3 is fitted, but its 1σ is about 50 times its anomaly.

Measurement	C – C_CD	1σ	In σ	What it says
$\bar{C}_{11}$	8.29×10^{-3}	2.74×10^{-3}	+3	center of mass toward +x
$\bar{C}_{20}$	-5.56×10^{-3}	7.07×10^{-5}	−79	J_2 higher: mass toward the equator
$\bar{C}_{21}$	1.72×10^{-3}	4.67×10^{-4}	+4	mass toward +x in the north, −x in the south
$\bar{C}_{22}$	-5.29×10^{-3}	5.43×10^{-5}	−97	mass toward ±y more than ±x
$\bar{S}_{22}$	-9.44×10^{-4}	8.77×10^{-5}	−11	mass toward longitudes −45° and 135°
$I_{xx}\,/\,MR^2$	3.01×10^{-2}	4.88×10^{-4}	+62	mass farther from the x-axis
$I_{yy}\,/\,MR^2$	1.63×10^{-2}	5.04×10^{-4}	+32	mass farther from the y-axis
$I_{zz}\,/\,MR^2$	3.55×10^{-2}	6.31×10^{-4}	+56	mass farther from the z-axis

The measurements more than 3σ from constant density.

The moments of inertia are the strongest signal: I_{xx} +62σ, I_{yy} +32σ, I_{zz} +56σ. Each moment is the sum of mass times squared distance from an axis, so all three larger means mass farther from the center: a light interior under a dense exterior. Degree 2 cannot see this, since it holds only differences of the moments. It shows that the move is uneven: $\bar{C}_{22}$ 97σ lower (mass toward the ends of the y-axis) and J_2 79σ higher (toward the equator). The center of mass is 1.3 m toward +x, 3σ.

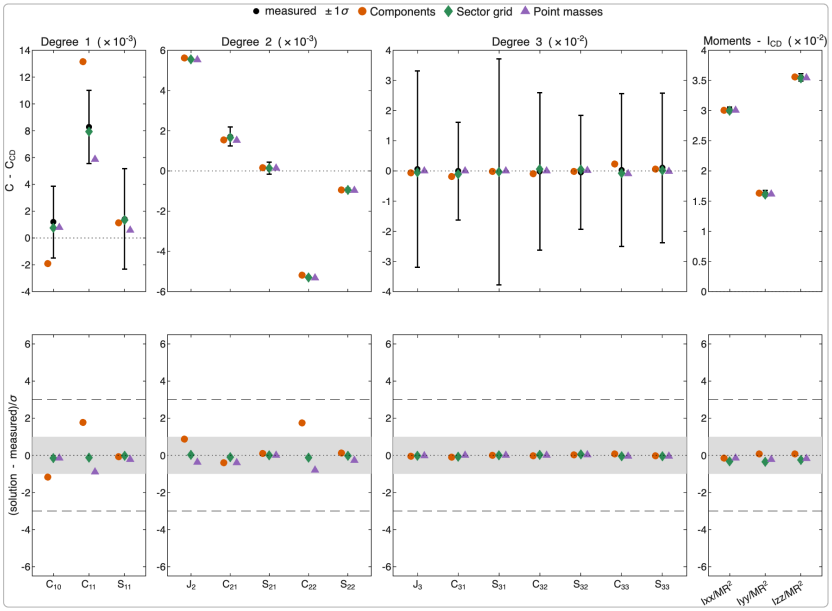
2. Where the mass is



Degree 2 of C − C_CD as a map of latitude and longitude from the center of figure. Red where a mass added near the surface in that direction would produce the pattern, blue where a mass taken away would. +: strongest excess; x: strongest deficit; star: direction of the center-of-mass offset.

Degree 2 puts the excess at the ends of the y-axis and the deficit near the poles and along x. The model built for this package uses what each measurement asks for. A light core is placed on the x-axis by its χ^2 profile: it takes mass from along x, making $\bar{C}_{22}$ lower, and moves the center of mass to +x. A dense torus in the equatorial plane makes J_2 higher. A dense surface layer puts mass far from the center, raising all three moments.

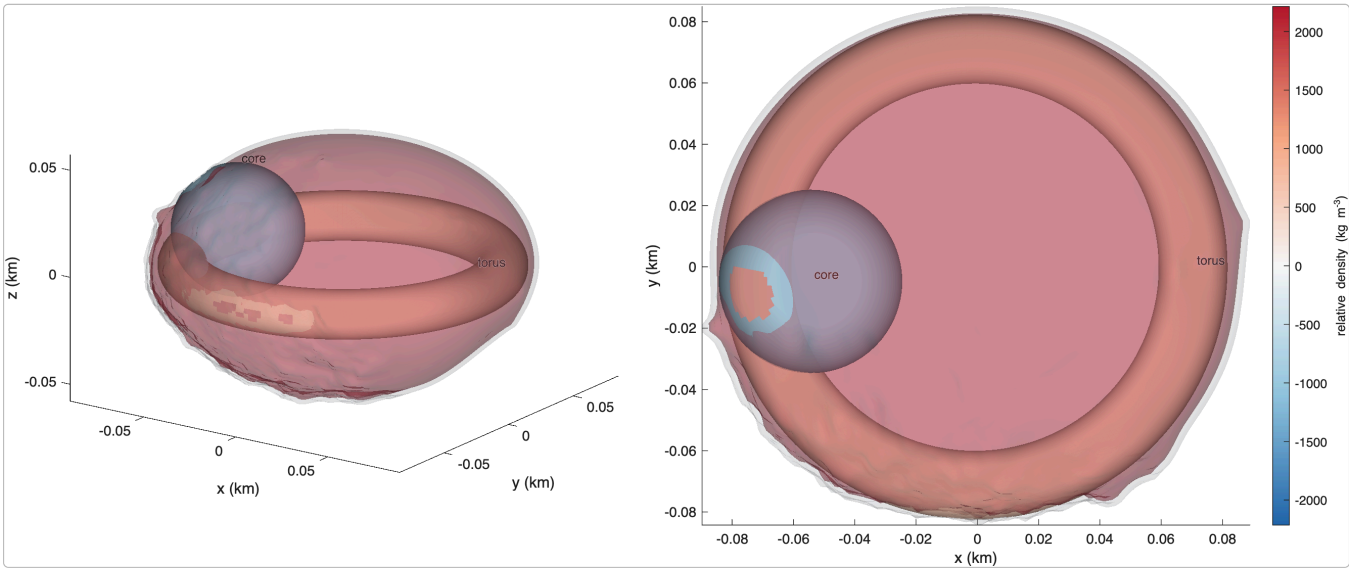
3. How well each solution fits



Top: C − C_CD measured (black, ±1σ) and of the best solution of each kind. Bottom: (solution − measured)/σ; the gray band is ±1σ, the dashed lines ±3σ.

The point masses (7.8 effective parameters) and the model with components (6) are 0.1 apart in AIC: a tie. The point masses match every measurement within 1σ; the model with components misses $\bar{C}_{11}$ and $\bar{C}_{22}$ by about 1.8σ, its center of mass too far toward +x. The sector grid fits as closely with 9.2 effective parameters and ranks 2 behind.

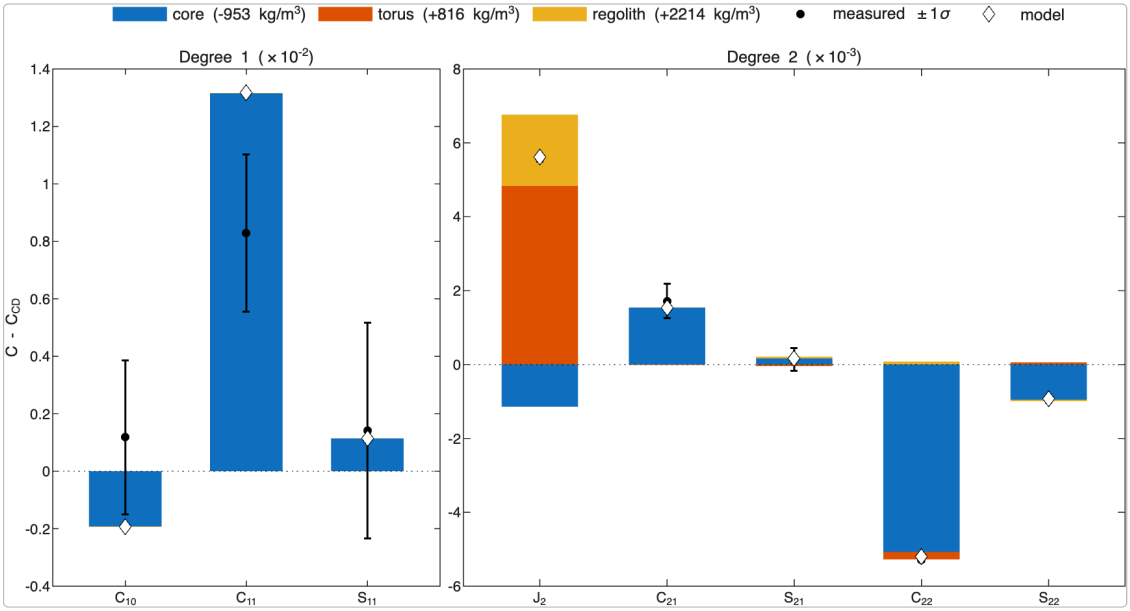
4. The model with components



The model with components, seen obliquely (left) and down the spin axis (right), colored by each component's density relative to the background density.

Component	Shape and place	Relative density (kg/m³)	Density (kg/m³)	Mass anomaly (kg); makes most of
core	sphere, radius 30 m, 54 m from the center toward lat 8°, lon −175°	−953 ± 371	430	-1.07×10^{8}; $\bar{C}_{22}$ −93σ, $\bar{C}_{20}$ +16σ
torus	torus, ring radius 71 m, tube radius 11 m, axis 0° from z, center 0 m from the center	+816 ± 66	2199	1.45×10^{8}; $\bar{C}_{20}$ −68σ, $\bar{C}_{22}$ −4σ
regolith	surface layer, 0 to 2 m deep	+2214 ± 345	3597	3.35×10^{8}; $\bar{C}_{20}$ −27σ, $\bar{C}_{22}$ +2σ

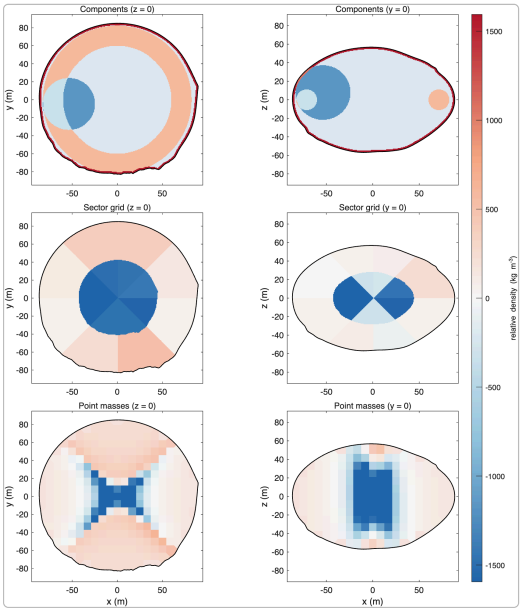
Background density 1383 kg/m³ (bulk density 1594 kg/m³). Makes most of: the component's part of each coefficient in units of its 1σ.



The part of each measured coefficient each component makes (stacked bars), the measured value with ±1σ (black) and the model's sum (white diamond).

- **The light core** (−953 ± 371 kg/m³, 30 m radius, 54 m toward −x) makes nearly all of $\bar{C}_{22}$.
- **The dense torus** (+816 ± 66 kg/m³, ring 71 m, tube 11 m) makes most of J_2.
- **The dense crust** (+2214 ± 345 kg/m³, 2 m thick) and the background 211 kg/m³ below the bulk density move mass outward and raise the moments.

5. Point masses and components side by side



The best solution of each kind on the planes z = 0 (left) and y = 0 (right) through the center of figure, colored by relative density on one scale. Components and sectors are drawn with their own boundaries; the point masses are drawn as the mean relative density of 8.3 m cells.

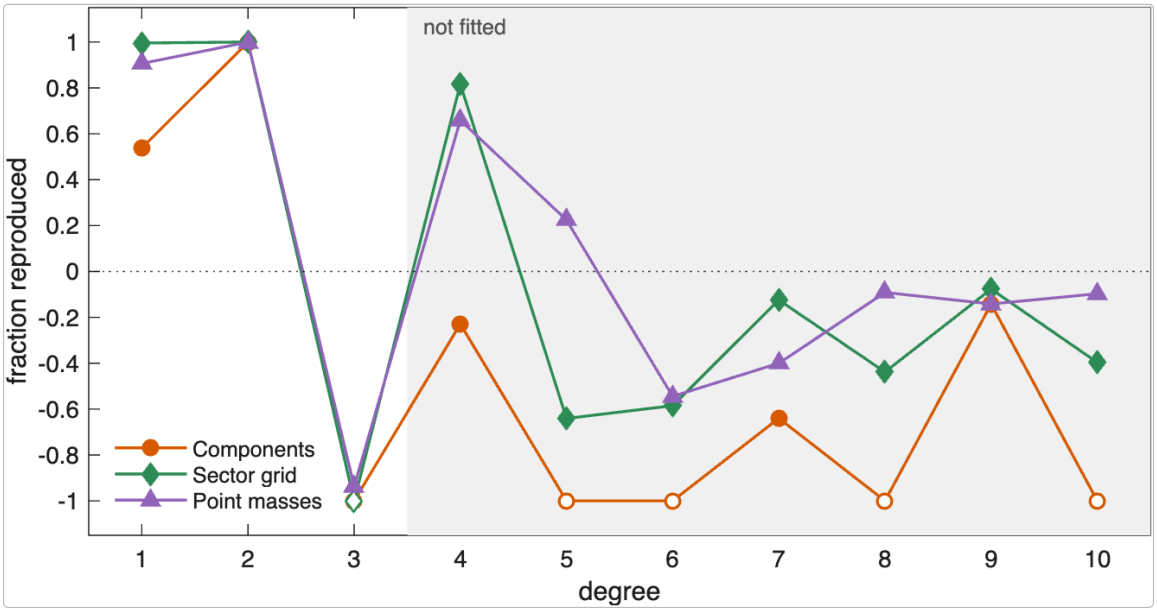
Solution	Mass moved (% of M)	Excess centered at (m)	Deficit centered at (m)
Components	5.9	(6, −1, −1)	(−14, −1, 2)
Sector grid	9.1	(12, 2, 0)	(0, 0, −1)
Point masses	11.3	(4, 0, 1)	(−4, 0, 0)

Correlation	Components	Sector grid	Point masses
Components	1.00	0.25	0.12
Sector grid	0.25	1.00	0.48
Point masses	0.12	0.48	1.00

Mass moved: the mass above the bulk density, as a percentage of the mass of the body. Excess and deficit centers are from the center of figure. Correlation: of the relative densities of the solutions, 8.3 m cell by 8.3 m cell.

The point masses and the model with components agree on the structure and disagree on where the light region is. Both take mass from along the x-axis inside the body and put it farther out. The model with components does it with one light core on the −x side. The point masses use a light region about 60 m long along x, centered on the center of figure, under an exterior denser than the bulk everywhere. Their relative densities correlate only 0.12, the lowest between the two in any package. The sector grid agrees with the point masses: a light inner shell, a denser outer one (correlation 0.48). The point masses move the most mass of any solution in the six packages, 11% of the mass of the body.

6. Against ROB's exact values and the GRASS point



The fraction of each degree of ROB's exact C − C_CD that each solution reproduces, 1 − |solution − exact|²/|exact|². 1: all of it; 0: no better than constant density; values below −1 are drawn at −1 as open markers. The gray region was not fitted.

In the fitted degrees the sector grid and the point masses reproduce 96% and 92%, the model with components 69%. The difference is degree 1: a light core 54 m to one side moves the center of mass more than the hidden interior does (54% of degree 1 against 91% for the point masses). No solution reproduces degree 3 (each about −1): it is fitted, but its 1σ is far larger than its anomaly. Over degrees 4 to 10 the sector grid and the point masses reproduce 50% and 46%, and the model with components does worse than constant density (−0.72). At degree 4 the sector grid and the point masses reproduce 82% and 67% and the model with components does not.

Solution	Acceleration minus ROB's (% of ROB's)	Angle from ROB's
Components	+4.1%	3.0°
Sector grid	+1.6%	2.3°
Point masses, sensitivity weighting	−0.7%	2.5°
Point masses, equal weighting	+1.6%	2.5°
Constant density	−1.8%	2.6°

The acceleration each solution predicts at the GRASS surface point minus ROB's, in magnitude (% of ROB's magnitude) and direction, on the full-resolution shape model.

The GRASS point lies at the +x end of the body. Constant density predicts the acceleration 1.8% low. The point masses predict it to 0.7%, the sector grid to 1.6%. The model with components is 4.1% high: its dense torus and crust put too much mass near the +x surface. The exact values and the GRASS point both favor a light interior near the center over one to the side.

API reference

The supported interface of SBTK: every function in the namespace packages below, generated from the help text and `arguments` block of its source file. Functions are called by their full name, `sbtk.<package>.<function>`. Arguments have no default values except where the Default column states one (three functions). Structures passed between functions are described in Data structures; error identifiers in Error identifiers. Helper functions in `sbtk.internal` are not part of the supported interface and are listed separately in Internal functions.

Package	Contents	Functions
`sbtk`	Version and repository folder	2
`sbtk.units`	Constants and unit conversions	6
`sbtk.shape`	Shape models	8
`sbtk.body`	Bodies	4
`sbtk.interior`	Density distributions	17
`sbtk.gravity`	Gravity and moment calculations	8
`sbtk.data`	Measurement sets	21
`sbtk.interpret`	Reading the gravity coefficients	7
`sbtk.invert`	Density estimation	12
`sbtk.search`	Searching the geometry	7
`sbtk.compare`	Comparison with a true model	6
`sbtk.exchange`	Exchange packages	10
`sbtk.io`	Results folders and text files	22
`sbtk.plot`	Figures	20

All supported functions

Function	Purpose
`sbtk.body.create`	A small body: shape model, bulk density and reference radius.
`sbtk.body.from_mass`	A small body defined by its mass instead of its bulk density.
`sbtk.body.summary`	Print a body's main numbers.
`sbtk.body.to_frame`	A body with its shape model in another frame: x' = (x - `origin_km`) * axes.
`sbtk.compare.evaluate`	Compare a fit with the true model, using a prepared reference (see sbtk.compare.reference).
`sbtk.compare.grid_summary`	Print a cell-by-cell comparison of two density grids.
`sbtk.compare.grids`	Compare two density grids cell by cell (e.g. a solution package with an interior-model package).
`sbtk.compare.reference`	The true model sampled once for comparisons with fits that use a given set of components.
`sbtk.compare.summary`	Print a comparison with the true model.
`sbtk.compare.with_truth`	Compare an estimated density with the true density (synthetic tests).
`sbtk.data.add_center_of_mass`	Add degree-1 coefficients that place the center of mass at a given point.
`sbtk.data.add_libration`	Add a measured libration ratio (B - A)/C.
`sbtk.data.add_moi`	Add one measured inertia element to a measurement set.
`sbtk.data.add_moi_row`	Add a measurement of any weighted sum of the inertia elements.
`sbtk.data.add_noise`	One noisy realization of a measurement set.
`sbtk.data.expand_about`	Measurements with the gravity field and the inertia about another point.
`sbtk.data.from_coefficients`	Measurement set from gravity-coefficient matrices (e.g. a mission solution).
`sbtk.data.libration_ratio_from_amplitude`	(B - A)/C from the forced libration of a synchronous satellite.
`sbtk.data.move_origin`	Express measurements about the center of mass relative to a new expansion origin.
`sbtk.data.select_coefficients`	Keep only the listed gravity coefficients.
`sbtk.data.select_degree`	Keep only the coefficients up to a degree.
`sbtk.data.select_moi`	Keep only the listed moment-of-inertia measurements.
`sbtk.data.set_covariance`	Replace the coefficient covariance with a full matrix.
`sbtk.data.set_moi_sigma`	Set the 1-sigma of moment-of-inertia measurements by label.
`sbtk.data.set_uncertainties`	Give measurements a 1-sigma uncertainty per degree and per inertia quantity.
`sbtk.data.simulate`	True model of an interior and its exact gravity coefficients and moments of inertia.
`sbtk.data.summary`	Print what is in a measurement set.
`sbtk.data.to_frame`	A measurement set in another frame: x' = (x - `origin_km`) * axes.

Function	Purpose
sbtk.data.to_reduced_shape	Measurements for a coarser version of the same shape model, for faster fits.
sbtk.data.uncertainties	1-sigma measurement uncertainties by degree and by inertia quantity, for sbtk.data.set_uncertainties.
sbtk.data.use_uncertainties	Give measurements the coefficient uncertainties of another data set (e.g. a precision file).
sbtk.exchange.density_grid	Density of a model at the centers of a cubic grid inside the body.
sbtk.exchange.locate	The exchange package of a given kind in a run folder.
sbtk.exchange.read_data	Measurements and body from a measurement package (sbtk.exchange.write_data format).
sbtk.exchange.read_grid	Density grid from an interior-model or solution package (`density_grid.txt`).
sbtk.exchange.read_shadr	Measurements and body from a gravity file in the PDS SHADR layout (e.g. other teams' .tab files).
sbtk.exchange.solution_name	Name of a solution package: <team>-SOL_<data team><number>.
sbtk.exchange.write_data	Write a measurement package to send to other teams (nothing about the interior).
sbtk.exchange.write_model	Write the interior-model package behind a measurement package (released after the solutions).
sbtk.exchange.write_potential	Add the gravitational potential and acceleration at given points to a package.
sbtk.exchange.write_solution	Write an inversion result as a solution package for the other teams.
sbtk.gravity.center_of_mass_frame	A gravity field about its center of mass, in the shape-model, spin or principal axes.
sbtk.gravity.component_integrals	Integrals over one component per unit density, clipped to the body.
sbtk.gravity.field_at	Gravitational potential and acceleration of a model of the body at given points.
sbtk.gravity.forward	Forward model: gravity coefficients, center of mass and inertia of an interior.
sbtk.gravity.geometry	Facet-cone geometry of a body about an expansion origin (the integration workspace).
sbtk.gravity.labels	Names of the gravity coefficients up to a degree, in SBTK order.
sbtk.gravity.polyhedron	Potential and acceleration of a homogeneous polyhedron, exact (Werner & Scheeres 1997).
sbtk.gravity.random_field_integrals	Density-weighted integrals of the random density field.
sbtk.interior.align_with_spin_axis	Set the axis of components along the axis of the largest moment of the whole model.
sbtk.interior.create	An interior model: components plus an optional random density field.
sbtk.interior.density_at	Density [kg/m^3] of a model at points inside the body.
sbtk.interior.ellipsoid	Uniform triaxial ellipsoid component, rotated, clipped to the body.
sbtk.interior.latlon	Position [x y z] (km) from latitude, longitude (deg) and distance (km).
sbtk.interior.layer	Uniform layer component between two depths below the surface.
sbtk.interior.no_components	An empty list of components (a homogeneous body).
sbtk.interior.no_random_field	No random density field.
sbtk.interior.random_field	Gaussian random density field added to the interior.
sbtk.interior.sector	Uniform sector component: radial fractions x latitude/longitude range.
sbtk.interior.sector_grid	Sectors that tile the whole body (a model without assumed structure).
sbtk.interior.set_axis	Turn a component so that its axis points along a given direction.
sbtk.interior.sphere	Uniform sphere component (e.g. a core, a boulder or a void).
sbtk.interior.sphere_grid	Small spheres on a cubic grid filling the body (mass concentrations, a model without assumed structure).
sbtk.interior.summary	Print a list of components.
sbtk.interior.to_frame	An interior or a true model in another frame: x' = (x - `origin_km`) * axes.
sbtk.interior.torus	Uniform torus component about the z-axis, clipped to the body.
sbtk.invert.choose_regularization	Tikhonov regularization parameter by the discrepancy principle or the lowest AIC.
sbtk.invert.goodness_of_fit	Reduced chi-square and chi-square test of a fit.
sbtk.invert.homogeneous	Fit a homogeneous body (no components) to the same measurements.
sbtk.invert.options	Inversion settings with their standard values.
sbtk.invert.predict	A fitted model's values of every measurement of a set, including those it was not fitted to.
sbtk.invert.recenter	Move one component so that the fitted model's center of mass matches the measured one.
sbtk.invert.replace_data	Use another realization of the same measurements without repeating the setup.
sbtk.invert.run	Estimate the densities for a prepared inversion (bounded, mass-constrained least squares).
sbtk.invert.scan_position	Fit the densities with one component placed at each of a list of centers: chi-square as a function of its position.
sbtk.invert.setup	Prepare an inversion: geometry and the design-matrix column of every component, computed once.
sbtk.invert.solve	Setup and run in one call.
sbtk.invert.summary	Print an inversion result.
sbtk.io.finish_run	Stop the run's log and say where the results are.
sbtk.io.latest_run	Folder of the most recent run with a given name.

Function	Purpose
sbtk.io.load_fit	Inversion result from a run folder (`fit.mat`) or a .mat file holding the variable fit.
sbtk.io.load_measurements	Measurements from a run folder, a .mat file or a text file.
sbtk.io.load_truth	True model from a run folder or a .mat file.
sbtk.io.new_run	Create a results folder for one run and start its log.
sbtk.io.read_jpl_gravity	Measurements from a JPL SSD gravity-field file (normalized coefficients + covariance).
sbtk.io.read_measurements	Measurement set from the SBTK text format.
sbtk.io.report_comparison	Save, print and plot a comparison with the true model.
sbtk.io.report_inversion	Save, print and plot an inversion result.
sbtk.io.report_simulation	Save, print and plot synthetic measurements and their true model.
sbtk.io.save_comparison	comparison.mat and comparison.txt in the run folder.
sbtk.io.save_figure	Save a figure as figures/<name>.png (200 dpi) in the run folder.
sbtk.io.save_fit	`fit.mat` and `fit.txt` in the run folder.
sbtk.io.save_measurements	measurements.mat and measurements.txt in the run folder.
sbtk.io.save_settings	Write the settings of a run to `settings.json` (readable, reproducible).
sbtk.io.save_truth	`truth.mat` and `truth.txt` in the run folder.
sbtk.io.subfolder	A subfolder of a run folder that the save and report functions write to.
sbtk.io.write_comparison	Plain-text copy of a comparison with the true model.
sbtk.io.write_fit	Plain-text summary of an inversion result.
sbtk.io.write_measurements	Plain-text copy of a measurement set (read back by sbtk.io.read_measurements).
sbtk.io.write_truth	Plain-text description of a true model (keep private in blind tests).
sbtk.plot.anomaly	The measured anomaly coefficient by coefficient, with the anomalies of models and how well they match it.
sbtk.plot.anomaly_map	Each degree of the anomaly as a map: where mass is added and where it is taken away.
sbtk.plot.coefficient_comparison	The measured anomaly C - `C_CD` coefficient by coefficient with +-1 sigma, the solutions' anomalies, and their residuals.
sbtk.plot.coefficient_residuals	Map of normalized residuals: one row per fit, one column per measurement, fitted ones outlined.
sbtk.plot.comparison	Estimate compared with the true model: R2 and each component's density.
sbtk.plot.contributions	How much of each coefficient's anomaly each component of a model makes.
sbtk.plot.degree_ratio	Each degree's change from the homogeneous body: scaling (radial structure) and the rest.
sbtk.plot.densities	Estimated density of the background and each component with its 1-sigma uncertainty.
sbtk.plot.difference_from_homogeneous	Gravity coefficients minus those of the homogeneous body, coefficient by coefficient.
sbtk.plot.interior3d	See-through 3D view of the body and every component of a model.
sbtk.plot.measurements	What the data contain: gravity signal and noise per degree, and the inertia rows.
sbtk.plot.point_masses	The mass anomalies of a point-mass solution on two cuts through the center of figure.
sbtk.plot.residuals	Normalized residuals of a fit, (predicted - measured)/sigma, per measurement.
sbtk.plot.search_masses	Where the measurements ask for compact masses, and the chi-square as masses are added.
sbtk.plot.search_profiles	The chi-square profile of each searched geometry parameter.
sbtk.plot.shape	The body's shape model with its center of figure and axes.
sbtk.plot.significance	Each measurement with its +-1 sigma interval against the constant-density (homogeneous) value.
sbtk.plot.slices	Absolute density on the equatorial plane and a meridional plane.
sbtk.plot.solution_cuts	Several solutions side by side on the same cuts and color scale, and their density in the same cells.
sbtk.plot.truth_vs_estimate	Truth, estimate and their difference on two planes.
sbtk.root	Folder of the SBTK repository (the one containing toolbox/, examples/, data/).
sbtk.shape.bundled	A shape model that ships with SBTK, by short name.
sbtk.shape.ellipsoid	Triaxial ellipsoid shape model (an icosphere stretched to the semi-axes).
sbtk.shape.from_arrays	Shape model from face and vertex arrays.
sbtk.shape.load	Read a polyhedral shape model (Wavefront OBJ) and convert it to km.
sbtk.shape.properties	Volume, centroid and closure checks of a shape model.
sbtk.shape.reduce	A shape model with fewer facets, checked to stay closed, outward and star-shaped.
sbtk.shape.reduce_spc	Every step-th grid point of an SPC shape model: the same surface, sampled step times coarser.
sbtk.shape.write_obj	Write a shape model to a Wavefront OBJ file (vertices in km).
sbtk.units.G	Gravitational constant in SBTK units [km^3 kg^-1 s^-2].
sbtk.units.length_scale	Factor that converts a length in UNIT to km.
sbtk.units.moi_to_norm	Moment of inertia in kg m^2 -> I / (M `R_ref`^2) (dimensionless).

Function	Purpose
`sbtk.units.norm_to_moi`	I / (M `R_ref`^2) (dimensionless) -> moment of inertia in kg m^2.
`sbtk.units.offset_to_degree1`	Fully normalized degree-1 coefficient of a center-of-mass offset.
`sbtk.units.table`	Print the SBTK unit convention.
`sbtk.version`	SBTK version string (semantic versioning).

Reference

`sbtk`: Version and repository folder

Function	Purpose
`root`	Folder of the SBTK repository (the one containing toolbox/, examples/, data/).
`version`	SBTK version string (semantic versioning).

`sbtk.root`

Folder of the SBTK repository (the one containing toolbox/, examples/, data/).

Syntax

```
MATLAB
r = sbtk.root()
```

Outputs

Name	Description
`r`	Absolute path of the SBTK repository folder (the folder that contains toolbox/).

Related functions

Called by

`sbtk.exchange.read_data`, `sbtk.io.latest_run`, `sbtk.io.new_run`, `sbtk.shape.bundled`

Implementation: `toolbox/+sbtk/root.m`, line 1.

`sbtk.version`

SBTK version string (semantic versioning).

Syntax

```
MATLAB
v = sbtk.version()      % e.g. '1.0.0'
```

Outputs

Name	Description
`v`	Version text, e.g. '1.0.0' (semantic versioning).

Related functions

Called by

`sbtk.io.save_settings`, `sbtk.io.write_truth`

Implementation: `toolbox/+sbtk/version.m`, line 1.

Reference

`sbtk.units`: Constants and unit conversions

Function	Purpose
`G`	Gravitational constant in SBTK units [km^3 kg^-1 s^-2].
`length_scale`	Factor that converts a length in UNIT to km.
`moi_to_norm`	Moment of inertia in kg m^2 -> I / (M `R_ref`^2) (dimensionless).
`norm_to_moi`	I / (M `R_ref`^2) (dimensionless) -> moment of inertia in kg m^2.
`offset_to_degree1`	Fully normalized degree-1 coefficient of a center-of-mass offset.
`table`	Print the SBTK unit convention.

`sbtk.units.G`

Gravitational constant in SBTK units [km^3 kg^-1 s^-2].

Syntax

MATLAB

```
g = sbtk.units.G()      % 6.67430e-20 km^3/(kg s^2)  (CODATA 2018)
```

Outputs

Name	Description
`g`	6.67430e-20 km³ kg⁻¹ s⁻² (CODATA 2018).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
GM_km3_s2 = sbtk.units.G()*body.mass_kg;
```

Related functions

Called by

`sbtk.body.create`, `sbtk.data.from_coefficients`, `sbtk.data.simulate`, `sbtk.exchange.read_data`, `sbtk.exchange.read_shadr`, `sbtk.gravity.field_at`, `sbtk.gravity.polyhedron`, `sbtk.io.read_jpl_gravity`

Implementation: `toolbox/+sbtk/+units/G.m`, line 1.

`sbtk.units.length_scale`

Factor that converts a length in UNIT to km.

Syntax

MATLAB

```
f = sbtk.units.length_scale(unit)      % unit: "km", "m" or "cm"
```

Arguments

Name	Size	Class	Unit	Default	Description
`unit`	1 × 1	string		required	Length unit: "km", "m" or "cm". Validation: `mustBeMember(unit, ["km", "m", "cm"])`.

Outputs

Name	Description
`f`	Factor that converts a length in that unit to km (1, 1e-3 or 1e-5).

Example

sbtk.units.length_scale("m") returns 1e-3.

Related functions

Called by

`sbtk.shape.from_arrays`, `sbtk.shape.load`

Implementation: `toolbox/+sbtk/+units/length_scale.m`, line 1.

`sbtk.units.moi_to_norm`

Moment of inertia in kg m^2 -> I / (M `R_ref`^2) (dimensionless).

Syntax

MATLAB

```
I_norm = sbtk.units.moi_to_norm(I_kgm2, mass_kg, R_ref_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`I_kgm2`		double	kg m²	required	Moment of inertia, or an array of them. Validation: `mustBeFinite`.
`mass_kg`	scalar	double	kg	required	Total mass M. Validation: `mustBePositive`.
`R_ref_km`	scalar	double	km	required	Reference radius R_{ref}. Validation: `mustBePositive`.

Outputs

Name	Description
`I_norm`	$I / (MR_{\mathrm{ref}}^2)$, same size as the input.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
Izz = sbtk.units.moi_to_norm(1.6e14, body.mass_kg, body.R_ref_km);    % kg m^2 -> I/(M R_ref^2)
```

Implementation: `toolbox/+sbtk/+units/moi_to_norm.m`, line 1.

`sbtk.units.norm_to_moi`

I / (M `R_ref`^2) (dimensionless) -> moment of inertia in kg m^2.

Syntax

MATLAB

```
I_kgm2 = sbtk.units.norm_to_moi(I_norm, mass_kg, R_ref_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`I_norm`		double	M R_ref²	required	$I / (MR_{\mathrm{ref}}^2)$, or an array of such values. Validation: `mustBeFinite`.
`mass_kg`	scalar	double	kg	required	Total mass M. Validation: `mustBePositive`.
`R_ref_km`	scalar	double	km	required	Reference radius R_{ref}. Validation: `mustBePositive`.

Outputs

Name	Description
`I_kgm2`	Moment of inertia in kg m², same size as the input.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
I_kgm2 = sbtk.units.norm_to_moi(obs.moi_tensor_norm, obs.mass_kg, obs.R_ref_km);
```

Related functions

Called by

`sbtk.io.write_measurements`

Implementation: `toolbox/+sbtk/+units/norm_to_moi.m`, line 1.

`sbtk.units.offset_to_degree1`

Fully normalized degree-1 coefficient of a center-of-mass offset.

Syntax

MATLAB

```
c = sbtk.units.offset_to_degree1(offset_km, R_ref_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`offset_km`		double	km	required	offset of the center of mass from the expansion origin along one axis, or its 1-sigma uncertainty [km] Validation: `mustBeFinite`.
`R_ref_km`	scalar	double	km	required	reference radius of the gravity coefficients [km] Validation: `mustBePositive`.

Outputs

Name	Description
`c`	Fully normalized degree-1 coefficient, d / ($\sqrt{3}$ R_{ref}). Fields: `component`.

Description

c = `offset_km` / (sqrt(3) `R_ref_km`): C10 for z, C11 for x, S11 for y. Use it to give the degree-1 entry of sbtk.data.uncertainties as a center-of-mass uncertainty.

Example

MATLAB, from the help text

```
% 2 m center of mass, 1e-4, 2e-4, 4e-4 for degrees 2-4
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
```

Related functions

Mentioned in the help text

`sbtk.data.uncertainties`

Implementation: `toolbox/+sbtk/+units/offset_to_degree1.m`, line 1.

`sbtk.units.table`

Print the SBTK unit convention.

Syntax

MATLAB

```
sbtk.units.table()
```

Example

MATLAB, from the help text

```
sbtk.units.table()                      % prints the unit conventions
```

Implementation: `toolbox/+sbtk/+units/table.m`, line 1.

Reference

sbtk.shape: Shape models

Function	Purpose
bundled	A shape model that ships with SBTK, by short name.
ellipsoid	Triaxial ellipsoid shape model (an icosphere stretched to the semi-axes).
from_arrays	Shape model from face and vertex arrays.
load	Read a polyhedral shape model (Wavefront OBJ) and convert it to km.
properties	Volume, centroid and closure checks of a shape model.
reduce	A shape model with fewer facets, checked to stay closed, outward and star-shaped.
reduce_spc	Every step-th grid point of an SPC shape model: the same surface, sampled step times coarser.
write_obj	Write a shape model to a Wavefront OBJ file (vertices in km).

---

sbtk.shape.bundled

A shape model that ships with SBTK, by short name.

Syntax

MATLAB

```
shape = sbtk.shape.bundled(name)
```

Arguments

Name	Size	Class	Unit	Default	Description
name	1 × 1	string		required	file in data/shapes/ (all in km) Validation: mustBeMember(name, ["didymos", "dimorphos", "didymos_highres", "dimorphos_highres", "didymos_every8", "dimorphos_placeholder", "bennu", "example_asteroid"]).

Outputs

Name	Description
shape	The shape model, lengths in km. Fields: shape.

Description

"didymos"	didymos_g_9309mm_spc_obj_0000n00000_v003.obj: DART SPC model v003 at 9.3 m sampling (49,152 facets), the version the Hera radio-science team uses
"dimorphos"	g_00243mm_spc_obj_dimo_0000n00000_v004_every8.obj: DART SPC model v004 (pre-impact) at 1.94 m sampling (49,152 facets), every 8th point of the high-resolution grid (sbtk.shape.reduce_spc)
"didymos_highres"	g_01165mm_spc_obj_didy_0000n00000_v003.obj: 1.165 m, 3,145,728 facets; the reference model of the HLP6-G test
"dimorphos_highres"	g_00243mm_spc_obj_dimo_0000n00000_v004.obj: 0.243 m, 3,145,728 facets
"didymos_every8"	g_01165mm_spc_obj_didy_0000n00000_v003_every8.obj: every 8th point of "didymos_highres" (49,152 facets)
"dimorphos_placeholder"	a placeholder shape (not a mission model; kept for the tests)
"bennu"	OSIRIS-REx shape model (bennu_higher_qual)
"example_asteroid"	a synthetic 1.2 km body (projects/template)

The two high-resolution files (about 160 MB each) are not in git: copy them from the Hera server (End-to-End Test/HLP6G BlindTest/inputs/Reference models/) or ESA's SPICE server (spiftp.esac.esa.int/data/SPICE/HERA/kernels/dsk/) into data/shapes/. See data/shapes/README.md for every file. For any other shape model use sbtk.shape.load(file, unit).

Example

MATLAB, from the help text

```
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
big  = sbtk.shape.bundled("dimorphos_highres");              % 3 s to read
```

Related functions

Mentioned in the help text

sbtk.body.from_mass, sbtk.shape.load, sbtk.shape.reduce_spc

Calls

sbtk.root, sbtk.shape.load

Implementation: toolbox/+sbtk/+shape/bundled.m, line 1.

`sbtk.shape.ellipsoid`

Triaxial ellipsoid shape model (an icosphere stretched to the semi-axes).

Syntax

MATLAB

```
shape = sbtk.shape.ellipsoid(name, semi_axes_km, level)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the shape.
`semi_axes_km`	1 × 3	double	km	required	[a b c] along x, y, z [km] Validation: `mustBePositive`.
`level`	scalar	double		required	icosphere refinement: 20*4^level facets (5 -> 20,480) Validation: `mustBeInteger, mustBeNonnegative`.

Outputs

Name	Description
`shape`	Triaxial ellipsoid with 20·4^level facets, vertices on the ellipsoid. Fields: `shape`.

Description

Vertices lie exactly on the ellipsoid. Useful for tests and stand-ins.

Example

MATLAB, from the help text

```
shape = sbtk.shape.ellipsoid("test", [0.40 0.35 0.30], 4);          % 5,120 facets
```

Implementation: `toolbox/+sbtk/+shape/ellipsoid.m`, line 1.

`sbtk.shape.from_arrays`

Shape model from face and vertex arrays.

Syntax

MATLAB

```
shape = sbtk.shape.from_arrays(name, faces, vertices, unit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the shape.
`faces`	any × 3	double		required	nf x 3 vertex indices (1-based), triangles Validation: `mustBeInteger, mustBePositive`.
`vertices`	any × 3	double		required	nv x 3 positions in UNIT ("km", "m" or "cm") Validation: `mustBeFinite`.
`unit`	1 × 1	string		required	Length unit of the vertices: "km", "m" or "cm". Validation: `mustBeMember(unit, ["km", "m", "cm"])`.

Outputs

Name	Description
`shape`	The shape model, vertices converted to km. Fields: `shape`.

Description

Returns the same struct as sbtk.shape.load.

Example

MATLAB, from the help text

```
V = [0 0 0; 100 0 0; 0 100 0; 0 0 100] - 25;                        % vertices in meters
shape = sbtk.shape.from_arrays("tetra", [1 3 2; 1 2 4; 1 4 3; 2 3 4], V, "m");
```

Related functions

Mentioned in the help text

`sbtk.shape.load`

Calls

`sbtk.units.length_scale`

Called by

`sbtk.shape.reduce`, `sbtk.shape.reduce_spc`

Implementation: `toolbox/+sbtk/+shape/from_arrays.m`, line 1.

`sbtk.shape.load`

Read a polyhedral shape model (Wavefront OBJ) and convert it to km.

Syntax

MATLAB

```
shape = sbtk.shape.load(file, unit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`file`	1 × 1	string		required	path to an .obj file with 'v x y z' and 'f i j k' records (normals, textures and 'f i/t/n' forms are accepted) Validation: `mustBeFile`.
`unit`	1 × 1	string		required	length unit of the file: "km", "m" or "cm" (required: there is no safe way to guess it) Validation: `mustBeMember(unit, ["km", "m", "cm"])`.

Outputs

Name	Description
`shape`	struct Fields: `shape`.

Description

`.name`	file name without extension
`.faces`	nf x 3 vertex indices
`.vertices_km`	nv x 3 vertex positions [km]
`.source`	the file it came from
`.frame_origin_km, .frame_axes`	[0 0 0] and eye(3): the frame of the file (changed by
`sbtk.body.to_frame; vertices = (file vertices - frame_origin_km)*frame_axes)`	

Example

MATLAB, from the help text

```
shape = sbtk.shape.load(fullfile(sbtk.root, 'data', 'shapes', ...
            'didymos_g_9309mm_spc_obj_0000n00000_v003.obj'), "km");
```

Related functions

Mentioned in the help text

`sbtk.body.to_frame`, `sbtk.root`

Calls

`sbtk.units.length_scale`

Called by

`sbtk.exchange.read_data`, `sbtk.shape.bundled`

Implementation: `toolbox/+sbtk/+shape/load.m`, line 1.

`sbtk.shape.properties`

Volume, centroid and closure checks of a shape model.

Syntax

MATLAB

```
props = sbtk.shape.properties(shape)
```

Arguments

Name	Size	Class	Unit	Default	Description
`shape`	1 × 1	struct		required	The shape model to check. Fields: `shape`.

Outputs

Name	Description
`props`	Volume, centroid and checks. Fields: `props`.

Description

`props.volume_km3`	enclosed volume [km^3]
`props.centroid_km`	center of figure [km]

`props.n_facets`, .n_vertices

`props.extent_km`	[min; max] of the vertices per axis [km]
`props.closed`	true if every edge is shared by exactly two facets, in opposite directions (a closed, consistently ordered surface)
`props.outward`	true if the facets are ordered outward
`props.star_shaped_about_centroid`	true if every facet faces away from the
`centroid (required to integrate about it)`	

Example

MATLAB, from the help text

```
p = sbtk.shape.properties(sbtk.shape.bundled("didymos"));
fprintf('volume %.4f km^3, star-shaped: %d\n', p.volume_km3, p.star_shaped_about_centroid);
```

Related functions

Mentioned in the help text

`sbtk.shape.bundled`

Called by

`sbtk.body.create`, `sbtk.body.from_mass`, `sbtk.exchange.write_data`, `sbtk.exchange.write_model`, `sbtk.shape.reduce`, `sbtk.shape.reduce_spc`

Implementation: `toolbox/+sbtk/+shape/properties.m`, line 1.

`sbtk.shape.reduce`

A shape model with fewer facets, checked to stay closed, outward and star-shaped.

Syntax

MATLAB

```
small = sbtk.shape.reduce(shape, n_facets)
```

Arguments

Name	Size	Class	Unit	Default	Description
`shape`	1 × 1	struct		required	from sbtk.shape.load (e.g. a high-resolution SPC model with millions of facets) Fields: `shape`.
`n_facets`	scalar	double		required	target number of facets Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`small`	The reduced shape model. Fields: `shape`.

Description

Uses MATLAB's reducepatch. Gravity coefficients to degree 4 or so need far fewer facets than a high-resolution shape model has; check what the reduction changes by comparing sbtk.shape.properties (volume, centroid) and the homogeneous-body coefficients of both (tools/make_reduced_shapes.m does both). Errors if the reduced surface is not closed, not ordered outward, or not star-shaped about its centroid. Save it with sbtk.shape.write_obj.

Example

MATLAB, from the help text

```
big   = sbtk.shape.load("g_00243mm_spc_obj_dimo_0000n00000_v004.obj", "km");
small = sbtk.shape.reduce(big, 50000);
sbtk.shape.write_obj(small, "dimorphos_v004_50k.obj");
```

Related functions

Mentioned in the help text

sbtk.shape.load, sbtk.shape.properties, sbtk.shape.write_obj

Calls

sbtk.shape.from_arrays, sbtk.shape.properties

Implementation: `toolbox/+sbtk/+shape/reduce.m`, line 1.

---

sbtk.shape.reduce_spc

Every step-th grid point of an SPC shape model: the same surface, sampled step times coarser.

Syntax

MATLAB

```
small = sbtk.shape.reduce_spc(shape, step)
```

Arguments

Name	Size	Class	Unit	Default	Description
`shape`	1 × 1	struct		required	an SPC (stereophotoclinometry) shape model in its grid order, as in the DART g_<sampling>_spc_obj_<body>_... OBJ files: 6 cube faces of (q+1) x (q+1) vertices each, the first index fastest, 12 q^2 triangles Fields: shape.
`step`	scalar	double		required	a divisor of q, e.g. 8 Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`small`	12 (q/step)^2 facets and 6 (q/step)^2 + 2 vertices: the grid points (i, j) with i and j multiples of step, triangulated like the original (diagonal from (i, j) to (i+1, j+1)), the copies along the cu hape.

Description

The grid is radial (one surface point per direction from the body's center), so the result stays star-shaped about the center, unlike a general mesh reduction (sbtk.shape.reduce). Each cube face's triangles are ordered outward.

Example

MATLAB, from the help text

```
big   = sbtk.shape.load("g_00243mm_spc_obj_dimo_0000n00000_v004.obj", "km");   % q = 512
small = sbtk.shape.reduce_spc(big, 8);                                        % q = 64, 49,152 facets
sbtk.shape.write_obj(small, "g_00243mm_spc_obj_dimo_0000n00000_v004_every8.obj");
```

Related functions

Mentioned in the help text

sbtk.shape.load, sbtk.shape.reduce, sbtk.shape.write_obj

Calls

sbtk.shape.from_arrays, sbtk.shape.properties

Implementation: `toolbox/+sbtk/+shape/reduce_spc.m`, line 1.

---

sbtk.shape.write_obj

Write a shape model to a Wavefront OBJ file (vertices in km).

Syntax

MATLAB

```
sbtk.shape.write_obj(shape, file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`shape`	1 × 1	struct		required	The shape model. Fields: `shape`.
`file`	1 × 1	string		required	Path of the OBJ file to write; vertices are written in km.

Example

MATLAB, from the help text

```
sbtk.shape.write_obj(sbtk.shape.ellipsoid("test", [0.40 0.35 0.30], 4), "test.obj");
```

Related functions

Mentioned in the help text

`sbtk.shape.ellipsoid`

Implementation: `toolbox/+sbtk/+shape/write_obj.m`, line 1.

sbtk.body: Bodies

Function	Purpose
create	A small body: shape model, bulk density and reference radius.
from_mass	A small body defined by its mass instead of its bulk density.
summary	Print a body's main numbers.
to_frame	A body with its shape model in another frame: x' = (x - origin_km) * axes.

sbtk.body.create

A small body: shape model, bulk density and reference radius.

Syntax

MATLAB

```
body = sbtk.body.create(name, shape, bulk_density_kgm3, R_ref_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
name	1 × 1	string		required	
shape	1 × 1	struct		required	from sbtk.shape.load / from_arrays / ellipsoid Fields: shape.
bulk_density_kgm3	scalar	double	kg/m³	required	mean density [kg/m^3]; sets the total mass Validation: mustBePositive.
R_ref_km	scalar	double	km	required	reference radius of the gravity coefficients [km] Validation: mustBePositive.

Outputs

Name	Description
body	struct Fields: body.

Description

.volume_km3	enclosed volume [km^3]
.mass_kg	bulk_density_kgm3 * volume [kg]
.GM_km3_s2	G * mass [km^3/s^2]
.centroid_km	center of figure [km]

Example

MATLAB, from the help text

```
shape = sbtk.shape.ellipsoid("test", [0.4 0.35 0.3], 4);
body  = sbtk.body.create("Test", shape, 2600, 0.4);
```

Related functions

Mentioned in the help text

sbtk.shape.ellipsoid, sbtk.shape.load

Calls

sbtk.shape.properties, sbtk.units.G

Called by

sbtk.body.from_mass

Implementation: toolbox/+sbtk/+body/create.m, line 1.

sbtk.body.from_mass

A small body defined by its mass instead of its bulk density.

Syntax

```
MATLAB
body = sbtk.body.from_mass(name, shape, mass_kg, R_ref_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the body.
`shape`	1 × 1	struct		required	The shape model. Fields: `shape`.
`mass_kg`	scalar	double	kg	required	Total mass M. Validation: `mustBePositive`.
`R_ref_km`	scalar	double	km	required	Reference radius of the gravity coefficients. Validation: `mustBePositive`.

Outputs

Name	Description
`body`	The body, with bulk density = mass / shape volume. Fields: `body`.

Description

Same as sbtk.body.create with bulk density = mass / shape volume. Use it when the mass (or GM / G) is what was measured.

Example

```
MATLAB fragment from the help text. Uses variables defined by earlier calls: shape.
body = sbtk.body.from_mass("Didymos", shape, 5.273e11, 0.43);
```

Related functions

Mentioned in the help text

`sbtk.body.create`

Calls

`sbtk.body.create`, `sbtk.shape.properties`

Called by

`sbtk.body.to_frame`, `sbtk.data.to_reduced_shape`, `sbtk.exchange.read_data`, `sbtk.exchange.read_shadr`

Implementation: `toolbox/+sbtk/+body/from_mass.m`, line 1.

`sbtk.body.summary`

Print a body's main numbers.

Syntax

```
MATLAB
sbtk.body.summary(body)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body to describe. Fields: `body`.

Example

```
MATLAB, from the help text
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
sbtk.body.summary(body)
```

Related functions

Mentioned in the help text

`sbtk.body.from_mass`, `sbtk.shape.bundled`

Implementation: `toolbox/+sbtk/+body/summary.m`, line 1.

`sbtk.body.to_frame`

A body with its shape model in another frame: x' = (x - `origin_km`) * axes.

Syntax

MATLAB

```
body = sbtk.body.to_frame(body, origin_km, axes, shape_name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create or sbtk.body.from_mass Fields: `body`.
`origin_km`	1 × 3	double	km	required	origin of the new frame in the current frame [km]; [0 0 0] keeps the origin Validation: `mustBeFinite`.
`axes`	3 × 3	double		required	3 x 3 rotation, columns: the new axes as unit vectors in the current frame, e.g. cf.axes of sbtk.gravity.center_of_mass_frame; eye(3) keeps the axes Validation: `mustBeFinite`.
`shape_name`	1 × 1	string		required	name of the transformed shape model; an exchange package writes it as <data set>_<shape_name>.obj

Outputs

Name	Description
`body`	from sbtk.body.create or sbtk.body.from_mass Fields: `body`.

Description

The vertices become (v - `origin_km`)*axes; the mass, reference radius and name of the body are kept, and the center of figure moves with the shape. `shape.source` keeps the name of the original file (not its path, which a package must not carry), and `shape.frame_origin_km` and `shape.frame_axes` give the frame relative to that file, combined over successive changes of frame: vertices = (file vertices - `frame_origin_km`)*`frame_axes`. An exchange package states them in conventions.txt. Use the same `origin_km` and axes for the measurements (sbtk.data.to_frame) and the interior or true model (sbtk.interior.to_frame).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `truth`, `body`.

```
spin   = sbtk.gravity.center_of_mass_frame(truth, body, "spin");
body_s = sbtk.body.to_frame(body, spin.com_km, spin.axes, "didymos_v003_com_spin_axes");
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.body.from_mass`, `sbtk.data.to_frame`, `sbtk.gravity.center_of_mass_frame`, `sbtk.interior.to_frame`

Calls

`sbtk.body.from_mass`

Called by

`sbtk.interior.to_frame`

Implementation: `toolbox/+sbtk/+body/to_frame.m`, line 1.

`sbtk.interior`: Density distributions

Function	Purpose
`align_with_spin_axis`	Set the axis of components along the axis of the largest moment of the whole model.
`create`	An interior model: components plus an optional random density field.
`density_at`	Density [kg/m^3] of a model at points inside the body.
`ellipsoid`	Uniform triaxial ellipsoid component, rotated, clipped to the body.
`latlon`	Position [x y z] (km) from latitude, longitude (deg) and distance (km).
`layer`	Uniform layer component between two depths below the surface.
`no_components`	An empty list of components (a homogeneous body).
`no_random_field`	No random density field.
`random_field`	Gaussian random density field added to the interior.
`sector`	Uniform sector component: radial fractions x latitude/longitude range.
`sector_grid`	Sectors that tile the whole body (a model without assumed structure).
`set_axis`	Turn a component so that its axis points along a given direction.
`sphere`	Uniform sphere component (e.g. a core, a boulder or a void).
`sphere_grid`	Small spheres on a cubic grid filling the body (mass concentrations, a model without assumed structure).
`summary`	Print a list of components.
`to_frame`	An interior or a true model in another frame: x' = (x - `origin_km`) * axes.
`torus`	Uniform torus component about the z-axis, clipped to the body.

`sbtk.interior.align_with_spin_axis`

Set the axis of components along the axis of the largest moment of the whole model.

Syntax

MATLAB

```
[interior, axis_dir, iterations] = sbtk.interior.align_with_spin_axis(body, interior, names, tol_deg, max_iter)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	the body Fields: `body`.
`interior`	1 × 1	struct		required	from sbtk.interior.create Fields: `interior`.
`names`	1 × any	string		required	the components to align (string array; tori, ellipsoids or sectors), e.g. "torus"; strings(1, 0) for none
`tol_deg`	scalar	double	deg	required	convergence: the axis moves by at most this angle in the last iteration [deg] Validation: `mustBePositive`.
`max_iter`	scalar	double		required	largest number of iterations (sbtk:iteration:notConverged beyond it) Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`interior`	from sbtk.interior.create Fields: `interior`.
`axis_dir`	1 x 3 axis of the largest moment of the aligned interior, in the frame of the shape model (pointing to +z's side)
`iterations`	number of simulations made

Description

A body in principal-axis rotation spins about its axis of the largest moment, so a component that formed about the spin axis, such as an equatorial torus, lies about that axis. The axis depends on every component, the aligned ones included, so the alignment is a fixed-point iteration (sbtk.internal.fixed_point): simulate to degree 2, take the axis of the largest moment (sbtk.gravity.center_of_mass_frame), turn the named components to it (sbtk.interior.set_axis), repeat. Each iteration is one simulation; the axis usually settles to 0.01 deg in 2 to 4.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `core`.

```
t = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
t.center_km = body.centroid_km;
interior = sbtk.interior.create([core, t], sbtk.interior.no_random_field());
[interior, axis_dir] = sbtk.interior.align_with_spin_axis(body, interior, "torus", 0.01, 20);
fprintf('axis %.2f deg from the shape-model z-axis\n', acosd(axis_dir(3)));
```

Related functions

Mentioned in the help text

sbtk.gravity.center_of_mass_frame, sbtk.interior.create, sbtk.interior.no_random_field, sbtk.interior.set_axis, sbtk.interior.torus, sbtk.internal.fixed_point

Calls

sbtk.data.simulate, sbtk.gravity.center_of_mass_frame, sbtk.interior.set_axis

Implementation: `toolbox/+sbtk/+interior/align_with_spin_axis.m`, line 1.

---

sbtk.interior.create

An interior model: components plus an optional random density field.

Syntax

MATLAB

```
interior = sbtk.interior.create(components, field)
```

Arguments

Name	Size	Class	Unit	Default	Description
`components`	1 × any	struct		required	1 x n struct array from the component constructors (sphere, ellipsoid, torus, layer, sector); sbtk.interior.no_components() for none. Overlapping relative densities add; everything outside the body is clipped. Fields: `component`.
`field`	1 × 1	struct		`sbtk.interior.no_random_field()`	sbtk.interior.random_field(...); omit it (or pass sbtk.interior.no_random_field()) for none Fields: `random_field`.

Outputs

Name	Description
`interior`	Components and random field. Fields: `interior`.

Description

The background density is not set here: sbtk.data.simulate sets it so that the total mass matches the body.

Example

MATLAB, from the help text

```
components = [sbtk.interior.sphere("core", [0 0 0], 0.170, +450), ...
              sbtk.interior.layer("regolith", 0, 0.012, -900)];
interior = sbtk.interior.create(components);                                % no random field
interior = sbtk.interior.create(components, sbtk.interior.random_field(40, 0.06, 7));
```

Related functions

Mentioned in the help text

sbtk.data.simulate, sbtk.interior.layer, sbtk.interior.no_components, sbtk.interior.no_random_field, sbtk.interior.random_field, sbtk.interior.sphere

Calls

sbtk.interior.no_random_field

Called by

sbtk.data.to_reduced_shape, sbtk.plot.difference_from_homogeneous

Implementation: `toolbox/+sbtk/+interior/create.m`, line 1.

---

sbtk.interior.density_at

Density [kg/m^3] of a model at points inside the body.

Syntax

MATLAB

```
rho_kgm3 = sbtk.interior.density_at(model, X_km, geo)
```

Arguments

Name	Size	Class	Unit	Default	Description
`model`	1 × 1	struct		required	anything with `rho_bg_kgm3`, components and `random_field`: a true model from sbtk.data.simulate or a fit from sbtk.invert.run
`X_km`	any × 3	double	km	required	n x 3 points in shape-model coordinates [km] Validation: `mustBeFinite`.
`geo`	1 × 1	struct		required	geometry about the model's frame (sbtk.gravity.geometry(body, `model.origin_km`, 0)) Fields: `geo`.

Outputs

Name	Description
`rho_kgm3`	Density at each point, n × 1. The points are not tested for being inside the body: a point outside gets the background density plus any component that contains it.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`.

```
geo = sbtk.gravity.geometry(body, truth.origin_km, 0);
rho = sbtk.interior.density_at(truth, [0 0 0; 0.3 0 0] + truth.origin_km, geo);   % center, 300 m along x
```

Related functions

Mentioned in the help text

sbtk.data.simulate, sbtk.gravity.geometry, sbtk.invert.run

Called by

sbtk.plot.solution_cuts

Implementation: `toolbox/+sbtk/+interior/density_at.m`, line 1.

---

sbtk.interior.ellipsoid

Uniform triaxial ellipsoid component, rotated, clipped to the body.

Syntax

MATLAB

```
c = sbtk.interior.ellipsoid(name, center_km, semi_axes_km, angles_deg, drho_kgm3)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the component; unique within a model.
`center_km`	1 × 3	double	km	required	[x y z] center [km] Validation: `mustBeFinite`.
`semi_axes_km`	1 × 3	double	km	required	[a b c] semi-axes [km]; before rotation along x, y, z Validation: `mustBePositive`.
`angles_deg`	1 × 3	double	deg	required	[yaw pitch roll] [deg]: rotation about z, then y, then x Validation: `mustBeFinite`.
`drho_kgm3`	scalar	double	kg/m³	required	relative density [kg/m^3] Validation: `mustBeFinite`.

Outputs

Name	Description
`p`	The component. Fields: `component`.

Example

MATLAB, from the help text

```
core = sbtk.interior.ellipsoid("core", [0 0 0], [0.20 0.15 0.12], [30 0 0], +400);
```

Implementation: `toolbox/+sbtk/+interior/ellipsoid.m`, line 1.

`sbtk.interior.latlon`

Position [x y z] (km) from latitude, longitude (deg) and distance (km).

Syntax

MATLAB

```
xyz_km = sbtk.interior.latlon(lat_deg, lon_deg, distance_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`lat_deg`	scalar	double	deg	required	Latitude, from the xy-plane toward +z. Validation: `mustBeInRange(lat_deg, -90, 90)`.
`lon_deg`	scalar	double	deg	required	Longitude, from +x toward +y. Validation: `mustBeFinite`.
`distance_km`	scalar	double	km	required	Distance from the origin of the coordinates. Validation: `mustBeNonnegative`.

Outputs

Name	Description
`xyz_km`	Position $d\,[\cos\phi\cos\lambda,\ \cos\phi\sin\lambda,\ \sin\phi]$, a 1 × 3 vector. Add a point to measure from another origin.

Example

a boulder 250 m from the center at 25 N, 60 E

MATLAB, from the help text

```
b = sbtk.interior.sphere("boulder", sbtk.interior.latlon(25, 60, 0.250), 0.040, +900);
```

Related functions

Mentioned in the help text

`sbtk.interior.sphere`

Implementation: `toolbox/+sbtk/+interior/latlon.m`, line 1.

`sbtk.interior.layer`

Uniform layer component between two depths below the surface.

Syntax

MATLAB

```
c = sbtk.interior.layer(name, top_depth_km, bottom_depth_km, drho_kgm3)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the component; unique within a model.
`top_depth_km`	scalar	double	km	required	Depth of the top of the layer below the surface, along rays from the expansion origin (0 = at the surface). Validation: `mustBeNonnegative`.
`bottom_depth_km`	scalar	double	km	required	Depth of the bottom of the layer; larger than the top depth. Validation: `mustBePositive, mustBeGreaterThan(bottom_depth_km, top_depth_km)`.
`drho_kgm3`	scalar	double	kg/m³	required	Relative density: density minus the background density. Validation: `mustBeFinite`.

Outputs

Name	Description
`p`	The component. Fields: `component`.

Description

Depth is measured along the ray from the expansion origin (the center of mass), so sbtk.interior.layer("regolith", 0, 0.012, -900) is the outermost 12 m of the body.

Example

MATLAB, from the help text

```
regolith = sbtk.interior.layer("regolith", 0, 0.012, -900);
```

Implementation: `toolbox/+sbtk/+interior/layer.m`, line 1.

`sbtk.interior.no_components`

An empty list of components (a homogeneous body).

Syntax

MATLAB

```
components = sbtk.interior.no_components()
```

Outputs

Name	Description
`p`	An empty 1 × 0 component array. Fields: `component`.

Example

MATLAB, from the help text

```
interior = sbtk.interior.create(sbtk.interior.no_components());      % a homogeneous body
```

Related functions

Mentioned in the help text

`sbtk.interior.create`

Called by

`sbtk.data.to_reduced_shape`, `sbtk.interior.sector_grid`, `sbtk.interior.sphere_grid`, `sbtk.plot.difference_from_homogeneous`

Implementation: `toolbox/+sbtk/+interior/no_components.m`, line 1.

`sbtk.interior.no_random_field`

No random density field.

Syntax

MATLAB

```
field = sbtk.interior.no_random_field()
```

Outputs

Name	Description
`field`	A random field with standard deviation 0. Fields: `random_field`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `components`.

```
interior = sbtk.interior.create(components, sbtk.interior.no_random_field());   % same as create(components)
```

Related functions

Mentioned in the help text

`sbtk.interior.create`

Called by

`sbtk.interior.create`, `sbtk.invert.run`

Implementation: `toolbox/+sbtk/+interior/no_random_field.m`, line 1.

`sbtk.interior.random_field`

Gaussian random density field added to the interior.

Syntax

MATLAB

```
field = sbtk.interior.random_field(sigma_kgm3, corr_length_km, seed)
```

Arguments

Name	Size	Class	Unit	Default	Description
`sigma_kgm3`	scalar	double	kg/m³	required	standard deviation of the density perturbation [kg/m^3] Validation: `mustBeNonnegative`.
`corr_length_km`	scalar	double	km	required	correlation length [km]: correlation exp(-|dx|^2 / (2 L^2)) Validation: `mustBePositive`.
`seed`	scalar	double		required	random seed (the same seed gives the same field) Validation: `mustBeInteger, mustBeNonnegative`.

Outputs

Name	Description
`field`	The random field. Fields: `random_field`.

Description

The field is a sum of 300 random Fourier modes, defined at every point and independent of the expansion origin. sbtk.interior.no_random_field() for none.

Example

MATLAB, from the help text

```
field = sbtk.interior.random_field(40, 0.06, 7);   % sigma 40 kg/m^3, correlation length 60 m
```

Related functions

Mentioned in the help text

`sbtk.interior.no_random_field`

Implementation: `toolbox/+sbtk/+interior/random_field.m`, line 1.

`sbtk.interior.sector`

Uniform sector component: radial fractions x latitude/longitude range.

Syntax

MATLAB

```
c = sbtk.interior.sector(name, frac, lat_deg, lon_deg, drho_kgm3)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the component; unique within a model.
`frac`	1 × 2	double		required	[`f_in` `f_out`]: fraction of the local distance from the expansion origin to the surface (0 = center, 1 = surface) Validation: `mustBeInRange(frac, 0, 1)`.
`lat_deg`	1 × 2	double	deg	required	[`lat_min` `lat_max`] in -90..90 [deg] Validation: `mustBeInRange(lat_deg, -90, 90)`.
`lon_deg`	1 × 2	double	deg	required	[`lon_min` `lon_max`] in -360..360 [deg], at most 360 apart; ranges past +-180 wrap around. Every range must increase. Validation: `mustBeInRange(lon_deg, -360, 360)`.
`drho_kgm3`	scalar	double	kg/m³	required	Relative density: density minus the background density. Validation: `mustBeFinite`.

Outputs

Name	Description
`p`	The component. Fields: `component`.

Description

`drho_kgm3` relative density [kg/m^3]

Latitude and longitude are measured about the expansion origin, in the axes of the shape-model frame (`c.axes` = eye(3); sbtk.interior.to_frame turns them with the frame). sbtk.interior.sector("shell", [0.5 0.75], [-90 90], [-180 180], 0) is a shell that follows the shape. See also sbtk.interior.sector_grid.

Example

MATLAB, from the help text

```
block = sbtk.interior.sector("block", [0.3 0.9], [40 90], [150 240], +200);   % radial fraction, latitude, longitude
shell = sbtk.interior.sector("shell", [0.5 0.75], [-90 90], [-180 180], -300);
```

Related functions

Mentioned in the help text

`sbtk.interior.sector_grid`, `sbtk.interior.to_frame`

Called by

`sbtk.interior.sector_grid`

Implementation: `toolbox/+sbtk/+interior/sector.m`, line 1.

`sbtk.interior.sector_grid`

Sectors that tile the whole body (a model without assumed structure).

Syntax

MATLAB

```
components = sbtk.interior.sector_grid(frac_edges, lat_edges_deg, lon_edges_deg)
```

Arguments

Name	Size	Class	Unit	Default	Description
`frac_edges`	1 × any	double		required	radial fraction edges from 0 to 1, e.g. [0 0.45 0.8 1] Validation: `mustBeInRange(frac_edges, 0, 1)`.
`lat_edges_deg`	1 × any	double	deg	required	latitude edges from -90 to 90, e.g. [-90 0 90] Validation: `mustBeInRange(lat_edges_deg, -90, 90)`.
`lon_edges_deg`	1 × any	double	deg	required	longitude edges spanning 360 deg, e.g. [-180 -90 0 90 180] Validation: `mustBeInRange(lon_edges_deg, -360, 360)`.

Outputs

Name	Description
`components`	One sector per radial shell, latitude band and longitude band, named s<shell><lat><lon>, with relative density 0. Fields: `component`.

Description

One sector per (radial shell, latitude band, longitude band), named 's<shell><lat><lon>' (s111 = innermost shell, southernmost band, first longitude band), `drho_kgm3` = 0. The example edges give 3 x 2 x 4 = 24 sectors. Many components usually need regularization (sbtk.invert.choose_regularization). Because the sectors fill the body, the background density and a common shift of all sector densities are interchangeable: use regularization or hold the background at the bulk density (`opts.background` = "bulk").

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `obs`.

```
sectors = sbtk.interior.sector_grid([0 0.5 0.85 1], [-90 -30 30 90], [-180 -90 0 90 180]);   % 36
prob = sbtk.invert.setup(body, sectors, obs, sbtk.invert.options(background="bulk"));
prob.opts.regularization = sbtk.invert.choose_regularization(prob, "discrepancy");
fit = sbtk.invert.run(prob);
```

Related functions

Mentioned in the help text

`sbtk.invert.choose_regularization`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`

Calls

`sbtk.interior.no_components`, `sbtk.interior.sector`

Implementation: `toolbox/+sbtk/+interior/sector_grid.m`, line 1.

`sbtk.interior.set_axis`

Turn a component so that its axis points along a given direction.

Syntax

MATLAB

```
c = sbtk.interior.set_axis(c, direction)
```

Arguments

Name	Size	Class	Unit	Default	Description
`c`	1 × 1	struct		required	a torus, an ellipsoid or a sector (a component with axes, `c.axes`) Fields: `component`.
`direction`	1 × 3	double		required	1 x 3 direction in the frame of the shape model (normalized here) Validation: `mustBeFinite`.

Outputs

Name	Description
`c`	a torus, an ellipsoid or a sector (a component with axes, `c.axes`) Fields: `component`.

Description

The axis of a component is its third axis, `c.axes`(:, 3): the axis a torus lies about, the third semi-axis of an ellipsoid, the pole of a sector's latitudes. The component is turned by the smallest rotation that takes this axis to the direction, about its own center (`c.center_km`; a sector about the expansion origin), so its first two axes stay as close as possible to where they were.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `See`, `also`.

```
  t = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
  t.center_km = body.centroid_km;                          % about the center of figure
  t = sbtk.interior.set_axis(t, [sind(5) 0 cosd(5)]);      % axis tilted 5 deg toward +x
See also sbtk.interior.align_with_spin_axis.
```

Related functions

Mentioned in the help text

`sbtk.interior.align_with_spin_axis`, `sbtk.interior.torus`

Called by

`sbtk.interior.align_with_spin_axis`, `sbtk.invert.run`

Implementation: `toolbox/+sbtk/+interior/set_axis.m`, line 1.

`sbtk.interior.sphere`

Uniform sphere component (e.g. a core, a boulder or a void).

Syntax

MATLAB

```
c = sbtk.interior.sphere(name, center_km, radius_km, drho_kgm3)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the component; unique within a model.
`center_km`	1 × 3	double	km	required	[x y z] center in shape-model coordinates [km] (sbtk.interior.latlon converts latitude, longitude and distance) Validation: `mustBeFinite`.
`radius_km`	scalar	double	km	required	radius [km] Validation: `mustBePositive`.
`drho_kgm3`	scalar	double	kg/m³	required	relative density: density minus the background density [kg/m^3] Validation: `mustBeFinite`.

Outputs

Name	Description
`p`	The component. Fields: `component`.

Description

A sphere entirely inside the body is integrated exactly as a point mass; one that intersects the surface is clipped to the body. For the inversion, set `c.estimate_position` = true to estimate its center as well (or a mask such as [false false true] for z only; sbtk.invert.run).

Example

MATLAB, from the help text

```
core = sbtk.interior.sphere("core", [0 0 0], 0.170, +450);
```

Related functions

Mentioned in the help text

`sbtk.interior.latlon`, `sbtk.invert.run`

Called by

`sbtk.interior.sphere_grid`

Implementation: `toolbox/+sbtk/+interior/sphere.m`, line 1.

`sbtk.interior.sphere_grid`

Small spheres on a cubic grid filling the body (mass concentrations, a model without assumed structure).

Syntax

MATLAB

```
components = sbtk.interior.sphere_grid(body, spacing_km, radius_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create Fields: `body`.
`spacing_km`	scalar	double	km	required	grid spacing [km]; the grid is centered on the center of figure Validation: `mustBePositive`.
`radius_km`	scalar	double	km	required	radius of each sphere [km]; a sphere entirely inside the body is an exact point mass of its volume times its relative density Validation: `mustBePositive`.

Outputs

Name	Description
`components`	The spheres, one per grid point inside the body (`component` array). Fields: `component`.

Description

One sphere per grid point whose sphere lies inside the body (with a margin of one radius), named 'm<k>', `drho_kgm3` = 0. With `radius_km` well below `spacing_km` the spheres are point masses on the grid, the classic mass-concentration (mascon) model: a density anomaly drho in sphere k carries the mass drho*4/3*pi*r^3, which the grid cell around it (spacing^3) represents. Thousands of them need regularization (sbtk.invert.choose_regularization), and the data determine only a few combinations of them: read the estimate as where the anomaly is concentrated, not as a density map.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `obs`.

```
grid = sbtk.interior.sphere_grid(body, 0.040, 0.005);          % Didymos: about 3,000 points
an   = sbtk.interpret.anomaly(obs, body, 5);
sg   = sbtk.interpret.signatures(an, body, grid, an.labels);
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.interpret.anomaly`, `sbtk.interpret.signatures`, `sbtk.invert.choose_regularization`

Calls

`sbtk.gravity.geometry`, `sbtk.interior.no_components`, `sbtk.interior.sphere`

Called by

`sbtk.plot.solution_cuts`

Implementation: `toolbox/+sbtk/+interior/sphere_grid.m`, line 1.

`sbtk.interior.summary`

Print a list of components.

Syntax

MATLAB

```
sbtk.interior.summary(components)       % components: struct array, or a model's .components
```

Arguments

Name	Size	Class	Unit	Default	Description
`components`	1 × any	struct		required	Component array, or the `components` field of a model. Fields: `component`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `truth`.

```
sbtk.interior.summary(truth.components)
```

Related functions

Called by

`sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+interior/summary.m`, line 1.

`sbtk.interior.to_frame`

An interior or a true model in another frame: x' = (x - `origin_km`) * axes.

Syntax

MATLAB

```
model = sbtk.interior.to_frame(model, origin_km, axes)
```

Arguments

Name	Size	Class	Unit	Default	Description
`model`	1 × 1	struct		required	an interior (sbtk.interior.create) or a true model (truth from sbtk.data.simulate)
`origin_km`	1 × 3	double	km	required	origin of the new frame in the current frame [km]; [0 0 0] keeps the origin Validation: `mustBeFinite`.
`axes`	3 × 3	double		required	3 x 3 rotation, columns: the new axes as unit vectors in the current frame, e.g. cf.axes of sbtk.gravity.center_of_mass_frame; eye(3) keeps the axes Validation: `mustBeFinite`.

Outputs

Name	Description
`model`	an interior (sbtk.interior.create) or a true model (truth from sbtk.data.simulate)

Description

Every position is transformed as x' = (x - `origin_km`)*axes and every direction as d' = d*axes: the centers of spheres, ellipsoids and tori, the axes of ellipsoids, tori and sectors (`c.axes`), the random field, and for a true model `origin_km` and `com_km`. Its coefficients are rotated (exact, sbtk.internal.sh_rotation) and its inertia elements become axes'*I*axes; both stay about the same point. The density at every point of the body is unchanged, so the model, the body from sbtk.body.to_frame and the measurements from sbtk.data.to_frame (same `origin_km` and axes) describe the same body in the new frame: simulating, gridding, plotting and comparing there gives the transformed results. An inversion result cannot be transformed: transform the measurements and the body and invert in the new frame.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");
spin  = sbtk.gravity.center_of_mass_frame(truth, body, "spin");   % origin: center of mass; z: largest moment
truth_s = sbtk.interior.to_frame(truth, spin.com_km, spin.axes);
body_s  = sbtk.body.to_frame(body, spin.com_km, spin.axes, "didymos_com_spin_axes");
grid = sbtk.exchange.density_grid(body_s, truth_s, 0.020);        % the grid in the new frame
```

Related functions

Mentioned in the help text

`sbtk.body.to_frame`, `sbtk.data.simulate`, `sbtk.data.to_frame`, `sbtk.exchange.density_grid`, `sbtk.gravity.center_of_mass_frame`, `sbtk.interior.create`, `sbtk.internal.sh_rotation`

Calls

`sbtk.body.to_frame`, `sbtk.data.to_frame`

Implementation: `toolbox/+sbtk/+interior/to_frame.m`, line 1.

`sbtk.interior.torus`

Uniform torus component about the z-axis, clipped to the body.

Syntax

MATLAB

```
c = sbtk.interior.torus(name, ring_radius_km, tube_radius_km, height_km, drho_kgm3)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the component; unique within a model.
`ring_radius_km`	scalar	double	km	required	radius of the tube's center circle [km] Validation: `mustBeNonnegative`.
`tube_radius_km`	scalar	double	km	required	radius of the tube [km] Validation: `mustBePositive`.
`height_km`	scalar	double	km	required	z of the center circle's plane [km] Validation: `mustBeFinite`.
`drho_kgm3`	scalar	double	kg/m³	required	relative density [kg/m^3] Validation: `mustBeFinite`.

Outputs

Name	Description
`p`	The component. Fields: `component`.

Description

The torus lies about the z-axis of the shape-model frame (`c.center_km` = [0 0 0], `c.axes` = eye(3); sbtk.interior.to_frame moves and turns both with the frame). It may be buried, intersect the surface or extend outside the body; only its intersection with the body counts. sbtk.interior.set_axis turns it; sbtk.interior.align_with_spin_axis puts it about the model's own axis of the largest moment. An inversion can estimate its center (`c.estimate_position` = true, or a mask such as [false false true] for z only) and the direction of its axis (`c.estimate_axis` = true) besides its density (sbtk.invert.run).

Example

MATLAB, from the help text

```
torus = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
```

Related functions

Mentioned in the help text

`sbtk.interior.align_with_spin_axis`, `sbtk.interior.set_axis`, `sbtk.interior.to_frame`, `sbtk.invert.run`

Implementation: `toolbox/+sbtk/+interior/torus.m`, line 1.

Reference

`sbtk.gravity`: Gravity and moment calculations

Function	Purpose
`center_of_mass_frame`	A gravity field about its center of mass, in the shape-model, spin or principal axes.
`component_integrals`	Integrals over one component per unit density, clipped to the body.
`field_at`	Gravitational potential and acceleration of a model of the body at given points.
`forward`	Forward model: gravity coefficients, center of mass and inertia of an interior.
`geometry`	Facet-cone geometry of a body about an expansion origin (the integration workspace).
`labels`	Names of the gravity coefficients up to a degree, in SBTK order.
`polyhedron`	Potential and acceleration of a homogeneous polyhedron, exact (Werner & Scheeres 1997).
`random_field_integrals`	Density-weighted integrals of the random density field.

`sbtk.gravity.center_of_mass_frame`

A gravity field about its center of mass, in the shape-model, spin or principal axes.

Syntax

MATLAB

```
cf = sbtk.gravity.center_of_mass_frame(set, body, axes)
```

Arguments

Name	Size	Class	Unit	Default	Description
`set`	1 × 1	struct		required	the field: a measurement set (sbtk.data.simulate, sbtk.exchange.read_data, sbtk.exchange.read_shadr, ...), a true model (truth from sbtk.data.simulate) or an inversion result (fit from sbtk.invert.run: the field of the fitted model)
`body`	1 × 1	struct		required	the body: the shape model for a fitted model, the reference radius for a true model or a fitted model Fields: `body`.
`axes`	1 × 1	string		required	"shape": the axes of the shape-model frame; "spin": z along the axis of the largest moment of the field (the spin axis of a body in principal-axis rotation), x the shape-model x-axis projected onto the plane normal to it, y = z x x: the frame of body-fixed radio-science fields such as the OSIRIS-REx field of Bennu, with C21 = S21 = 0 and S22 not 0; "principal": the principal axes of inertia of the field (x the axis of the smallest moment, z of the largest, each pointing to the positive side of the shape-model axis nearest to it, right-handed), with C21 = S21 = S22 = 0 Validation: `mustBeMember(axes, ["shape", "spin", "principal"])`.

Outputs

Name	Description
`cf`	The field about its center of mass in the chosen axes, with its principal axes and moments. Fields: `cf`.

Description

`cf.labels`	1 x n names of the coefficients, 'C10' ... (degree 1 to cf.degree)
`cf.coef`	1 x n fully normalized coefficients about the center of mass: degree 1 is 0; in spin axes C21 = S21 = 0, in principal axes C21 = S21 = S22 = 0
`cf.sigma`	1 x n 1-sigma (NaN without uncertainties); degree 1: of the center of mass
`cf.cov`	n x n covariance (NaN without uncertainties)
`cf.com_km`	the center of mass in the shape-model frame [km]
`cf.axes`	3 x 3, columns: the axes of cf.coef as unit vectors in the shape-model axes (eye(3) for "shape"); sbtk.body.to_frame, sbtk.data.to_frame and sbtk.interior.to_frame take cf.com_km and cf.axes to express everything in the frame with the center of mass as origin and these axes
`cf.principal`	3 x 3, columns: the principal axes (smallest, middle, largest moment) in the shape-model axes
`cf.moments`	1 x 3 principal moments minus their mean, A <= B <= C, as I / (M `R_ref`^2) (the degree-2 field fixes the moments up to their trace)

cf.R_ref_km, cf.degree

Moving the expansion point to the center of mass makes degree 1 vanish: about a point O, C10, C11, S11 = (z, x, y of the center of mass - O)/(sqrt(3) `R_ref`). The translation of solid harmonics mixes lower degrees into higher ones (degree 2 into 3, ...), so all coefficients change; it is exact (sbtk.internal.sh_translation). Rotating the axes mixes the 2l+1 coefficients within each degree and leaves every degree's power unchanged; in the principal axes the degree-2 inertia tensor is diagonal, so C21 = S21 = S22 = 0 and C20 = (A + B - 2C)/(2 M R^2), C22 = (B - A)/(4 M R^2) (unnormalized). The rotation is exact too (sbtk.internal.sh_rotation). In the spin axes only the tilt of the axis of the largest moment is removed: C21 = S21 = 0, and the angle of the axis of the smallest moment from x is (1/2) atan2(S22, C22).

The center of mass: a true model's `com_km`; a fitted model's own center of mass (computed from its densities); for a measurement set, `origin_km` plus the offset its degree-1 coefficients give. A measurement set with `coef_about` 'origin' whose degree-1 coefficients put the center of mass more than 3 sigma from `origin_km` (more than 1 mm for exact values), e.g. a SHADR field about the shape-model origin, is moved to that center of mass; otherwise it is taken as about its center of mass already (its degree-1 values are then noise about zero). A set without degree-1 coefficients (e.g. sbtk.io.read_jpl_gravity) is taken as about its center of mass, `origin_km`; its degree-1 sigma is NaN. Moving a set and the principal axes need every coefficient of degree >= 2 up to the set's degree; a set that lacks some can be used only with axes "shape" and only if it is about its center of mass (missing coefficients are NaN).

The covariance is propagated through the translation and the rotation as linear maps; the uncertainty of the center of mass and of the principal axes themselves, which the data also determine, is not added.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");
cs = sbtk.gravity.center_of_mass_frame(truth, body, "shape");       % degree 1 = 0
cz = sbtk.gravity.center_of_mass_frame(truth, body, "spin");        % and C21 = S21 = 0
cp = sbtk.gravity.center_of_mass_frame(truth, body, "principal");   % and C21 = S21 = S22 = 0
disp([cs.labels(1:8); string(cs.coef(1:8)); string(cp.coef(1:8))])
tilt_deg = acosd(cp.principal(3, 3))         % largest-moment axis from the shape z-axis
```

Related functions

Mentioned in the help text

sbtk.body.to_frame, sbtk.data.simulate, sbtk.data.to_frame, sbtk.exchange.read_data, sbtk.exchange.read_shadr, sbtk.interior.to_frame, sbtk.internal.sh_rotation, sbtk.internal.sh_translation, sbtk.invert.run, sbtk.io.read_jpl_gravity

Calls

sbtk.gravity.component_integrals, sbtk.gravity.geometry

Called by

sbtk.interior.align_with_spin_axis, sbtk.plot.difference_from_homogeneous

Implementation: `toolbox/+sbtk/+gravity/center_of_mass_frame.m`, line 1.

---

sbtk.gravity.component_integrals

Integrals over one component per unit density, clipped to the body.

Syntax

MATLAB

```
I = sbtk.gravity.component_integrals(geo, c)
```

Arguments

Name	Size	Class	Unit	Default	Description
`geo`	1 × 1	struct		required	Geometry from `sbtk.gravity.geometry`: the expansion origin and degree of the integrals. Fields: `geo`.
`c`	1 × 1	struct		required	One component. Fields: `component`.

Outputs

Name	Description
`I`	Integrals of the part of the component inside the body, per unit density. Fields: `integrals`.

Description

`I.U`	1 x (L+1)^2 int (r/R)^l `Ybar_lm` dV (packed as `geo.pk`) [km^3]
`I.V`	volume [km^3]
`I.m1`	first moment [km^4]
`I.Q`	second moments [xx yy zz xy xz yz] [km^5]

all about `geo.origin_km` and limited to the body.

A sphere entirely inside the body is an exact point mass. Every other component is integrated ray by ray: each interval of a sub-ray inside the component, between scaled facets `s_a` and `s_b`, adds its exact cone-scaling integrals (see sbtk.gravity.geometry). Layers and sectors give the interval directly; for spheres, ellipsoids and tori the crossings of each ray with their surface are the real roots of a quadratic (sphere, ellipsoid) or quartic (torus) in the distance along the ray, so a ray may cross a component any number of times and the integrals change smoothly with the component's geometry.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
geo = sbtk.gravity.geometry(body, [0 0 0], 4);
I = sbtk.gravity.component_integrals(geo, sbtk.interior.sphere("core", [0 0 0], 0.170, 0));
fprintf('volume inside the body %.4f km^3\n', I.V);
```

Related functions

Mentioned in the help text

`sbtk.gravity.geometry`, `sbtk.interior.sphere`

Called by

`sbtk.gravity.center_of_mass_frame`, `sbtk.gravity.forward`, `sbtk.invert.recenter`, `sbtk.invert.run`, `sbtk.invert.scan_position`

Implementation: `toolbox/+sbtk/+gravity/component_integrals.m`, line 1.

`sbtk.gravity.field_at`

Gravitational potential and acceleration of a model of the body at given points.

Syntax

MATLAB

```
f = sbtk.gravity.field_at(body, model, points_km, grid_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create Fields: `body`.
`model`	1 × 1	struct		required	a true model (sbtk.data.simulate) or a fit (sbtk.invert.run): `rho_bg_kgm3`, components, `random_field`
`points_km`	any × 3	double	km	required	n x 3 points in the frame of the shape model [km]: outside, on or inside the body Validation: `mustBeFinite`.
`grid_km`	scalar	double	km	required	cell edge [km] of the grid that carries the departures from the background density (components, random field); not used for a homogeneous model Validation: `mustBePositive`.

Outputs

Name	Description
`f`	Potential `U_km2_s2` (n x 1) and acceleration `acc_km_s2` (n x 3) at the points, and `method`, how they were computed.

Description

The background density fills the shape model, and its field is exact (sbtk.gravity.polyhedron). A sphere entirely inside the body is a point mass outside it and a uniform ball inside it, also exact (a grid of point masses, sbtk.interior.sphere_grid, as well as a core). The other departures from the background (components that reach the surface, layers, tori, sectors, a random field) are taken at the cell centers of the density grid of the model (sbtk.exchange.density_grid), each cell a uniform cube (sbtk.internal.prism_field): the cubes follow the boundaries of these components and of the body only to within a cell, so a finer grid gives a more accurate field near them.

`f.U_km2_s2`	n x 1 potential [km^2/s^2], positive: U = G integral rho / |r - r'| dV
`f.acc_km_s2`	n x 3 acceleration [km/s^2], the gradient of U (the rotation of the body is not included)
`f.method`	the method, for the file header: "Werner and Scheeres 1997", and the cubes for a model that is not homogeneous

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
[~, truth] = sbtk.data.simulate(body, sbtk.interior.create(sbtk.interior.no_components()), 4, "homogeneous");
f = sbtk.gravity.field_at(body, truth, [0.5 0 0; 0 0.6 0], 0.020);
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.data.simulate`, `sbtk.exchange.density_grid`, `sbtk.gravity.polyhedron`, `sbtk.interior.create`, `sbtk.interior.no_components`, `sbtk.interior.sphere_grid`, `sbtk.internal.prism_field`, `sbtk.invert.run`

Calls

`sbtk.exchange.density_grid`, `sbtk.gravity.geometry`, `sbtk.gravity.polyhedron`, `sbtk.units.G`

Called by

`sbtk.exchange.write_potential`

Implementation: `toolbox/+sbtk/+gravity/field_at.m`, line 1.

`sbtk.gravity.forward`

Forward model: gravity coefficients, center of mass and inertia of an interior.

Syntax

MATLAB

```
fm = sbtk.gravity.forward(geo, interior, mass_kg)
```

Arguments

Name	Size	Class	Unit	Default	Description
`geo`	1 × 1	struct		required	Geometry from `sbtk.gravity.geometry`; the coefficients and inertia are about `geo.origin_km`. Fields: `geo`.
`interior`	1 × 1	struct		required	Components and random field. Fields: `interior`.
`mass_kg`	scalar	double	kg	required	Total mass; sets the background density. Validation: `mustBePositive`.

Outputs

Name	Description
`fm`	Background density, coefficients, center of mass and inertia. Fields: `fm`.

Description

density = background + sum of component relative densities + random field, with the background density set so that the total mass equals `mass_kg`.

`fm.rho_bg_kgm3`	background density [kg/m^3]
`fm.coef`	1 x (L+1)^2 fully normalized coefficients about `geo.origin_km`, packed as `geo.pk` (C00, C10, C11, S11, C20, ...)
`fm.com_km`	center of mass in shape-model coordinates [km]
`fm.inertia_norm`	[Ixx Iyy Izz Ixy Ixz Iyz] about `geo.origin_km` / (M `R_ref`^2)
`fm.mass_kg`	`mass_kg`

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
geo = sbtk.gravity.geometry(body, [0 0 0], 4);
fm  = sbtk.gravity.forward(geo, interior, body.mass_kg);
```

Related functions

Mentioned in the help text

`sbtk.gravity.geometry`

Calls

`sbtk.gravity.component_integrals`, `sbtk.gravity.random_field_integrals`

Called by

`sbtk.data.simulate`, `sbtk.data.to_reduced_shape`, `sbtk.plot.difference_from_homogeneous`

Implementation: `toolbox/+sbtk/+gravity/forward.m`, line 1.

`sbtk.gravity.geometry`

Facet-cone geometry of a body about an expansion origin (the integration workspace).

Syntax

MATLAB

```
geo = sbtk.gravity.geometry(body, origin_km, degree)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create Fields: `body`.
`origin_km`	1 × 3	double	km	required	expansion origin in shape-model coordinates [km] (1x3). Gravity coefficients, first moments and inertia are all about this point; layers and sectors are measured along rays from it. Validation: `mustBeFinite`.

Name	Size	Class	Unit	Default	Description
`degree`	scalar	double		required	highest spherical-harmonic degree (0 when only volumes, inertia or densities are needed: much faster) Validation: `mustBeInteger, mustBeNonnegative`.

Outputs

Name	Description
`geo`	The facet-cone geometry and integrals. Fields: `geo`.

Description

The body is the union of tetrahedral cones with apex at the origin and a surface facet as base. For every cone geo stores, per unit density,

`T`	spherical-harmonic integrals int (r/R)^l `Ybar_lm` dV [km^3] (nf x (L+1)^2)
`V`	volume [km^3], m1 first moment [km^4],
`Q`	second moments [xx yy zz xy xz yz] [km^5]

(T from the polyhedral kernel sbtk.internal.ComputePolyhedralCSBar_mex). Shrinking a cone about its apex by s multiplies r^l dV by s^(l+3), so the portion of a cone between the scaled facets `s_a` and `s_b` contributes exactly T (`s_b`^(l+3) - `s_a`^(l+3)), V (`s_b`^3 - `s_a`^3), m1 (`s_b`^4 - `s_a`^4), Q (`s_b`^5 - `s_a`^5). Interior components are described by such radial intervals along 9 sub-rays of every cone (sbtk.gravity.component_integrals).

geo fields (km units throughout)

```
body, origin_km, degree, R_ref_km, pk (coefficient packing, sbtk.internal.sh_packing)
nf, K                number of facets, sub-rays per facet (9)
faces, verts         mesh with vertices relative to origin_km
D, E, F, det6, EF, FD, DE    facet corners, 6 x cone volumes, edge cross products
T, V, m1, Q          per-cone integrals (above)
U_body, V_body, m1_body, Q_body   whole-body sums
ray_t, ray_dir       sub-ray surface distances [km] and unit directions (nf*K rows)
```

Didymos (49,152 facets): about 1 s at degree 4.

Every facet should face away from the origin (the shape star-shaped about it). Facets that face it are accepted when their cones hold less than 1e-6 of the volume: small overhangs of a high-resolution model, such as the 0.243 m Dimorphos model seen from a point a few meters from its center. Their cones count with their sign (negative), which is exact for the body and leaves an error of the order of their volume in layers and sectors. Otherwise sbtk:gravity:notStarShaped.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
geo = sbtk.gravity.geometry(body, body.centroid_km, 4);
fprintf('%d facets, volume %.4f km^3\n', geo.nf, geo.V_body);
```

Related functions

Mentioned in the help text

sbtk.body.create, sbtk.gravity.component_integrals, sbtk.internal.ComputePolyhedralCSBar_mex, sbtk.internal.sh_packing

Called by

sbtk.compare.reference, sbtk.data.simulate, sbtk.data.to_reduced_shape, sbtk.exchange.density_grid, sbtk.exchange.read_data, sbtk.gravity.center_of_mass_frame, sbtk.gravity.field_at, sbtk.interior.sphere_grid, sbtk.invert.setup, sbtk.plot.difference_from_homogeneous, sbtk.plot.interior3d, sbtk.plot.point_masses, sbtk.plot.search_masses, sbtk.plot.slices, sbtk.plot.solution_cuts, sbtk.plot.truth_vs_estimate

Implementation: `toolbox/+sbtk/+gravity/geometry.m`, line 1.

`sbtk.gravity.labels`

Names of the gravity coefficients up to a degree, in SBTK order.

Syntax

MATLAB

```
lab = sbtk.gravity.labels(degree)     % e.g. degree 2 -> {'C00','C10','C11','S11','C20','C21','S21','C22','S22'}
```

Arguments

Name	Size	Class	Unit	Default	Description
`degree`	scalar	double		required	Highest degree L. Validation: `mustBeInteger, mustBeNonnegative`.

Outputs

Name	Description
`lab`	1 × (L+1)² cell array of names in SBTK order, 'C00' … .

Example

MATLAB, from the help text

```
lab = sbtk.gravity.labels(4);              % 25 labels, C00 ... S44
```

Implementation: `toolbox/+sbtk/+gravity/labels.m`, line 1.

`sbtk.gravity.polyhedron`

Potential and acceleration of a homogeneous polyhedron, exact (Werner & Scheeres 1997).

Syntax

MATLAB

```
[U, acc] = sbtk.gravity.polyhedron(shape, density_kgm3, points_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`shape`	1 × 1	struct		required	from sbtk.shape.load (a closed mesh, triangles ordered so that their normals point outward) Fields: `shape`.
`density_kgm3`	scalar	double	kg/m³	required	uniform density [kg/m^3] Validation: `mustBeFinite`.
`points_km`	any × 3	double	km	required	n x 3 points in the frame of the shape model [km]: outside, on or inside the body Validation: `mustBeFinite`.

Outputs

Name	Description
`U`	n x 1 gravitational potential [km^2/s^2], positive: U = G integral rho / |r - r'| dV
`acc`	n x 3 gravitational acceleration [km/s^2], the gradient of U (no rotation)

Description

The closed form of Werner and Scheeres (1997, Celestial Mechanics and Dynamical Astronomy 65, 313-344), with `r_e` and `r_f` the vectors from the point to an edge and to a face, `n_f` the outward normal of the face and `n_fe` that of its edge in the plane of the face:

```
acc = -G rho sum_f sum_(e of f) n_f (n_fe . r_e) L_e + G rho sum_f n_f (r_f . n_f) w_f
```

where `L_e` = ln((a + b + e)/(a + b - e)) for an edge of length e whose ends are at distances a and b, and `w_f` is the signed solid angle of the face seen from the point (the sum of `w_f` is 4 pi inside the body and 0 outside). Each edge enters once for each of its two faces, which is the edge dyad of the paper. The expressions hold on the surface too. On an edge or a vertex `L_e` is infinite, but both factors in front of it are zero there and the terms go to zero (as d^2 ln d with the distance d from the edge), so they are taken as zero. About 1 s per point for 3 million facets.

Example

MATLAB, from the help text

```
shape = sbtk.shape.bundled("didymos_highres");
[U, acc] = sbtk.gravity.polyhedron(shape, 2600, [0.5 0 0]);        % 500 m along x
fprintf('U %.6e km^2/s^2, |acc| %.6e km/s^2\n', U, norm(acc));
```

Related functions

Mentioned in the help text

`sbtk.shape.bundled`, `sbtk.shape.load`

Calls

`sbtk.units.G`

Called by

`sbtk.gravity.field_at`

Implementation: `toolbox/+sbtk/+gravity/polyhedron.m`, line 1.

`sbtk.gravity.random_field_integrals`

Density-weighted integrals of the random density field.

Syntax

MATLAB

```
I = sbtk.gravity.random_field_integrals(geo, field)
```

Arguments

Name	Size	Class	Unit	Default	Description
`geo`	1 × 1	struct		required	Geometry from `sbtk.gravity.geometry`. Fields: `geo`.
`field`	1 × 1	struct		required	The random field. Fields: `random_field`.

Outputs

Name	Description
`I`	Density-weighted integrals of the random field. Fields: `integrals`.

Description

`I.U` = int drho (r/R)^l `Ybar_lm` dV, `I.V` = int drho dV, `I.m1`, `I.Q` likewise (units: kg/m^3 times km^n). The field is evaluated at the centroid of every radial shell of every facet cone (sbtk.internal.body_samples) and each shell contributes its exact cone-scaling integrals with that value. No field (`sigma_kgm3` = 0) gives zeros without sampling.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
geo = sbtk.gravity.geometry(body, [0 0 0], 4);
I = sbtk.gravity.random_field_integrals(geo, sbtk.interior.random_field(40, 0.06, 7));
```

Related functions

Mentioned in the help text

`sbtk.gravity.geometry`, `sbtk.interior.random_field`, `sbtk.internal.body_samples`

Called by

`sbtk.gravity.forward`

Implementation: `toolbox/+sbtk/+gravity/random_field_integrals.m`, line 1.

sbtk.data: Measurement sets

Function	Purpose
add_center_of_mass	Add degree-1 coefficients that place the center of mass at a given point.
add_libration	Add a measured libration ratio (B - A)/C.
add_moi	Add one measured inertia element to a measurement set.
add_moi_row	Add a measurement of any weighted sum of the inertia elements.
add_noise	One noisy realization of a measurement set.
expand_about	Measurements with the gravity field and the inertia about another point.
from_coefficients	Measurement set from gravity-coefficient matrices (e.g. a mission solution).
libration_ratio_from_amplitude	(B - A)/C from the forced libration of a synchronous satellite.
move_origin	Express measurements about the center of mass relative to a new expansion origin.
select_coefficients	Keep only the listed gravity coefficients.
select_degree	Keep only the coefficients up to a degree.
select_moi	Keep only the listed moment-of-inertia measurements.
set_covariance	Replace the coefficient covariance with a full matrix.
set_moi_sigma	Set the 1-sigma of moment-of-inertia measurements by label.
set_uncertainties	Give measurements a 1-sigma uncertainty per degree and per inertia quantity.
simulate	True model of an interior and its exact gravity coefficients and moments of inertia.
summary	Print what is in a measurement set.
to_frame	A measurement set in another frame: x' = (x - `origin_km`) * axes.
to_reduced_shape	Measurements for a coarser version of the same shape model, for faster fits.
uncertainties	1-sigma measurement uncertainties by degree and by inertia quantity, for sbtk.data.set_uncertainties.
use_uncertainties	Give measurements the coefficient uncertainties of another data set (e.g. a precision file).

---

sbtk.data.add_center_of_mass

Add degree-1 coefficients that place the center of mass at a given point.

Syntax

MATLAB

```
obs = sbtk.data.add_center_of_mass(obs, com_km, sigma_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set, without degree-1 coefficients. Fields: `obs`.
`com_km`	1 × 3	double	km	required	center of mass in shape-model coordinates [km]; `obs.origin_km` when the coefficients are expanded about the center of mass (degree 1 = 0) Validation: `mustBeFinite`.
`sigma_km`	scalar	double	km	required	1-sigma uncertainty of each component of the center of mass [km] Validation: `mustBePositive`.

Outputs

Name	Description
`obs`	The measurement set with C10, C11 and S11 added and `coef_about` set to '`center_of_mass`'. Fields: `obs`.

Description

With fully normalized coefficients, C10 = dz/(sqrt(3) R), C11 = dx/(sqrt(3) R), S11 = dy/(sqrt(3) R), with [dx dy dz] = `com_km` - `obs.origin_km`. In an inversion these terms constrain the first moment of the density, which sphere positions (`estimate_position`) can satisfy. Errors if the data already have degree-1 terms.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `file`, `body`.

```
% a field about the center of mass, which is at the shape-model origin within 1 m
obs = sbtk.io.read_jpl_gravity(file, body, "bennu_jpl", [0 0 0], 4);
obs = sbtk.data.add_center_of_mass(obs, [0 0 0], 0.001);
```

Related functions

Mentioned in the help text

`sbtk.io.read_jpl_gravity`

Called by

`sbtk.invert.setup`

Implementation: `toolbox/+sbtk/+data/add_center_of_mass.m`, line 1.

`sbtk.data.add_libration`

Add a measured libration ratio (B - A)/C.

Syntax

MATLAB

```
obs = sbtk.data.add_libration(obs, ratio, ratio_sigma, Izz_norm)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set; it must contain C22 and S22. Fields: `obs`.
`ratio`	scalar	double		required	measured (B - A)/C: A <= B are the principal moments of inertia about the axes in the xy-plane (the long and intermediate axes of a body spinning about z) and C = Izz. This is what the libration of a synchronously rotating body depends on (amplitude 2e/(3 ratio - 1) for forced libration in an orbit of eccentricity e). Validation: `mustBeFinite`.
`ratio_sigma`	scalar	double		required	its 1-sigma Validation: `mustBePositive`.
`Izz_norm`	scalar	double	M R_ref²	required	Izz / (M `R_ref`^2) used to scale the uncertainty (a homogeneous-body value is accurate enough) Validation: `mustBePositive`.

Outputs

Name	Description
`obs`	The measurement set with the `libration` row added. Fields: `obs`.

Description

The principal axes need not be the shape-model axes: their direction in the xy-plane comes from the measured C22 and S22 of obs (which must have them), and the measurement is stored as the linear row

```
    cos(2a) (Iyy − Ixx) − sin(2a) 2 Ixy − ratio Izz = 0 +− ratio_sigma Izz_norm,
```

(B - A) written with the angle a of the A axis from the x axis (sbtk.internal.libration_weights). Together with C22 and S22, which give B - A, the ratio measures C to a relative precision of about ratio_sigma/ratio.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
% (B − A)/C = 0.20 +− 0.03, with Izz/(M R_ref^2) about 0.39
obs = sbtk.data.add_libration(obs, 0.20, 0.03, 0.39);
```

Related functions

Mentioned in the help text

`sbtk.internal.libration_weights`

Calls

`sbtk.data.add_moi_row`

Called by

`sbtk.exchange.read_data`

Implementation: `toolbox/+sbtk/+data/add_libration.m`, line 1.

`sbtk.data.add_moi`

Add one measured inertia element to a measurement set.

Syntax

MATLAB

```
obs = sbtk.data.add_moi(obs, label, value_norm, sigma_norm)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`label`	1 × 1	string		required	'Ixx', 'Iyy', 'Izz', 'Ixy', 'Ixz' or 'Iyz' (about `obs.origin_km`, shape-model axes; products of inertia with the minus sign) Validation: `mustBeMember(label, ["Ixx", "Iyy", "Izz", "Ixy", "Ixz", "Iyz"])`.
`value_norm`	scalar	double	M R_ref²	required	measured I / (M `R_ref`^2) (sbtk.units.moi_to_norm converts kg m^2) Validation: `mustBeFinite`.
`sigma_norm`	scalar	double	M R_ref²	required	its 1-sigma, same unit Validation: `mustBePositive`.

Outputs

Name	Description
`obs`	The measurement set with the row added. Fields: `obs`.

Description

For any other combination of elements use sbtk.data.add_moi_row.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
obs = sbtk.data.add_moi(obs, "Izz", 0.0752, 2e-3);
```

Related functions

Mentioned in the help text

`sbtk.data.add_moi_row`, `sbtk.units.moi_to_norm`

Calls

`sbtk.data.add_moi_row`

Called by

`sbtk.exchange.read_data`

Implementation: `toolbox/+sbtk/+data/add_moi.m`, line 1.

`sbtk.data.add_moi_row`

Add a measurement of any weighted sum of the inertia elements.

Syntax

MATLAB

```
obs = sbtk.data.add_moi_row(obs, label, weights, value, sigma)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`label`	1 × 1	string		required	Name of the new row; must differ from the existing labels.
`weights`	1 × 6	double		required	Weights on $[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}]$ / (MR_{ref}^2). Validation: `mustBeFinite`.
`value`	scalar	double		required	Measured value of the weighted sum. Validation: `mustBeFinite`.
`sigma`	scalar	double		required	Its 1σ. Validation: `mustBePositive`.

Outputs

Name	Description
`obs`	The measurement set with the row added. Fields: `obs`.

Description

The measurement is weights * [Ixx Iyy Izz Ixy Ixz Iyz]' / (M `R_ref` ^2) = value +- sigma. label must not already be in `obs.moi`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
% Izz - (Ixx + Iyy)/2 = 0.09 +- 0.002 (as a fraction of M R_ref^2)
obs = sbtk.data.add_moi_row(obs, "polar_minus_mean", [-0.5 -0.5 1 0 0 0], 0.09, 0.002);
```

Related functions

Called by

`sbtk.data.add_libration`, `sbtk.data.add_moi`

Implementation: `toolbox/+sbtk/+data/add_moi_row.m`, line 1.

`sbtk.data.add_noise`

One noisy realization of a measurement set.

Syntax

MATLAB

```
obs = sbtk.data.add_noise(obs, seed)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	Measurement set with exact values and stated uncertainties. Fields: `obs`.
`seed`	scalar	double		required	Seed of the random number stream (non-negative integer); the same seed gives the same noise. Validation: `mustBeInteger, mustBeNonnegative`.

Outputs

Name	Description
`obs`	One noisy realization: coefficients plus noise drawn from `coef_cov` (full covariance), each inertia row plus noise from its σ; `noise_seed` set. Fields: `obs`.

Description

Adds Gaussian noise drawn from the stated uncertainties: the full coefficient covariance and each moment-of-inertia row's sigma. The same seed gives the same noise. The informational tensor `obs.moi_tensor_norm` is rebuilt from the noisy rows and the seed stored in `obs.noise_seed`. Errors if the measurements have no uncertainties (the exact values of sbtk.data.simulate).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`, `unc`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");
exact = sbtk.data.set_uncertainties(exact, unc);
obs = sbtk.data.add_noise(exact, 1);        % seed 1: the same noise every time
```

Related functions

Mentioned in the help text

`sbtk.data.set_uncertainties`, `sbtk.data.simulate`

Calls

`sbtk.data.set_covariance`, `sbtk.data.set_moi_sigma`, `sbtk.data.set_uncertainties`, `sbtk.data.use_uncertainties`

Called by

`sbtk.data.to_frame`

Implementation: `toolbox/+sbtk/+data/add_noise.m`, line 1.

`sbtk.data.expand_about`

Measurements with the gravity field and the inertia about another point.

Syntax

```
MATLAB
obs = sbtk.data.expand_about(obs, point_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	a measurement set whose coefficients are about `obs.origin_km` (`coef_about` = 'origin', as those of sbtk.data.simulate), with every coefficient of degree 1 to `obs.degree` and with moment-of-inertia element rows only (select them with sbtk.data.select_moi: the libration ratio is defined about the center of mass) Fields: obs.
`point_km`	1 × 3	double	km	required	the new expansion point in the frame of the shape model [km]; [0 0 0] is the origin of the shape model file Validation: `mustBeFinite`.

Outputs

Name	Description
`obs`	a measurement set whose coefficients are about `obs.origin_km` (`coef_about` = 'origin', as those of sbtk.data.simulate), with every coefficient of degree 1 to `obs.degree` and with moment-of-inertia element rows only (select them with sbtk.data.select_moi: the libration ratio is defined about the center of mass) Fields: obs.

Description

The coefficients are expanded about `point_km` exactly (sbtk.internal.sh_translation), with their covariance; degree 1 then gives the center of mass relative to `point_km`. The inertia elements are moved to `point_km` by the parallel-axis theorem, from the center of mass that degree 1 gives: I(P) = I(C) + M (|b|^2 `delta_ij` - `b_i` `b_j`), b = P - C. A package written from a set about [0 0 0] gives the gravity field, degree 1 included, and the moments of inertia about the origin of the shape model file (sbtk.exchange.write_data).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 6, "T1");
raw = sbtk.data.expand_about(sbtk.data.select_moi(exact, ["Ixx" "Iyy" "Izz"]), [0 0 0]);
```

Related functions

Mentioned in the help text

sbtk.data.select_moi, sbtk.data.simulate, sbtk.exchange.write_data, sbtk.internal.sh_translation

Calls

sbtk.data.move_origin, sbtk.data.select_moi

Implementation: `toolbox/+sbtk/+data/expand_about.m`, line 1.

`sbtk.data.from_coefficients`

Measurement set from gravity-coefficient matrices (e.g. a mission solution).

Syntax

```
MATLAB
obs = sbtk.data.from_coefficients(body, name, origin_km, C, S, C_sigma, S_sigma)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create: gives the mass and `R_ref`. `R_ref` must be the reference radius the coefficients are normalized with. Fields: body.
`name`	1 × 1	string		required	name of this data set
`origin_km`	1 × 3	double	km	required	point the coefficients are expanded about, in shape-model coordinates [km] (usually the center of mass) Validation: `mustBeFinite`.
`C`	any × any	double		required	(L+1) x (L+1) fully normalized coefficients, C(l+1, m+1) = `C_lm` (Kaula normalization, no Condon-Shortley phase); entries with m > l, the column S(:,1) and C00 are ignored Validation: `mustBeFinite`.
`S`	any × any	double		required	(L+1) x (L+1) fully normalized coefficients, C(l+1, m+1) = `C_lm` (Kaula normalization, no Condon-Shortley phase); entries with m > l, the column S(:,1) and C00 are ignored Validation: `mustBeFinite`.
`C_sigma`	any × any	double		required	1-sigma uncertainties, same layout (must be > 0 where used) Validation: `mustBeNonnegative`.
`S_sigma`	any × any	double		required	1-sigma uncertainties, same layout (must be > 0 where used) Validation: `mustBeNonnegative`.

Outputs

Name	Description
`obs`	The measurement set: coefficients of degree 1 … L, diagonal covariance from the uncertainties, no inertia rows. Fields: `obs`.

Description

Degrees 1..L are used. Add moments of inertia with sbtk.data.add_moi / `add_libration` and a full covariance with sbtk.data.set_covariance.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `C`, `S`, `Cs`, `Ss`.

```
obs = sbtk.data.from_coefficients(body, "hera_v1", [0 0 0], C, S, Cs, Ss);
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.data.add_moi`, `sbtk.data.set_covariance`

Calls

`sbtk.units.G`

Called by

`sbtk.exchange.read_shadr`

Implementation: `toolbox/+sbtk/+data/from_coefficients.m`, line 1.

`sbtk.data.libration_ratio_from_amplitude`

(B - A)/C from the forced libration of a synchronous satellite.

Syntax

MATLAB

```
[ratio, ratio_sigma] = sbtk.data.libration_ratio_from_amplitude(amplitude_deg, amplitude_sigma_deg, e, e_sigma)
```

Arguments

Name	Size	Class	Unit	Default	Description
`amplitude_deg`	scalar	double	deg	required	signed amplitude G of the forced libration gamma = G sin(M) [deg]: gamma is the angle of the long axis from the direction to the primary, M the mean anomaly. G < 0 (the long axis lags behind the primary direction while the satellite moves fastest) for (B - A)/C < 1/3, which covers elongations up to about 1.4 : 1; G > 0 above that. Validation: `mustBeNonzero`.
`amplitude_sigma_deg`	scalar	double	deg	required	1-sigma of G [deg] Validation: `mustBeNonnegative`.
`e`	scalar	double		required	orbital eccentricity and its 1-sigma Validation: `mustBeInRange(e, 0, 1, 'exclude-upper')`.
`e_sigma`	scalar	double		required	orbital eccentricity and its 1-sigma Validation: `mustBeNonnegative`.

Outputs

Name	Description
`ratio`	(B - A)/C (principal moments, see sbtk.data.add_libration) and its 1-sigma, from first-order error propagation
`ratio_sigma`	(B - A)/C (principal moments, see sbtk.data.add_libration) and its 1-sigma, from first-order error propagation

Description

Planar forced libration in a Keplerian orbit about a point-mass primary, to first order in e: gamma'' + 3 n^2 ratio gamma = 2 e n^2 sin(M), so G = 2e/(3 ratio - 1) and ratio = (1 + 2e/G)/3. Free libration, the primary's oblateness and the mutual dynamics of a binary are not included. The sensitivity grows near ratio = 1/3, where the free libration period equals the orbital period: an elongated body gives a large amplitude and a precise ratio, a nearly axisymmetric one a small amplitude that depends little on the ratio.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
% amplitude −12 +− 0.2 deg, e = 0.027 +− 0.001: (B − A)/C = 0.247 +− 0.0035 (1.4%)
[k, sk] = sbtk.data.libration_ratio_from_amplitude(−12, 0.2, 0.027, 0.001);
obs = sbtk.data.add_libration(obs, k, sk, 0.39);
% a nearly axisymmetric body: −4.0 +− 0.2 deg gives 0.075 +− 0.016 (21%)
```

Related functions

Mentioned in the help text

`sbtk.data.add_libration`

Implementation: `toolbox/+sbtk/+data/libration_ratio_from_amplitude.m`, line 1.

`sbtk.data.move_origin`

Express measurements about the center of mass relative to a new expansion origin.

Syntax

MATLAB

```
obs = sbtk.data.move_origin(obs, origin_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	Measurement set with `coef_about` = '`center_of_mass`'. Fields: `obs`.
`origin_km`	1 × 3	double	km	required	New expansion origin in the shape-model frame. Validation: `mustBeFinite`.

Outputs

Name	Description
`obs`	The same measurements with the degree-1 values relative to the new origin. Fields: `obs`.

Description

For `obs.coef_about` = '`center_of_mass`' only: the coefficients of degree >= 2 are about the body's center of mass wherever the origin is, and the degree-1 coefficients give the measured center of mass relative to the origin (C10 = dz/(sqrt(3) R), C11 = dx/(sqrt(3) R), S11 = dy/(sqrt(3) R)). Moving the origin changes only the degree-1 values; uncertainties are unchanged.

Example

MATLAB, from the help text

```
obs = sbtk.exchange.read_data("UCB-DATA1-Didymos");
moved = sbtk.data.move_origin(obs, obs.origin_km + [0.001 0 0]);     % 1 m along x
```

Related functions

Mentioned in the help text

`sbtk.exchange.read_data`

Called by

`sbtk.data.expand_about`

Implementation: `toolbox/+sbtk/+data/move_origin.m`, line 1.

`sbtk.data.select_coefficients`

Keep only the listed gravity coefficients.

Syntax

MATLAB

```
obs = sbtk.data.select_coefficients(obs, labels)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`labels`	1 × any	string		required	cell array or string array of names, e.g. {'C20', 'C22', 'S22'}

Outputs

Name	Description
`obs`	The measurement set with only the listed coefficients and the matching rows and columns of the covariance. Fields: `obs`.

Description

Use it to fit a subset of the coefficients (e.g. only those a mission determined well). The covariance keeps the matching rows and columns. See also sbtk.data.select_degree.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: obs.

```
obs = sbtk.data.select_coefficients(obs, ["C10" "C11" "S11" "C20" "C22" "S22"]);
```

Related functions

Mentioned in the help text

sbtk.data.select_degree

Called by

sbtk.exchange.read_shadr

Implementation: `toolbox/+sbtk/+data/select_coefficients.m`, line 1.

---

sbtk.data.select_degree

Keep only the coefficients up to a degree.

Syntax

MATLAB

```
obs = sbtk.data.select_degree(obs, degree)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: obs.
`degree`	scalar	double		required	Highest degree to keep. Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`obs`	The measurement set with the coefficients up to that degree. Fields: obs.

Description

Useful to test what a lower-degree solution can tell about the interior.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: obs.

```
obs2 = sbtk.data.select_degree(obs, 2);      % degrees 1 and 2 only
```

Implementation: `toolbox/+sbtk/+data/select_degree.m`, line 1.

---

sbtk.data.select_moi

Keep only the listed moment-of-inertia measurements.

Syntax

MATLAB

```
obs = sbtk.data.select_moi(obs, labels)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: obs.
`labels`	1 × any	string		required	e.g. {'Izz', 'libration'}; {} keeps none

Outputs

Name	Description
`obs`	The measurement set with only the listed inertia rows. Fields: `obs`.

Description

Use it to decide which inertia measurements a data set contains (for example before sbtk.exchange.write_data). See also sbtk.data.select_coefficients.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
obs = sbtk.data.select_moi(obs, ["Izz" "libration"]);
obs = sbtk.data.select_moi(obs, {});                % gravity coefficients only
```

Related functions

Mentioned in the help text

`sbtk.data.select_coefficients`, `sbtk.exchange.write_data`

Called by

`sbtk.data.expand_about`, `sbtk.data.set_moi_sigma`, `sbtk.data.set_uncertainties`, `sbtk.data.to_frame`

Implementation: `toolbox/+sbtk/+data/select_moi.m`, line 1.

`sbtk.data.set_covariance`

Replace the coefficient covariance with a full matrix.

Syntax

MATLAB

```
obs = sbtk.data.set_covariance(obs, coef_cov)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`coef_cov`	any × any	double		required	n x n covariance of `obs.coef`, in the order of `obs.coef_labels` (symmetric positive definite) Validation: `mustBeFinite`.

Outputs

Name	Description
`obs`	The measurement set with `coef_cov` replaced. Fields: `obs`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
sig = sqrt(diag(obs.coef_cov));
obs = sbtk.data.set_covariance(obs, (0.3 + 0.7*eye(numel(sig))).*(sig*sig'));   % 30% correlation
```

Related functions

Called by

`sbtk.data.add_noise`, `sbtk.exchange.write_data`, `sbtk.invert.replace_data`, `sbtk.invert.setup`

Implementation: `toolbox/+sbtk/+data/set_covariance.m`, line 1.

`sbtk.data.set_moi_sigma`

Set the 1-sigma of moment-of-inertia measurements by label.

Syntax

MATLAB

```
obs = sbtk.data.set_moi_sigma(obs, sigma)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`sigma`	1 × 1	struct		required	struct whose field names are measurement labels and whose values are 1-sigma: Ixx, Iyy, Izz, Ixy, Ixz, Iyz (or a label of sbtk.data.add_moi_row) as a fraction of M `R_ref`^2 libration of the ratio (B - A)/C; the row's 1-sigma becomes this times the Izz / (M `R_ref`^2) of the Izz row Rows not named keep their sigma. Errors for a label the data do not have.

Outputs

Name	Description
`obs`	The measurement set with the σ of the named rows replaced. Fields: `obs`.

Example

MATLAB fragment. Uses variables defined by earlier calls: `obs`, `body`.

```
% 1-sigma of the diagonal elements given in kg km^2: divide by M R_ref^2 in kg km^2
s = struct('Ixx', 2.0e9, 'Iyy', 2.0e9, 'Izz', 2.0e9);
obs = sbtk.data.set_moi_sigma(obs, structfun(@(v) v/(body.mass_kg*body.R_ref_km^2), s, 'UniformOutput', false));
```

Related functions

Mentioned in the help text

sbtk.data.add_moi_row

Calls

sbtk.data.select_moi

Called by

sbtk.data.add_noise, sbtk.exchange.write_data, sbtk.invert.replace_data, sbtk.invert.setup

Implementation: `toolbox/+sbtk/+data/set_moi_sigma.m`, line 1.

---

sbtk.data.set_uncertainties

Give measurements a 1-sigma uncertainty per degree and per inertia quantity.

Syntax

MATLAB

```
obs = sbtk.data.set_uncertainties(obs, unc)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	measurements, e.g. the exact values from sbtk.data.simulate, which have none Fields: `obs`.
`unc`	1 × 1	struct		required	from sbtk.data.uncertainties Fields: `unc`.

Outputs

Name	Description
`obs`	measurements, e.g. the exact values from sbtk.data.simulate, which have none Fields: `obs`.

Description

Every gravity coefficient of degree l gets the 1-sigma `unc.coef_sigma_by_degree`(l), and the coefficient covariance becomes diagonal. Every inertia element (Ixx, Iyy, Izz, Ixy, Ixz, Iyz) gets `unc.moi_sigma`. The libration row gets `unc.libration_sigma` times the Izz / (M `R_ref`^2) of the Izz row (set the uncertainties before sbtk.data.select_moi removes that row). Rows with other labels (sbtk.data.add_moi_row) keep their sigma. The values themselves do not change; sbtk.data.add_noise then draws noise from these uncertainties.

For other uncertainties: sbtk.data.use_uncertainties (those of another data set), sbtk.data.set_covariance (a full coefficient covariance), sbtk.data.set_moi_sigma (inertia rows by label).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
```

Related functions

Mentioned in the help text

sbtk.data.add_moi_row, sbtk.data.add_noise, sbtk.data.select_moi, sbtk.data.set_covariance, sbtk.data.set_moi_sigma, sbtk.data.simulate, sbtk.data.uncertainties, sbtk.data.use_uncertainties, sbtk.units.offset_to_degree1

Calls

sbtk.data.select_moi

Called by

sbtk.data.add_noise, sbtk.data.summary, sbtk.data.to_frame, sbtk.exchange.write_data, sbtk.invert.replace_data, sbtk.invert.setup

Implementation: `toolbox/+sbtk/+data/set_uncertainties.m`, line 1.

---

sbtk.data.simulate

True model of an interior and its exact gravity coefficients and moments of inertia.

Syntax

MATLAB

```
[exact, truth, geo] = sbtk.data.simulate(body, interior, degree, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create (shape, total mass, reference radius) Fields: `body`.
`interior`	1 × 1	struct		required	from sbtk.interior.create (components + random field) Fields: `interior`.
`degree`	scalar	double		required	maximum degree of the gravity coefficients Validation: `mustBeInteger, mustBePositive`.
`name`	1 × 1	string		required	name of the data set, e.g. "`didymos_T1`"

Outputs

Name	Description
`exact`	the exact values as a measurement set (docs/data_formats.md), without uncertainties (`coef_cov` and every inertia sigma are NaN); for Didymos to degree 4: `coef_labels` 24 x 1 {'C10';'C11';'S11';'C20';...}, coef 24 x 1, moi 7 rows (Ixx Iyy Izz Ixy Ixz Iyz, I/(M `R_ref`^2); libration: (B - A)/C with principal moments, see sbtk.data.add_libration), `origin_km` the center of mass the coefficients are about [km] Fields: `obs`.
`truth`	the true model (`rho_bg_kgm3`, components, `random_field`, `origin_km`, exact values) Fields: `truth`.
`geo`	geometry about the center of mass (for plots and comparisons) Fields: `geo`.

Description

The gravity field and inertia are expanded about the center of mass. Layers and sectors are defined along rays from the expansion origin, so the interior, and with it the center of mass, depend on the origin: the geometry is rebuilt about the center of mass until it moves by less than 1 mm (sbtk.internal.fixed_point, at most 50 times; sbtk:iteration:notConverged otherwise). No measurement uncertainties are involved: the values are exact.

To make measurements of the true model, give the exact values uncertainties (sbtk.data.set_uncertainties: one 1-sigma per degree and per inertia quantity; sbtk.data.use_uncertainties: those of another data set; sbtk.data.set_covariance, sbtk.data.set_moi_sigma) and draw noise from them (sbtk.data.add_noise).

Errors when the density is negative anywhere.

Example

MATLAB, from the help text

```
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
interior = sbtk.interior.create(sbtk.interior.sphere("core", [0 0 0], 0.170, +450));
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");          % the true model
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);  % measurements of it
```

Related functions

Mentioned in the help text

sbtk.body.create, sbtk.body.from_mass, sbtk.data.add_libration, sbtk.data.add_noise, sbtk.data.set_covariance, sbtk.data.set_moi_sigma, sbtk.data.set_uncertainties, sbtk.data.uncertainties, sbtk.data.use_uncertainties, sbtk.interior.create, sbtk.interior.sphere, sbtk.internal.fixed_point, sbtk.shape.bundled, sbtk.units.offset_to_degree1

Calls

sbtk.gravity.forward, sbtk.gravity.geometry, sbtk.units.G

Called by

sbtk.data.to_frame, sbtk.interior.align_with_spin_axis

Implementation: `toolbox/+sbtk/+data/simulate.m`, line 1.

`sbtk.data.summary`

Print what is in a measurement set.

Syntax

MATLAB

```
sbtk.data.summary(obs)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set to describe. Fields: `obs`.

Example output

Output

```
Didymos measurements (didymos_T1)
  mass 5.273e+11 kg, reference radius 430 m, gravity to degree 4
  24 gravity coefficients: 3 of degree 1, 5 of degree 2, 7 of degree 3, 9 of degree 4
  7 moment-of-inertia measurements: Ixx Iyy Izz Ixy Ixz Iyz libration
```

Related functions

Calls

`sbtk.data.set_uncertainties`

Called by

`sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+data/summary.m`, line 1.

`sbtk.data.to_frame`

A measurement set in another frame: x' = (x - `origin_km`) * axes.

Syntax

MATLAB

```
obs = sbtk.data.to_frame(obs, origin_km, axes)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	a measurement set; to change the axes it must have all six inertia elements without uncertainties, as the exact values of sbtk.data.simulate have (or no inertia rows): a measurement is made in the frame it is reported in, so transform the exact values, then give uncertainties, select and add noise Fields: `obs`.
`origin_km`	1 × 3	double	km	required	origin of the new frame in the current frame [km]; [0 0 0] keeps the origin Validation: `mustBeFinite`.
`axes`	3 × 3	double		required	3 x 3 rotation, columns: the new axes as unit vectors in the current frame, e.g. cf.axes of sbtk.gravity.center_of_mass_frame; eye(3) keeps the axes Validation: `mustBeFinite`.

Outputs

Name	Description
`obs`	a measurement set; to change the axes it must have all six inertia elements without uncertainties, as the exact values of sbtk.data.simulate have (or no inertia rows): a measurement is m bs.

Description

`origin_km` becomes (`origin_km` - o)*axes and `com_minus_cof_km` becomes `com_minus_cof_km`*axes. The coefficients stay about the same point and are rotated degree by degree, with their covariance (exact, sbtk.internal.sh_rotation); every degree present must be complete. The inertia elements Ixx ... Iyz are recomputed as the elements in the new axes (axes'*I*axes); the libration row and `libration_ratio` use the new z-axis (C = Izz) and the principal axes in the new xy-plane; any other row keeps its meaning (its weights are transformed). A translation alone (axes = eye(3)) applies to any set. Use the same `origin_km` and axes for the body (sbtk.body.to_frame) and the interior or true model (sbtk.interior.to_frame).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `truth`, `body`, `exact`, `unc`.

```
spin    = sbtk.gravity.center_of_mass_frame(truth, body, "spin");
exact_s = sbtk.data.to_frame(exact, spin.com_km, spin.axes);      % C21 = S21 = 0
obs_s   = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact_s, unc), 1);
```

Related functions

Mentioned in the help text

`sbtk.body.to_frame`, `sbtk.data.add_noise`, `sbtk.data.set_uncertainties`, `sbtk.data.simulate`, `sbtk.gravity.center_of_mass_frame`, `sbtk.interior.to_frame`, `sbtk.internal.sh_rotation`

Calls

`sbtk.data.add_noise`, `sbtk.data.select_moi`, `sbtk.data.set_uncertainties`, `sbtk.data.simulate`

Called by

`sbtk.interior.to_frame`

Implementation: `toolbox/+sbtk/+data/to_frame.m`, line 1.

`sbtk.data.to_reduced_shape`

Measurements for a coarser version of the same shape model, for faster fits.

Syntax

MATLAB

```
[obs, reduced] = sbtk.data.to_reduced_shape(obs, body, shape)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	measurements of a body with the shape model of body (e.g. 3 million facets) Fields: `obs`.
`body`	1 × 1	struct		required	that body Fields: `body`.
`shape`	1 × 1	struct		required	a coarser model of the same surface (sbtk.shape.reduce, sbtk.shape.reduce_spc, or a bundled reduced file such as "`didymos_every8`") Fields: `shape`.

Outputs

Name	Description
`obs`	measurements of a body with the shape model of body (e.g. 3 million facets) Fields: `obs`.
`reduced`	the coarser body (same mass and reference radius)

Description

The interior's part of the measurements depends on the shape only through the edge of the body, which the two models put within a few meters of each other, so fits on the coarse body give nearly the same densities as on the full one; the homogeneous part follows the facets and is replaced exactly. For the HLP6 Didymos model (1.165 m sampling, every 8th point) the replaced part is up to 1.5 sigma of the Hera radio-science precision at degree 2: without it, a fit on the coarse model would be biased. Check final results on the full model.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`, `body`.

```
[obs8, body8] = sbtk.data.to_reduced_shape(obs, body, sbtk.shape.bundled("didymos_every8"));
```

Related functions

Mentioned in the help text

`sbtk.shape.bundled`, `sbtk.shape.reduce`, `sbtk.shape.reduce_spc`

Calls

`sbtk.body.from_mass`, `sbtk.gravity.forward`, `sbtk.gravity.geometry`, `sbtk.interior.create`, `sbtk.interior.no_components`

Implementation: `toolbox/+sbtk/+data/to_reduced_shape.m`, line 1.

`sbtk.data.uncertainties`

1-sigma measurement uncertainties by degree and by inertia quantity, for sbtk.data.set_uncertainties.

Syntax

MATLAB

```
unc = sbtk.data.uncertainties(coef_sigma_by_degree, moi_sigma, libration_sigma)
```

Arguments

Name	Size	Class	Unit	Default	Description
`coef_sigma_by_degree`	1 × any	double		required	1 x L: 1-sigma of every fully normalized coefficient of degree 1, 2, ..., L (dimensionless). A center-of-mass uncertainty of d km corresponds to d/(sqrt(3) `R_ref`) at degree 1. Validation: `mustBePositive`.
`moi_sigma`	scalar	double		required	1-sigma of each element Ixx..Iyz / (M `R_ref`^2) Validation: `mustBePositive`.
`libration_sigma`	scalar	double		required	1-sigma of the libration ratio (B - A)/C (principal moments) Validation: `mustBePositive`.

Outputs

Name	Description
`unc`	The uncertainties. Fields: `unc`.

Description

Only measurements use it: the true model and its exact values (sbtk.data.simulate) do not. sbtk.data.set_uncertainties gives them to a measurement set.

Example

Didymos placeholders

MATLAB fragment from the help text. Uses variables defined by earlier calls: `exact`.

```
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
```

Related functions

Mentioned in the help text

sbtk.data.add_noise, sbtk.data.set_uncertainties, sbtk.data.simulate, sbtk.units.offset_to_degree1

Implementation: `toolbox/+sbtk/+data/uncertainties.m`, line 1.

---

sbtk.data.use_uncertainties

Give measurements the coefficient uncertainties of another data set (e.g. a precision file).

Syntax

MATLAB

```
obs = sbtk.data.use_uncertainties(obs, ref)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	measurements whose uncertainties are replaced (usually exact values from sbtk.data.simulate, before sbtk.data.add_noise) Fields: `obs`.
`ref`	1 × 1	struct		required	measurements holding the uncertainties to use, e.g. a precision file read with sbtk.exchange.read_shadr or sbtk.exchange.read_data Fields: `ref`.

Outputs

Name	Description
`obs`	measurements whose uncertainties are replaced (usually exact values from sbtk.data.simulate, before sbtk.data.add_noise) Fields: `obs`.

Description

Every coefficient of obs that ref also has takes ref's variances and covariances; the others (e.g. degree 1 if ref has none) keep theirs and become uncorrelated with them (exact values from sbtk.data.simulate have none: give them with sbtk.data.set_uncertainties first). If ref is about another point than obs (`ref.origin_km`, e.g. a radio-science field about the center of mass for data about the origin of a shape model file), its covariance is first moved to `obs.origin_km` exactly, K' = T K T' (sbtk.internal.sh_translation): moving a field to another point mixes the degrees, so the uncertainty of degree 1 (the center of mass) enters the others; ref then needs every coefficient of degree 1 to its degree. If the reference radii differ, ref's covariance is converted to `obs.R_ref_km`: a coefficient of degree l scales by (`R_ref` / `R_obs`)^l. Moment-of-inertia uncertainties are not changed (sbtk.data.set_moi_sigma sets them).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
shape = sbtk.shape.load("didymos_g_9309mm_spc_obj_0000n00000_v003.obj", "km");
prec  = sbtk.exchange.read_shadr("Didymos_gravity_fromRS.tab", "Didymos", shape, [0 0 0], 4);
[exact, truth] = sbtk.data.simulate(body, interior, 4, "didymos_T1");
exact = sbtk.data.use_uncertainties(exact, prec);      % every coefficient, degree 1 included
exact = sbtk.data.select_moi(exact, {});               % gravity only in this example
obs = sbtk.data.add_noise(exact, 1);
```

Related functions

Mentioned in the help text

sbtk.data.add_noise, sbtk.data.select_moi, sbtk.data.set_moi_sigma, sbtk.data.set_uncertainties, sbtk.data.simulate, sbtk.exchange.read_data, sbtk.exchange.read_shadr, sbtk.internal.sh_translation, sbtk.shape.load

Called by

sbtk.data.add_noise, sbtk.exchange.write_data, sbtk.invert.replace_data, sbtk.invert.setup

Implementation: `toolbox/+sbtk/+data/use_uncertainties.m`, line 1.

`sbtk.interpret`: Reading the gravity coefficients

Function	Purpose

Reference

`sbtk.invert`: Density estimation

Function	Purpose
`choose_regularization`	Tikhonov regularization parameter by the discrepancy principle or the lowest AIC.
`goodness_of_fit`	Reduced chi-square and chi-square test of a fit.
`homogeneous`	Fit a homogeneous body (no components) to the same measurements.
`options`	Inversion settings with their standard values.
`predict`	A fitted model's values of every measurement of a set, including those it was not fitted to.
`recenter`	Move one component so that the fitted model's center of mass matches the measured one.
`replace_data`	Use another realization of the same measurements without repeating the setup.
`run`	Estimate the densities for a prepared inversion (bounded, mass-constrained least squares).
`scan_position`	Fit the densities with one component placed at each of a list of centers: chi-square as a function of its position.
`setup`	Prepare an inversion: geometry and the design-matrix column of every component, computed once.
`solve`	Setup and run in one call.
`summary`	Print an inversion result.

`sbtk.invert.choose_regularization`

Tikhonov regularization parameter by the discrepancy principle or the lowest AIC.

Syntax

MATLAB

```
lambda = sbtk.invert.choose_regularization(prob, rule)
prob.opts.regularization = lambda;  fit = sbtk.invert.run(prob);
```

Arguments

Name	Size	Class	Unit	Default	Description
`prob`	1 × 1	struct		required	from sbtk.invert.setup Fields: `prob`.
`rule`	1 × 1	string		required	"discrepancy": the largest lambda for which chi^2 <= max(n, chi^2_min + sqrt(2n)), n measurements, chi^2_min the value at lambda = 10^-2: chi^2 may rise by about one standard deviation of its distribution. Bisection on log10(lambda) over [-2, 8] (9 solves). For noisy measurements of known precision. "aic": the lambda of the lowest Akaike information criterion, chi^2 + 2 x the effective number of parameters (`fit.n_obs` - `fit.dof`). A scan of log10(lambda) from -3 to 8 in steps of 0.25, then golden-section search about the lowest (about 75 solves). For comparing a regularized model with other models on equal terms. Validation: `mustBeMember(rule, ["discrepancy", "aic"])`.

Outputs

Name	Description
`lambda`	Tikhonov parameter chosen by the discrepancy principle.

Description

The regularization term lambda * sum((`drho_c` / `rho_mean`)^2) damps the relative densities. Models with many components (a sector grid) or weak data need it to keep noise from producing large alternating densities. The result is returned, not stored: set `prob.opts.regularization` before the final run.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `obs`.

```
sectors = sbtk.interior.sector_grid([0 0.5 1], [-90 0 90], [-180 0 180]);
prob = sbtk.invert.setup(body, sectors, obs, sbtk.invert.options(background="bulk"));
prob.opts.regularization = sbtk.invert.choose_regularization(prob, "discrepancy");
fit = sbtk.invert.run(prob);
```

Related functions

Mentioned in the help text

`sbtk.interior.sector_grid`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`

Calls

`sbtk.invert.run`

Implementation: `toolbox/+sbtk/+invert/choose_regularization.m`, line 1.

`sbtk.invert.goodness_of_fit`

Reduced chi-square and chi-square test of a fit.

Syntax

MATLAB

```
[chi2_reduced, p_value, consistent] = sbtk.invert.goodness_of_fit(chi2, dof)
```

Arguments

Name	Size	Class	Unit	Default	Description
`chi2`	scalar	double		required	sum of squared normalized residuals (with the full covariance) Validation: `mustBeNonnegative`.
`dof`	scalar	double		required	degrees of freedom: number of measurements minus the effective number of estimated parameters Validation: `mustBeFinite`.

Outputs

Name	Description
`chi2_reduced`	chi2/dof; about 1 when the model and the stated uncertainties are right
`p_value`	probability that a correct model gives a chi2 at least this large
`consistent`	`p_value` >= 0.05: the model is consistent with the measurements at the 5% significance level

Description

With dof <= 1e-6 (at least as many parameters as measurements) the test is undefined: `chi2_reduced` and `p_value` are NaN and consistent is false.

Example

MATLAB, from the help text

```
[chi2_reduced, p, consistent] = sbtk.invert.goodness_of_fit(30.2, 24);   % 1.26, 0.18, true
```

Related functions

Called by

`sbtk.invert.run`

Implementation: `toolbox/+sbtk/+invert/goodness_of_fit.m`, line 1.

`sbtk.invert.homogeneous`

Fit a homogeneous body (no components) to the same measurements.

Syntax

MATLAB

```
fit = sbtk.invert.homogeneous(prob)
```

Arguments

Name	Size	Class	Unit	Default	Description
`prob`	1 × 1	struct		required	Prepared inversion; its components are ignored. Fields: `prob`.

Outputs

Name	Description
`fit`	Fit of a homogeneous body: the bulk density everywhere, no free parameters. Fields: `fit`.

Description

The density is the bulk density everywhere (fixed by the mass), so there are no free parameters. If this fit is already consistent with the measurements (`fit.consistent`), the data do not require internal structure.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`.

```
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
h = sbtk.invert.homogeneous(prob);
fprintf('homogeneous body: reduced chi2 %.2f, p = %.2g\n', h.chi2_reduced, h.p_value);
```

Related functions

Mentioned in the help text

`sbtk.invert.options`, `sbtk.invert.setup`

Calls

`sbtk.invert.run`

Implementation: `toolbox/+sbtk/+invert/homogeneous.m`, line 1.

`sbtk.invert.options`

Inversion settings with their standard values.

Syntax

MATLAB

```
opts = sbtk.invert.options()                    % the standard values
opts = sbtk.invert.options(Name=Value, ...)     % with some of them changed
```

Arguments

Name	Size	Class	Unit	Default	Description
`moi_labels`				`{'Ixx', 'Iyy', 'Izz'}`	Name-value argument; moment-of-inertia measurements to fit, e.g. {'Ixx','Iyy','Izz'}, {'libration'}, or {} (gravity only) Validation: `mustBeText`.
`regularization`	scalar	double		`0`	Name-value argument; Tikhonov parameter lambda (0 = none). The penalty is lambda * sum((`drho_c` / `rho_mean`)^2) in units of chi^2. Use sbtk.invert.choose_regularization for many components. Validation: `mustBeNonnegative`.
`background`	1 × 1	string		`"estimate"`	Name-value argument; "estimate": the background density is estimated (with the mass constraint); "bulk": it is held at the mean density M/V, so the relative densities of the components conserve the mass by themselves Validation: `mustBeMember(settings.background, ["estimate", "bulk"])`.
`rho_max_kgm3`	scalar	double	kg/m³	`4500`	Name-value argument; upper bound on any component's density [kg/m^3] Validation: `mustBePositive`.
`max_iter`	scalar	double		`200`	Name-value argument; largest number of Gauss-Newton steps for the geometry of components with `estimate_position` or `estimate_axis` Validation: `mustBeInteger, mustBeNonnegative`.
`step_km`	scalar	double	km	`0.010`	Name-value argument; trust-region radius of a step of a center coordinate [km] Validation: `mustBePositive`.
`position_sigma_km`	scalar	double	km	`Inf`	Name-value argument; a priori 1-sigma of each estimated center coordinate about its Validation: `mustBePositive`.
`step_deg`	scalar	double	deg	`1`	Name-value argument; trust-region radius of a step of an axis tilt or turn [deg] Validation: `mustBePositive`.
`axis_sigma_deg`	scalar	double	deg	`Inf`	Name-value argument; a priori 1-sigma of each estimated axis tilt and turn about the starting orientation [deg]; Inf = no constraint Validation: `mustBePositive`.
`max_shift_km`	scalar	double	km	`Inf`	Name-value argument; largest distance of an estimated center from its starting point [km]; Inf = no limit (a sphere is kept inside the body in any case) Validation: `mustBePositive`.
`max_tilt_deg`	scalar	double	deg	`Inf`	Name-value argument; largest angle of an estimated axis from its starting direction [deg]; Inf = no limit Validation: `mustBePositive`.
`geometry_tol`	scalar	double		`1e-3`	Name-value argument; the geometry has converged when an accepted step changes every geometry parameter by less than this fraction of its step limit Validation: `mustBeNonnegative`.
`max_turn_deg`	scalar	double	deg	`Inf`	Name-value argument; largest turn of an estimated ellipsoid about its axis from its starting orientation [deg]; Inf = no limit Validation: `mustBePositive`.

Outputs

Name	Description
`opts`	The settings. Fields: `opts`.

Description

`starting point [km]; Inf = no constraint. Needed when more`	
`geometry is estimated than the measurements determine.`	

Standard values: `moi_labels` {'Ixx','Iyy','Izz'}, regularization 0, background "estimate", `rho_max_kgm3` 4500, `max_iter` 200, `step_km` 0.010, `step_deg` 1, `position_sigma_km` Inf, `axis_sigma_deg` Inf, `max_shift_km` Inf, `max_tilt_deg` Inf, `max_turn_deg` Inf, `geometry_tol` 1e-3. The fields can also be changed afterwards (`opts.regularization` = 10).

Example

MATLAB, from the help text

```
opts = sbtk.invert.options(moi_labels={'libration'}, background="bulk");
opts = sbtk.invert.options(moi_labels={});                % gravity coefficients only
```

Related functions

Mentioned in the help text

`sbtk.invert.choose_regularization`

Called by

`sbtk.invert.predict`, `sbtk.invert.solve`

Implementation: `toolbox/+sbtk/+invert/options.m`, line 1.

`sbtk.invert.predict`

A fitted model's values of every measurement of a set, including those it was not fitted to.

Syntax

MATLAB

```
pred = sbtk.invert.predict(fit, body, obs)
```

Arguments

Name	Size	Class	Unit	Default	Description
`fit`	1 × 1	struct		required	from sbtk.invert.run, fitted to some of the coefficients of obs (for example after sbtk.data.select_coefficients) or to all of them Fields: `fit`.
`body`	1 × 1	struct		required	the body of the fit Fields: `body`.
`obs`	1 × 1	struct		required	the measurements with every coefficient to predict (and the inertia rows the fit used, `fit.moi_labels`) Fields: `obs`.

Outputs

Name	Description
`pred`	The fitted model's values of every measurement of `obs`. Fields: `pred`.

Description

The fitted model (its components with their fitted geometry, background and relative densities) is set up for obs (sbtk.invert.setup) and evaluated: the same predictions as the fit for the measurements it was fitted to, and predictions for the others. For measurements about the center of mass (`coef_about` '`center_of_mass`') the coefficients of degree >= 2 and the inertia are predicted about the model's own center of mass, as the fit does. The model must be about the same point as the fit: obs must give the same measured center of mass (the same degree-1 coefficients), so that layers and sectors are measured from the same point; otherwise sbtk:invert:predictOrigin. Comparing the predictions of coefficients left out of a fit with their measurements tests the model on data it was not fitted to.

pred.labels, pred.measured, pred.predicted, pred.sigma every measurement of obs

```
                (coefficients, then the inertia rows of the fit), its prediction, 1-sigma
```

`pred.normalized_residual`	(predicted - measured)/sigma
`pred.chi2, pred.n_obs`	chi-square of every row (full covariance) and their number
`pred.fitted`	logical, one per row: the rows the fit was fitted to
`pred.chi2_fitted, pred.n_fitted`	chi-square and number of those rows (with their covariance)
`pred.chi2_other, pred.n_other`	the same for the other rows
`pred.fit_labels`	the rows the fit was fitted to

pred.data_name, pred.body

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`, `body`, `components`.

```
part = sbtk.data.select_coefficients(obs, ["C10" "C11" "S11" "C20"]);
fit  = sbtk.invert.solve(body, components, part, sbtk.invert.options(moi_labels={}));
pred = sbtk.invert.predict(fit, body, obs);
fprintf('fitted: chi2/n = %.2f; other coefficients: chi2/n = %.2f\n', ...
        pred.chi2_fitted/pred.n_fitted, pred.chi2_other/pred.n_other);
```

Related functions

Mentioned in the help text

`sbtk.data.select_coefficients`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`, `sbtk.invert.solve`

Calls

`sbtk.invert.options`, `sbtk.invert.setup`

Implementation: `toolbox/+sbtk/+invert/predict.m`, line 1.

`sbtk.invert.recenter`

Move one component so that the fitted model's center of mass matches the measured one.

Syntax

MATLAB

```
[prob, step_km] = sbtk.invert.recenter(prob, fit, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`prob`	1 × 1	struct		required	from sbtk.invert.setup, with degree-1 coefficients among the measurements Fields: `prob`.
`fit`	1 × 1	struct		required	sbtk.invert.run(prob) Fields: `fit`.
`name`	1 × 1	string		required	the component to move: a sphere, ellipsoid or torus

Outputs

Name	Description
`prob`	from sbtk.invert.setup, with degree-1 coefficients among the measurements Fields: `prob`.
`step_km`	1 x 3 the move [km]

Description

Degree 1 is the first moment of the density: moving a component of relative density drho and volume `V_c` by s moves the model's center of mass by drho `V_c` s / M. The component is moved by

```
    s = M (measured center of mass − model center of mass) / (drho V_c),
```

which makes the model's degree 1 equal to the measured one at the fitted densities (for a sphere inside the body exactly, otherwise to first order), and its integrals and design-matrix column are updated (no new geometry). A sphere is kept inside the body. Fitting the densities again and repeating until the step is small solves the degree-1 equations for the position while the densities fit every measurement: a fixed-point iteration. Gauss-Newton on the position (`estimate_position`, sbtk.invert.run) fits the position to every measurement at once instead; this step is a starting point for it or a check on it.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`, `opts`.

```
prob = sbtk.invert.setup(body, components, obs, opts);
for k = 1:10
    fit = sbtk.invert.run(prob);
    [prob, s] = sbtk.invert.recenter(prob, fit, "core");
    if norm(s) < 1e-4, break, end                       % 0.1 m
end
fit = sbtk.invert.run(prob);
```

Related functions

Mentioned in the help text

`sbtk.invert.run`, `sbtk.invert.setup`

Calls

`sbtk.gravity.component_integrals`

Implementation: `toolbox/+sbtk/+invert/recenter.m`, line 1.

`sbtk.invert.replace_data`

Use another realization of the same measurements without repeating the setup.

Syntax

MATLAB

```
prob = sbtk.invert.replace_data(prob, obs)
```

Arguments

Name	Size	Class	Unit	Default	Description
`prob`	1 × 1	struct		required	Prepared inversion. Fields: `prob`.
`obs`	1 × 1	struct		required	Another realization of the same measurements (same labels, inertia weights, origin, degree and mass). Fields: `obs`.

Outputs

Name	Description
`prob`	The inversion with the new measurement values and uncertainties; geometry and design matrix reused. Fields: `prob`.

Description

obs must have the same coefficient labels, moment-of-inertia rows (weights), origin, degree and mass as the data prob was set up with; only the values and uncertainties may differ (e.g. another noise seed). The geometry and design matrix are reused and a run takes milliseconds; for data about a fixed point, layers and sectors stay measured from the center of mass of the data prob was set up with.

Example

100 noise realizations

MATLAB fragment from the help text. Uses variables defined by earlier calls: `prob`, `exact`.

```
for k = 1:100
    fit = sbtk.invert.run(sbtk.invert.replace_data(prob, sbtk.data.add_noise(exact, k)));
end
```

Related functions

Mentioned in the help text

`sbtk.data.add_noise`, `sbtk.invert.run`

Calls

`sbtk.data.set_covariance`, `sbtk.data.set_moi_sigma`, `sbtk.data.set_uncertainties`, `sbtk.data.use_uncertainties`, `sbtk.invert.setup`

Implementation: `toolbox/+sbtk/+invert/replace_data.m`, line 1.

`sbtk.invert.run`

Estimate the densities for a prepared inversion (bounded, mass-constrained least squares).

Syntax

MATLAB

```
fit = sbtk.invert.run(prob)          % prob from sbtk.invert.setup
```

Arguments

Name	Size	Class	Unit	Default	Description
`prob`	1 × 1	struct		required	Prepared inversion from `sbtk.invert.setup` (or `sbtk.invert.replace_data`). Fields: `prob`.

Outputs

Name	Description
`fit`	Inversion result. Fields: `fit`.

Description

Unknowns: the background density, the relative density of each component, and the geometry asked for: the center of spheres, ellipsoids and tori with `estimate_position` (all coordinates, or those of a mask [x y z]) and the axis of ellipsoids and tori with `estimate_axis` = true. The total mass is an equality constraint and every component's density is bounded to [0, `opts.rho_max_kgm3`]. Measurements are whitened by the Cholesky factor of their covariance and the columns scaled to unit norm; the problem is solved with lsqlin (active set, or interior point if the active set does not converge). The geometry is refined by Gauss-Newton (at most `opts.max_iter` steps) with a trust region (`opts.step_km` for center coordinates, `opts.step_deg` for axis tilts): a step

that does not lower the objective is undone and the trust region halved, and a step that does doubles it up to those limits; it has converged when the objective changes by less than 1e-10 relative or a step changes every parameter by less than `opts.geometry_tol` of its step limit. Optional a priori constraints keep the geometry near its start (`opts.position_sigma_km`, `opts.axis_sigma_deg`), hard limits bound how far it may go (`opts.max_shift_km`, `opts.max_tilt_deg`, `opts.max_turn_deg`), and spheres are kept inside the body. The parameters are listed by sbtk.internal.geometry_parameters. Optional Tikhonov regularization (`opts.regularization`) penalizes the relative densities. The covariance is the linearized one with the mass constraint eliminated (inequality bounds are not included; see `fit.at_bound`).

fit (a model: `rho_bg_kgm3`, components, `random_field`, `origin_km`, plus)

`rho_bg_kgm3, rho_bg_sigma_kgm3`	background density and its 1-sigma [kg/m^3]
`components`	the components with the estimated relative densities `drho_kgm3`
`drho_sigma_kgm3`	1-sigma of each relative density
`rho_kgm3, rho_sigma_kgm3`	density of each component (background + relative) and its 1-sigma
`center_sigma_km`	n x 3 1-sigma of the center coordinates (NaN unless estimated)
`axis_sigma_deg`	n x 3 1-sigma of the rotations about the component's first, second and third axes [deg] (NaN unless estimated; a rotation about an axis of symmetry never is, sbtk.internal.free_rotations)
`at_bound`	density at 0 or `rho_max`
`resolved`	|relative density| > its 1-sigma
`determined`	false if the measurements (with any prior or regularization) do not determine all free parameters; cov is then a pseudo-inverse and the solution is not unique
`chi2, n_obs, dof`	chi-square, number of measurements, degrees of freedom (`n_obs` minus the effective number of parameters, trace of the data resolution matrix)
`chi2_reduced, p_value chi2/dof and P(chi2 >= observed) for a correct model`	
`consistent`	`p_value` >= 0.05 (model consistent with the measurements)
`labels, measured, predicted, sigma, normalized_residual`	
`every measurement, its predicted value, (predicted - measured)/sigma`	
`names, cov, corr`	parameters, their covariance and correlation
`iterations`	1 plus the Gauss-Newton steps to the best geometry (1 when none is estimated)
`geometry_converged`	false when `opts.max_iter` steps ended before the geometry converged
`regularization, moi_labels, background, body, data_name`	
`coef_about`	'origin' or '`center_of_mass`' (from the measurements)
`com_shift_km`	the point the predictions are about minus `origin_km`: for '`center_of_mass`' data the model's center of mass (coefficients of degree >= 2 and inertia); for 'origin' data the point the data are about (every coefficient and the inertia), `origin_km` being their measured center of mass
`frame_iterations`	Gauss-Newton steps on the model's center of mass (0 for 'origin')
`frame_offset_km`	change of that center in the last step (below 1e-6 km when converged)

When `obs.coef_about` is '`center_of_mass`' (packages read with sbtk.exchange.read_data, fields from sbtk.data.add_center_of_mass), the predicted coefficients of degree >= 2 and inertia are moved to the fitted model's own center of mass (exact translation), and the fit repeated until that center moves less than 1 mm. The model itself stays fixed: `origin_km` is the measured center of mass, from which layers and sectors are measured.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`.

```
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
fit  = sbtk.invert.run(prob);
sbtk.invert.summary(fit)
```

Related functions

Mentioned in the help text

sbtk.data.add_center_of_mass, sbtk.exchange.read_data, sbtk.internal.free_rotations, sbtk.internal.geometry_parameters, sbtk.invert.options, sbtk.invert.setup, sbtk.invert.summary

Calls

sbtk.gravity.component_integrals, sbtk.interior.no_random_field, sbtk.interior.set_axis, sbtk.invert.goodness_of_fit

Called by

sbtk.invert.choose_regularization, sbtk.invert.homogeneous, sbtk.invert.scan_position, sbtk.invert.solve

Implementation: `toolbox/+sbtk/+invert/run.m`, line 1.

`sbtk.invert.scan_position`

Fit the densities with one component placed at each of a list of centers: chi-square as a function of its position.

Syntax

MATLAB

```
sc = sbtk.invert.scan_position(prob, name, centers_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`prob`	1 × 1	struct		required	from sbtk.invert.setup Fields: `prob`.
`name`	1 × 1	string		required	the component to place: a sphere, ellipsoid or torus
`centers_km`	any × 3	double	km	required	k x 3 centers to try [km], e.g. points along the measured center-of-mass offset from the center of figure, or a grid Validation: `mustBeFinite`.

Outputs

Name	Description
`sc`	Chi-square and densities at each center. Fields: `sc`.

Description

For each center the component is moved there (its integrals and design-matrix column recomputed, no new geometry) and sbtk.invert.run fits the densities (and any geometry the other components estimate). The chi-square profile shows where the data put the component and whether more than one position fits: degree 1 fixes only the first moment, drho `V_c` times the offset, so a dense core on one side and a light one on the other give the same degree 1, and other components can make both fit. Start Gauss-Newton (`estimate_position`) from the lowest point, or compare the minima.

`sc.centers_km`	k x 3
`sc.chi2`	1 x k
`sc.chi2_reduced, sc.p_value`	1 x k
`sc.drho_kgm3`	k x n relative density of every component at each center
`sc.rho_bg_kgm3`	1 x k background density
`sc.best`	index of the lowest chi-square
`sc.fit`	the fit at sc.best

sc.name, sc.names (the components)

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`, `body`, `prob`.

```
an  = sbtk.interpret.anomaly(obs, body, 5);
u   = an.com_offset_km/norm(an.com_offset_km);              % direction of the center-of-mass offset
t   = (-0.08:0.005:0.08)';                                   % km along it
sc  = sbtk.invert.scan_position(prob, "core", body.centroid_km + t*u);
plot(1e3*t, sc.chi2)
```

Related functions

Mentioned in the help text

`sbtk.interpret.anomaly`, `sbtk.invert.run`, `sbtk.invert.setup`

Calls

`sbtk.gravity.component_integrals`, `sbtk.invert.run`

Implementation: `toolbox/+sbtk/+invert/scan_position.m`, line 1.

`sbtk.invert.setup`

Prepare an inversion: geometry and the design-matrix column of every component, computed once.

Syntax

MATLAB

```
prob = sbtk.invert.setup(body, components, obs, opts)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create (shape; the mass used is `obs.mass_kg`) Fields: `body`.
`components`	1 × any	struct		required	the model's components (sbtk.interior constructors). Their geometry is fixed except for the center of spheres, ellipsoids and tori with `estimate_position` (true, or a mask [x y z]) and the axis of ellipsoids and tori with `estimate_axis` = true (sbtk.invert.run). Their `drho_kgm3` is estimated (the value given is not used), except for components with `estimate_density` = false, whose value is kept. Fields: `component`.
`obs`	1 × 1	struct		required	the measurements (sbtk.data.simulate, sbtk.io.load_measurements, ...) Fields: `obs`.
`opts`	1 × 1	struct		required	from sbtk.invert.options Fields: `opts`.

Outputs

Name	Description
`prob`	everything sbtk.invert.run needs. Further runs with other options (`prob.opts.regularization` = ...), sbtk.invert.homogeneous and sbtk.invert.replace_data reuse it (Didymos, three components: setup 0.7 s, each run about 0.2 s; tools/benchmark.m). Fields: `prob`.

Description

Layers and sectors are measured along rays from the measured center of mass, as in sbtk.data.simulate: for data about the center of mass it is `obs.origin_km`, and for data about a fixed point (`coef_about` 'origin', e.g. a package about the origin of its shape model file) it is the center of mass their degree-1 coefficients give, unless that is within 3 sigma of the point (the 1-sigma of the measured center of mass along the offset): the data then do not tell the point from the center of mass, and the point is kept. The geometry is built about it (`prob.geo.origin_km`, the fit's `origin_km`), and the predictions are moved exactly to the point the data are about (`prob.shift_km`, sbtk.internal.observe).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`.

```
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
fit  = sbtk.invert.run(prob);                    % about 0.2 s
prob.opts.regularization = 5;                    % settings that do not select measurements
fit5 = sbtk.invert.run(prob);                    % can change between runs
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.data.simulate`, `sbtk.internal.observe`, `sbtk.invert.homogeneous`, `sbtk.invert.options`, `sbtk.invert.replace_data`, `sbtk.invert.run`, `sbtk.io.load_measurements`

Calls

`sbtk.data.add_center_of_mass`, `sbtk.data.set_covariance`, `sbtk.data.set_moi_sigma`, `sbtk.data.set_uncertainties`, `sbtk.data.use_uncertainties`, `sbtk.gravity.geometry`

Called by

`sbtk.invert.predict`, `sbtk.invert.replace_data`, `sbtk.invert.solve`

Implementation: `toolbox/+sbtk/+invert/setup.m`, line 1.

`sbtk.invert.solve`

Setup and run in one call.

Syntax

MATLAB

```
fit = sbtk.invert.solve(body, components, obs)          % standard options
fit = sbtk.invert.solve(body, components, obs, opts)    % opts from sbtk.invert.options
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body (shape model; the mass used is `obs.mass_kg`). Fields: `body`.
`components`	1 × any	struct		required	The model's components; their geometry is fixed and their relative densities are estimated (the values given are not used) unless `estimate_density` is false. Fields: `component`.
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`opts`	1 × 1	struct		`sbtk.invert.options()`	Settings from `sbtk.invert.options`. Fields: `opts`.

Outputs

Name	Description
`fit`	Inversion result. Fields: `fit`.

Description

Same as sbtk.invert.run(sbtk.invert.setup(body, components, obs, opts)). Use setup + run when solving the same problem more than once (other options, sbtk.invert.homogeneous, other noise realizations).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`.

```
fit = sbtk.invert.solve(body, components, obs);
fit = sbtk.invert.solve(body, components, obs, sbtk.invert.options(moi_labels={'Izz'}));
sbtk.invert.summary(fit)
```

Related functions

Mentioned in the help text

`sbtk.invert.homogeneous`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`, `sbtk.invert.summary`

Calls

`sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`

Implementation: `toolbox/+sbtk/+invert/solve.m`, line 1.

`sbtk.invert.summary`

Print an inversion result.

Syntax

MATLAB

```
sbtk.invert.summary(fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`fit`	1 × 1	struct		required	Inversion result to print. Fields: `fit`.

Example output

Didymos demo

Output

```
Didymos inversion of didymos_T1: 27 measurements
  chi2 = 26.74, dof = 24.0, reduced chi2 = 1.11, p = 0.32 (consistent with the measurements at the 5% level)
  component     density [kg/m^3]     relative density [kg/m^3]
  background      2708 +-    65
  core            3162 +-   205         +453 +-   268
```

Related functions

Called by

`sbtk.io.report_inversion`

Implementation: `toolbox/+sbtk/+invert/summary.m`, line 1.

`sbtk.search`: Searching the geometry

Function	Purpose

`sbtk.compare`: Comparison with a true model

Function	Purpose
`evaluate`	Compare a fit with the true model, using a prepared reference (see sbtk.compare.reference).
`grid_summary`	Print a cell-by-cell comparison of two density grids.
`grids`	Compare two density grids cell by cell (e.g. a solution package with an interior-model package).
`reference`	The true model sampled once for comparisons with fits that use a given set of components.
`summary`	Print a comparison with the true model.
`with_truth`	Compare an estimated density with the true density (synthetic tests).

`sbtk.compare.evaluate`

Compare a fit with the true model, using a prepared reference (see sbtk.compare.reference).

Syntax

MATLAB

```
cmp = sbtk.compare.evaluate(ref, fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`ref`	1 × 1	struct		required	Prepared reference from `sbtk.compare.reference`. Fields: `ref`.
`fit`	1 × 1	struct		required	Inversion result with the same components and origin as the reference. Fields: `fit`.

Outputs

Name	Description
`cmp`	Comparison with the true model. Fields: `cmp`.

Description

fit must use the same components (same geometry) as ref. Returns the fields described in sbtk.compare.with_truth.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `components`, `prob`, `exact`.

```
% many fits of the same components against one true model
ref = sbtk.compare.reference(body, truth, components, prob.geo.origin_km);   % once, about 0.5 s
for k = 1:100
    fit = sbtk.invert.run(sbtk.invert.replace_data(prob, sbtk.data.add_noise(exact, k)));
    cmp = sbtk.compare.evaluate(ref, fit);                                   % milliseconds
    R2(k) = cmp.R2;
end
```

Related functions

Mentioned in the help text

`sbtk.compare.reference`, `sbtk.compare.with_truth`, `sbtk.data.add_noise`, `sbtk.invert.replace_data`, `sbtk.invert.run`

Calls

`sbtk.compare.with_truth`

Called by

`sbtk.compare.with_truth`

Implementation: `toolbox/+sbtk/+compare/evaluate.m`, line 1.

`sbtk.compare.grid_summary`

Print a cell-by-cell comparison of two density grids.

Syntax

MATLAB

```
lines = sbtk.compare.grid_summary(cmp)
```

Arguments

Name	Size	Class	Unit	Default	Description
`cmp`	1 × 1	struct		required	from sbtk.compare.grids Fields: `gridcmp`.

Outputs

Name	Description
`lines`	the printed lines (string array), e.g. for writelines into a results folder

Example

MATLAB, from the help text

```
cmp = sbtk.compare.grids(sbtk.exchange.read_grid("UCB-MOD1-Didymos/density_grid.txt"), ...
                         sbtk.exchange.read_grid("UCB-SOL_UCB1/density_grid.txt"));
sbtk.compare.grid_summary(cmp);
```

Related functions

Mentioned in the help text

`sbtk.compare.grids`, `sbtk.exchange.read_grid`

Implementation: `toolbox/+sbtk/+compare/grid_summary.m`, line 1.

`sbtk.compare.grids`

Compare two density grids cell by cell (e.g. a solution package with an interior-model package).

Syntax

MATLAB

```
cmp = sbtk.compare.grids(truth, estimate)
```

Arguments

Name	Size	Class	Unit	Default	Description
`truth`	1 × 1	struct		required	from sbtk.exchange.read_grid or sbtk.exchange.density_grid, with the same spacing; cells present in both are compared (every cell has the same volume, so all have equal weight) Fields: `grid`.
`estimate`	1 × 1	struct		required	from sbtk.exchange.read_grid or sbtk.exchange.density_grid, with the same spacing; cells present in both are compared (every cell has the same volume, so all have equal weight) Fields: `grid`.

Outputs

Name	Description
`cmp`	Comparison of the grids. Fields: `gridcmp`.

Description

`cmp.R2`	1 - sum (estimate - true)^2 / sum (true - mean true)^2: the fraction of the variance of the true density explained
`cmp.rms_kgm3`	RMS of estimate - true
`cmp.mean_error_kgm3`	mean of estimate - true
`cmp.std_true_kgm3`	standard deviation of the true density
`cmp.within_2sigma`	fraction of cells with |estimate - true| <= 2 sigma of the estimate (NaN if the estimate has no uncertainties)

`cmp.n_common`, `cmp.n_truth`, `cmp.n_estimate` numbers of cells

Example

MATLAB, from the help text

```
cmp = sbtk.compare.grids(sbtk.exchange.read_grid("UCB-MOD1-Didymos/density_grid.txt"), ...
                         sbtk.exchange.read_grid("XYZ-SOL_UCB1/density_grid.txt"));
```

Related functions

Mentioned in the help text

`sbtk.exchange.density_grid`, `sbtk.exchange.read_grid`

Implementation: `toolbox/+sbtk/+compare/grids.m`, line 1.

`sbtk.compare.reference`

The true model sampled once for comparisons with fits that use a given set of components.

Syntax

MATLAB

```
ref = sbtk.compare.reference(body, truth, components, origin_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`truth`	1 × 1	struct		required	True model from `sbtk.data.simulate`. Fields: `truth`.
`components`	1 × any	struct		required	the components of the fits to be compared (their geometry; densities are not used) Fields: `component`.
`origin_km`	1 × 3	double	km	required	the point their layers and sectors are measured from: the fits' `origin_km` (the true model's origin for data in its frame, the measured center of mass for data about the center of mass) Validation: `mustBeFinite`.

Outputs

Name	Description
`ref`	The prepared reference. Fields: `ref`.

Description

Evaluates the true density at points filling the body (24 radial shells per facet cone, weighted by volume, in the true model's frame), the membership of every point in every component, and `R2_max` for these components. A model with more than 50,000 facets is sampled in the cones of 50,000 of them, chosen at random with a fixed seed (the same on every run, whatever the order of the facets in the file): about 1.2 million points, as for the 49,152-facet Didymos model, instead of 75 million for the 3,145,728-facet one, with R2 within 0.002 of the value from 9.4 million points. sbtk.compare.evaluate(ref, fit) then compares any such fit in milliseconds.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `components`, `prob`.

```
ref = sbtk.compare.reference(body, truth, components, prob.geo.origin_km);
cmp = sbtk.compare.evaluate(ref, sbtk.invert.run(prob));
% (see sbtk.compare.evaluate for a loop over noise realizations)
```

Related functions

Mentioned in the help text

`sbtk.compare.evaluate`, `sbtk.invert.run`

Calls

`sbtk.gravity.geometry`

Called by

`sbtk.compare.with_truth`

Implementation: `toolbox/+sbtk/+compare/reference.m`, line 1.

`sbtk.compare.summary`

Print a comparison with the true model.

Syntax

MATLAB

```
sbtk.compare.summary(cmp)          % cmp from sbtk.compare.with_truth
```

Arguments

Name	Size	Class	Unit	Default	Description
`cmp`	1 × 1	struct		required	Comparison from `sbtk.compare.with_truth` or `sbtk.compare.evaluate`. Fields: `cmp`.

Example output

Didymos demo

Output

```
Didymos: estimate compared with the true model
  R2 = 0.89 (fraction of the variance of the true density explained); R2_max = 0.98 for these components
```

Related functions

Mentioned in the help text

sbtk.compare.with_truth

Called by

sbtk.io.report_comparison

Implementation: `toolbox/+sbtk/+compare/summary.m`, line 1.

---

sbtk.compare.with_truth

Compare an estimated density with the true density (synthetic tests).

Syntax

MATLAB

```
cmp = sbtk.compare.with_truth(body, truth, fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`truth`	1 × 1	struct		required	the true model, from sbtk.data.simulate Fields: `truth`.
`fit`	1 × 1	struct		required	the estimate, from sbtk.invert.run Fields: `fit`.

Outputs

Name	Description
`cmp`	Comparison with the true model. Fields: `cmp`.

Description

Both densities are evaluated at points filling the body (24 radial shells per facet cone, 1.18 million for Didymos, each weighted by its volume); the true model's layers and sectors are measured from `truth.origin_km`, the fit's from `fit.origin_km`. About 0.5 s for Didymos.

`cmp.R2`	coefficient of determination of the estimated density: R2 = 1 - sum w (`rho_est` - `rho_true`)^2 / sum w (`rho_true` - mean)^2, the fraction of the variance of the true density (about its volume-weighted mean) that the estimate explains. 1 = exact; 0 = no better than a homogeneous body; < 0 = worse than that.
`cmp.R2_max`	the largest R2 attainable with the same components (weighted least-squares fit of the true density by the background and the components): the limit set by the choice of components
`cmp.rms_kgm3`	volume-weighted RMS density error [kg/m^3]
`cmp.std_true_kgm3`	volume-weighted standard deviation of the true density [kg/m^3]
`cmp.components`	components with the same name in both: name, `estimate_kgm3`, `sigma_kgm3`, `true_kgm3` (densities), `deviation_sigma` (|estimate - true| / sigma)
`cmp.chi2_reduced, cmp.p_value`	copied from the fit

For many fits with the same components against one true model, call sbtk.compare.reference once and sbtk.compare.evaluate for each fit.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `fit`.

```
cmp = sbtk.compare.with_truth(body, truth, fit);
sbtk.compare.summary(cmp)
sbtk.plot.comparison(cmp);
```

Related functions

Mentioned in the help text

sbtk.compare.evaluate, sbtk.compare.reference, sbtk.compare.summary, sbtk.data.simulate, sbtk.invert.run, sbtk.plot.comparison

Calls

sbtk.compare.evaluate, sbtk.compare.reference

Called by

sbtk.compare.evaluate

Implementation: `toolbox/+sbtk/+compare/with_truth.m`, line 1.

`sbtk.exchange`: Exchange packages

Function	Purpose
`density_grid`	Density of a model at the centers of a cubic grid inside the body.
`locate`	The exchange package of a given kind in a run folder.
`read_data`	Measurements and body from a measurement package (sbtk.exchange.write_data format).
`read_grid`	Density grid from an interior-model or solution package (`density_grid.txt`).
`read_shadr`	Measurements and body from a gravity file in the PDS SHADR layout (e.g. other teams' .tab files).
`solution_name`	Name of a solution package: <team>-SOL_<data team><number>.
`write_data`	Write a measurement package to send to other teams (nothing about the interior).
`write_model`	Write the interior-model package behind a measurement package (released after the solutions).
`write_potential`	Add the gravitational potential and acceleration at given points to a package.
`write_solution`	Write an inversion result as a solution package for the other teams.

`sbtk.exchange.density_grid`

Density of a model at the centers of a cubic grid inside the body.

Syntax

MATLAB

```
grid = sbtk.exchange.density_grid(body, model, spacing_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create Fields: `body`.
`model`	1 × 1	struct		required	a true model (sbtk.data.simulate) or a fit (sbtk.invert.run)
`spacing_km`	scalar	double	km	required	cell edge [km] Validation: `mustBePositive`.

Outputs

Name	Description
`grid`	The density grid. Fields: `grid`.

Description

Cell centers lie at integer multiples of `spacing_km` along each axis of the shape-model frame, so grids with the same spacing coincide whoever makes them; cells whose center is inside the shape model are kept (tested along rays from the center of figure, so every model of a body has the same cells). The density is the model's density at the cell center (random field included).

`grid.spacing_km`, `grid.x_km` (n x 3), `grid.density_kgm3` (n x 1), `grid.density_sigma_kgm3` (n x 1, zero here; sbtk.exchange.write_solution fills it from the fit's covariance), `grid.member` (n x ncomp: the cell center is inside component c)

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`.

```
grid = sbtk.exchange.density_grid(body, truth, 0.020);              % 20 m cells
scatter3(grid.x_km(:,1), grid.x_km(:,2), grid.x_km(:,3), 4, grid.density_kgm3, 'filled');
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.data.simulate`, `sbtk.exchange.write_solution`, `sbtk.invert.run`

Calls

`sbtk.gravity.geometry`

Called by

`sbtk.exchange.write_model`, `sbtk.exchange.write_solution`, `sbtk.gravity.field_at`

Implementation: `toolbox/+sbtk/+exchange/density_grid.m`, line 1.

`sbtk.exchange.locate`

The exchange package of a given kind in a run folder.

Syntax

MATLAB

```
folder = sbtk.exchange.locate(parent, kind)
```

Arguments

Name	Size	Class	Unit	Default	Description
`parent`	1 × 1	string		required	a run folder (or any folder holding a package) Validation: `mustBeFolder`.
`kind`	1 × 1	string		required	"DATA" (measurement package, <team>-DATA<n>...), "MOD" (interior-model package, <team>-MOD<n>...) or "SOL" (solution package, <team>-SOL_...) Validation: `mustBeMember(kind, ["DATA", "MOD", "SOL"])`.

Outputs

Name	Description
`folder`	Path of the package folder.

Description

Looks in parent and in its redacted/ and unredacted/ subfolders (the layout of simulation runs) and returns the one package of that kind. Errors if there is none or more than one.

Example

MATLAB, from the help text

```
run = sbtk.io.latest_run("hera_hlp6", "didymos_T1");
[obs, body] = sbtk.exchange.read_data(sbtk.exchange.locate(run, "DATA"));
model = sbtk.exchange.locate(run, "MOD");
```

Related functions

Mentioned in the help text

`sbtk.exchange.read_data`, `sbtk.io.latest_run`

Implementation: `toolbox/+sbtk/+exchange/locate.m`, line 1.

`sbtk.exchange.read_data`

Measurements and body from a measurement package (sbtk.exchange.write_data format).

Syntax

MATLAB

```
[obs, body] = sbtk.exchange.read_data(folder)
```

Arguments

Name	Size	Class	Unit	Default	Description
`folder`	1 × 1	string		required	the package folder (README.txt, `gravity_field.txt`, `gravity_covariance.txt`, `center_of_mass.txt`, `moments_of_inertia.txt`, conventions.txt, shape model). The shape model file the package names is read from the package, or from data/shapes/ when the package does not include it (sbtk.exchange.write_data) Validation: `mustBeFolder`.

Outputs

Name	Description
`obs`	the measurements, ready for sbtk.invert, in one of two forms: - a package about the origin of its shape model file (`gravity_field.txt` lists degree 1): the coefficients of degree 1..L as given, about `obs.origin_km` = [0 0 0] (`coef_about` = 'origin'), and the moments of inertia about the same origin; - a package about the center of mass: coefficients of degree 2..L with their full covariance, expanded about the measured c `bs.origin_km`, `coef_about` = '`center_of_mass`'), plus degree-1 coefficients 0 +- sigma that carry the uncertainty of the center of mass, and the moment-of-inertia rows Fields: `obs`.
`body`	from the package's shape model (lengths in km), mass = GM / G and the package's reference radius Fields: `body`.

Description

A package of exact values (no 1-sigma columns, no `gravity_covariance.txt`) gives a set with NaN uncertainties: give it uncertainties (e.g. sbtk.data.use_uncertainties with the precision file of the test) before sbtk.invert.

The libration ratio (B - A)/C becomes the row (B - A) - ratio*Izz = 0 with sigma `ratio_sigma` * Izz, where Izz is the measured Izz if the package has one, otherwise the Izz of the homogeneous body.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `components`.

```
[obs, body] = sbtk.exchange.read_data("UCB-DATA1-Didymos");
fit = sbtk.invert.solve(body, components, obs, sbtk.invert.options());
```

Related functions

Mentioned in the help text

`sbtk.data.use_uncertainties`, `sbtk.exchange.write_data`, `sbtk.invert.options`, `sbtk.invert.solve`

Calls

`sbtk.body.from_mass`, `sbtk.data.add_libration`, `sbtk.data.add_moi`, `sbtk.gravity.geometry`, `sbtk.root`, `sbtk.shape.load`, `sbtk.units.G`

Implementation: `toolbox/+sbtk/+exchange/read_data.m`, line 1.

`sbtk.exchange.read_grid`

Density grid from an interior-model or solution package (`density_grid.txt`).

Syntax

MATLAB

```
grid = sbtk.exchange.read_grid(file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`file`	1 × 1	string		required	Path of a `density_grid.txt` file of an interior-model or solution package. Validation: `mustBeFile`.

Outputs

Name	Description
`grid`	The density grid (without `member`). Fields: `grid`.

Description

`grid.spacing_km`, `grid.x_km` (n x 3), `grid.density_kgm3`, `grid.density_sigma_kgm3` Compare two grids with sbtk.compare.grids.

Example

MATLAB, from the help text

```
grid = sbtk.exchange.read_grid("UCB-MOD1-Didymos/density_grid.txt");
```

Related functions

Mentioned in the help text

`sbtk.compare.grids`

Implementation: `toolbox/+sbtk/+exchange/read_grid.m`, line 1.

`sbtk.exchange.read_shadr`

Measurements and body from a gravity file in the PDS SHADR layout (e.g. other teams' .tab files).

Syntax

MATLAB

```
[obs, body] = sbtk.exchange.read_shadr(file, body_name, shape, origin_km, degree)
```

Arguments

Name	Size	Class	Unit	Default	Description
`file`	1 × 1	string		required	text file in the layout of the PDS Spherical Harmonic ASCII Data Record: a header row `R_ref` [km], GM [km^3/s^2], sigma GM, degree, order, normalization state (1 = fully normalized), reference longitude, latitude then one row per (l, m): l, m, C, S, `sigma_C`, `sigma_S` Values may be separated by commas or spaces; lines starting with '#' and lines that do not start with a number (column names) are skipped. Validation: `mustBeFile`.
`body_name`	1 × 1	string		required	name of the body, e.g. "Didymos"
`shape`	1 × 1	struct		required	from sbtk.shape.load: the shape model the field refers to Fields: `shape`.
`origin_km`	1 × 3	double	km	required	point the coefficients are expanded about, in shape-model coordinates [km]: [0 0 0] if the field is about the shape-model origin (it then has degree-1 terms), the center of mass if it is about the center of mass Validation: `mustBeFinite`.
`degree`	scalar	double		required	highest degree to use Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`obs`	coefficients of degree 1..degree present in the file, covariance diag(sigma^2) (the format has no correlations), mass = GM / G Fields: `obs`.
`body`	sbtk.body.from_mass(`body_name`, shape, GM / G, `R_ref` of the file) Fields: `body`.

Description

A field about the center of mass has no degree-1 terms; add the measured center of mass with sbtk.data.add_center_of_mass.

Example

MATLAB, from the help text

```
shape = sbtk.shape.load("didymos_g_9309mm_spc_obj_0000n00000_v003.obj", "km");
[obs, body] = sbtk.exchange.read_shadr("Didymos_gravity.tab", "Didymos", shape, [0 0 0], 4);
```

Related functions

Mentioned in the help text

`sbtk.body.from_mass`, `sbtk.data.add_center_of_mass`, `sbtk.shape.load`

Calls

`sbtk.body.from_mass`, `sbtk.data.from_coefficients`, `sbtk.data.select_coefficients`, `sbtk.units.G`

Implementation: `toolbox/+sbtk/+exchange/read_shadr.m`, line 1.

`sbtk.exchange.solution_name`

Name of a solution package: <team>-SOL_<data team><number>.

Syntax

MATLAB

```
name = sbtk.exchange.solution_name(team, data_set)
```

Arguments

Name	Size	Class	Unit	Default	Description
`team`	1 × 1	string		required	the team that made the solution, e.g. "UCB"
`data_set`	1 × 1	string		required	the measurement package solved, named <data team>-DATA<number>[-comment]

Outputs

Name	Description
`name`	Package name `<team>-SOL_<data team><number>`.

Example

MATLAB, from the help text

```
sbtk.exchange.solution_name("UCB", "ROB-DATA1")          % "UCB-SOL_ROB1"
```

Implementation: `toolbox/+sbtk/+exchange/solution_name.m`, line 1.

`sbtk.exchange.write_data`

Write a measurement package to send to other teams (nothing about the interior).

Syntax

MATLAB

```
folder = sbtk.exchange.write_data(parent, data_set, body, obs, prepared_by)
```

Arguments

Name	Size	Class	Unit	Default	Description
`parent`	1 × 1	string		required	folder in which the package folder is created
`data_set`	1 × 1	string		required	package name, e.g. "UCB-DATA1-Didymos" (team - DATA<number> - comment)
`body`	1 × 1	struct		required	from sbtk.body.create Fields: `body`.
`obs`	1 × 1	struct		required	the measurements with degree-1 coefficients (the center of mass): exact values (a maker's package; the receiving team applies its own uncertainties), or values with uncertainties and noise (sbtk.data.add_noise) Fields: `obs`.
`prepared_by`	1 × 1	string		required	who made the package, e.g. "University of Colorado Boulder, name (e-mail)"

Outputs

Name	Description
`folder`	Path of the package folder that was created.

Description

Creates <parent>/<data_set>/ with

`README.txt`	summary and list of files
`gravity_field.txt`	coefficients of degree 1..L about the origin of the shape model file, or of degree 2..L about the center of mass (`exchange_data_files`)
`gravity_field_unnormalized.txt`	the same coefficients unnormalized
`gravity_covariance.txt`	their full covariance (values with uncertainties only)
`center_of_mass.txt`	center of mass and center of figure in the shape-model frame
`moments_of_inertia.txt`	inertia tensor elements and/or libration ratio
`conventions.txt`	units, frame, normalization (with code), degree 1
`<shape model>.obj the shape model, unless it is one of SBTK's shape models as`	
`distributed (data/shapes/): the files then name it, and`	
`sbtk.exchange.read_data finds it there`	

The files state only what applies: 1-sigma columns and the noise when the values have them (sbtk.internal.exchange_data_files). Nothing else of obs is written: not the expansion origin of the simulation (the true center of mass), the noise seed or the notes. Format: docs/exchange_format.md; read back with sbtk.exchange.read_data.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `body`, `exact`, `prepared_by`.

```
sbtk.exchange.write_data(fullfile(out.folder, "redacted"), "UCB-DATA1-Didymos", body, exact, prepared_by);
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.data.add_noise`, `sbtk.exchange.read_data`, `sbtk.internal.exchange_data_files`

Calls

`sbtk.data.set_covariance`, `sbtk.data.set_moi_sigma`, `sbtk.data.set_uncertainties`, `sbtk.data.use_uncertainties`, `sbtk.shape.properties`

Implementation: `toolbox/+sbtk/+exchange/write_data.m`, line 1.

`sbtk.exchange.write_model`

Write the interior-model package behind a measurement package (released after the solutions).

Syntax

MATLAB

```
folder = sbtk.exchange.write_model(parent, model_set, data_set, body, truth, obs_exact, spacing_km, prepared_by)
```

Arguments

Name	Size	Class	Unit	Default	Description
`parent`	1 × 1	string		required	folder in which the package folder is created
`model_set`	1 × 1	string		required	package name, e.g. "UCB-MOD1-Didymos"
`data_set`	1 × 1	string		required	name of the measurement package made from this model, e.g. "UCB-DATA1-Didymos"
`body`	1 × 1	struct		required	from sbtk.body.create Fields: `body`.
`truth`	1 × 1	struct		required	the true model (sbtk.data.simulate) Fields: `truth`.
`obs_exact`	1 × 1	struct		required	the measurements without noise: the exact values of sbtk.data.simulate, with or without uncertainties (without, their 1-sigma are written as NaN)
`spacing_km`	scalar	double	km	required	cell edge of the density grid [km] Validation: `mustBePositive`.
`prepared_by`	1 × 1	string		required	who made the package

Outputs

Name	Description
`folder`	Path of the package folder that was created.

Description

Creates <parent>/<model_set>/ with

`README.txt`	summary and list of files
`interior_model.txt`	background density and every component (type, densities, geometry), random density field, principal axes and moments of inertia
`density_grid.txt`	true density at the centers of a cubic grid (sbtk.exchange.density_grid)
`noise_free_data/`	the measurements of `data_set` without noise, in the same files
`figures/`	3-D view and slices of the true density

Compare a solution with it by sbtk.compare.grids on the two density grids.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `body`, `truth`, `exact`, `prepared_by`.

```
% exact: the measurements before sbtk.data.add_noise
sbtk.exchange.write_model(fullfile(out.folder, "unredacted"), "UCB-MOD1-Didymos", "UCB-DATA1-Didymos", ...
                          body, truth, exact, 0.020, prepared_by);
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.compare.grids`, `sbtk.data.add_noise`, `sbtk.data.simulate`, `sbtk.exchange.density_grid`

Calls

`sbtk.exchange.density_grid`, `sbtk.plot.interior3d`, `sbtk.plot.slices`, `sbtk.shape.properties`

Implementation: `toolbox/+sbtk/+exchange/write_model.m`, line 1.

`sbtk.exchange.write_potential`

Add the gravitational potential and acceleration at given points to a package.

Syntax

MATLAB

```
file = sbtk.exchange.write_potential(folder, body, model, points_km, grid_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`folder`	1 × 1	string		required	a package folder made by sbtk.exchange.write_data (or `write_model`) Validation: `mustBeFolder`.

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	the body of the package Fields: `body`.
`model`	1 × 1	struct		required	the model whose field is wanted: a true model (sbtk.data.simulate) or a fit
`points_km`	any × 3	double	km	required	n x 3 points in the frame of the shape model file [km] Validation: `mustBeFinite`.
`grid_km`	scalar	double	km	required	cell edge [km] for the departures from the background density (sbtk.gravity.field_at; not used for a homogeneous model) Validation: `mustBePositive`.

Outputs

Name	Description
`file`	Path of the `potential_acceleration.txt` file that was written.

Description

Writes `potential_acceleration.txt`, one row per point, in SI units as the HLP6 makers' cross-validation asks: the position [m], the potential U [m^2/s^2] (positive, U = G integral rho / |r - r'| dV) and the acceleration, the gradient of U [m/s^2], without the rotation of the body. The header names the method (sbtk.gravity.field_at). The file is added to the list of files in README.txt.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `data_set`, `body`, `exact`, `prepared_by`, `truth`, `points_km`.

```
pkg = sbtk.exchange.write_data(fullfile(out.folder, "redacted"), data_set, body, exact, prepared_by);
sbtk.exchange.write_potential(pkg, body, truth, points_km, 0.020);
```

Related functions

Mentioned in the help text

`sbtk.data.simulate`, `sbtk.exchange.write_data`, `sbtk.gravity.field_at`

Calls

`sbtk.gravity.field_at`

Implementation: `toolbox/+sbtk/+exchange/write_potential.m`, line 1.

`sbtk.exchange.write_solution`

Write an inversion result as a solution package for the other teams.

Syntax

MATLAB

```
folder = sbtk.exchange.write_solution(parent, solution_set, body, fit, homogeneous, spacing_km, prepared_by)
```

Arguments

Name	Size	Class	Unit	Default	Description
`parent`	1 × 1	string		required	folder in which the package folder is created
`solution_set`	1 × 1	string		required	package name, e.g. "UCB-`SOL_ROB1`" (team - SOL_<data team><number>)
`body`	1 × 1	struct		required	the body used in the inversion Fields: `body`.
`fit`	1 × 1	struct		required	from sbtk.invert.run (`fit.data_name` names the measurement package) Fields: `fit`.
`homogeneous`	1 × 1	struct		required	from sbtk.invert.homogeneous (the same data fitted by a homogeneous body) Fields: `fit`.
`spacing_km`	scalar	double	km	required	cell edge of the density grid [km]; use the spacing of the interior-model package for a cell-by-cell comparison Validation: `mustBePositive`.
`prepared_by`	1 × 1	string		required	who made the solution

Outputs

Name	Description
`folder`	Path of the package folder that was created.

Description

Creates <parent>/<solution_set>/ with

`README.txt`	method, measurements used, chi-square test, list of files
`solution.txt`	background and component densities with 1-sigma, geometry
`solution_covariance.txt`	covariance of the estimated parameters

`fitted_measurements.txt`	every measurement used: measured, predicted, sigma, normalized residual
`density_grid.txt`	estimated density and its 1-sigma on the cubic grid
`figures/`	residuals, densities, 3-D view, slices

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`, `out`, `prepared_by`.

```
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
fit  = sbtk.invert.run(prob);
sbtk.exchange.write_solution(out.folder, sbtk.exchange.solution_name("UCB", obs.name), body, fit, ...
                             sbtk.invert.homogeneous(prob), 0.020, prepared_by);
```

Related functions

Mentioned in the help text

`sbtk.exchange.solution_name`, `sbtk.invert.homogeneous`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`

Calls

`sbtk.exchange.density_grid`, `sbtk.plot.densities`, `sbtk.plot.interior3d`, `sbtk.plot.residuals`, `sbtk.plot.slices`

Implementation: `toolbox/+sbtk/+exchange/write_solution.m`, line 1.

Reference

`sbtk.io`: Results folders and text files

Function	Purpose
`finish_run`	Stop the run's log and say where the results are.
`latest_run`	Folder of the most recent run with a given name.
`load_fit`	Inversion result from a run folder (`fit.mat`) or a .mat file holding the variable fit.
`load_measurements`	Measurements from a run folder, a .mat file or a text file.
`load_truth`	True model from a run folder or a .mat file.
`new_run`	Create a results folder for one run and start its log.
`read_jpl_gravity`	Measurements from a JPL SSD gravity-field file (normalized coefficients + covariance).
`read_measurements`	Measurement set from the SBTK text format.
`report_comparison`	Save, print and plot a comparison with the true model.
`report_inversion`	Save, print and plot an inversion result.
`report_simulation`	Save, print and plot synthetic measurements and their true model.
`save_comparison`	comparison.mat and comparison.txt in the run folder.
`save_figure`	Save a figure as figures/<name>.png (200 dpi) in the run folder.
`save_fit`	`fit.mat` and `fit.txt` in the run folder.
`save_measurements`	measurements.mat and measurements.txt in the run folder.
`save_settings`	Write the settings of a run to `settings.json` (readable, reproducible).
`save_truth`	`truth.mat` and `truth.txt` in the run folder.
`subfolder`	A subfolder of a run folder that the save and report functions write to.
`write_comparison`	Plain-text copy of a comparison with the true model.
`write_fit`	Plain-text summary of an inversion result.
`write_measurements`	Plain-text copy of a measurement set (read back by sbtk.io.read_measurements).
`write_truth`	Plain-text description of a true model (keep private in blind tests).

`sbtk.io.finish_run`

Stop the run's log and say where the results are.

Syntax

MATLAB

```
sbtk.io.finish_run(out)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder whose log to close. Fields: `out`.

Example

MATLAB, from the help text

```
out = sbtk.io.new_run("my_project", "T1");
% ... everything printed until here goes to out.folder/log.txt
sbtk.io.finish_run(out);
```

Related functions

Mentioned in the help text

`sbtk.io.new_run`

Implementation: `toolbox/+sbtk/+io/finish_run.m`, line 1.

`sbtk.io.latest_run`

Folder of the most recent run with a given name.

Syntax

```
MATLAB
folder = sbtk.io.latest_run(project, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`project`	1 × 1	string		required	Project name.
`name`	1 × 1	string		required	Run name; only folders whose name after the time stamp equals it match.

Outputs

Name	Description
`folder`	Path of the newest matching run folder.

Example

an inversion reads the newest simulation run

```
MATLAB, from the help text
obs = sbtk.io.load_measurements(sbtk.io.latest_run("hera_hlp6", "didymos_T1"));
```

Related functions

Mentioned in the help text

`sbtk.io.load_measurements`

Calls

`sbtk.root`

Implementation: `toolbox/+sbtk/+io/latest_run.m`, line 1.

`sbtk.io.load_fit`

Inversion result from a run folder (`fit.mat`) or a .mat file holding the variable fit.

Syntax

```
MATLAB
fit = sbtk.io.load_fit(source)
```

Arguments

Name	Size	Class	Unit	Default	Description
`source`	1 × 1	string		required	A run folder (reads `fit.mat`) or a .mat file holding the variable fit.

Outputs

Name	Description
`fit`	Inversion result. Fields: `fit`.

Example

```
MATLAB, from the help text
fit = sbtk.io.load_fit(sbtk.io.latest_run("hera_hlp6", "didymos_T1_inversion"));
```

Related functions

Mentioned in the help text

`sbtk.io.latest_run`

Implementation: `toolbox/+sbtk/+io/load_fit.m`, line 1.

`sbtk.io.load_measurements`

Measurements from a run folder, a .mat file or a text file.

Syntax

```
MATLAB
obs = sbtk.io.load_measurements(source)
```

Arguments

Name	Size	Class	Unit	Default	Description
`source`	1 × 1	string		required	a run folder (reads its measurements.mat, or unredacted/measurements.mat for a simulation run that splits its output), a .mat file holding the variable obs, or a text file in the SBTK format (sbtk.io.read_measurements). Measurement packages for other teams: sbtk.exchange.read_data.

Outputs

Name	Description
`obs`	The measurement set. Fields: `obs`.

Example

```
MATLAB, from the help text
obs = sbtk.io.load_measurements(sbtk.io.latest_run("hera_hlp6", "didymos_T1"));
obs = sbtk.io.load_measurements("their_data.txt");
```

Related functions

Mentioned in the help text

`sbtk.exchange.read_data`, `sbtk.io.latest_run`, `sbtk.io.read_measurements`

Calls

`sbtk.io.read_measurements`

Implementation: `toolbox/+sbtk/+io/load_measurements.m`, line 1.

`sbtk.io.load_truth`

True model from a run folder or a .mat file.

Syntax

```
MATLAB
truth = sbtk.io.load_truth(source)
```

Arguments

Name	Size	Class	Unit	Default	Description
`source`	1 × 1	string		required	a run folder (reads its `truth.mat`, or unredacted/truth.mat for a simulation run that splits its output) or a .mat file holding the variable truth

Outputs

Name	Description
`truth`	The true model. Fields: `truth`.

Example

```
MATLAB, from the help text
truth = sbtk.io.load_truth(sbtk.io.latest_run("hera_hlp6", "didymos_T1"));
```

Related functions

Mentioned in the help text

`sbtk.io.latest_run`

Implementation: `toolbox/+sbtk/+io/load_truth.m`, line 1.

`sbtk.io.new_run`

Create a results folder for one run and start its log.

Syntax

MATLAB

```
out = sbtk.io.new_run(project, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`project`	1 × 1	string		required	Project name: the folder results/<project>/.
`name`	1 × 1	string		required	Run name; the folder is results/<project>/<yyyy-mm-`dd_HHMMSS`>_<name>/.

Outputs

Name	Description
`out`	The results folder. Fields: `out`.

Description

Creates <SBTK>/results/<project>/<yyyy-mm-`dd_HHMMSS`>_<name>/ (figures go to its figures/ subfolder) and records everything printed in the command window to log.txt in it until sbtk.io.finish_run(out). Standard file names inside:

```
  measurements.txt / .mat   truth.txt / .mat   fit.txt / .mat   homogeneous.txt
  comparison.txt / .mat
  settings.json             figures/<name>.png  log.txt
```

`out.project`, `out.name`, `out.stamp`, `out.folder`, `out.figures`

Example

MATLAB, from the help text

```
out = sbtk.io.new_run("hera_hlp6", "didymos_T1");
```

Related functions

Mentioned in the help text

`sbtk.io.finish_run`

Calls

`sbtk.root`

Implementation: `toolbox/+sbtk/+io/new_run.m`, line 1.

`sbtk.io.read_jpl_gravity`

Measurements from a JPL SSD gravity-field file (normalized coefficients + covariance).

Syntax

MATLAB

```
obs = sbtk.io.read_jpl_gravity(file, body, name, origin_km, degree)
```

Arguments

Name	Size	Class	Unit	Default	Description
`file`	1 × 1	string		required	JPL SSD gravity file in MATLAB-assignment form (`REF_RADIUS`, NORMALIZED, NAMES = {...}, VALS = [...], COV = [...]), e.g. the OSIRIS-REx Bennu field Validation: `mustBeFile`.
`body`	1 × 1	struct		required	from sbtk.body.create; `body.R_ref_km` must equal the file's `REF_RADIUS` Fields: `body`.
`name`	1 × 1	string		required	name of the data set
`origin_km`	1 × 3	double	km	required	point the field is expanded about, in shape-model coordinates [km] (the center of mass of the gravity solution) Validation: `mustBeFinite`.
`degree`	scalar	double		required	highest degree to keep (2 .. the file's DEGREE) Validation: `mustBeInteger, mustBeGreaterThanOrEqual(degree, 2)`.

Outputs

Name	Description
`obs`	The measurement set: coefficients of degree 2 … degree with their covariance, `coef_about` = 'origin'. Fields: `obs`.

Description

The file is parsed as text (not executed). Zonal terms `J_l` are converted to `C_l0` = -`J_l` (with the matching sign changes in the covariance); `C_lm`, `S_lm` are kept. The covariance of the kept coefficients is the full submatrix of the file's covariance (GM row and column removed). The mass is GM / G. Degree-1 terms are absent in such files (the expansion is about the center of mass); add them with sbtk.data.add_center_of_mass.

Example

MATLAB, from the help text

```
body = sbtk.body.create("Bennu", sbtk.shape.bundled("bennu"), 1190, 0.290);
file = fullfile(sbtk.root, "data", "gravity", "bennu_osirisrex_jpl_2019.txt");
obs  = sbtk.io.read_jpl_gravity(file, body, "bennu_jpl2019", [0 0 0], 4);
obs  = sbtk.data.add_center_of_mass(obs, [0 0 0], 0.001);
```

Related functions

Mentioned in the help text

sbtk.body.create, sbtk.data.add_center_of_mass, sbtk.root, sbtk.shape.bundled

Calls

sbtk.units.G

Implementation: `toolbox/+sbtk/+io/read_jpl_gravity.m`, line 1.

---

sbtk.io.read_measurements

Measurement set from the SBTK text format.

Syntax

MATLAB

```
obs = sbtk.io.read_measurements(file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`file`	1 × 1	string		required	Path of a text file in the SBTK measurement format. Validation: `mustBeFile`.

Outputs

Name	Description
`obs`	The measurement set (diagonal covariance). Fields: `obs`.

Description

Keywords (see sbtk.io.write_measurements and docs/data_formats.md):

```
body, name, mass_kg, GM_km3_s2, R_ref_km, degree
origin_km          expansion origin, shape-model coordinates [km]
com_minus_cof_km   center-of-mass offset from the center of figure [km]
coef l m C S sigma_C sigma_S        one line per degree and order
inertia label value sigma w(1..6)   one line per moment-of-inertia measurement
```

Other keywords are ignored. This is the entry point for data from other teams: convert their files to this format, or build the measurements from arrays with sbtk.data.from_coefficients.

Example

MATLAB, from the help text

```
obs = sbtk.io.read_measurements("measurements.txt");
```

Related functions

Mentioned in the help text

sbtk.data.from_coefficients, sbtk.io.write_measurements

Called by

sbtk.io.load_measurements

Implementation: `toolbox/+sbtk/+io/read_measurements.m`, line 1.

`sbtk.io.report_comparison`

Save, print and plot a comparison with the true model.

Syntax

MATLAB

```
sbtk.io.report_comparison(out, body, truth, fit, cmp)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`body`	1 × 1	struct		required	The body. Fields: `body`.
`truth`	1 × 1	struct		required	The true model. Fields: `truth`.
`fit`	1 × 1	struct		required	The inversion result. Fields: `fit`.
`cmp`	1 × 1	struct		required	Their comparison from `sbtk.compare.with_truth`. Fields: `cmp`.

Description

Writes to `out.folder`: comparison.txt/.mat, `settings.json`, figures/comparison.png and figures/truth_vs_estimate.png.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `fit`.

```
out = sbtk.io.new_run("my_project", "T1_comparison");
sbtk.io.report_comparison(out, body, truth, fit, sbtk.compare.with_truth(body, truth, fit));
sbtk.io.finish_run(out);
```

Related functions

Mentioned in the help text

`sbtk.compare.with_truth`, `sbtk.io.finish_run`, `sbtk.io.new_run`

Calls

`sbtk.compare.summary`, `sbtk.io.save_comparison`, `sbtk.io.save_figure`, `sbtk.io.save_settings`, `sbtk.plot.comparison`, `sbtk.plot.truth_vs_estimate`

Implementation: `toolbox/+sbtk/+io/report_comparison.m`, line 1.

`sbtk.io.report_inversion`

Save, print and plot an inversion result.

Syntax

MATLAB

```
sbtk.io.report_inversion(out, body, fit, homogeneous)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`body`	1 × 1	struct		required	The body. Fields: `body`.
`fit`	1 × 1	struct		required	from sbtk.invert.run Fields: `fit`.
`homogeneous`	1 × 1	struct		required	from sbtk.invert.homogeneous (the same data fitted by a homogeneous body) Fields: `fit`.

Description

Writes to `out.folder`:

`fit.txt/.mat`	densities with 1-sigma uncertainties and normalized residuals
`homogeneous.txt`	the homogeneous fit
`settings.json`	body, data, components, options
`figures/residuals.png, densities.png, estimate_3d.png, estimate_slices.png`	

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`.

```
out  = sbtk.io.new_run("my_project", "T1_inversion");
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
sbtk.io.report_inversion(out, body, sbtk.invert.run(prob), sbtk.invert.homogeneous(prob));
sbtk.io.finish_run(out);
```

Related functions

Mentioned in the help text

`sbtk.invert.homogeneous`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`, `sbtk.io.finish_run`, `sbtk.io.new_run`

Calls

`sbtk.invert.summary`, `sbtk.io.save_figure`, `sbtk.io.save_fit`, `sbtk.io.save_settings`, `sbtk.io.write_fit`, `sbtk.plot.densities`, `sbtk.plot.interior3d`, `sbtk.plot.residuals`, `sbtk.plot.slices`

Implementation: `toolbox/+sbtk/+io/report_inversion.m`, line 1.

`sbtk.io.report_simulation`

Save, print and plot synthetic measurements and their true model.

Syntax

MATLAB

```
sbtk.io.report_simulation(out, body, obs, truth)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder (or a subfolder from `sbtk.io.subfolder`). Fields: `out`.
`body`	1 × 1	struct		required	The body. Fields: `body`.
`obs`	1 × 1	struct		required	The simulated measurements. Fields: `obs`.
`truth`	1 × 1	struct		required	The true model. Fields: `truth`.

Description

Writes to `out.folder`:

`measurements.txt/.mat`	the measurements (the .txt is the SBTK text format, sbtk.io.read_measurements)
`truth.txt/.mat`	the true model (keep private in blind tests)
`settings.json`	body, degree, uncertainties, components, random field, noise seed
`figures/truth_3d.png, truth_slices.png, measurements.png`	

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `obs`, `truth`.

```
out = sbtk.io.new_run("my_project", "T1");
sbtk.io.report_simulation(sbtk.io.subfolder(out, "unredacted"), body, obs, truth);   % or out itself
sbtk.io.finish_run(out);
```

Related functions

Mentioned in the help text

`sbtk.io.finish_run`, `sbtk.io.new_run`, `sbtk.io.read_measurements`, `sbtk.io.subfolder`

Calls

`sbtk.data.summary`, `sbtk.interior.summary`, `sbtk.io.save_figure`, `sbtk.io.save_measurements`, `sbtk.io.save_settings`, `sbtk.io.save_truth`, `sbtk.plot.interior3d`, `sbtk.plot.measurements`, `sbtk.plot.slices`

Implementation: `toolbox/+sbtk/+io/report_simulation.m`, line 1.

`sbtk.io.save_comparison`

comparison.mat and comparison.txt in the run folder.

Syntax

```
MATLAB
sbtk.io.save_comparison(out, cmp)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`cmp`	1 × 1	struct		required	Comparison from `sbtk.compare.with_truth`. Fields: `cmp`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `cmp`.

```
sbtk.io.save_comparison(out, cmp);          % comparison.mat and comparison.txt in out.folder
```

Related functions

Calls

`sbtk.io.write_comparison`

Called by

`sbtk.io.report_comparison`

Implementation: `toolbox/+sbtk/+io/save_comparison.m`, line 1.

`sbtk.io.save_figure`

Save a figure as figures/<name>.png (200 dpi) in the run folder.

Syntax

```
MATLAB
file = sbtk.io.save_figure(out, fig, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder; the file goes to `out.figures`. Fields: `out`.
`fig`	1 × 1	matlab.ui.Figure		required	Figure handle.
`name`	1 × 1	string		required	File name without extension.

Outputs

Name	Description
`file`	Path of the PNG file written.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `fit`.

```
sbtk.io.save_figure(out, sbtk.plot.residuals(fit), "residuals");   % out.folder/figures/residuals.png
```

Related functions

Mentioned in the help text

`sbtk.plot.residuals`

Called by

`sbtk.io.report_comparison`, `sbtk.io.report_inversion`, `sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+io/save_figure.m`, line 1.

`sbtk.io.save_fit`

`fit.mat` and `fit.txt` in the run folder.

Syntax

```
MATLAB
sbtk.io.save_fit(out, fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`fit`	1 × 1	struct		required	Inversion result. Fields: `fit`.

Example

```
MATLAB fragment from the help text. Uses variables defined by earlier calls: out, fit.
sbtk.io.save_fit(out, fit);              % fit.mat and fit.txt in out.folder
```

Related functions

Calls

`sbtk.io.write_fit`

Called by

`sbtk.io.report_inversion`

Implementation: `toolbox/+sbtk/+io/save_fit.m`, line 1.

`sbtk.io.save_measurements`

measurements.mat and measurements.txt in the run folder.

Syntax

```
MATLAB
sbtk.io.save_measurements(out, obs)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.

Example

```
MATLAB fragment from the help text. Uses variables defined by earlier calls: out, obs.
sbtk.io.save_measurements(out, obs);        % measurements.mat and measurements.txt in out.folder
```

Related functions

Calls

`sbtk.io.write_measurements`

Called by

`sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+io/save_measurements.m`, line 1.

`sbtk.io.save_settings`

Write the settings of a run to `settings.json` (readable, reproducible).

Syntax

```
MATLAB
sbtk.io.save_settings(out, settings)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`settings`	1 × 1	struct		required	any struct: body name, shape file, components, uncertainties, seeds, inversion options... Matrices and struct arrays are written as JSON.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`.

```
sbtk.io.save_settings(out, struct('noise_seed', 1, 'note', "first try"));
```

Related functions

Calls

`sbtk.version`

Called by

`sbtk.io.report_comparison`, `sbtk.io.report_inversion`, `sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+io/save_settings.m`, line 1.

`sbtk.io.save_truth`

`truth.mat` and `truth.txt` in the run folder.

Syntax

MATLAB

```
sbtk.io.save_truth(out, truth)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`truth`	1 × 1	struct		required	The true model. Fields: `truth`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `truth`.

```
sbtk.io.save_truth(out, truth);            % truth.mat and truth.txt in out.folder
```

Related functions

Calls

`sbtk.io.write_truth`

Called by

`sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+io/save_truth.m`, line 1.

`sbtk.io.subfolder`

A subfolder of a run folder that the save and report functions write to.

Syntax

MATLAB

```
sub = sbtk.io.subfolder(out, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	from sbtk.io.new_run Fields: `out`.
`name`	1 × 1	string		required	Name of the subfolder.

Outputs

Name	Description
`sub`	the same run with `sub.folder` = <run>/<name> and `sub.figures` = <run>/<name>/figures; the log stays in <run>/log.txt Fields: `out`.

Example

keep the true model apart from the data package to send

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `obs`, `truth`.

```
out = sbtk.io.new_run("hera_hlp6", "didymos_T1");
sbtk.io.report_simulation(sbtk.io.subfolder(out, "unredacted"), body, obs, truth);
```

Related functions

Mentioned in the help text

`sbtk.io.new_run`, `sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+io/subfolder.m`, line 1.

`sbtk.io.write_comparison`

Plain-text copy of a comparison with the true model.

Syntax

MATLAB

```
sbtk.io.write_comparison(cmp, file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`cmp`	1 × 1	struct		required	Comparison from `sbtk.compare.with_truth`. Fields: `cmp`.
`file`	1 × 1	string		required	Path of the text file to write.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `cmp`.

```
sbtk.io.write_comparison(cmp, "comparison.txt");
```

Related functions

Called by

`sbtk.io.save_comparison`

Implementation: `toolbox/+sbtk/+io/write_comparison.m`, line 1.

`sbtk.io.write_fit`

Plain-text summary of an inversion result.

Syntax

MATLAB

```
sbtk.io.write_fit(fit, file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`fit`	1 × 1	struct		required	Inversion result. Fields: `fit`.
`file`	1 × 1	string		required	Path of the text file to write.

Description

Lines starting with '#' describe the lines below them. Example:

```
  chi2 26.742   dof 24.0   chi2_reduced 1.114   p_value 0.316
  density core 453.423 268.160 3161.834 205.147
  residual C20 -3.822967e-02 -3.823088e-02 1.000000e-04 -0.012
```

Related functions

Called by

`sbtk.io.report_inversion`, `sbtk.io.save_fit`

Implementation: `toolbox/+sbtk/+io/write_fit.m`, line 1.

`sbtk.io.write_measurements`

Plain-text copy of a measurement set (read back by sbtk.io.read_measurements).

Syntax

```
MATLAB
sbtk.io.write_measurements(obs, file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`file`	1 × 1	string		required	Path of the text file to write.

Description

One keyword per line, values separated by spaces, '#' lines are comments, so every block can be pasted into a spreadsheet or another code. Only the diagonal of the coefficient covariance is written (the .mat keeps it all). Keywords: body, name, `mass_kg`, `GM_km3_s2`, `R_ref_km`, degree, `origin_km`, `com_minus_cof_km`, coef (l m C S `sigma_C` `sigma_S`), inertia (label value sigma `w_Ixx` `w_Iyy` `w_Izz` `w_Ixy` `w_Ixz` `w_Iyz`), plus informational lines.

Example

```
MATLAB fragment from the help text. Uses variables defined by earlier calls: obs.
sbtk.io.write_measurements(obs, "measurements.txt");
```

Related functions

Calls

`sbtk.units.norm_to_moi`

Called by

`sbtk.io.save_measurements`

Implementation: `toolbox/+sbtk/+io/write_measurements.m`, line 1.

`sbtk.io.write_truth`

Plain-text description of a true model (keep private in blind tests).

Syntax

```
MATLAB
sbtk.io.write_truth(truth, file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`truth`	1 × 1	struct		required	The true model. Fields: `truth`.
`file`	1 × 1	string		required	Path of the text file to write.

Example

```
MATLAB fragment from the help text. Uses variables defined by earlier calls: truth.
sbtk.io.write_truth(truth, "truth.txt");
```

Related functions

Calls

`sbtk.version`

Called by

`sbtk.io.save_truth`

Implementation: `toolbox/+sbtk/+io/write_truth.m`, line 1.

`sbtk.plot`: Figures

Function	Purpose
`anomaly`	The measured anomaly coefficient by coefficient, with the anomalies of models and how well they match it.
`anomaly_map`	Each degree of the anomaly as a map: where mass is added and where it is taken away.
`coefficient_comparison`	The measured anomaly C - `C_CD` coefficient by coefficient with +-1 sigma, the solutions' anomalies, and their residuals.
`coefficient_residuals`	Map of normalized residuals: one row per fit, one column per measurement, fitted ones outlined.
`comparison`	Estimate compared with the true model: R2 and each component's density.
`contributions`	How much of each coefficient's anomaly each component of a model makes.
`degree_ratio`	Each degree's change from the homogeneous body: scaling (radial structure) and the rest.
`densities`	Estimated density of the background and each component with its 1-sigma uncertainty.
`difference_from_homogeneous`	Gravity coefficients minus those of the homogeneous body, coefficient by coefficient.
`interior3d`	See-through 3D view of the body and every component of a model.
`measurements`	What the data contain: gravity signal and noise per degree, and the inertia rows.
`point_masses`	The mass anomalies of a point-mass solution on two cuts through the center of figure.
`residuals`	Normalized residuals of a fit, (predicted - measured)/sigma, per measurement.
`search_masses`	Where the measurements ask for compact masses, and the chi-square as masses are added.
`search_profiles`	The chi-square profile of each searched geometry parameter.
`shape`	The body's shape model with its center of figure and axes.
`significance`	Each measurement with its +-1 sigma interval against the constant-density (homogeneous) value.
`slices`	Absolute density on the equatorial plane and a meridional plane.
`solution_cuts`	Several solutions side by side on the same cuts and color scale, and their density in the same cells.
`truth_vs_estimate`	Truth, estimate and their difference on two planes.

`sbtk.plot.anomaly`

The measured anomaly coefficient by coefficient, with the anomalies of models and how well they match it.

Syntax

MATLAB

```
fig = sbtk.plot.anomaly(an, models, names)
```

Arguments

Name	Size	Class	Unit	Default	Description
`an`	1 × 1	struct		required	the measurements' anomaly (sbtk.interpret.anomaly of a measurement set) Fields: `an`.
`models`	1 × any	cell		required	cell array of model anomalies (sbtk.interpret.anomaly of a fit or a true model, same degree), or {} for the measurements alone
`names`				required	one name per model Validation: `mustBeText`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Top: each coefficient's anomaly (field minus homogeneous body, about the center of figure) divided by its 1-sigma, so that every degree can be read on one axis: black, the measurements with their +-1 sigma; colors, the models. A model fitted to some of the coefficients (a fit, an.fitted_labels) has filled markers on those and open markers on the others, which it predicts. Bottom: (model - measurement)/sigma; the gray band is +-1, the dashed lines +-3. Both axes are symmetric logarithmic, sign(v) log10(1 + |v|): linear near 0, logarithmic beyond a few sigma. Zonal coefficients are shown as `J_l` = -`C_l0` (the sign of their anomaly flips with them); degree 1 is the center-of-mass offset (C11, S11, C10 = x, y, z / (sqrt(3) `R_ref`)). Vertical lines separate the degrees.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`, `body`, `fit`.

```
an  = sbtk.interpret.anomaly(obs, body, 5);
fig = sbtk.plot.anomaly(an, {sbtk.interpret.anomaly(fit, body, 5)}, "core + torus");
```

Related functions

Mentioned in the help text

`sbtk.interpret.anomaly`

Implementation: `toolbox/+sbtk/+plot/anomaly.m`, line 1.

`sbtk.plot.anomaly_map`

Each degree of the anomaly as a map: where mass is added and where it is taken away.

Syntax

MATLAB

```
fig = sbtk.plot.anomaly_map(ge)
```

Arguments

Name	Size	Class	Unit	Default	Description
`ge`	1 × 1	struct		required	from sbtk.interpret.geometry Fields: `ge`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

One map per degree from 2: the anomaly C - `C_CD` of that degree as a pattern on the sphere (latitude and longitude from the center of figure, in the axes of the shape model). Red where a mass added near the surface in that direction would make the pattern, blue where a mass taken away would; each map on its own color scale symmetric about 0. + marks the strongest excess, x the strongest deficit; the star is the direction of the center-of-mass offset (degree 1). The same direction of excess at every degree points to one compact mass; extremes along the equator and at the poles in the even degrees to a belt, a ring or polar caps.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`, `body`.

```
ge  = sbtk.interpret.geometry(sbtk.interpret.anomaly(obs, body, 5));
fig = sbtk.plot.anomaly_map(ge);
```

Related functions

Mentioned in the help text

`sbtk.interpret.anomaly`, `sbtk.interpret.geometry`

Implementation: `toolbox/+sbtk/+plot/anomaly_map.m`, line 1.

`sbtk.plot.coefficient_comparison`

The measured anomaly C - `C_CD` coefficient by coefficient with +-1 sigma, the solutions' anomalies, and their residuals.

Syntax

MATLAB

```
fig = sbtk.plot.coefficient_comparison(an, models, moi, names, max_degree, style)
```

Arguments

Name	Size	Class	Unit	Default	Description
`an`	1 × 1	struct		required	the measurements' anomaly (sbtk.interpret.anomaly of a measurement set) Fields: `an`.
`models`	1 × any	cell		required	1 x k cell of the solutions' anomalies (sbtk.interpret.anomaly of a fit or of the model of a point-mass solution), or {} for the measurements alone
`moi`	any × 3	double		required	k x 3 each solution's (predicted - measured)/sigma of the inertia rows of an (an.moi_labels), e.g. from the fit's normalized residuals; zeros(0, 3) without solutions or without inertia rows
`names`				required	one name per solution Validation: `mustBeText`.
`max_degree`	scalar	double		required	the highest degree shown Validation: `mustBeInteger, mustBePositive`.

Name	Size	Class	Unit	Default	Description
`style`	1 × any	double		required	1 x k the color and marker of each solution, as a number from 1 to 6 (orange circle, blue square, green diamond, purple triangle, brown triangle, pink pentagram), so that a kind of solution looks the same in every figure Validation: `mustBeInteger, mustBeInRange(style, 1, 6)`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

One column per degree (its width proportional to its number of coefficients) and one for the moments of inertia, if the measurements have them. Top: the measured coefficient minus that of the constant-density body, with +-1 sigma, in the units of the coefficients (zonal terms as `J_l` = -`C_l0`, degree 1 as C10, C11, S11); each solution's difference from the constant-density body as a marker; 0 is the constant-density body. Each degree has its own linear scale, its power of ten in its title. Bottom (with solutions): (solution - measurement)/sigma, the band +-1, the dashed lines +-3; values beyond +-6 drawn at the edge as triangles. The moments are I/(M `R_ref`^2) about the point the measurements are about.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`, `body`, `fit`.

```
an  = sbtk.interpret.anomaly(obs, body, 4);
ms  = {sbtk.interpret.anomaly(fit, body, 4)};
j   = ismember(fit.labels, an.moi_labels);
fig = sbtk.plot.coefficient_comparison(an, ms, fit.normalized_residual(j)', "core and torus", 4, 1);
```

Related functions

Mentioned in the help text

`sbtk.interpret.anomaly`

Implementation: `toolbox/+sbtk/+plot/coefficient_comparison.m`, line 1.

`sbtk.plot.coefficient_residuals`

Map of normalized residuals: one row per fit, one column per measurement, fitted ones outlined.

Syntax

MATLAB

```
fig = sbtk.plot.coefficient_residuals(preds, names)
```

Arguments

Name	Size	Class	Unit	Default	Description
`preds`	1 × any	cell		required	cell array of sbtk.invert.predict results for the same measurements (fits to different subsets of them, or with different models)
`names`				required	one name per row Validation: `mustBeText`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Each cell is (predicted - measured)/sigma of one measurement for one fit, colored on a symmetric logarithmic scale, sign(v) log10(1 + |v|), saturated at 100 (blue: predicted too small, red: too large; nearly white within 1 sigma). The measurements a fit was fitted to are outlined, so one row shows how well a fit matches its own coefficients (outlined cells) and how well it predicts the others. Right: chi-square per measurement of the fitted rows and of the others (with their covariance). Zonal coefficients are labeled `J_l`; their residual is that of `C_l0` (the same size, opposite sign of `J_l`).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`, `opts`, `preds`.

```
sets = {["C10" "C11" "S11" "C20"], ["C10" "C11" "S11" "C20" "C30"]};
for k = 1:2
    fit = sbtk.invert.solve(body, components, sbtk.data.select_coefficients(obs, sets{k}), opts);
    preds{k} = sbtk.invert.predict(fit, body, obs);
end
fig = sbtk.plot.coefficient_residuals(preds, ["degree 1 + J2", "+ J3"]);
```

Related functions

Mentioned in the help text

`sbtk.data.select_coefficients`, `sbtk.invert.predict`, `sbtk.invert.solve`

Implementation: `toolbox/+sbtk/+plot/coefficient_residuals.m`, line 1.

`sbtk.plot.comparison`

Estimate compared with the true model: R2 and each component's density.

Syntax

MATLAB

```
fig = sbtk.plot.comparison(cmp)          % cmp from sbtk.compare.with_truth
```

Arguments

Name	Size	Class	Unit	Default	Description
`cmp`	1 × 1	struct		required	Comparison from `sbtk.compare.with_truth`. Fields: `cmp`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Left: R2 of the estimated density (fraction of the variance of the true density explained) next to `R2_max`, the largest R2 the same components can reach. Right: each component's estimated density with +-1 sigma (bar) and the true density (cross).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `fit`.

```
fig = sbtk.plot.comparison(sbtk.compare.with_truth(body, truth, fit));
```

Related functions

Mentioned in the help text

`sbtk.compare.with_truth`

Called by

`sbtk.io.report_comparison`

Implementation: `toolbox/+sbtk/+plot/comparison.m`, line 1.

`sbtk.plot.contributions`

How much of each coefficient's anomaly each component of a model makes.

Syntax

MATLAB

```
fig = sbtk.plot.contributions(an, fit, body, max_degree)
```

Arguments

Name	Size	Class	Unit	Default	Description
`an`	1 × 1	struct		required	the measurements' anomaly (sbtk.interpret.anomaly of a measurement set) Fields: `an`.
`fit`	1 × 1	struct		required	a fitted model with a few components (sbtk.invert.run) Fields: `fit`.
`body`	1 × 1	struct		required	the body Fields: `body`.
`max_degree`	scalar	double		required	the highest degree shown Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

The anomaly of a model, its coefficients minus those of the constant-density body, is the sum over its components of their relative density times their signature (sbtk.interpret.signatures: the coefficients of the component with the mass it moves taken evenly from the rest of the body). For every coefficient of degrees 1 to `max_degree` a bar is split into these parts, positive ones stacked up and negative ones down; the black dot with its bar is the measured anomaly +-1 sigma, the white diamond the model's sum. Which component makes a coefficient, and with which sign, is the reason it sits where it does: a dense core off the center of figure makes degree 1 and lowers J2, a light torus about the z-axis raises J2 and changes J4, and so on. Zonal terms are shown as `J_l` = -`C_l0`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`, `body`, `fit`.

```
fig = sbtk.plot.contributions(sbtk.interpret.anomaly(obs, body, 4), fit, body, 4);
```

Related functions

Mentioned in the help text

`sbtk.interpret.anomaly`, `sbtk.interpret.signatures`, `sbtk.invert.run`

Implementation: `toolbox/+sbtk/+plot/contributions.m`, line 1.

`sbtk.plot.degree_ratio`

Each degree's change from the homogeneous body: scaling (radial structure) and the rest.

Syntax

MATLAB

```
fig = sbtk.plot.degree_ratio(an, models, names)
```

Arguments

Name	Size	Class	Unit	Default	Description
`an`	1 × 1	struct		required	an anomaly (sbtk.interpret.anomaly): measurements, or a model Fields: `an`.
`models`	1 × any	cell		required	cell array of further anomalies (fits, true models), or {}
`names`				required	one name per model Validation: `mustBeText`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Left: 100 (`k_l` - 1), the change of degree l as a scaled homogeneous field, in % of the homogeneous field (an.ratio; measurements with +-1 sigma, cut at the range of the values). A compact central mass excess q M changes every degree by -100 q %; a light surface layer of thickness t changes degree l by about 100 (drho/rho) l t/r % (r the mean radius), more at higher degree; a torus or anything not centrally symmetric adds to some degrees only. Right: the RMS of the rest of each degree (an.residual_rms), the part that is not a scaled homogeneous field, with the RMS 1-sigma of the measurements (dashed): above it, the degree contains structure that is not centrally symmetric.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`, `body`, `truth`.

```
fig = sbtk.plot.degree_ratio(sbtk.interpret.anomaly(obs, body, 5), ...
                             {sbtk.interpret.anomaly(truth, body, 5)}, "true model");
```

Related functions

Mentioned in the help text

`sbtk.interpret.anomaly`

Implementation: `toolbox/+sbtk/+plot/degree_ratio.m`, line 1.

`sbtk.plot.densities`

Estimated density of the background and each component with its 1-sigma uncertainty.

Syntax

```
MATLAB
fig = sbtk.plot.densities(fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`fit`	1 × 1	struct		required	Inversion result. Fields: `fit`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

One row per component (background first): the density [kg/m^3] and a +-1 sigma bar. The dashed line is the background density. Gray: components whose relative density is not resolved (|relative density| < 1 sigma).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `fit`.

```
fig = sbtk.plot.densities(fit);
```

Related functions

Called by

`sbtk.exchange.write_solution`, `sbtk.io.report_inversion`

Implementation: `toolbox/+sbtk/+plot/densities.m`, line 1.

`sbtk.plot.difference_from_homogeneous`

Gravity coefficients minus those of the homogeneous body, coefficient by coefficient.

Syntax

```
MATLAB
[fig, differences] = sbtk.plot.difference_from_homogeneous(body, sets, names, scale, frame)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	the body: its shape model gives the homogeneous body (density M/V everywhere) Fields: `body`.
`sets`	1 × any	cell		required	cell array of coefficient sets, each one of a measurement set exact values (sbtk.data.simulate), measurements (sbtk.data.add_noise), a package (sbtk.exchange.read_data), ... a true model truth from sbtk.data.simulate an inversion result fit from sbtk.invert.run: the field of the fitted model
`names`				required	one legend entry per set, e.g. ["true model" "estimate"] Validation: `mustBeText`.
`scale`	1 × 1	string		required	"linear": the signed difference, with +-3 sigma bars for sets that have uncertainties (measurement sets with `coef_cov`); "log": the absolute difference, with 3 sigma drawn as a separate line Validation: `mustBeMember(scale, ["linear", "log"])`.
`frame`	1 × 1	string		required	the axes both fields are expressed in (sbtk.gravity.center_of_mass_frame): "shape": the axes of the shape-model frame; "spin": z along the axis of the largest moment of the set, x the shape-model x-axis projected onto the plane normal to it, as in body-fixed radio-science fields (the OSIRIS-REx field of Bennu). The set has C21 = S21 = 0, so the homogeneous body's C21, S21 show how far its axis of the largest moment is tilted from the set's; "principal": the principal axes of inertia of the set (x the axis of the smallest moment, z of the largest). The set has C21 = S21 = S22 = 0, so the homogeneous body's C21, S21, S22 show how its principal axes are tilted and turned. Validation: `mustBeMember(frame, ["shape", "spin", "principal"])`.

Outputs

Name	Description
`fig`	the figure
`differences`	struct array, one per set: name, labels (as plotted: C10, C11, S11, J2, ...), difference, sigma (1-sigma; NaN where the set has none), `axes_deg` ([tilt turn]: angle between the axes of the largest moment of the set and of the homogeneous body, and angle about that axis between their axes of the smallest moment, deg)

Description

Both fields are about their own center of mass, as radio-science fields are:

```
  degrees >= 2   the set about its center of mass minus the homogeneous body about its
                 center of mass (the center of figure)
  degree 1       the set's center of mass minus the center of figure, as C11, S11, C10 =
                 (x, y, z)/(sqrt(3) R_ref) in the axes of the plot: about its own center of
                 mass each field's degree 1 is zero, so this shows where the interior moves
                 the center of mass instead
```

sbtk.gravity.center_of_mass_frame moves each set to its center of mass and into the axes (it states how the center of mass of each kind of set is found); the homogeneous field is rotated into the same axes. Differences are fully normalized, in SBTK order; zonal coefficients are shown as `J_l` = -`C_l0`, so their difference has the opposite sign of the `C_l0` difference. The reference radius is the set's (`body.R_ref_km` for truth and fit). One geometry is built per set (about 1 s for 49,152 facets at degree 4, about a minute for 3 million facets).

The plot shows how far the gravity field of an interior departs from that of a uniform density, and, against 3 sigma, which departures measurements resolve. The three frames show the same information: the spin and principal axes move the difference in orientation of the principal axes into the homogeneous body's degree-2 terms.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`, `unc`, `components`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");
sbtk.plot.difference_from_homogeneous(body, {truth}, "true model", "linear", "principal");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
fit = sbtk.invert.solve(body, components, obs);
sbtk.plot.difference_from_homogeneous(body, {obs, fit}, ["measured" "estimate"], "log", "shape");
```

Related functions

Mentioned in the help text

sbtk.data.add_noise, sbtk.data.set_uncertainties, sbtk.data.simulate, sbtk.exchange.read_data, sbtk.gravity.center_of_mass_frame, sbtk.invert.run, sbtk.invert.solve

Calls

sbtk.gravity.center_of_mass_frame, sbtk.gravity.forward, sbtk.gravity.geometry, sbtk.interior.create, sbtk.interior.no_components

Implementation: `toolbox/+sbtk/+plot/difference_from_homogeneous.m`, line 1.

---

sbtk.plot.interior3d

See-through 3D view of the body and every component of a model.

Syntax

MATLAB

```
fig = sbtk.plot.interior3d(body, model, title_str)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`model`	1 × 1	struct		required	a truth or a fit (anything with components and `origin_km`)
`title_str`	1 × 1	string		required	Title of the figure.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Two views, oblique and looking down the z-axis. The surface is drawn translucent; spheres, ellipsoids and tori where they lie inside the body, layers by their lower boundary, sectors as the solid between their latitudes, longitudes and radial fractions (measured from `origin_km`, as the model does). Color is the relative density (blue lighter, red denser than the background), on one scale for the model; sectors and layers are the more opaque the larger their relative density compared with the strongest of them, so that the strongest stand out of a grid. Spheres, ellipsoids and tori carry their names.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `fit`.

```
fig = sbtk.plot.interior3d(body, truth, "true model");
fig = sbtk.plot.interior3d(body, fit, "estimate");
```

Related functions

Calls

sbtk.gravity.geometry

Called by

`sbtk.exchange.write_model`, `sbtk.exchange.write_solution`, `sbtk.io.report_inversion`, `sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+plot/interior3d.m`, line 1.

`sbtk.plot.measurements`

What the data contain: gravity signal and noise per degree, and the inertia rows.

Syntax

MATLAB

```
fig = sbtk.plot.measurements(obs)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Left: root-mean-square size of the coefficients of each degree (dots) and of their 1-sigma uncertainties (line). Degrees where the dots sit well above the line carry information about the interior; where they meet, the data are noise (degree 1 is near zero about the center of mass). Right: each moment-of-inertia measurement with its 1-sigma ('libration' is the row (B - A) - ratio*Izz, measured as 0).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
fig = sbtk.plot.measurements(obs);
```

Related functions

Called by

`sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+plot/measurements.m`, line 1.

`sbtk.plot.point_masses`

The mass anomalies of a point-mass solution on two cuts through the center of figure.

Syntax

MATLAB

```
fig = sbtk.plot.point_masses(body, pm, spacing_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`pm`	1 × 1	struct		required	from sbtk.interpret.point_masses Fields: `pm`.
`spacing_km`	scalar	double	km	required	the grid spacing of the points (sbtk.interior.sphere_grid) [km] Validation: `mustBePositive`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Each point within half a spacing of the plane z = 0 (left) or y = 0 (right) through the center of figure is a square colored by its mass anomaly spread over its grid cell, mass/spacing^3 in kg/m^3 (red: more mass than the homogeneous body, blue: less), on one color scale symmetric about 0. The crosses are the mass-weighted centroids of the positive (+) and negative (x) parts, projected onto each plane; the line is the outline of the body.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `prob`.

```
fig = sbtk.plot.point_masses(body, sbtk.interpret.point_masses(prob, "sensitivity", "aic"), 0.040);
```

Related functions

Mentioned in the help text

`sbtk.interior.sphere_grid`, `sbtk.interpret.point_masses`

Calls

`sbtk.gravity.geometry`

Implementation: `toolbox/+sbtk/+plot/point_masses.m`, line 1.

`sbtk.plot.residuals`

Normalized residuals of a fit, (predicted - measured)/sigma, per measurement.

Syntax

MATLAB

```
fig = sbtk.plot.residuals(fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`fit`	1 × 1	struct		required	Inversion result. Fields: `fit`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Gray band: +-1 sigma; dashed: +-2 sigma. Blue within 1 sigma, amber 1-2, red beyond 2. For a correct model about 68% of the residuals fall within +-1 sigma; systematic residuals in one group of coefficients (e.g. all of degree 3) indicate structure the model does not contain.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `fit`.

```
fig = sbtk.plot.residuals(fit);
```

Related functions

Called by

`sbtk.exchange.write_solution`, `sbtk.io.report_inversion`

Implementation: `toolbox/+sbtk/+plot/residuals.m`, line 1.

`sbtk.plot.search_masses`

Where the measurements ask for compact masses, and the chi-square as masses are added.

Syntax

MATLAB

```
fig = sbtk.plot.search_masses(ms, body)
```

Arguments

Name	Size	Class	Unit	Default	Description
`ms`	1 × 1	struct		required	from sbtk.search.add_masses (with `max_count` >= 1) Fields: `ms`.
`body`	1 × 1	struct		required	the body Fields: `body`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Left and middle: the chi-square reduction of one mass at each candidate position of the first step, on the planes z = 0 and y = 0 through the center of figure (candidates within half a grid spacing of the plane), red where the mass would be denser than the model, blue where lighter; the masses kept are marked (+ dense, x light) at their fitted positions, projected onto each plane. Right: chi-square of the model with 0, 1, 2, ... masses (filled: kept; open: the best tried and rejected) and its 95% point for the degrees of freedom of each fit (dashed): below it the model is consistent.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `prob`, `fit`, `cand`, `body`.

```
ms  = sbtk.search.add_masses(prob, fit, cand, 4, 0.05, 3, 0);
fig = sbtk.plot.search_masses(ms, body);
```

Related functions

Mentioned in the help text

`sbtk.search.add_masses`

Calls

`sbtk.gravity.geometry`

Implementation: `toolbox/+sbtk/+plot/search_masses.m`, line 1.

`sbtk.plot.search_profiles`

The chi-square profile of each searched geometry parameter.

Syntax

MATLAB

```
fig = sbtk.plot.search_profiles(gr)
```

Arguments

Name	Size	Class	Unit	Default	Description
`gr`	1 × 1	struct		required	from sbtk.search.grid Fields: `gr`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

One panel per parameter: at each of its values, the lowest chi-square over all the other parameters (densities fitted without bounds), on a logarithmic axis (values that share a coordinate, such as axis directions of one tilt, as separate points). A narrow minimum is a parameter the measurements determine; a flat profile is one they do not, or one that other parameters can make up for. The dashed line is the number of measurements, about the chi-square a correct model has; the dotted line marks the value of the best combination fitted with the density bounds.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `prob`, `P`.

```
gr  = sbtk.search.grid(prob, P, 5);
fig = sbtk.plot.search_profiles(gr);
```

Related functions

Mentioned in the help text

`sbtk.search.grid`

Implementation: `toolbox/+sbtk/+plot/search_profiles.m`, line 1.

`sbtk.plot.shape`

The body's shape model with its center of figure and axes.

Syntax

MATLAB

```
fig = sbtk.plot.shape(body)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Example

MATLAB, from the help text

```
fig = sbtk.plot.shape(sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43));
```

Related functions

Mentioned in the help text

`sbtk.body.from_mass`, `sbtk.shape.bundled`

Implementation: `toolbox/+sbtk/+plot/shape.m`, line 1.

`sbtk.plot.significance`

Each measurement with its +-1 sigma interval against the constant-density (homogeneous) value.

Syntax

MATLAB

```
fig = sbtk.plot.significance(an)       % an from sbtk.interpret.anomaly of a measurement set
```

Arguments

Name	Size	Class	Unit	Default	Description
`an`	1 × 1	struct		required	Anomaly of a measurement set from `sbtk.interpret.anomaly`. Fields: `an`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

For every gravity coefficient (about the center of figure) and every inertia row: (measured - homogeneous)/sigma with its +-1 sigma interval. The homogeneous body, the same shape model with the bulk density everywhere, is the line at 0: an interval that does not reach it is a measurement that differs from constant density at 1 sigma. Gray: within 1 sigma of constant density; amber: 1 to 3 sigma; red: more than 3 sigma. The axis is symmetric logarithmic, sign(v) log10(1 + |v|). Zonal coefficients are shown as `J_l` = -`C_l0`; degree 1 is the center-of-mass offset from the center of figure.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`, `body`.

```
fig = sbtk.plot.significance(sbtk.interpret.anomaly(obs, body, 5));
```

Related functions

Mentioned in the help text

`sbtk.interpret.anomaly`

Implementation: `toolbox/+sbtk/+plot/significance.m`, line 1.

`sbtk.plot.slices`

Absolute density on the equatorial plane and a meridional plane.

Syntax

MATLAB

```
fig = sbtk.plot.slices(body, models, titles)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`models`	1 × any	cell		required	cell array of models (truth from sbtk.data.simulate, fit from sbtk.invert.run): one row of panels each, one shared color scale
`titles`	1 × any	cell		required	cell array, one label per model

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Left: the plane z = 0 (looking down the spin axis). Right: the plane y = 0. Both pass through the first model's `origin_km` (its center of mass).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`.

```
fig = sbtk.plot.slices(body, {truth}, {'truth'});
```

Related functions

Mentioned in the help text

`sbtk.data.simulate`, `sbtk.invert.run`

Calls

`sbtk.gravity.geometry`

Called by

`sbtk.exchange.write_model`, `sbtk.exchange.write_solution`, `sbtk.io.report_inversion`, `sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+plot/slices.m`, line 1.

`sbtk.plot.solution_cuts`

Several solutions side by side on the same cuts and color scale, and their density in the same cells.

Syntax

MATLAB

```
[fig, cs] = sbtk.plot.solution_cuts(body, models, names, spacing_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	the body Fields: `body`.
`models`	1 × any	cell		required	1 x k cell of solutions: fits (sbtk.invert.run), point-mass models (pm.model of sbtk.interpret.point_masses), true models
`names`				required	one name per solution Validation: `mustBeText`.
`spacing_km`	scalar	double	km	required	the cell size [km]; cells on the grid of sbtk.interior.sphere_grid Validation: `mustBePositive`.

Outputs

Name	Description
`fig`	Handle of the new figure.
`cs`	Each solution's density minus the bulk density in the same cells, the cells' volumes inside the body, and the correlation of each pair of solutions.

Description

One row per solution, the planes z = 0 (left) and y = 0 (right) through the center of figure: its density relative to the bulk density M/V (red: denser, blue: lighter), on one color scale for all, at the median of the solutions' ranges (the strongest saturate). Components and sectors are drawn with their own boundaries. A sphere smaller than half a cell inside the body (a point mass of a grid) has no shape at this resolution: its mass is spread over the cell of its center, so point masses show as cells.

To compare the solutions in numbers, each is also averaged over the same cubic cells (the small spheres in the cell of their center, everything else sampled at 27 points per cell inside the body):

`cs.centers_km`	K x 3 cell centers
`cs.volume_m3`	K x 1 the volume of each cell inside the body
`cs.anomaly_kgm3`	K x k each solution's cell density minus the bulk density
`cs.correlation`	k x k correlation of the cells of each pair of solutions

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `fit1`, `fit2`, `pm`.

```
fig = sbtk.plot.solution_cuts(body, {fit1, fit2, pm.model}, ["core", "sectors", "point masses"], 0.040);
```

Related functions

Mentioned in the help text

`sbtk.interior.sphere_grid`, `sbtk.interpret.point_masses`, `sbtk.invert.run`

Calls

`sbtk.gravity.geometry`, `sbtk.interior.density_at`, `sbtk.interior.sphere_grid`

Implementation: `toolbox/+sbtk/+plot/solution_cuts.m`, line 1.

`sbtk.plot.truth_vs_estimate`

Truth, estimate and their difference on two planes.

Syntax

MATLAB

```
fig = sbtk.plot.truth_vs_estimate(body, truth, fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`truth`	1 × 1	struct		required	The true model. Fields: `truth`.
`fit`	1 × 1	struct		required	The inversion result. Fields: `fit`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Rows: truth, estimate (same density color scale), and estimate minus truth (blue = estimate too light, red = too dense). Left: plane z = 0; right: plane y = 0, both through the truth's center of mass.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `fit`.

```
fig = sbtk.plot.truth_vs_estimate(body, truth, fit);
```

Related functions

Calls

`sbtk.gravity.geometry`

Called by

`sbtk.io.report_comparison`

Implementation: `toolbox/+sbtk/+plot/truth_vs_estimate.m`, line 1.

Data structures

SBTK passes MATLAB structures between its functions. Each structure has a fixed set of fields, listed below; the field names carry their units (see Units). Sizes refer to the Didymos examples (49,152 facets, degree 4) unless stated: *n* is the number of measurements, *nc* the number of components, *nf* the number of facets, *L* the highest degree.

A *model* is any structure with the fields `rho_bg_kgm3`, `components`, `random_field` and `origin_km`: a true model (`truth`) or an inversion result (`fit`). Functions that evaluate, plot or compare densities accept either.

Structure	Contents	Created by
`shape`	Shape model	`sbtk.shape.load`, `sbtk.shape.bundled`, `sbtk.shape.from_arrays`, `sbtk.shape.ellipsoid`, `sbtk.shape.reduce`, `sbtk.shape.reduce_spc`
`props`	Shape properties	`sbtk.shape.properties`
`body`	Body	`sbtk.body.create`, `sbtk.body.from_mass`
`component`	Interior component	`sbtk.interior.sphere`, `sbtk.interior.ellipsoid`, `sbtk.interior.torus`, `sbtk.interior.layer`, `sbtk.interior.sector`, `sbtk.interior.sector_grid`
`interior`	Interior model	`sbtk.interior.create`
`random_field`	Gaussian random density field	`sbtk.interior.random_field`, `sbtk.interior.no_random_field`
`unc`	Uncertainties for a simulation	`sbtk.data.uncertainties`
`obs`	Measurement set	`sbtk.data.simulate`, `sbtk.data.from_coefficients`, `sbtk.exchange.read_data`, `sbtk.exchange.read_shadr`, `sbtk.io.read_jpl_gravity`, `sbtk.io.read_measurements`, `sbtk.io.load_measurements`
`moi`	Moment-of-inertia measurement row	`sbtk.data.add_moi`, `sbtk.data.add_moi_row`, `sbtk.data.add_libration`, `sbtk.data.simulate`
`truth`	True model of a simulation	`sbtk.data.simulate`, `sbtk.io.load_truth`
`geo`	Facet-cone geometry	`sbtk.gravity.geometry`
`integrals`	Integrals of a component	`sbtk.gravity.component_integrals`, `sbtk.gravity.random_field_integrals`
`fm`	Forward-model result	`sbtk.gravity.forward`
`cf`	Field about its center of mass	`sbtk.gravity.center_of_mass_frame`
`opts`	Inversion settings	`sbtk.invert.options`
`prob`	Prepared inversion	`sbtk.invert.setup`, `sbtk.invert.replace_data`
`fit`	Inversion result	`sbtk.invert.run`, `sbtk.invert.solve`, `sbtk.invert.homogeneous`, `sbtk.io.load_fit`
`pred`	Predictions of a fitted model	`sbtk.invert.predict`
`an`	Anomaly of the gravity coefficients	`sbtk.interpret.anomaly`
`cm`	Central mass	`sbtk.interpret.central_mass`
`sg`	Signatures of candidate components	`sbtk.interpret.signatures`
`sel`	Stepwise selection of components	`sbtk.interpret.select`
`ge`	Geometric reading of the anomaly	`sbtk.interpret.geometry`
`pm`	Point masses fitted to the measurements	`sbtk.interpret.point_masses`
`sc`	Chi-square along positions of a component	`sbtk.invert.scan_position`
`pa`	A searched geometry parameter	`sbtk.search.parameter`
`gr`	Grid search of the geometry	`sbtk.search.grid`
`rf`	Gauss-Newton from several starts	`sbtk.search.refine`
`ms`	Compact masses added to a model	`sbtk.search.add_masses`
`sr`	Search of the geometry	`sbtk.search.run`
`cmp`	Comparison with the true model	`sbtk.compare.with_truth`, `sbtk.compare.evaluate`
`ref`	Prepared true model for comparisons	`sbtk.compare.reference`
`grid`	Density grid	`sbtk.exchange.density_grid`, `sbtk.exchange.read_grid`
`gridcmp`	Comparison of two density grids	`sbtk.compare.grids`
`out`	Results folder	`sbtk.io.new_run`, `sbtk.io.subfolder`
`pk`	Coefficient order	`sbtk.internal.sh_packing`
`samples`	Points filling the body	`sbtk.internal.body_samples`
`slices`	Plotting planes	`sbtk.internal.slice_planes`

`shape`: Shape model

Created by `sbtk.shape.load`, `sbtk.shape.bundled`, `sbtk.shape.from_arrays`, `sbtk.shape.ellipsoid`, `sbtk.shape.reduce`, `sbtk.shape.reduce_spc`.

A closed triangular mesh with vertex positions in kilometers.

Fields of `shape` (6)

Field	Size	Unit	Description
`name`	text		File name without extension, or the name given.
`faces`	nf × 3		Vertex indices of each triangle (1-based), ordered so that the normal points outward.
`vertices_km`	nv × 3	km	Vertex positions in the shape-model frame.
`source`	text		Path of the file the shape was read from (`'arrays'` or `'ellipsoid'` for shapes not read from a file). After `sbtk.body.to_frame`, the name of the original file without its path. `sbtk.exchange.write_data` copies the file into a measurement package if the shape model is that file unchanged, unless it is one of SBTK's shape models in `data/shapes/`, which the package names without including it.
`frame_origin_km`	1 × 3	km	Origin of the shape model's frame in the coordinates of `source`: `[0 0 0]`, or as set by `sbtk.body.to_frame` (combined over successive changes of frame).
`frame_axes`	3 × 3		Axes of the shape model's frame (columns) in the coordinates of `source`: `eye(3)`, or as set by `sbtk.body.to_frame`. The vertices are `(source vertices - frame_origin_km)*frame_axes`.

`props`: Shape properties

Created by `sbtk.shape.properties`.

Fields of `props` (8)

Field	Size	Unit	Description
`volume_km3`	scalar	km³	Enclosed volume.
`centroid_km`	1 × 3	km	Centroid of the enclosed volume (center of figure).
`n_facets`	scalar		Number of triangles.
`n_vertices`	scalar		Number of vertices.
`extent_km`	2 × 3	km	Minimum (row 1) and maximum (row 2) vertex coordinate along each axis.
`closed`	logical		True if every edge is shared by exactly two facets that traverse it in opposite directions.
`outward`	logical		True if the facets are ordered outward (positive enclosed volume).
`star_shaped_about_centroid`	logical		True if every facet faces away from the centroid, which the facet-cone integration about the centroid requires.

`body`: Body

Created by `sbtk.body.create`, `sbtk.body.from_mass`.

A shape model with the total mass and the reference radius of the gravity field.

Fields of `body` (8)

Field	Size	Unit	Description
`name`	text		Name of the body, used in printed output and file contents.
`shape`	struct		The `shape`.
`bulk_density_kgm3`	scalar	kg/m³	Mass divided by the shape volume.
`R_ref_km`	scalar	km	Reference radius of the gravity coefficients.
`volume_km3`	scalar	km³	Shape volume.
`mass_kg`	scalar	kg	Total mass.
`GM_km3_s2`	scalar	km³/s²	Gravitational parameter, GM with G from `sbtk.units.G`.
`centroid_km`	1 × 3	km	Center of figure in the shape-model frame.

`component`: Interior component

Created by `sbtk.interior.sphere`, `sbtk.interior.ellipsoid`, `sbtk.interior.torus`, `sbtk.interior.layer`, `sbtk.interior.sector`, `sbtk.interior.sector_grid`.

One region of uniform relative density. Every component has every field, so components of different types can be concatenated into one structure array (`[c1, c2]`); fields that a type does not use are zero. The geometry definitions are in Density distributions.

Fields of `component` (17)

Field	Size	Unit	Description
`name`	text		Name; unique within a model.
`type`	text		`'sphere'`, `'ellipsoid'`, `'torus'`, `'layer'` or `'sector'`.
`drho_kgm3`	scalar	kg/m³	Relative density: density minus the background density. In an inversion it is estimated unless `estimate_density` is false.
`center_km`	1 × 3	km	Center of a sphere or ellipsoid in the shape-model frame; for a torus, the point on its axis from which `height_km` is measured (`[0 0 0]` as made by `sbtk.interior.torus`; it may be set, estimated, or moved by `sbtk.interior.to_frame`).
`radius_km`	scalar	km	Sphere radius.
`semi_axes_km`	1 × 3	km	Ellipsoid semi-axes along its own axes.
`axes`	3 × 3		Columns: the component's own axes in the shape-model frame. Ellipsoid: $R_z(\text{yaw})\,R_y(\text{pitch})\,R_x(\text{roll})$. Torus (about its third axis) and sector (latitude and longitude in these axes): `eye(3)` unless turned by `sbtk.interior.set_axis`, `sbtk.interior.align_with_spin_axis` or `sbtk.interior.to_frame`.
`ring_radius_km`	scalar	km	Torus: radius of the circle at the center of the tube.
`tube_radius_km`	scalar	km	Torus: radius of the tube.
`height_km`	scalar	km	Torus: distance of the plane of the center circle from `center_km` along the torus axis (z for a torus made by `sbtk.interior.torus`).
`depth_km`	1 × 2	km	Layer: top and bottom depth below the surface, measured along rays from the expansion origin.
`frac`	1 × 2		Sector: inner and outer fraction of the distance from the expansion origin to the surface (0 at the origin, 1 at the surface).
`lat_deg`	1 × 2	deg	Sector: latitude range, within −90 to 90.
`lon_deg`	1 × 2	deg	Sector: longitude range, within −360 to 360 and at most 360 wide; ranges beyond ±180 wrap.
`estimate_density`	logical		True (set by the constructors): the inversion estimates `drho_kgm3`. False: the value is held fixed (a prescribed component).
`estimate_position`	logical, 1 or 1 × 3		Spheres, ellipsoids and tori. False (set by the constructors). True: the inversion also estimates `center_km`; a mask `[x y z]` estimates only those coordinates, e.g. `[false false true]` for the height (Estimated geometry).
`estimate_axis`	logical		Ellipsoids and tori. False (set by the constructors). True: the inversion also estimates the orientation: the rotation about each of the component's axes that is not an axis of symmetry. For a torus, two tilts of its axis (the third of `axes`); for an ellipsoid, also its turn about that axis, without the rotation about an axis of symmetry if two semi-axes are equal, and none if all three are.

`interior`: Interior model

Created by `sbtk.interior.create`.

Input of `sbtk.data.simulate` and `sbtk.gravity.forward`. The background density is not part of it: it follows from the mass of the body.

Fields of `interior` (2)

Field	Size	Unit	Description
`components`	1 × nc struct		The `component` array.
`random_field`	struct		The `random_field`; standard deviation 0 means no field.

`random_field`: Gaussian random density field

Created by `sbtk.interior.random_field`, `sbtk.interior.no_random_field`.

A sum of 300 Fourier modes with random wave vectors and phases, $\delta\rho(\mathbf{x}) = a\sum_{j=1}^{300}\cos(\mathbf{k}_j\cdot\mathbf{x}+\varphi_j)$ with $a = \sigma\sqrt{2\,/\,300}$ and $\mathbf{k}_j$ drawn from a normal distribution with standard deviation $1\,/\,L$ per component, which gives the correlation $\exp(-\,|\Delta\mathbf{x}|^2\,/\,(2L^2))$. The field is defined at every point and does not depend on the expansion origin.

Fields of `random_field` (6)

Field	Size	Unit	Description
`sigma_kgm3`	scalar	kg/m³	Standard deviation σ of the density perturbation; 0 means no field.
`corr_length_km`	scalar	km	Correlation length L.
`seed`	scalar		Seed of the random number stream; the same seed gives the same field.
`k_per_km`	300 × 3	1/km	Wave vectors $\mathbf{k}_j$.
`phase`	300 × 1	rad	Phases φ_j.
`amp_kgm3`	scalar	kg/m³	Amplitude a of each mode.

`unc`: Uncertainties for a simulation

Created by `sbtk.data.uncertainties`.

1σ uncertainties by degree and by inertia quantity, which `sbtk.data.set_uncertainties` gives to a measurement set (for example the exact values of `sbtk.data.simulate`, which have none) and `sbtk.data.add_noise` then draws noise from.

Fields of `unc` (3)

Field	Size	Unit	Description
`coef_sigma_by_degree`	1 × L		1σ of every fully normalized coefficient of degree 1, 2, ..., L. For degree 1, `sbtk.units.offset_to_degree1` converts a center-of-mass uncertainty in km.
`moi_sigma`	scalar		1σ of each inertia element divided by MR_{ref}^2.
`libration_sigma`	scalar		1σ of the libration ratio $(B - A) / C$.

`obs`: Measurement set

Created by `sbtk.data.simulate`, `sbtk.data.from_coefficients`, `sbtk.exchange.read_data`, `sbtk.exchange.read_shadr`, `sbtk.io.read_jpl_gravity`, `sbtk.io.read_measurements`, `sbtk.io.load_measurements`.

Gravity coefficients with their covariance, moment-of-inertia measurements, mass, reference radius and expansion origin. See Measurement sets.

Fields of `obs` (18)

Field	Size	Unit	Description
`body`	text		Name of the body.
`name`	text		Name of the data set (for packages, the package name).
`mass_kg`	scalar	kg	Total mass; the inversion keeps the model mass equal to it.
`GM_km3_s2`	scalar	km³/s²	GM.
`R_ref_km`	scalar	km	Reference radius of the coefficients.
`degree`	scalar		Highest degree L.
`origin_km`	1 × 3	km	Expansion origin in the shape-model frame.
`coef_about`	text		`'origin'`: all coefficients, degree 1 included, and the inertia rows are about `origin_km` (sets from `sbtk.data.simulate` and `sbtk.data.expand_about`, and packages about the origin of their shape model file). `'center_of_mass'`: coefficients of degree ≥ 2 are about the body's center of mass and the degree-1 coefficients give its measured position relative to `origin_km` (Density estimation).
`com_minus_cof_km`	1 × 3	km	Center of mass minus center of figure.
`coef_labels`	n × 1 cell		Names of the coefficients, e.g. `'C10'`, `'S22'`, in the order of `coef`; C00 is not included.
`coef`	n × 1		Fully normalized coefficients.
`coef_cov`	n × n		Covariance of `coef` (diagonal or full); NaN for exact values without uncertainties.
`moi`	1 × k struct		Moment-of-inertia measurements, one `moi` row each.
`moi_tensor_norm`	3 × 3		Inertia tensor divided by MR_{ref}^2, assembled from the rows `Ixx` ... `Iyz`; NaN if any of them is missing. Informational: the inversion uses the rows.
`libration_ratio`	scalar		$(B - A) / C$ used in the `libration` row; NaN if there is none.
`libration_sigma`	scalar		1σ of the libration ratio; NaN if there is none.
`noise_seed`	scalar		Seed of the simulated noise; NaN for exact values and real data.
`notes`	text		Origin of the data.

`moi`: Moment-of-inertia measurement row

Created by `sbtk.data.add_moi`, `sbtk.data.add_moi_row`, `sbtk.data.add_libration`, `sbtk.data.simulate`.

One measurement of a linear combination of the inertia elements: $\mathbf{w} \cdot [I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}] \,/\, (MR_{\mathrm{ref}}^2) = \mathrm{value} \pm \sigma$.

Fields of `moi` (4)

Field	Size	Unit	Description
`label`	text		`'Ixx'`, `'Iyy'`, `'Izz'`, `'Ixy'`, `'Ixz'`, `'Iyz'`, `'libration'`, or a name given to `sbtk.data.add_moi_row`.
`weights`	1 × 6		Weights $\mathbf{w}$ on $[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}]$. Single elements have one weight 1; the libration row is $[-\cos(2a), \cos(2a), -k, 2\sin(2a), 0, 0]$ (Moments of inertia).
`value`	scalar		Measured value (0 for the libration row).
`sigma`	scalar		1σ (for the libration row, the 1σ of the ratio times $I_{zz} \,/\, (MR_{\mathrm{ref}}^2)$); NaN for exact values without uncertainties.

`truth`: True model of a simulation

Created by `sbtk.data.simulate`, `sbtk.io.load_truth`.

A model with the exact values computed from it.

Fields of `truth` (11)

Field	Size	Unit	Description
`body`	text		Name of the body.
`name`	text		Name of the data set.
`rho_bg_kgm3`	scalar	kg/m³	Background density, set so that the model mass equals the body mass.
`components`	1 × nc struct		The `component` array with the true relative densities.
`random_field`	struct		The `random_field`.
`origin_km`	1 × 3	km	The expansion origin: the center of mass of the model. Layers and sectors are measured from it.
`mass_kg`	scalar	kg	Total mass.
`degree`	scalar		Highest degree.
`coef`	1 × (L+1)²		Exact fully normalized coefficients about `origin_km`, in the order of `pk`, C00 included.
`com_km`	1 × 3	km	Exact center of mass.
`inertia_norm`	1 × 6		Exact $[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}] / (MR_{\mathrm{ref}}^2)$ about `origin_km`.

`geo`: Facet-cone geometry

Created by `sbtk.gravity.geometry`.

The integrals of every facet cone about one expansion origin, per unit density; the workspace of the forward model and the inversion (Gravity and moment calculations). Not saved to files (about 30 MB for Didymos).

Fields of `geo` (26)

Field	Size	Unit	Description
`body`	text		Name of the body.
`origin_km`	1 × 3	km	Expansion origin (the apex of every cone).
`degree`	scalar		Highest degree L.
`R_ref_km`	scalar	km	Reference radius.
`pk`	struct		Coefficient order (`pk`).
`nf`	scalar		Number of facets.
`K`	scalar		Number of sub-rays per facet (9).
`faces`	nf × 3		Vertex indices of each facet.
`verts`	nv × 3	km	Vertices relative to `origin_km`.
`D`	nf × 3	km	First corner of each facet, relative to the origin.
`E`	nf × 3	km	Second corner.
`F`	nf × 3	km	Third corner.
`det6`	nf × 1	km³	Six times the cone volume, $\mathbf{D}\cdot(\mathbf{E}\times\mathbf{F})$.
`EF`	nf × 3	km²	$\mathbf{E}\times\mathbf{F}$, used in ray–facet tests.
`FD`	nf × 3	km²	$\mathbf{F}\times\mathbf{D}$.
`DE`	nf × 3	km²	$\mathbf{D}\times\mathbf{E}$.
`T`	nf × (L+1)²	km³	Per cone, $\int (r/R)^l \bar{Y}_{lm}\, dV$ (Gravity and moment calculations).
`V`	nf × 1	km³	Per-cone volume.
`m1`	nf × 3	km⁴	Per-cone first moment $\int \mathbf{x}\, dV$.
`Q`	nf × 6	km⁵	Per-cone second moments $\int [x^2, y^2, z^2, xy, xz, yz]\, dV$.
`U_body`	1 × (L+1)²	km³	Sum of `T` over the body.
`V_body`	scalar	km³	Body volume.
`m1_body`	1 × 3	km⁴	First moment of the body.
`Q_body`	1 × 6	km⁵	Second moments of the body.
`ray_t`	nf·K × 1	km	Distance from the origin to the surface along each sub-ray.
`ray_dir`	nf·K × 3		Unit direction of each sub-ray.

`integrals`: Integrals of a component

Created by `sbtk.gravity.component_integrals`, `sbtk.gravity.random_field_integrals`.

Integrals over the part of a component inside the body, about `geo.origin_km`, per unit density. From `sbtk.gravity.random_field_integrals` the same fields are weighted by the density perturbation (units multiplied by kg/m³).

Fields of `integrals` (4)

Field	Size	Unit	Description
`U`	1 × (L+1)²	km³	$\int (r/R)^l \bar{Y}_{lm}\, dV$ in the order of `pk`.
`V`	scalar	km³	Volume inside the body.
`m1`	1 × 3	km⁴	First moment.
`Q`	1 × 6	km⁵	Second moments $[xx, yy, zz, xy, xz, yz]$.

`fm`: Forward-model result

Created by `sbtk.gravity.forward`.

Fields of `fm` (5)

Field	Size	Unit	Description
`rho_bg_kgm3`	scalar	kg/m³	Background density that gives the stated mass.
`coef`	1 × (L+1)²		Fully normalized coefficients about `geo.origin_km`, order of `pk`.
`com_km`	1 × 3	km	Center of mass in the shape-model frame.
`inertia_norm`	1 × 6		$[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}] / (MR_{\text{ref}}^2)$ about `geo.origin_km`.
`mass_kg`	scalar	kg	The mass given.

`cf`: Field about its center of mass

Created by `sbtk.gravity.center_of_mass_frame`.

A gravity field moved to its center of mass, in the shape-model axes or its principal axes (Frames of a gravity field). n = (L+1)² − 1.

Fields of `cf` (10)

Field	Size	Unit	Description
`labels`	1 × n		Names of the coefficients of degree 1 to L, order of `pk`.
`coef`	1 × n		Fully normalized coefficients about the center of mass in the axes `axes`: degree 1 is 0, and in principal axes $\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0$. NaN for a coefficient a measurement set does not have.
`sigma`	1 × n		1σ, NaN without uncertainties; degree 1: of the center of mass.
`cov`	n × n		Covariance propagated through the translation and the rotation.
`com_km`	1 × 3	km	Center of mass in the shape-model frame.
`axes`	3 × 3		Columns: the axes of `coef` as unit vectors in the shape-model axes (`eye(3)` for `"shape"`).
`principal`	3 × 3		Columns: the principal axes of the smallest, middle and largest moment in the shape-model axes.
`moments`	1 × 3		Principal moments minus their mean, $A \le B \le C$, as $I / (MR_{\text{ref}}^2)$; the degree-2 field fixes them up to their trace.
`R_ref_km`	scalar	km	Reference radius.
`degree`	scalar		Highest degree L.

`opts`: Inversion settings

Created by `sbtk.invert.options`.

Fields of `opts` (13)

Field	Size	Unit	Description
`moi_labels`	1 × k cell		Labels of the `moi` rows to fit. Standard value `{'Ixx','Iyy','Izz'}`; `{}` fits gravity coefficients only.
`regularization`	scalar		Tikhonov parameter $\lambda \ge 0$. Standard value 0.
`background`	string		`"estimate"` (standard): the background density is a free parameter. `"bulk"`: it is held at the bulk density M / V.
`rho_max_kgm3`	scalar	kg/m³	Upper bound on the background density and on every component's density. Standard value 4500.
`max_iter`	scalar		Largest number of Gauss–Newton steps for estimated geometry. Standard value 200.

Field	Size	Unit	Description
`step_km`	scalar	km	Largest trust-region radius of one step of a center coordinate. A step that does not lower the objective is undone and the radius halved; a step that does doubles it, up to this value. Standard value 0.010.
`step_deg`	scalar	deg	Largest trust-region radius of one step of an axis tilt or turn, halved and doubled with `step_km`. Standard value 1.
`position_sigma_km`	scalar	km	A priori 1σ of each estimated center coordinate about its starting value; `Inf` (standard) means no a priori constraint.
`axis_sigma_deg`	scalar	deg	A priori 1σ of each estimated axis tilt and turn about the starting orientation; `Inf` (standard) means none.
`max_shift_km`	scalar	km	Largest distance of an estimated center from its starting point; `Inf` (standard) means no limit.
`max_tilt_deg`	scalar	deg	Largest angle of an estimated axis from its starting direction; `Inf` (standard) means no limit.
`max_turn_deg`	scalar	deg	Largest turn of an estimated ellipsoid about its axis from its starting orientation; `Inf` (standard) means no limit.
`geometry_tol`	scalar		The geometry has converged when an accepted step changes every parameter by less than this fraction of its step limit. Standard value 10^{-3}.

`prob`: Prepared inversion

Created by `sbtk.invert.setup`, `sbtk.invert.replace_data`.

Everything `sbtk.invert.run` needs, computed once. Sizes for 27 measurements and two components.

Fields of `prob` (27)

Field	Size	Unit	Description
`body`	struct		The `body`.
`obs_name`	text		Name of the measurement set.
`obs`	struct		The `obs`.
`coef_about`	text		Copied from `obs.coef_about`.
`opts`	struct		The `opts`; fields that do not select measurements may be changed between runs.
`geo`	struct		Geometry about the measured center of mass, from which layers and sectors are measured (`geo`; `fit.origin_km`).
`mass_kg`	scalar	kg	Mass of the measurement set.
`rho_mean_kgm3`	scalar	kg/m³	Bulk density M / V.
`jc`	1 × n		Position of each measured coefficient in the packed vector of `pk`.
`W`	k × 6		Weights of the inertia rows used.
`y`	n × 1		Measurement vector (coefficients, then inertia rows), including the linearization offset of the center-of-mass iteration.
`Cov`	n × n		Measurement covariance: the coefficient covariance and the inertia variances.
`Lw`	n × n		Lower Cholesky factor of `Cov`, used to whiten the problem.
`labels`	n × 1 cell		Measurement names.
`shift_km`	1 × 3	km	Point the predictions are about, relative to `geo.origin_km`: for data about a fixed point, `obs.origin_km` (every coefficient and the inertia); for data about the center of mass, the model's center of mass (degree ≥ 2 and inertia).
`T`	(L+1)² × (L+1)²		Solid-harmonic translation matrix for `shift_km`.
`D`	n × 3		Derivative of the predictions with respect to the expansion point, the coupling term of the center-of-mass iteration.
`y_measured`	n × 1		Measurement vector as given.
`Ibody`	struct		Integrals of the whole body (`integrals`).
`gbody`	n × 1		Predicted measurements of a body of unit density.
`components`	1 × nc struct		The `component` array.
`Ic`	1 × nc struct		`integrals` of each component.
`mov`	1 × m		Indices of the components whose geometry is estimated.
`gp`	1 × p struct		The estimated geometry parameters, one per scalar (`sbtk.internal.geometry_parameters`): `component`, `kind` (`'position'` or `'tilt'`: a rotation about the component's own axis `index`), `index`, `name`.
`start`	1 × nc struct		The components as given: priors and limits on the geometry refer to them.
`G`	n × nc		Design matrix: change of each measurement per unit relative density of each component (kg/m³).
`Vc`	1 × nc	km³	Volume of each component inside the body.

`fit`: Inversion result

Created by `sbtk.invert.run`, `sbtk.invert.solve`, `sbtk.invert.homogeneous`, `sbtk.io.load_fit`.

A model (`rho_bg_kgm3`, `components`, `random_field`, `origin_km`) with the estimates, their uncertainties and the fit statistics. See Density estimation and Goodness of fit.

Fields of `fit` (38)

Field	Size	Unit	Description
`body`	text		Name of the body.
`data_name`	text		Name of the measurement set.
`rho_bg_kgm3`	scalar	kg/m³	Estimated background density.
`components`	1 × nc struct		Components with the estimated `drho_kgm3` (and `center_km`, `axes` where estimated).
`random_field`	struct		No field (standard deviation 0).
`origin_km`	1 × 3	km	Origin of the model geometry, from which layers and sectors are measured: the measured center of mass (`obs.origin_km` for data about the center of mass; for data about a fixed point, the center of mass their degree 1 gives, or `obs.origin_km` if that is within 3σ of it).
`rho_bg_sigma_kgm3`	scalar	kg/m³	1σ of the background density.
`drho_sigma_kgm3`	1 × nc	kg/m³	1σ of each relative density.
`rho_kgm3`	1 × nc	kg/m³	Density of each component: background plus relative density.
`rho_sigma_kgm3`	1 × nc	kg/m³	1σ of each component density, including the covariance with the background.
`center_sigma_km`	nc × 3	km	1σ of the estimated center coordinates; NaN where not estimated.
`axis_sigma_deg`	nc × 3	deg	1σ of the estimated rotations about the component's first, second and third axes (the tilts of its third axis and the turn about it); NaN where not estimated, as for a rotation about an axis of symmetry.
`at_bound`	1 × nc logical		Density at 0 or at `opts.rho_max_kgm3`. The stated 1σ does not account for the bound.
`resolved`	1 × nc logical		Absolute relative density larger than its 1σ.
`determined`	logical		False if the measurements, with any regularization or a priori constraint, do not determine every free parameter; the covariance is then a pseudo-inverse.
`chi2`	scalar		χ^2 of the measurements.
`n_obs`	scalar		Number of measurements n.
`dof`	scalar		Degrees of freedom $\nu = n - p_{\mathrm{eff}}$ (Goodness of fit).
`chi2_reduced`	scalar		χ^2 / ν.
`p_value`	scalar		$P(\chi^2_\nu \geq \chi^2)$.
`consistent`	logical		`p_value >= 0.05`.
`labels`	n × 1 cell		Measurement names.
`measured`	n × 1		Measured values.
`predicted`	n × 1		Values predicted by the fitted model.
`sigma`	n × 1		1σ of each measurement (square root of the covariance diagonal).
`normalized_residual`	n × 1		(predicted − measured)/σ.
`names`	1 × p cell		Names of the parameters: `'background'`, component names, `<name>_x`, `_y`, `_z` for estimated center coordinates, and `<name>_tilt1`, `_tilt2`, `_turn` for the estimated rotations about the component's first, second and third axes.
`cov`	p × p		Covariance of the parameters (each entry in the product of the units of its two parameters: kg/m³, km, deg).
`corr`	p × p		Correlation matrix of the parameters.
`iterations`	scalar		1 plus the Gauss–Newton steps to the best geometry (1 if no geometry is estimated).
`geometry_converged`	logical		False when `opts.max_iter` steps ended before the estimated geometry converged (true if none is estimated).
`regularization`	scalar		Tikhonov parameter used.
`moi_labels`	1 × k cell		Inertia rows used.
`background`	text		`'estimate'` or `'bulk'`.
`frame_iterations`	scalar		Iterations of the center-of-mass frame (0 when `coef_about` is `'origin'`).

Field	Size	Unit	Description
`frame_offset_km`	scalar	km	Change of the model's center of mass in the last frame iteration.
`coef_about`	text		Copied from the measurements.
`com_shift_km`	1 × 3	km	The point the predictions are about minus `origin_km`: the model's center of mass (coefficients of degree ≥ 2 and inertia) when `coef_about` is `'center_of_mass'`; `obs.origin_km` (every coefficient and the inertia) when it is `'origin'`.

`pred`: Predictions of a fitted model

Created by `sbtk.invert.predict`.

The fitted model evaluated for every measurement of a set, including those it was not fitted to. See Fitting part of the coefficients.

Fields of `pred` (15)

Field	Size	Unit	Description
`labels`	n × 1 cell		Every measurement of the set: its coefficients, then the inertia rows of the fit.
`measured`	n × 1		Measured values.
`predicted`	n × 1		Values of the fitted model.
`sigma`	n × 1		1σ of each measurement.
`normalized_residual`	n × 1		(predicted − measured)/σ.
`chi2`	scalar		χ^2 of every row with the full covariance.
`n_obs`	scalar		Number of rows.
`fitted`	n × 1 logical		The rows the fit was fitted to.
`n_fitted`	scalar		Number of those rows.
`chi2_fitted`	scalar		χ^2 of those rows with their covariance (equal to the fit's χ^2).
`n_other`	scalar		Number of the other rows.
`chi2_other`	scalar		χ^2 of the other rows with their covariance: the test on measurements the model was not fitted to.
`fit_labels`	n_fit × 1 cell		The measurements of the fit.
`data_name`	text		Name of the measurement set.
`body`	text		Name of the body.

`an`: Anomaly of the gravity coefficients

Created by `sbtk.interpret.anomaly`.

The coefficients of a measurement set or of a model minus those of the homogeneous body, about the center of figure in the shape-model axes. See Reading the gravity coefficients; *n* is the number of coefficients of degree 1 to *L*.

Fields of `an` (32)

Field	Size	Unit	Description
`fitted_labels`	1 × k cell		For a fit: the measurements it was fitted to; otherwise empty.
`moi_labels`	1 × p cell		Inertia rows of a measurement set (empty for a model).
`moi_measured`	1 × p		Their measured values, I/(M R_ref²) about the point the set is about.
`moi_homogeneous`	1 × p		The homogeneous body's values about the same point.
`moi_sigma`	1 × p		Their 1σ.
`kind`	text		`"measurements"` or `"model"`.
`name`	text		Name of the measurement set, or of the data a fit was fitted to (`"model"` for a true model).
`body`	text		Name of the body.
`degree`	scalar		Highest degree *L*.
`R_ref_km`	scalar	km	Reference radius.
`mass_kg`	scalar	kg	Mass.
`centroid_km`	1 × 3	km	Center of figure, the point the coefficients are about.
`labels`	1 × n cell		Coefficients `'C10'`, `'C11'`, ... of degree 1 to *L*.
`deg`	1 × n		Degree of each.
`ord`	1 × n		Order of each.

Field	Size	Unit	Description
`field`	1 × n		The set's fully normalized coefficients about the center of figure.
`homogeneous`	1 × n		The homogeneous body's coefficients (degree 1 is 0).
`value`	1 × n		`field − homogeneous`: the anomaly.
`cov`	n × n		Its covariance (0 for a model; NaN for exact values).
`sigma`	1 × n		Its 1σ.
`significance`	1 × n		`value./sigma`.
`com_offset_km`	1 × 3	km	Center of mass minus center of figure, from degree 1.
`com_offset_cov_km2`	3 × 3	km²	Its covariance.
`rays_from_km`	1 × 3	km	The point layers and sectors are measured from: the measured center of mass, or the model's `origin_km`.
`ratio`	1 × L		k_l: the factor by which the homogeneous field of degree l is scaled in the set, fitted by weighted least squares with the covariance of the degree for measurements (NaN at degree 1 and where the homogeneous body has no field of that degree).
`ratio_sigma`	1 × L		Its 1σ.
`residual_rms`	1 × L		RMS of the part of each degree that is not a scaled homogeneous field.
`residual_chi2`	1 × L		χ^2 of that part with the covariance of the degree ($2l$ degrees of freedom; NaN without uncertainties).
`signal_rms`	1 × L		RMS of the anomaly of each degree.
`sigma_rms`	1 × L		RMS of its 1σ.
`residual`	1 × n		Each coefficient's part that is not a scaled homogeneous field (degree 1: the anomaly).
`residual_significance`	1 × n		`residual./sigma`.

`cm`: Central mass

Created by `sbtk.interpret.central_mass`.

One compact mass with the same degrees 1 and 2 as an anomaly. See Degree 1 and the core.

Fields of `cm` (9)

Field	Size	Unit	Description
`fraction`	scalar		q: the mass excess over the total mass (negative for a mass deficit).
`fraction_sigma`	scalar		Its 1σ.
`excess_kg`	scalar	kg	qM.
`offset_km`	1 × 3	km	d: its position from the center of figure.
`offset_cov_km2`	3 × 3	km²	Covariance of `offset_km`.
`offset_sigma_km`	1 × 3	km	1σ of `offset_km`.
`center_km`	1 × 3	km	Center of figure plus `offset_km`.
`iterations`	scalar		Iterations used.
`converged`	logical		False if the iteration did not settle (q not distinguished from 0, or no single compact mass fits).

`sg`: Signatures of candidate components

Created by `sbtk.interpret.signatures`.

Fields of `sg` (13)

Field	Size	Unit	Description
`names`	1 × k cell		Names of the candidates.
`types`	1 × k cell		Their types.
`candidates`	1 × k struct		The candidates (`component`).
`labels`	1 × m cell		The coefficients compared.
`signature`	k × n	per kg/m³	Anomaly of a unit relative density in each candidate, for every coefficient of the anomaly (`an.labels`).
`volume_km3`	1 × k	km³	Volume of each candidate inside the body.
`drho_kgm3`	1 × k	kg/m³	Relative density of each candidate fitted alone to the anomaly.
`drho_sigma_kgm3`	1 × k	kg/m³	Its 1σ.
`delta_chi2`	1 × k		Reduction of χ^2 from the homogeneous body by each candidate alone.
`correlation`	1 × k		Correlation of each weighted signature with the weighted anomaly (1: the same shape).
`chi2_homogeneous`	scalar		χ^2 of the homogeneous body over the coefficients compared.

Field	Size	Unit	Description
`whitened`	m × k		The weighted signatures (whitened by the Cholesky factor of the covariance).
`whitened_anomaly`	m × 1		The weighted anomaly.

`sel`: Stepwise selection of components

Created by `sbtk.interpret.select`.

Fields of `sel` (9)

Field	Size	Unit	Description
`index`	1 × c		The candidates selected, in order (indices into the signatures).
`names`	1 × c cell		Their names.
`chi2`	1 × (c + 1)		χ^2 of the homogeneous body and after each step.
`objective`	1 × (c + 1)		χ^2 plus the prior term.
`delta_chi2`	1 × c		Reduction of the objective at each step.
`drho_kgm3`	1 × c	kg/m³	Relative densities of the selected candidates fitted together.
`drho_sigma_kgm3`	1 × c	kg/m³	Their 1σ.
`components`	1 × c struct		The selected candidates with these densities.
`n_obs`	scalar		Number of coefficients compared.

`ge`: Geometric reading of the anomaly

Created by `sbtk.interpret.geometry`.

Fields of `ge` (34)

Field	Size	Unit	Description
`offset_km`	1 × 3	km	Center of mass minus center of figure.
`offset_sigma_km`	scalar	km	Its 1σ along the offset (NaN without uncertainties).
`offset_lat_deg`	scalar	deg	Its direction.
`offset_lon_deg`	scalar	deg	Its direction.
`offset_significance`	scalar		Its length over its 1σ.
`quad_values`	1 × 3		Principal values of the second moment of degree 2, largest first, in units of MR_{ref}^2.
`quad_axes`	3 × 3		The principal axes (columns), each with $z \geq 0$.
`quad_lat_deg`	1 × 3	deg	Their directions.
`quad_lon_deg`	1 × 3	deg	Their directions.
`quad_sigma_deg`	1 × 3	deg	Their 1σ, from 500 samples of the covariance.
`quad_shape`	text		`axis`, `plane` or `triaxial`.
`quad_ratio`	scalar		$(a_2 - a_3) / (a_1 - a_3)$: 0 an axis, 1 a plane.
`quad_chi2`	scalar		χ^2 of degree 2 (5 degrees of freedom).
`rest_quad_values`			The same for the rest of degree 2, after the scaled homogeneous field.
`rest_quad_axes`			The same for the rest of degree 2, after the scaled homogeneous field.
`rest_quad_lat_deg`			The same for the rest of degree 2, after the scaled homogeneous field.
`rest_quad_lon_deg`			The same for the rest of degree 2, after the scaled homogeneous field.
`rest_quad_shape`			The same for the rest of degree 2, after the scaled homogeneous field.
`rest_quad_ratio`			The same for the rest of degree 2, after the scaled homogeneous field.
`lat_deg`	1 × 91, 1 × 181	deg	The grid of the maps.
`lon_deg`	1 × 91, 1 × 181	deg	The grid of the maps.
`map`	1 × L cell		Each degree of the anomaly and of its rest as a pattern on the sphere.
`rest_map`	1 × L cell		Each degree of the anomaly and of its rest as a pattern on the sphere.
`max_lat_deg`	1 × L	deg	Directions of the strongest excess and deficit of each degree.
`max_lon_deg`	1 × L	deg	Directions of the strongest excess and deficit of each degree.
`min_lat_deg`	1 × L	deg	Directions of the strongest excess and deficit of each degree.
`min_lon_deg`	1 × L	deg	Directions of the strongest excess and deficit of each degree.
`rest_max_lat_deg`	1 × L	deg	The same for the rest.
`rest_max_lon_deg`	1 × L	deg	The same for the rest.

Field	Size	Unit	Description
`rest_min_lat_deg`	1 × L	deg	The same for the rest.
`rest_min_lon_deg`	1 × L	deg	The same for the rest.
`chi2`	1 × L		χ^2 of each degree and its number of coefficients.
`dof`	1 × L		χ^2 of each degree and its number of coefficients.
`lines`	text		The reading in words.

`pm`: Point masses fitted to the measurements

Created by `sbtk.interpret.point_masses`.

Fields of `pm` (21)

Field	Size	Unit	Description
`centers_km`	k × 3	km	Positions of the points.
`drho_kgm3`	1 × k	kg/m³	Relative density of each sphere.
`mass_kg`	1 × k	kg	Mass anomaly of each point relative to the homogeneous body; they sum to 0.
`lambda`	scalar		The Tikhonov parameter.
`chi2`	scalar		χ^2 and number of measurements.
`n_obs`	scalar		χ^2 and number of measurements.
`labels`	n × 1		Every measurement and (predicted − measured)/σ.
`normalized_residual`	n × 1		Every measurement and (predicted − measured)/σ.
`p_eff`	scalar		Effective number of parameters (trace of the data resolution matrix) and the degrees of freedom left.
`dof`	scalar		Effective number of parameters (trace of the data resolution matrix) and the degrees of freedom left.
`chi2_reduced`	scalar		The χ^2 test.
`p_value`	scalar		The χ^2 test.
`consistent`	scalar		The χ^2 test.
`model`	struct		The solution as a model: `rho_bg_kgm3`, `components`, `random_field`, `origin_km`, `labels`, `data_name`.
`chi2_homogeneous`	scalar		χ^2 of the homogeneous body.
`positive_kg`	scalar, 1 × 3	kg, km	Total positive mass anomaly and its centroid.
`positive_centroid_km`	scalar, 1 × 3	kg, km	Total positive mass anomaly and its centroid.
`negative_kg`	scalar, 1 × 3	kg, km	The same for the negative part.
`negative_centroid_km`	scalar, 1 × 3	kg, km	The same for the negative part.
`weighting`	text		`equal` or `sensitivity`.
`rule`	text		`discrepancy`, `consistent` or `aic`.

`sc`: Chi-square along positions of a component

Created by `sbtk.invert.scan_position`.

Fields of `sc` (10)

Field	Size	Unit	Description
`name`	text		The component placed.
`names`	1 × nc cell		Names of every component of the model.
`centers_km`	k × 3	km	The centers tried.
`chi2`	1 × k		χ^2 of the fit at each center (NaN where the component lies outside the body).
`chi2_reduced`	1 × k		χ^2 / ν.
`p_value`	1 × k		$P(\chi^2_\nu \geq \chi^2)$.
`drho_kgm3`	k × nc	kg/m³	Relative density of every component at each center.
`rho_bg_kgm3`	1 × k	kg/m³	Background density at each center.
`best`	scalar		Index of the lowest χ^2.
`fit`	struct		The `fit` at `best`.

`pa`: A searched geometry parameter

Created by `sbtk.search.parameter`.

Fields of `pa` (5)

Field	Size	Unit	Description
`component`	text		Name of the component.
`field`	text		The geometry field varied: `center_km`, `radius_km`, `semi_axes_km`, `ring_radius_km`, `tube_radius_km`, `height_km`, `depth_km`, `frac`, `lat_deg`, `lon_deg` or `axis`.
`values`	k × w		One row per value tried (w = 1, 2 or 3 numbers, as the field).
`coordinate`	1 × k		A number per value, for plotting the χ^2 profile.
`label`	text		What the coordinate is, with its unit.

`gr`: Grid search of the geometry

Created by `sbtk.search.grid`.

Fields of `gr` (14)

Field	Size	Unit	Description
`parameters`	1 × p struct		The searched parameters (`pa`).
`n_combinations`	scalar		Number of combinations fitted.
`n_integrals`	scalar		Number of component integrals computed.
`index`	N × p		Index of each parameter's value in each combination.
`chi2`	1 × N		χ^2 of each combination, densities fitted without bounds (Inf: a component outside the body).
`within_bounds`	1 × N logical		The densities of the combination are within the bounds of `sbtk.invert.run`.
`drho_kgm3`	N × nc	kg/m³	Relative densities without bounds.
`profile`	1 × p cell		Lowest χ^2 at each value of each parameter.
`valleys`	1 × v		Combinations at the lowest point of a valley of some parameter's profile, lowest χ^2 first: the starts tried first.
`top`	1 × keep		The combinations fitted with the bounds, best first.
`fits`	1 × keep cell		Those `fit`s.
`components`	1 × keep cell		Their components at the grid's geometry, with the estimate flags of the model.
`best_fit`	struct		The first of `fits`.
`time_s`	scalar	s	Computing time.

`rf`: Gauss-Newton from several starts

Created by `sbtk.search.refine`.

Fields of `rf` (5)

Field	Size	Unit	Description
`fits`	1 × k cell		The `fit` from each start, lowest χ^2 first.
`chi2`	1 × k		Their χ^2.
`start`	1 × k		Index of each fit's start.
`best_fit`	struct		The first of `fits`.
`time_s`	scalar	s	Computing time.

`ms`: Compact masses added to a model

Created by `sbtk.search.add_masses`.

Fields of `ms` (14)

Field	Size	Unit	Description
`fits`	1 × (n + 1) cell		The `fit` given and the fit after each mass kept.
`chi2`	1 × (n + 1)		Their χ^2.
`dof`	1 × (n + 1)		Their degrees of freedom, less the `n_fixed` parameters of the model held here (sizes chosen on a grid).
`p_value`	1 × (n + 1)		Their p-values with these degrees of freedom.
`n_masses`	scalar		Number of masses kept.
`tried_chi2`	1 × steps		χ^2 of the best model tried at each step.
`delta_chi2`	1 × steps		Its reduction of χ^2.

Field	Size	Unit	Description
`threshold`	scalar		The reduction a mass needs: the $1-\alpha$ quantile of χ^2_4.
`stop_reason`	text		`consistent`, `not significant` or `max_count`.
`candidates_km`	K × 3	km	Candidate positions.
`candidate_delta_chi2`	steps × K		χ^2 reduction of one mass at each candidate position, at each step.
`candidate_drho_kgm3`	steps × K	kg/m³	Its relative density there.
`best_fit`	struct		The last of `fits`.
`time_s`	scalar	s	Computing time.

`sr`: Search of the geometry

Created by `sbtk.search.run`.

Fields of `sr` (5)

Field	Size	Unit	Description
`grid`	struct		The grid search (`gr`; empty without parameters).
`refine`	struct		Gauss-Newton from the best combinations (`rf`).
`masses`	struct		Masses added (`ms`; empty without candidates or with `max_masses` 0).
`best_fit`	struct		The final `fit`. Its degrees of freedom, reduced χ^2 and p-value count each size chosen on the grid as a parameter, besides those Gauss–Newton estimates, so that `n_obs - dof` is the number of parameters of the Akaike information criterion.
`log`	table		Step, χ^2, degrees of freedom (counted as for `best_fit`), reduced χ^2, p-value, number of components and time of each step.

`cmp`: Comparison with the true model

Created by `sbtk.compare.with_truth`, `sbtk.compare.evaluate`.

See Comparison with a true model.

Fields of `cmp` (8)

Field	Size	Unit	Description
`body`	text		Name of the body.
`R2`	scalar		Fraction of the volume-weighted variance of the true density explained by the estimate.
`R2_max`	scalar		Largest R^2 the same components can reach.
`rms_kgm3`	scalar	kg/m³	Volume-weighted root-mean-square density error.
`std_true_kgm3`	scalar	kg/m³	Volume-weighted standard deviation of the true density.
`chi2_reduced`	scalar		Copied from the fit.
`p_value`	scalar		Copied from the fit.
`components`	struct array		Components present in both models, with fields `name`, `estimate_kgm3`, `sigma_kgm3`, `true_kgm3` (densities) and `deviation_sigma` (absolute difference divided by `sigma_kgm3`).

`ref`: Prepared true model for comparisons

Created by `sbtk.compare.reference`.

Sizes for Didymos (1,179,648 sample points).

Fields of `ref` (9)

Field	Size	Unit	Description
`body`	text		Name of the body.
`truth`	struct		The `truth`.
`components`	1 × nc struct		Components of the fits to be compared.
`origin_km`	1 × 3	km	Origin of those fits.
`w`	1179648 × 1	km³	Volume of each sample point.
`rho_true`	1179648 × 1	kg/m³	True density at each sample point.
`member`	1179648 × nc sparse		1 where a sample point lies inside a component.
`var_true`	scalar	(kg/m³)²	Volume-weighted variance of the true density.
`R2_max`	scalar		Largest R^2 of these components.

`grid`: Density grid

Created by `sbtk.exchange.density_grid`, `sbtk.exchange.read_grid`.

Fields of `grid` (5)

Field	Size	Unit	Description
`spacing_km`	scalar	km	Cell edge.
`x_km`	n × 3	km	Cell centers, at integer multiples of the spacing in the shape-model frame.
`density_kgm3`	n × 1	kg/m³	Density at the cell centers.
`density_sigma_kgm3`	n × 1	kg/m³	1σ of the density: 0 for a true model; propagated from the parameter covariance in a solution package.
`member`	n × nc logical		Cell center inside each component. Only from `sbtk.exchange.density_grid`.

`gridcmp`: Comparison of two density grids

Created by `sbtk.compare.grids`.

Fields of `gridcmp` (8)

Field	Size	Unit	Description
`R2`	scalar		$1 - \sum(\rho_{\text{est}} - \rho_{\text{true}})^2 \,/\, \sum(\rho_{\text{true}} - \bar{\rho}_{\text{true}})^2$ over the common cells.
`rms_kgm3`	scalar	kg/m³	Root-mean-square of estimate minus truth.
`mean_error_kgm3`	scalar	kg/m³	Mean of estimate minus truth.
`std_true_kgm3`	scalar	kg/m³	Standard deviation of the true density.
`within_2sigma`	scalar		Fraction of cells with an absolute error at most twice the 1σ of the estimate; NaN if the estimate has no uncertainties.
`n_common`	scalar		Number of cells in both grids.
`n_truth`	scalar		Number of cells in the true grid.
`n_estimate`	scalar		Number of cells in the estimated grid.

`out`: Results folder

Created by `sbtk.io.new_run`, `sbtk.io.subfolder`.

Fields of `out` (5)

Field	Size	Unit	Description
`project`	text		Project name.
`name`	text		Run name.
`stamp`	text		Creation time, `yyyy-mm-dd_HHMMSS`.
`folder`	text		Folder the save and report functions write to: `results/<project>/<stamp>_<name>`, or a subfolder of it.
`figures`	text		Folder for figures: `<folder>/figures`.

`pk`: Coefficient order

Created by `sbtk.internal.sh_packing`.

Packed coefficient vectors are ordered C00, C10, C11, S11, C20, C21, S21, C22, S22, C30, ...: for each degree, C_{l0}, then C_{lm}, S_{lm} for $m = 1 \ldots l$. Sizes for $L = 4$.

Fields of `pk` (6)

Field	Size	Unit	Description
`deg`	1 × 25		Degree of each entry.
`ord`	1 × 25		Order of each entry.
`isS`	1 × 25 logical		True for sine coefficients.
`idx`	1 × 25		Linear index into an (L+1) × (L+1) matrix with `C(l+1, m+1)` = C_{lm}.
`n`	scalar		Number of entries, $(L+1)^2$.
`labels`	1 × 25 cell		Names, `'C00'` ... `'S44'`.

`samples`: Points filling the body

Created by `sbtk.internal.body_samples`.

Each facet cone is divided into 24 radial shells of equal steps in the radial fraction; each shell is represented by its centroid and volume. Used by the comparisons and the random-field integrals. Sizes for Didymos.

Fields of `samples` (5)

Field	Size	Unit	Description
`X`	1179648 × 3	km	Shell centroids in the shape-model frame.
`w`	1179648 × 1	km³	Shell volumes.
`ts`	1179648 × 1	km	Distance from the origin to the surface along each point's ray.
`ds`	24 × np		$s_k^p - s_{k-1}^p$ for the shell bounds s and powers $p = 3 \ldots \max(L+3, 5)$.
`nr`	scalar		Number of shells per cone (24).

`slices`: Plotting planes

Created by `sbtk.internal.slice_planes`.

A 1 × 2 structure array: plane 1 is $z = 0$ and plane 2 is $y = 0$, both through the geometry's origin.

Fields of `slices` (7)

Field	Size	Unit	Description
`g`	1 × 280	km	Grid coordinates along both plane axes.
`in`	280 × 280 logical		Grid point inside the body.
`X`	k × 3	km	Those points in the shape-model frame.
`ts`	k × 1	km	Surface distance along each point's ray.
`outline`	1441 × 2	km	Outline of the body in the plane.
`xlabel`	text		Axis label.
`ylabel`	text		Axis label.

Reference

Error identifiers

SBTK raises an error instead of substituting a value when an input is invalid or a computation cannot proceed. Every error has an identifier `sbtk:<package>:<condition>`, usable with `verifyError` in tests. The messages below are the fixed parts of the message text; values inserted at run time appear as format specifiers (`%s`, `%d`, `%g`).

Identifier	Raised by	Message
`sbtk:compare:differentComponents`	`sbtk.compare.evaluate`	The fit's components differ from the reference's or origin (e.g. a sphere position was estimated): use sbtk.compare.with_truth.
`sbtk:compare:gridSpacing`	`sbtk.compare.grids`	The grids have different spacings (%g and %g km).
`sbtk:data:axisymmetric`	`sbtk.internal.libration_weights`	Iyy - Ixx = Ixy = 0 (C22 = S22 = 0): the body has no preferred equatorial axis, so (B - A)/C is not defined.
`sbtk:data:coefLabel`	`sbtk.data.select_coefficients`	The data have no coefficient %s.
`sbtk:data:coefSize`	`sbtk.data.from_coefficients`	C, S, C_sigma and S_sigma must all be (L+1) x (L+1) with L >= 1.
`sbtk:data:covPositive`	`sbtk.data.set_covariance`	The covariance must be positive definite.
`sbtk:data:covSize`	`sbtk.data.set_covariance`	Covariance is %d x %d; the data have %d coefficients.
`sbtk:data:covSymmetric`	`sbtk.data.set_covariance`	The covariance must be symmetric.
`sbtk:data:degreeOne`	`sbtk.data.add_center_of_mass`	The data already have degree-1 coefficients.
`sbtk:data:duplicateMoi`	`sbtk.data.add_moi_row`	The data already have a moment-of-inertia row named %s.
`sbtk:data:expandAbout`	`sbtk.data.expand_about`	The coefficients must be about obs.origin_km (coef_about = 'origin'); these are about the center of mass: sbtk.data.move_origin moves their degree 1.
`sbtk:data:expandAboutRow`	`sbtk.data.expand_about`	Only inertia elements can be moved to another point; %s is defined about the center of mass. Select the elements with sbtk.data.select_moi first.
`sbtk:data:incompleteDegree`	`sbtk.data.expand_about`	Expanding about another point needs every coefficient of degree 1 to %d.
`sbtk:data:incompleteDegree`	`sbtk.data.to_frame`	Rotating the coefficients needs every coefficient of each degree present; degree %d is incomplete.
`sbtk:data:incompleteDegree`	`sbtk.data.use_uncertainties`	%s is about another point than the measurements; moving its covariance needs every coefficient of degree 1 to %d.
`sbtk:data:moiLabel`	`sbtk.data.select_moi`	The data have no moment-of-inertia measurement %s.
`sbtk:data:moiLabel`	`sbtk.data.set_moi_sigma`	The data have no moment-of-inertia measurement %s.
`sbtk:data:moveOrigin`	`sbtk.data.move_origin`	Only measurements about the center of mass can move their origin; these are about obs.origin_km.
`sbtk:data:negativeDensity`	`sbtk.data.simulate`	The interior has negative density (minimum %.0f kg/m^3): make the light components less light.
`sbtk:data:noC22`	`sbtk.data.add_libration`	The libration ratio needs the measured C22 and S22 (they give the direction of the principal axes).
`sbtk:data:noCommonCoefficients`	`sbtk.data.use_uncertainties`	The two data sets have no gravity coefficient in common.
`sbtk:data:noIzz`	`sbtk.data.set_moi_sigma`	The 1-sigma of the libration row is the ratio's 1-sigma times Izz / (M R_ref^2) of the Izz row, which these data do not have: set it before sbtk.data.select_moi.
`sbtk:data:noIzz`	`sbtk.data.set_uncertainties`	The 1-sigma of the libration row is the ratio's 1-sigma times Izz / (M R_ref^2) of the Izz row, which these data do not have: set the uncertainties before sbtk.data.select_moi.
`sbtk:data:noUncertainties`	`sbtk.data.add_noise`	The measurements have no uncertainties to draw noise from: give them with sbtk.data.set_uncertainties, sbtk.data.use_uncertainties, sbtk.data.set_covariance or sbtk.data.set_moi_sigma.
`sbtk:data:noUncertainties`	`sbtk.exchange.write_data`	Some of the measurements have uncertainties and some do not. Give all of them (sbtk.data.set_uncertainties, sbtk.data.use_uncertainties, sbtk.data.set_covariance, sbtk.data.set_moi_sigma) or none (exact values).
`sbtk:data:noUncertainties`	`sbtk.interpret.signatures`	The measurements have no uncertainties, which the comparison needs as weights (sbtk.data.set_uncertainties, sbtk.data.use_uncertainties, ...).
`sbtk:data:noUncertainties`	`sbtk.invert.replace_data`	The measurements have no uncertainties, which the inversion needs as weights: give them with sbtk.data.set_uncertainties, sbtk.data.use_uncertainties, sbtk.data.set_covariance or sbtk.data.set_moi_sigma.
`sbtk:data:noUncertainties`	`sbtk.invert.setup`	The measurements have no uncertainties, which the inversion needs as weights: give them with sbtk.data.set_uncertainties, sbtk.data.use_uncertainties, sbtk.data.set_covariance or sbtk.data.set_moi_sigma.
`sbtk:data:transformMeasured`	`sbtk.data.to_frame`	Inertia rows can be expressed in new axes only from the full inertia tensor without uncertainties: transform the exact values of sbtk.data.simulate, then use sbtk.data.set_uncertainties, sbtk.data.select_moi and sbtk.data.add_noise.
`sbtk:data:uncertainties`	`sbtk.data.set_uncertainties`	unc.coef_sigma_by_degree has %d values; degree %d needs one per degree.

Identifier	Raised by	Message
`sbtk:data:zeroSigma`	`sbtk.data.from_coefficients`	Uncertainty of %s is zero; every used coefficient needs sigma > 0.
`sbtk:exchange:correlatedCenterOfMass`	`sbtk.internal.exchange_data_files`	The center of mass is correlated with the other coefficients; the exchange format gives them separately.
`sbtk:exchange:covariance`	`sbtk.exchange.read_data`	gravity_covariance.txt has no row for %s.
`sbtk:exchange:dataSetName`	`sbtk.exchange.solution_name`	Data set %s is not named <team>-DATA<number>.
`sbtk:exchange:locate`	`sbtk.exchange.locate`	Found %d %s packages in %s (and its redacted/, unredacted/); expected one.
`sbtk:exchange:missingKey`	`sbtk.internal.exchange_number`	The file has no "# %s:" line.
`sbtk:exchange:moiRow`	`sbtk.internal.exchange_data_files`	Moment-of-inertia row %s has no exchange-format equivalent.
`sbtk:exchange:noCenterOfMass`	`sbtk.internal.exchange_data_files`	The data need the degree-1 coefficients (the measured center of mass): add them with sbtk.data.add_center_of_mass.
`sbtk:exchange:normalization`	`sbtk.exchange.read_shadr`	The header of %s gives normalization state %g; SBTK needs 1 (fully normalized).
`sbtk:exchange:shapeModel`	`sbtk.exchange.read_data`	The package names the shape model %s, which is neither in the package nor in data/shapes/: copy that file into data/shapes/.
`sbtk:frame:notRotation`	`sbtk.internal.check_rotation`	The axes must be a rotation: orthonormal columns (the new axes as unit vectors in the current frame) with determinant +1.
`sbtk:gravity:incompleteSet`	`sbtk.gravity.center_of_mass_frame`	The field is about a point %.1f m from its center of mass and does not have every coefficient up to degree %d, which moving it needs.
`sbtk:gravity:incompleteSet`	`sbtk.gravity.center_of_mass_frame`	The principal axes need every coefficient of degree 2 to %d; the field lacks some.
`sbtk:gravity:notStarShaped`	`sbtk.gravity.geometry`	%d facets face the expansion origin: the shape is not star-shaped about [%g %g %g] km.
`sbtk:interior:duplicateName`	`sbtk.interior.create`	Component names must be unique (they label the results): %s
`sbtk:interior:noAxis`	`sbtk.interior.set_axis`	Component %s is a %s, which has no axis.
`sbtk:interior:noAxis`	`sbtk.interior.set_axis`	The direction for the axis of %s is zero.
`sbtk:interior:sectorCount`	`sbtk.interior.sector_grid`	Use 1 to 9 bands in each direction (got %d, %d, %d).
`sbtk:interior:sectorRange`	`sbtk.interior.sector`	Sector %s: each range must increase (frac %s, lat %s, lon %s) and span at most 360 deg of longitude.
`sbtk:interior:transformFit`	`sbtk.interior.to_frame`	An inversion result cannot be transformed. Transform the measurements (sbtk.data.to_frame) and the body (sbtk.body.to_frame) and invert in the new frame.
`sbtk:interior:unknownComponent`	`sbtk.interior.align_with_spin_axis`	The interior has no component named %s.
`sbtk:interpret:coefLabel`	`sbtk.interpret.signatures`	The anomaly has no coefficient %s.
`sbtk:interpret:degree`	`sbtk.interpret.anomaly`	The set has coefficients to degree %d, not %d.
`sbtk:interpret:degree`	`sbtk.interpret.central_mass`	The central mass needs degree 2.
`sbtk:interpret:incompleteDegree`	`sbtk.interpret.anomaly`	The anomaly needs every coefficient of degree 1 to %d; the set has no %s. Compute it before selecting coefficients.
`sbtk:interpret:outsideBody`	`sbtk.interpret.signatures`	Candidate %s lies entirely outside the body.
`sbtk:interpret:pointMasses`	`sbtk.interpret.point_masses`	Every component must be a sphere (sbtk.interior.sphere_grid).
`sbtk:invert:coefLabel`	`sbtk.invert.setup`	Coefficient %s is above obs.degree = %d.
`sbtk:invert:differentData`	`sbtk.invert.replace_data`	The new data differ in labels, origin, degree, mass or inertia weights: run sbtk.invert.setup instead.
`sbtk:invert:duplicateName`	`sbtk.internal.prepare_components`	Component names must be unique: %s
`sbtk:invert:geometryParameter`	`sbtk.internal.geometry_parameters`	estimate_position of %s must be true, false or a mask of 3 (x, y, z).
`sbtk:invert:geometryParameter`	`sbtk.internal.geometry_parameters`	Component %s is a %s, which has no center to estimate.
`sbtk:invert:geometryParameter`	`sbtk.internal.geometry_parameters`	Component %s is a %s, which has no axis to estimate.

Identifier	Raised by	Message
`sbtk:invert:infeasible`	`sbtk.invert.run`	Prescribed densities violate the density bounds.
`sbtk:invert:moiLabel`	`sbtk.invert.setup`	opts.moi_labels asks for %s; the data have: %s.
`sbtk:invert:noCenterOfMass`	`sbtk.invert.setup`	Coefficients about the center of mass need its measured position as degree-1 coefficients (sbtk.data.add_center_of_mass).
`sbtk:invert:outsideBody`	`sbtk.internal.prepare_components`	Component %s lies entirely outside the body.
`sbtk:invert:predictOrigin`	`sbtk.invert.predict`	The fit measures layers and sectors from [%g %g %g] km; these measurements give [%g %g %g] km (another center of mass). Predict with the measurements the fit was fitted to, plus other coefficients.
`sbtk:invert:recenter`	`sbtk.invert.recenter`	The model has no sphere, ellipsoid or torus named %s.
`sbtk:invert:recenter`	`sbtk.invert.recenter`	Recentering needs the degree-1 coefficients among the measurements.
`sbtk:invert:scanPosition`	`sbtk.invert.scan_position`	The model has no sphere, ellipsoid or torus named %s.
`sbtk:invert:solver`	`sbtk.invert.run`	The bounded least-squares solution did not converge (lsqlin exit flag %d).
`sbtk:io:degree`	`sbtk.io.read_jpl_gravity`	The file has coefficients up to degree %d only.
`sbtk:io:jplFormat`	`sbtk.io.read_jpl_gravity`	NAMES, VALS and COV sizes do not agree in %s.
`sbtk:io:jplFormat`	`sbtk.io.read_jpl_gravity`	Coefficient %s is missing from %s.
`sbtk:io:jplFormat`	`sbtk.io.read_jpl_gravity`	Keyword %s not found.
`sbtk:io:jplFormat`	`sbtk.io.read_jpl_gravity`	Block %s not found.
`sbtk:io:noRun`	`sbtk.io.latest_run`	No run named %s in results/%s.
`sbtk:io:notNormalized`	`sbtk.io.read_jpl_gravity`	The coefficients in %s are not normalized.
`sbtk:io:referenceRadius`	`sbtk.io.read_jpl_gravity`	The file's reference radius is %g km; body.R_ref_km is %g km.
`sbtk:iteration:notConverged`	`sbtk.internal.fixed_point`	The iteration for %s did not converge in %d steps (last change %.3g, tolerance %.3g).
`sbtk:plot:labels`	`sbtk.plot.coefficient_residuals`	Every prediction must have the same measurements as the first.
`sbtk:plot:names`	`sbtk.plot.anomaly`	Give one name per model: %d models, %d names.
`sbtk:plot:names`	`sbtk.plot.coefficient_residuals`	Give one name per prediction: %d predictions, %d names.
`sbtk:plot:names`	`sbtk.plot.degree_ratio`	Give one name per model: %d models, %d names.
`sbtk:plot:names`	`sbtk.plot.difference_from_homogeneous`	Give one name per set: %d sets, %d names.
`sbtk:plot:noUncertainties`	`sbtk.plot.anomaly`	The anomaly has no uncertainties: give the measurements uncertainties first.
`sbtk:plot:noUncertainties`	`sbtk.plot.significance`	The significance needs measurements with uncertainties.
`sbtk:search:candidates`	`sbtk.search.add_masses`	The candidate masses must be spheres (sbtk.interior.sphere_grid).
`sbtk:search:component`	`sbtk.search.grid`	The model has no component named %s.
`sbtk:search:field`	`sbtk.internal.set_geometry`	Component %s is a %s, which has no geometry field %s.
`sbtk:search:field`	`sbtk.internal.set_geometry`	The field %s of %s takes %d values, not %d.
`sbtk:search:parameter`	`sbtk.search.parameter`	Give one coordinate per row of values: %d rows, %d coordinates.
`sbtk:shape:notDownloaded`	`sbtk.shape.bundled`	%s is not in data/shapes/: the high-resolution models are not in git; copy it from the Hera server (End-to-End Test/HLP6G BlindTest/inputs/Reference models/).
`sbtk:shape:notSpc`	`sbtk.shape.reduce_spc`	%d vertices and %d facets: not an SPC grid model (6 (q+1)^2 vertices, 12 q^2 facets).
`sbtk:shape:reduce`	`sbtk.shape.reduce`	The reduced shape (%d facets) is closed %d, outward %d, star-shaped %d: try another number of facets.
`sbtk:shape:spcReduce`	`sbtk.shape.reduce_spc`	The reduced grid (%d vertices) is closed %d, outward %d, star-shaped %d: the vertices are not in SPC grid order.
`sbtk:shape:spcStep`	`sbtk.shape.reduce_spc`	step %d does not divide the grid size q = %d.

Internal functions

Functions in `sbtk.internal` implement the supported functions. They are not part of the supported interface: their arguments and behavior may change without notice, and scripts should not call them. They are listed here because the methods pages refer to some of them. The toolbox contains no deprecated or legacy functions.

Function	Purpose	Used by (supported functions)
`sbtk.internal.body_samples`	Points that fill the body, with the volume each one represents.	`sbtk.compare.reference`, `sbtk.data.simulate`, `sbtk.gravity.random_field_integrals`
`sbtk.internal.body_settings`	The numbers that define a body, for `settings.json`.	`sbtk.io.report_comparison`, `sbtk.io.report_inversion`, `sbtk.io.report_simulation`
`sbtk.internal.check_rotation`	Stop unless axes is a proper rotation (orthonormal columns, right-handed).	`sbtk.body.to_frame`, `sbtk.data.to_frame`, `sbtk.interior.to_frame`
`sbtk.internal.component_columns`	Design-matrix columns and volumes of components idx at their current geometry.	`sbtk.invert.recenter`, `sbtk.invert.run`, `sbtk.invert.scan_position`
`sbtk.internal.component_member`	True for points X inside component c.	`sbtk.compare.reference`, `sbtk.exchange.density_grid`, `sbtk.gravity.component_integrals`
`sbtk.internal.component_settings`	Components as plain records for `settings.json` (only the fields each type uses).	`sbtk.io.report_inversion`, `sbtk.io.report_simulation`
`sbtk.internal.component_text`	One-line description of a component's geometry (printouts and text files).	`sbtk.exchange.write_model`, `sbtk.exchange.write_solution`, `sbtk.io.write_truth`
`sbtk.internal.ComputePolyhedralCSBar_mex`	Fully normalized spherical-harmonic coefficients of a uniform tetrahedron with one vertex at the expansion origin (compiled C).	`sbtk.gravity.geometry`
`sbtk.internal.density_at_points`	Density [kg/m^3] of a model at points inside the body.	`sbtk.compare.reference`, `sbtk.data.simulate`, `sbtk.exchange.density_grid`, `sbtk.interior.density_at`
`sbtk.internal.density_map`	Sequential dark-blue -> teal -> yellow color map (viridis-like) for densities.	`sbtk.plot.slices`, `sbtk.plot.truth_vs_estimate`
`sbtk.internal.difference_map`	Blue - white - red color map for signed differences (blue lighter, red denser).	`sbtk.plot.anomaly_map`, `sbtk.plot.interior3d`, `sbtk.plot.point_masses`, `sbtk.plot.search_masses`, `sbtk.plot.solution_cuts`, `sbtk.plot.truth_vs_estimate`
`sbtk.internal.empty_moi`	Empty list of moment-of-inertia measurement rows.	`sbtk.data.from_coefficients`, `sbtk.exchange.read_data`, `sbtk.io.read_jpl_gravity`, `sbtk.io.read_measurements`
`sbtk.internal.exchange_conventions`	Text of conventions.txt: units, frame, normalization (with code), degree 1.	other internal functions
`sbtk.internal.exchange_data_files`	Write the measurement files of an exchange package.	`sbtk.exchange.write_data`, `sbtk.exchange.write_model`
`sbtk.internal.exchange_grid_write`	Write `density_grid.txt` of an interior-model or solution package.	`sbtk.exchange.write_model`, `sbtk.exchange.write_solution`
`sbtk.internal.exchange_number`	Numeric value of a metadata key of an exchange file (error if missing).	`sbtk.exchange.read_data`, `sbtk.exchange.read_grid`
`sbtk.internal.exchange_read`	Read one exchange-format text file (see sbtk.internal.exchange_write).	`sbtk.exchange.read_data`, `sbtk.exchange.read_grid`
`sbtk.internal.exchange_write`	Write one exchange-format text file.	`sbtk.exchange.write_model`, `sbtk.exchange.write_potential`, `sbtk.exchange.write_solution`
`sbtk.internal.fewer_dof`	A fit with n more parameters counted: fewer degrees of freedom, and its chi-square test again.	other internal functions
`sbtk.internal.fixed_point`	Iterate x = update(x) until a step moves x by at most tol; at most `max_iter` steps.	`sbtk.data.simulate`, `sbtk.interior.align_with_spin_axis`
`sbtk.internal.free_rotations`	The component's own axes about which a rotation changes its shape.	other internal functions
`sbtk.internal.geometry_deviation`	How far one geometry parameter of a component is from its start.	`sbtk.invert.run`
`sbtk.internal.geometry_parameters`	The geometry an inversion estimates: one entry per scalar parameter.	other internal functions
`sbtk.internal.geometry_step`	A component with one geometry parameter changed by a step.	`sbtk.invert.run`
`sbtk.internal.icosphere`	Unit sphere mesh: an icosahedron with every triangle split into four LEVEL times.	`sbtk.shape.ellipsoid`

Function	Purpose	Used by (supported functions)
sbtk.internal.inertia_from_Q	Inertia tensor from second moments.	sbtk.exchange.read_data, sbtk.gravity.forward
sbtk.internal.label_degree_order	Degree and order of coefficient labels such as 'C20', 'S22', 'C100', 'C1010'.	sbtk.plot.coefficient_residuals, sbtk.plot.difference_from_homogeneous
sbtk.internal.libration_weights	Weights on [Ixx Iyy Izz Ixy Ixz Iyz] of the libration row (B - A) - ratio*C = 0.	sbtk.data.add_libration, sbtk.data.simulate, sbtk.data.to_frame
sbtk.internal.linear_densities	Densities of fixed components by weighted least squares with the mass constraint, without bounds.	other internal functions
sbtk.internal.moi_tensor	3x3 normalized inertia tensor from measurement rows Ixx..Iyz (NaN if any is missing).	sbtk.data.add_moi_row, sbtk.data.add_noise, sbtk.data.select_moi, sbtk.data.simulate, sbtk.data.to_frame, sbtk.exchange.read_data, sbtk.io.read_measurements
sbtk.internal.new_component	Fields shared by every component (use the sbtk.interior constructors).	sbtk.interior.ellipsoid, sbtk.interior.layer, sbtk.interior.no_components, sbtk.interior.sector, sbtk.interior.sphere, sbtk.interior.torus
sbtk.internal.observe	Predicted measurements from per-unit-density integrals I (struct with U and Q).	sbtk.invert.predict, sbtk.invert.run, sbtk.invert.setup
sbtk.internal.point_mass_sh	Normalized spherical harmonics of unit point masses.	sbtk.gravity.component_integrals
sbtk.internal.prepare_components	Put components into a prepared inversion: their integrals, design-matrix columns and estimated geometry.	sbtk.invert.setup
sbtk.internal.principal_axes	Principal axes of inertia from the degree-2 gravity coefficients.	sbtk.exchange.write_model, sbtk.gravity.center_of_mass_frame, sbtk.plot.difference_from_homogeneous
sbtk.internal.prism_field	Potential and acceleration of uniform cubes, exact (Nagy, Papp and Benedek 2000).	sbtk.gravity.field_at
sbtk.internal.random_field_value	Random-field density perturbation [kg/m^3] at points X (n x 3, km).	sbtk.gravity.random_field_integrals
sbtk.internal.read_obj	Triangular faces and vertices of a Wavefront OBJ shape model.	sbtk.shape.load
sbtk.internal.rotate_inertia	Inertia elements [Ixx Iyy Izz Ixy Ixz Iyz] in rotated axes: axes' * I * axes.	sbtk.data.to_frame, sbtk.interior.to_frame
sbtk.internal.set_geometry	A component with one geometry field set to a value (sbtk.search).	other internal functions
sbtk.internal.sh_packing	Ordering of packed spherical-harmonic coefficient vectors.	sbtk.data.expand_about, sbtk.data.from_coefficients, sbtk.data.select_degree, sbtk.data.set_uncertainties, sbtk.data.summary, sbtk.data.to_frame, sbtk.data.to_reduced_shape, sbtk.data.use_uncertainties, sbtk.exchange.read_shadr, sbtk.exchange.write_model, sbtk.gravity.center_of_mass_frame, sbtk.gravity.geometry, sbtk.gravity.labels, sbtk.interior.to_frame, sbtk.io.read_jpl_gravity, sbtk.io.report_simulation, sbtk.io.write_measurements, sbtk.plot.difference_from_homogeneous, sbtk.plot.measurements
sbtk.internal.sh_rotation	Matrix that expresses spherical-harmonic coefficients in rotated axes.	sbtk.data.to_frame, sbtk.gravity.center_of_mass_frame, sbtk.interior.to_frame, sbtk.plot.difference_from_homogeneous
sbtk.internal.sh_translation	Matrix that moves spherical-harmonic integrals to a shifted expansion origin.	sbtk.data.expand_about, sbtk.data.use_uncertainties, sbtk.exchange.write_model, sbtk.gravity.center_of_mass_frame, sbtk.invert.predict, sbtk.invert.run, sbtk.invert.setup
sbtk.internal.shape_file_name	File name of a shape model in an exchange package.	other internal functions
sbtk.internal.slice_axes	Draw one density slice (image + body outline) on axes ax.	sbtk.plot.slices, sbtk.plot.truth_vs_estimate
sbtk.internal.slice_images	Absolute density of each model on the two slice planes.	sbtk.plot.slices, sbtk.plot.solution_cuts, sbtk.plot.truth_vs_estimate
sbtk.internal.slice_planes	Grid points on the two plotting planes, for density slices.	sbtk.plot.point_masses, sbtk.plot.search_masses
sbtk.internal.sphere_inside	True when a sphere lies entirely inside the body (checked on 600 points of its surface).	sbtk.gravity.component_integrals, sbtk.gravity.field_at
sbtk.internal.surface_distance	Distance from the expansion origin to the surface along unit directions [km].	sbtk.compare.reference, sbtk.exchange.density_grid, sbtk.interior.density_at, sbtk.interior.sphere_grid, sbtk.invert.recenter, sbtk.invert.run, sbtk.plot.interior3d, sbtk.plot.solution_cuts
sbtk.internal.surface_distance_indexed	sbtk.internal.surface_distance with the facets sorted by direction first.	other internal functions
sbtk.interpret.anomaly	Gravity coefficients minus those of the homogeneous body, about the center of figure.	other internal functions
sbtk.interpret.central_mass	The mass concentrated near the center and where it sits, read from degrees 1 and 2.	other internal functions

Function	Purpose	Used by (supported functions)
`sbtk.interpret.geometry`	Where the anomaly puts mass: the center-of-mass offset, the principal axes of degree 2, and each degree's directions of excess and deficit.	other internal functions
`sbtk.interpret.point_masses`	Many small masses fitted together to the measurements: where the data put the density anomaly.	other internal functions
`sbtk.interpret.select`	Forward stepwise selection of candidate components that explain the anomaly.	other internal functions
`sbtk.interpret.signatures`	The anomaly each candidate component makes per kg/m^3, and how well each alone explains the data.	`sbtk.plot.contributions`
`sbtk.interpret.summary`	Print what the anomaly says: center-of-mass offset, each degree's size and ratio, the central mass.	other internal functions
`sbtk.search.add_masses`	Add compact masses (boulders, voids) to a model one at a time where the measurements ask for them.	other internal functions
`sbtk.search.directions`	Directions spread evenly within a cone about an axis, for searching the axis of a component.	other internal functions
`sbtk.search.grid`	Fit the densities for every combination of the values of the searched geometry parameters.	other internal functions
`sbtk.search.parameter`	One geometry parameter of a component to search, with the values to try.	other internal functions
`sbtk.search.refine`	Gauss-Newton on the estimated geometry from several starting models; the fits ranked by chi-square.	other internal functions
`sbtk.search.run`	Search the geometry of a model: grid, Gauss-Newton from the best points, compact masses.	other internal functions
`sbtk.search.settings`	Settings of sbtk.search.run with their standard values.	other internal functions

Validation and tests

SBTK 1.0.0 has 196 unit and integration tests in MATLAB's unit-test framework. They run with

```
MATLAB
run(fullfile(sbtk.root, "tests", "run_tests.m"))      % about 35 minutes; writes tests/reports/test_report.txt
```

Most of the time goes to `tests/integration/test_maker_scripts.m`, which runs three HLP6 maker scripts on the full-resolution shape models (3 million facets, degree 10).

This page states what the tests check and with what tolerance, as written in the test files. It makes no claims beyond them. Unit tests use a small ellipsoid body (`tests/unit/small_body.m`); validation tests use an ellipsoid with semi-axes 0.5 × 0.4 × 0.3 km (20,480 facets) and the 9.3 m Didymos model with a bulk density of 2600 kg/m³ and R_{ref} = 0.40 km.

Comparison with independent results

From `tests/integration/test_validation.m`:

Test	Check	Tolerance
`testKernelVolume`	The kernel's degree-0 integral of every facet cone equals the cone volume.	relative 10^{-12}
`testEllipsoidClosedForm`	$\bar{C}_{20}$, $\bar{C}_{22}$ and the principal moments of a homogeneous triaxial ellipsoid against the closed forms $C_{20} = (2c^2 - a^2 - b^2) / (10R^2)$, $C_{22} = (a^2 - b^2) / (20R^2)$, $I = M(b^2 + c^2) / 5$, ...	relative 2×10^{-3} (20,480-facet mesh)
`testConeScaling`	Kernel output for a cone scaled by $s = 0.37$ against s^{l+3} times the unscaled integrals.	relative 10^{-13}
`testSpherePointMassVsRays`	A sphere integrated as a point mass against the same sphere integrated along rays (as an ellipsoid).	relative 10^{-3}
`testTorus`	Volume and I_{zz} of a buried torus against $2\pi^2 R a^2$ and $V(R^2 + 3a^2 / 4)$.	relative 10^{-4} (volume), 10^{-3} (I_{zz})
`testShellSector`	A full-sphere sector between radial fractions 0.5 and 0.75 has volume $(0.75^3 - 0.5^3)V$.	relative 10^{-12}
`testSectorsTileBody`	The 24 sectors of a sector grid fill the body exactly once.	relative 10^{-12}
`testRandomFieldMonteCarlo`	Mass and degree-2 integrals of a random field (σ = 120 kg/m³, L = 100 m) on Didymos against Monte Carlo integration with 4×10^6 points.	2% of the largest value
`testExactRecovery`	Exact Didymos data of a core (off-center), torus and regolith: densities and the core center are recovered, starting with the core at the origin.	10^{-6} kg/m³, 10^{-7} km, $\chi^2 < 10^{-10}$
`testExactRecoverySectors`	Exact data of an 18-sector interior with random densities are fitted by the same sectors.	$\chi^2 < 10^{-6}$, densities within 20 kg/m³
`testCalibration`	Over 40 noise realizations, the standard deviation of (estimate − truth)/σ.	within 0.25 of 1
`testFullCovariance`	The same with coefficients correlated by 0.6 and noise drawn from that covariance.	within 0.25 of 1
`testInertiaHelps`	Adding I_{xx}, I_{yy}, I_{zz} to the gravity coefficients reduces every density uncertainty.	strict
`testDimorphosExact`	Exact data of degree ≤ 3 and the libration ratio only (no other inertia data): an ellipsoidal core and a regolith layer are recovered.	10^{-6} kg/m³
`testTextRoundTrip`	Writing and reading the measurement text format preserves values, covariance, origin, seed and libration ratio, and the fits agree.	relative 10^{-14}; densities 10^{-6} kg/m³
`testFromCoefficients`	Data built from coefficient matrices equal the original measurement set and give the same fit.	exact; 10^{-6} kg/m³

Conventions and frames

From `tests/unit/test_exchange.m` and `tests/unit/test_data.m`:

Test	Check	Tolerance
`testDegreeTwoAndInertia`	The relations between $\bar{C}_{20}$, $\bar{C}_{21}$, $\bar{S}_{21}$, $\bar{C}_{22}$, $\bar{S}_{22}$ and the inertia elements (Moments of inertia).	10^{-12}
`testNormalizationCode`	The normalization function printed in `conventions.txt` gives $N_{20}, N_{21}, N_{22}, N_{30} = \sqrt{5}\ , \sqrt{5/3}\ , \sqrt{5/12}\ , \sqrt{7}$.	relative 10^{-15}
`testShTranslation`	Integrals moved by the solid-harmonic translation equal integrals computed about the shifted origin, including inertia; the predictions move degree ≥ 2 for data about the center of mass and every degree for data about a fixed point.	10^{-12} of the largest value
`testUseUncertainTiesAboutAnotherPoint`	A reference covariance about another point is moved to the data's point (TKT^{T}, the same as `sbtk.data.expand_about`), with the correlations it brings; a reference without degree 1 is rejected.	10^{-12} of the largest value
`testPredictionsAboutTheModelCenterOfMass`	For data about the center of mass, the predicted coefficients and inertia of a fit equal a forward model computed from scratch about the fitted center of mass.	10^{-6}

Test	Check	Tolerance
`testCenterOfMassFrameMatchesTheTrueFrame`	Inverting a measurement package (expansion about the model's center of mass) gives the densities of an inversion about the true center of mass.	0.2σ; χ^2 within 10%
`testFrameIterationWithSectors`	The center-of-mass iteration converges with layers and many sectors.	offset < 10^{-6} km in fewer than 20 iterations
`testNothingAboutTheInterior`	A measurement package does not contain component names, the data-set name of the model, the seed, the expansion origin or the true center of mass.	
`testExactPackage`	A package of exact values has the columns `l,m,C,S`, no covariance file and no uncertainty or noise lines; it is read back with NaN uncertainties, the inversion asks for uncertainties, and with them it fits.	values 10^{-17}; $\chi^2 < 10^{-6}$
`testPackageAboutTheShapeOrigin`	A set expanded about the origin of the shape model file is written with degree 1 and the moments of inertia about that origin, read back as it is (`coef_about = 'origin'`), and inverted without a change of frame.	values 10^{-17}; $\chi^2 < 10^{-6}$
`testReportStatesWhatApplies`	Uncertainties, noise and the libration ratio are stated only when the package has them; exact values with the center of mass as origin in principal axes, or with x, y turned 20° about z, are described as such in `conventions.txt`, and the package's shape model has its center of mass at the origin.	10^{-12} km; obj precision for the center of figure
`testUnnormalizedCoefficients`	`gravity_field_unnormalized.txt` gives the coefficients of `gravity_field.txt` times N_{lm}, and their 1σ for values with uncertainties.	relative 10^{-14}
`testShippedShapeModelNotIncluded`	A package made with one of SBTK's shape model files names it but does not include it; `sbtk.exchange.read_data` reads it from `data/shapes/`.	vertices exact
`testWritePotential`	`potential_acceleration.txt` gives the positions, potential and acceleration of `sbtk.gravity.polyhedron` in SI units and is listed once in `README.txt`.	potential relative 10^{-14}; acceleration 10^{-12} of the largest value
`testGridDefinition`	Density-grid cell centers lie on multiples of the spacing and the cells fill the body; grids of different spacing are rejected.	volume within 5% at 50 m cells
`testLibrationRatioUsesPrincipalMoments`	The simulated libration ratio equals $(B-A)/C$ from the eigenvalues of the tensor, and the true tensor satisfies the libration row.	relative 10^{-12}, 10^{-14}
`testLibrationAmplitudeFormula`	$G = 2e/(3k-1)$ against numerical integration of planar libration in a Keplerian orbit (e = 0.01; k = 0.07, 0.25, 0.5), and the inverse formula.	3%; relative 10^{-12}

Inversion behavior

From `tests/unit/test_invert.m`: exact recovery with $\nu = n - p$; the mass constraint (relative 10^{-10}); the homogeneous fit has the bulk density and $\nu = n$; regularization reduces the norm of the relative densities and increases the degrees of freedom; the λ of `sbtk.invert.choose_regularization` with the rule `"aic"` has an Akaike information criterion no higher than that of any λ of a scan from 10^{-3} to 10^{8}; the p-value of $\chi^2 = 10$ with $\nu = 10$ equals 0.44049; density bounds hold and are flagged; settings by name and value; starting values of the relative densities are not used; unknown inertia labels and inconsistent replacement data are rejected. `testEstimateTorusAxisAndHeight`: the axis of a torus (true tilt 4.5°) and its height (12 m), started 3.5° and 15 m away, are recovered from exact values to 0.02° and 0.2 m with its density (5 kg/m³). `testGeometryLimits`: a hard limit on the tilt holds, an a priori constraint keeps the axis at its start, and geometry asked of components without a center or an axis is rejected. `testEstimateEllipsoidOrientation`: the full orientation of an ellipsoid (the direction of its third axis and its turn about it), started 8° away in yaw and 5° in pitch (9.4° in all), is recovered from exact values to 0.05°, and a limit of 3° on the turn holds. `testRotationsAboutAxesOfSymmetryAreNotEstimated`: with two equal semi-axes, whichever axis is the axis of symmetry, only the two rotations that change the shape are estimated, without a warning, each counted once, and the axis of symmetry is recovered from exact values to 0.05°; with three equal semi-axes, no rotation is estimated. The sphere-position results of `test_regression_baseline.m` (including the number of iterations) did not change when the estimated geometry was generalized.

Reproduction of published and earlier results

Test file	Check	Tolerance
`tests/integration/test_bennu_paper.m`	The Bennu inversion of Cavale et al. (Icarus) (Bennu): core relative density; boulders; torus mass contrast; core center; background equal to the bulk density 1190 kg/m³; coefficients of degrees 3–4 not fitted by the paper.	1 kg/m³; 2%; 1%; 0.5 m; 10^{-9} kg/m³; 0.15 of the JPL 1σ
`tests/integration/test_regression_baseline.m`	Geometry, integrals, sample points, simulation, noise, inversion, regularization choice, comparison and position estimation against values stored from the code before the toolkit (`tests/reference/baseline_v0.mat`). Results that include an ellipsoid or a torus differ because their ray crossings are now exact: the old code sampled 256 points per chord and missed rays that only graze them (the exact crossings equal 4096 samples to 10^{-15}).	relative 10^{-10} unless stated in the test; 2×10^{-6} with an ellipsoid or a torus

Other tests

Test file	Contents
`tests/unit/test_units.m`	G, length scales, inertia conversions, degree-1 conversion.
`tests/unit/test_shape.m`	OBJ reading with units, write–read round trip, ellipsoid volume (relative 3×10^{-3}) and centroid, detection of inward and open meshes, units required, bundled models, SPC reduction (closed, outward, star-shaped).
`tests/unit/test_body.m`	Mass from density, centroid of a shifted shape, bodies from mass, invalid densities rejected.

Test file	Contents
`tests/unit/test_interior.m`	Constructors, ellipsoid rotation, layer depths, latitude–longitude, duplicate names, sector grids and ranges, random field reproducible and with standard deviation within 15% of σ. `testSetAxis`: the axis points along the direction, by the smallest rotation. `testAlignWithSpinAxis`: a torus aligned with the axis of the largest moment of the model it belongs to (to 0.002°, in fewer than 10 simulations), the other components unchanged, unknown names and non-convergence reported.
`tests/unit/test_invert.m` (additions)	`testLayersFromMeasuredCenterOfMass`: data about a point 50 m from the center of mass, inverted with exactly the components of the true model (a core, a layer and a sector), give its densities (to 10^{-4} kg/m³), the measured center of mass as origin and residuals below 10^{-3}σ. `testMassAfterCenterOfMassIterations`: data about the center of mass and an estimated component the body clips: after the center-of-mass iterations the fitted model has the measured mass (to 10^{-12}).
`tests/unit/test_gravity.m`	Uniform body, mass kept by the background, center-of-mass shift, labels, non-star-shaped meshes rejected, components clipped to the body. `testOneRayGrazingSphere`: a sphere on the surface whose bounding sphere only one ray crosses, and that ray misses it, has volume 0. `testPolyhedronAgreesWithCube`: the polyhedron formula and the cube formula give the same potential and acceleration of a cube, outside, on a face (on the edge between its triangles) and inside (to 10^{-10}). `testPolyhedronField`: inside the body the divergence of the acceleration is $-4\pi G\rho$, outside the acceleration is the gradient of the potential (to 10^{-6}), and far away the potential is GM / r. `testFieldAt`: a homogeneous model gives the polyhedron (to 10^{-12}); a sphere inside the body adds the field of a point mass outside it and of a uniform ball inside it, exactly (to 10^{-12}); one through the surface, on a 5 m grid, within 2%. `testIndexedSurfaceDistance`: facets sorted by direction give the surface distance of the search over every facet (to 10^{-13} km), about the center of figure and a point off it, also along the edges of the cube map. `testSmallOverhangAccepted`: seen from a point where one facet of a 0.1 mm dent faces it, the signed cones give the volume (to 10^{-12}) and the center of figure (to 10^{-12} km); a point outside the body is rejected. `testCenterOfMassFrame`: about the center of mass degree 1 is 0; in principal axes $\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0$, each degree's power and each degree's total variance are unchanged, and $\bar{C}_{20}$, $\bar{C}_{22}$ equal the values from the principal moments; a measurement set, the same set without degree 1, and a set lacking a coefficient.
`tests/unit/test_interpret.m`	`testHomogeneousBodyHasNoAnomaly`: a homogeneous body about any point has no anomaly (to 10^{-12}), degree scaling 1 and no center-of-mass offset. `testAnomalyIndependentOfExpansionPoint`: measurements about the center of mass, about another point and the true model give the same anomaly (to 10^{-12}). `testAnomalyCovariance`: the covariance moves with the measurements; the center-of-mass 1σ is $\sqrt{3}\ R$ times that of degree 1. `testCentralMassOfOneSphere`: degrees 1 and 2 of one sphere give its mass excess (to 10^{-9}) and center (to 10^{-9} km). `testCentralMassUncertainty`: the 1σ of the mass excess matches the spread over 60 noise realizations (within 30%). `testSignaturesAddUp`: the anomaly of a model is the sum of its densities times their signatures. `testSelectFindsTheSphere`: forward selection picks the true sphere from four candidates with its density. `testPredictReproducesTheFit`: a fit to degree 1 and J_2 predicts its own rows as the fit does and the other coefficients of the true model to $\chi^2 < 10^{-6}$. `testPredictNeedsTheSameCenterOfMass`: a fit to data about another point is refused. `testRecenterMatchesDegreeOne`: the fixed-point iteration moves a sphere to its true center (to 1 mm). `testScanPositionFindsTheCenter`: the χ^2 profile is 0 at the true center. `testPointMassesFindTheAnomaly`: point masses fitted to a dense sphere keep the mass and put their positive part within 50 m of it; with noise, the consistent rule leaves at least one degree of freedom, and the AIC rule's λ has a χ² + 2 p_eff no higher than half or twice that λ. `testReducedShapeKeepsTheInterior`: the anomaly on a coarser model of the surface equals the original's (within 5%). `testGeometryOfOneSphere`: for a dense sphere in a round body the offset points at it (to 10^{-6} deg), degree 2 is an axis through it (to 1°) and the excess of degrees 2–4 points at it (within 3°). `testGeometryOfATorus`: the smallest axis of degree 2 of a torus about z is z (within 5°), with a 1σ below 1°. `testPointMassesFindTheAnomaly` also: the point-mass solution as a model gives its own χ^2; with the consistent rule and no consistent smoothing, the best p-value is reported and the fit marked not consistent.
`tests/unit/test_search.m`	`testParameterChecks`: unknown components, fields and value widths are refused. `testDirections`: the axis first, unit vectors, the tilts as given. `testGridMatchesRun`: exact data pick the true layer thickness; with noise, the densities without bounds give the χ^2 of `sbtk.invert.run` (to 10^{-6}). `testGridFindsTorus`: the true ring and tube radii and the tilted axis among 84 combinations, $\chi^2 < 10^{-8}$. `testGridStartsInEachValley`: the profile of a core with a layer has a valley on each side (a dense and a light core), and both are starts. `testRefineFromSeveralStarts`: Gauss–Newton from two starts recovers the core. `testAddMassesFindsBoulder`: one mass is added, at the hidden boulder (to 0.1 m) with its density, and the search stops when the model is consistent. `testRunAllSteps`: grid, Gauss–Newton and masses recover a core, a layer and a boulder from exact data; the figures draw.
`tests/unit/test_frames.m`	`testSpinAxes`: in the spin axes $\bar{C}_{21} = \bar{S}_{21} = 0$ to 10^{-13}, z is the axis of the largest moment, x lies in the plane of the shape-model x-axis and z, $\frac{1}{2}\operatorname{atan2}(\bar{S}_{22}, \bar{C}_{22})$ is the longitude of the axis of the smallest moment, each degree's power is unchanged, and the comparison with the homogeneous body gives its own terms. `testChangeOfFrameEqualsSimulationInIt`: a body, an interior with every component type and a random field, carried into a rotated and shifted frame and simulated there, give the transformed true model, coefficients, inertia rows and libration ratio of the original simulation (coefficients and inertia to 10^{-7}, the libration ratio to 10^{-6}), and the density grid in the new frame equals the original model at the same points (to 10^{-9} kg/m³). `testPackageInSpinAxes`: packages in the spin axes contain their own shape-model file, the data have $\bar{C}_{21}$, $\bar{S}_{21}$ zero within the noise, and the grid and the component descriptions are in the same frame. `testChangeOfFrameRules`: a shift applies to any set; new axes need exact inertia rows and complete degrees; fits are not transformed; non-rotations are rejected. `testNoLocalPathInPackage`: a package with a shape model changed in frame names the original file but not its folder. `testChangesOfFrameCombine`: after two changes of frame the shape model's vertices equal `(original vertices - frame_origin_km)*frame_axes` to 10^{-12} km. `testGeneratedShapeNamedAfterPackage`: a shape model changed in frame and written into two packages gets two file names, `<data set>_<shape name>.obj`, and the interior-model package names the same file as its data package.
`tests/unit/test_data.m`	`testExpandAbout`: coefficients and inertia expanded about another point equal those computed about that point for a homogeneous body (to 10^{-14}); incomplete degrees and non-element inertia rows are rejected. Measurement fields, center-of-mass frame, simulation without uncertainties and `sbtk.data.set_uncertainties`, noise, negative density rejected, selection, covariance, inertia and libration rows, inertia uncertainties.
`tests/unit/test_compare.m`	$R^2 = 1$ for the true model; $R^2 = 0$ (within 0.01) for a homogeneous estimate.
`tests/unit/test_io.m`	Run folders, missing runs, report functions; a run that only makes a true model (simulation without uncertainties, the comparison with the homogeneous body in principal axes, the interior-model package with the principal axes and moments of the true model).
`tests/unit/test_plot.m`	Every figure function runs, except the two search figures, which `test_search.m` draws (`testInterpretationPlots`: the anomaly, anomaly-map, degree-ratio, coefficient-residual, significance, point-mass, coefficient-comparison (with and without solutions), contribution figures and the solutions side by side). `testDifferenceFromHomogeneous`: a homogeneous body differs from itself by 0; one interior as a true model, a package, a field about the shape-model origin and a fitted model gives the same differences. `testDifferenceInPrincipalAxes`: the rotated field has $\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0$ to 10^{-13}, and the homogeneous field of the rotated shape model equals the rotated homogeneous coefficients to 10^{-12}.
`tests/integration/test_examples.m`	The eleven tutorial scripts run, ex01 also as code evaluated outside its file (as Run Section does).
`tests/integration/test_template_project.m`	The project template runs: simulation, inversion, comparison.
`tests/integration/test_maker_scripts.m`	The seven HLP6 maker scripts in `examples/` pass the code analyzer; `make_didymos_3`, `make_dimorphos_3` and `make_didymos_homogeneous` run on the full-resolution models, and their packages have exact values only, to degree 10, normalized and unnormalized, in the frame and about the origin of the shape model file (degree 1 listed, $\bar{C}_{21}$, $\bar{S}_{21}$ not zero), which they name and do not include, every boulder or void of the model, and no local paths. The homogeneous package has the agreed GM and reference radius, its degree 1 gives the center of figure (to 10^{-9} km), it has all six inertia elements and the potential and acceleration at the 20 points of the makers' cross-validation.

Longer studies in `tests/studies/` (recovery over interiors and data sets, edge cases of the forward model against brute-force integration, uncertainty calibration, reading the coefficients, the geometry search, point masses) are run separately and are not part of the test suite.